User Manual

Autonomous Navigation Solutions

Inertial Sense, Inc.

©2022

Table of Contents

1. Overview 5
   1.1 IMX (IMU, AHRS, and GPS-INS) 5
   1.2 Features 6
   1.3 Interfaces 7
   1.4 Applications 7
2. Data Sheets 9
   2.1 IMX-6 (IMU, AHRS, GNSS-INS) 9
   2.2 RUG-4 GNSS Variant Comparison 9
   2.3 IMX-5 (IMU, AHRS, GNSS-INS) - Legacy Product 9
   2.4 GPX-1 (Multi-Band L1/L5 Dual GNSS Receiver) 9
3. Dimensions and Pinouts 10
   3.1 Hardware Design Files 10
4. Getting Started 11
   4.1 Getting Started 11
   4.2 GPX-1 Quick Start Guide 13
5. Hardware Integration 15
   5.1 Hardware Integration: IMX-6 Module 15
   5.2 Hardware Integration: IMX-5 Module 24
   5.3 Hardware Integration: GPX-1 Module 31
   5.4 Hardware Integration: RUG-4 (Rugged-4) 41
   5.5 Rugged-3 46
   5.6 Hardware Integration: IG-2 (IMX5 + GPX1) 52
   5.7 Hardware Integration: IG-1-IMX-5 59
   5.8 Hardware Integration: IK-1 (IMX5 or GPX1) 66
   5.9 Hardware Design Files 72
   5.10 Reflow Soldering 73
6. Software / SDK 75
   6.1 EvalTool 75
   6.2 CLTool 89
   6.3 Log Inspector 94
   6.4 SDK 97
   6.5 Example Projects 99
7. IMX 117
   7.1 Allan Deviation / Allan Variance (ADEV/AVAR) 117
   7.2 External Aiding 122

©2022

7.3 GNSS Aiding 127
7.4 Configuration 162
7.5 IMX System Status 179
7.6 Magnetometer 185
7.7 Dead Reckoning 187
7.8 IMU Shock Rejection 196
7.9 IMX-5 Bootloader 205
7.10 Troubleshooting 206
8. GPX 212
8.1 GPX Configuration 212
8.2 GPX System Status 213
9. Communication Protocols 214
9.1 Protocol Overview 214
9.2 Data Sets (DIDs) 215
9.3 Inertial Sense Binary (ISB) Protocol 305
9.4 NMEA 0183 (ASCII) Protocol 311
9.5 SPI Protocol 330
9.6 CAN Protocol 335
10. GNSS - RTK 343
10.1 Multi-band GNSS 343
10.2 GNSS Antennas 347
10.3 GNSS Satellite Constellations 348
10.4 RTK Positioning 349
10.5 Dual GNSS RTK Compassing 365
11. Data Logging/Plotting 372
11.1 Data Logging/Plotting 372
11.2 Logging 375
11.3 Plotting 377
12. Reference 378
12.1 Coordinate Frames 378
12.2 Definitions 383
12.3 IMU Specifications 385
12.4 Interference Considerations 386
13. User Manual PDF 388
14. Frequently Asked Questions 389
14.1 What is a Tactical Grade IMU? 389
14.2 What is Inertial Navigation? 390
14.3 What does an Inertial Navigation System (INS) offer over GPS alone? 391

©2022

14.4 Our Sensors - IMU vs AHRS vs INS 391
14.5 How long can the IMX dead reckoning estimate position without GPS? 391
14.6 Can the IMX estimate position without GPS? 392
14.7 How does the IMX estimate roll/pitch during airborne coordinate turns (acceleration only in the Z axis and not in the X and Y axes)? 392
14.8 How does vibration affect navigation accuracy? 392
14.9 Can the IMX operate underwater? 392
14.10 Can the IMX operate at >4g acceleration? 392
14.11 Customer Support 392
15. Inertial Sense Navigation Glossary 393
16. Navigation Systems 393
17. Navigation State Variables 394
18. Coordinate Frames 395
19. Attitude Representations 396
20. Strapdown Navigation Equations 397
21. IMU Measurement Terms 397
22. Sensor Error Models 398
23. Allan Variance Terms 399
24. GNSS Terminology 400
25. GNSS Measurement Models 401
26. Kalman Filtering 401
27. INS/GNSS Integration 402
28. Navigation Performance Metrics 403
29. Inertial Sense Protocol 403
30. Software Tools 403
31. Hardware Products 404
32. Common Abbreviations 405
33. Troubleshooting 406
33.1 Firmware Troubleshooting 406
33.2 IMX Firmware Troubleshooting 410

©2022

Software Release 3.1.1 - October 1, 2026

Older (non-current) versions of the User Manual can be found on GitHub under the specific release tags.

INERTIALSENSE
autonomous navigation solutions

1. Overview

1.1 IMX (IMU, AHRS, and GPS-INS)

The **IMX**™ family of sensors are 10-DOF sensor modules consisting of a tactical grade Inertial Measurement Unit (IMU), magnetometer, and barometer. Output includes angular rate, linear acceleration, magnetic vector, and barometric pressure and altitude. IMU calibration consists of bias, scale factor, cross-axis alignment, and temperature compensation. The IMX sensors

©2022

include Attitude Heading Reference System (**AHRS**) sensor fusion to estimate roll, pitch, and heading. Adding GNSS input to an IMX enables onboard Inertial Navigation System (**INS**) sensor fusion for roll, pitch, heading, velocity, and position.

The **RUGGED** series of products include a rugged aluminum enclosure, RS232, RS485, and CAN bus to the IMX.

The **RUGGED RTK** series include a multi-frequency GNSS receiver with RTK precision position enabling INS sensor fusion for roll, pitch, heading, velocity, and position.

The **RUGGED DUAL** series include two multi-frequency GNSS receivers with RTK precision position and/or dual GNSS heading/compass.

The **Inertial Sense SDK** is an open-source software development kit for quick integration to configure and communicate with Inertial Sense products. The SDK includes data logger, math libraries, and interface for Linux, Windows, and embedded platforms.

1.2 Features

- **Tactical-Grade IMU**
- **Reflow-Compatible PCB Module**
- **Performance Specifications**

	IMX-5	**IMX-6**
Gyro Bias Instability	2.0 °/hr	0.9 °/hr
Gyro ARW	0.23 °/√hr	0.10 °/√hr
Accel Bias Instability	27 µg	4.0 µg
Accel VRW	0.020 m/s/√hr	0.011 m/s/√hr
Dynamic Roll/Pitch	0.04°	0.03°
Dynamic Heading	0.13°	0.09°
Performance specifications per IEEE-STD-952-1997.		

- Output Data Rate Up to 8 kHz IMU and 500 Hz INS AHRS
- Small Form Factor: 15.6 x 12.5 x 2.9 mm
- Light Weight: 0.9 g
- Low power consumption: <120mW
- External GNSS Support (Multi-Band)
- Attitude (Roll, Pitch, Yaw, Quaternions), Velocity, and Position UTC Time Synchronized
- Triple Redundant IMUs Calibrated for Bias, Scale Factor, Cross-axis Alignment, and G-sensitivity
- -40°C to 85°C Sensor Temperature Calibration
- Binary and NMEA Protocol
- Barometric Pressure and Humidity
- Strobe In/Out Data Sync (Camera Shutter Event)
- Fast Integration with SDK and Example Software

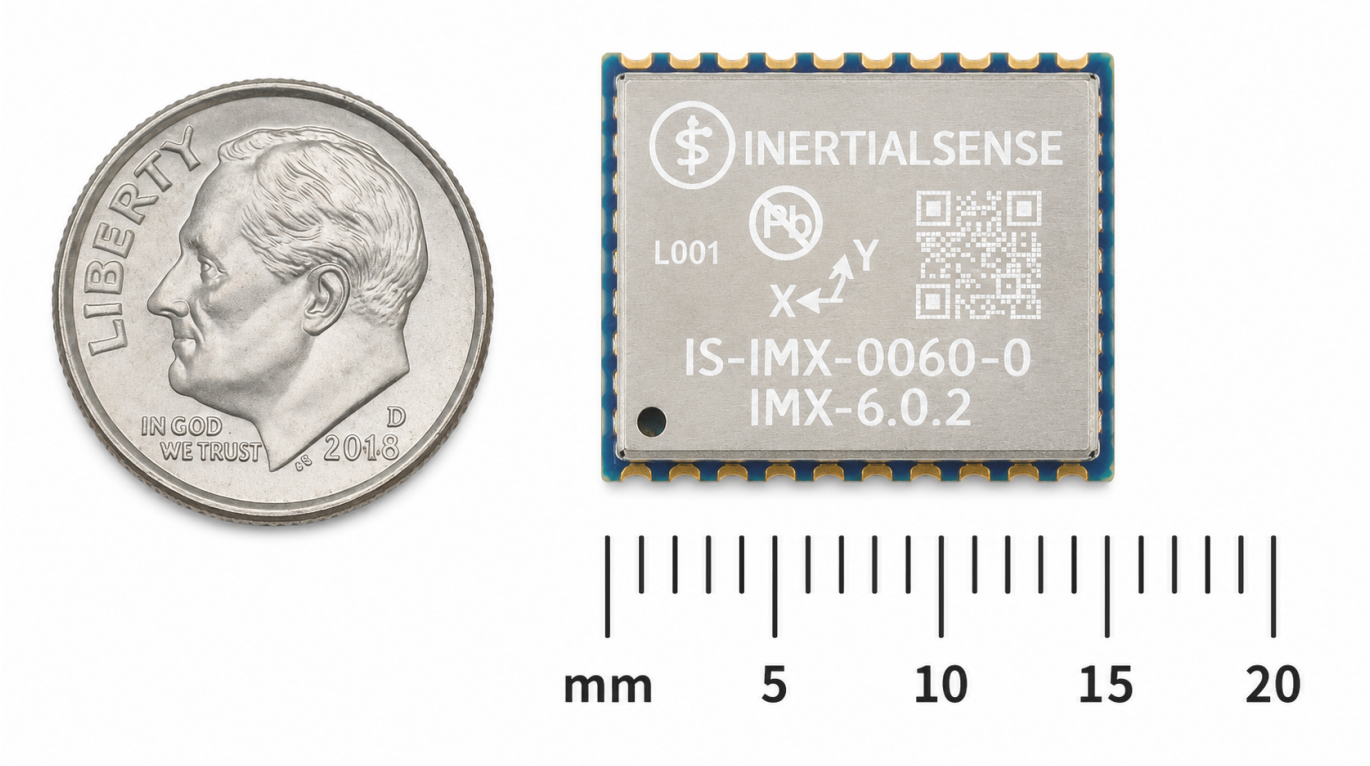
©2022

1.3 Interfaces

	IMX Module	Rugged
USB	Yes	Yes
UART	Yes	Yes
RS232/RS422/RS485	No	Yes
CAN	Yes	Yes
SPI	Yes	Yes
GPS Antenna Ports (Dual=Compassing)	No	Dual (Option)

1.4 Applications

- Drone Navigation
- Unmanned Vehicle Payloads
- Stabilized Platforms
- Antenna and Camera Pointing
- First Responder and Personnel Tracking
- Pedestrian and Auto Outdoor / Indoor Navigation
- Health, Fitness, and Sport Monitors
- Hand-held Devices
- Robotics and Ground Vehicles
- Maritime



©2022

Inertial Sense, Inc.
3000 S Sierra Vista Way Suite 5, Provo, UT 84606 USA
Phone 801-610-6771
Email support@inertialsense.com
Website: InertialSense.com

© 2014-2026 Inertial Sense

Inertial Sense®, Inertial Sense logo and combinations thereof are registered trademarks or trademarks of Inertial Sense, Inc. Other terms and product names may be trademarks of others.

DISCLAIMER: The information in this document is provided in connection with Inertial Sense products. No license, express or implied, by estoppel or otherwise, to any intellectual property right is granted by this document or in connection with the sale of Inertial Sense products. EXCEPT AS SET FORTH IN THE INERTIAL SENSE TERMS AND CONDITIONS OF SALES LOCATED ON THE INERTIAL SENSE WEBSITE, INERTIAL SENSE ASSUMES NO LIABILITY WHATSOEVER AND DISCLAIMS ANY EXPRESS, IMPLIED OR STATUTORY WARRANTY RELATING TO ITS PRODUCTS INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR NON-INFRINGEMENT. IN NO EVENT SHALL INERTIAL SENSE BE LIABLE FOR ANY DIRECT, INDIRECT, CONSEQUENTIAL, PUNITIVE, SPECIAL OR INCIDENTAL DAMAGES (INCLUDING, WITHOUT LIMITATION, DAMAGES FOR LOSS AND PROFITS, BUSINESS INTERRUPTION, OR LOSS OF INFORMATION) ARISING OUT OF THE USE OR INABILITY TO USE THIS DOCUMENT, EVEN IF INERTIAL SENSE HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. Inertial Sense makes no representations or warranties with respect to the accuracy or completeness of the contents of this document and reserves the right to make changes to specifications and products descriptions at any time without notice. Inertial Sense does not make any commitment to update the information contained herein. Unless specifically provided otherwise, Inertial Sense products are not suitable for, and shall not be used in, automotive applications. Inertial Sense products are not intended, authorized, or warranted for use as components in applications intended to support or sustain life. SAFETY-CRITICAL, MILITARY, AND AUTOMOTIVE APPLICATIONS DISCLAIMER: Inertial Sense products are not designed for and will not be used in connection with any applications where the failure of such products would reasonably be expected to result in significant personal injury or death (“Safety-Critical Applications”) without an Inertial Sense officer's specific written consent. Safety-Critical Applications include, without limitation, life support devices and systems, equipment or systems for the operation of nuclear facilities and weapons systems. Inertial Sense products are not designed nor intended for use in military or aerospace applications or environments unless specifically designated by Inertial Sense as military-grade.

©2022

2. Data Sheets

2.1 IMX-6 (IMU, AHRS, GNSS-INS)

Download Datasheet

IMX-6.0 Datasheet

2.2 RUG-4 GNSS Variant Comparison

Download Comparison Table

2.3 IMX-5 (IMU, AHRS, GNSS-INS) - Legacy Product

End of Life Notification

Download Datasheet

2.4 GPX-1 (Multi-Band L1/L5 Dual GNSS Receiver)

Download Datasheet

©2022

3. Dimensions and Pinouts

3.0.1 IMX-6 (Module)

Download PDF

3.0.2 IMX-5 (Module)

Download PDF

3.0.3 GPX-1 (Module)

Download PDF

3.0.4 RUG-3-IMX-5

Download PDF

3.0.5 RUG-3-IMX-5-Dual

Download PDF

3.0.6 IG-1-IMX-5-Dual (Module)

Download PDF

3.1 Hardware Design Files

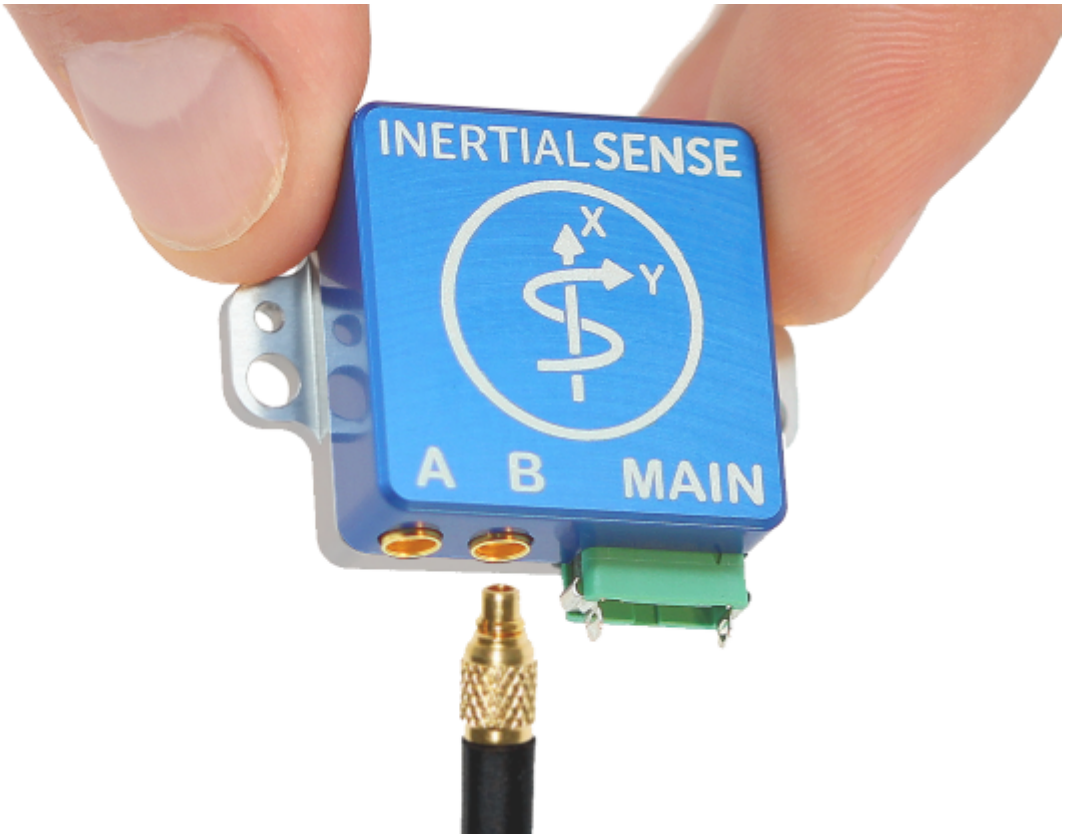
The Inertial Sense hardware design files are available on our IS-hdw repository to facilitate product hardware development and integration.

- **Products** - 3D models and resources for the IMX, Rugged, EVB, and products useful for CAD and circuit board designs. Libraries for schematic and layout designs for printed circuit board.

©2022

4. Getting Started

4.1 Getting Started



The Inertial Sense Development platform was designed to provide a method of rapid evaluation and integration. The following steps will provide a simple method to begin basic integration.

4.1.1 1. Install Software

Inertial Sense software provides a way to view, manipulate, stream, and record the data generated by an IMX-6, GPX-1, and the accompanying products.

- **EvalTool (Windows and Linux)**

  The EvalTool is a graphical Windows-based desktop program that allows users to explore, configure, and test functionality of the Inertial Sense products in real-time. Download the EvalTool installer from the Inertial Sense releases page. Run the .exe file and follow the instructions to complete the installation.

- **CLTool (Windows, Linux, and OS X)**

  The CLTool is a command line utility that can be used to read and display data, update firmware, and log data from Inertial Sense products. CLTool must be compiled from our source code. Follow the instructions on the CLTool page.

- **SDK (Windows, Linux)**

  Software development kit to interface with Inertial Sense products. Download the file named "Source code" from our releases page. The extracted folder contains code libraries as well as example projects.

©2022

4.1.2 2. Connect Hardware

Select the evaluation product from the list below to view instructions on basic connection to a computer.

- IMX-6 PCB Module
- GPX-1 PCB Module
- Rugged-3 Units
- IG-1 Units
- IG-2 Units
- IK-1 Units

4.1.3 3. Configuring Settings

The following sections contain instructions for basic configuration of IMX-6 based products and GPX-1 based products respectively:

- IMX-6 Basic Configuration
- GPX-1 Basic Configuration

4.1.4 4. Evaluation and Testing

Once a connection to the unit has been established, please follow one of the following guides to get started with the software tool of choice:

- EvalTool
- CLTool
- SDK Example Projects

©2022

4.2 GPX-1 Quick Start Guide

4.2.1 Basic Configuration

> **Note**
>
> When the GPX-1 is paired with an IMX-6, the IMX-6 will configure the GPX-1 automatically based on the configuration of the IMX-6. No further action is required.

The GPX-1 should be connected to a host via a communications port: USB, UART, or SPI.

The configuration settings found in DID_GPX_FLASH_CFG are used to configure the various features of the device. These can be modified directly to the appropriate values using either the EvalTool, the CLTool, or the SDK.

Configure the antenna offsets. When a GPX is not paired with an IMX the antenna offsets must be defined. In this scenario simply measure the distance between Antenna 1 and Antenna 2 and enter it as the value of gnss1AntOffset[0]. All other offset values should then be set to zero. This can be done easily in the EvalTool by going to the Data Sets Tab and selecting DID_GPX_FLASH_CFG. The values of each field can then be entered.

1. gnss1AntOffset[X-Z] - Position of GPS1 antenna with respect to sensor frame.
2. gnss2AntOffset[X-Z] - Position of GPS2 antenna with respect to sensor frame.

©2022

GNSS data sets can then be requested by one of several methods:

1. Requesting NMEA data using the ASCE command. There is a convenient tool in the lower left corner of the Data Logs tab of the EvalTool.
2. Request data using the SDK commands: SDK Function
3. Use the EvalTool to modify the value of DID_GPX_RMC.bits as outlined in the SDK.

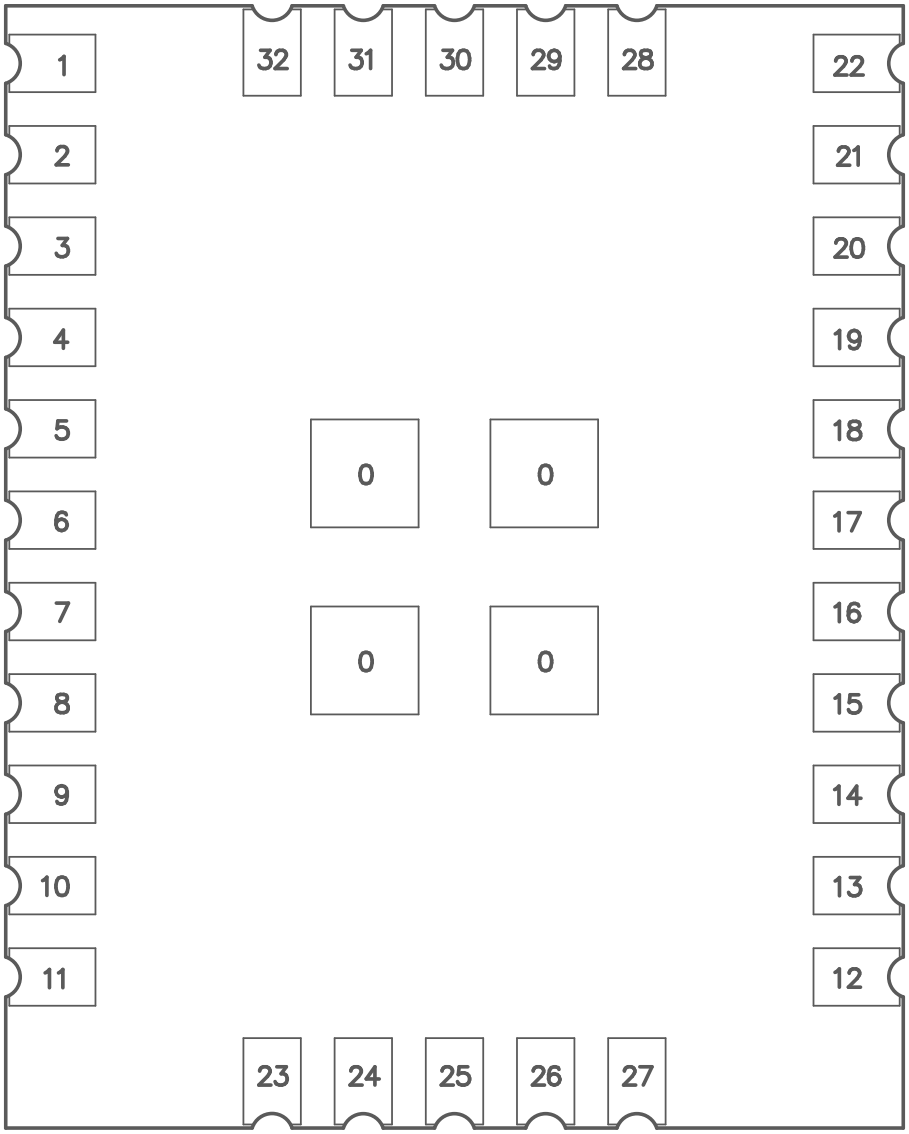
©2022

5. Hardware Integration

5.1 Hardware Integration: IMX-6 Module

©2022

5.1.1 Pinout



IMX-6 Pinout from the TOP

©2022

©2022

Pin	Name	I/O	Description
23-27	Not Connected	-	Not connected internally. Connect to ground (GND).
0,11,21	GND	-	Supply ground
1	USB_P	I/O	USB Data Positive Line
2	USB_N	I/O	USB Data Negative Line
3	VBKUP	I	Optional battery backup feature (reserved for future use, not currently supported) intended to enable deep-sleep and warm/hot start operation in future firmware. For future use, VBKUP may be connected to a 1.4V–3.6V backup battery. If no backup battery is used, VBKUP **must** be connected (tied) to VCC.
4	G1/Rx2/ RxCAN/SCL	I/O	GPIO1 Serial 2 input (TTL) Serial input pin from CAN transceiver* I2C SCL line
5	G2/Tx2/ TxCAN/SDA/ STRB	I/O	GPIO2 Serial 2 output (TTL) Serial output pin to CAN transceiver* I2C SDA line Strobe time sync input
6	G6/Rx1/MOSI	I/O	GPIO6 Serial 1 input (TTL) SPI MOSI
7	G7/Tx1/MISO	I/O	GPIO7 Serial 1 output (TTL) SPI MISO
8	G8/CS/STRB	I/O	GPIO8 SPI CS Strobe time sync input
9	G5/SCLK/ STRB	I/O	GPIO5 SPI SCLK Strobe time sync input
10	G9/nSPI_EN/ STRB/DRDY	I/O	GPIO9 SPI Enable: Hold LOW during boot to enable SPI on G5-G8 Strobe time sync input or output. SPI data ready (DRDY) is active-high on this alternate pin; data is available when high
12	nRESET	I	System reset on logic low. May be left unconnected if not used.
13	G14/SWCLK	I/O	GPIO14
14	G13/XSDA/ PPS2/DRDY	I/O	GPIO13 GNSS2 PPS Data Ready (DRDY) is active high Alt I2C SDA
15	G12/XSCL/ SWO	I/O	GPIO12 Alt I2C SCL
16	G11/SWDIO	I/O	GPIO11
17	G10/BOOT	I/O	

©2022

Pin	Name	I/O	Description
			Leave unconnected. BOOT mode used in manufacturing. !!! WARNING !!! Asserting a logic high (+3.3V) will cause the IMX to reboot into bootloader (DFU) mode.
18	G4/Rx0	I/O	GPIO4 Serial 0 input (TTL)
19	G3/Tx0	I/O	GPIO3 Serial 0 output (TTL)
20	G15/PPS1	I	Input for GNSS PPS for time synchronization pulse.
22	VCC	I	3.3V supply input
28	QDEC0.A	I	Ground vehicle wheel sensor 0 quadrature channel A input. Has an internal pull-up/pull-down resistor and can be left floating if not used.
29	QDEC0.B	I	Ground vehicle wheel sensor 0 quadrature channel B input. Has an internal pull-up/pull-down resistor and can be left floating if not used.
30	VUSB	I	3.0V to 3.6V required for USB operation. Can be left floating if USB is not needed and must not be connected to ground.
31	QDEC1.A	I	Ground vehicle wheel sensor 1 quadrature channel A input. Has an internal pull-up/pull-down resistor and can be left floating if not used.
32	QDEC1.B	I	Ground vehicle wheel sensor 1 quadrature channel B input. Has an internal pull-up/pull-down resistor and can be left floating if not used.

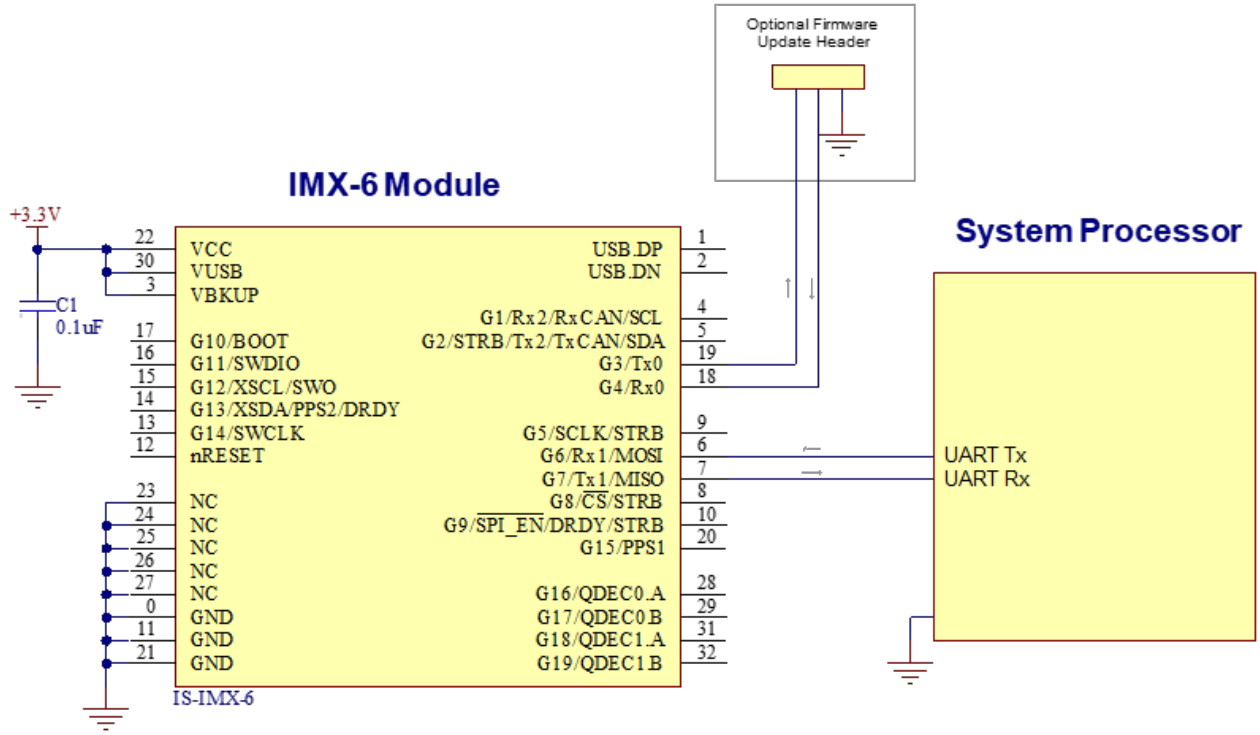
*External transceiver required for CAN interface.

5.1.2 Application

UART Interface

The following schematic demonstrates a typical setup for the IMX-6 module. If the system processor is not capable of updating the IMX firmware, it is recommended to add a header to an alternate IMX serial port for firmware updates via an external computer. The reset line is not necessary for typical use.

©2022



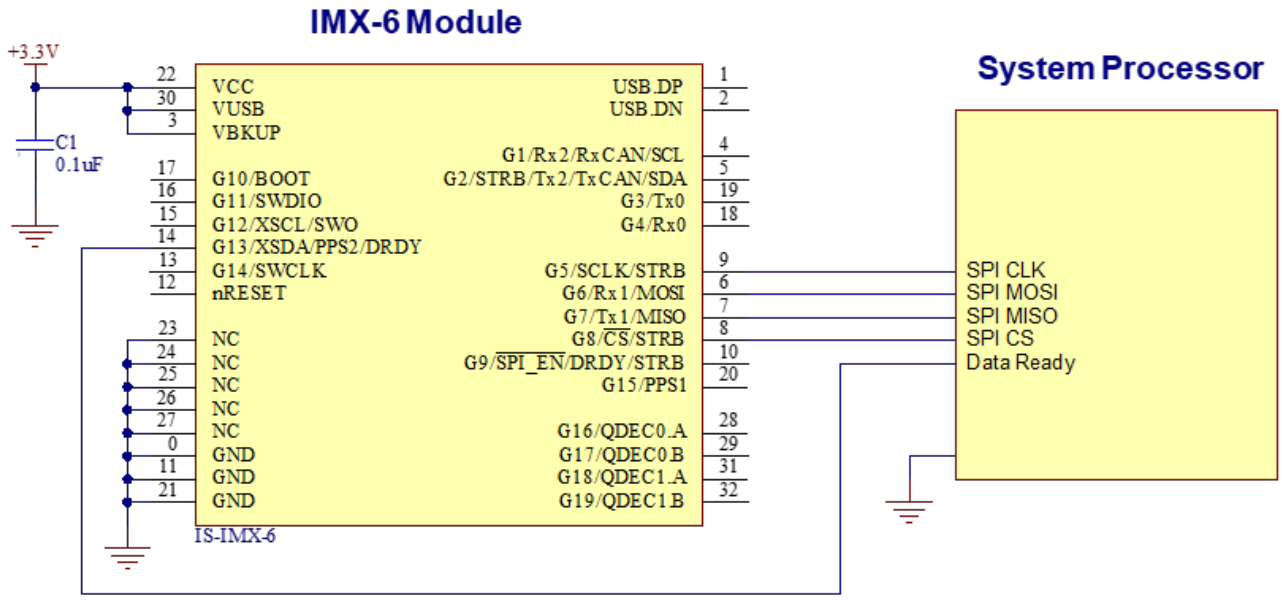
USB Interface

To use the USB interface, VUSB (pin 30) must be supplied with 3.0 V to 3.6 V (typically +3.3 V). If USB functionality is not required, VUSB may be left floating.

SPI Interface

The SPI interface is enabled by holding the pin 10 low during boot up.

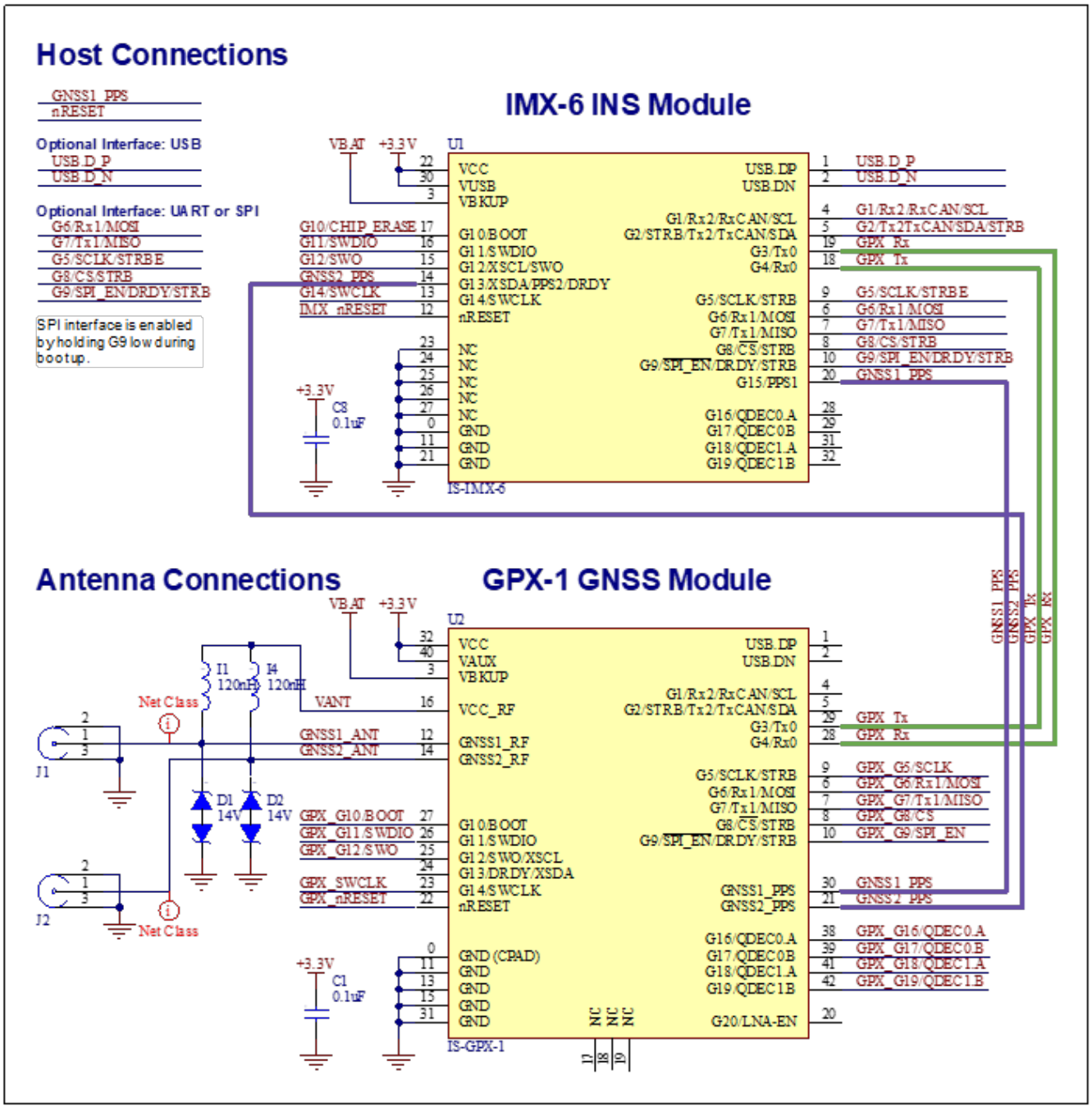
©2022



GNSS Interface

The following is an example of how to interface the IMX with a GNSS receiver.

©2022



5.1.3 Manufacturing

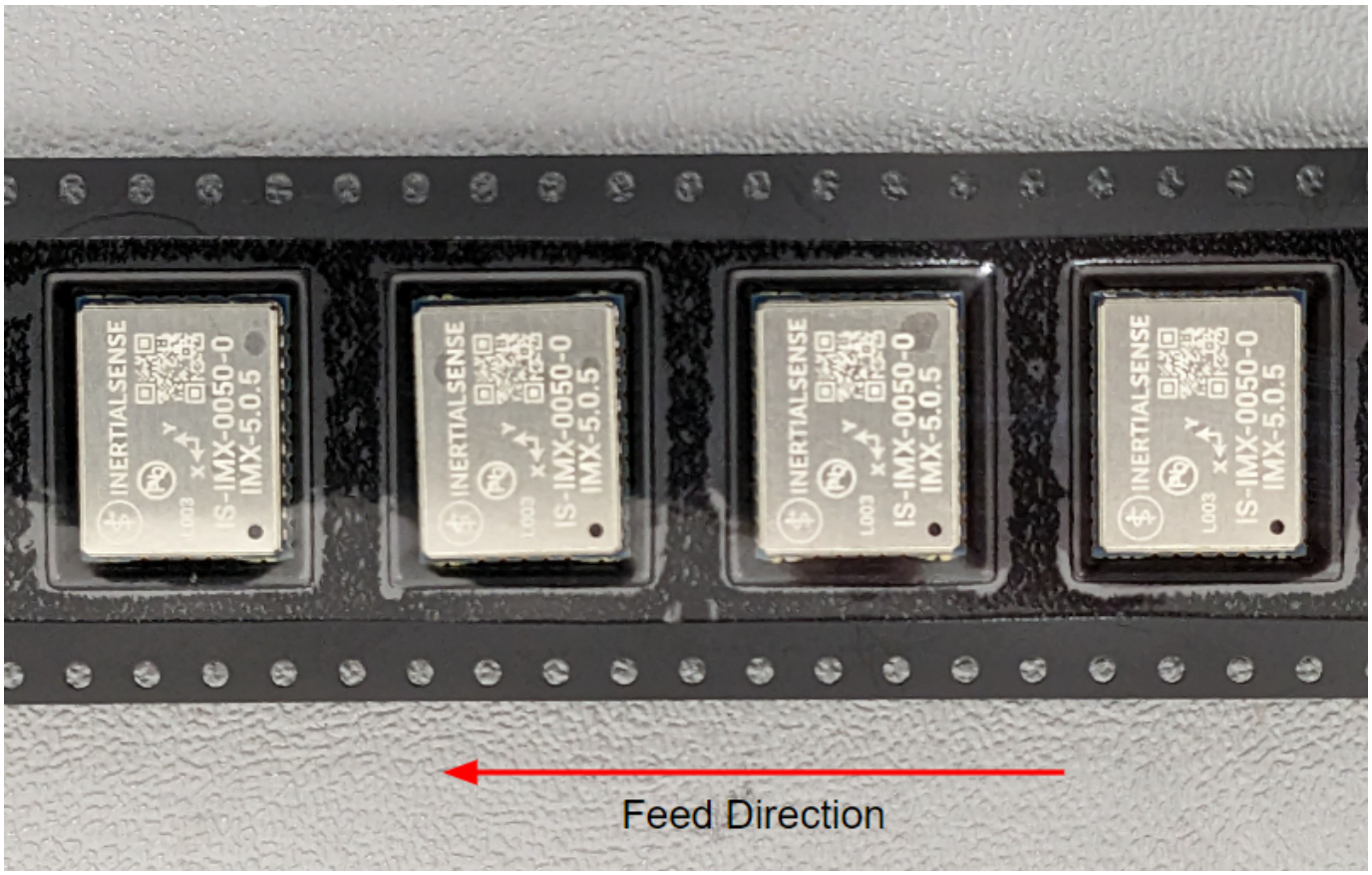
Soldering

The IMX-6 can be reflow soldered. Reflow information can be found in the Reflow Information page of this manual.

Tape Packaging

The IMX-6 modules are available in **cut tape** as well as **tape and reel** packaging. The following image shows the feed direction and illustrates the orientation of the IMX-6 module on the tape:

©2022



The feed direction to the pick and place pick-up is shown by the orientation of the IMX-6 pin 1 location. With pin 1 location on the bottom of the tape, the feed direction into the pick and place pick-up is from the reel (located to the right of the figure) towards the left.

The dimensions of the tapes for the IMX-6 are shown in the drawing below:

5.1.4 Hardware Design

Recommended PCB Footprint and Layout

A single ceramic 100nF decoupling capacitor should be placed between and in close proximity to the IMX pins 21 and 22 (GND and Vcc). It is recommended that this capacitor be on the same side of the PCB as the IMX and that there not be any vias between the capacitor and the Vcc and GND pins. The default forward direction is indicated in the PCB footprint figure and on the IMX shield as the X axis. The forward direction is reconfigurable in software as necessary.

Download PDF

5.1.5 Design Files

Open source hardware design files, libraries, and example projects for the IMX module are found at the Inertial Sense Hardware Design repository hosted on GitHub. These include schematic and layout files for printed circuit board designs, and 3D step models of the InertialSense products usable for CAD and circuit board designs.

open source hardware

Reference Design Projects

The IG-2.1 circuit board project serves as a reference design that illustrates implementation of the IMX PCB module.

IG-2.1 module

©2022

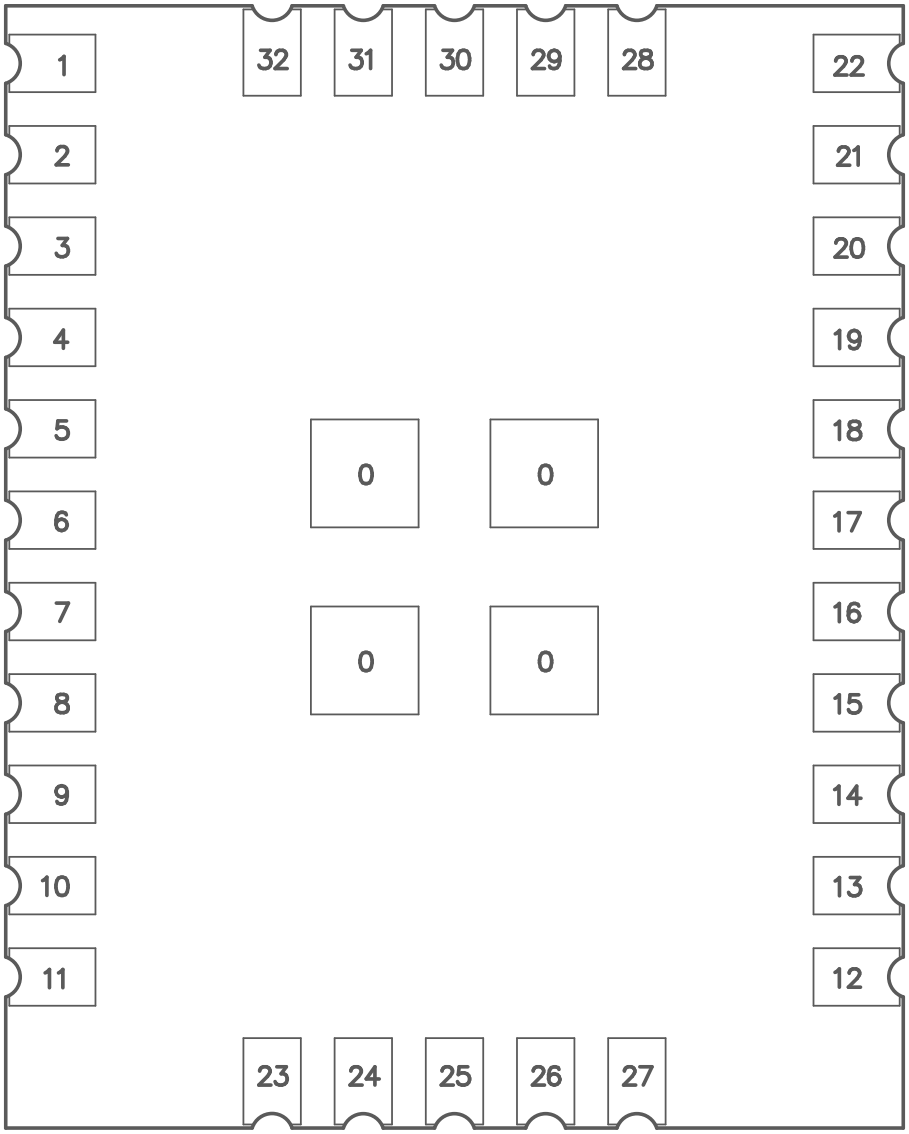
5.2 Hardware Integration: IMX-5 Module

Warning

The IMX-5 series of products is approaching its End of Life. Full Notification.

©2022

5.2.1 Pinout



IMX-5 Pinout from the TOP

©2022

©2022

Pin	Name	I/O	Description
23-29, 31,32	Not Connected	-	Not connected internally. Connect to ground (GND).
30	Not Connected	-	Not connected internally. (For IMX-6 compatibility, this pin must be tied to 3.3V to enable IMX-6 USB operation.)
0,11,21	GND	-	Supply ground
1	USB_P	I/O	USB Data Positive Line
2	USB_N	I/O	USB Data Negative Line
3	VBKUP	I	Connect VBKUP to VCC.
4	G1/Rx2/RxCAN/ SCL	I/O	GPIO1 Serial 2 input (TTL) Serial input pin from CAN transceiver* I2C SCL line
5	G2/Tx2/TxCAN/ SDA/STRB	I/O	GPIO2 Serial 2 output (TTL) Serial output pin to CAN transceiver* I2C SDA line Strobe time sync input
6	G6/Rx1/MOSI	I/O	GPIO6 Serial 1 input (TTL) SPI MOSI
7	G7/Tx1/MISO	I/O	GPIO7 Serial 1 output (TTL) SPI MISO
8	G8/CS/STRB	I/O	GPIO8 SPI CS Strobe time sync input
9	G5/SCLK/STRB	I/O	GPIO5 SPI SCLK Strobe time sync input
10	G9/nSPI_EN/ STRB/DRDY	I/O	GPIO9 SPI Enable: Hold LOW during boot to enable SPI on G5-G8 Strobe time sync input or output. SPI DRDY alternate pin (active-high: data available when high)
12	nRESET	I	System reset on logic low. May be left unconnected if not used.
13	G14/SWCLK	I/O	GPIO14
14	G13/XSDA/ PPS2/DRDY	I/O	GPIO13 GNSS2 PPS SPI data available when high Alt I2C SDA
15	G12/XSCL/SWO	I/O	GPIO12 Alt I2C SCL
16	G11/SWDIO	I/O	GPIO11
17	G10/BOOT	I/O	Leave unconnected. BOOT mode used in manufacturing. !!! WARNING !!! Asserting a logic high (+3.3V) will cause the IMX to reboot into ROM bootloader (DFU) mode.

©2022

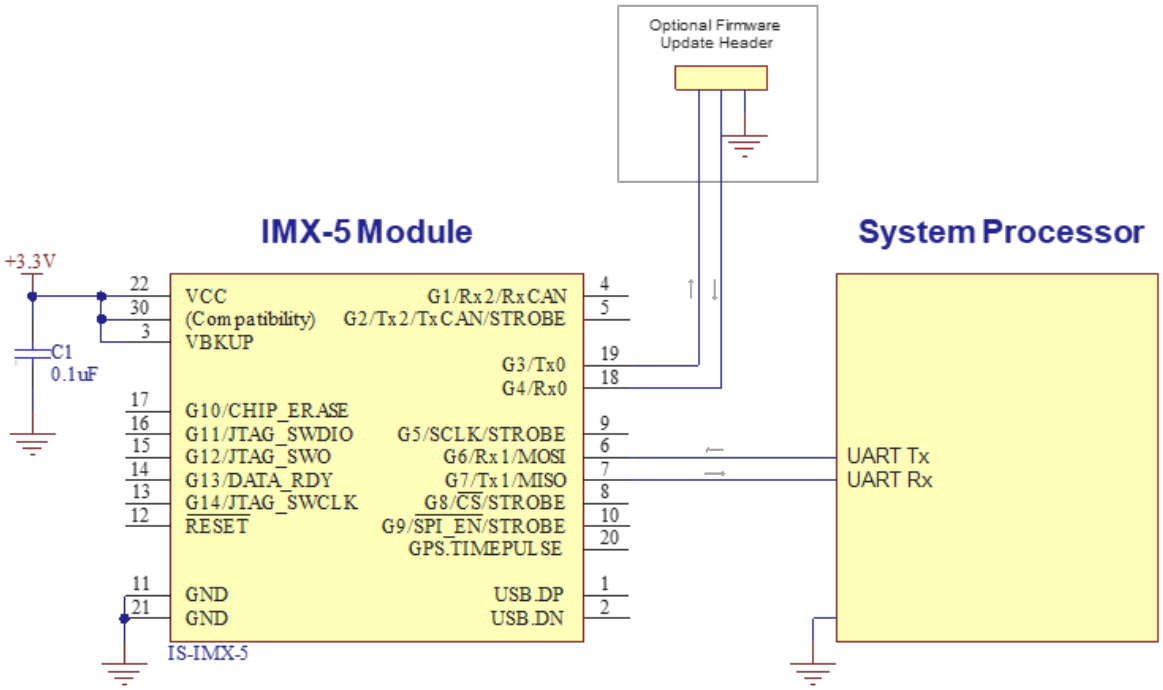
Pin	Name	I/O	Description
18	G4/Rx0	I/O	GPIO4 Serial 0 input (TTL)
19	G3/Tx0	I/O	GPIO3 Serial 0 output (TTL)
20	G15/PPS1	I	Input for GNSS PPS for time synchronization pulse.
22	VCC	I	3.3V supply input

*External transceiver required for CAN interface.

5.2.2 Application

UART Interface

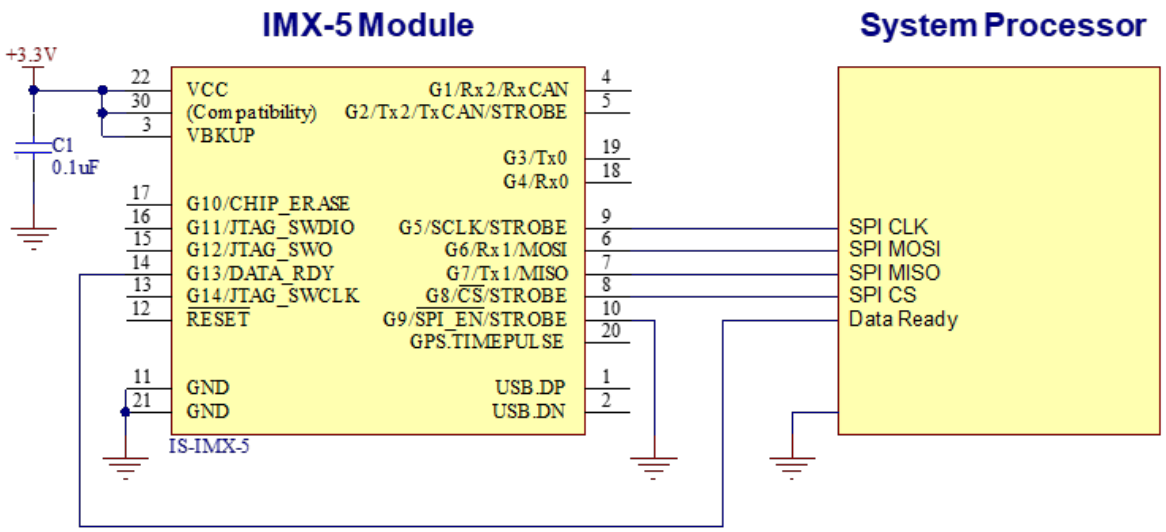
The following schematic demonstrates a typical setup for the IMX-5 module. If the system processor is not capable of updating the IMX firmware, it is recommended to add a header to an alternate IMX serial port for firmware updates via an external computer. The reset line is not necessary for typical use.



SPI Interface

The SPI interface is enabled by holding the pin 10 low during boot up.

©2022



5.2.3 Manufacturing

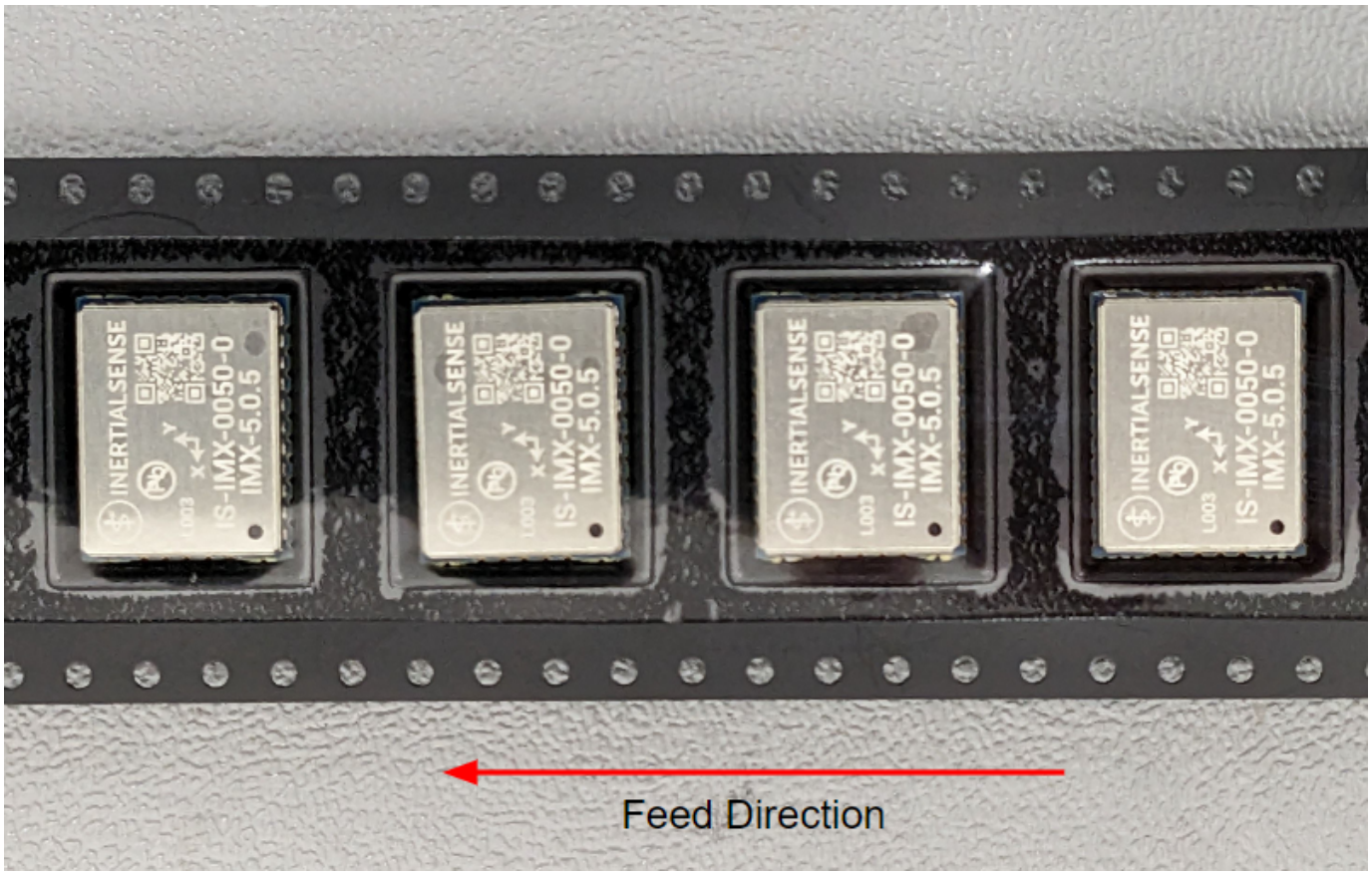
Soldering

The IMX-5 can be reflow soldered. Reflow information can be found in the Reflow Information page of this manual.

Tape Packaging

The IMX-5 modules are available in **cut tape** as well as **tape and reel** packaging. The following image shows the feed direction and illustrates the orientation of the IMX-5 module on the tape:

©2022



The feed direction to the pick and place pick-up is shown by the orientation of the IMX-5 pin 1 location. With pin 1 location on the bottom of the tape, the feed direction into the pick and place pick-up is from the reel (located to the right of the figure) towards the left.

The dimensions of the tapes for the IMX-5 are shown in the drawing below:

5.2.4 Hardware Design

Recommended PCB Footprint and Layout

A single ceramic 100nF decoupling capacitor should be placed between and in close proximity to the IMX pins 21 and 22 (GND and Vcc). It is recommended that this capacitor be on the same side of the PCB as the IMX and that there not be any vias between the capacitor and the Vcc and GND pins. The default forward direction is indicated in the PCB footprint figure and on the IMX shield as the X axis. The forward direction is reconfigurable in software as necessary.

Download PDF

5.2.5 Design Files

Open source hardware design files, libraries, and example projects for the IMX module are found at the Inertial Sense Hardware Design repository hosted on GitHub. These include schematic and layout files for printed circuit board designs, and 3D step models of the InertialSense products usable for CAD and circuit board designs.

Reference Design Projects

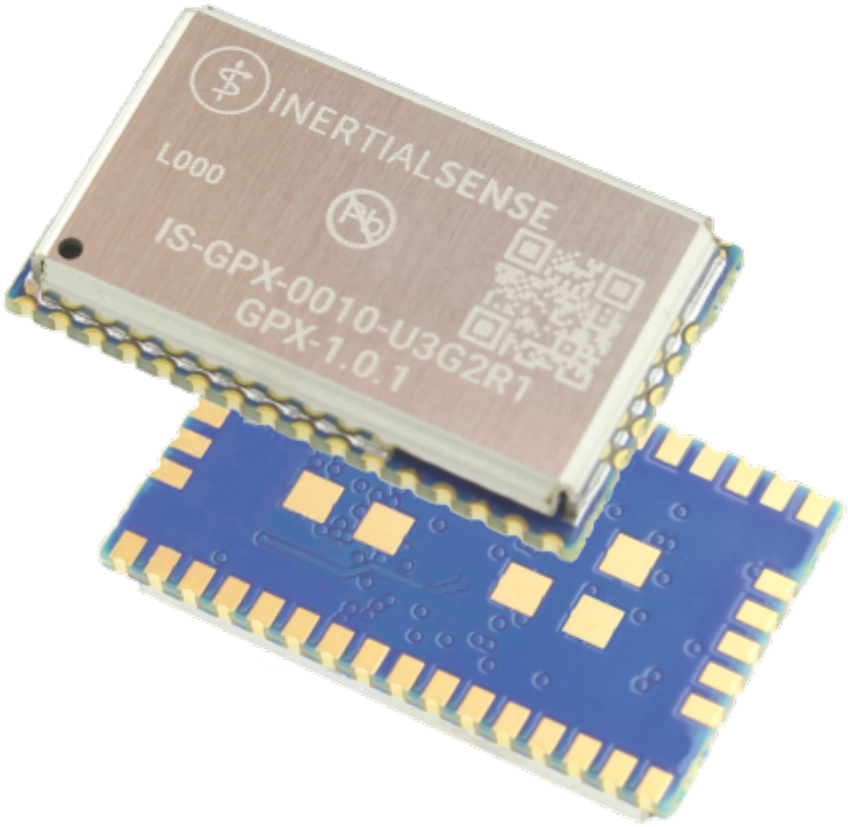
The EVB-2 and IG-1 circuit board projects serve as reference designs that illustrate implementation of the IMX PCB module.

EVB-2 evaluation board

IG-1 module

©2022

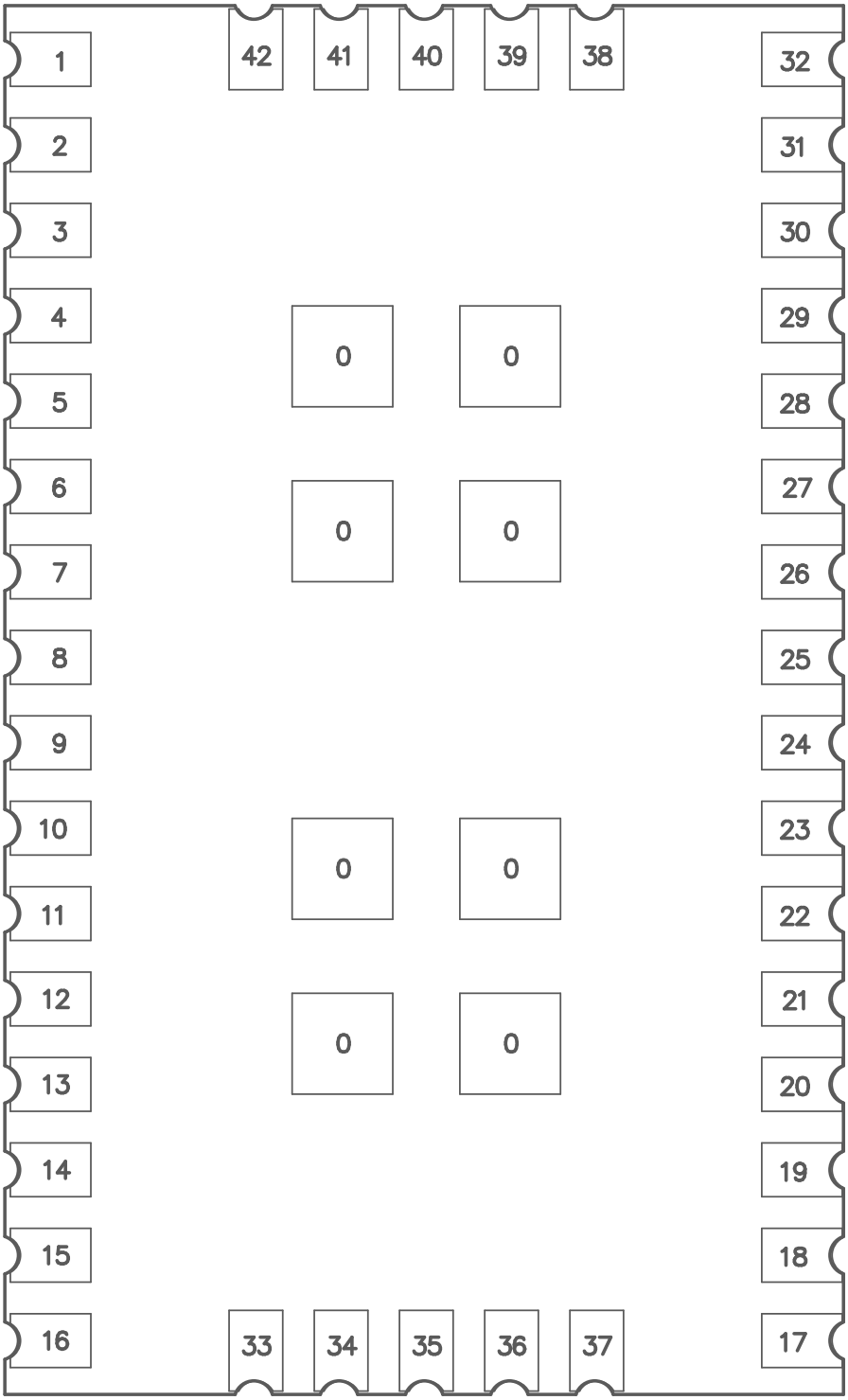
5.3 Hardware Integration: GPX-1 Module



5.3.1 Pinout

The GPX-1 module footprint and pinout similar that of the IMX-5 such that the common power and interface pins are at the same location. The GPX-1 is extended to accommodate additional GNSS inputs and output. The GPX-1 is designed to work in conjunction with the IMX-5.

©2022



GPX-1 Pinout from the TOP

©2022

©2022

Pin	Name	Type	Description
33-37	Not Connected	-	Not connected internally. Connect to ground (GND).
0	GND	Power	Supply ground on center pads.
11,13,15,31	GND	Power	Supply ground
1	USB_P	I/O	USB full-speed Positive Line. USB will be supported in future firmware updates.
2	USB_N	I/O	USB full-speed Negative Line. USB will be supported in future firmware updates.
3	VBKUP	Power	Backup supply voltage input (1.75V to 3.6V). Future firmware updates will use voltage applied on this pin to backup GNSS ephemeris, almanac, and other operating parameters for a faster startup when VCC is applied again. This pin MUST be connected to a backup battery or VCC.
4	G1/Rx2/ RxCAN/SCL	I/O	GPIO1 Serial 2 input (TTL) Serial input pin from CAN transceiver* I2C SCL line
5	G2/Tx2/ TxCAN/SDA/ STRB	I/O	GPIO2 Serial 2 output (TTL) Serial output pin to CAN transceiver* I2C SDA line Strobe time sync input
6	G6/Rx1/ MOSI	I/O	GPIO6 Serial 1 input (TTL) SPI MOSI
7	G7/Tx1/MISO	I/O	GPIO7 Serial 1 output (TTL) SPI MISO
8	G8/CS/STRB	I/O	GPIO8 SPI CS Strobe time sync input
9	G5/SCLK/ STRB	I/O	GPIO5 SPI SCLK Strobe time sync input
10	G9/nSPI_EN/ STRB/DRDY	I/O	GPIO9 SPI Enable: Hold LOW during boot to enable SPI on G5-G8 Strobe time sync input or output. SPI data ready alternate location
12	GNSS1_RF	I	GNSS1 antenna RF input. Use an active antenna or LNA with a gain of 15-25dB. Place the LNA as close to the antenna as possible. Filtered 3.3V from VAUX is injected onto the pad to power active antennas (power injection can be disabled in software). Connect to ground with 5V-14V TVS diode for ESD and surge projection (e.g. Littlefuse PESD0402-140).
14	GNSS2_RF	I	GNSS2 antenna RF input. Same requirements as GNSS1_RF
16	VCC_RF	O	Supply output for GNSS active antenna. Connect VCC_RF through 33-120nH inductor (e.g. Murata LQW15ANR12J00D, 110mA max) to GNSS1_RF and GNSS2_RF to inject DC supply for active antenna(s). VCC_RF is supplied from VAUX through an onboard 1A load switch and 10 ohm resistor. Disable by setting `GPX_SYS_CFG_BITS_DISABLE_VCC_RF` (0x00000001) in `DID_GPX_FLASH_CFG.sysCfgBits`.

©2022

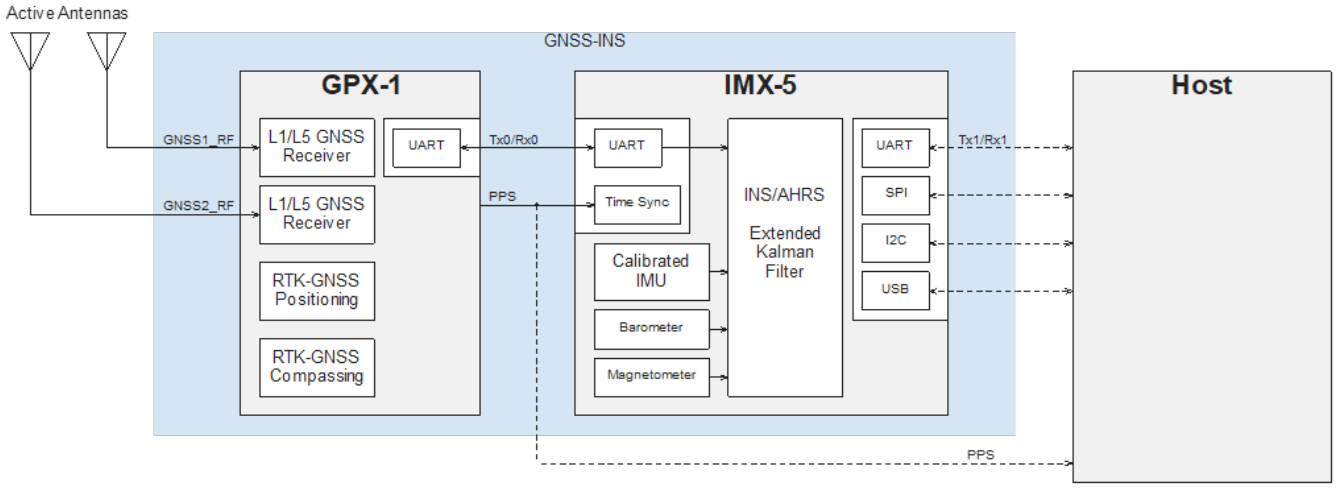
Pin	Name	Type	Description
17-19	Not Connected	-	Not connected internally. Connect to ground (GND).
20	G20/LNA-EN	I/O	GPIO20
21	GNSS2_PPS	O	GNSS2 PPS time synchronization output pulse (1Hz, 10% duty cycle)
22	nRESET	I	System reset on logic low. May be left unconnected if not used.
23	G14/SWCLK	I/O	GPIO14
24	G13/DRDY/ XSDA	I/O	GPIO13 SPI Data Ready Alt I2C SDA
25	G12/XSCL	I/O	GPIO12 Alt I2C SCL
26	G11/SWDIO	I/O	GPIO11
27	G10/BOOT	I/O	Leave unconnected. BOOT mode used in manufacturing. !!! WARNING !!! Asserting a logic high (+3.3V) will cause the IMX to reboot into ROM bootloader (DFU) mode.
28	G4/Rx0	I/O	GPIO4 Serial 0 input (TTL)
29	G3/Tx0	I/O	GPIO3 Serial 0 output (TTL)
30	GNSS1_PPS	O	GNSS1 PPS time synchronization output pulse (1Hz, 10% duty cycle)
32	VCC	Power	1.8V to 3.3V supply input.
38	G16/ QDEC0.A	I/O	GPIO16
39	G17/ QDEC0.B	I/O	GPIO17
40	VAUX	Power	Input supplies for the USB and VCC_RF (GNSS antenna supply). Connect to +3.3V (3.0V to 3.6V) to supply USB and VCC_RF. Can be left floating if USB or VCC_RF are not needed and must not be connected to ground.
41	G18/ QDEC1.A	I/O	GPIO18
42	G19/ QDEC1.B	I/O	GPIO19

*External transceiver required for CAN interface.

©2022

5.3.2 Application

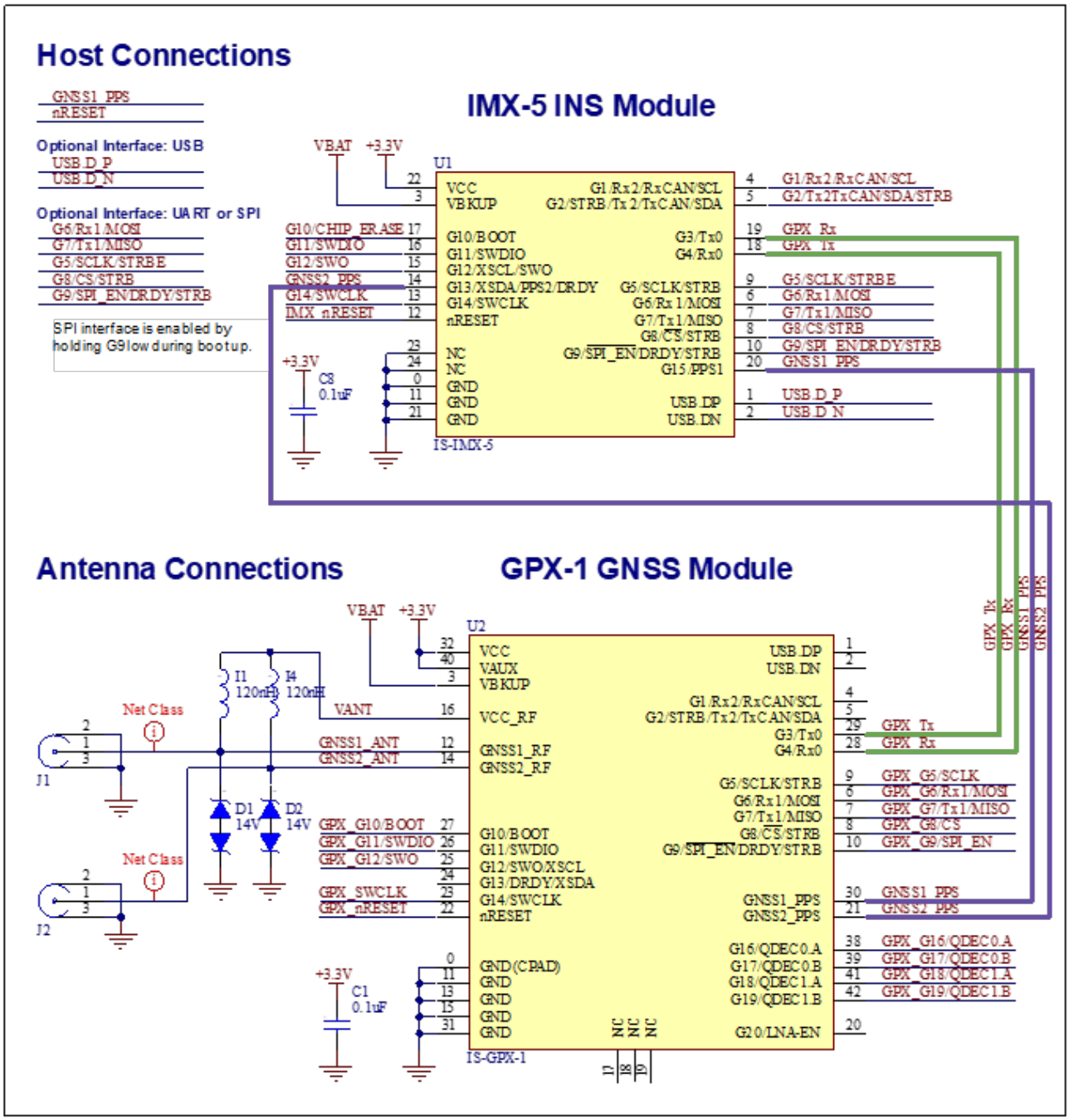
GNSS-INS Block Diagram



Typical Application: GPX-1 IMX-5

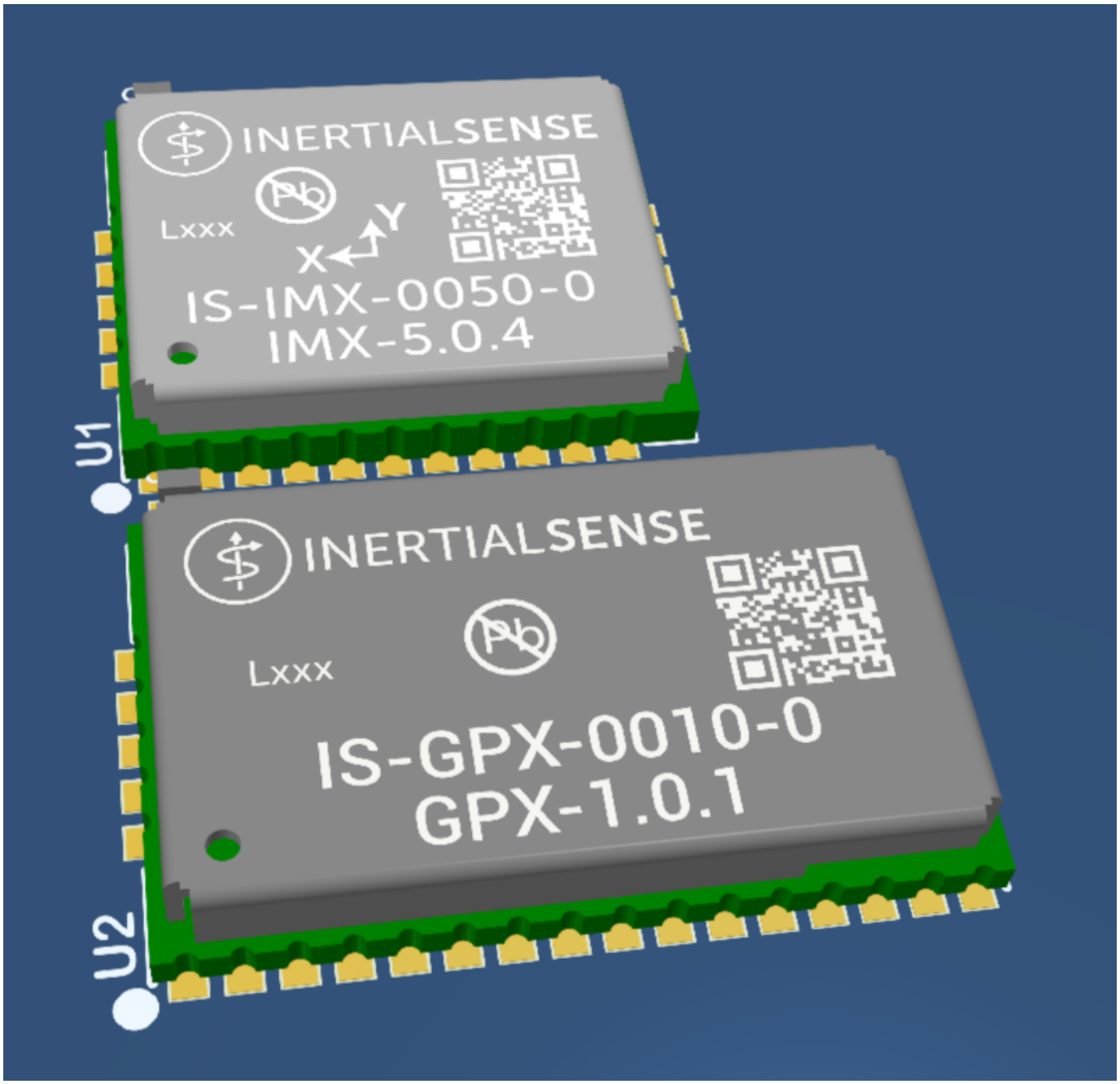
The following schematic shows the recommended interconnection for the GPX-1 and IMX-5. To enable GNSS PPS1 and PPS2 time synchronization input on the IMX-5 the corresponding enable bits must be set in `DID_FLASH_CONFIG.ioConfig` and `DID_FLASH_CONFIG.ioConfig2`.

©2022



Designator	Manufacturer	Part Number	Description
D1, D2	Littlefuse	PESD0402-140	TVS DIODE 14VWM 40VC 0402
I1, I4	Murata	LQW15ANR12J00D	FIXED IND 120NH 110MA 2.66OHM SM

©2022



5.3.3 Layout Guidance

GNSS_RF Trace

The GNSS_RF trace should be designed to work in the combined GNSS L1 + L5 signal band.

For FR-4 PCB material with a dielectric permittivity of for example 4.2, the trace width for the 50 Ω line impedance can be calculated.

A grounded co-planar RF trace is recommended as it provides the maximum shielding from noise with adequate vias to the ground layer.

The RF trace must be shielded by vias to ground along the entire length of the trace and the ZEDF9P RF_IN pad should be surrounded by vias as shown in the figure below.

[INSERT LAYOUT FIGURE HERE]

©2022

5.3.4 Design Guidance

Backup Battery

For achieving a minimal Time To First Fix (TTFF) after a power down (warm starts, hot starts), make sure to connect a backup battery to V_BCKP.

- Verify your battery backup supply can provide the battery backup current specified in the ZEDF9P datasheet.
- Allow all I/O including UART and other interfaces to float/high impedance in battery backup mode (battery back-up connected with VCC removed).

RF Front-end Circuit Options

> **Important**
>
> Active antenna(s) are required for the GPX-1.

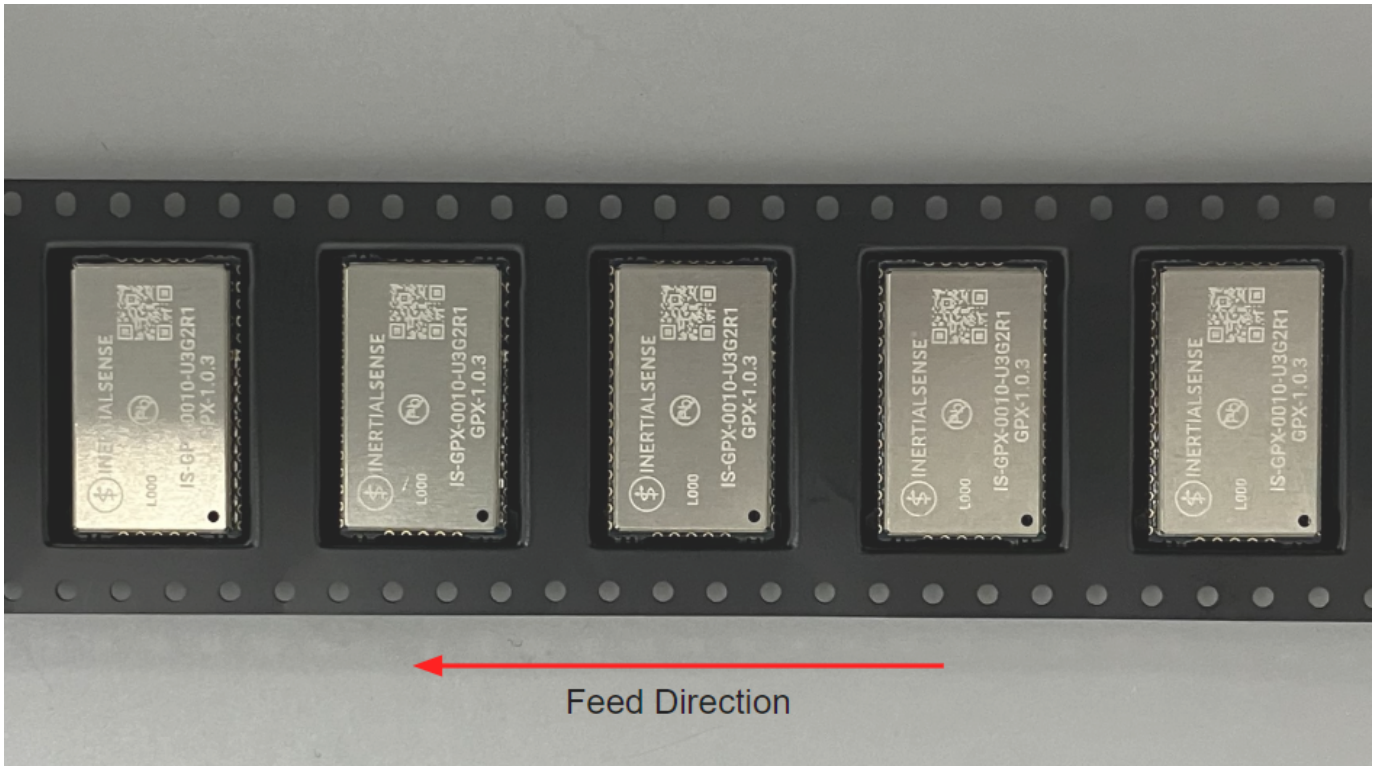
5.3.5 Manufacturing

Soldering

The GPX-1 can be reflow soldered. Reflow information can be found in the Reflow Information page of this manual.

Tape Packaging

The GPX-1 modules are available in **cut tape** as well as **tape and reel** packaging. The following image shows the feed direction and illustrates the orientation of the GPX-1 module on the tape:



The feed direction to the pick and place pick-up is shown by the orientation of the GPX-1 pin 1 location. With pin 1 location on the bottom of the tape, the feed direction into the pick and place pick-up is from the reel (located to the right of the figure) towards the left.

The dimensions of the tapes for the GPX-1 are shown in the drawing below:

©2022

5.3.6 Hardware Design

Recommended PCB Footprint and Layout

A single ceramic 100nF decoupling capacitor should be placed between and in close proximity to the module pins 31 and 32 (GND and Vcc). It is recommended that this capacitor be on the same side of the PCB as the GPX and that there not be any vias between the capacitor and the Vcc and GND pins.

Download PDF

5.3.7 Design Files

Open source hardware design files, libraries, and example projects for the GPX module are found at the Inertial Sense Hardware Design repository hosted on GitHub. These include schematic and layout files for printed circuit board designs, and 3D step models of the InertialSense products usable for CAD and circuit board designs.

Reference Design Projects

Coming soon

©2022

5.4 Hardware Integration: RUG-4 (Rugged-4)

The **RUG-4-G0** series adds a rugged aluminum enclosure and RS232, RS485, and CAN bus to the IMX-6.

The **RUG-4-GPX, RUG-4-X20, RUG-4-G5P3, and RUG-4-G5P8** expand on the RUG-4-G0 to include multi-frequency GNSS receiver(s) enabling INS sensor fusion for roll, pitch, heading, velocity, and position. The multi-frequency GNSS receivers are also RTK precision position and dual GNSS heading/compass capable.

5.4.1 Features

- **Tactical Grade IMU**
- **Gyro: 0.9 °/hr Bias Instability, 0.10 °/√hr ARW**
- **Accel: 4.0 µg Bias Instability, 0.011 m/s/√hr VRW**
- **INS, AHRS**
- **Dynamic: 0.03° Roll/Pitch, 0.09° Heading**
- **Static: 0.09° Roll/Pitch, 0.4° Heading**
- Output Data Rate: 2, 4, 8 kHz IMU, 500 INS AHRS
- Dual onboard all-band (L1/L2/L5/L6) GNSS receivers
- Dual MMCX antenna ports for GPS compassing
- Size: 25.4 x 25.4 x 20.0 mm
- Light weight: 14g
- Low power consumption: <1500mW
- UART x3, RS232, RS485, CAN, and SPI interfaces
- Integrated CAN and RS232 / RS485 transceivers
- Voltage regulation for 3.1V - 23V input

5.4.2 RUG-4 Comparison Table

Each variant of the GNSS enabled RUG-4 modules has different specifications and capabilities. More information can be found here:

Comparison Table

©2022

5.4.3 Applications

- Drone Navigation
- Unmanned Vehicle Payloads
- Ground and Aerial Survey
- Automotive Navigation
- Stabilized Platforms
- Antenna and Camera Pointing
- First Responder and Trackers
- Health, Fitness, and Sport Monitors
- Robotics and Ground Vehicles
- Maritime

5.4.4 Connecting Your Unit

For the purposes of basic evaluation, the easiest interface available on the rugged is the included USB to Gecko connector cable, included in the evaluation kit. The cable provides power and communications with the installed module via USB virtual communications port.

GPS Antenna Ports

If using GPS with the module, connect an appropriate antenna to MMCX port ***1*** (***GPS1***). If the module is used for RTK compassing, connect a second antenna to MMCX port ***2***, (***GPS2***).

5.4.5 Pinout

Warning

The pin numbering of the Rugged main connector does not match that of the connector manufacturer. Please refer to the drawings in the Dimensions and Pinouts page for the correct pin numbering.

©2022

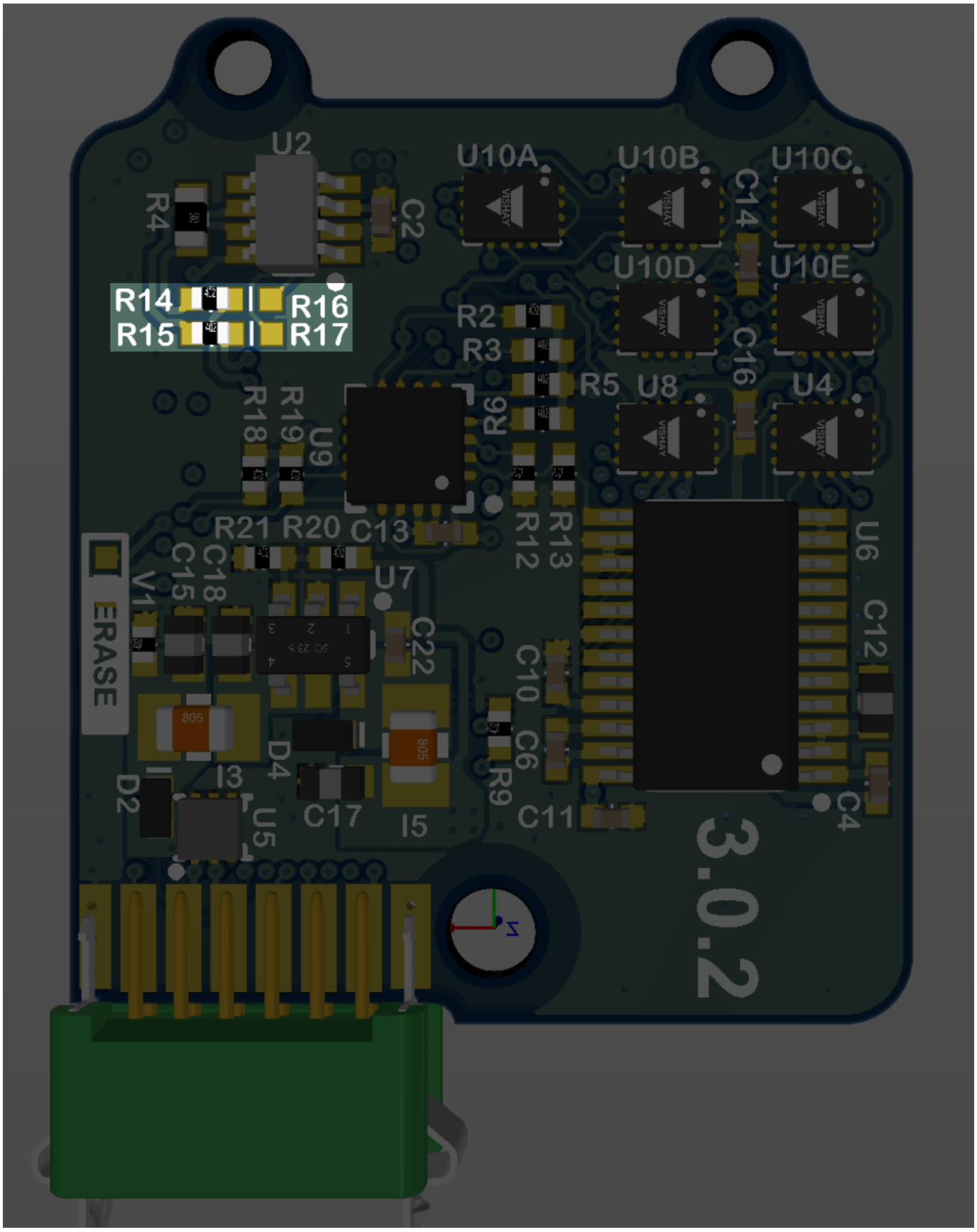
The following table shows the Rugged-4 pinout. Note that pin function can change based on changing `DID_FLASH_CONFIG.platformConfig` (see I/O Configuration below).

Rugged Pin	IMX Pin	Name	I/O	Description
1		GND	PWR	-
2	G9	nSPI_EN/STRB/ DRDY	I/O	(Includes 3K ohm series resistor) SPI Enable: Hold LOW during boot to enable SPI on G5-G8 G9-Strobe time sync input or output. SPI data ready
3	-	VIN	PWR	3.1V - 23V supply voltage input. Input voltage can be extended up to 42V by loading R1 SMT jumper.
4	1	USB.D+	I/O	USB Data Positive Line
5		PPS1	O	GNSS 1 PPS time synchronization output pulse (1Hz, 10% duty cycle)
6	2	USB.D-	I/O	USB Data Negative Line
7	G3 G2 G5	Tx0 485Tx1+ SCLK	O O I	Serial 0 output (TTL or RS232) Serial 1 output+ (RS485/RS422) SPI clock
8	G2 G2 G7	Tx2 485Tx1- Tx1, MISO	O O O	Serial 2 output (TTL) Serial 1 output- (RS485/RS422) Serial 1 output (TTL or RS232), SPI MISO
9	G4 G1 G8	Rx0 485Rx1- CS, G8_STROBE	I I I	Serial 0 input (TTL or RS232) Serial 1 input- (RS485/RS422) SPI chip select, G8-Strobe time sync input
10	G1 G1 G6	Rx2 485Rx1+ Rx1, MOSI	I I I	Serial 2 input (TTL) Serial 1 input+ (RS485/RS422) Serial 1 input (TTL or RS232), SPI MOSI
11	G1 G1	CANL* Rx2**	I/O I	High level (CAN bus) Serial 2 input (TTL)**
12	G2 G2	CANH* Tx2, G2_STROBE**	I/O I/O	Low level (CAN bus)*. Serial 2 output (TTL)**, G2-Strobe time sync input**

* The CAN bus is enabled by default on pins 11,12 (R16,R17 removed and R14,R15 loaded with 0402 zero ohm jumpers).

** To disable CAN bus and enable Serial2 TTL or STROBE on pins 11,12, remove R14,R15 and load R16,R17 with 0402 zero ohm jumpers.

©2022


U2
U10A
U10B
U10C
U10D
U10E
R4
C2
C14
C16
R14
R15
R16
R17
R2
R3
R5
U8
U4
R18
R19
U9
R6
R21
R20
C13
R12
R13
U6
C12
C18
C15
V1
U7
C22
ERASE
C10
C6
R9
C11
C4
D4
I3
D2
U5
C17
I5
3.0.2

©2022

5.4.6 I/O Configuration

The Rugged 4 "MAIN" connector pinout can be configured for USB, TTL, RS232, RS485, and CAN by setting the `DID_FLASH_CONFIG.platformConfig`.

RUG-4 Pin **IMX Pin** **I/O Preset**	**7,9** **G3,G4** **(G5,G8)**	**8,10** **G1,G2**	**11,12** **G1,G2**	**GPS1** **GPS2**
1 *	S0-RS232		CAN	S1
2	S0-TTL		CAN	S1
3	S0-TTL	S2-TTL or G2-STROBE		S1
4	S0-RS232	S1-RS232		S2
5	S1-RS485	S1-RS485		S2
6	SPI or G8-STROBE	SPI		S2
7 **		S1-RS232		S2
8			CAN	S1
9		S2-TTL		S1

* RUG-4-G0 default

** RUG-4-G2 default

5.4.7 Related Parts

Part	**Manufacturer**	**Manufacturer #**	**Description**
Main Connector	Harwin	G125- FC11205L0-0150L	1.25MM F/F 12POS 26AWG 150MM
GPS antenna SMA adapter	Crystek Corporation	CCSMX-FBM-RG178-6	6" MMCX to SMA GPS antenna adapter cable.
GPS antenna SMA adapter	Crystek Corporation	CCSMX1-FBM- RG178-6	6" R/A MMCX to SMA GPS antenna adapter cable.

See the Multi-Band GNSS page for GNSS antenna options for an L1/L2 receiver (Rugged-4).

See the Multi-Band GNSS page for GNSS antenna options for an L1/L5 receiver (GPX-1).

5.4.8 Using with Inertial Sense Software

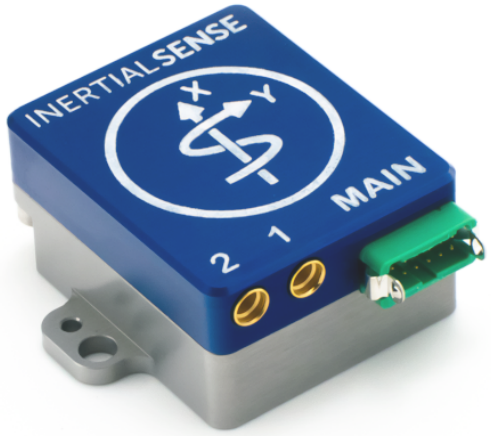
Please return to the getting started page to get started programming, updating firmware, viewing data, and logging.

©2022

5.5 Rugged-3

Warning

The IMX-5 series of products is approaching its End of Life. Full Notification.



The **RUG-3-IMX-5** series adds a rugged aluminum enclosure and RS232, RS485, and CAN bus to the IMX-5.

The **RUG-3-IMX-5-RTK** includes a multi-frequency GNSS receiver with RTK precision position enabling INS sensor fusion for roll, pitch, heading, velocity, and position.

The **RUG-3-IMX-5-Dual** includes two multi-frequency GNSS receivers with RTK precision position and dual GNSS heading/ compass.

- Integrated CAN transceiver, RS232, RS485, TTL serial, USB, and SPI interfaces.
- Dual onboard multi-band GNSS receiver(s).
- Dual antenna ports for GPS compassing.

5.5.1 Features

- **Tactical Grade IMU**
- **Gyro: 1.7 °/hr Bias Instability, 0.16 °/√hr ARW**
- **Accel: 19 µg Bias Instability, 0.02 m/s/√hr VRW**
- **INS, AHRS**
- **Dynamic: 0.04° Roll/Pitch, 0.13° Heading**
- **Static: 0.1° Roll/Pitch, 0.5° Heading**
- Up to 8 kHz IMU Output Data Rate
- Dual onboard multi-band (L1/L2/E5) GNSS receivers
- Dual MMCX antenna ports for GPS compassing
- Size: 25.4 x 25.4 x 20.0 mm
- Light weight: 14g

©2022

- Low power consumption: <1500mW
- UART x3, RS232, RS485, CAN, and SPI interfaces
- Integrated CAN and RS232 / RS485 transceivers
- Voltage regulation for 4V - 20V input

5.5.2 Applications

- Drone Navigation
- Unmanned Vehicle Payloads
- Ground and Aerial Survey
- Automotive Navigation
- Stabilized Platforms
- Antenna and Camera Pointing
- First Responder and Trackers
- Health, Fitness, and Sport Monitors
- Robotics and Ground Vehicles
- Maritime

5.5.3 Connecting Your Unit

For the purposes of basic evaluation, the easiest interface available on the rugged is the included USB to Gecko connector cable, included in the evaluation kit. The cable provides power and communications with the installed module via USB virtual communications port.

GPS Antenna Ports

If using GPS with the module, connect an appropriate antenna to MMCX port ***1*** (***GPS1***). If the module is used for RTK compassing, connect a second antenna to MMCX port ***2***, (***GPS2***).

5.5.4 Pinout

> **Warning**
>
> The pin numbering of the Rugged main connector does not match that of the connector manufacturer. Please refer to the drawings in the Dimensions and Pinouts page for the correct pin numbering.

©2022

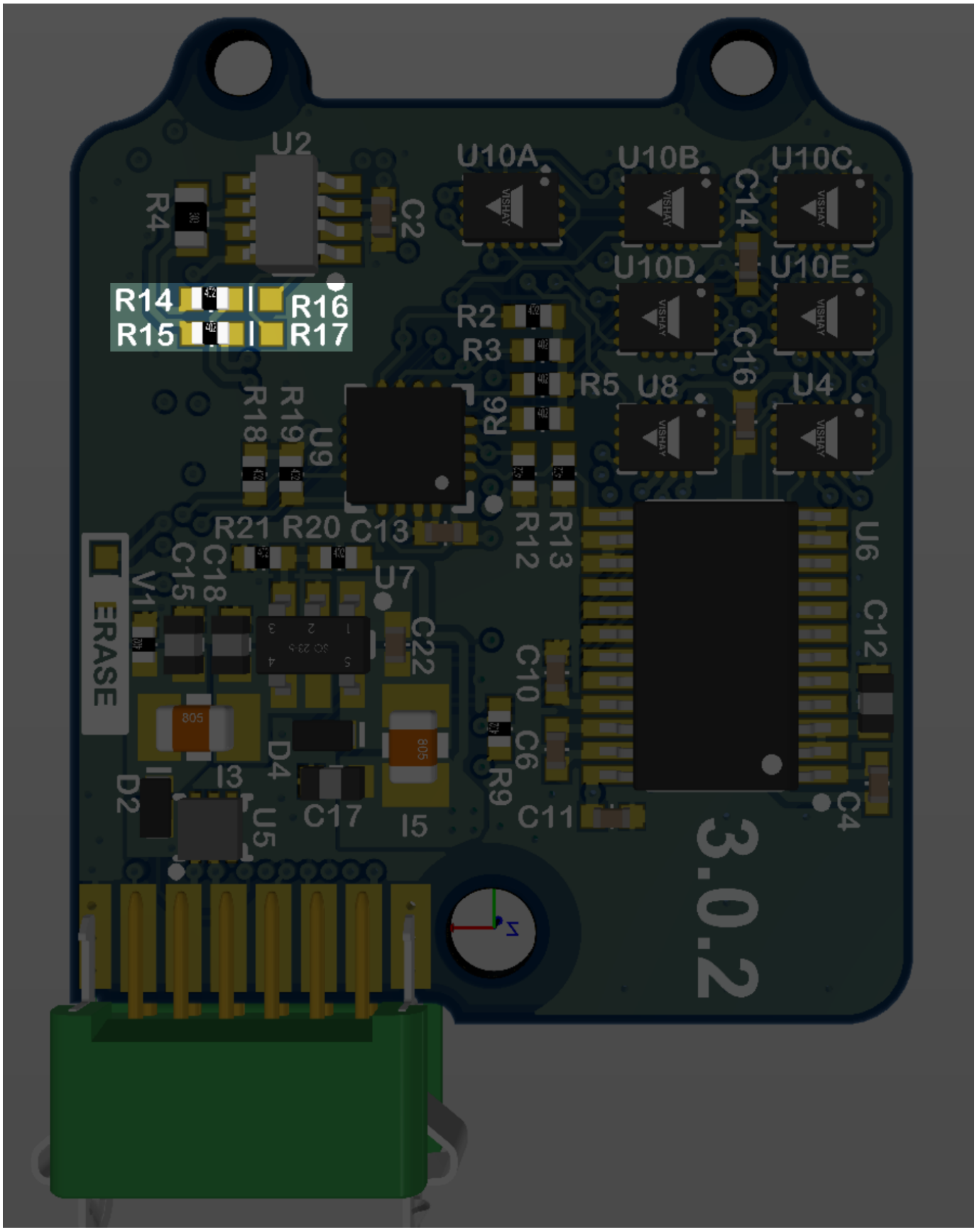
The following table shows the Rugged-3 pinout. Note that pin function can change based on changing `DID_FLASH_CONFIG.platformConfig` (see I/O Configuration below).

Rugged Pin	IMX Pin	Name	I/O	Description
1		GND	PWR	-
2	G9	nSPI_EN/STRB/ DRDY	I/O	(Includes 3K ohm series resistor) SPI Enable: Hold LOW during boot to enable SPI on G5-G8 G9-Strobe time sync input or output. SPI data ready
3		VIN	PWR	4V-20V supply voltage input
4		USB.D+	I/O	USB Data Positive Line
5		PPS1	O	GNSS 1 PPS time synchronization output pulse (1Hz, 10% duty cycle)
6		USB.D-	I/O	USB Data Negative Line
7	G3 G2 G5	Tx0 485Tx1+ SCLK	O O I	Serial 0 output (TTL or RS232) Serial 1 output+ (RS485/RS422) SPI clock
8	G2 G2 G7	Tx2 485Tx1- Tx1, MISO	O O O	Serial 2 output (TTL) Serial 1 output- (RS485/RS422) Serial 1 output (TTL or RS232), SPI MISO
9	G4 G1 G8	Rx0 485Rx1- CS, G8_STROBE	I I I	Serial 0 input (TTL or RS232) Serial 1 input- (RS485/RS422) SPI chip select, G8-Strobe time sync input
10	G1 G1 G6	Rx2 485Rx1+ Rx1, MOSI	I I I	Serial 2 input (TTL) Serial 1 input+ (RS485/RS422) Serial 1 input (TTL or RS232), SPI MOSI
11	G1 G1	CANL* Rx2**	I/O I	High level (CAN bus) Serial 2 input (TTL)**
12	G2 G2	CANH* Tx2, G2_STROBE**	I/O I/O	Low level (CAN bus)*. Serial 2 output (TTL)**, G2-Strobe time sync input**

* The CAN bus is enabled by default on pins 11,12 (R16,R17 removed and R14,R15 loaded with 0402 zero ohm jumpers).

** To disable CAN bus and enable Serial2 TTL or STROBE on pins 11,12, remove R14,R15 and load R16,R17 with 0402 zero ohm jumpers.

©2022


U2
U10A
U10B
U10C
U10D
U10E
U8
U4
U9
U7
U6
R4
C2
C14
C16
R14
R15
R16
R17
R2
R3
R5
R6
R18
R19
R21
R20
C13
R12
R13
C18
C15
V1
C22
C10
C12
C6
R9
C11
C4
C17
I5
I3
D4
D2
U5
ERASE
805
3.0.2

©2022

5.5.5 I/O Configuration

The Rugged 3 "MAIN" connector pinout can be configured for USB, TTL, RS232, RS485, and CAN by setting the `DID_FLASH_CONFIG.platformConfig`.

RUG-3 Pin **IMX Pin** **I/O Preset**	**7,9** **G3,G4** **(G5,G8)**	**8,10** **G1,G2**	**11,12** **G1,G2**	**GPS1**	**GPS2**
1 *	S0-RS232		CAN	S1	
2	S0-TTL		CAN	S1	
3	S0-TTL	S2-TTL or G2-STROBE		S1	
4	S0-RS232	S1-RS232		S2	
5	S1-RS485	S1-RS485		S2	S0
6	SPI or G8-STROBE	SPI		S2	S0
7 **		S1-RS232		S2	S0
8			CAN	S1	S0
9		S2-TTL		S1	S0

* RUG-3-G0 default

** RUG-3-G2 default

5.5.6 Hardware Versions

The following outlines differences in the RUG-3 hardware versions.

RUG-3.1

Use platform config RUG-3 with this hardware version.

- GPS1 PPS line connected to IMX G15 (pin 20).
- RS485 Tx pins polarity swapped between pins 7,8 (from RUG-2)

RUG-2.1 (RUG-3.0)

Use platform config RUG-2.1 with this hardware version.

- GPS1 PPS line connected to IMX G9 (pin 10).
- MAIN pin 5 is connected to IMX G15 (pin 20).
- MAIN pin 5 is not connected to GPS1 PPS.

©2022

5.5.7 Related Parts

Part	Manufacturer	Manufacturer #	Description
Main Connector	Harwin	G125-FC11205L0-0150L	1.25MM F/F 12POS 26AWG 150MM
GPS antenna SMA adapter	Crystek Corporation	CCSMX-FBM-RG178-6	6" MMCX to SMA GPS antenna adapter cable.
GPS antenna SMA adapter	Crystek Corporation	CCSMX1-FBM-RG178-6	6" R/A MMCX to SMA GPS antenna adapter cable.

See the Multi-Band GNSS page for GNSS antenna options for an L1/L2 receiver (Rugged-3).

See the Multi-Band GNSS page for GNSS antenna options for an L1/L5 receiver (GPX-1).

5.5.8 Using with Inertial Sense Software

Please return to the getting started page to get started programming, updating firmware, viewing data, and logging.

©2022

5.6 Hardware Integration: IG-2 (IMX5 + GPX1)

> **Warning**
>
> The IMX-5 series of products is approaching its End of Life. Full Notification.

The Inertial Sense IG-2 is a PCB module with IMX-5 and GPX-1 multi-frequency GNSS receiver.

- Surface mount reflowable.
- Onboard dual GNSS for simultaneous RTK positioning and GPS compassing.
- Micro USB and 14 pin I/O header for convenient evaluation.

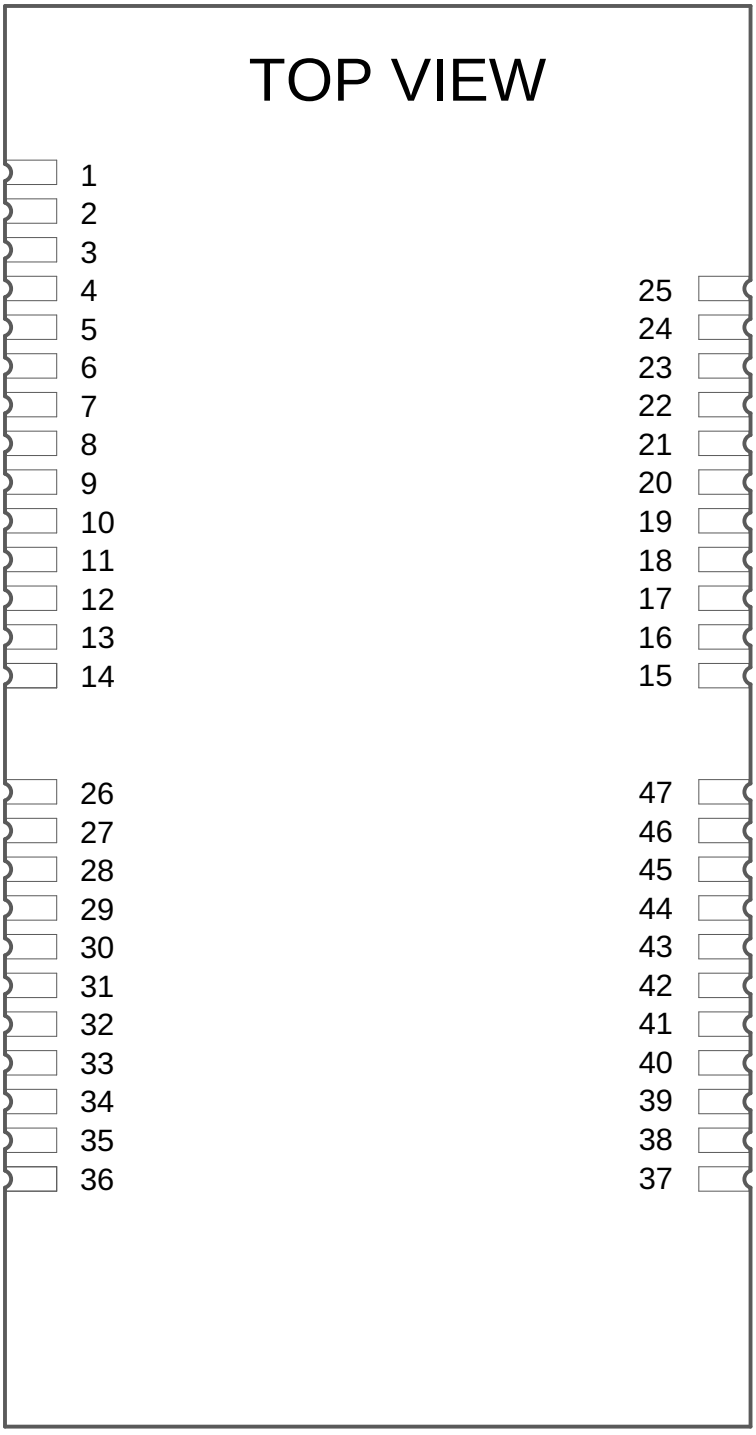
5.6.1 Connecting Your Unit

For the purposes of basic evaluation, the easiest interface available on the IG-2 is by using a micro-USB cable. A cable included in the evaluation kit. The cable provides power and communications with the installed module via USB virtual communications port.

5.6.2 Pinout

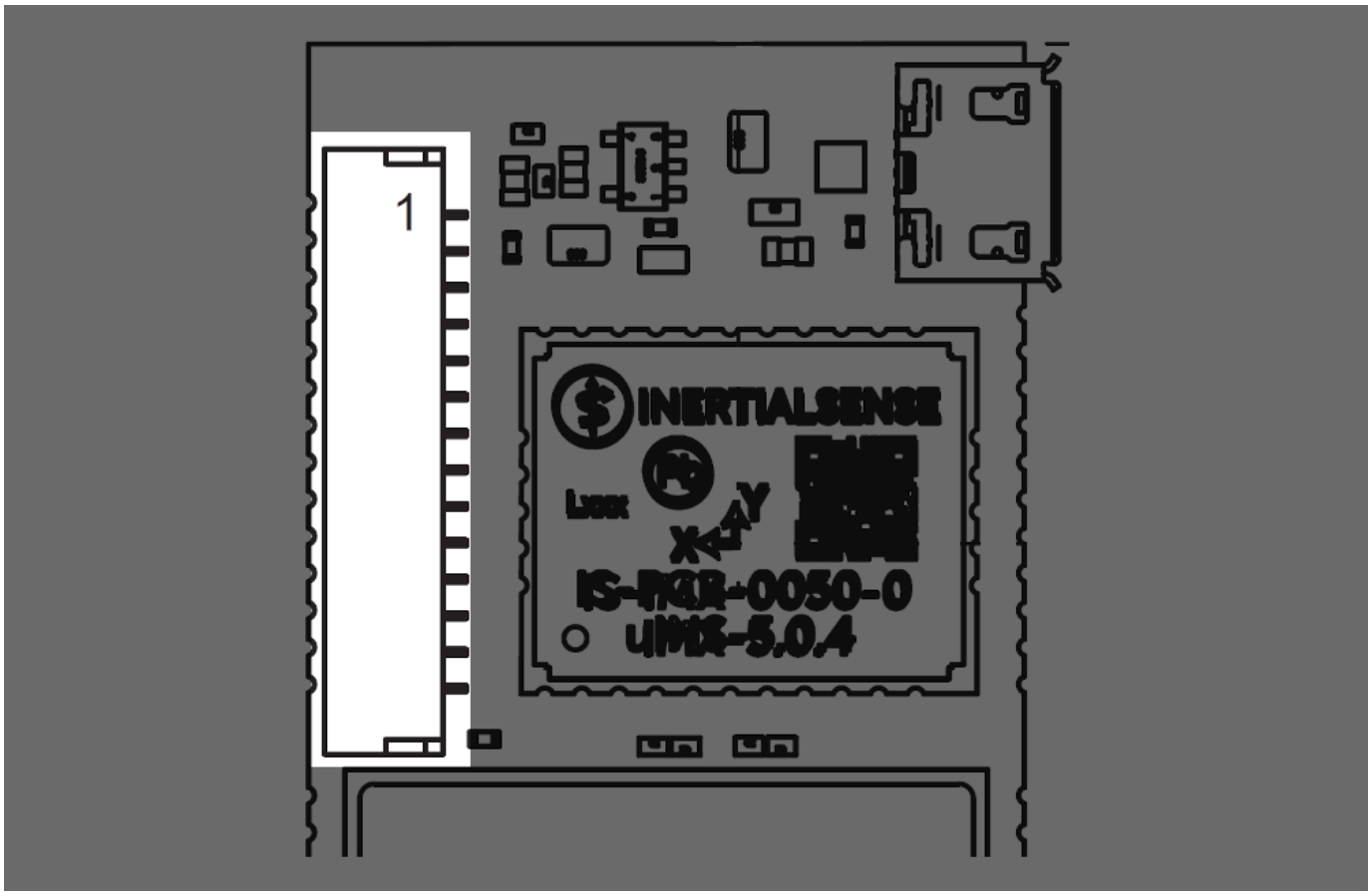
Module Pinout

©2022



Header H1 Pinout

©2022



©2022

The IG-2 module and IG-2 header H1 have the same pinout assignment for pins 1-14. Because H1 only has 14 pins, pins 15 and above listed in the following table are only on the IG-2 module.

©2022

IG-2 Module & IG-2 H1 Pin	Name	I/O	Description
1	GND	PWR	-
2	VIN	PWR	4V-20V supply voltage input
3	+3.3V	PWR	Regulated 3.3V supply input/output.
4	Reserved		Not Connected
5	G1/Rx2/RxCAN/SCL	I/O	IMX GPIO1 Serial 2 input (TTL) Serial input pin from CAN transceiver* I2C SCL line6
6	G2/Tx2/TxCAN/SDA/ STROBE	I/O	IMX GPIO2 Serial 2 output (TTL) Serial output pin to CAN transceiver* I2C SDA line Strobe time sync input
7	G3/Tx0	I/O	IMX GPIO3 Serial 0 output (TTL)
8	G4/Rx0	I/O	IMX GPIO4 Serial 0 input (TTL)
9	G5/SCLK/STROBE	I/O	IMX GPIO5 SPI SCLK Strobe time sync input
10	G6/Rx1/MOSI	I/O	IMX GPIO6 Serial 1 input (TTL) SPI MOSI
11	G7/Tx1/MISO	I/O	IMX GPIO7 Serial 1 output (TTL) SPI MISO
12	G8/CS/STROBE	I/O	IMX GPIO8 SPI CS Strobe time sync input
13	G9/nSPI_EN/ STROBE /STROBE_OUT/ SPI_DRDY	I/O	IMX GPIO9 SPI Enable: Hold LOW during bootup to enable SPI on G5-G8 Strobe time sync input or output. SPI data ready alternate location.
14	GPS.TIMEPULSE	O	GPS PPS UTC time synchronization signal.
15	GND	PWR	-
16	VBAT	I/O	GPS backup supply voltage. (1.4V to 3.6V) enables GPS hardware backup mode for hot or warm startup (faster GPS lock acquisition). MUST connect GPS_VBAT to VCC if no backup battery is used.
17	G10/BOOT_MODE	I/O	Leave unconnected. IMX BOOT MODE used in manufacturing. !!! WARNING !!! Asserting a logic high (+3.3V) will cause the IMX to reboot into ROM bootloader (DFU) mode.

©2022

IG-2 Module & IG-2 H1 Pin	Name	I/O	Description
18	G11	I/O	IMX GPIO11
19	G12	I/O	IMX GPIO12 GPS reset
20	G13/DRDY	I/O	IMX GPIO13 SPI data ready
21	G14/SWCLK	I/O	IMX GPIO14
22	nRESET	I	System reset (IMX and GPX) on logic low. May be left unconnected if not used.
23	GND	PWR	-
24	USB_N	I/O	IMX USB Data Negative Line
25	USB_P	I/O	IMX USB Data Positive Line
26	GPX_G16/QDEC0A	I/O	GPX GPIO16
27	GPX_G17/QDEC0B	I/O	GPX GPIO17
28	GPX_G18/QDEC1A	I/O	GPX GPIO18
29	GPX_G19/QDEC1B	I/O	GPX GPIO19
30	+3.3V	PWR	Regulated 3.3V supply input/output.
31	GPX_G10_BOOT	I/O	GPX GPIO10
32	GPX_G5/SCLK	I/O	GPX GPIO5 GPX SPI clock
33	GPX_G6/Rx1/MOSI	I/O	GPX GPIO6 GPX Serial 1 input (TTL) GPX SPI MOSI
34	GPX_G7/Tx1/MISO	I/O	GPX GPIO7 GPX Serial 1 output (TTL) GPX SPI MISO
35	GPX_G8/CS	I/O	GPX GPIO8 GPX SPI chip select
36	GPX_G9/SPI_EN	I/O	GPX GPIO9 GPX SPI Enable: Hold LOW during bootup to enable SPI on G5-G8
37-47	GND	PWR	-

5.6.3 Schematic

Download Schematic

5.6.4 Hardware Design

Recommended PCB Footprint and Layout

The default forward direction is indicated in the PCB footprint figure and on the silkscreen as the X axis. The forward direction is reconfigurable in software as necessary.

©2022

Download PDF

5.6.5 Soldering

The IMX-5 can be reflow soldered. Reflow information can be found in the Reflow Information section of this manual.

5.6.6 Design Files

Open source hardware design files, libraries, and example projects for the IMX module are found at the Inertial Sense Hardware Design repository hosted on GitHub. These include schematic and layout files for printed circuit board designs, and 3D step models of the InertialSense products usable for CAD and circuit board designs.

Reference Design Projects

The EVB-2, IG-1, and IG-2 circuit board projects serve as reference designs that illustrate implementation of the IMX PCB module.

EVB-2 evaluation board

IG-1 module

IG-2 module

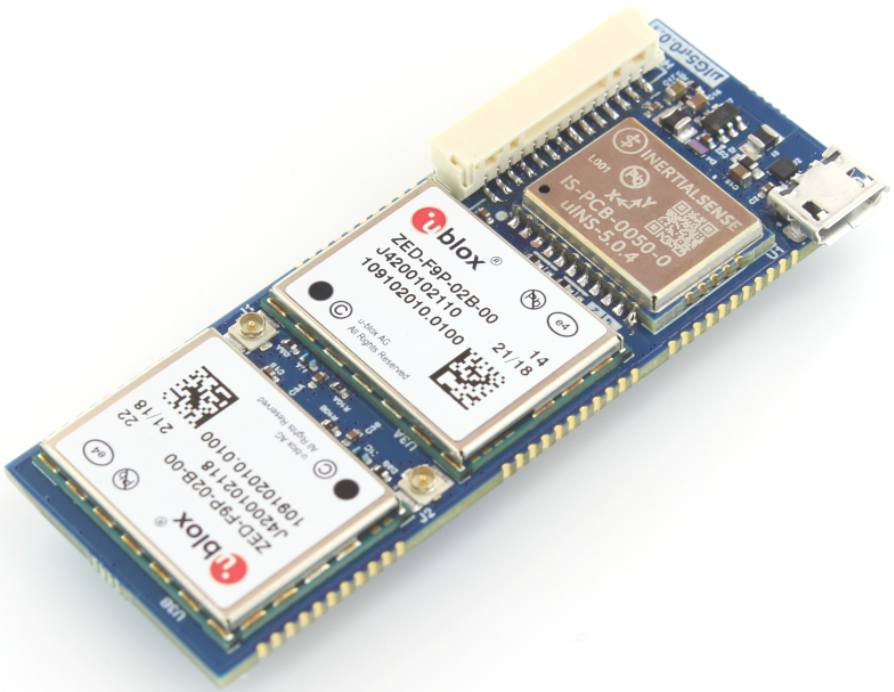
5.6.7 Related Parts

Part	Manufacturer	Manufacturer #	Description
H1	JST	GHR-14V-S	14 pin connector 1.25mm pitch for IMX I/O connection.

©2022

5.7 Hardware Integration: IG-1-IMX-5

> **Warning**
>
> The IMX-5 series of products is approaching its End of Life. Full Notification.



The Inertial Sense IG-1 is a PCB module with IMX-5 and dual ublox ZED-F9P multi-frequency GNSS receivers.

- Surface mount reflowable.
- Onboard dual GNSS for simultaneous RTK positioning and GPS compassing.
- Micro USB and 14 pin I/O header for convenient evaluation.

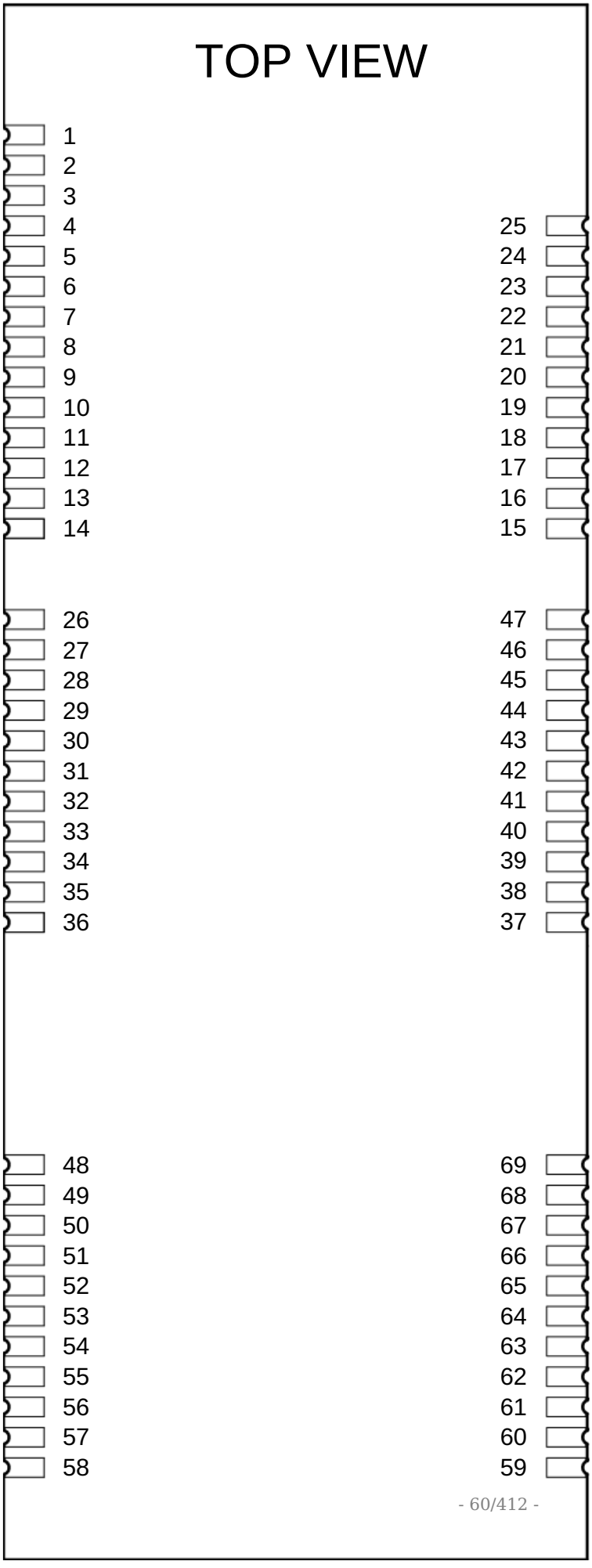
5.7.1 Connecting Your Unit

For the purposes of basic evaluation, the easiest interface available on the IG-1 is by using a micro-USB cable. A cable is included in the evaluation kit. The cable provides power and communications with the installed module via USB virtual communications port.

5.7.2 Pinout

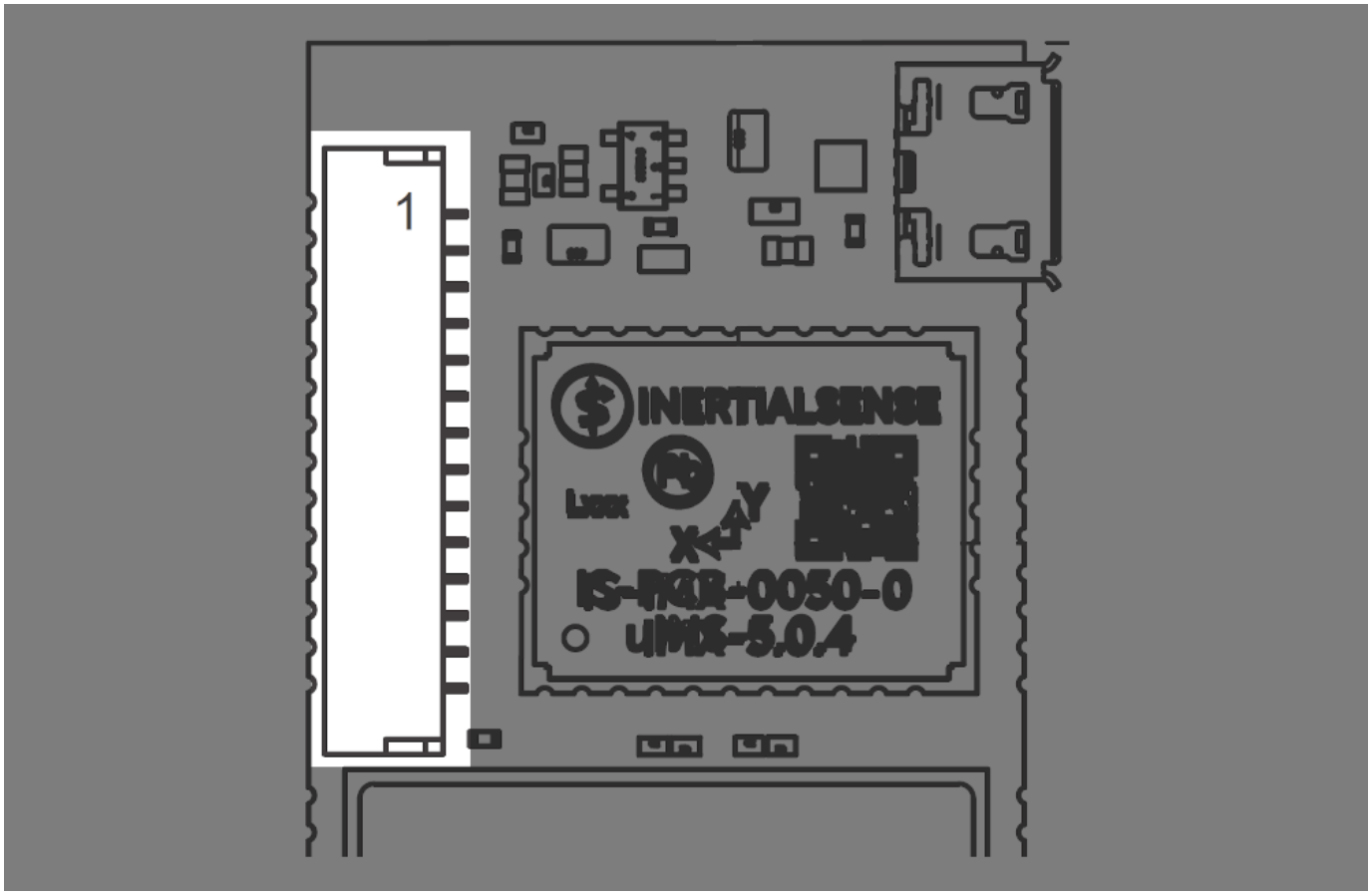
Module Pinout

©2022


TOP VIEW
1
2
3
4
5
6
7
8
9
10
11
12
13
14
25
24
23
22
21
20
19
18
17
16
15
26
27
28
29
30
31
32
33
34
35
36
47
46
45
44
43
42
41
40
39
38
37
48
49
50
51
52
53
54
55
56
57
58
69
68
67
66
65
64
63
62
61
60
59

©2022

Header H1 Pinout



©2022

The module and header H1 have the same pinout assignment for pins 1-14. All pins 15 and above are only on the module.

©2022

Module & H1 Pin	Name	I/O	Description
1	GND	PWR	-
2	VIN	PWR	4V-20V supply voltage input
3	+3.3V	PWR	Regulated 3.3V supply input/output.
4	Reserved		Not Connected
5	G1/Rx2/RxCAN/SCL	I/O	**(Internally connected to GPS2 ZED-F9P TXD)** GPIO1 Serial 2 input (TTL) Serial input pin from CAN transceiver* I2C SCL line
6	G2/Tx2/TxCAN/SDA/ STROBE	I/O	**(Internally connected to GPS2 ZED-F9P RXD)** GPIO2 Serial 2 output (TTL) Serial output pin to CAN transceiver* I2C SDA line Strobe time sync input
7	G3/Tx0	I/O	**(Internally connected to GPS1 ZED-F9P RXD)** GPIO3 Serial 0 output (TTL)
8	G4/Rx0	I/O	**(Internally connected to GPS1 ZED-F9P TXD)** GPIO4 Serial 0 input (TTL)
9	G5/SCLK/STROBE	I/O	GPIO5 SPI SCLK Strobe time sync input
10	G6/Rx1/MOSI	I/O	GPIO6 Serial 1 input (TTL) SPI MOSI
11	G7/Tx1/MISO	I/O	GPIO7 Serial 1 output (TTL) SPI MISO
12	G8/CS/STROBE	I/O	GPIO8 SPI CS Strobe time sync input
13	G9/nSPI_EN/STROBE /STROBE_OUT/ SPI_DRDY	I/O	GPIO9 SPI Enable: Hold LOW during boot to enable SPI on G5-G8 Strobe time sync input or output. SPI data ready alternate location.
14	GPS_TIMEPULSE	O	GPS1 PPS UTC time synchronization signal.
15	GND	I/O	-
16	VBAT	I/O	GPS backup supply voltage. (1.4V to 3.6V) enables GPS hardware backup mode for hot or warm startup (faster GPS lock acquisition). MUST connect GPS_VBAT to VCC if no backup battery is used.
17	G10/BOOT_MODE	I/O	

©2022

Module & H1 Pin	Name	I/O	Description
			Leave unconnected. BOOT MODE used in manufacturing. !!! WARNING !!! Asserting a logic high (+3.3V) will cause the IMX to reboot into ROM bootloader (DFU) mode.
18	G11	I/O	GPIO11
19	G12	I/O	GPIO12 GPS reset
20	G13/DRDY	I/O	GPIO13 SPI data ready
21	G14/SWCLK	I/O	GPIO14
22	nRESET	I	System reset on logic low. May be left unconnected if not used.
23	GND	PWR	-
24	USB_N	I/O	USB Data Negative Line
25	USB_P	I/O	USB Data Positive Line
26	GPS1_RX2	I	Ublox ZED-F9P RXD2 (GPS1)
27	GPS1_TX2	O	Ublox ZED-F9P TXD2 (GPS1)
28	GPS2_RX2	I	Ublox ZED-F9P RXD2 (GPS2)
29	GPS2_TX2	O	Ublox ZED-F9P TXD2 (GPS2)
30	+3.3V	PWR	Regulated 3.3V supply input/output.
31	GPS2_TIMEPULSE	O	GPS2 PPS UTC time synchronization signal.
32-36	NC	-	Not connected internally
37-69	GND	PWR	-

5.7.3 Hardware Versions

The following outlines differences in the IG-1.x hardware versions.

IG-1.2

- GPS1 ZED-F9P RXD2/TXD2 lines connected to IG-1 pins 26, 27.
- GPS2 ZED-F9P RXD2/TXD2 lines connected to IG-1 pins 28, 29.
- GPS1 ZED-F9P PPS (TIMEPULSE) line connected to IG-1 pin 31.
- IG-1 pins 32-36 are not connected internally (not connected to ground).

IG-1.1

- GPS1 PPS line connected to IMX TIMEPULSE G15 (pin 20).
- IG-1 pins 26-36 are connected to ground.

IG-1.0

- GPS1 PPS line connected to IMX G8 (pin 8).
- IG-1 pins 26-36 are connected to ground.

5.7.4 Schematic

Download Schematic

©2022

5.7.5 Hardware Design

Recommended PCB Footprint and Layout

The default forward direction is indicated in the PCB footprint figure and on the silkscreen as the X axis. The forward direction is reconfigurable in software as necessary.

Download PDF

5.7.6 Soldering

The IMX-5 can be reflow soldered. Reflow information can be found in the Reflow Information section of this manual.

5.7.7 Design Files

Open source hardware design files, libraries, and example projects for the IMX module are found at the Inertial Sense Hardware Design repository hosted on GitHub. These include schematic and layout files for printed circuit board designs, and 3D step models of the InertialSense products usable for CAD and circuit board designs.

Reference Design Projects

The IG-1 circuit board projects serve as reference designs that illustrate implementation of the IMX PCB module.

IG-1 module

5.7.8 Related Parts

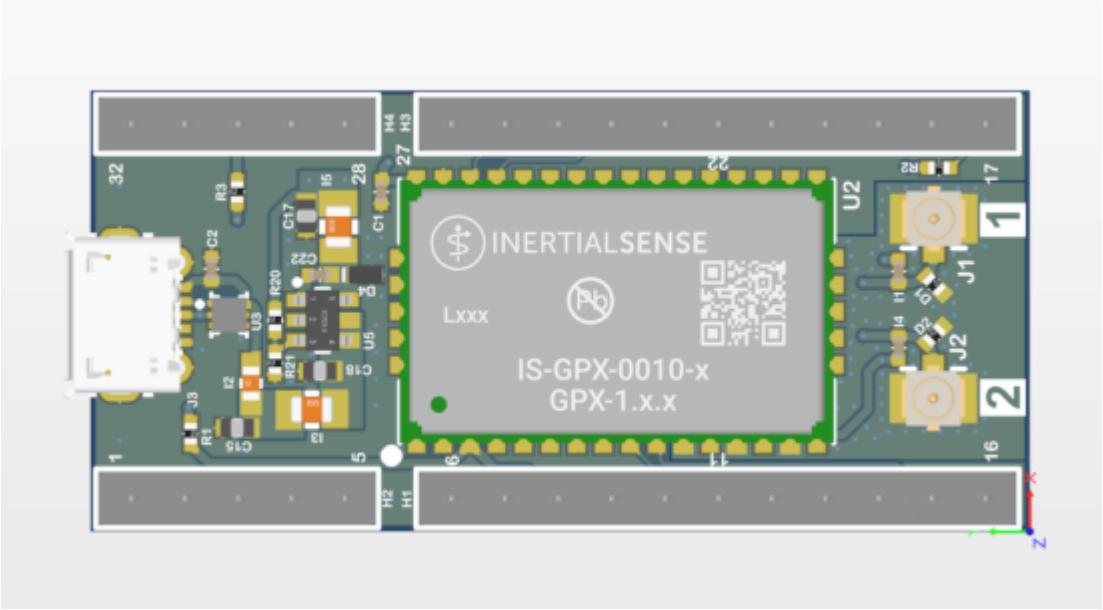
Part	Manufacturer	Manufacturer #	Description
H1	JST	GHR-14V-S	14 pin connector 1.25mm pitch for IMX I/O connection.

©2022

5.8 Hardware Integration: IK-1 (IMX5 or GPX1)

Notice

The IMX-5 series of products is approaching its End of Life. Full Notification.



The Inertial Sense IK-1 is a breakout evaluation board for either the IMX-5 or GPX-1 multi-frequency GNSS receiver.

- 0.1" pitch header for convenient interface.
- Interfaces with standard breadboard.
- Onboard Micro USB connector
- Onboard voltage regulation.
- Dual U.FL connectors for GPX GNSS antennas.

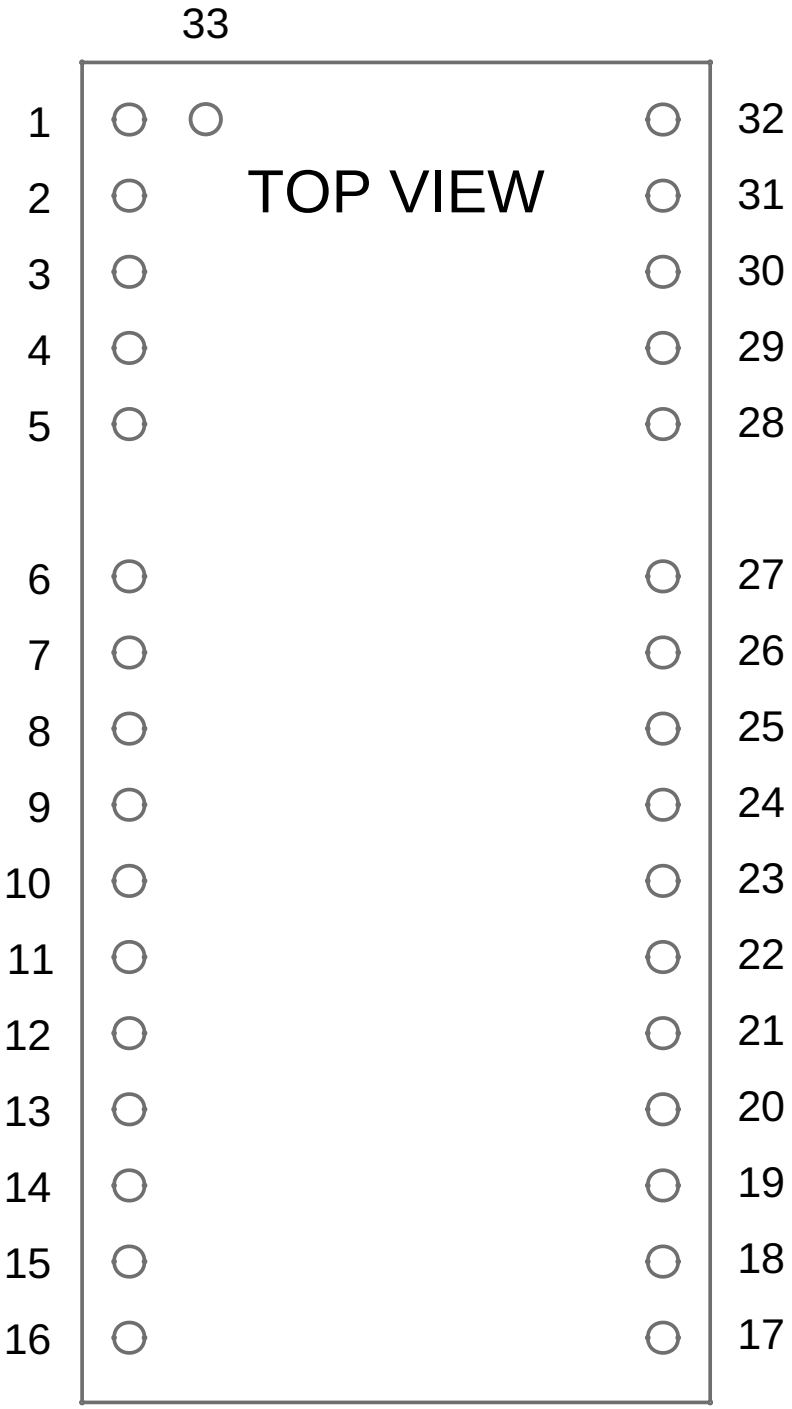
5.8.1 Connecting Your Unit

For the purposes of basic evaluation, the easiest interface available on the IK-1 is by using a micro-USB cable. A cable is included in the evaluation kit. The cable provides power and communications with the installed module via USB virtual communications port.

5.8.2 Pinout

Module Pinout

©2022


33
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
TOP VIEW
32
31
30
29
28
27
26
25
24
23
22
21
20
19
18
17

©2022

The IK-1 module pinout is as follows

©2022

IK1	IMX	GPX	Name	Type	Description
1-3,16,26	11,21	11,13,15,31	GND	Power	Supply ground
4	-	20	G20/LNA-EN	I/O	GPIO20, GPX LNA enable
5	-	21	GNSS2_PPS	O	GNSS2 PPS time synchronization output pulse (1Hz, 10% duty cycle)
6	1	1	USB_P	I/O	USB full-speed Positive Line. USB will be supported in future firmware updates.
7	2	2	USB_N	I/O	USB full-speed Negative Line. USB will be supported in future firmware updates.
8	3	3	VBKUP	Power	Backup supply voltage input (1.75V to 3.6V). Future firmware updates will use voltage applied on this pin to backup GNSS ephemeris, almanac, and other operating parameters for a faster startup when VCC is applied again. This pin MUST be connected to a backup battery or VCC.
9	4	4	G1/Rx2/ RxCAN/SCL	I/O	GPIO1 Serial 2 input (TTL) Serial input pin from CAN transceiver* I2C SCL line
10	5	5	G2/Tx2/ TxCAN/SDA/ STRB	I/O	GPIO2 Serial 2 output (TTL) Serial output pin to CAN transceiver* I2C SDA line Strobe time sync input
11	6	6	G6/Rx1/ MOSI	I/O	GPIO6 Serial 1 input (TTL) SPI MOSI
12	7	7	G7/Tx1/ MISO	I/O	GPIO7 Serial 1 output (TTL) SPI MISO
13	8	8	G8/CS/STRB	I/O	GPIO8 SPI CS Strobe time sync input
14	9	9	G5/SCLK/ STRB	I/O	GPIO5 SPI SCLK Strobe time sync input
15	10	10	G9/nSPI_EN/ STRB/DRDY	I/O	GPIO9 SPI Enable: Hold LOW during boot to enable SPI on G5-G8 Strobe time sync input or output. SPI data ready alternate location
17	12	22	nRESET	I	System reset on logic low. May be left unconnected if not used.
18	13	23	G14/SWCLK	I/O	GPIO14
19	14	24	G13/XSDA/ PPS2/DRDY	I/O	GPIO13 Alt I2C SDA GNSS2 PPS time synchronization input

©2022

IK1	IMX	GPX	Name	Type	Description
					pulse (1Hz, 10% duty cycle) SPI Data Ready
20	15	25	G12/XSCL	I/O	GPIO12 Alt I2C SCL
21	16	26	G11/SWDIO	I/O	GPIO11
22	17	27	G10/BOOT	I/O	Leave unconnected. BOOT mode used in manufacturing. !!! WARNING !!! Asserting a logic high (+3.3V) will cause the IMX to reboot into ROM bootloader (DFU) mode.
23	18	28	G4/Rx0	I/O	GPIO4 Serial 0 input (TTL)
24	19	29	G3/Tx0	I/O	GPIO3 Serial 0 output (TTL)
25	20	30	GNSS1_PPS	O (GPX) I (IMX)	GNSS1 PPS time synchronization pulse (1Hz, 10% duty cycle)
27	22	32	VCC	Power	1.8V to 3.3V supply input.
28	-	38	G16/ QDEC0.A	I/O	GPIO16
29	-	39	G17/ QDEC0.B	I/O	GPIO17
30	-	40	VAUX	Power	Input supplies for the USB and VCC_RF (GNSS antenna supply). Connect to +3.3V (3.0V to 3.6V) to supply USB and VCC_RF. Can be left floating if USB or VCC_RF are not needed.
31	-	41	G18/ QDEC1.A	I/O	GPIO18
32	-	42	G19/ QDEC1.B	I/O	GPIO19
33	-	-	VIN	Power	4V-20V supply voltage input
U.FL1	-	12	GNSS1_RF	I	GNSS1 antenna RF input. Use an active antenna or LNA with a gain of 15-25dB. Place the LNA as close to the antenna as possible. Filtered 3.3V from VCC is injected onto the pad to power active antennas (power injection can be disabled in software). Connect to ground with 5V-14V TVS diode for ESD and surge protection (e.g. Littlefuse PESD0402-140).
U.FL2	-	14	GNSS2_RF	I	GNSS2 antenna RF input. Same requirements as GNSS1_RF

5.8.3 Schematic

Download Schematic

©2022

5.8.4 Design Files

Open source hardware design files, libraries, and example projects for the IMX module are found at the Inertial Sense Hardware Design repository hosted on GitHub. These include schematic and layout files for printed circuit board designs, and 3D step models of the InertialSense products usable for CAD and circuit board designs.

Reference Design Projects

The EVB-2, IG-1, IG-2, and IK-1 circuit board projects serve as reference designs that illustrate implementation of the IMX PCB module.

EVB-2 evaluation board

IG-1 module

IG-2 module

IK-1 module

5.8.5 Related Parts

Part	Manufacturer	Manufacturer #	Description

©2022

5.9 Hardware Design Files

The Inertial Sense hardware design files are available on our IS-hdw repository to facilitate product hardware development and integration.

- **PCB Libraries** - Schematic and layout files for printed circuit board designs.
- **Products** - 3D models and resources for the IMX, Rugged, EVB, and products useful for CAD and circuit board designs.

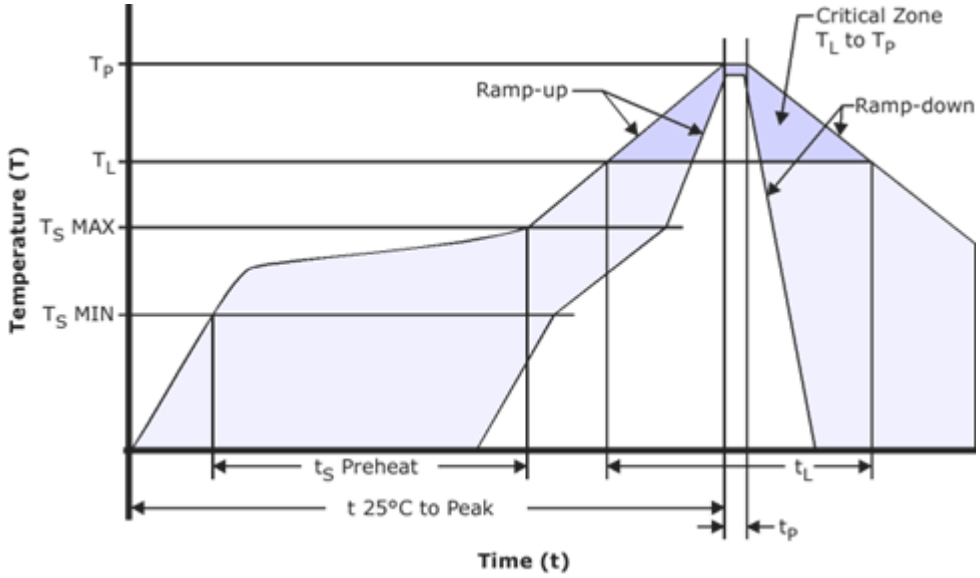
©2022

5.10 Reflow Soldering

Use of "No Clean" soldering paste is recommended as it does not require cleaning after the soldering process. The following examples of paste meet these criteria.

Solder	Details
Soldering Paste	OM338 SAC405 / Nr.143714 (Cookson Electronics)
Allow Specification	Sn 95.5/ Ag 4/ Cu 0.5 (95.5% Tin/ 4% Silver/ 0.5% Copper)
Melting temperatures	217 °C

5.10.1 The following reflow profile is recommended for soldering:



Phase	Name	Recommended	Details
Preheat			
	dT/dt	3°C/sec	Preheat Temperature Rise Rate
	T_SMIN	150°C	Preheat Minimum Temperature
	T_SMAX	200°C	Preheat Maximum Temperature
	t_SPreheat	60 - 120 sec	Time Spent Between Preheat MIN and Max temperatures
Reflow			
	T_L	217°C	Reflow Liquidus temperatures
	T_P	245°C	Reflow Peak temperatures
	t_L	40-60 sec	Time Spent above Reflow Liquidus temperatures
Cooling			
	dT/dt	4°C/sec	Maximum Cooling Temperature Fall Rate

©2022

Important

A convection soldering oven is highly recommended over an infrared type radiation oven as it allows precision control of the temperature and all parts will be heated evenly.

Warning

The IMX should be located on the topside of a PCB during reflow to avoid falling off.

Care should be taken to not disturb the components on the IMX during reflow as the solder on the IMX will also reflow.

The part must not be soldered with a damp heat process.

©2022

6. Software / SDK

6.1 EvalTool

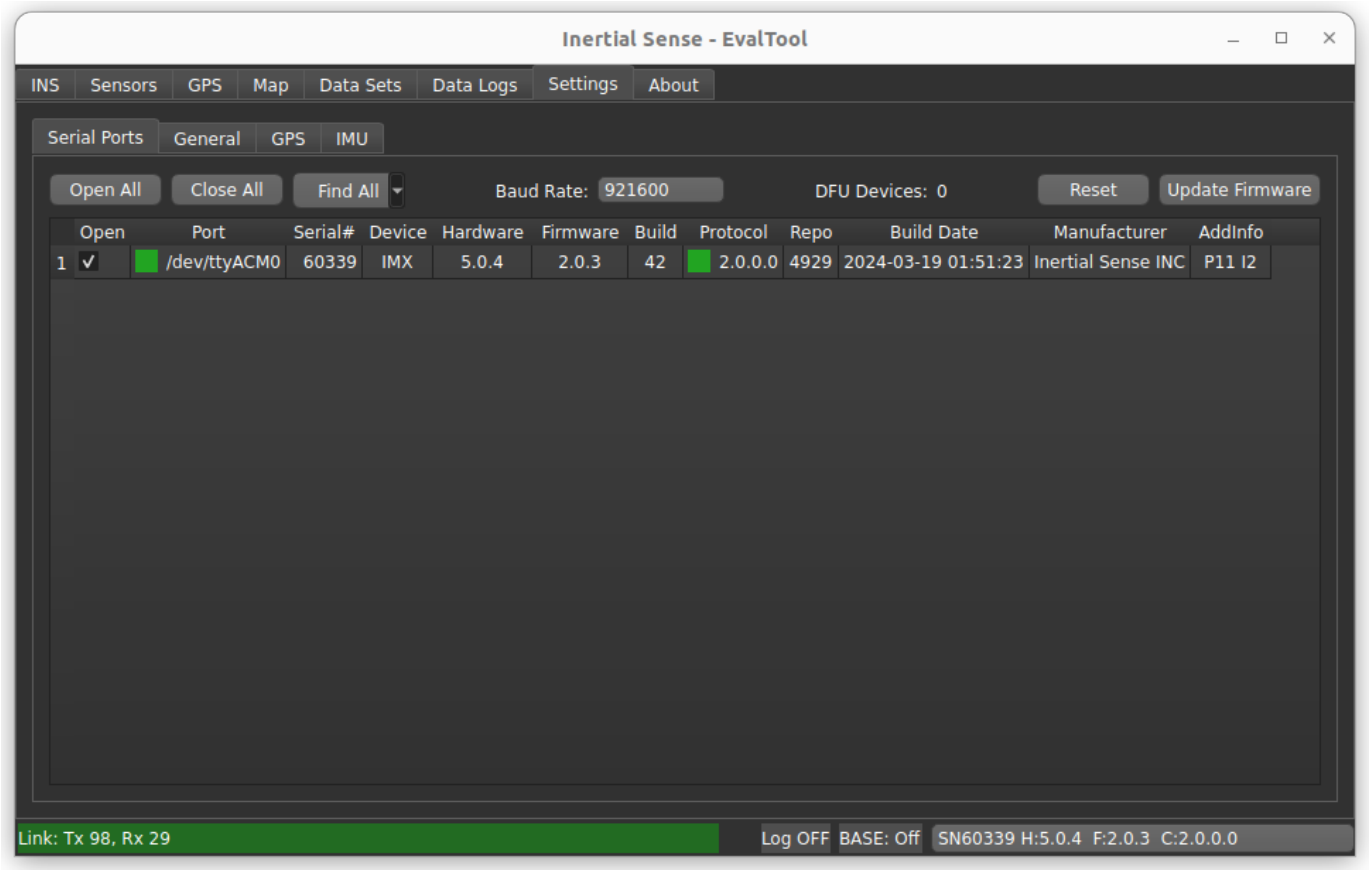
6.1.1 Overview

The EvalTool (Evaluation Tool) is a desktop GUI application that allows you to explore and test functionality of the Inertial Sense products in real-time. It has scrolling plots, 3D model representation, table views of all data, data logger, and firmware updating interface for the IMX. The EvalTool can simultaneously interface with multiple Inertial Sense devices.

6.1.2 Download and Install

The EvalTool Windows desktop app installer (.exe) can be downloaded from the Inertial Sense releases page.

6.1.3 Getting Started



With a device connected to your computer:

1. Connect your INS to your computer using directions in the IS Hardware section of this guide corresponding to the correct model.
2. Open the **Settings** > **Serial Ports** tab.
3. Click the **Find All** button, or open the port to your device by checking the **Open** checkbox.
4. The status box in the **Port** column will turn green and the **Link** status bar will turn green while data is being received from the device.
5. You can specify the serial port baud rate using the **Baud Rate** dropdown menu when using a serial interface like RS232.

©2022

DATA LOGGING STEPS

In order to log data from your INS device, follow the steps listed below:

1. Connect your device to the EvalTool and open the port.
2. Go to the **Data Logs** tab.
3. Enable the data that you would like collected in the **Data Streams** area:

a. Select one of the **RMC Preset** menu options. This automatically enables standard messages commonly used for logging. **PPD** (Post Processed Data) is the recommended default.

b. Enable any of the **DID** messages listed below by checking the **On** checkbox and setting the **Period Mult**.

4. Configure and enable logging in the **Data Log** area:

a. The **Open Folder** button opens the log directory in a file explorer window where logs are saved.

b. The **Format** dropdown menu selects the output log file format (.raw .dat .csv .kml).

c. Press **Log** to start recording a new log.

5. The data you are currently recording will be shown in the “Log Summary” sub-tab.
6. When you are finished recording data, press “Disable”. Your data will be saved in the location shown in “Open Folder”.

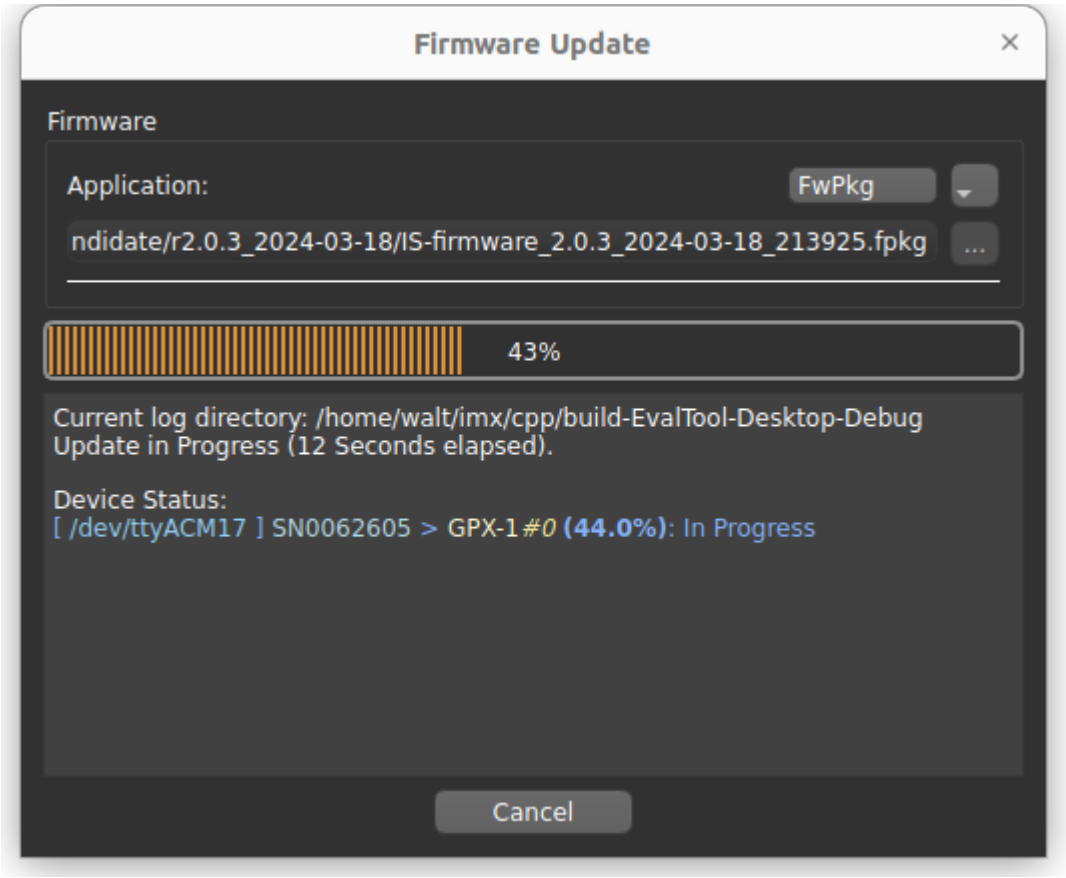
6.1.4 Info Bar

The Info Bar can be seen from any tab and shows basic connection information for the unit selected.

1. Link Status - Shows Packets being Transmitted and Received. counts to 99 then resets to 0.
2. Error Message - Shows error messages for the selected unit. The kinds of messages vary from data packets lost to system had a reset.
3. RTK Base Messages - The number in this field will increment as your rover unit continues to receive RTK messages from your base station. Use this field as the main signifier that RTK messages are coming through.
4. Currently selected unit - The unit with the serial number shown here will have its live data shown on each tab in EvalTool.

©2022

6.1.5 Update Firmware



1. Go to the **Settings > Serial Port** tab.
2. Open the Ports of the units you would like to update. If the units don't open up, you may have to change the baud rate.
3. Click **Update Firmware**.
4. Select the update type using the drop down menu:

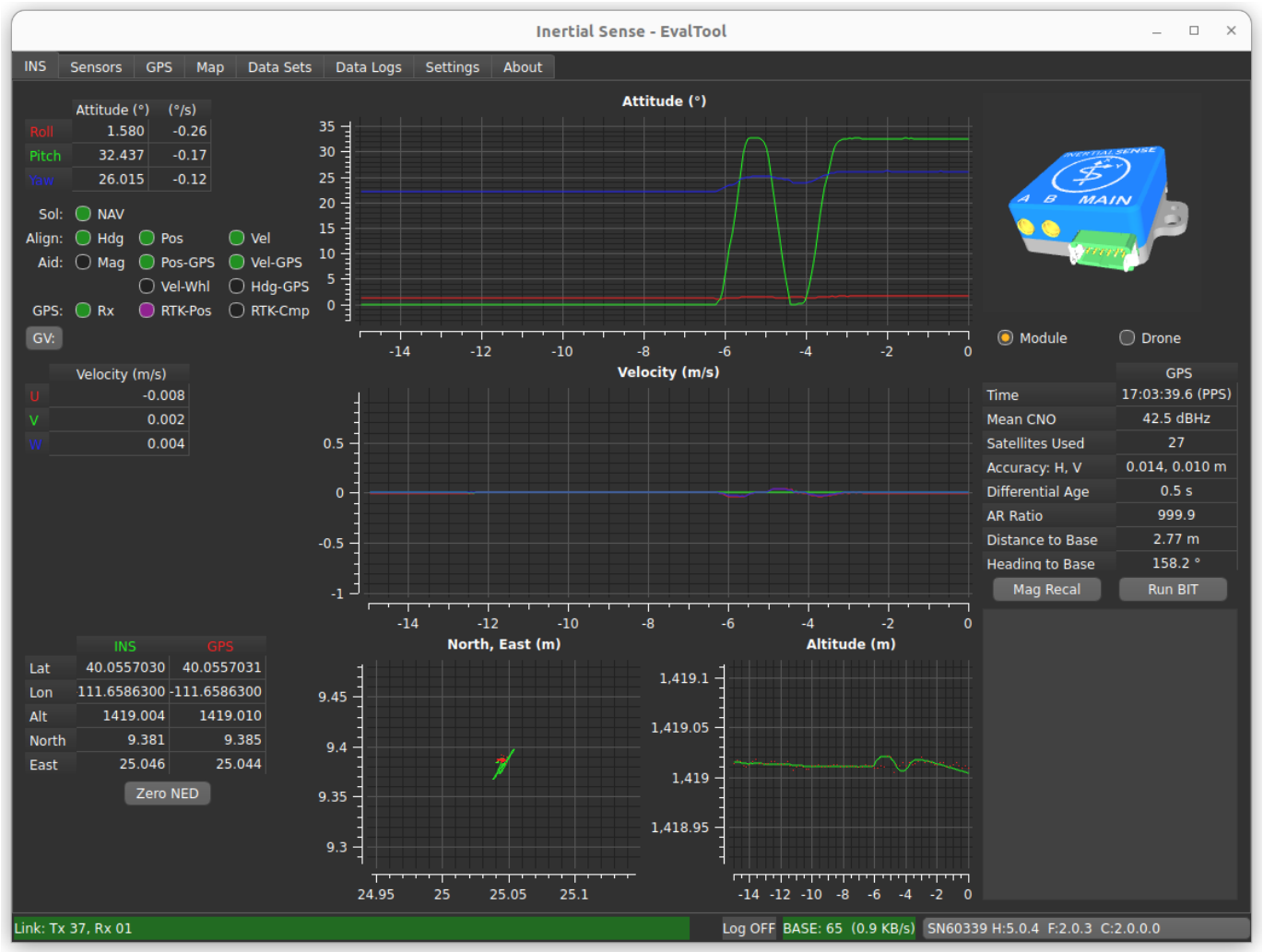
Update Type	**Description**
FwPkg (.fpkg)	Batch firmware update method for updating multiple devices in one process. The .fpkg file contains multiple firmware files and instructions for sequencing firmware updates for all available devices. **NOTE:** IMX-5 firmware update is not yet supported but will be in a future update.
GPX-1 (.bin)	GPX-1 firmware update.
IMX-5 (.hex)	(Legacy mode) IMX-5 firmware update that used the legacy InertialSense bootloader (ISB).
1. Select the firmware file by clicking on the ellipsis (three dots) button next to the file name and navigating to and opening the file.	
1. For the IMX-5 update type, you can optionally select the bootloader .bin file. The bootloader will only be updated if the selected file is newer than the bootloader on the connected unit.	
1. Click **Start**.	
1. Wait for the progress to reach 100% and click **Done**.	

©2022

*Note: The firmware can only be updated at the following baud rates: 3000000, 921600, 460800, 230400, 115200.

6.1.6 Tab Descriptions

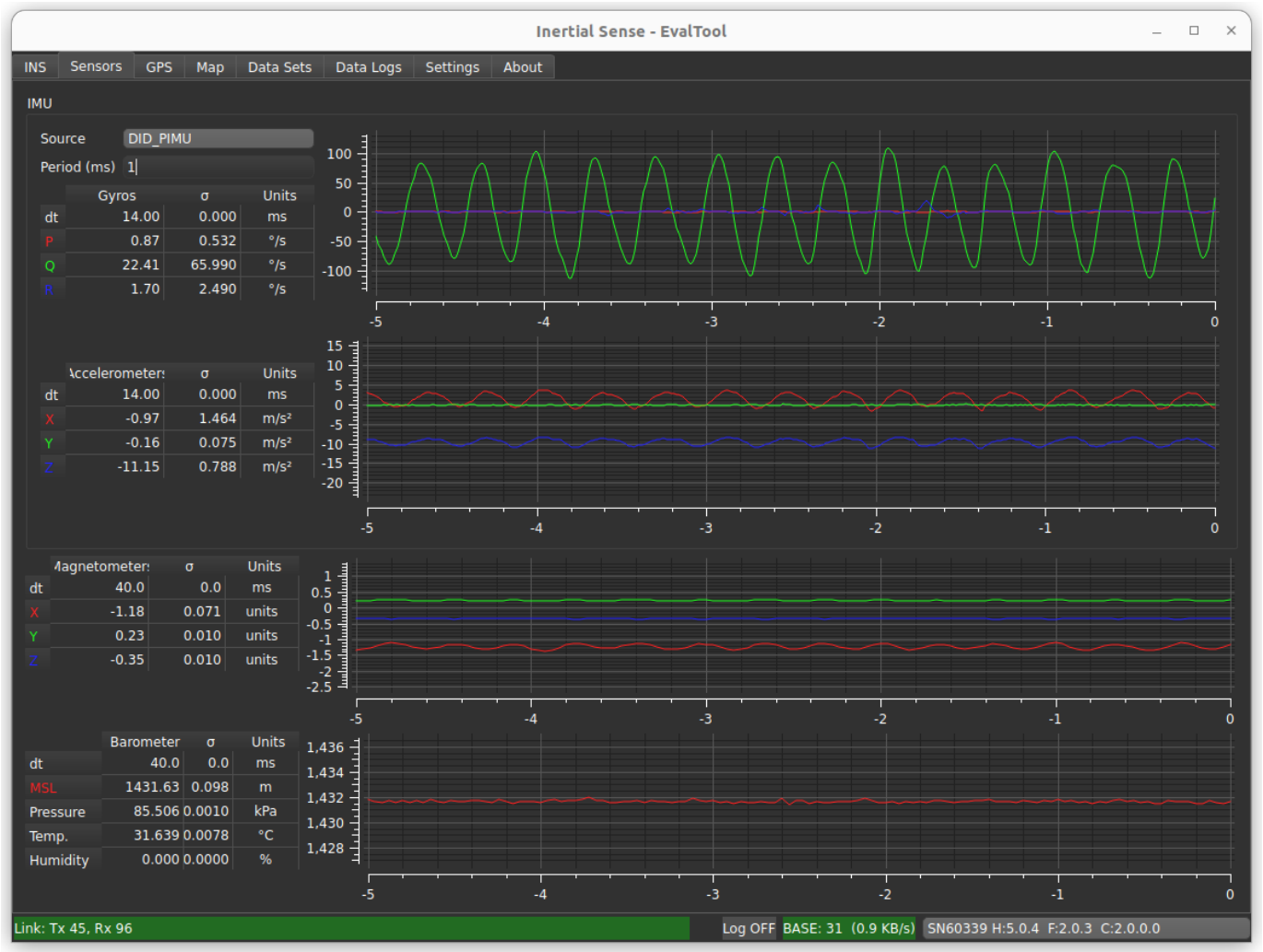
INS Tab



1. Attitude Plot and Table - shows the Roll, Pitch, and Yaw values of the selected unit. Hover the cursor over the radio buttons to see more descriptions.
2. Velocity Plot and Table - U,V,W velocities.
3. LLA Plot and Table - Tabular values and plot of Latitude, Longitude, and Altitude.
4. Simulation - Real-time, simulated image of the INS orientation.
5. GPS Summary - Strength of GPS signal and accuracy.
6. Mag Recal Button - Allows you to calibrate your units about either a single axis (for heavy, ground based vehicles) or multi-axes.
7. BIT (Built-In Test) Button - Runs a system of checks on your unit.
8. Link Messages - shows the performance information on connected units and displays error messages.

©2022

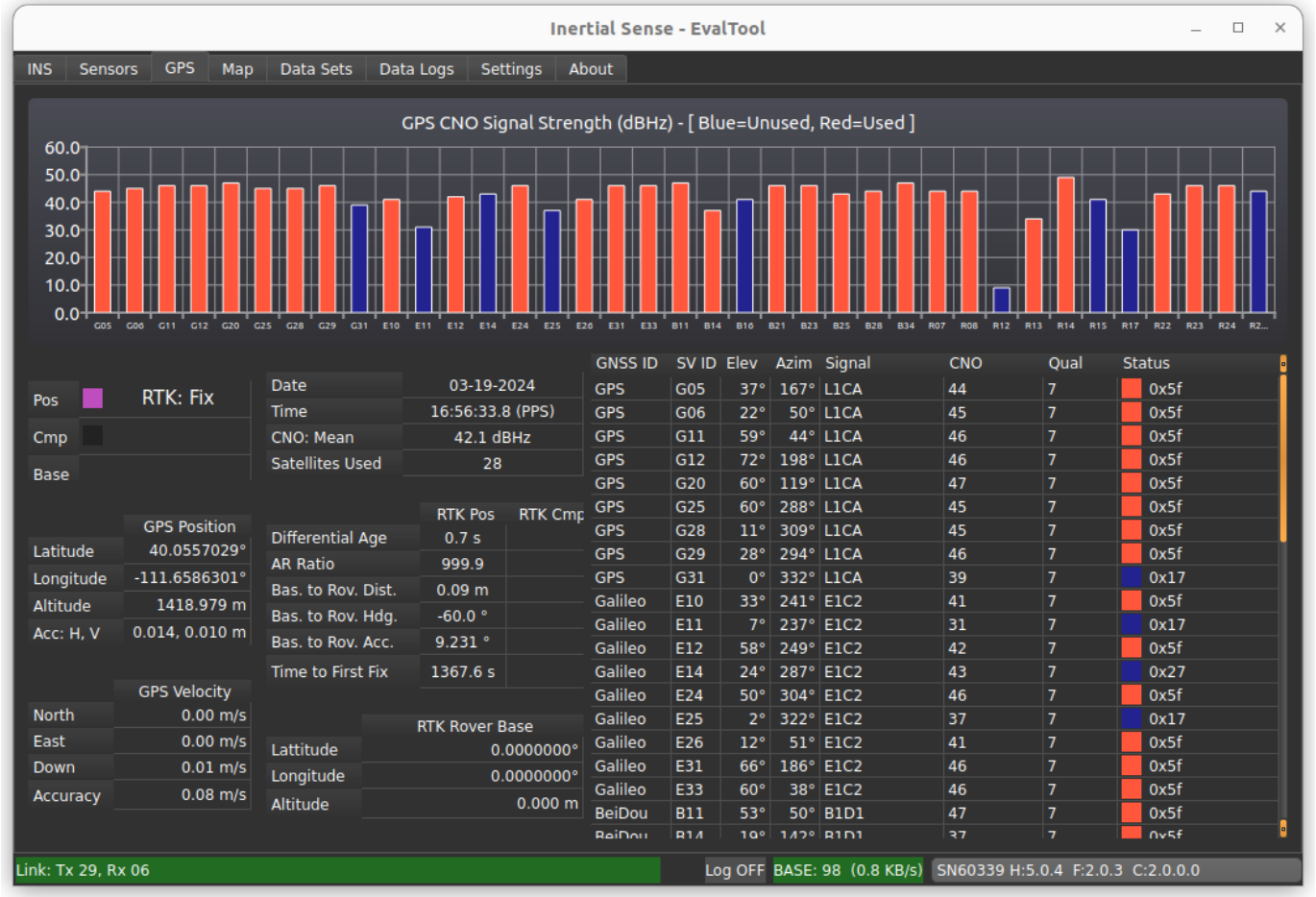
Sensors Tab



1. Gyros Plot and Table - Gyroscopic data on the selected unit. Includes standard deviation.
2. Accelerometers Plot and Table - Accelerometer data on the selected unit. Includes standard deviation.
3. Magnetometers Plot and Table - Magnetometer data on the selected unit. Includes standard deviation.
4. Barometer Plot and Table - Barometric, temperature, and humidity data on the selected unit. Includes standard deviation.

©2022

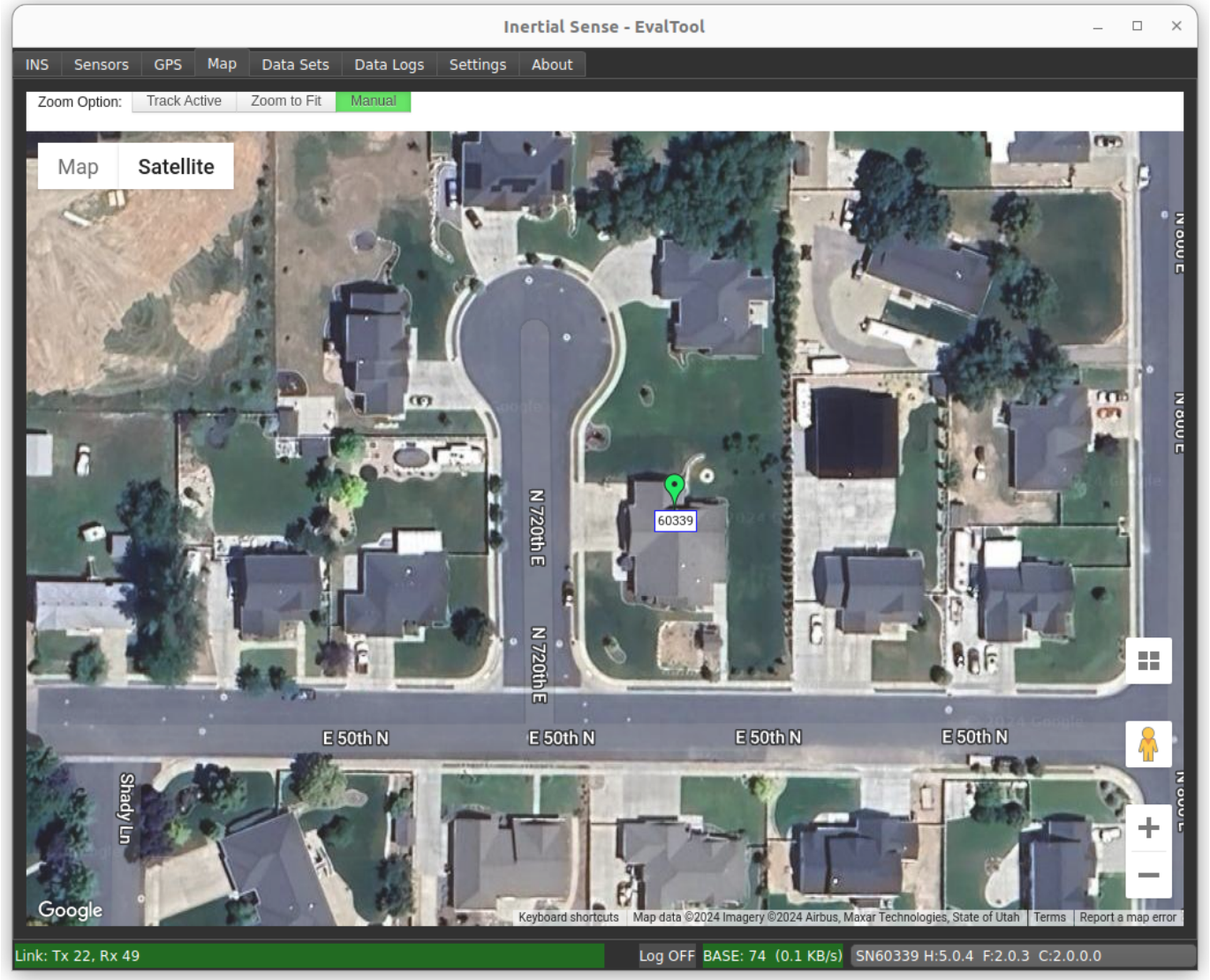
GPS Tab



1. GPS CNO Signal Strength - Bar graphs of each satellite being used in your solution and its strength in dBHz(CNO).
2. Position Accuracy Plot and Table - RTK mode and status. Includes number of satellites used in the RTK solution (max and mean).
3. Satellites Used Table - The GNSS ID for each satellite seen by your unit and the subsequent connection details.

©2022

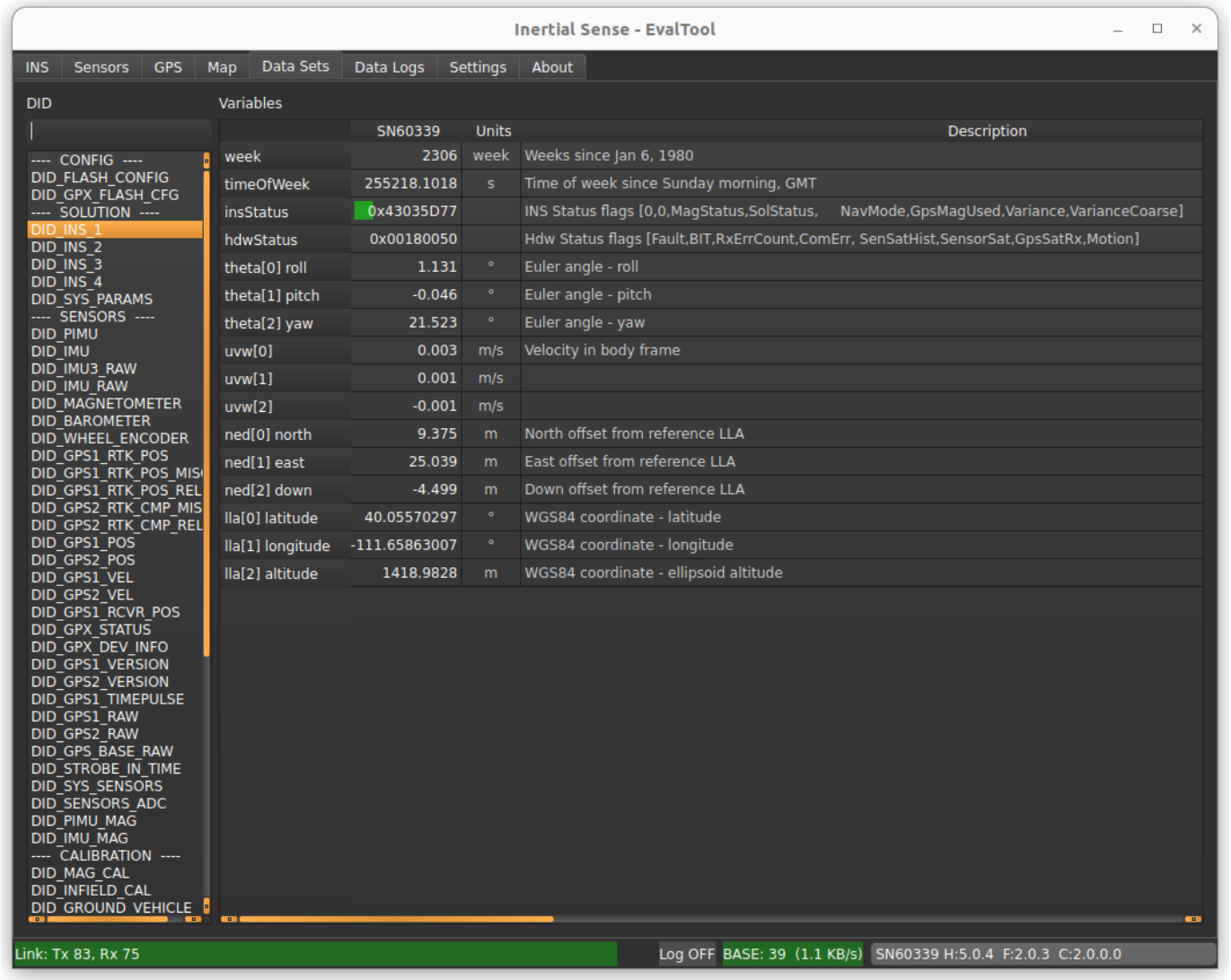
Map Tab



1. Track Active - Tracks all units on window view.
2. Zoom to Fit - Zooms your window view around each unit being used.
3. Manual - Requires manual movement of the window view.
4. Location of Units - GPS location of each of your units. Shows RTK, GPS ublox, and INS solution.

©2022

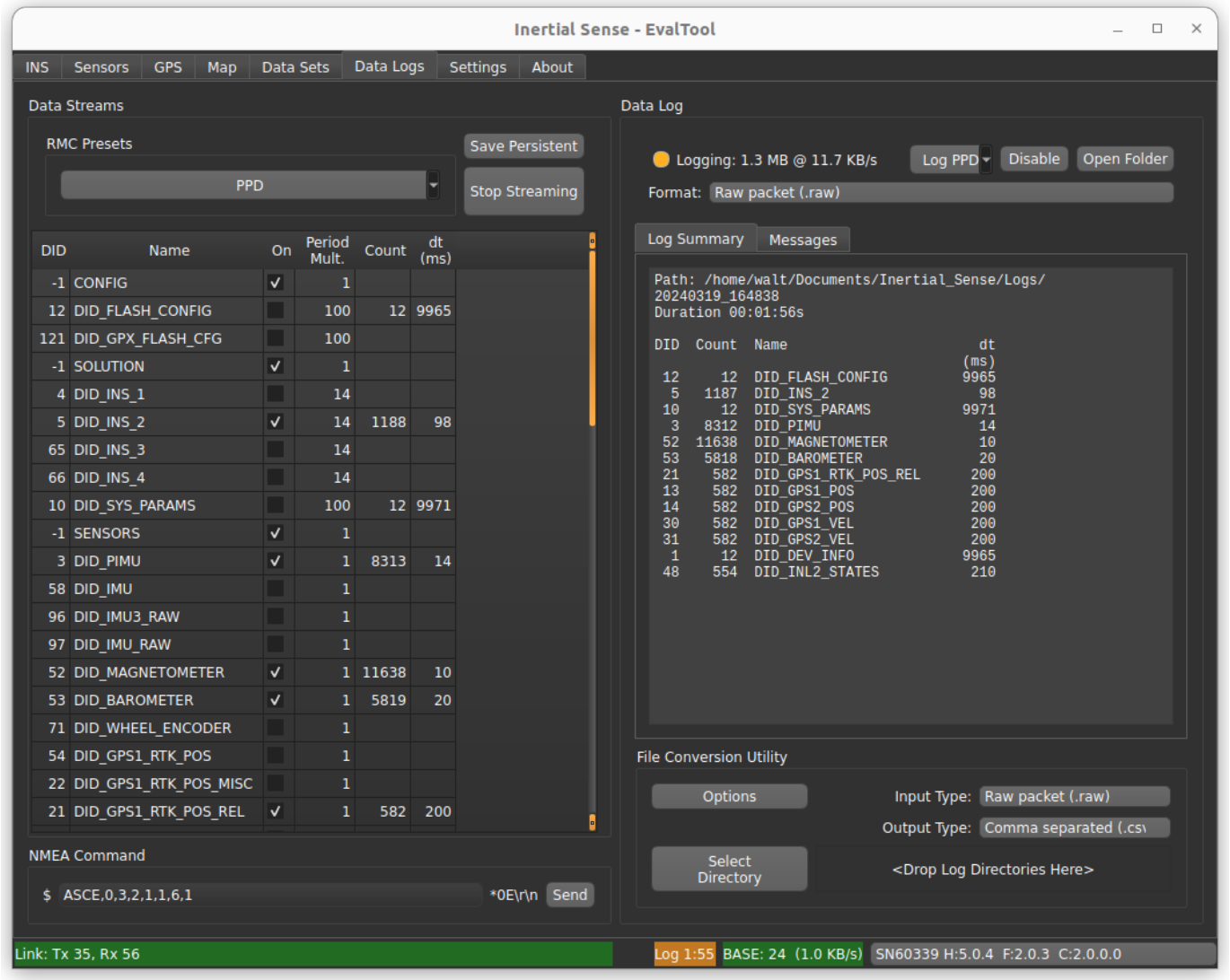
Data Sets Tab



1. List of DIDs (Data IDentifiers) - The data identifiers that you might need to view for measurements. See the User Manual (Binary Protocol Data Sets) for a detailed description of frequently used DIDs.
2. List of Variables within DIDs - shows what is recorded in each DID in real-time.

©2022

Data Logs



DATA STREAMS

This area allow users to enable streaming of various DIDs.

1. RMC Presets Button - Enable a group of data sets. PPD (post process data) is the preferred preset for post processing and debug analysis.
2. Save Persistent Button - Save currently enabled data streams to automatically begin streaming after system restart. To clear persistent streams, first stop streaming and then click Save Persistent.
3. Stop Streaming - Stops all data streams. Any streams previously saved as persistent will begin streaming at startup.

DATA LOG

1. Enable/Disable Button - Starts/stops a log of all currently streaming data and saves it to a sub-folder with the current time-stamp within your "Logs" folder.
2. Open Folder Button - Opens the "Logs" folder where your previous logs are saved.
3. Format Dropdown - Select the file output type of the data log , such as .raw, .dat, .csv, or .kml.

©2022

Log Format	Description
Raw packet (.raw)	Binary file containing byte for byte data received over the serial ports. All packets remain in their native form. Used for logging InertialSense binary (ISB), NMEA, RTCM3, uBlox UBX binary and SPARTN, and any other packet formats. ***Recommended for logging all data formats and post processing***.
Serial binary (.dat)	Binary file containing InertialSense binary (ISB) DID data sets in "chunk" groups containing data in serial order as they appear over the serial port. Default file format. ***Recommended for post processing.***
Comma separated (.csv)	Plain text file that uses specific structuring to arrange tabular data. Its basic format involves separating each data field (or cell in a table) with a comma and each record (or row) is on a new line. This simple format allows for ease in data import and export between programs that handle tabular data, such as databases and spreadsheets.

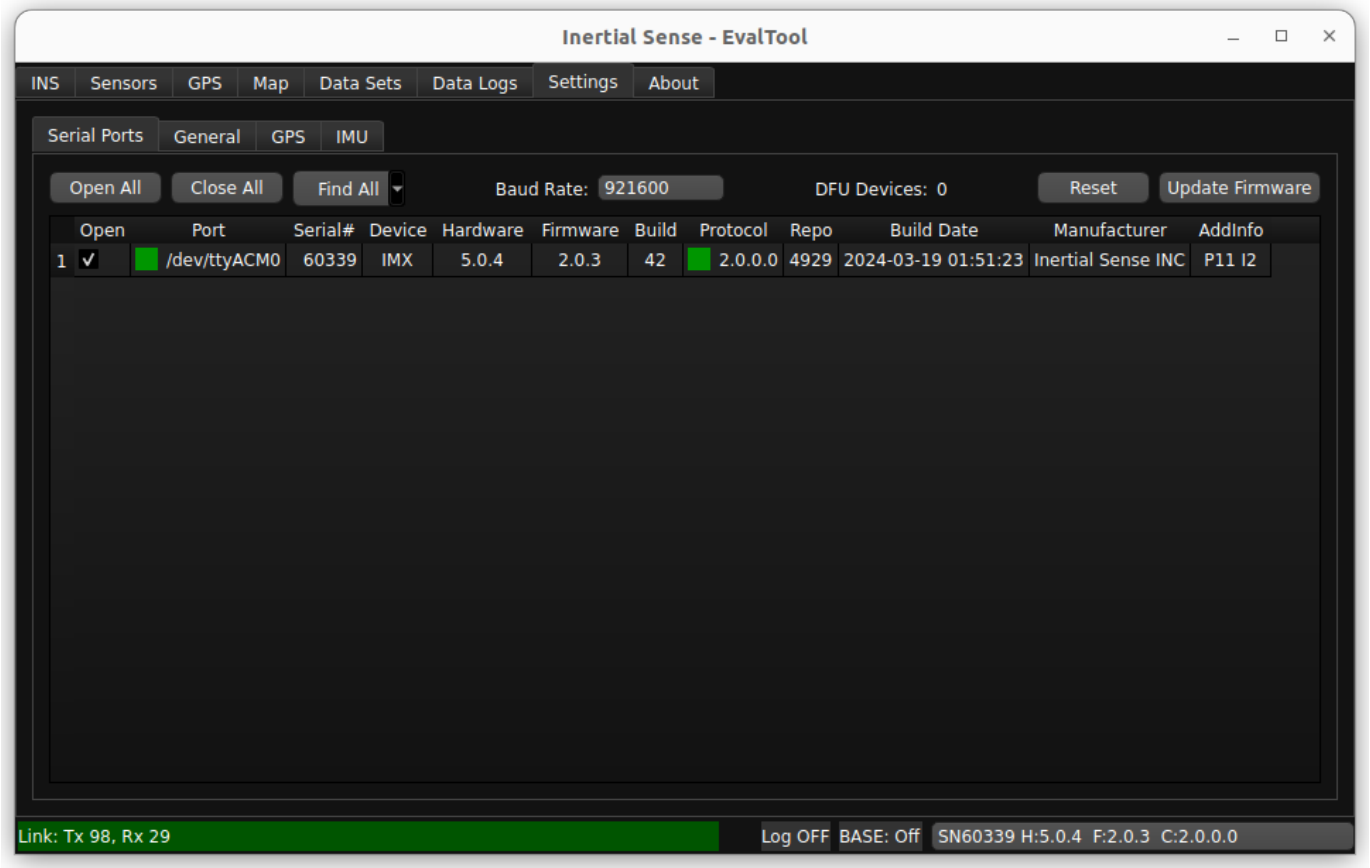
4. Summary Window - Shows the log directory path, the elapsed time the data log has been running, the total size of the log file, and a list of currently recording DIDs with corresponding dt (time between measurements).
5. File Conversion Utility - Enables you to convert the data log file type in a specified directory. (e.g. .dat to .csv)

Settings Tab

The Settings tab has 3 sub tabs and they are as follows:

Settings - Serial Ports Tab

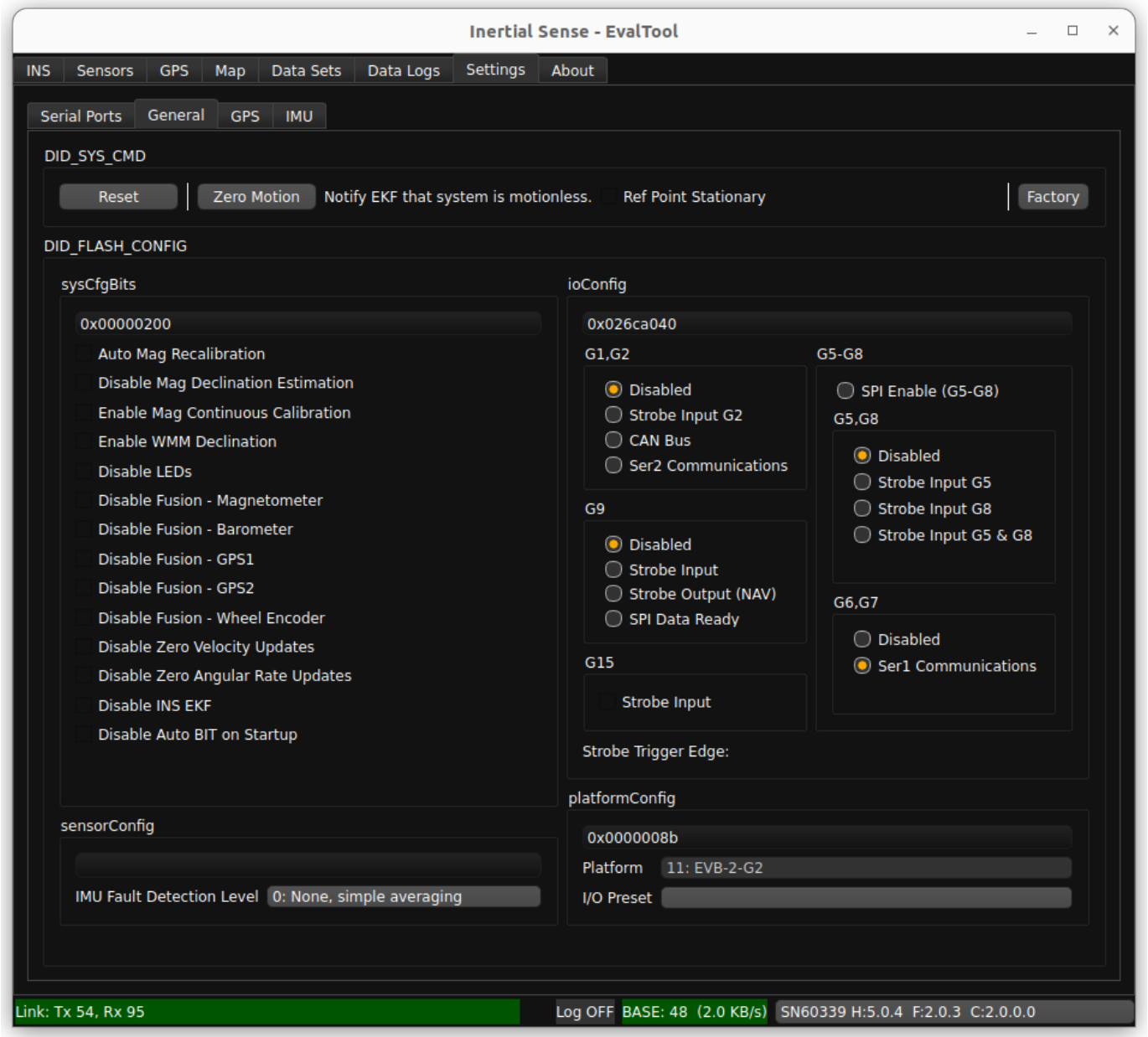
©2022



1. Open All - Opens all of the ports shown.
2. Close All - Closes all of the ports shown.
3. Find Devices - Determines which peripherals connected to your computer are Inertial Sense units, and opens those ports while closing the others.
4. Baud Rate - The rate at which data will be communicated over your data channel.
5. Update Firmware - Allows you to update your unit's firmware when an update is released from Inertial Sense.
6. Port Status - Shows a list of all connected comports and basic information for each of them. Clicking the check box opens the port.

Settings - General Tab

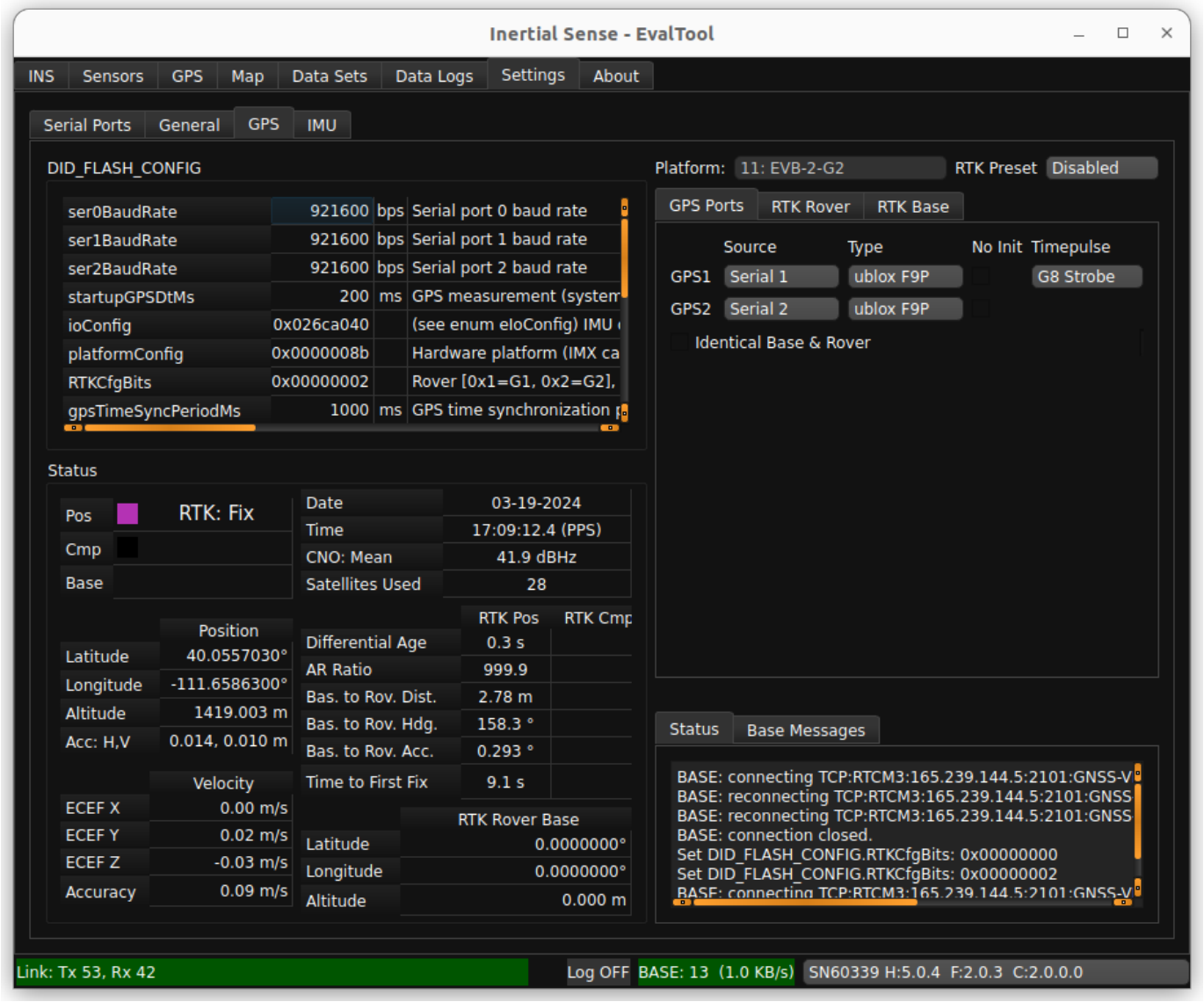
©2022



1. Software Reset - Allows the user to issue a reset to the unit. Has options for all open comports and only the currently connected unit.
2. Zero Motion - Allows the user to inform the EKF that the system is stationary on the ground and is used to aid in IMU bias estimation which can reduce drift in the INS attitude.
3. DID_Flash_Config - Gives the user option to disable or enable different features normally found in the "Data Sets" tab. For more information about the Flash Config see Data sets.

Settings - GPS Tab

©2022

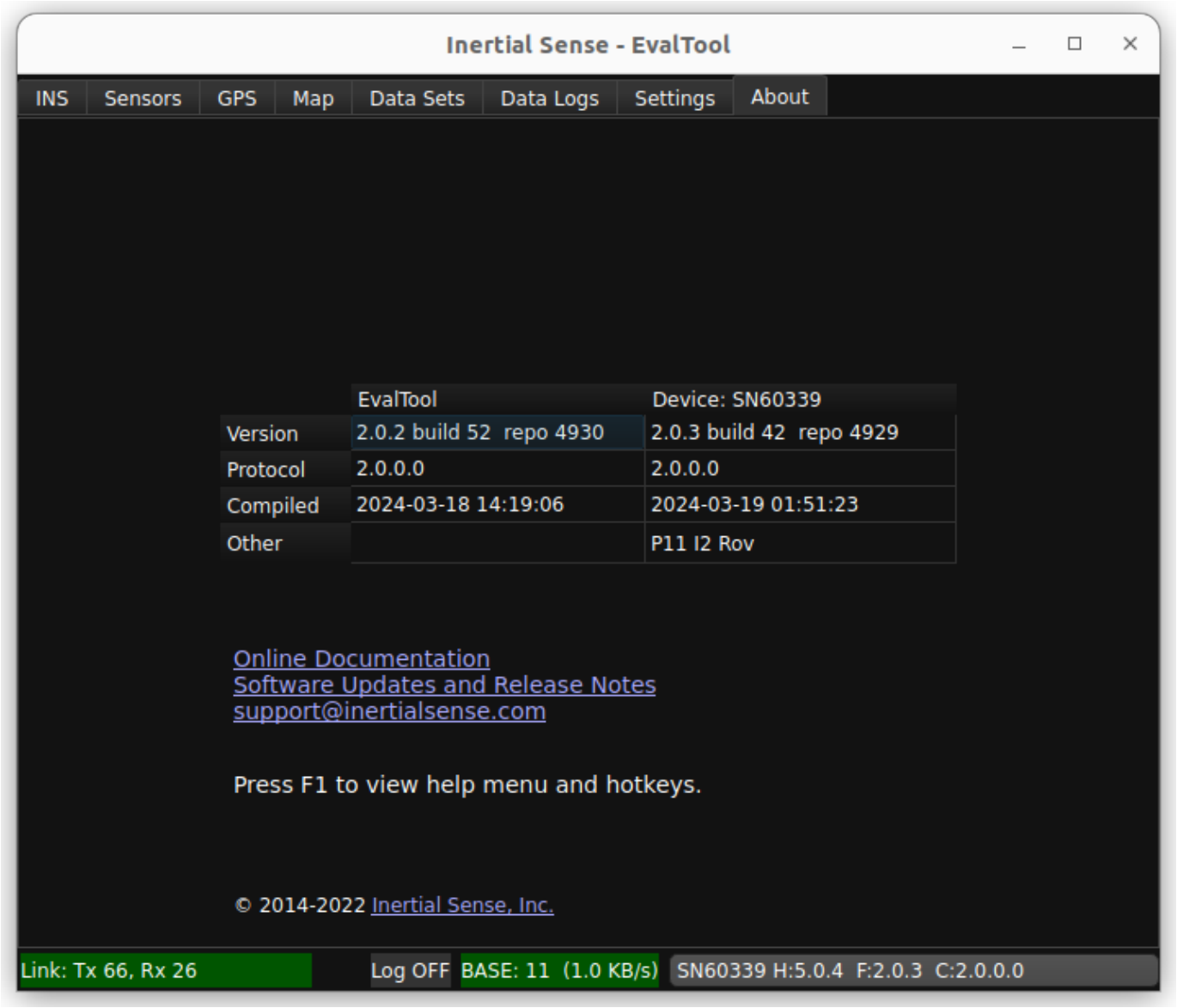


1. IMX Parameters - Shows flash config settings commonly used when setting up RTK units
2. Status - Shows information important to using RTK.
3. Rover/Base Mode - Used in setup of RTK Rovers and RTK base Stations.
4. Message Window - Shows confirmation messages and Flash Config writes.

About Tab

The about tab shows version information for the EvalTool and connected device. It also provides helpful links to online documentation and software release information.

©2022


Inertial Sense - EvalTool
INS
Sensors
GPS
Map
Data Sets
Data Logs
Settings
About
EvalTool
Device: SN60339
Version
2.0.2 build 52 repo 4930
2.0.3 build 42 repo 4929
Protocol
2.0.0.0
2.0.0.0
Compiled
2024-03-18 14:19:06
2024-03-19 01:51:23
Other
P11 I2 Rov
Online Documentation
Software Updates and Release Notes
support@inertialsense.com
Press F1 to view help menu and hotkeys.
© 2014-2022 Inertial Sense, Inc.
Link: Tx 66, Rx 26
Log OFF
BASE: 11 (1.0 KB/s)
SN60339 H:5.0.4 F:2.0.3 C:2.0.0.0

©2022

6.2 CLTool

6.2.1 Overview

The Inertial Sense CLTool is a command line utility that can be used to read and display data, update firmware, and log data from Inertial Sense products. Additionally, CLTool serves as example source code that demonstrates integration of the Inertial Sense SDK into your own source code. The CLTool can be compiled in Linux, Mac, Windows and embedded platforms.

6.2.2 Help Menu

```
# Command line utility for communicating, logging, and updating firmware with Inertial Sense product line.

EXAMPLES
    cltool -c /dev/ttyS2 -did DID_INS_1 DID_GNSS1_POS DID_PIMU      # stream DID messages
    cltool -c /dev/ttyS2 -did 4 13 3                # stream same as line above
    cltool -c /dev/ttyS2 -did 3=5                   # stream DID_PIMU at startupNavDtMs x 5
    cltool -c /dev/ttyS2 -presetPPD                 # stream post processing data (PPD) with INS2
    cltool -c /dev/ttyS2 -presetPPD -lon -lts=1     # stream PPD + INS2 data, logging, dir timestamp
    cltool -c /dev/ttyS2 -edit DID_FLASH_CONFIG     # edit DID_FLASH_CONFIG message
    cltool -c /dev/ttyS2 -baud=115200 -did 5 13=10  # stream at 115200 bps, GPS streamed at 10x startupGnssDtMs
    cltool -c * -baud=921600                        # 921600 bps baudrate on all serial ports
    cltool -rp logs/20170117_222549                 # replay log files from a folder
    cltool -c /dev/ttyS2 -rover=RTCM3:192.168.1.100:7777:mount:user:password         # Connect to RTK NTRIP base
    cltool -c /dev/ttyS2 -get 1,4,13,DID_GNSS1_POS                                   # Return specific DIDs
    cltool -c /dev/ttyS2 -get "{DID_INS_1: {insStatus, theta}, DID_INS_2: {qn2b}}"   # Return portion of two DIDs
    cltool -c /dev/ttyS2 -set "{DID_FLASH_CONFIG: {gnss1AntOffset[1]: 0.8}}"         # Set one value in DID array
    cltool -c /dev/ttyS2 -set "{DID_FLASH_CONFIG: {gnss1AntOffset: [0.8, 0.0, 1.2]}}" # Set values in DID

EXAMPLES (Firmware Update)
    cltool -c /dev/ttyS2 -ufpkg fw/IS-firmware.fpkg
    cltool -c /dev/ttyS2 -uf fw/IS_IMX-5.hex -ub fw/IS_bootloader-STM32L4.hex -uv

OPTIONS (General)
    -baud=BAUDRATE  Set serial port baudrate.  Options: 115200, 230400, 460800, 921600 (default)
    -c DEVICE_PORT  Select serial port. Set DEVICE_PORT to "*" for all ports or "*4" for only first four.
    -dboc           Send stop-broadcast command `$STPB` on close.
    -h --help       Display this help menu.
    -list-devices   Discovers and prints a list of discovered Inertial Sense devices and connected ports.
    -lm             Listen mode for ISB. Disables device verification (-vd) and does not send stop-broadcast command on start.
    -magRecal[n]    Recalibrate magnetometers: 0=multi-axis, 1=single-axis
    -nmea=[s]       Send NMEA message s with added checksum footer. Display rx messages. (`-nmea=ASCE,0,GxGGA,1`)
    -nmea           Listen mode for NMEA message without sending stop-broadcast command `$STPB` at start.
    -q              Quiet mode, no display.
    -raw-out        Outputs all data in a human-readable raw format (used for debugging/learning the ISB protocol).
    -reset          Issue software reset.
    -s              Scroll displayed messages to show history.
    -stats          Display statistics of data received.
    -survey=[s],[d] Survey-in and store base position to refLla: s=[2=3D, 3=float, 4=fix], d=durationSec
    -sysCmd=[c]     Send DID_SYS_CMD c (see eSystemCommand) command then exit the program.
    -vd             Disable device validation.  Use to keep port(s) open even if device response is not received.
    -verbose[=n]    Enable verbose event logging. Use optional '=n' to specify log level between 0 (errors only) and 99 (all events)
    -v              Print version information.

OPTIONS (Special)
    -factoryReset   Reset IMX flash config to factory defaults.
    -romBootloader  Reboot into ROM bootloader mode.  Requires power cycle and reloading bootloader and firmware.

OPTIONS (Event)
    -evf=[t],[po],[pr],[id]    Sets which DID_EVENT's can be broadcast for debug purposes.
         target:        t=[0=device, 1=device's GNSS1 port, 2=device's GNSS2 port],
         portMask:      po=[0x80=currentPort, 0x08=USB port, 0x04=UART2, 0x02=UART1, 0x01=UART)],
         priorityLevel: pr=[Priority ID's to be enabled. See:eEventPriority for protocol EV_ID values].
             It is recommended to have a minimum level of 1 at all times to allow broadcast of critical errors.
         msgTypeIdMask: id=[Protocol ID's to be enabled. Mask together protocol EV_ID value (0x01 << EV_ID).
             See:eEventProtocol for protocol EV_ID values]. It is recommended to mask (0x01 << EVENT_MSG_TYPE_ID_ASCII)
             at all times to allow broadcast of critical errors.

OPTIONS (Firmware Update)
    -ufpkg FILEPATH Update firmware using firmware package file (.fpkg) at FILEPATH.
    -uf FILEPATH    Update app firmware using .hex file FILEPATH.  Add -baud=115200 for systems w/ baud limits.
    -ub FILEPATH    Update bootloader using .bin file FILEPATH if version is old. Must be used with option -uf.
    -fb             Force bootloader update regardless of the version.
    -uv             Run verification after application firmware update.

OPTIONS (Messages)
    -get <DID1>,<DID2>,...                  Return values of dataset(s). DID may be a name or number.
    -get "{<DID>: {<FIELD1>,<FIELD2>,...}}" Return values of dataset(s). DID may be a name or number. YAML input format.
                                            Examples: -get 1,4,12,DID_GNSS1_POS
                                                      -get "{DID_INS_1,DID_GNSS1_POS}"
                                                      -get "{DID_INS_1: {insStatus, theta}, DID_INS_2: {qn2b}}"
    -set "{<DID>: {<FIELD1>: <VALUE>, ...}}" Set values of dataset(s). DID may be a number or name. YAML input format.
                                            Examples: -set "{DID_FLASH_CONFIG: {gnss1AntOffset: [0.8, 0.0, 1.2]}}"
                                                      -set "{DID_FLASH_CONFIG: {gnss1AntOffset[2]: 1.2}}"
                                                      -set "{12: {ioConfig: 0x1a2b012c, ser2BaudRate: 921600}}"
```


©2022

```
    -did [DID#<=PERIODMULT> DID#<=PERIODMULT> ...]  Stream 1 or more datasets and display w/ compact view.
    -edit [DID#<=PERIODMULT>]                       Stream and edit 1 dataset.
          Each DID# can be the DID number or name and appended with <=PERIODMULT> to decrease message frequency.
          Message period = source period x PERIODMULT. PERIODMULT is 1 if not specified.
          Common DIDs: DID_INS_1, DID_INS_2, DID_INS_4, DID_PIMU, DID_IMU, DID_GNSS1_POS,
          DID_GNSS2_RTK_CMP_REL, DID_BAROMETER, DID_MAGNETOMETER, DID_FLASH_CONFIG (see data_sets.h for complete list)
    -dids           Print list of all DID datasets
    -persistent     Save current streams as persistent messages enabled on startup
    -presetPPD      Send RMC preset to enable IMX post processing data (PPD) stream
    -presetINS      Send RMC preset to enable INS data stream
    -presetGPXPPD   Send RMC preset to enable GPX post processing data (PPD) stream

OPTIONS (Logging to file, disabled by default)
    -lon            Enable logging
    -lt=TYPE        Log type: raw (default), dat, sdat, kml or csv
    -lp PATH        Log data to path (default: ./IS_logs)
    -lmb=MB         File culling: Log drive usage limit in MB. (default: 0). `-lmb=0 -lms=0` disables file culling.
    -lms=PERCENT    File culling: Log drive space limit in percent of total drive, 0.0 to 1.0. (default: 0.5)
    -lmf=BYTES      Log max file size in bytes (default: 5242880)
    -lts=0          Log sub folder, 0 or blank for none, 1 for timestamp, else use as is
    -r              Replay data log from default path
    -rp PATH        Replay data log from PATH
    -rs=SPEED       Replay data log at x SPEED. SPEED=0 runs as fast as possible.

OPTIONS (READ flash config) - DEPRECATED, use `-get` instead
    -imxFlashCfg                                # List all "keys" and "values" in IMX
    -gpxFlashCfg                                # List all "keys" and "values" in GPX
    "-imxFlashCfg=[key]|[key]|[key]"            # List specific IMX values
    "-gpxFlashCfg=[key]|[key]|[key]"            # List specific GPX values

OPTIONS (WRITE flash config) - DEPRECATED, use `-set` instead
    "-imxFlashCfg=[key]=[value]|[key]=[value]"  # Set key / value pairs in IMX flash config.
    "-gpxFlashCfg=[key]=[value]|[key]=[value]"  # Set key / value pairs in GPX flash config.
                                                # Surround with "quotes" when using pipe operator.
EXAMPLES
    cltool -c /dev/ttyS2 -imxFlashCfg  # Read from device and print all keys and values
    cltool -c /dev/ttyS2 "-imxFlashCfg=insOffset[1]=1.2|ser2BaudRate=115200"  # Set multiple values

OPTIONS (RTK Rover / Base)
    -rover=[type]:[IP or URL]:[port]:[mountpoint]:[username]:[password]
        As a rover (client), receive RTK corrections.  Examples:
            -rover=TCP:RTCM3:192.168.1.100:7777:mountpoint:username:password   (NTRIP)
            -rover=TCP:RTCM3:192.168.1.100:7777
            -rover=TCP:UBLOX:192.168.1.100:7777
            -rover=SERIAL:RTCM3:/dev/ttyS2:57600        (port, baud rate)
    -base=[IP]:[port]   As a Base (sever), send RTK corrections.  Examples:
            -base=TCP::7777                             (IP is optional)
            -base=TCP:192.168.1.43:7777
            -base=SERIAL:/dev/ttyS2:921600
```

6.2.3 Compile & Run (Linux/Mac)

1. You must have cmake installed on your machine. To do this, download the cmake application at https://cmake.org/download/. Then, using the command line, you will need to install cmake with either of the following commands depending on your platform:

```
Mac:
sudo "/Applications/CMake.app/Contents.bin/cmake-gui" --install

Linux:
sudo apt-get install cmake
```

2. Create build directory

```
cd cltool
mkdir build
```

3. Run cmake from within build directory

```
cd build
cmake ..
```

4. Compile using make

```
make
```

5. If necessary, add current user to the "dialout" group in order to read and write to the USB serial communication ports:

```
sudo usermod -a -G dialout $USER
sudo usermod -a -G plugdev $USER
(reboot computer)
```

6. Run executable

©2022

```
cd build
./cltool
```

6.2.4 Compile & Run (Windows CMake CL)

1. Install CMake for Windows
2. Create build directory ```bash cd cltool mkdir build
3. Run cmake from within build directory

```
cd build
cmake ..
```

4. Compile

```
cmake --build .
```

5. Run executable

```
cd Release (or Debug depending on CMake configuration you selected)
cltool.exe
```

6.2.5 Compile & Run (Windows CMake Visual Studio)

Windows Visual Studio supports CMake projects. Follow the instructions provided by Microsoft: https://learn.microsoft.com/en-us/cpp/build/cmake-projects-in-visual-studio?view=msvc-170

6.2.6 Updating Firmware with CLTool

Updating using Firmware Package

Updating firmware using a firmware package file provides a simple method to update multiple devices in one process. This include the ability to update an IMX-GPX module pair in one step. The cltool only needs know the file path of the firmware package file and the serial port of the device to be updated. The file extension for a firmware package is `.fpkg`.

NOTE: Updating the IMX firmware using a firmware package currently not supported and will become available in a future update.

```
cltool -c DEVICE_PORT -ufpkg FILEPATH
```

The following is a specific example of using a firmware package file:

```
cltool -c /dev/ttyACM0 -ufpkg IS-firmware_2.0.3_2024-03-18_213925.fpkg
```

Updating using Single Firmware File (Legacy Mode)

The CLTool can be used to update device firmware with the following options. This is the legacy firmware update methods that works only with the IMX-5 and earlier products (IMX-3, EVB-2, etc.).

©2022

```
cltool -c DEVICE_PORT -uf [FW_FILEPATH] -ub [BL_FILEPATH] -uv
```

Options	Description
`-c DEVICE_PORT`	Specifies the device serial or USB port (i.e. `/dev/ttyACM0`).
`-uf [FW_FILEPATH]`	Specifies the application firmware file path.
`-ub [BL_FILEPATH]`	(Optional) Specified the bootloader firmware file. The bootloader is only updated if the version of the file provided is newer than the bootloader version currently on the device.
`-uv`	(Optional) Run verification after application firmware update.

The following is a specific example:

```
cltool -c /dev/ttyS2 -uf fw/IS_IMX-5.hex -ub fw/IS_bootloader-STM32L4.hex -uv
```

Note: The firmware can only be updated at the following baud rates: 3000000, 921600, 460800, 230400, 115200

6.2.7 Logging with CLTool

The CLTool can be used to log data to file with the following options:

```
cltool -c DEVICE_PORT -lon -lt=LOG_TYPE -lp DIRECTORY
```

Options	Description
`-lon`	Enable logging.
`-lt=LOG_TYPE`	Specifies the log file type to be written. LOG_TYPE can be `raw`, `dat`, or `csv`.
`-lp DIRECTORY`	(Optional) Specifies the path where log files will be written. When not specified, the default location will be the current working directory.

Log File Types

Log Type

Log Type	Description
`raw`	Binary file containing byte for byte data received over the serial ports. All packets remain in their native form. Used for logging InertialSense binary (ISB), NMEA, RTCM3, uBlox UBX binary and SPARTN, and any other packet formats. ***Recommended for logging all data formats and post processing***.
`dat`	Binary file containing InertialSense binary (ISB) DID data sets in "chunk" groups containing data in serial order as they appear over the serial port. Default file format. ***Recommended for post processing.***
`csv`	Comma-Separated Values - Plain text file that uses specific structuring to arrange tabular data. Its basic format involves separating each data field (or cell in a table) with a comma and each record (or row) is on a new line. This simple format allows for ease in data import and export between programs that handle tabular data, such as databases and spreadsheets.

The following is an example of enabling the logger with type raw and specifying the output directory:

```
./cltool -c /dev/ttyACM0 -lon -lt=raw -lp /media/usbdrive/data
```

6.2.8 Command Line Options

Navigate to the directory `/cpp/SDK/cltool/build` and run the CLTool with the help option, "`-h`"

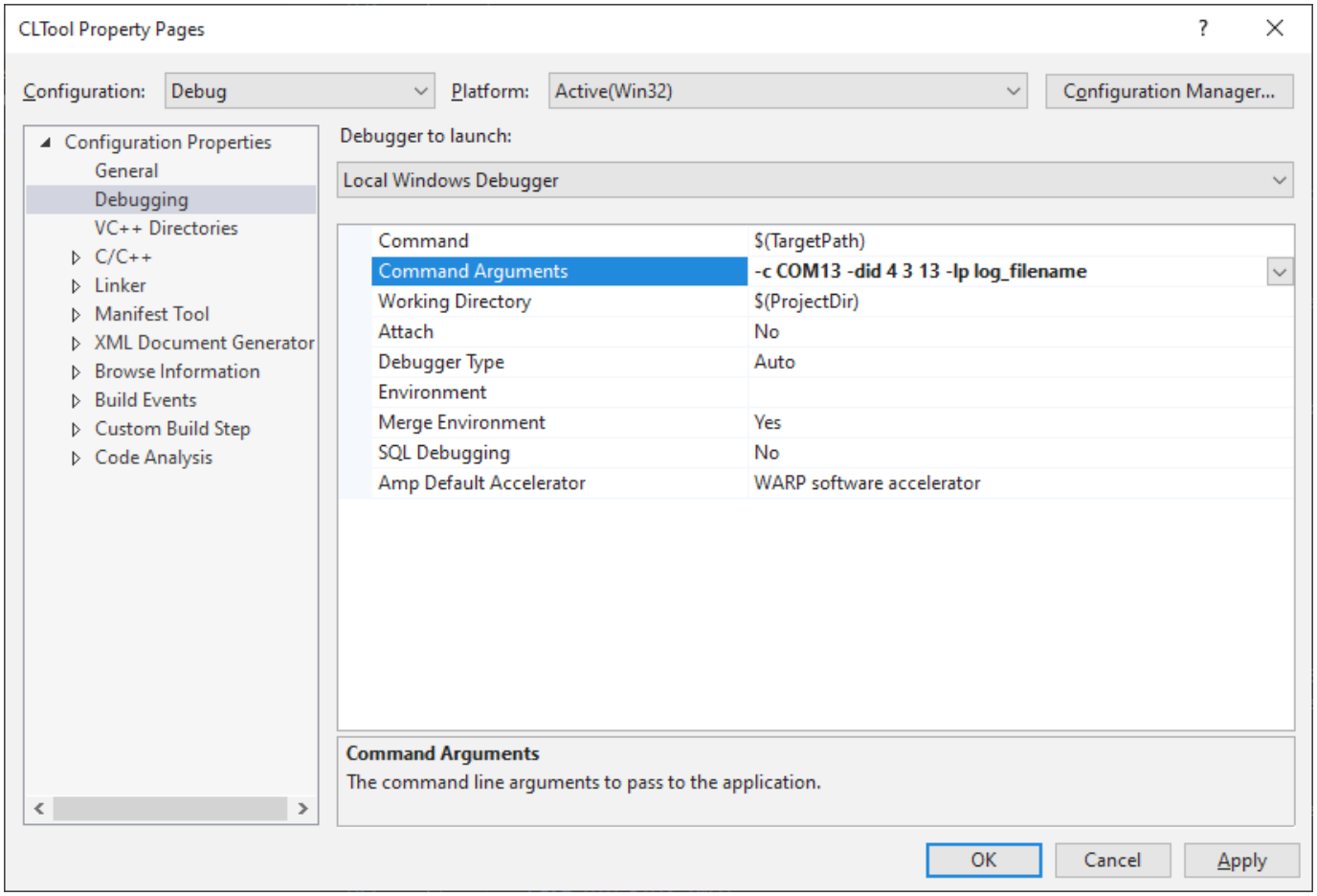
©2022

```
./cltool -h
```

to display the command line options

6.2.9 Command Line Options in MS Visual Studio

When using MS Visual Studio IDE, command line arguments can be supplied by right clicking the project in the solution explorer and then selecting **Configuration Properties -> Debugging -> Command Arguments** (see image below).



©2022

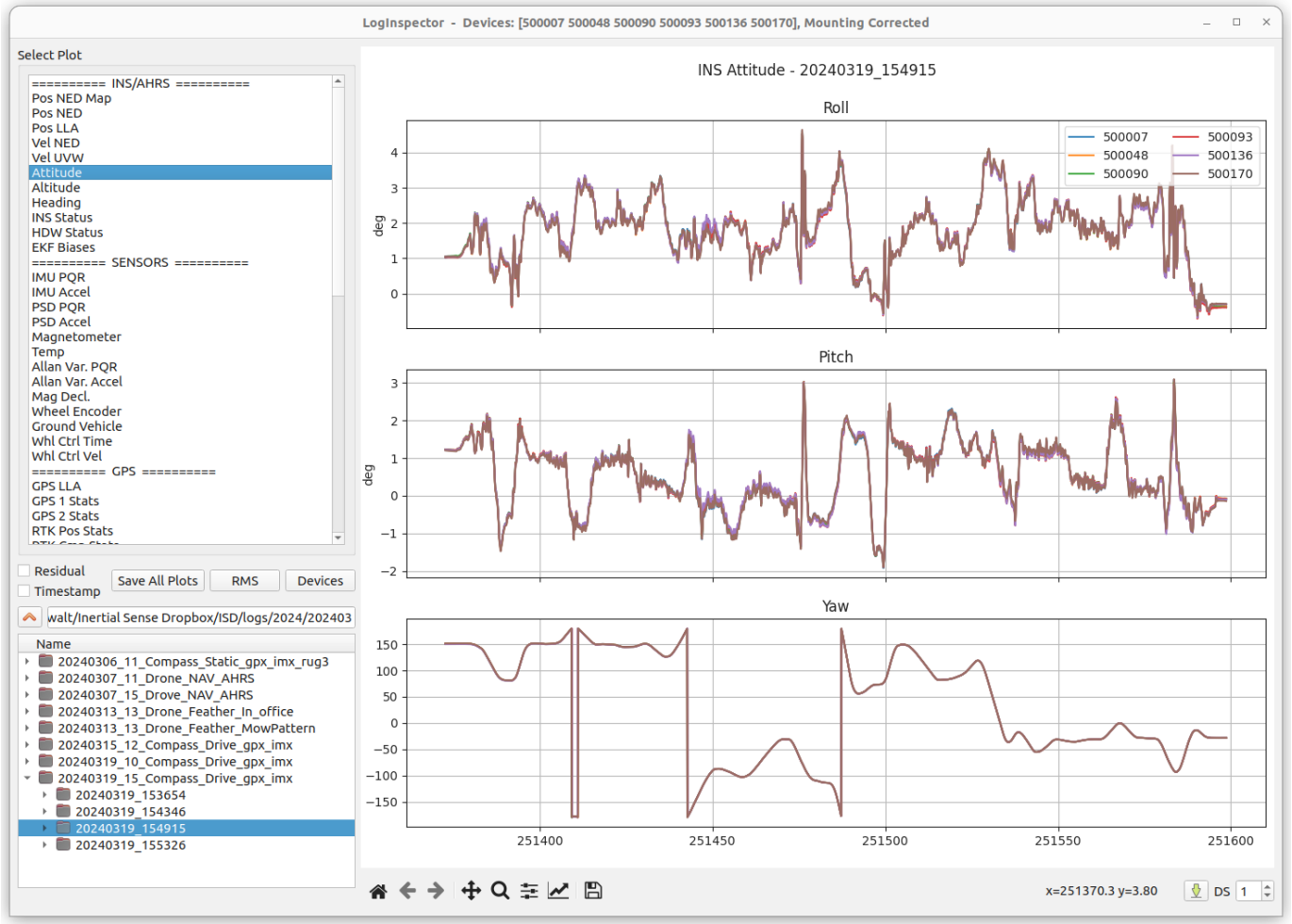
6.3 Log Inspector

6.3.1 Overview

Log Inspector is an open source python utility for viewing and scrubbing InertialSense data log (.dat) files.

6.3.2 Getting Started

Log Inspector can open and plot .dat PPD log files. The lower left hand corner file browser allows you to enter a "working directory" in the directory field. The whole directory containing the desired is selected from the directory tree. Once the log is opened, the buttons in the upper left hand corner are used to graph various data sets.



©2022

6.3.3 Standard data sets

- POS NED Map - Used to plot INS position data in NED frame.
- POS NED - INS position in NED frame.
- POS LLA - INS and GNSS position in LLA.
- GPS LLA - GNSS LLA position.
- Vel NED - Velocity in NED frame.
- Vel UVW - Velocity in body frame.
- Attitude - Euler angle attitude in degrees.
- Heading - Heading data from magnetometer, INS, and RTK.
- INS Status - Plots of status flags vs time.
- HDW Status - plots of hardware status flags vs time.

6.3.4 Building

Note - logInspector requires Python 3.

Navigate to the Inertial Sense SDK directory

```
cd python
pip3 install .
```

Create a config file.

```
C:\Users\[USER]\Documents\Inertial_Sense\log_inspector.yaml
```

Add the following or similar contents to this file.

```
directory: C:\Users\<username>\Documents\Inertial_Sense\Logs\20181116_SKI\morning_run_1\back\20181116_175352
logs_directory: C:\Users\<username>\Documents\Inertial_Sense\Logs

serials: ["ALL"]
```

6.3.5 Running

To run logInspector open a shell and navigate to the logInspector directory and enter the following commands:

```
python3 logInspector.py
```

6.3.6 Other Directory Contents

The *logInspector* also contains some example implementations for dealing with log files directly in python.

logReader

This python module is responsible for loading the log file through a pybind11 interface. All the data in the log is eventually put in the `log.data` array.

©2022

logPlotter

This python module is responsible for creating plots. Adding new plots is easy, data is directly accessed using the member `logReader` object.

logInspector

A pyqt5 GUI which uses logPlotter to generate plots.

©2022

6.4 SDK

Overview

The Inertial Sense open source software development kit (SDK) provides quick integration for communication with the Inertial Sense product line, including the IMX. It includes data logger, math libraries, and serial port interface for Linux and Windows environments.

6.4.1 C vs. C++ Implementation

The Inertial Sense SDK provides both C and C++ programming language implementation. The following compares differences between these implementations.

C

- Easier implementation
- Light weight
- Smaller code size
- Minimal subset of SDK files
- Recommended for smaller projects that require lower memory usage.
- SDK files: ISComm.c

C++

- Object oriented device representation
- Fully integrated support for:
- Commutations: single or multiple data type callback functions.
- Serial port handling included
- Datalogging
- Firmware update (bootloader)
- Recommended for typical to advanced C++ applications and production level integration.
- SDK files: InertialSense.cpp

6.4.2 Installing and Configuring Visual Studio

The SDK example projects can be conveniently compiled using gcc with cmake or Visual Studio. The following sections outline how to setup Visual Studio for use with the SDK example projects.

Installing

The SDK example projects can be compiled using the Community (free) version of Visual Studio. Windows SDK should be installed in addition to Visual Studio, as an added option in the Visual Studio installer or using the separate Windows SDK installer.

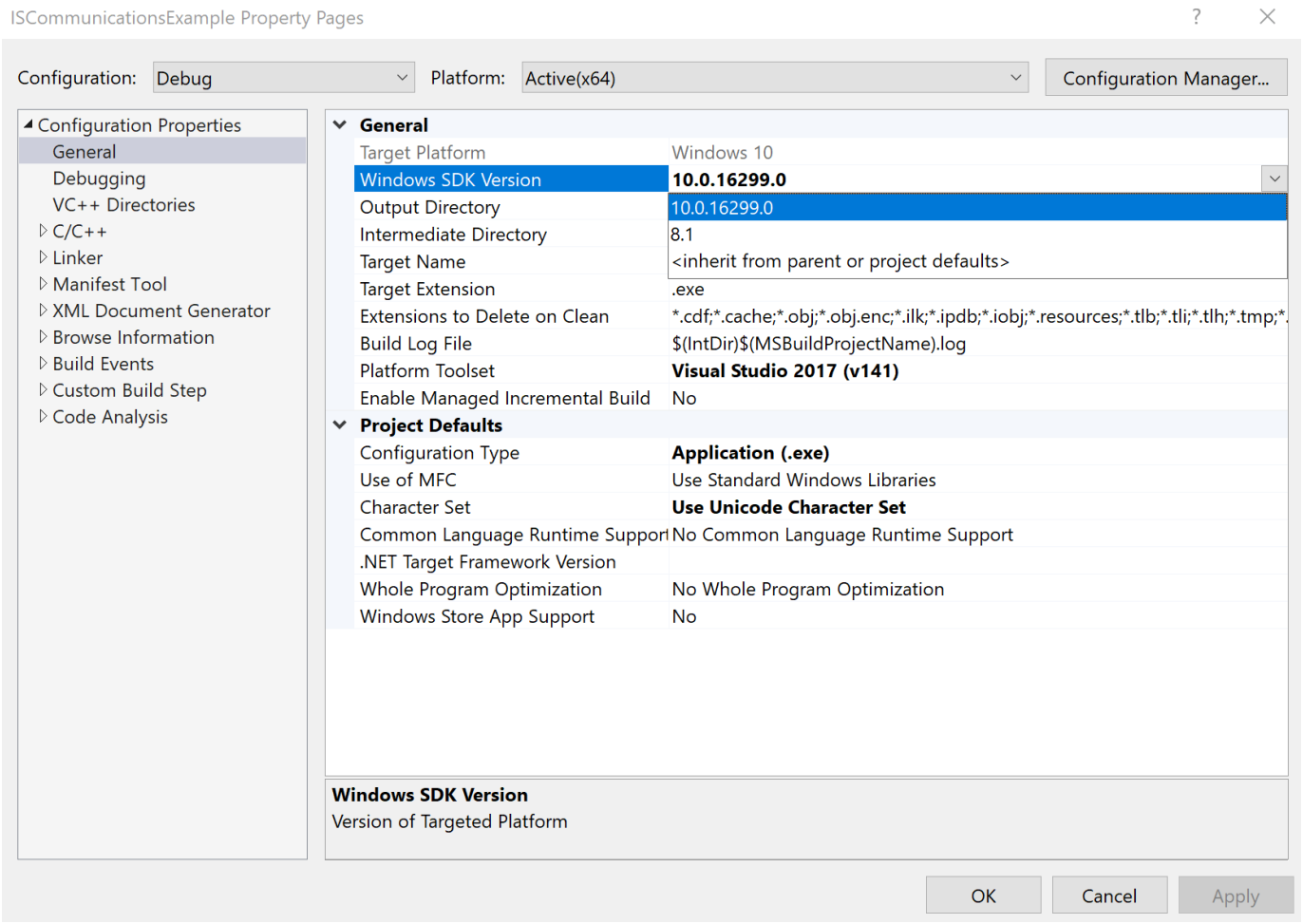
- Visual Studio
- Windows SDK - Can be installed using option in Visual Studio Installer or using separate Windows SDK installer.

©2022

Configuring

When compiling an Inertial Sense SDK example project in Visual Studio, the currently installed version of Windows SDK must be selected in the project properties, as illustrated below:

```
Project properties > General > Windows SDK Version > [Currently Installed Version]
```



6.4.3 SDK Example Projects Overview

Example Project	Language	Description
NMEA Communications	C	How to use SDK for NMEA communications.
Binary Communications	C	How to use SDK for binary communications.
Firmware Update	C	How to use bootloader for embedded firmware update.
Data Logger	C++	How to use SDK data logging.
CLTool	C++	Open source project illustrating how to use the InertialSense C++ class. It combines all SDK capabilities including serial communications, data logging to file, and embedded firmware update.

6.4.4

©2022

6.5 Example Projects

6.5.1 Binary Communications Example Project

This ISCommExample project demonstrates binary communications with the Inertial Sense Products (IMX) using the Inertial Sense SDK.

Files

Project Files

- ISCommExample.cpp

SDK Files

- data_sets.c
- data_sets.h
- ISComm.c
- ISComm.h
- PortFactory.cpp
- PortFactory.h
- serialPort.c
- serialPort.h

Implementation

STEP 1: ADD INCLUDES

```
// Change these include paths to the correct paths for your project
#include "ISComm.h"
#include "ISPose.h"
#include "ISUtilities.h"
#include "PortFactory.h"
#include "protocol_nmea.h"
```

STEP 2: INITIALIZE AND OPEN SERIAL PORT

```
    // Initialize the serial port (Windows, MAC or Linux) - if using an embedded system like Arduino,
    //  you will need to handle the serial port creation, open and reads yourself.

    SerialPortFactory& spf = SerialPortFactory::getInstance();
    spf.setBaudRate(921600);
    port_handle_t port = spf.bindPort(argv[1]);

    if (port == nullptr) {
        printf("Failed to allocate port\r\n");
        return -2;
    }

    // Binding a port does not open a port.. so let's open it
    if (!portIsOpened(port) && (portOpen(port) != PORT_ERROR__NONE)) {
        printf("Failed to open port\r\n");
        return -3;
    }
```

STEP 3: STOP ANY MESSAGE BROADCASTING

```
    is_comm_stop_broadcasts_all_ports(port);
```

STEP 4: BIND CALLBACKS TO THE PORT

```
    // Any ISB protocol messages will call into this handler (defined above).
    is_comm_register_port_isb_handler(port, isbDataHandler);
```


©2022

The handler receives a `p_data_t*` for every successfully parsed message; `data->hdr.id` identifies the DID and `data->ptr` points at the payload struct:

```
int isbDataHandler(void* ctx, p_data_t* data, port_handle_t port) {
    switch (data->hdr.id)
    {
    case DID_INS_1:
        handleIns1Message((ins_1_t*)data->ptr);
        break;

    case DID_INS_2:
        handleIns2Message((ins_2_t*)data->ptr);
        break;

    case DID_GNSS1_POS:
        handleGpsMessage((gnss_pos_t*)data->ptr);
        break;

    case DID_IMU:
        handleImuMessage((imu_t*)data->ptr);
        break;

        // TODO: add other cases for other data ids that you care about
    }

    return 0;
}
```

STEP 5: SET CONFIGURATION (OPTIONAL)

```
    // Set INS output Euler rotation in radians to 90 degrees roll for mounting
    float rotation[3] = { 90.0f*C_DEG2RAD_F, 0.0f, 0.0f };
    is_comm_set_data(port, DID_FLASH_CONFIG, sizeof(float) * 3, offsetof(nvm_flash_cfg_t, insRotation), rotation);
```

STEP 6: ENABLE MESSAGE BROADCASTING

```
    // Ask for INS message w/ update 40ms period (4ms source period x 10).  Set data rate to zero to disable broadcast and pull a single packet.
    is_comm_get_data(port, DID_INS_1, 0, 0, 10);

    // Ask for GPS message at period of 200ms (200ms source period x 1).  Size and offset can be left at 0 unless you want to just pull a specific field from
a data set.
    is_comm_get_data(port, DID_GNSS1_POS, 0, 0, 1);

    // Ask for IMU message at period of 96ms (DID_FLASH_CONFIG.startupNavDtMs source period x 6).  This could be as high as 1000 times a second (period
multiple of 1)
    is_comm_get_data(port, DID_IMU, 0, 0, 6);
```

STEP 7: SAVE PERSISTENT MESSAGES

(OPTIONAL) Save currently enabled streams as persistent messages enabled after reboot.

```
    system_command_t cfg;
    cfg.command = SYS_CMD_SAVE_PERSISTENT_MESSAGES;
    cfg.invCommand = ~cfg.command;

    is_comm_set_data(port, DID_SYS_CMD, 0, 0, &cfg);
```

STEP 8: PROCESS AND PARSE MESSAGES FROM THE PORT

```
    // This should run a fairly fast rate, (1ms is typical) to avoid data from filling
    // the COMM buffer, which could lead to data drop.
    while (portIsOpened(port)) {
        is_comm_port_parse_messages(port);
        SLEEP_MS(1);
    }
```

Compile & Run (Linux/Mac)

1. Create build directory

```
cd InertialSenseSDK/ExampleProjects/ISComm
mkdir build
```

2. Run cmake from within build directory

```
cd build
cmake ..
```


©2022

3. Compile using make

```
make
```

4. If necessary, add current user to the "dialout" group in order to read and write to the USB serial communication ports:

```
sudo usermod -a -G dialout $USER
sudo usermod -a -G plugdev $USER
(reboot computer)
```

5. Run executable

```
./ISCommExample /dev/ttyUSB0
```

Compile & Run (Windows Powershell)

*Note - Install CMake for Windows natively, or install the CMake for Windows extension for Visual Studio

1. Create build directory

```
cd InertialSenseSDK/ExampleProjects/ISComm
mkdir build
```

2. Run cmake from within build directory

```
cd build
cmake ..
```

3. Compile using make

```
cmake --build .
```

4. Run executable

```
C:\InertialSenseSDK\ExampleProjects\ISComm\build\Release\ISCommExample.exe COM3
```

Summary

This section has covered the basic functionality you need to set up and communicate with Inertial Sense products. If this doesn't cover everything you need, feel free to reach out to us on the Inertial Sense SDK GitHub repository, and we will be happy to help.

©2022

6.5.2 ASCII Communications Example Project

This ISAsciiExample project demonstrates ASCII communications with the Inertial Sense Products (IMX) using the Inertial Sense SDK. See the ASCII protocol section for details on the ASCII packet structures.

Files

Project Files

- ISAsciiExample.cpp

SDK Files

- data_sets.c
- data_sets.h
- ISComm.c
- ISComm.h
- ISConstants.h
- serialPort.c
- serialPort.h
- serialPortPlatform.c
- serialPortPlatform.h

Implementation

STEP 1: ADD INCLUDES

```
// Change these include paths to the correct paths for the project
#include "../../src/serialPortPlatform.h"
#include "../../src/ISComm.h"
```

STEP 2: INITIALIZE AND OPEN SERIAL PORT

```
    serial_port_t serialPort;

    // Initialize the serial port (Windows, MAC or Linux) - if using an embedded system like Arduino,
    //  you will need to handle the serial port creation, open and reads yourself. In this
    //  case, you do not need to include serialPort.h/.c and serialPortPlatform.h/.c in your project.
    serialPortPlatformInit(&serialPort);

    // Open serial, last parameter is a 1 which means a blocking read, you can set as 0 for non-blocking
    // you can change the baudrate to a supported baud rate (IS_BAUDRATE_*), make sure to reboot the IMX
    //  if you are changing baud rates, you only need to do this when you are changing baud rates.
    if (!serialPortOpen(&serialPort, argv[1], IS_BAUDRATE_921600, 1))
    {
        printf("Failed to open serial port on com port %s\r\n", argv[1]);
    }
```

STEP 3: DISABLE PRIOR MESSAGE BROADCASTING

```
// Stop all broadcasts on the device on all ports.  We don't want binary message coming through while we are doing ASCII
if (!portWriteAscii(&serialPort, "STPB", 4))
{
    printf("Failed to encode stop broadcasts message\r\n");
}
```

STEP 4: ENABLE MESSAGE BROADCASTING

```
    // ASCII protocol is based on NMEA protocol https://en.wikipedia.org/wiki/NMEA_0183
    // turn on the INS message at a period of 100 milliseconds (10 hz)
    // portWriteAscii takes care of the leading $ character, checksum and ending \r\n newline
    // ASCE message enables ASCII broadcasts
    // ASCE fields: 1:options, ID0, Period0, ID1, Period1, ........ ID19, Period19
    // IDs:
    // NMEA_MSG_ID_INVALID   = 0,
    // NMEA_MSG_ID_PIMU      = 1,
    // NMEA_MSG_ID_PPIMU     = 2,
```


©2022

```
    // NMEA_MSG_ID_PRIMU     = 3,
    // NMEA_MSG_ID_PINS1     = 4,
    // NMEA_MSG_ID_PINS2     = 5,
    // NMEA_MSG_ID_PGPSP     = 6,
    // NMEA_MSG_ID_GNGGA     = 7,
    // NMEA_MSG_ID_GNGLL     = 8,
    // NMEA_MSG_ID_GNGSA     = 9,
    // NMEA_MSG_ID_GNRMC     = 10,
    // NMEA_MSG_ID_GNZDA     = 11,
    // NMEA_MSG_ID_PASHR     = 12,
    // NMEA_MSG_ID_PSTRB     = 13,
    // NMEA_MSG_ID_INFO      = 14,
    // NMEA_MSG_ID_GNGSV     = 15,
    // NMEA_MSG_ID_GNVTG     = 16,
    // NMEA_MSG_ID_INTEL     = 17,

    // options can be 0 for current serial port, 1 for serial 0, 2 for serial 1 or 3 for both serial ports
    // Instead of a 0 for a message, it can be left blank (,,) to not modify the period for that message
    // please see the user manual for additional updates and notes

    // Get PINS1 @ 5Hz on the connected serial port, leave all other broadcasts the same, and save persistent messages.
    const char* asciiMessage = "ASCE,0,4,1";

    // Get PINS1 @ 1Hz and PGPSP @ 1Hz on the connected serial port, leave all other broadcasts the same
    // const char* asciiMessage = "ASCE,0,6,5";

    // Get PIMU @ 50Hz, GGA @ 5Hz, serial0 and serial1 ports, set all other periods to 0
    //  const char* asciiMessage = "ASCE,3,7,1";

    if (!portWriteAscii(&serialPort, asciiMessage, (int)strnlen(asciiMessage, 128)))
    {
        printf("Failed to encode ASCII get INS message\r\n");
    }
```

STEP 5: SAVE PERSISTENT MESSAGES

(OPTIONAL) This remembers the current communications and automatically streams data following reboot.

```
if (!portWriteAscii(&serialPort, "PERS", 4))
{
    printf("Failed to encode ASCII save persistent message\r\n");
}
```

STEP 6: HANDLE RECEIVED DATA

```
    unsigned char* asciiData;
    unsigned char asciiLine[512];

    // you can set running to false with some other piece of code to break out of the loop and end the program
    while (running)
    {
        if (serialPortReadAscii(&serialPort, asciiLine, sizeof(asciiLine), &asciiData) > 0)
        {
            printf("%s\n", asciiData);
        }
    }
```

Compile & Run (Linux/Mac)

1. Create build directory

```
cd InertialSenseSDK/ExampleProjects/Ascii
mkdir build
```

2. Run cmake from within build directory

```
cd build
cmake ..
```

3. Compile using make

```
make
```

4. If necessary, add current user to the "dialout" group in order to read and write to the USB serial communication ports:

```
sudo usermod -a -G dialout $USER
sudo usermod -a -G plugdev $USER
(reboot computer)
```

5. Run executable

©2022

```
./ISAsciiExample /dev/ttyUSB0
```

Compile & Run (Windows Powershell)

*Note - Install CMake for Windows natively, or install the CMake for Windows extension for Visual Studio

1. Create build directory

```
cd InertialSenseSDK/ExampleProjects/Ascii
mkdir build
```

2. Run cmake from within build directory

```
cd build
cmake ..
```

3. Compile using make

```
cmake --build .
```

4. Run executable

```
C:\InertialSenseSDK\ExampleProjects\Ascii\build\Release\ISAsciiExample.exe COM3
```

Summary

This section has covered the basic functionality you need to set up and communicate with Inertial Sense products. If this doesn't cover everything you need, feel free to reach out to us on the Inertial Sense SDK GitHub repository, and we will be happy to help.

©2022

6.5.3 Basic Arduino Communications Example Project

Interfacing with the IMX over serial

This example shows how to communicate with the IMX using the Inertial Sense Binary Communications Protocol. The example code can be found in the Inertial Sense SDK Arduino project.

> **Important**
>
> Update the IMX to the latest firmware

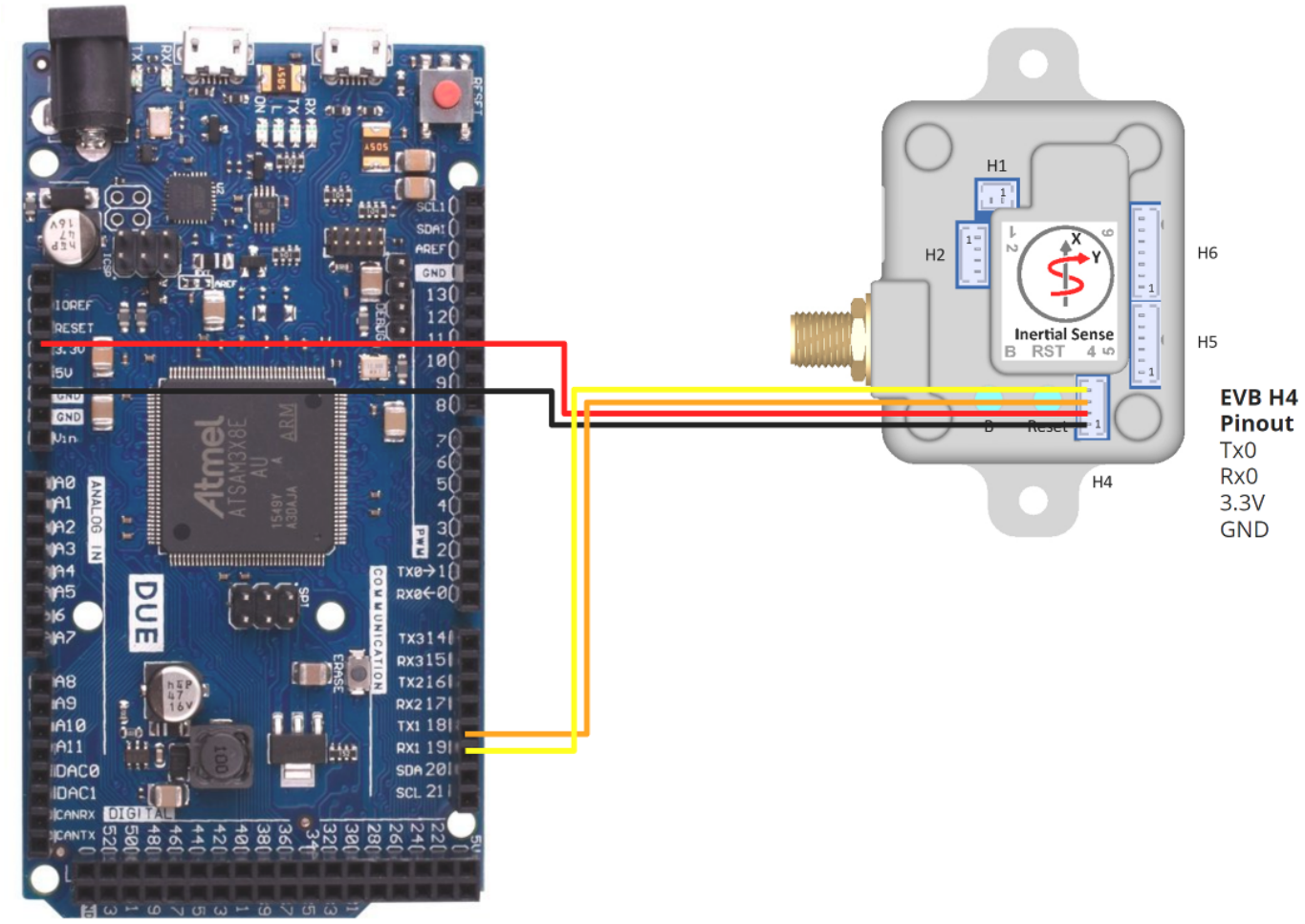
This example demonstrates how to use the Inertial Sense EVB with an Arduino Due. The Due was selected because it has two serial ports. This way the Arduino can communicate with the IMX using one of the ports, and write the output over the Serial Monitor to the computer using the other.

> **Warning**
>
> The InertialSense SDK requires 64-bit double support. 32-bit processors (Arduino Due, Zero, and M0) are supported. 8-bit processors (i.e. Arduino Mega and Uno) are NOT supported. The ASCII protocol (not covered in this example) may be used on an 8-bit Arduino.

> **Note**
>
> A Raspberry PI (similar in price to the Arduino) is a good alternative to the Arduino. Either the Binary Communications and ASCII Communications example projects can be run on a Raspberry PI.

©2022

Wiring Guide



After downloading the Inertial Sense SDK, navigate to ExampleProjects/Arduino/ReadIS. Use the ImportSdkFiles.bat (Windows) or ImportSdkFiles.sh (Linux) to copy the required files from the SDK into the src/ISsdk directory. The resulting file structure for the ReadIS Arduino sketch should look like the following:

```
|-ReadIS
  | - ImportSdkFiles.bat
  | - ImportSdkFiles.sh
  | - ReadIS.ino
  | - src
    | - ISsdk
      | - data_sets.c
      | - data_sets.h
      | - ISComm.c
      | - ISComm.h
      | - ISConstants.h
      | - rtk_defines.h
      | - core
        | - types.h
    | - hw-libs
      | - printf
        | - printf.c
        | - printf.h
```

What is an ino file?

An `.ino` file is the arduino extension for a sketch. It is actually C++ code.

©2022

> **Note**
>
> Note that there are `.c` files in the tree. You'll need to make sure that these files are compiled by the toolchain, otherwise `xxxx is not defined` errors can occur.

SDK Implementation

`ReadIS.ino` file explained:

STEP 1: ADD INCLUDES

The `"ISComm.h"` header file includes all the other required code. `stddef.h` file from the standard library is required for the `offsetof` function.

```
#include "src/ISsdk/ISComm.h"
#include <stddef.h>
```

STEP 2: CREATE BUFFERS

Next, define a buffer to hold data. As the IMX sends data, this buffer is used to hold the data until a full message arrives. This buffer only needs to be as big as the largest message expected, multiplied by two + 32 (worst case scenario if there is a bad transmission). For this example a 1KB buffer is used.

```
// This buffer is going to be used to hold messages as they come in.
// You can make this 512 size if memory is tight.
static uint8_t s_buffer[1024];
// create an instance to hold communications state
static is_comm_instance_t comm;
```

STEP 3: SERIAL PORT INITIALIZATION

```
void setup()
{
    // Initialize both serial ports:
    Serial.begin(115200);
    Serial1.begin(115200);

    if (sizeof(double) != 8)
    {
        Serial.println("Inertial Sense SDK requires 64 bit double support");
        while (true)
        {
        };
    }

    Serial.println("initializing");

    // Initialize comm interface - call this before doing any comm functions
    is_comm_init(&comm, s_buffer, sizeof(s_buffer), NULL); // TODO: Use callbacks
    is_comm_enable_protocol(&comm, _PTYPE_INERTIAL_SENSE_DATA);
    is_comm_enable_protocol(&comm, _PTYPE_NMEA);

    // Stop all the broadcasts on the device
    int messageSize = is_comm_write_to_buf(s_buffer, sizeof(s_buffer), &comm, PKT_TYPE_STOP_BROADCASTS_ALL_PORTS, 0, 0, 0, NULL);
    Serial1.write(comm.rxBuf.start, messageSize); // Transmit the message to the inertialsense device

    // Ask for ins_1 message 20 times per second.  Ask for the whole thing, so
    // set 0's for the offset and size
    messageSize = is_comm_get_data_to_buf(s_buffer, sizeof(s_buffer), &comm, DID_INS_1, sizeof(ins_1_t), 0, 1000);
    Serial1.write(comm.rxBuf.start, messageSize); // Transmit the message to the inertialsense device
}
```

Initialize the communication using the following steps as shown above:

1. Initialize the serial ports
2. Tell the communication interface where to find the buffer to use to hold messages, and how big that buffer is.
3. Reset communications on the device
4. Perform configuration of the IMX
5. Tell the IMX what data to stream, and how often

©2022

Whenever sending a command to the IMX, the command is put into the buffer, and the length of the message is returned by one of the configuration functions. That buffer needs to be written out to the IMX for the command to be received.

Tip

It is recommended to use the enumerations in `data_sets.h` such as `SYS_CFG_BITS_RTK_ROVER` to configure the device. This aids code readability and reduces the chance for errors.

In this example, the `DID_INS_1` message is streamed. All available messages can be found in the `data_sets.h` file, defined as C-style structs.

STEP 4: HANDLE RECEIVED DATA

```
void loop()
{
    // Read from port 1, and see if we have a complete inertialsense packet
    if (Serial1.available())
    {
        uint8_t inByte = Serial1.read();
        // This function returns the DID of the message that was just parsed, we can then point the buffer to
        // the right function to handle the message.  We can use a cast to interpret the s_buffer as the
        // kind of message that we received.
        uint32_t message_type = is_comm_parse_byte(&comm, inByte);
        switch (message_type)
        {
        case _PTYPE_INERTIAL_SENSE_DATA:
            switch (comm.rxPkt.dataHdr.id)
            {
            case DID_NULL:
                break;
            case DID_INS_1:
                handleINSMessage((ins_1_t *)(comm.rxPkt.data.ptr));
                break;
            default:
                Serial.print("Got an unexpected message DID: ");
                Serial.println(message_type, DEC);
            }
        }
    }
}
```

In this code, every byte that we receive from the IMX is passed to the `is_comm_parse` function. For each byte received, this function waits for a complete message in the buffer and decodes it. Once a full message is received, it identifies what kind of message is in the buffer so it can be handled correctly. The easiest way to deal with this is to us a case structure as shown above, with separate "callback" functions for each message type.

The INS message handler is just printing the position in lla, velocity and euler angle attitude to the screen. Other parameterizations of position and attitude are available in other `DID_INS_x` messages.

```
static void handleINSMessage(ins_1_t *ins)
{
    Serial.print("Lat: ");
    Serial.print((float)ins->lla[0], 6);
    Serial.print("\t");
    Serial.print(", Lon: ");
    Serial.print((float)ins->lla[1], 6);
    Serial.print("\t");
    Serial.print(", Alt: ");
    Serial.print((float)ins->lla[2], 2);
    Serial.print("\t");
    Serial.print(", roll: ");
    Serial.print(ins->theta[0] * C_RAD2DEG_F);
    Serial.print("\t");
    Serial.print(", pitch: ");
    Serial.print(ins->theta[1] * C_RAD2DEG_F);
    Serial.print("\t");
    Serial.print(", yaw: ");
    Serial.print("\t");
    Serial.println(ins->theta[2] * C_RAD2DEG_F);
}
```


©2022

6.5.4 Firmware Update (Bootloader) Example Project

This ISBootloaderExample project demonstrates firmware update with the InertialSense products (IMX) using the Inertial Sense SDK.

Files

Project Files

- ISBootloaderExample.cpp

SDK Files

- data_sets.c
- data_sets.h
- ISBootloaderBase.cpp
- ISBootloaderBase.h
- ISBootloaderThread.cpp
- ISBootloaderThread.h
- ISSerialPort.cpp
- ISSerialPort.h
- ISComm.c
- ISComm.h

Implementation

STEP 1: ADD INCLUDES

```
// Change these include paths to the correct paths for your project
#include "../../src/ISComm.h"
#include "../../src/serialPortPlatform.h"
#include "../../src/ISBootloaderThread.h"
#include "../../src/ISBootloaderBase.h"
#include "../../src/ISSerialPort.h"

using namespace ISBootloader;
```

STEP 2: DEFINE PROGRESS AND STATUS CALLBACKS

The bootloader thread reports upload progress, verify progress, and status/log text through three callbacks that you supply:

```
static is_operation_result bootloaderUploadProgress(const std::any& obj, float pct, const std::string& stepName, int stepNo, int totalSteps)
{
    int percent = (int)(pct * 100.0f);
    printf("\rUpload Progress: %d%%\r", percent);
    return IS_OP_OK;
}

static is_operation_result bootloaderVerifyProgress(const std::any& obj, float pct, const std::string& stepName, int stepNo, int totalSteps)
{
    int percent = (int)(pct * 100.0f);
    printf("\rVerify Progress: %d%%\r", percent);
    return IS_OP_OK;
}

static void bootloaderStatusText(const std::any& obj, eLogLevel level, const char* str, ...)
{
    static char buffer[256];
    va_list ap;
    va_start(ap, str);
    vsnprintf(buffer, sizeof(buffer) - 1, str, ap);
    va_end(ap);
    printf("%s\r\n", buffer);
}
```


©2022

STEP 3: DISCOVER CONNECTED DEVICES AND SELECT FIRMWARE FILES

Rather than targeting one specific port directly, the example enumerates every connected serial port and lets the bootloader logic identify which firmware/bootloader file is appropriate for each device it finds. The same firmware file path is assigned to every device-type slot in `firmwares_t`; the bootloader only uses the slot that matches what it actually finds on that port.

```
// Enumerate all connected serial ports
std::vector<std::string> portStrings;
cISSerialPort::GetComPorts(portStrings);

// Assign the same file to every device-type slot; the bootloader identifies
// the connected device and only applies the file to the matching slot(s).
firmwares_t files;
files.fw_uINS_3.path = std::string(argv[2]);
files.bl_uINS_3.path = std::string(argv[2]);
files.fw_IMX_5.path  = std::string(argv[2]);
files.bl_IMX_5.path  = std::string(argv[2]);
files.fw_EVB_2.path  = std::string(argv[2]);
files.bl_EVB_2.path  = std::string(argv[2]);
```

STEP 4: PUT DEVICES INTO BOOTLOADER MODE

```
std::vector<cISBootloaderThread::confirm_bootload_t> confirm_device_list;
if (!cISBootloaderThread::set_mode_and_check_devices(
        portStrings,
        atoi(argv[1]),
        files,
        bootloaderUploadProgress,
        bootloaderVerifyProgress,
        bootloaderStatusText,
        NULL,
        &confirm_device_list))
    return -1;   // Error or no devices found
```

STEP 5: RUN THE FIRMWARE UPDATE

```
std::vector<std::string> all_ports;
cISSerialPort::GetComPorts(all_ports);

cISBootloaderThread::update(
    all_ports,
    true,
    atoi(argv[1]),
    files,
    bootloaderUploadProgress,
    bootloaderVerifyProgress,
    bootloaderStatusText,
    NULL);
```

Note

The example's own usage message (`{COMx} {Baudrate} {Firmware file} {Bootloader file (optional)}`) is out of date relative to what the code actually reads: `argv[1]` is the baud rate and `argv[2]` is the firmware file path, applied to every device-type slot as shown in Step 3 above. There is no separate command-line argument to target one specific COM port — the tool auto-discovers and updates every connected, supported device.

Compile & Run (Linux/Mac)

1. Create build directory

```
cd InertialSenseSDK/ExampleProjects/Bootloader
mkdir build
```

2. Run cmake from within build directory

```
cd build
cmake ..
```

3. Compile using make

```
make
```

4. If necessary, add current user to the "dialout" group in order to read and write to the USB serial communication ports:

©2022

```
sudo usermod -a -G dialout $USER
sudo usermod -a -G plugdev $USER
(reboot computer)
```

5. Run executable (baud rate, then firmware file path)

```
./ISBootloaderExample 921600 IS_uINS-3.hex
```

Compile & Run (Windows Powershell)

*Note - Install CMake for Windows natively, or install the CMake for Windows extension for Visual Studio

1. Create build directory

```
cd InertialSenseSDK/ExampleProjects/Bootloader
mkdir build
```

2. Run cmake from within build directory

```
cd build
cmake ..
```

3. Compile using make

```
cmake --build .
```

4. Run executable (baud rate, then firmware file path)

```
C:\InertialSenseSDK\ExampleProjects\Bootloader\build\Release\ISBootloaderExample.exe 921600 IS_uINS-3.hex
```

Summary

This section has covered the basic functionality you need to set up and communicate with Inertial Sense products. If this doesn't cover everything you need, feel free to reach out to us on the Inertial Sense SDK GitHub repository, and we will be happy to help.

©2022

6.5.5 C++ API - Inertial Sense Class and CLTool Example Project

The InertialSense C++ class, defined in InertialSense.h/.cpp, provides all SDK capabilities including serial communications, data logging to file, and embedded firmware update for InertialSense products.

CLTool Example

The Command Line Tool (CLTool) is an open source project designed to illustrate InertialSense C++ class implementation. The CLTool project can be compiled on most operating systems using cmake and gcc and can be used to communicate, log data, and update firmware for Inertial Sense products. A Visual Studio project for Windows is also included. See Using CLTool for details on compiling and running the CLTool.

IMPLEMENTATION KEYWORDS

The following keywords are found in the CLTool soure code identify the steps for InertialSense class implementation.

```
/* SDK Implementation Keywords:
 * [C++ COMM INSTRUCTION] - C++ binding API, InertialSense class with binary
 * communication protocol and serial port support for Linux and Windows.
 * [LOGGER INSTRUCTION] - Data logger.
 * [BOOTLOADER INSTRUCTION] - Firmware update feature.
 */
```

Serial Communications

STEP 1: INSTANTIATE INERTIALSENSE CLASS

Include the InertialSense header file. Create InertialSense object, passing in the port factories (empty here to use the defaults) and device factories to use.

```
#include "InertialSense.h"

// [C++ COMM INSTRUCTION] 1.) Create InertialSense object, passing in the port and device factories to use.
InertialSense inertialSenseInterface({}, {&CltoolDeviceFactory::getInstance()});
```

STEP 2: OPEN SERIAL PORT

Open the serial port by specifying the com port number, baud rate, and the serial port used for communications

```
if (!inertialSenseInterface.Open(g_commandLineOptions.comPort.c_str(),
   g_commandLineOptions.baudRate,
   g_commandLineOptions.disableBroadcastsOnClose))
{
    cout << "Failed to open serial port at " << g_commandLineOptions.comPort.c_str() << endl;
    return -1; // Failed to open serial port
}
```

STEP 3: ENABLE DATA BROADCASTING

The following enables data broadcasting from the IMX at a specified data rate or period in milliseconds.

```
cltool_setupCommunications(inertialSenseInterface)
```

STEP 4: READ DATA

Call the Update() method at regular intervals to send and receive data.

```
// Main loop. Could be in separate thread if desired.
while (!g_inertialSenseDisplay.ControlCWasPressed())
{
    if (!inertialSenseInterface.Update())
    {
        // device disconnected, exit
        break;
    }
}
```


©2022

STEP 5: HANDLE RECEIVED DATA

New data is delivered through a virtual method on the `ISDevice`-derived device class (`CltoolDevice`) that CLTool registers with its device factory, rather than a free-standing callback function.

```
int CltoolDevice::onIsbDataHandler(p_data_t *data, port_handle_t port)
{
    // Print data to terminal
    g_inertialSenseDisplay.ProcessData(data);

    // uDatasets is a union of all datasets that we can receive. See data_sets.h for a full list of all available datasets.
    uDatasets d = {};
    copyDataPToStructP(&d, data, sizeof(uDatasets));

    // Example of how to access dataset fields.
    switch (data->hdr.id)
    {
    case DID_INS_2:
        d.ins2.qn2b; // quaternion attitude
        d.ins2.uvw; // body velocities
        d.ins2.lla; // latitude, longitude, altitude
        break;
    case DID_INS_1:
        d.ins1.theta; // euler attitude
        d.ins1.lla; // latitude, longitude, altitude
    break;
    case DID_IMU: d.dualImu; break;
    case DID_PIMU: d.dThetaVel; break;
    case DID_GNSS1_POS: d.gpsPos; break;
    case DID_MAGNETOMETER: d.mag; break;
    case DID_BAROMETER: d.baro; break;
    case DID_SYS_SENSORS: d.sysSensors; break;
    }

    return 0;
}
```

STEP 6: CLOSE INTERFACE

Close the interface when your application finishes.

```
// Close cleanly to ensure serial port and logging are shutdown properly. (optional)
inertialSenseInterface.Close();
```

Data Logging

STEP 1: CONFIGURE AND START LOGGING

```
// [LOGGER INSTRUCTION] Setup and start data logger
if (!cltool_setupLogger(inertialSenseInterface))
{
    cout << "Failed to setup logger!" << endl;
    return -1;
}
```

Compile & Run (Linux/Mac)

1. Create build directory

```
cd cltool
mkdir build
```

2. Run cmake from within build directory

```
cd build
cmake ..
```

3. Compile using make

```
make
```

4. If necessary, add current user to the "dialout" group in order to read and write to the USB serial communication ports:

```
sudo usermod -a -G dialout $USER
sudo usermod -a -G plugdev $USER
(reboot computer)
```


©2022

5. Run executable

```
./cltool
```

Compile & Run (Windows Powershell)

*Note - Install CMake for Windows natively, or install the CMake for Windows extension for Visual Studio

1. Create build directory

```
cd InertialSenseSDK/cltool
mkdir build
```

2. Run cmake from within build directory

```
cd build
cmake ..
```

3. Compile using make

```
cmake --build .
```

4. Run executable

```
C:\InertialSenseSDK\cltool\build\Release\cltool.exe
```

Summary

This section has covered the basic functionality you need to set up and communicate with Inertial Sense products. If this doesn't cover everything you need, feel free to reach out to us on the Inertial Sense SDK GitHub repository, and we will be happy to help.

©2022

6.5.6 Data Logging Example Project

This ISLoggerExample project demonstrates data logging with the InertialSense products (IMX) using the Inertial Sense SDK.

Files

Project Files

- ISLoggerExample.cpp

SDK Files

- SDK

Implementation

STEP 1: ADD INCLUDES

```
// Change these include paths to the correct paths for your project
#include "../../src/InertialSense.h"
```

STEP 2: INSTANTIATE INERTIALSENSE CLASS

```
    // InertialSense class wraps communications and logging in a convenient, easy to use class
    InertialSense inertialSense(dataCallback);
    if (!inertialSense.Open(argv[1]))
    {
        std::cout << "Failed to open com port at " << argv[1] << std::endl;
    }
```

STEP 3: ENABLE DATA LOGGER

```
    // get log type from command line
    cISLogger::sSaveOptions options;
    options.logType = (argc < 3 ? cISLogger::LOGTYPE_DAT : cISLogger::ParseLogType(argv[2]));
    inertialSense.EnableLogger(true, "", options);
```

STEP 4: ENABLE DATA BROADCASTING

```
    // broadcast the standard set of post processing messages (ins, imu, etc.)
    inertialSense.BroadcastBinaryDataRmcPreset();

    // instead of the rmc preset (real-time message controller) you can request individual messages...
    // inertialSense.BroadcastBinaryData(DID_IMU, 6); // (startupNavDtMs default)
```

By default, data logs will be stored in the "IS_logs" directory in the current directory.

```
build/IS_logs/LOG_SN30664_20180323_112822_0001.dat
```

Compile & Run (Linux/Mac)

1. Create build directory

```
cd InertialSenseSDK/ExampleProjects/InertialSense_logger
mkdir build
```

2. Run cmake from within build directory

```
cd build
cmake ..
```

3. Compile using make

```
make
```

4. If necessary, add current user to the "dialout" group in order to read and write to the USB serial communication ports:

©2022

```
sudo usermod -a -G dialout $USER
sudo usermod -a -G plugdev $USER
(reboot computer)
```

5. Run executable

```
./ISLoggerExample /dev/ttyUSB0
```

Compile & Run (Windows Powershell)

*Note - Install CMake for Windows natively, or install the CMake for Windows extension for Visual Studio

1. Create build directory

```
cd InertialSenseSDK/ExampleProjects/InertialSense_logger
mkdir build
```

2. Run cmake from within build directory

```
cd build
cmake ..
```

3. Compile using make

```
cmake --build .
```

4. Run executable

```
C:\InertialSenseSDK\ExampleProjects\Logger\build\Release\ISLoggerExample.exe COM3
```

Summary

This section has covered the basic functionality you need to set up and communicate with Inertial Sense products. If this doesn't cover everything you need, feel free to reach out to us on the Inertial Sense SDK GitHub repository, and we will be happy to help.

©2022

7. IMX

7.1 Allan Deviation / Allan Variance (ADEV/AVAR)

Allan Deviation (ADEV), the square root of Allan Variance (AVAR), is the industry-standard statistical method for characterizing inertial sensor noise as a function of averaging time. Unlike a single RMS noise specification, Allan Deviation separates the various stochastic error processes present in MEMS gyroscopes and accelerometers, allowing white noise, bias instability, random walk, and long-term drift to be independently quantified.

The Allan Deviation plots below were generated from static IMU data collected under controlled laboratory conditions. Each point represents the expected sensor uncertainty after averaging measurements over the corresponding integration time.

7.1.1 Test Conditions

- Sample Rate: **1000 Hz**
- Test Duration: **24 hours**
- Environment: **Static, vibration-free**
- Ambient Temperature: **25 °C**

7.1.2 IMX-6 IMU Performance

Parameter	Value
Gyro Bias Instability	**0.9 °/hr**
Gyro Angle Random Walk (ARW)	**0.10 °/√hr**
Accelerometer Bias Instability	**4.0 µg**
Accelerometer Velocity Random Walk (VRW)	**0.011 m/s/√hr**

Bias Instability values are reported using the IEEE Std 952-1997 convention and are computed as the minimum Allan Deviation divided by **0.664**, yielding the bias instability coefficient attributable to flicker (1/f) noise. This value is sometimes loosely referred to as the 1σ bias instability, though strictly speaking it is a power-spectral-density-derived coefficient rather than a standard deviation of a Gaussian process.

7.1.3 Interpreting Allan Deviation

Different stochastic error sources appear as characteristic slopes on the log-log Allan Deviation plot.

Slope	Dominant Noise Process	Typical Effect
−½	White Noise (ARW / VRW)	Short-term measurement noise
0	Bias Instability	Minimum achievable bias uncertainty
+½	Rate Random Walk	Slowly varying bias drift
+1	Rate Ramp	Very low-frequency systematic drift

At short averaging times, white sensor noise dominates, producing the characteristic **−½** slope. As averaging time increases, random noise is reduced through averaging until the Allan Deviation reaches its minimum, corresponding to the sensor's bias instability. For longer averaging times, low-frequency drift mechanisms become dominant, causing the Allan Deviation to increase.

©2022

7.1.4 Gyroscope

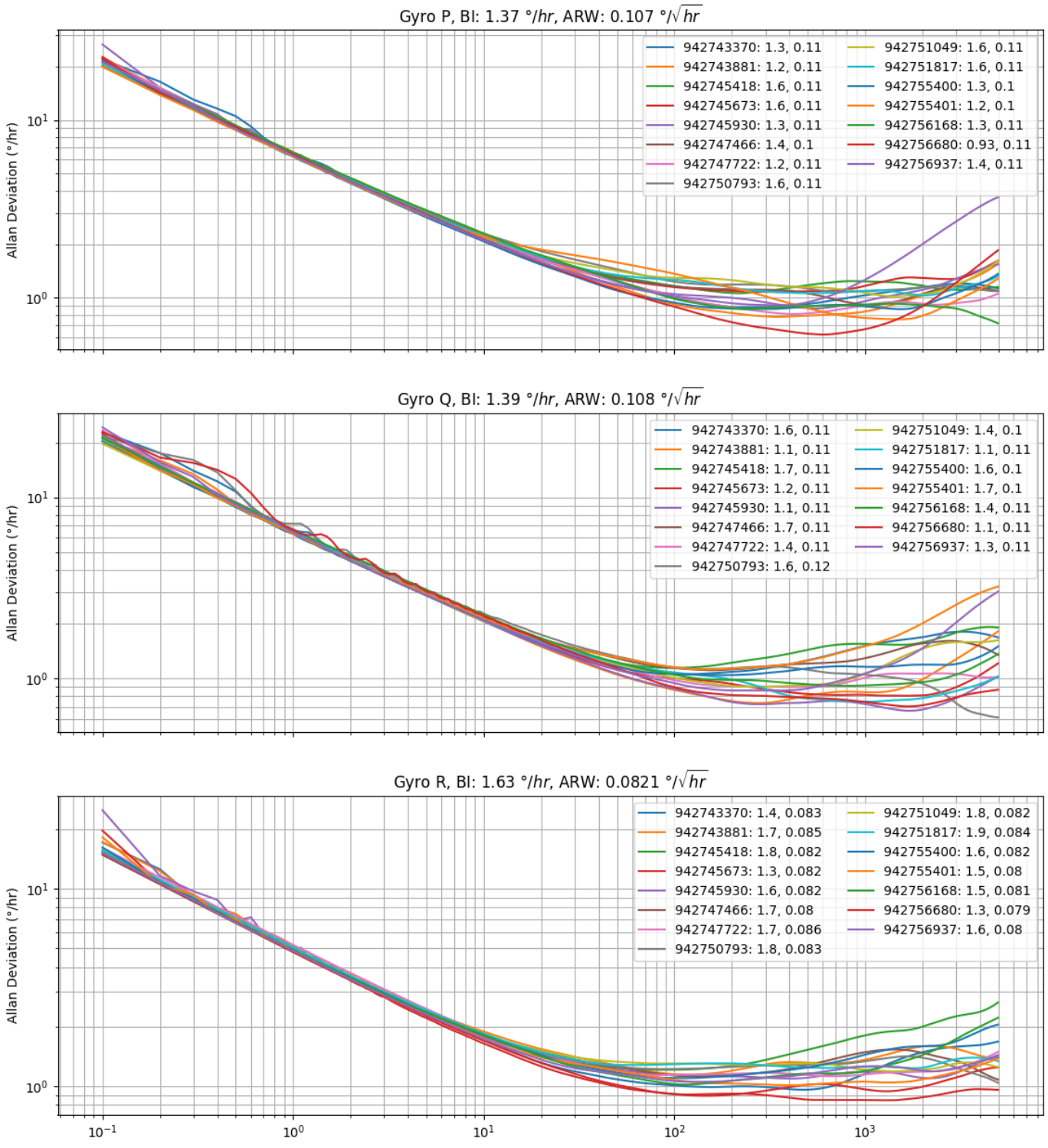
The gyroscope Allan Deviation plot is used to characterize both short-term angular rate noise and long-term bias stability.

- **Angle Random Walk (ARW)** quantifies the white angular-rate noise present at short averaging times and determines the rate at which attitude error accumulates during inertial integration.
- **Bias Instability** represents the minimum achievable bias uncertainty resulting primarily from flicker (1/f) noise.
- Positive-slope regions identify lower-frequency stochastic processes such as rate random walk and rate ramp.

The IMX-6 gyroscope exhibits an Angle Random Walk of **0.10 °/√hr** and a Bias Instability of **0.95 °/hr**, providing excellent short-term stability while maintaining exceptionally low long-term drift.

©2022

PQR Allan Dev. : 20260416_151931_spec, **BI :** 1.46 °/*hr*, **ARW :** 0.0992 °/$\sqrt{hr}$



©2022

7.1.5 Accelerometer

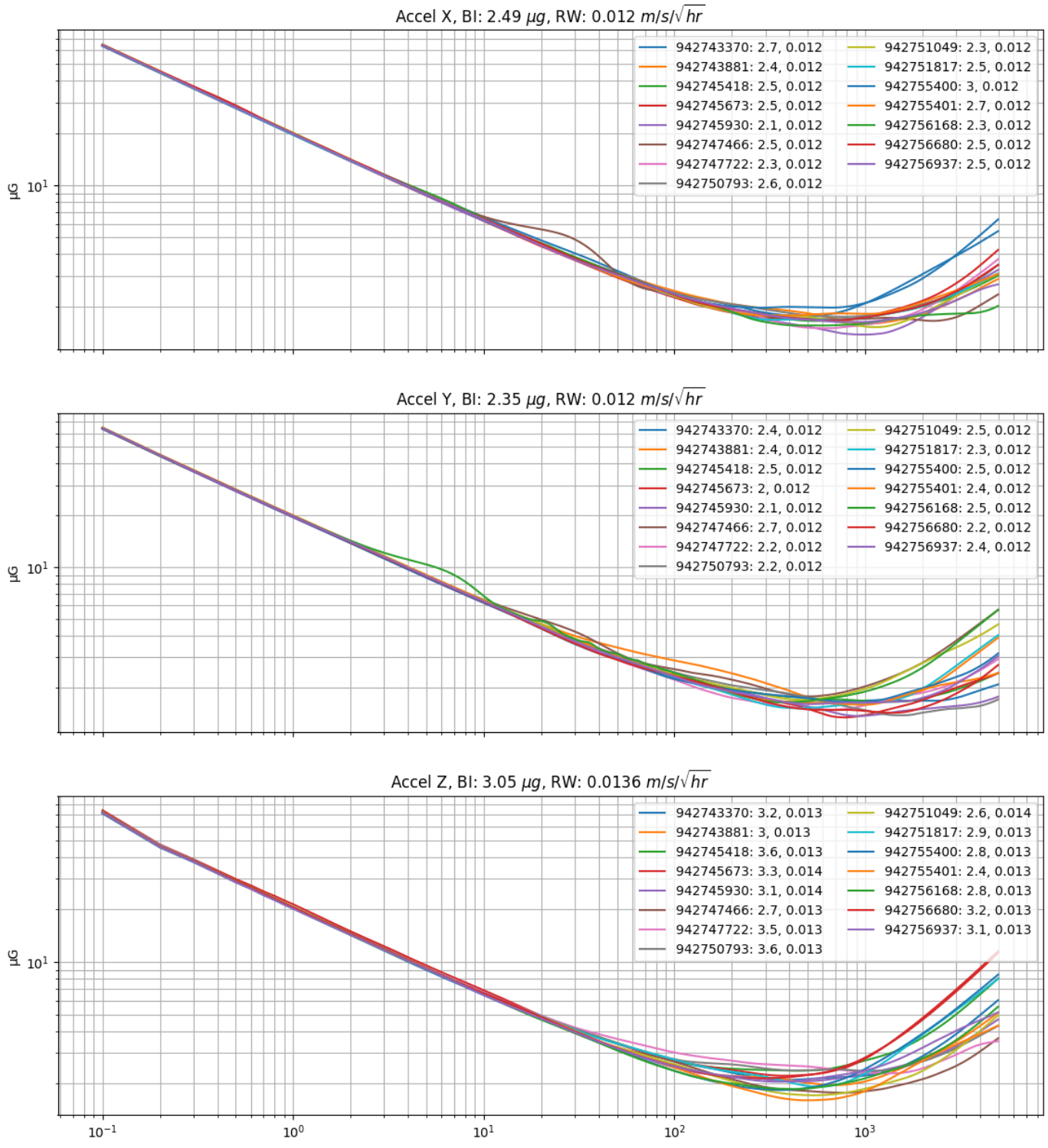
The accelerometer Allan Deviation plot characterizes both short-term acceleration noise and long-term bias stability.

- **Velocity Random Walk (VRW)** quantifies the white acceleration noise and determines velocity error accumulation during inertial integration.
- **Bias Instability** represents the minimum achievable accelerometer bias uncertainty.
- Positive-slope regions indicate long-term stochastic drift mechanisms that become significant over extended averaging periods.

The IMX-6 accelerometer exhibits a Velocity Random Walk of **0.011 m/s/√hr** and a Bias Instability of **4.0 µg**, providing excellent performance for precision inertial navigation and long-duration dead reckoning.

©2022

Accel Allan Dev. : 20260416_151931_spec, **BI :** 2.63 μg, **VRW :** 0.0132 $m/s/\sqrt{hr}$



©2022

7.2 External Aiding

7.2.1 Overview

External aiding lets a host computer or an external sensor feed observations directly into the IMX EKF, independent of any GNSS receiver. This is useful when the platform has a sensor the IMX cannot otherwise see — for example a second INS, a vision or SLAM system, an alternate GNSS receiver, an airspeed/pitot sensor, a compass, or an RTK correction service running on the host.

Seven data sets are available:

DID	Struct	Observation	Fused by the EKF
`DID_EXT_AIDING_POS`	`ext_aiding_pos_t`	Position	Yes
`DID_EXT_AIDING_VEL`	`ext_aiding_vel_t`	Velocity	Yes
`DID_EXT_AIDING_SPEED`	`ext_aiding_speed_t`	Scalar speed (3D or horizontal)	Yes
`DID_EXT_AIDING_DIR_SPEED`	`ext_aiding_dir_speed_t`	Directional speed (e.g. airspeed)	Yes
`DID_EXT_AIDING_HEADING`	`ext_aiding_heading_t`	Heading (true, magnetic, or course)	Yes
`DID_EXT_AIDING_ATTITUDE`	`ext_aiding_attitude_t`	Full attitude (euler or quaternion)	Yes
`DID_EXT_IMU`	`imu_t`	Gyro/accel	No — see below

All seven are independent — supply whichever ones match the sensors you have. Each carries its own GPS time of week and a discard-on-invalid observation variance (or, for attitude, a full covariance matrix).

> **Important**
>
> A non-positive variance (or, for `DID_EXT_AIDING_ATTITUDE`, a non-positive covariance diagonal) causes the entire observation to be discarded by the EKF. Variance must always reflect a realistic (non-zero) uncertainty of the measurement — do not use zero to mean "perfect" or "unknown."

7.2.2 Position and Velocity

```
/** Frame of measurement for an external aiding observation, held in the low nibble of
 *  ext_aiding_pos_t.status / ext_aiding_vel_t.status. Note 0 is not a valid frame. */
enum eExtAidingFrame
{
    EXT_AIDING_FRAME_MASK    = 0x0000000F,  // Mask for the frame of measurement
    EXT_AIDING_FRAME_ECEF    = 1,           // ECEF frame
    EXT_AIDING_FRAME_NED     = 2,           // NED frame
    EXT_AIDING_FRAME_BODY    = 3            // Body frame
};

/** (DID_EXT_AIDING_POS) External aiding position observation, supplied by a host or external
 *  sensor as an input to the INS/EKF. Position is expected in ECEF; var is expected in NED. */
typedef struct PACKED
{
    uint32_t   timeOfWeekMs; // GPS time of week (since Sunday morning) in milliseconds
    uint32_t   status;       // Status bitfield; set (status & EXT_AIDING_FRAME_MASK) to frame: 1=ECEF, 2=NED, 3=Body (see eExtAidingFrame)
    double     pos[3];       // position {x,y,z} (m)
    float      offset[3];    // point of measurement relative to IMU origin in IMU/body frame {x,y,z} (m)
    float      var[3];       // observation variance, per axis, in NED (m^2). Must be non-zero or the observation is discarded.
} ext_aiding_pos_t;

/** (DID_EXT_AIDING_VEL) External aiding velocity observation, supplied by a host or external
 *  sensor as an input to the INS/EKF. Velocity is expected in ECEF; var is expected in NED. */
typedef struct PACKED
{
    uint32_t   timeOfWeekMs; // GPS time of week (since Sunday morning) in milliseconds
    uint32_t   status;       // Status bitfield; set (status & EXT_AIDING_FRAME_MASK) to frame: 1=ECEF, 2=NED, 3=Body (see eExtAidingFrame)
    float      vel[3];       // velocity {vx,vy,vz} (m/s)
    float      offset[3];    // point of measurement relative to IMU origin in IMU/body frame {x,y,z} (m)
```


©2022

```
    float      var[3];       // observation variance, per axis, in NED (m^2/s^2). Must be non-zero or the observation is discarded.
} ext_aiding_vel_t;
```

- **`timeOfWeekMs`** — GPS time of week the observation was taken. Used by the EKF to align the observation against its own delayed-state history. If the observation is more than 200 ms old relative to the IMX's current IMU time, its timestamp is clamped and the fix loses some of its benefit — keep aiding latency well under this.
- **`status`** — the coordinate frame the measurement was taken in (see `eExtAidingFrame`); `0` is invalid and causes the observation to be rejected. `vel` fully honors this — `ECEF`, `NED`, and `BODY` are all correctly rotated into the NED innovation. `pos` currently only checks that `status` is non-zero; the position itself is always processed as if it were ECEF, so flagging `pos` as `NED` or `BODY` will *not* error but will silently produce a wrong result — always send `pos` in ECEF today. `var` is always expressed in NED regardless of `status`.
- **`offset`** — the lever arm from the IMU origin to the point of measurement (e.g. the antenna phase center of an external GNSS receiver), expressed in the IMU/body frame. Set to `[0,0,0]` if the source measures at the IMU origin.
- **`var`** — per-axis observation variance in NED; report your source's actual uncertainty, since this is what the EKF weights the observation by and what its outlier (NIS) gate checks the innovation against. In dynamic environments, inflate it to also cover the error that `timeOfWeekMs` timestamp uncertainty induces from vehicle motion — for a position observation this is timestamp uncertainty × speed, and for a velocity observation it's timestamp uncertainty × acceleration — not just the sensor's own noise.

7.2.3 Speed

`DID_EXT_AIDING_SPEED` carries a scalar speed with no direction — for example a wheel-speed sensor or a GNSS-derived ground speed that doesn't expose a velocity vector.

```
enum eExtAidingSpeedType
{
    EXT_AIDING_SPEED_TYPE_MASK       = 0x0000000F,
    EXT_AIDING_SPEED_TYPE_3D         = 1,  // full 3D velocity magnitude, |v|
    EXT_AIDING_SPEED_TYPE_HORIZONTAL = 2   // horizontal (ground) velocity magnitude only, |v_NE|
};

/** (DID_EXT_AIDING_SPEED) External aiding scalar speed observation. */
typedef struct PACKED
{
    uint32_t   timeOfWeekMs; // GPS time of week (since Sunday morning) in milliseconds
    uint32_t   status;       // Speed type, 1=3D magnitude, 2=horizontal magnitude (see eExtAidingSpeedType)
    float      speed;        // speed (m/s)
    float      var;          // observation variance (m^2/s^2). Must be positive or the observation is discarded.
    float      offset[3];    // point of measurement relative to IMU origin in IMU/body frame {x,y,z} (m)
} ext_aiding_speed_t;
```

7.2.4 Directional Speed (Airspeed)

`DID_EXT_AIDING_DIR_SPEED` carries a speed measured along a fixed direction in the IMU/body frame — the natural fit for an airspeed/pitot sensor, which measures flow along the direction it's pointed (typically the body X axis).

```
/** (DID_EXT_AIDING_DIR_SPEED) External aiding directional speed observation. `direction` is a
 *  unit vector in the IMU/body frame (e.g. [1,0,0] for a forward-pointing airspeed sensor);
 *  `speed` is the velocity component along that direction at the point of measurement. */
typedef struct PACKED
{
    uint32_t   timeOfWeekMs; // GPS time of week (since Sunday morning) in milliseconds
    uint32_t   status;       // reserved, set to 0
    float      speed;        // speed along `direction` (m/s)
    float      var;          // observation variance (m^2/s^2). Must be non-zero or the observation is discarded.
    float      offset[3];    // point of measurement relative to IMU origin in IMU/body frame {x,y,z} (m)
    float      direction[3]; // unit vector, in IMU/body frame, along which `speed` is measured
} ext_aiding_dir_speed_t;
```

7.2.5 Heading

`DID_EXT_AIDING_HEADING` carries a single heading angle, in one of three senses:

```
enum eExtAidingHeadingType
{
    EXT_AIDING_HEADING_TYPE_MASK     = 0x0000000F,
    EXT_AIDING_HEADING_TYPE_TRUE     = 1, // true heading (body X axis bearing relative to true north)
    EXT_AIDING_HEADING_TYPE_MAGNETIC = 2, // magnetic heading (converted to true using the EKF's declination estimate)
    EXT_AIDING_HEADING_TYPE_COURSE   = 3  // course over ground (direction of travel — not necessarily the same as body heading)
```


©2022

```
};

/** (DID_EXT_AIDING_HEADING) External aiding heading observation. */
typedef struct PACKED
{
    uint32_t   timeOfWeekMs; // GPS time of week (since Sunday morning) in milliseconds
    uint32_t   status;       // Heading type, 1=true, 2=magnetic, 3=course over ground (see eExtAidingHeadingType)
    float      heading;      // heading (rad), 0 = north, positive clockwise, range [-pi, pi]
    float      var;          // observation variance (rad^2). Must be non-zero or the observation is discarded.
} ext_aiding_heading_t;
```

Note

`TRUE` / `MAGNETIC` heading are treated as an attitude observation (the bearing the vehicle is physically pointed). `COURSE` (over ground) is treated as a velocity-direction observation instead — the direction the vehicle is actually moving, which can differ from body heading under sideslip, wind, or current. Don't use `COURSE` in place of `TRUE` / `MAGNETIC` if what you actually have is a compass.

7.2.6 Attitude

`DID_EXT_AIDING_ATTITUDE` carries a full 3-axis attitude, from a source such as a second INS, with a full 3x3 attitude-error covariance rather than a per-axis variance.

```
enum eExtAidingAttitudeType
{
    EXT_AIDING_ATTITUDE_TYPE_MASK       = 0x0000000F,
    EXT_AIDING_ATTITUDE_TYPE_EULER      = 1, // att = {roll, pitch, yaw} (rad); att[3] unused
    EXT_AIDING_ATTITUDE_TYPE_QUATERNION = 2  // att = {w, x, y, z}
};

/** (DID_EXT_AIDING_ATTITUDE) External aiding attitude observation. `var` is the 3x3
 *  attitude-error covariance (row-major), expressed as a small-angle roll/pitch/yaw rotation
 *  vector regardless of whether `att` is euler or quaternion. */
typedef struct PACKED
{
    uint32_t   timeOfWeekMs; // GPS time of week (since Sunday morning) in milliseconds
    uint32_t   status;       // Attitude representation, 1=euler, 2=quaternion (see eExtAidingAttitudeType)
    float      att[4];       // attitude, interpreted per `status`: euler {roll,pitch,yaw,-} or quaternion {w,x,y,z}
    float      var[9];       // 3x3 row-major attitude-error covariance (rad^2). Must have a non-zero diagonal or the observation is discarded.
} ext_aiding_attitude_t;
```

7.2.7 DID_EXT_IMU

`DID_EXT_IMU` carries a raw gyro/accel sample from an external IMU. It reuses the SDK's existing `imu_t` — the same struct as `DID_IMU` / `DID_IMU_RAW` / `DID_REFERENCE_IMU` — rather than a bespoke type. Unlike the other six types, **it is not currently fused by the EKF** — the DID exists so an external IMU can be streamed to and logged by the IMX (e.g. for offline analysis or a future consistency-check/blended time-update path), but it has no effect on the navigation solution today.

```
/** (DID_IMU, DID_IMU_RAW, DID_REFERENCE_IMU, DID_EXT_IMU) Single combined Inertial Measurement
 *  Unit (IMU) sample, in body/sensor frame. */
typedef struct PACKED
{
    double      time;    // Time since boot up in seconds. Convert to GPS time of week by adding gps.towOffset
    uint32_t    status;  // IMU status flags (eImuStatus)
    imui_t      I;       // Combined IMU sample: angular rate and acceleration
} imu_t;

/** Single IMU sample: angular rate and acceleration, in the sensor/body frame. */
typedef struct PACKED
{
    float       pqr[3];  // Gyroscope P, Q, R (angular rate about body X, Y, Z) in radians/second
    float       acc[3];  // Acceleration X, Y, Z in meters/second^2, in body frame
} imui_t;
```

Because `DID_EXT_IMU` reuses `imu_t` as-is, its `time` field is seconds since boot (matching `DID_IMU`'s convention), not the GPS time-of-week-in-milliseconds convention the other six external aiding types use — convert using the same `gps.towOffset` relationship other IMU DIDs use if you need GPS time.

Note

`DID_EXT_IMU` also has no relay/logging RMC bit (unlike the six fused types below), so it can't be relayed to another port for logging the way the others can — poll or stream it directly from the port it's written to.

©2022

7.2.8 Sending Data to the IMX

All seven DIDs are writable. A host streams an observation to the IMX using the SDK's Set Data method on any connected port, the same as any other writable DID. Each write is consumed on arrival — there is no need to hold a channel open or stream continuously; send an observation whenever a new one is available from your source.

For bring-up or quick testing, an observation can be injected directly with the CLTool:

```
cltool -c /dev/ttyUSB0 -set "{DID_EXT_AIDING_POS: {status: 1, timeOfWeekMs: <tow>, pos: [<x>, <y>, <z>], offset: [0,0,0], var: [4.0, 4.0, 9.0]}}"

cltool -c /dev/ttyUSB0 -set "{DID_EXT_AIDING_VEL: {status: 1, timeOfWeekMs: <tow>, vel: [<vx>, <vy>, <vz>], offset: [0,0,0], var: [0.25, 0.25, 0.25]}}"

cltool -c /dev/ttyUSB0 -set "{DID_EXT_AIDING_SPEED: {status: 1, timeOfWeekMs: <tow>, speed: <s>, offset: [0,0,0], var: 0.25}}"

cltool -c /dev/ttyUSB0 -set "{DID_EXT_AIDING_DIR_SPEED: {status: 0, timeOfWeekMs: <tow>, speed: <s>, offset: [0,0,0], direction: [1,0,0], var: 0.25}}"

cltool -c /dev/ttyUSB0 -set "{DID_EXT_AIDING_HEADING: {status: 1, timeOfWeekMs: <tow>, heading: <rad>, var: 0.01}}"

cltool -c /dev/ttyUSB0 -set "{DID_EXT_AIDING_ATTITUDE: {status: 1, timeOfWeekMs: <tow>, att: [<roll>,<pitch>,<yaw>,0], var: [0.01,0,0, 0,0.01,0, 0,0,0.01]}}"
```

7.2.9 Logging Received Observations

Whatever external aiding observations the IMX receives — from a host or from a GPX-1 (see below) — are relayed unmodified to any other port that has the corresponding RMC bit enabled, so the aiding data supplied to the IMX can be logged or inspected on a host PC:

RMC Bit	Relays
`RMC_BITS_EXT_AIDING_POS`	`DID_EXT_AIDING_POS`
`RMC_BITS_EXT_AIDING_VEL`	`DID_EXT_AIDING_VEL`
`RMC_BITS_EXT_AIDING_SPEED`	`DID_EXT_AIDING_SPEED`
`RMC_BITS_EXT_AIDING_DIR_SPEED`	`DID_EXT_AIDING_DIR_SPEED`
`RMC_BITS_EXT_AIDING_HEADING`	`DID_EXT_AIDING_HEADING`
`RMC_BITS_EXT_AIDING_ATTITUDE`	`DID_EXT_AIDING_ATTITUDE`

`DID_EXT_IMU` has no RMC bit and is not relayed (see above). All six bits above are included in the IMX PPD presets by default. All seven DIDs appear in the EvalTool's **Data Sets** and **Logger** tabs.

> **Note**
>
> Enabling these bits on the IMX only controls whether received observations are relayed back out for logging — it has no effect on whether the EKF consumes them. The EKF always fuses whatever valid external aiding data it receives on any port (except `DID_EXT_IMU`, which it doesn't consume at all today).

7.2.10 Using a GPX-1 as an External Aiding Source

The GPX-1 can restate its own GNSS1 position/velocity solution as external aiding observations and stream them to a connected IMX, providing a ready-made source for exercising the IMX's external aiding path without a separate host integration.

This is enabled per output port with two opt-in GRMC bits, set on `DID_GPX_RMC.bits` the same way as any other GPX RMC bit (see GPX-1 Quick Start):

GRMC Bit	Emits
`GRMC_BITS_EXT_AIDING_POS`	`DID_EXT_AIDING_POS`
`GRMC_BITS_EXT_AIDING_VEL`	`DID_EXT_AIDING_VEL`

©2022

Behavior:

- Observations are emitted once per GNSS1 epoch, alongside `DID_GNSS1_POS`. GNSS2 is not supported — it is the compassing/moving-base receiver and does not carry a navigation solution.
- The fix must be at least 3D-class (3D, GNSS+dead-reckoning, DGPS, SBAS, or any RTK fix level). 2D, dead-reckoning-only, time-only, and surveyed/reference-position fixes are excluded.
- `var` is derived from the receiver's reported `hAcc` / `vAcc` / `sAcc`, floored at 0.01 m (position) / 0.01 m/s (velocity) so an optimistic or unreported accuracy is never presented as noise-free.
- `offset` is taken from `gnss1AntOffset` in the GPX's flash configuration.

Important

These bits are deliberately excluded from `GRMC_PRESET_GPX_IMX`. If the IMX is already using this same GPX-1 as its GNSS receiver, also enabling external aiding from it would fuse the same solution into the EKF twice. Only enable these bits when the GPX-1's GNSS solution is being used to aid an *independent* IMX, or for bench testing the external aiding path itself.

7.2.11 Related Change Log Entries

- (IMX/SDK) SN-8317, SN-8318 — `DID_EXT_AIDING_POS` / `DID_EXT_AIDING_VEL` added.
- (GPX-1) SN-8472 — GPX-1 restates its GNSS1 solution as external aiding observations.
- (IMX/EvalTool) SN-8317 — external aiding relay/logging support.
- (IMX/SDK) SN-8318 — `DID_EXT_AIDING_SPEED` / `DID_EXT_AIDING_DIR_SPEED` / `DID_EXT_AIDING_HEADING` / `DID_EXT_AIDING_ATTITUDE` / `DID_EXT_IMU` added.

©2022

7.3 GNSS Aiding

7.3.1 GNSS Aiding IMX

The IMX INS/EKF can be aided by one or more externally connected GNSS receivers. The IMX interfaces with a variety of GNSS manufacturers and protocols, so the receiver hardware can be chosen to fit the application, while the IMX handles the details of communicating with it and fusing its position, velocity, and (optionally) dual-antenna heading data into the EKF solution.

Supported GNSS Manufacturers and Protocols

Manufacturer / Protocol	GNSS Type	Description	Guide
Inertial Sense	GPX-1	Inertial Sense's own multi-band GNSS/RTK receiver module, interfaced over Inertial Sense Binary (ISB).	GPX-1
u-blox	ublox	u-blox multi-band GNSS/RTK receivers (ZED-F9P, ZED-X20), interfaced using the u-blox binary protocol.	u-blox
Septentrio	Septentrio	Septentrio multi-band GNSS/RTK receivers (e.g. Mosaic-G5), interfaced using Septentrio Binary Format (SBF).	Septentrio
Any NMEA-compliant receiver	NMEA	Any GNSS receiver that outputs standard NMEA ASCII sentences (GNS/GGA, ZDA, RMC, GSA).	NMEA
Inertial Sense Binary (ISB)	ISB	Any device capable of generating Inertial Sense Binary GNSS DID messages directly — another Inertial Sense product, a companion computer, or a custom host.	see below

GNSS Antenna Offset

If the setup includes a significant distance (40cm or more) between the GNSS antenna and the IMX central unit, enter a non-zero value for the GNSS antenna lever arm, `DID_FLASH_CONFIG.gnss1AntOffset` (or `DID_FLASH_CONFIG.gnss2AntOffset`) X,Y,Z offset in meters from the Sensor Frame origin to the GNSS antenna. The Sensor Frame origin and Hardware Frame origin are always at the same location but may differ in direction according to the Sensor Rotation.

INS-GNSS Dynamic Model

The `DID_FLASH_CONFIG.dynamicModel` setting allows the user to adjust how the EKF and GNSS receivers behave in different dynamic environments. This parameter is applied to the GNSS receiver(s) engine as well as the INS EKF.

©2022

7.3.2 Inertial Sense Binary (ISB) Aiding IMX

The IMX INS/EKF can be aided directly with GNSS position and velocity data supplied as Inertial Sense Binary (ISB) messages from an external device, such as another Inertial Sense product, a companion computer, or a custom host generating GNSS solutions. Because the data arrives already formatted as native IMX Data Identifiers (DIDs), the IMX does not need to parse a third-party GNSS receiver protocol (uBlox, NMEA, Septentrio, etc.) — it simply accepts and fuses the incoming DID messages into the EKF.

ISB GNSS aiding data can be supplied over any of the IMX serial ports (Serial 0, Serial 1, or Serial 2). The `DID_FLASH_CONFIG.ioConfig` GNSS1 and/or GNSS2 source fields must be set to the corresponding serial port so the IMX knows which port to use for GNSS1 and GNSS2.

This is the same mechanism the GPX-1 uses internally to supply GNSS1 and GNSS2 data to the IMX-5. When `DID_FLASH_CONFIG.ioConfig` GNSS type is set to **GPX**, the IMX-5 requests this exact set of GNSS DID messages from the GPX-1 over ISB. Setting the GNSS type to **ISB** allows any external source capable of generating these same DID messages to aid the INS in the same way.

Configure the IMX for ISB GNSS Input

1. Connect the external ISB GNSS source to Serial 0, Serial 1, or Serial 2 (3.3V TTL UART) on the IMX. See the PCB Module hardware page for pinout.
2. Set the serial port baudrate to match `DID_FLASH_CONFIG.serXBaudRate` for the port used.
3. Configure GNSS1 (and GNSS2, if using dual antenna heading) using the EvalTool GPS Setting tab or `DID_FLASH_CONFIG.ioConfig`:

DID_FLASH_CONFIG	Value
GNSS1 Source	Serial 0, Serial 1, or Serial 2
GNSS1 Type	**ISB** (Inertial Sense Binary)
GNSS2 Source	Serial 0, Serial 1, or Serial 2 (optional, for dual antenna heading)
GNSS2 Type	**ISB** (Inertial Sense Binary)

4. Stream the required DID messages (below) from the external device to the IMX at the desired GNSS update rate. A 200ms (5Hz) period is typical and matches `DID_FLASH_CONFIG.startupGnssDtMs`.
5. Ensure the `status` field of each `DID_GNSS1_POS` / `DID_GNSS1_VEL` / `DID_GNSS2_POS` / `DID_GNSS2_VEL` message accurately reports fix type, satellites used, and accuracy (see `eGnssStatus`). The EKF uses these status flags to determine whether the data is trustworthy enough to fuse.

Required GNSS DID Messages

The following DID messages are needed to properly aid the INS, matching the same set the GPX-1 provides to the IMX-5. Full field definitions for each DID are available in the DID Descriptions reference.

GNSS1 (REQUIRED)

DID	Description
`DID_GNSS1_POS`	GPS 1 position data. Used directly by the EKF for INS position aiding.
`DID_GNSS1_VEL`	GPS 1 velocity data. Used directly by the EKF for INS velocity aiding.

©2022

GNSS2 (REQUIRED ONLY IF USING A SECOND ANTENNA FOR HEADING/RTK COMPASSING)

DID	Description
`DID_GNSS2_POS`	GPS 2 position data.
`DID_GNSS2_VEL`	GPS 2 velocity data.
`DID_GNSS2_RTK_CMP_REL`	Dual GNSS RTK compassing relative info (baseline vector/heading between GNSS1 and GNSS2 antennas). Used by the EKF to aid INS heading.

SUPPLEMENTARY (OPTIONAL, RECOMMENDED FOR FULL PARITY WITH THE GPX-1)

These messages are not required for basic position/velocity/heading aiding, but provide status, diagnostic, and accuracy information used by the EvalTool and logging tools.

DID	Description
`DID_GNSS1_SAT` / `DID_GNSS2_SAT`	Satellite information: identifiers, CNO, elevation/azimuth.
`DID_GNSS1_SIG` / `DID_GNSS2_SIG`	GNSS signal information.
`DID_GNSS1_VERSION` / `DID_GNSS2_VERSION`	GNSS receiver software/hardware version info.
`DID_GNSS1_RTK_POS_REL` / `DID_GNSS1_RTK_POS_MISC`	RTK precision positioning relative and accuracy/diagnostic info for GNSS1.
`DID_GNSS2_RTK_CMP_MISC`	RTK compassing accuracy/diagnostic info for GNSS2.

Electrical Interface

The external ISB GNSS source can be connected to Serial 0, Serial 1, and Serial 2 (3.3V TTL UART) on the IMX.

©2022

7.3.3 GPX GNSS Aiding IMX

The IMX can be configured for use with the Inertial Sense GPX-1 multi-band GNSS receiver. This can be done using either the EvalTool GPS Setting tab or the IMX `DID_FLASH_CONFIG.ioConfig` and `DID_FLASH_CONFIG.RTKCfgBits` fields.

GPS Ports	**Value**
GPS Source	serial 0, serial 1, or serial 2
GPS Type	GPX-1
GPS1 Timepulse	*Disable* or IMX pin connected to GPX-1 PPS

RTK Rover	**Value**
GPS RTK Mode	Precision Position or Compass

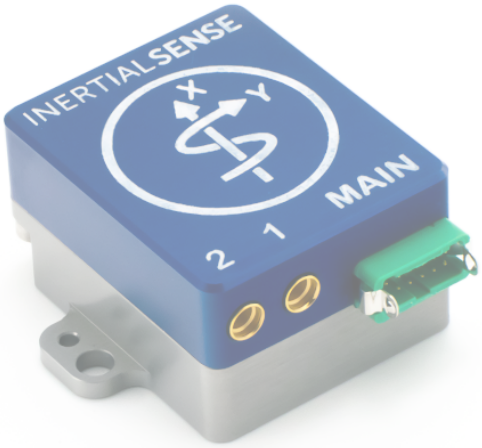
RTK Base	**Value**
Serial Port 0 (Single GNSS only)	GNSS1 - RTCM3
USB Port	GNSS1 - RTCM3

The following sections detail how to interface and configure the IMX for operation using the GPX-1. See RTK precision positioning and RTK compassing for RTK operation principles.

TYPICAL INTERFACE

The IMX will automatically configure the GPX-1 for communications.

RUGGED-4 (COMING SOON)



The Rugged-4 INS contains the GPX-1 onboard supporting RTK positioning and compassing. GPS 1 and GPS 2 are connected to serial port 0 on the IMX-5.

©2022

The following is a list of compatible antenna options for GPX-1 (L1/L5) installations.

©2022

Item

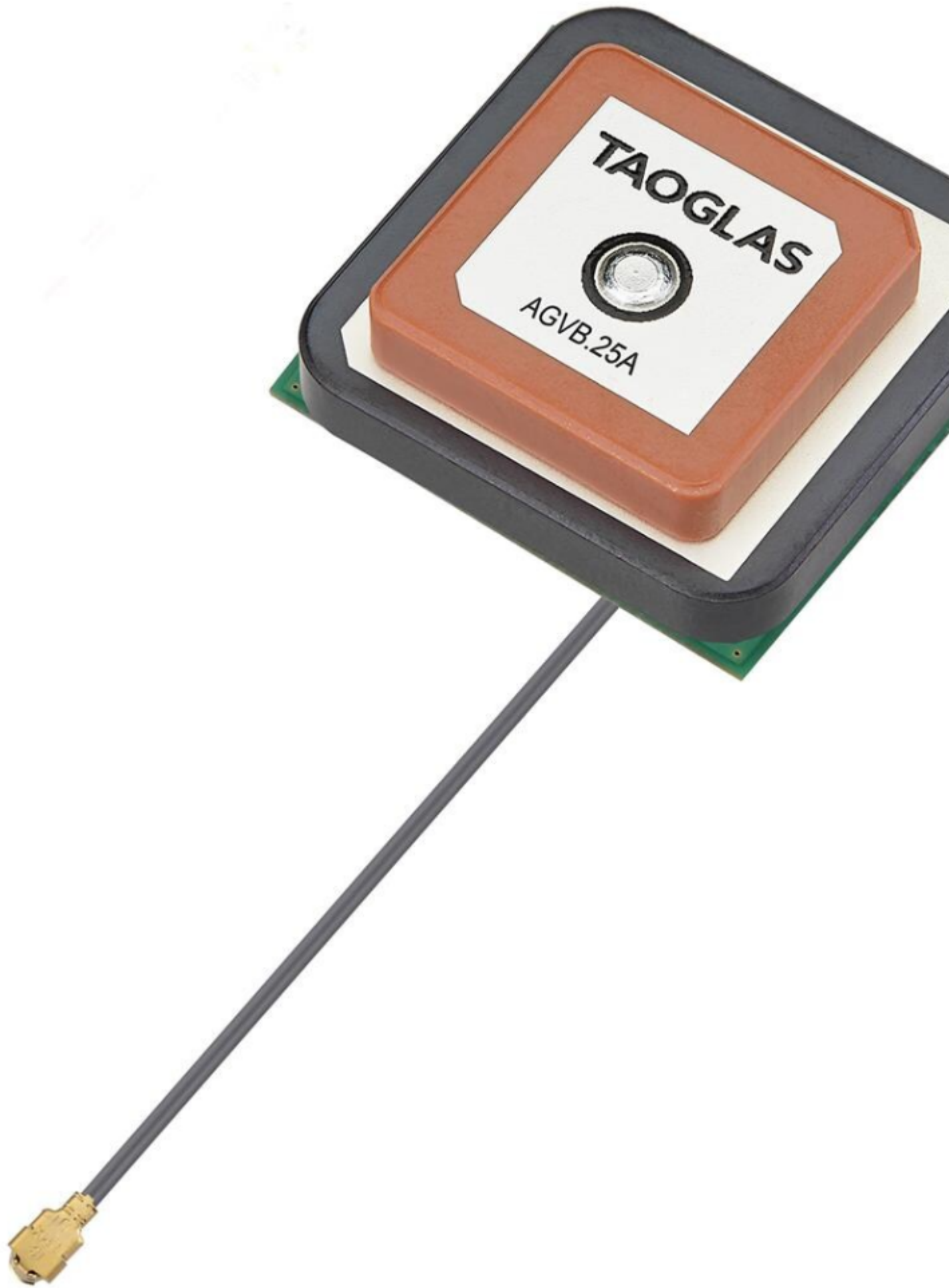
©2022

Item

©2022

Item

©2022



©2022

Item

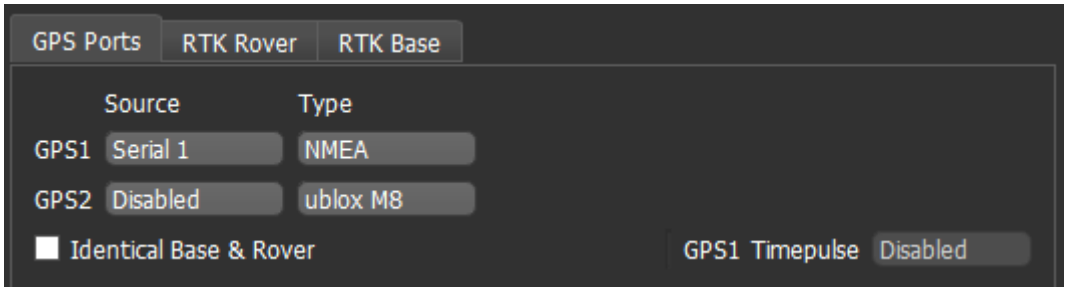
©2022

7.3.4 NMEA GNSS Aiding IMX

GNSS receivers that output NMEA ascii protocol can be used to aid the IMX INS.

Configure IMX for NMEA GNSS Input

1. Set serial port baudrate, matching DID_FLASH_CONFIG.serXBaudRate.
2. Configure GNSS1 using EvalTool GNSS Setting tab or the DID_FLASH_CONFIG.ioConfig.

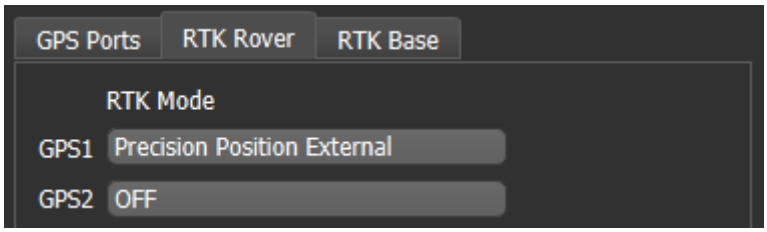


DID_FLASH_CONFIG	Value
ioConfig (firmware >=1.8.5)	0x00840040

3. Enable the NMEA messages on the external GNSS:

Message	Description
GNS	GNSS Fix data (preferred) or **GGA** - Global positioning System Fix Data.
ZDA	UTC time and date.
RMC	Recommended Minimum Specific GNSS Data.
GSA	GNSS DOP and Active Satellites.

4. If RTK positioning is supported by the NMEA receiver, Enable RTK rover mode by selecting **Precision Position External**. This will run the INS kalman filter in high accuracy mode and forward any RTK base station corrections to the external GNSS receiver.



DID_FLASH_CONFIG	Value
RTKCfgBits	0x00000002

Electrical Interface

The external NMEA GNSS receiver can be connected to Serial 0, Serial 1, and Serial 2 ports (3.3V TTL UART) on the IMX. See the PCB Module hardware page for a description of the IMX pinout. Serial 0 and 2 can be accessed on the main connector of Rugged-1 and Rugged-2 and all serial ports can be accessed on header H7 of the EVB-2.

©2022

Enabling NMEA on ZED-F9P

The recommended protocol with the IMX and ZED-F9P receiver is the uBlox binary protocol. However, the ZED-F9P can operate using NMEA protocol if necessary. The following steps can be used to enable NMEA protocol output on the ublox ZED-F9P receiver.

1. Enable NMEA output using the u-blox u-center application.

- **Set the configuration**: (ublox u-center menu -> View -> Configuration View) change the following. You must press the "Send" button to apply each change.
- **PRT (Ports)** - Set Baudrate to match the GPS port baudrate (i.e. ser1BaudRate 921600)
- **PRT (Ports)** - Enable NMEA on the connected port/UART
- **PRT (Ports)** - Enable RTCM3 on the connected port/UART if using RTK
- **RATE (Rates)** - Measurement Period: 200ms
- **RATE (Rates)** - Navigation Rate: 1cyc
- **MSG (Messages)** - Enable NMEA messages listed above for the connected port/UART (i.e. UART1 On)
- F0-0D NMEA GxGNS
- F0-08 NMEA GxZDA
- F0-04 NMEA GxRMC
- F0-03 NMEA GxGSV
- **Save the configuration**: Send the CFG (Configuration) to 1 - FLASH or press the "Save Config" button with the small gear save icon (or menu Receiver -> Action -> Save Config).

©2022

7.3.5 Septentrio GNSS Aiding IMX

The IMX can be configured for use with Septentrio GNSS receivers such as the Mosaic-G5. This can be done using either the EvalTool GNSS Setting tab or the IMX `DID_FLASH_CONFIG.ioConfig` and `DID_FLASH_CONFIG.RTKCfgBits` fields.

GNSS Ports	**Value**
GNSS Source	serial 0, serial 1, or serial 2
GNSS Type	Septentrio
GNSS1 Timepulse	*Disable* or IMX pin connected to Septentrio PPS
GNSS2 Timepulse	*Disable*

RTK Rover	**Value**
GNSS RTK Mode	Precision Position or Compass

RTK Base	**Value**
Serial Port 0 (Single GNSS only)	GNSS1 - RTCM3
USB Port	GNSS1 - RTCM3

The following sections detail how to interface and configure the IMX for operation using a Septentrio receiver. See RTK precision positioning and RTK compassing for RTK operation principles.

Single-Receiver, Dual-Antenna Architecture

Unlike the ublox ZED or the GPX-1, which use two independent receiver modules for dual-antenna RTK compassing (one wired to GNSS1, a second to GNSS2), the Septentrio Mosaic-G5 is a **single receiver with two antenna inputs** (Main and Aux1). Only GNSS1 needs a **Source** and **Type** of **Septentrio** configured — GNSS2 stays disabled. Enabling RTK Compassing in `RTKCfgBits` tells the IMX to command the Mosaic-G5 into its own internal dual-antenna heading mode, rather than routing a second physical receiver.

How the IMX Talks to the Septentrio Receiver

The IMX communicates with the Septentrio receiver using its native **SBF (Septentrio Binary Format)** protocol rather than translating to/from RTCM or NMEA. Once a serial port's GNSS Type is set to **Septentrio**, the IMX automatically initializes the receiver:

1. Raises the receiver's baud rate from its factory default of 115200 to the IMX's operating baud rate of 921600, and switches its own UART to match.
2. Configures the receiver's antenna mode — multi-antenna heading mode if RTK Compassing is enabled, single-antenna otherwise.
3. Enables SBF output on the port.
4. Streams the position, velocity, satellite-visibility, and dual-antenna measurement messages the IMX needs, at a 500 ms (2 Hz) interval.
5. Configures a 1 Hz PPS output from the receiver.

RTK correction data (RTCM3) received on any IMX serial port is forwarded byte-for-byte to the Septentrio receiver. The IMX withholds corrections until the initialization sequence above has completed.

Firmware Update

Septentrio GNSS firmware updates are performed through the IMX's own fwUpdate protocol — the same mechanism used for IMX application firmware updates — rather than the serial-bridge-plus-vendor-tool procedure used for ublox ZED receivers. Use

©2022

`cltool -ufpkg <package>` or the EvalTool firmware update dialog; the IMX forwards the update to the Septentrio receiver internally.

©2022

7.3.6 u-blox GNSS Aiding IMX

The IMX can be configured for use with u-blox GNSS receivers, such as the ZED-X20 and ZED-F9P. This can be done using either the EvalTool GNSS Setting tab or the IMX `DID_FLASH_CONFIG.ioConfig` and `DID_FLASH_CONFIG.RTKCfgBits` fields.

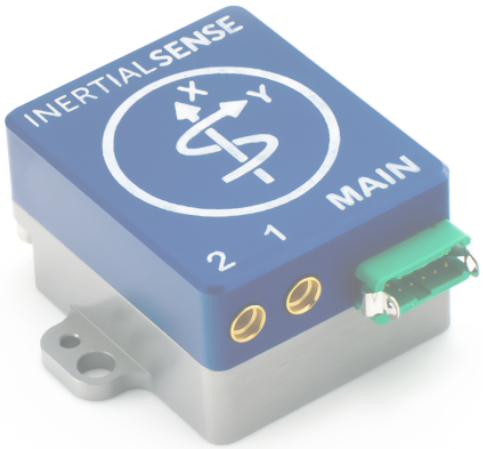
GNSS Ports	**Value**
GNSS Source	serial 0, serial 1, or serial 2
GNSS Type	ublox
GNSS1 Timepulse	*Disable* or IMX pin connected to 1st ZED PPS
GNSS2 Timepulse	*Disable* or IMX pin connected to 2nd ZED PPS

RTK Rover	**Value**
GNSS RTK Mode	Precision Position or Compass

RTK Base	**Value**
Serial Port 0 (Single GNSS only)	GNSS1 - RTCM3
USB Port	GNSS1 - RTCM3

The following sections detail how to interface and configure the IMX for operation using the ZED. See RTK precision positioning and RTK compassing for RTK operation principles.

RUGGED-3



The Rugged-3 INS contains either a single or dual ZED receiver onboard, supporting RTK positioning and compassing. GNSS1 and GNSS2 are connected to serial ports 1 and 0 respectively on the IMX.

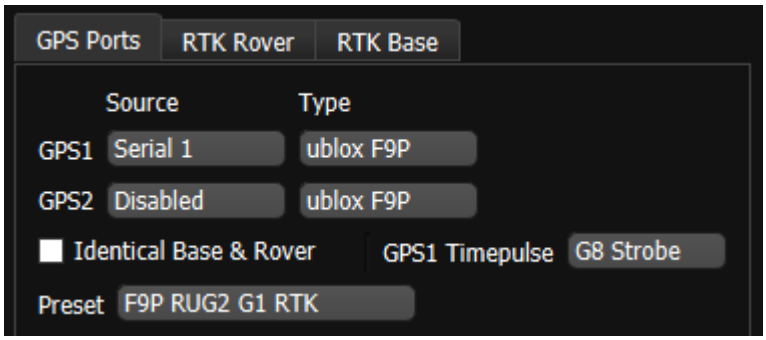
Single GNSS Settings

Use the following IMX settings with the Rugged-3-G1 (single GNSS receiver). These settings can be applied either using the EvalTool GNSS Settings tab or the IMX `DID_FLASH_CONFIG.ioConfig` and `DID_FLASH_CONFIG.RTKCfgBits` fields.

GNSS Ports

Set the GNSS1 source to **Serial 1** and type to **ublox**.

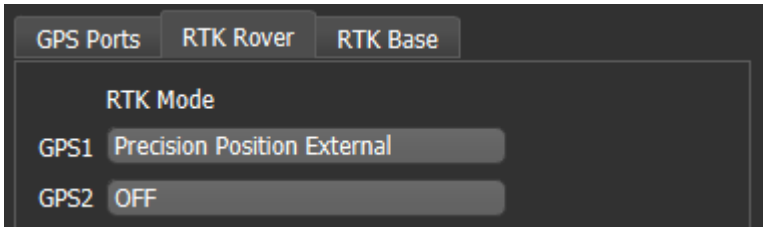
©2022



DID_FLASH_CONFIG	Value
ioConfig (firmware >=1.8.5)	0x0244a040

RTK Rover

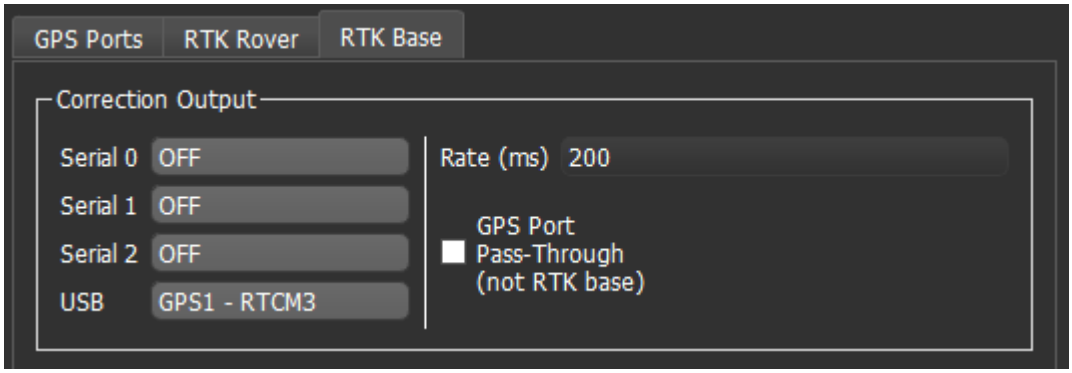
Enable RTK rover mode by selecting **ZED Precision Position**.



DID_FLASH_CONFIG	Value
RTKCfgBits	0x00000002

RTK Base

To configuring a system as an RTK base, disable the RTK Rover by setting the GNSS1 and GNSS2 RTK Mode to **OFF**, and select the appropriate correction output port on the IMX.



DID_FLASH_CONFIG	Value
RTKCfgBits	0x00000900

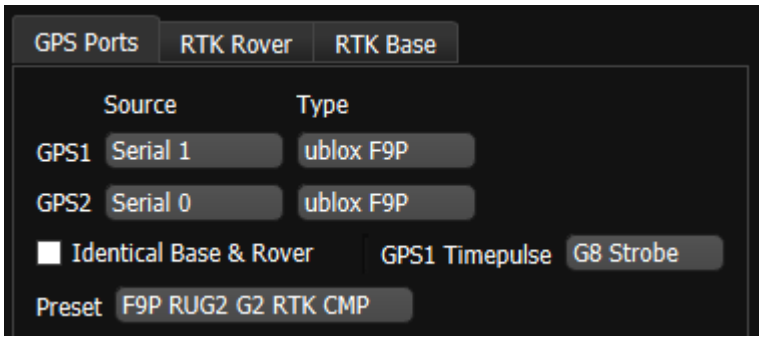
Dual GNSS Settings

Use the following IMX settings with the Rugged-3-G2 (dual GNSS receivers). These settings can be applied either using the EvalTool GNSS Settings tab or the IMX `DID_FLASH_CONFIG.ioConfig` and `DID_FLASH_CONFIG.RTKCfgBits` fields.

GNSS Ports

Set GNSS1 and GNSS2 to source **Serial 1** and **Serial 0**. the serial port that the ZED is connected to and type to **ublox**.

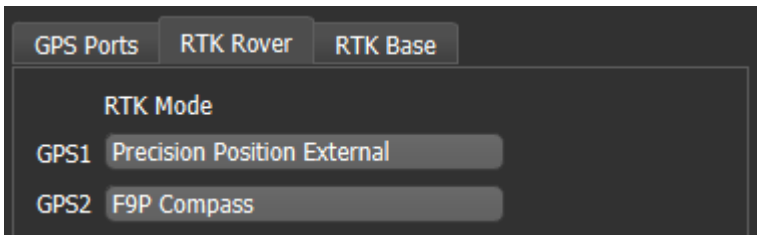
©2022



DID_FLASH_CONFIG	Value
ioConfig (firmware >=1.8.5)	0x025ca040

RTK Rover

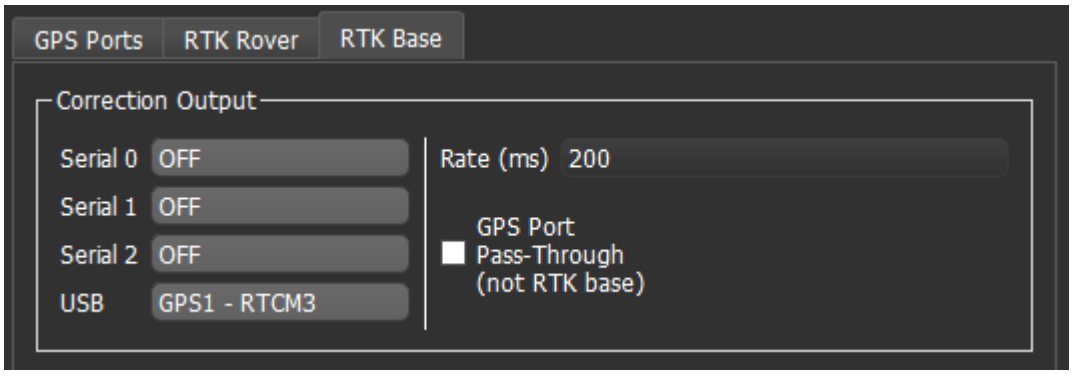
Enable RTK rover mode by selecting **Precision Position External**. **GNSS1** is designated for **Precision Position External** and **GNSS2** for **ZED Compass settings**. Either or both can be enabled at the same time.



DID_FLASH_CONFIG	Value
RTKCfgBits	0x00000006

RTK Base

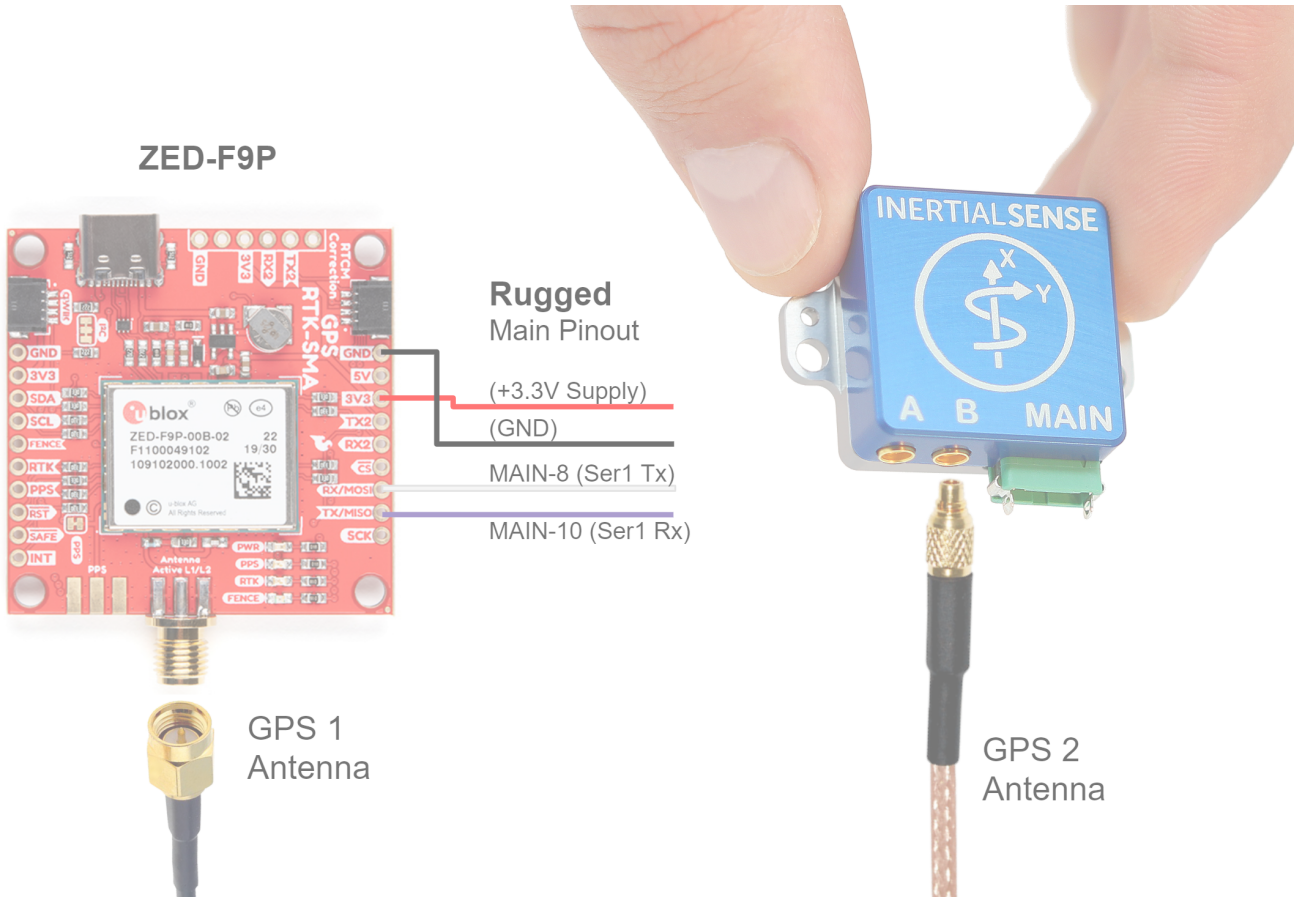
To configuring a system as an RTK base, skip the RTK rover settings, and select the appropriate correction output port on the IMX. Notice that IMX serial port 0 and 1 may be unavailable and occupied by the dual ZED receivers.



DID_FLASH_CONFIG	Value
RTKCfgBits	0x00000900

©2022

RUGGED-3-IMX-5 TO ZED



A +3.3V or +5V supply is needed to power the ZED when using the Rugged-1 IMX. A USB +5V supply can be used if available. The Rugged-1 must be configured for Serial Port 1 TTL voltage. See hardware configuration for Rugged v1.0 or Rugged v1.1 for details.

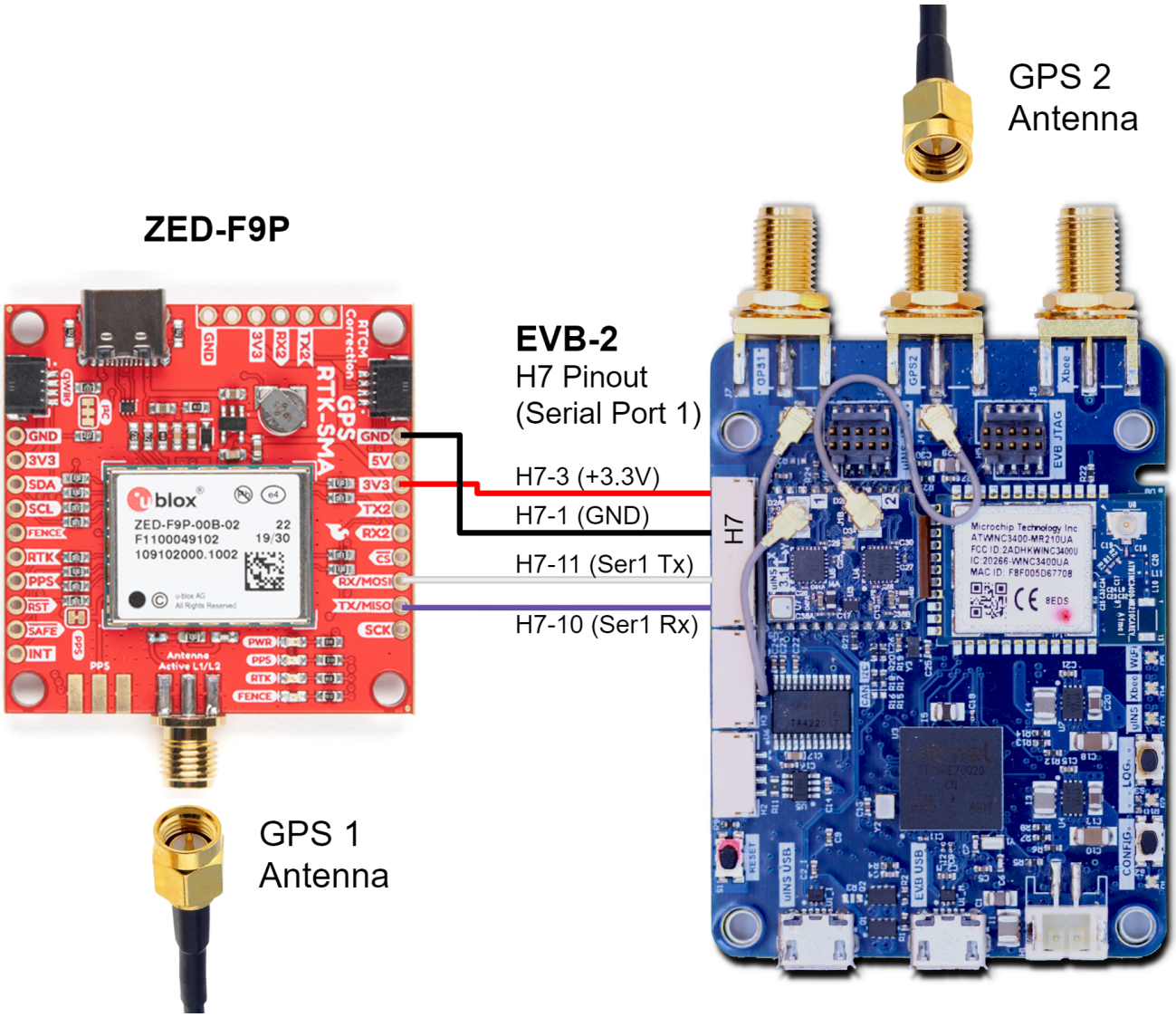
Settings

See the single GNSS settings.

EVB-2 to ZED Interface

Use the following wiring when connecting a single ZED receiver to the EVB-2. GNSS1 is typically routed to serial port 1 and provides PPS to the IMX.

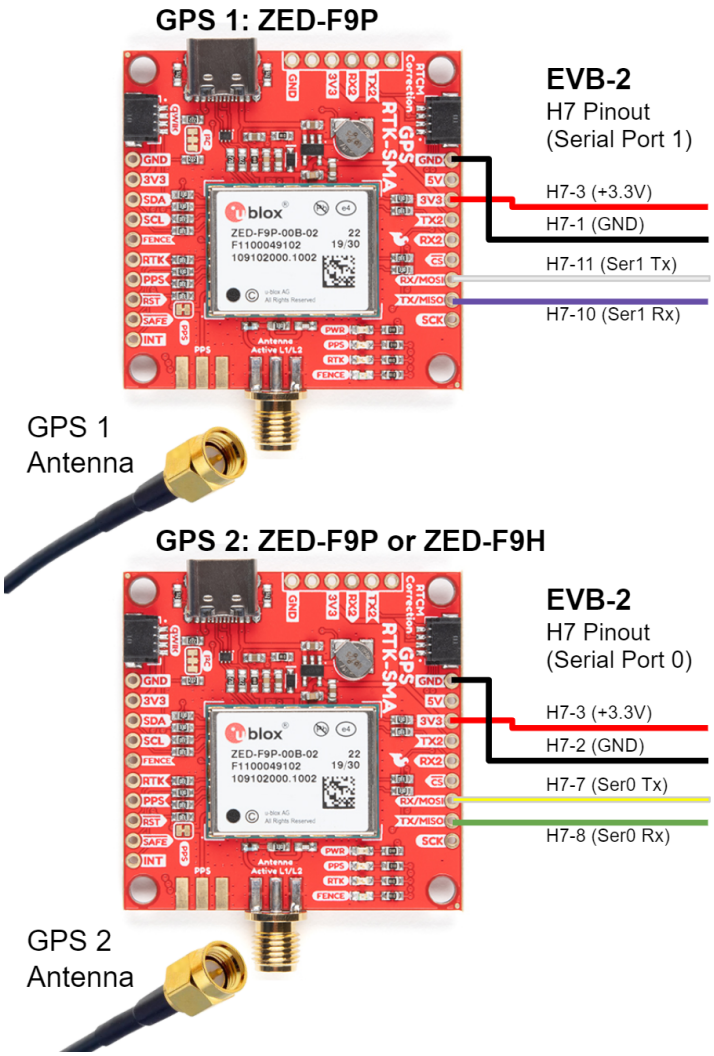
©2022

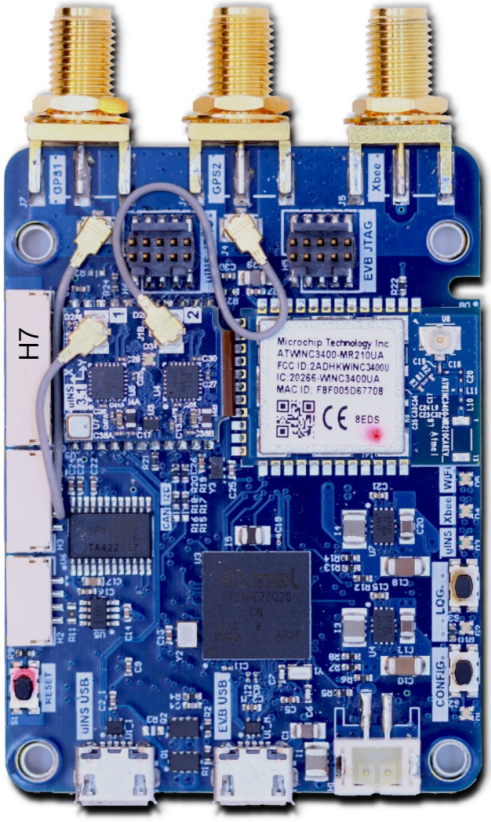


EVB-2 to Dual ZED Interface

For dual-receiver RTK compassing setups, connect GNSS1 and GNSS2 as shown below. Verify both receivers share the same reference clock and PPS to maintain synchronization.

©2022





RTK Base Messages

In RTK mode, the ZED requires RTCM version 3 messages supporting DGNSS according to RTCM 10403.3.

©2022

ZED ROVER MESSAGES

The ZED operating in RTK rover mode can decode the following RTCM 3.3 messages.

Message type	Description
RTCM 1001	L1-only GPS RTK observables
RTCM 1002	Extended L1-only GPS RTK observables
RTCM 1003	L1/L2 GPS RTK observables
RTCM 1004	Extended L1/L2 GPS RTK observables
RTCM 1005	Stationary RTK reference station ARP
RTCM 1006	Stationary RTK reference station ARP with antenna height
RTCM 1007	Antenna descriptor
RTCM 1009	L1-only GLONASS RTK observables
RTCM 1010	Extended L1-only GLONASS RTK observables
RTCM 1011	L1/L2 GLONASS RTK observables
RTCM 1012	Extended L1/L2 GLONASS RTK observables
RTCM 1033	Receiver and antenna description
RTCM 1074	GPS MSM4
RTCM 1075	GPS MSM5
RTCM 1077	GPS MSM7
RTCM 1084	GLONASS MSM4
RTCM 1085	GLONASS MSM5
RTCM 1087	GLONASS MSM7
RTCM 1094	Galileo MSM4
RTCM 1095	Galileo MSM5
RTCM 1097	Galileo MSM7
RTCM 1124	BeiDou MSM4
RTCM 1125	BeiDou MSM5
RTCM 1127	BeiDou MSM7
RTCM 1230	GLONASS code-phase biases
RTCM 4072.0	Reference station PVT (u-blox proprietary RTCM Message)

©2022

ZED BASE OUTPUT MESSAGES

The ZED operating in RTK base mode will generate the following RTCM 3.3 output messages depending on whether the satellite constellation have been enabled. See the Constellation Selection for information on enabling and disabling satellite constellations.

Message Type	Period (sec)	Description
RTCM 1005	2	Stationary RTK reference station ARP
RTCM 1074	0.4	GPS MSM4
RTCM 1077	0.4	GPS MSM7
RTCM 1084	0.4	GLONASS MSM4
RTCM 1087	0.4	GLONASS MSM7
RTCM 1094	0.4	Galileo MSM4
RTCM 1097	0.4	Galileo MSM7
RTCM 1124	0.4	BeiDou MSM4
RTCM 1127	0.4	BeiDou MSM7
RTCM 1230	2	GLONASS code-phase biases

NTRIP MESSAGES

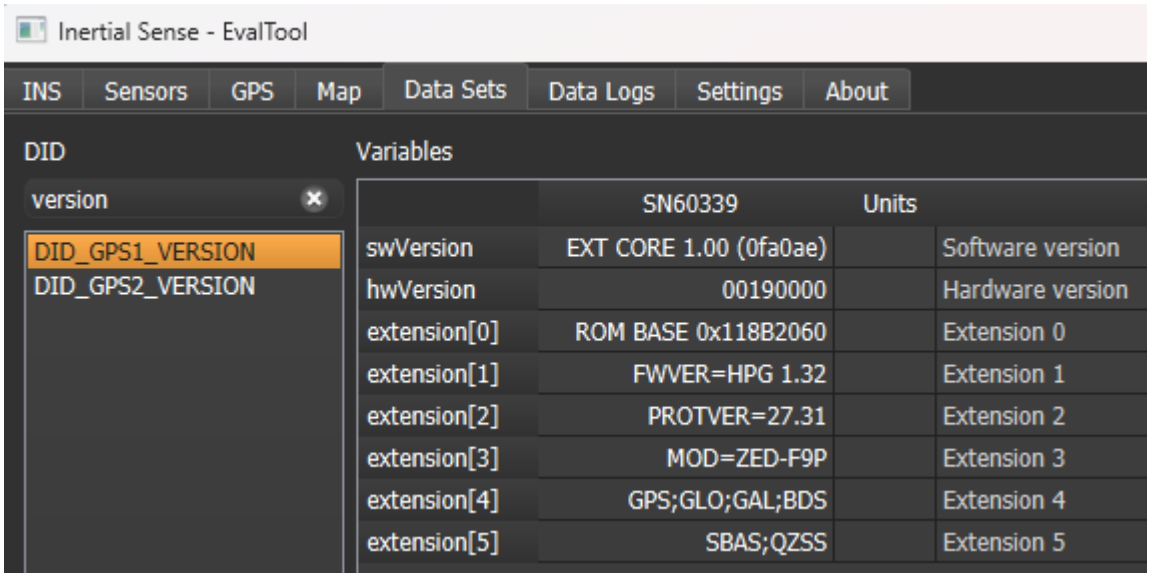
The NTRIP server must provide the necessary subset of RTCM3 messages supported by the IMX-RTK. See the NTRIP page for an overview of NTRIP.

ZED Firmware Update

The following section describes how to view the current GNSS firmware version and how to update the firmware on the u-blox ZED GNSS receiver through the IMX.

GNSS FIRMWARE VERSION

The current GNSS firmware version can be read through the `DID_GNSS1_VERSION` and `DID_GNSS2_VERSION` messages.



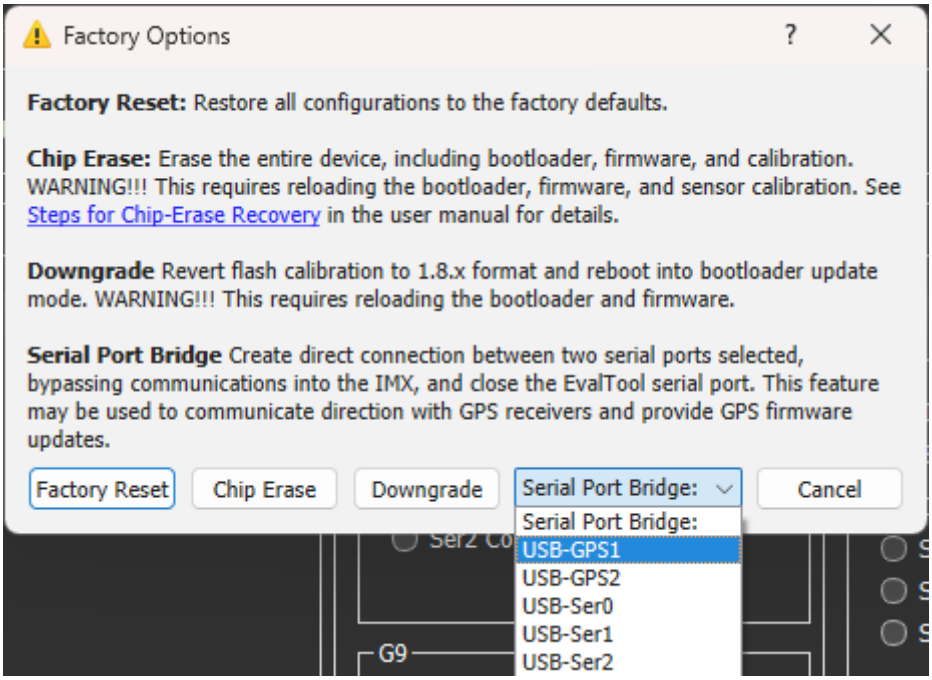
©2022

FIRMWARE UPDATE

The following steps describe how to update the u-blox ZED firmware using the u-blox U-Center application. The U-Center application and firmware binary can be downloaded from the u-blox ZED documentation and resources webpage.

©2022

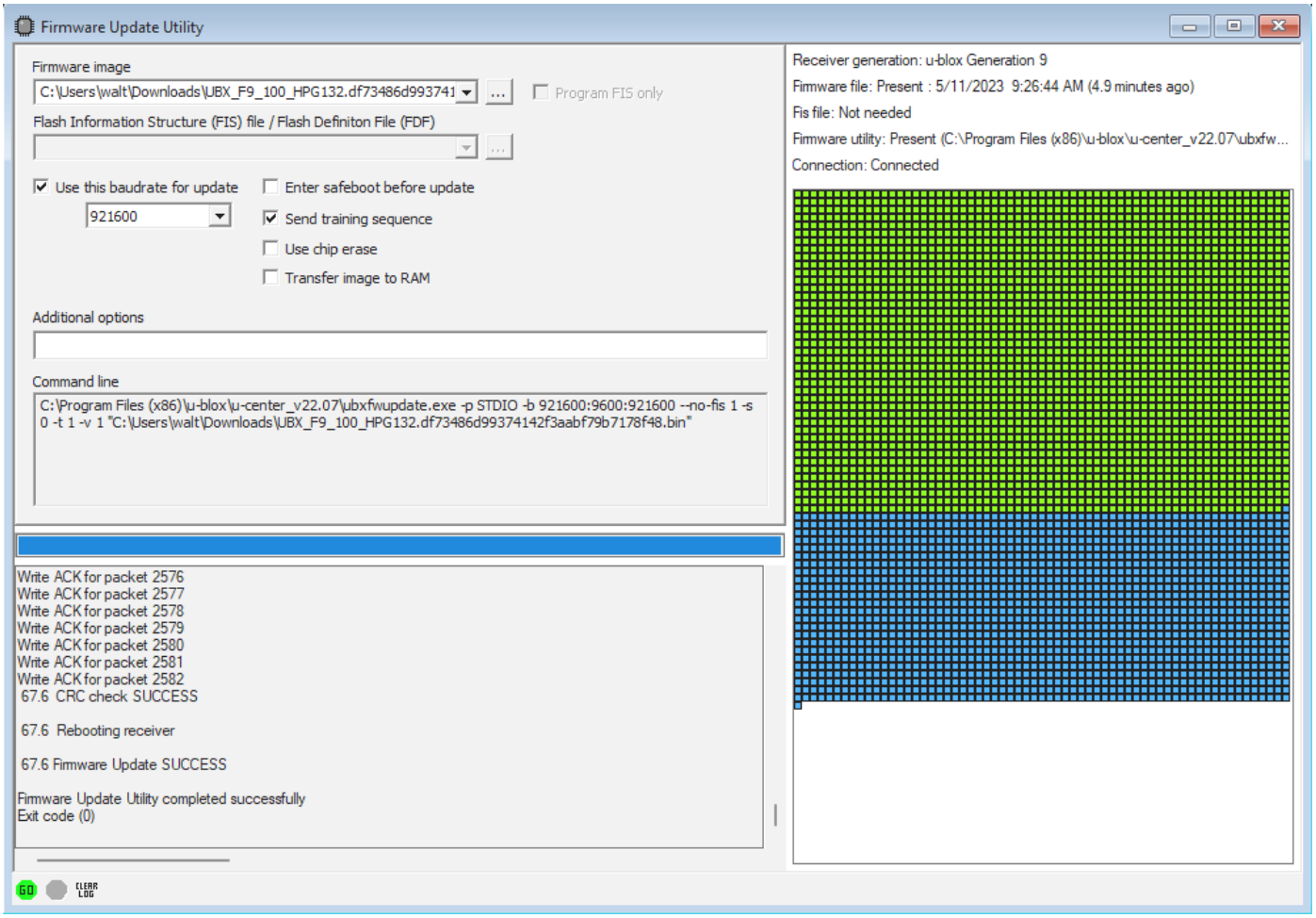
1. **Enable IMX Serial Bypass** - Send the system command (`DID_SYS_CMD`) `SYS_CMD_ENABLE_SERIAL_PORT_BRIDGE_USB_TO_GPS1` or `SYS_CMD_ENABLE_SERIAL_PORT_BRIDGE_USB_TO_GPS2` to enable serial bypass on the IMX. This will create a direct connection between the current IMX serial port and the GNSS receiver. This is done in the EvalTool using the Factory Options dialog in the Settings -> General tab.



©2022

2. **Update Using U-Center** - With the IMX serial bypass enabled, the u-blox U-Center software can connect directly to the ZED GNSS receiver. Use the following steps in the ublox U-Center app:

- Open the serial port with baudrate 921600.
- Select Tool -> Firmware Update and specify the ublox firmware file (i.e. `UBX_F9_100_HPG132...bin`).
- Enable "Use this baudrate for update" as 921600.
- Disable "Enter safeboot before update".
- Enable "Send training sequence".
- Start the firmware update by pressing the small green "GO" circle in the bottom left corner of the Firmware Update Utility dialog.



- Power cycle the IMX.

Purchasing the ZED

The following components work well when sourcing ZED receivers and matching antennas for multi-band GNSS installations.

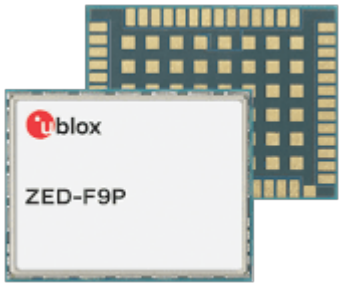
©2022

Multi-Band GNSS Components

The following is a list of the ZED GNSS receivers and compatible antenna(s).

©2022

Item


ublox
ZED-F9P

RTCM Correction
GPS
RTK-SMA
TX2
RX2
3V3
GND
GND
5V
3V3
TX2
RX2
SCL
RX/MOSI
TX/MISO
SCK
PWR
PPS
RTK
FENCE
ZED-F9P-00B-02
F1100048102
109102000.1002
Antenna Active (LNA)
I2C
QWIIC
PPS
GND
3V3
SDA
SCL
FENCE
RTK
PPS
RST
SAFE
INT

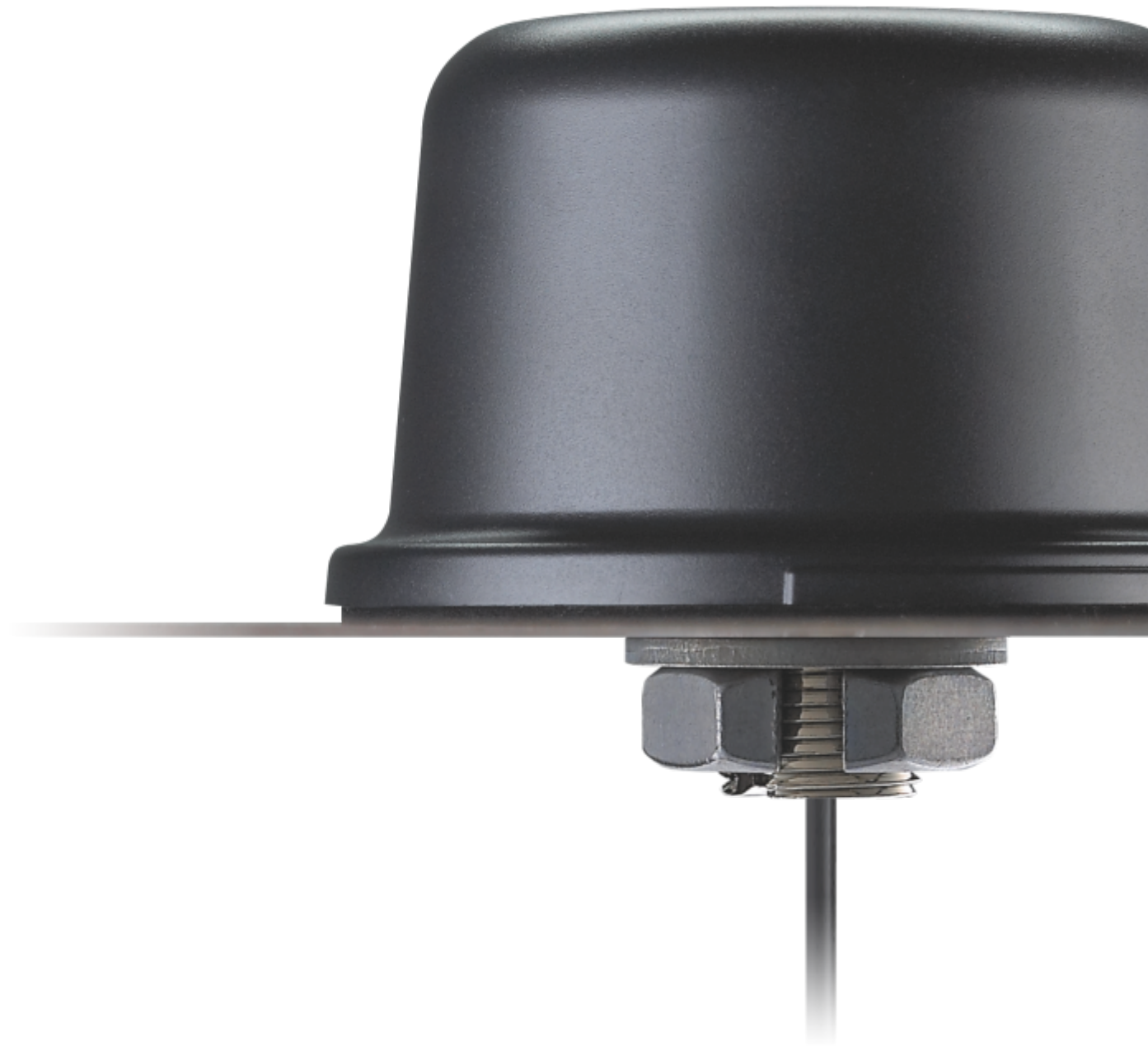
©2022

Item

©2022

©2022

Item



©2022

Item

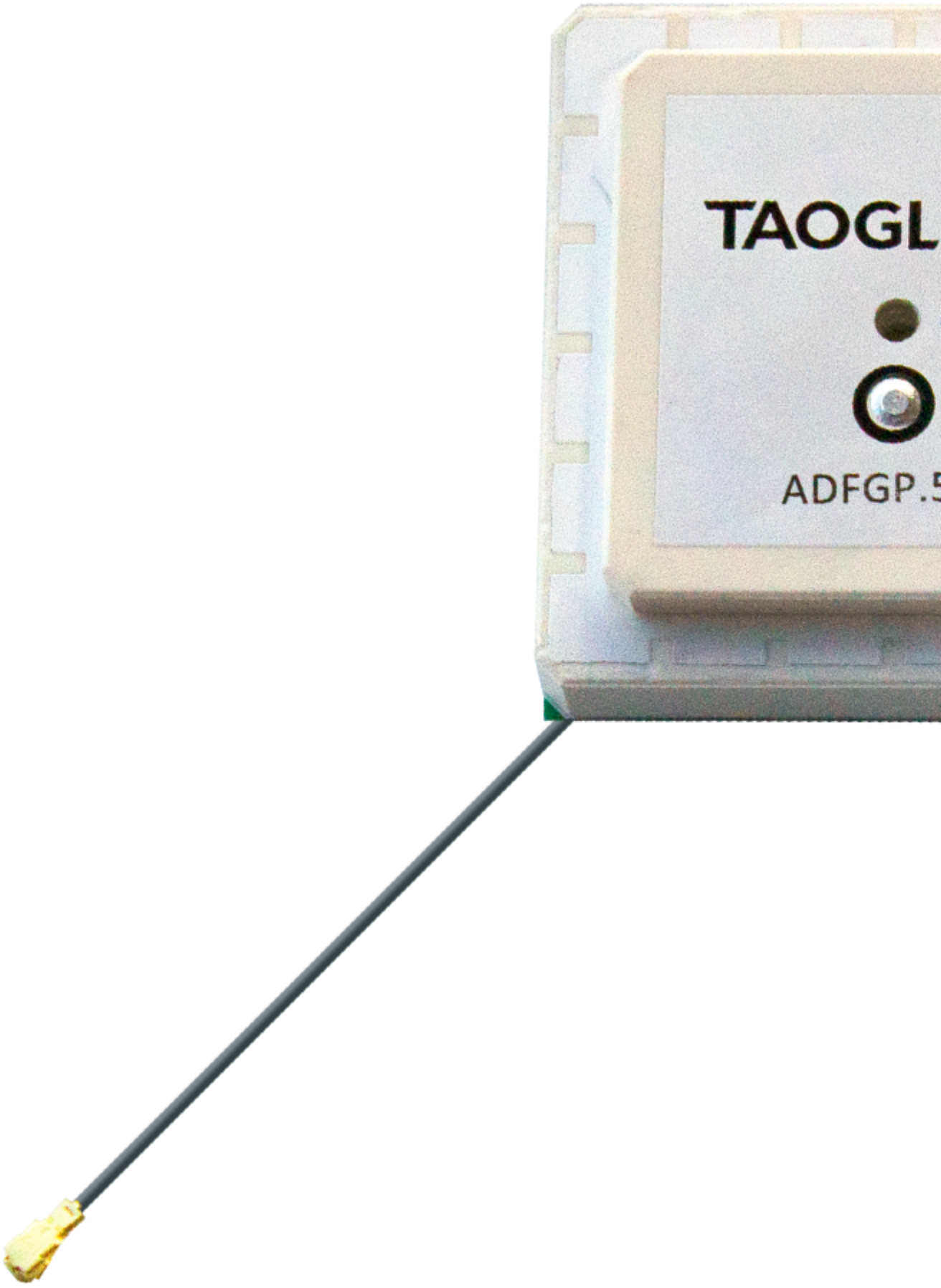
©2022

Item

©2022

Item

©2022



©2022

Item

©2022

7.4 Configuration

7.4.1 IMU INS Configuration

Translation

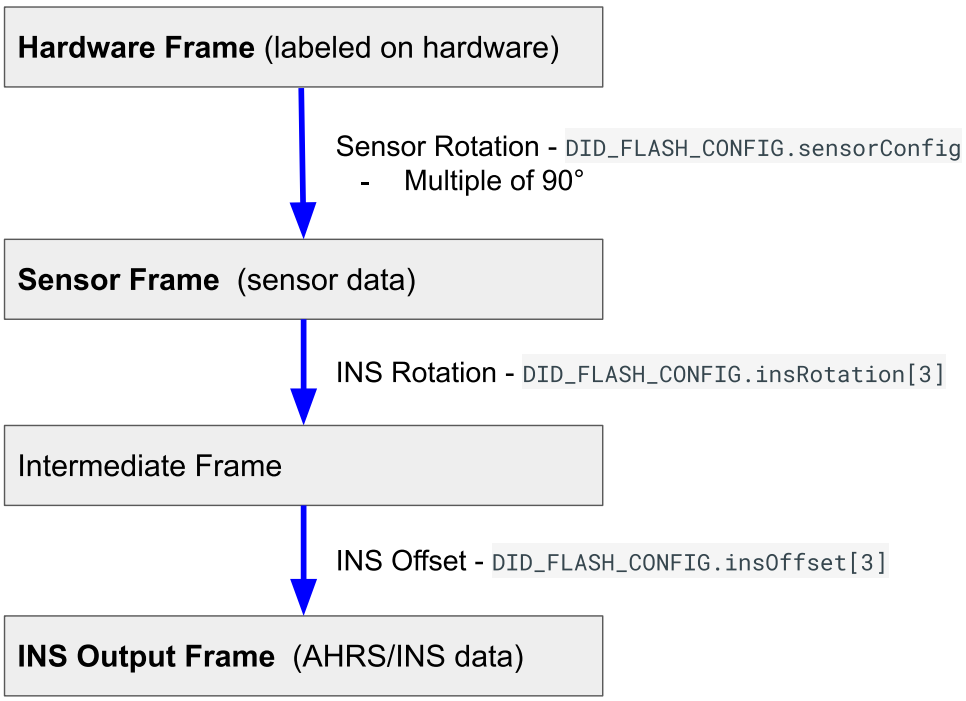
The IMX can be mounted and operated in any arbitrary orientation. It is often desirable and conventional to translate the IMX output so that it is translated into the vehicle frame located at certain point for control and navigation of the vehicle. This is done using the ***Sensor Rotation***, ***INS Rotation***, and ***INS Offset*** parameters.

In most common applications, output is translated to the vehicle frame (X to the front, Y to the right, and Z down):

- *Sensor Rotation* provides gross rotation of the IMU output in multiples of 90°.
- *INS Rotation* provides small angle alignment of the INS output.
- *INS Offset* shifts the location from the INS output.

COORDINATE FRAME RELATIONSHIP

The relationship between the Hardware Frame, Sensor Frame, and INS Output Frame are as follows.



The ***Hardware Frame*** and ***Sensor Frame*** are equivalent when the ***Sensor Rotation*** in `DID_FLASH_CONFIG.sensorConfig` is zero. The ***Hardware Frame origin*** and ***Sensor Frame origin*** are always at the same location and may differ in direction according to the ***Sensor Rotation*** in `DID_FLASH_CONFIG.sensorConfig`. The ***Sensor Frame*** and ***INS output Frame*** are equivalent when the `DID_FLASH_CONFIG.insRotation` and `DID_FLASH_CONFIG.insOffset` are zero.

SENSOR ROTATION (HARDWARE FRAME TO SENSOR FRAME)

The *Sensor Rotation* is used to rotate the IMU and magnetometer output from the hardware frame to the sensor frame by **multiples of 90°**. This is done using the `SENSOR_CFG_SENSOR_ROTATION_MASK` bits of the `DID_FLASH_CONFIG.sensorConfig` as defined in `enum eSensorConfig`. The Sensor Rotation is defined in X,Y,Z rotations about the corresponding axes and applied in the order of Z,Y,X. This rotation is recommended for gross rotations.

©2022

INS ROTATION

The *INS rotation* is used to convert the INS output from the sensor frame to the vehicle frame. This is useful if the sensor frame and vehicle frame are not aligned. The actual INS rotation parameters are `DID_FLASH_CONFIG.insRotation[3]` (X, Y, Z) in radians. The *INS rotation* values describe the rotation from the INS sensor frame to the intermediate frame in order of Z, Y, X.

INS OFFSET

The *INS offset* is used to shift the location of the INS output and is applied following the INS Rotation. This offset can be used to move the IMX location from the origin of the sensor frame to any arbitrary location, often a control navigation point on the vehicle.

MANUALLY ALIGNING THE INS AFTER MOUNTING

NOTE for use:

- The Infield Calibration process can be used instead of this process to automatically measure and align the INS with the vehicle frame for INS rotations less than 15°.
- If using software release 1.8.4 or newer, we recommend using the `DID_FLASH_CONFIG.sensorConfig` to rotate the sensor frame by 90° to near level before following the steps below.

The following process uses the IMX to measure and correct for the IMX mounting angle.

1. Set `DID_FLASH_CONFIG.insRotation` to zero.
2. Set the sensor on the ground at various known orientations and record the INS quaternion output (DID_INS_2). Using the Euler output (DID_INS_1) can be used if the pitch is less than 15°. It is recommended to use the EKF Zero Motion Command to ensure the EKF bias estimation and attitude have stabilized quickly before measuring the INS attitude.
3. Find the difference between the known orientations and the measured INS orientations and average these differences together.
4. Negate this average difference and enter that into the `DID_FLASH_CONFIG.insRotation`. This value is in Euler, however it is OK for this step as this rotation should have just been converted from quaternion to Euler and will be converted back to quaternion on-board for the runtime rotation.

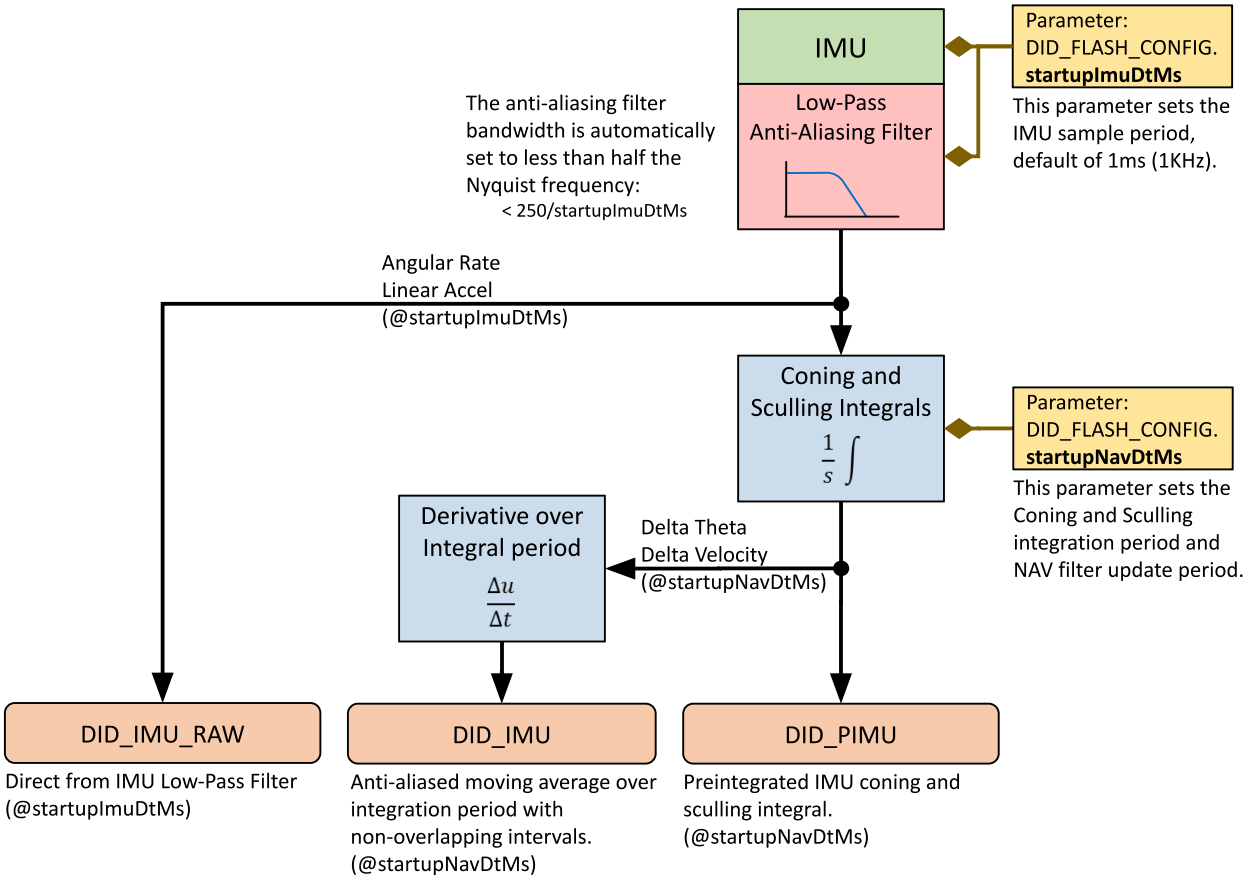
Infield Calibration

The *Infield Calibration* provides a method to 1.) zero IMU biases and 2.) zero INS attitude to align the INS output frame with the vehicle frame. These steps can be run together or independently.

IMU Sample and Navigation Periods

The IMU sample period is configured by setting `DID_FLASH_CONFIG.startupImuDtMs` in milliseconds. This parameter determines how frequently the IMU is measured and data integrated into the `DID_PIMU` data. `DID_FLASH_CONFIG.startupImuDtMs` also automatically sets the bandwidth of the IMU anti-aliasing filter to less than one half the Nyquist frequency (i.e. < 250 / startupImuDtMs).

©2022



The preintegrated IMU (PIMU) a.k.a. Coning and Sculling (delta theta, delta velocity) integrals serve as an anti-aliased moving average of the IMU value. The `DID_IMU` is the derivative of the `DID_PIMU` value over a single integration period.

IMU LATENCY

The IMU low-pass filter (LPF) adds latency (delay) to the signal in the IMU output. This latency can be expressed as:

IMU Latency \cong \frac{2.197}{LPF bandwidth}

The default IMU sensor bandwidths (cutoff frequencies) and corresponding signal latencies are:

Sensor	Bandwidth	Signal Latency
Gyro	539 Hz	4.1 ms
Accelerometer	416 Hz	5.3 ms

NAVIGATION UPDATE AND OUTPUT PERIODS

The navigation filter output period should be set using the flash parameter `DID_FLASH_CONFIG.startupNavDtMs`. This value sets the `DID_SYS_PARAMS.navOutputDtMs` and `DID_SYS_PARAMS.navUpdateDtMs` during startup of the IMX.

The **navigation filter output period** (`DID_SYS_PARAMS.navOutputDtMs`) determines the EKF output data rate, the maximum rate for messages DID_INS_1, DID_INS_2, and DID_INS_3.

The **navigation filter update period** (`DID_SYS_PARAMS.navUpdateDtMs`) controls the EKF update rate and sets the standard integration period for the preintegrated IMU (PIMU) output. This parameter is automatically adjusted based on the value of `DID_SYS_PARAMS.navOutputDtMs` and the amount of CPU available.

©2022

Minimum NAV Output and Update Period (Maximum Data Rate)

The following table lists the output and update period minimum limits for the IMX.

Operation Mode	IMX-5 Minimum Output Period / Update Period	IMX-6 Minimum Output Period / Update Period
INS (GPS enabled)	7 ms (142 Hz) / 14 ms	2 ms (500 Hz) / 4 ms
AHRS (GPS disabled)	5 ms (200 Hz) / 10 ms	2 ms (500 Hz) / 4 ms
VRS (GPS and magnetometer disabled)	4 ms (250 Hz) / 8 ms	2 ms (500 Hz) / 4 ms

INS-GNSS Dynamic Model

The `DID_FLASH_CONFIG.dynamicModel` setting allows the user to adjust how the EKF behaves in different dynamic environments. All values except for 2 (STATIONARY) and 8 (AIR <4g) are experimental. The user is encouraged to attempt to use different settings to improve performance, however in most applications the default setting, 8: airborne <4g, will yield best performance. This parameter is applied to the GNSS receiver(s) engine as well as the INS EKF.

The STATIONARY configuration (dynamicModel = 2) can be used to configure the EKF for static applications. It is a permanent implementation of the Zero Motion Command which will reduce EKF drift under stationary conditions.

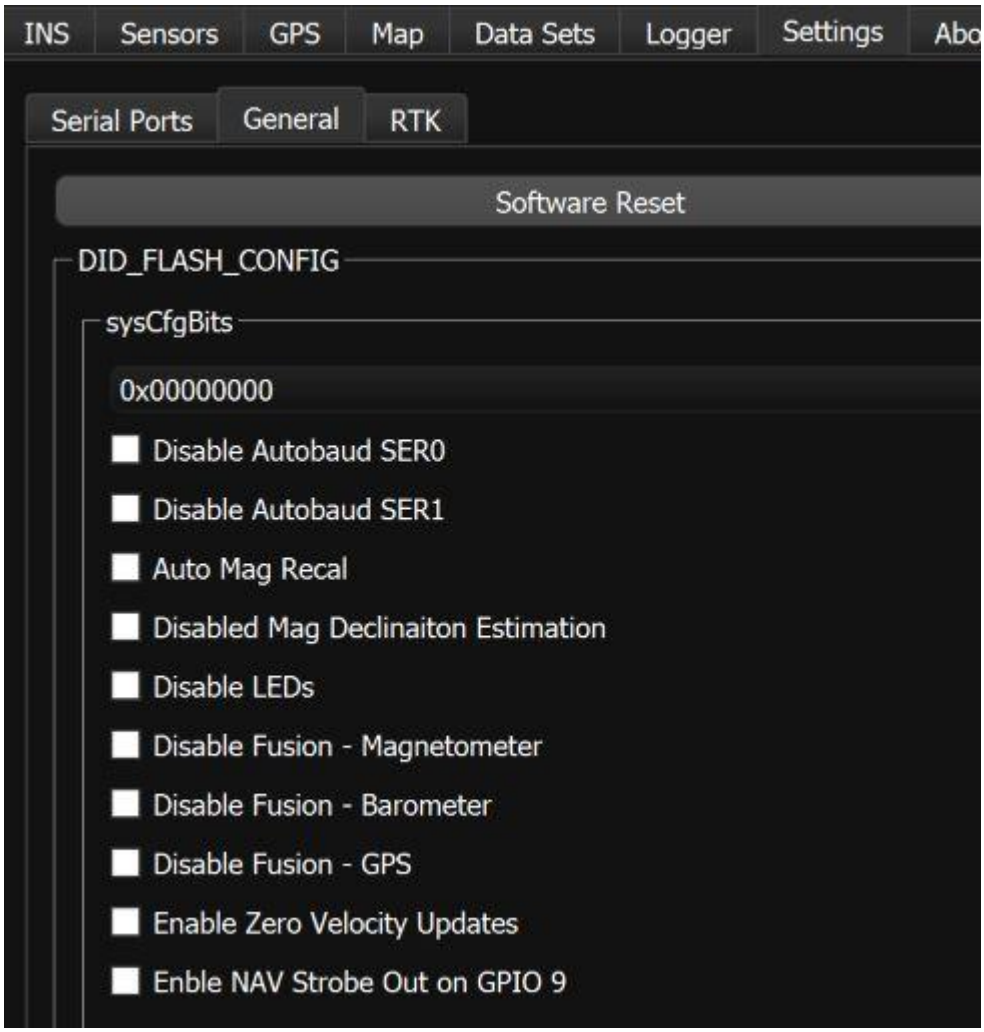
Disable Magnetometer and Barometer Updates

Magnetometer and barometer updates (fusion) into the INS and AHRS filter (Kalman filter) can be disabled by setting the following bits in `DID_FLASH_CONFIG.sysCfgBits`.

Bit Name	Bit Value	Description
SYS_CFG_BITS_DISABLE_MAGNETOMETER_FUSION	0x00001000	Disable magnetometer fusion into EKF
SYS_CFG_BITS_DISABLE_BAROMETER_FUSION	0x00002000	Disable barometer fusion into EKF

These settings can be disabled using the General Settings tab of the EvalTool.

©2022



Disable Zero Velocity Updates

Zero velocity updates (ZUPT) rely on GPS and/or wheel encoder data. In some cases there can be a slight lag/deviation when starting motion while simultaneously rotating. This is because GPS data is updated at 5 Hz and it takes a few samples to detect motion after a period of no motion. When ZUPT is enabled, it acts as a virtual velocity sensor telling the system that its velocity is zero. It may conflict briefly with GPS velocity observation when starting motion. If a slight lag at the beginning of motion is an issue, ZUPT may be disabled. Generally it should be enabled (Default). It can be disabled using `DID_FLASH_CONFIG.sysCfgBits` or using the General Settings tab of the EvalTool.

Disable Zero Angular Rate Updates

Zero angular rate updates (ZARU) rely on analysis of either IMU (gyro) data or wheel encoders when available. When angular motion is very slow and no wheel encoders are available a zero angular rate may be mistakenly detected, which will lead to gyro bias estimation errors. In these cases it can be beneficial to disable ZARU if the applications has slow rotation rates (approximately below 3 deg/s). It is not encouraged to disable ZARU if there is no rotation or faster rotation. It can be disabled using `DID_FLASH_CONFIG.sysCfgBits` or using the General Settings tab of the EvalTool.

©2022

7.4.2 Infield Calibration

The *Infield Calibration* provides a method to 1.) zero IMU biases and 2.) zero INS attitude to align the INS output frame with the vehicle frame. These steps can be run together or independently.

Zeroing IMU Bias

Zeroing IMU bias is a way to remove permanent offsets in the sensor output that may have occurred as a result of manufacturing or high shock. The system must be completely stationary for accurate bias measurement. The current value for IMU biases stored in flash memory is viewable in `DID_INFIELD_CAL.imu` when infield calibration is inactive and `DID_INFIELD_CAL.sampleTimeMs` is zero.

ACCELEROMETER BIAS

In order to correct accelerometer bias on a given axis, that axis must be sampled while measuring full gravity. Thus, only the accelerometer axes that are sampled while in the vertical direction can be corrected. In order to correct all accelerometer axes, all three axes must be sampled while oriented vertically. The sample can be done while the axis is pointed up, down, or both up and down for averaging.

GYRO BIAS

All three axes of the gyros are sampled simultaneously, and the bias is stored in flash memory. The system must be completely stationary for accurate bias measurement. The system does not need to be level to zero the gyro biases.

Zeroing INS Attitude

The Infield Calibration process can be used to align or level the INS output frame with the vehicle frame. This is done by observing the X,Y,Z axes rotations necessary to level the orientation(s) sampled. Zeroing the INS attitude as part of the Infield Calibration routine provides a optimal and highly accurate method for measuring the attitude while stationary by averaging raw bias corrected accelerations.

Rotations cannot be computed for axes that are pointed vertically. For example, a single orientation sample with X and Y in the horizontal plane and Z pointed down will only be able to produce an X,Y rotation, and the Z rotation will remain zero. To compute all three rotations for the X,Y,Z axes, the system must be sampled at least twice, once while level and once while on its side.

The infield calibration process is generally useful for only small angle INS rotations and is not intended for those larger than 15° per axis. The user must set the INS rotation manually for larger rotations. The INS rotation is stored and accessed in `DID_FLASH_CONFIG.insRotation` in flash memory.

Because the sampled orientations are averaged together, it is recommended to only sample orientations that are at true 90° multiples of the vehicle frame.

The zero INS attitude feature assumes there are flat rigid surface(s) attached to the IMX about which the system can be leveled. If the working surface is not level or additional precision is desired, each orientation sampled can have an additional sample taken with ~180° yaw offset to cancel out tilt of the working surface.

If Infield Calibration is not adequate, the INS may be leveled or aligned manually.

©2022

Infield Calibration Process

The following process can be used to used to improve the IMU calibration accuracy and also align or level the INS to the vehicle frame.

1. **Prepare Leveling Surface** - Ensure the system is stable and stationary on a near-level surface with one of three axes in the vertical direction.
2. **Initialize the Mode** - Clear any prior samples and set the calibration mode by setting `DID_INFIELD_CAL.state` to one of the following:

```
INFIELD_CAL_STATE_CMD_INIT_ZERO_IMU            = 1, // Zero accel and gyro biases.
INFIELD_CAL_STATE_CMD_INIT_ZERO_GYRO           = 2, // Zero only gyro  biases.
INFIELD_CAL_STATE_CMD_INIT_ZERO_ACCEL          = 3, // Zero only accel biases.
INFIELD_CAL_STATE_CMD_INIT_ZERO_ATTITUDE       = 4, // Zero (level) INS attitude by adjusting INS rotation.
INFIELD_CAL_STATE_CMD_INIT_ZERO_ATTITUDE_IMU   = 5, // Zero gyro and accel biases.  Zero (level) INS attitude by adjusting INS rotation.
INFIELD_CAL_STATE_CMD_INIT_ZERO_ATTITUDE_GYRO  = 6, // Zero only gyro  biases.  Zero (level) INS attitude by adjusting INS rotation.
INFIELD_CAL_STATE_CMD_INIT_ZERO_ATTITUDE_ACCEL = 7, // Zero only accel biases.  Zero (level) INS attitude by adjusting INS rotation.

INFIELD_CAL_STATE_CMD_INIT_OPTION_DISABLE_MOTION_DETECT    = 0x00010000, // Bitwise AND this with the above init commands to disable motion detection during
sampling (allow for more tolerant sampling).
 INFIELD_CAL_STATE_CMD_INIT_OPTION_DISABLE_REQUIRE_VERTIAL = 0x00020000, // Bitwise AND this with the above init commands to disable vertical alignment
requirement for accelerometer bias calibration (allow for more tolerant sampling).
```

   Zeroing accelerometer biases requires that any of the X,Y,Z axes be vertically aligned with gravity during sampling. This is indicated by bit `INFIELD_CAL_STATUS_AXIS_NOT_VERTICAL = 0x10000000` in `DID_INFIELD_CAL.status`.

   By default, the system must also be stationary without any movement during sampling. This is indicated by bit `INFIELD_CAL_STATUS_MOTION_DETECTED = 0x20000000` is set in `DID_INFIELD_CAL.status`. Motion detection can be disabled to make the system more tolerant during sampling. To do this, bitwise and `INFIELD_CAL_STATE_CMD_INIT_OPTION_DISABLE_MOTION_DETECT = 0x00010000` with the initialization command. As an example, the command to initialize *INS alignment with zero IMU bias* with motion detection disabled is as follows:

```
(INFIELD_CAL_STATE_CMD_INIT_ZERO_ATTITUDE_IMU | INFIELD_CAL_STATE_CMD_INIT_OPTION_DISABLE_MOTION_DETECT);

0x00010005 = (0x00000005 | 0x00010000);
```

3. **Sample Orientation(s)** - Initiate sampling of one or more orientations by setting `DID_INFIELD_CAL.state` to `INFIELD_CAL_STATE_CMD_START_SAMPLE = 8`. Sampling per orientation will take 5 seconds and completion is indicated when `DID_INFIELD_CAL.state` switches to `INFIELD_CAL_STATE_READY_FOR_SAMPLING = 50`.

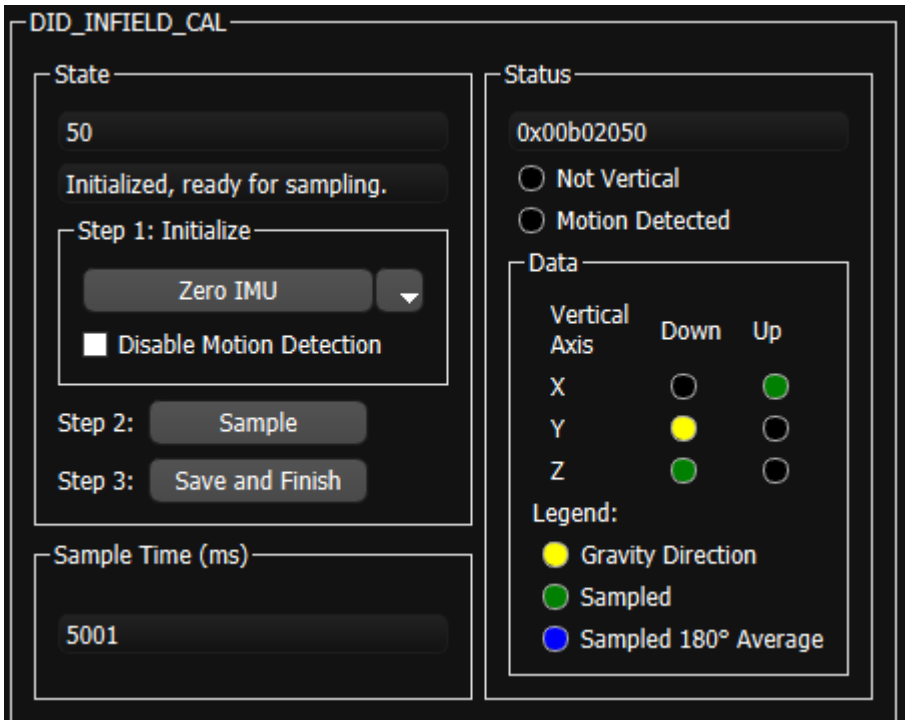
- **Sample Same Orientation w/ +180° Yaw** - If the working surface is not level, two samples per orientation can be taken to cancel out the tilt of the working surface. Rotate the system approximately 180° in yaw (heading) and initiate the sampling a second time for a given orientation.
- **Sample Up to Six Orientations** - The sampling process can be done for up to six orientations (X,Y,Z pointed up and down). Each sample will be automatically associated with the corresponding vertical axis and direction. All orientations will be averaged together for both the zero IMU bias and zero INS attitude.

4. **Store IMU Bias and/or Align INS** - Following sampling of the orientations, set `DID_INFIELD_CAL.state` to `INFIELD_CAL_STATE_CMD_SAVE_AND_FINISH = 9` to process and save the infield calibration to flash memory. The built-in test (BIT) will run once following this to verify the newly adjusted calibration and `DID_INFIELD_CAL.state` will be set to `INFIELD_CAL_STATE_SAVED_AND_FINISHED`.

EVALTOOL OR CLTOOL FOR INFIELD CAL

The EvalTool IMU Settings tab provides a user interface to read and write the DID_INFIELD_CAL message.

©2022



CLTool Infield Cal

The following options can be used with the CLTool to edit the infield calibration (DID_INFIELD_CAL).

```
cltool -c /dev/ttyS2 -edit DID_INFIELD_CAL
```

Below is an example of the CLTool edit view of the DID_INFIELD_CAL message.

```
$ Inertial Sense.  Connected.  Press CTRL-C to terminate.  Rx 13657

(94) DID_INFIELD_CAL:      W up, S down
                     0   calData[2].down.dev[1].acc[1]
                     0   calData[2].down.dev[1].acc[2]
                     0   calData[2].down.yaw
                     0   calData[2].up.dev[0].acc[0]
                     0   calData[2].up.dev[0].acc[1]
                     0   calData[2].up.dev[0].acc[2]
                     0   calData[2].up.dev[1].acc[0]
                     0   calData[2].up.dev[1].acc[1]
                     0   calData[2].up.dev[1].acc[2]
                     0   calData[2].up.yaw
          0.0398919582   imu[0].acc[0]
        0.000717461109   imu[0].acc[1]
            9.67872334   imu[0].acc[2]
         0.00583727891   imu[0].pqr[0]
          0.0135380113   imu[0].pqr[1]
        -0.00342554389   imu[0].pqr[2]
          0.0874974579   imu[1].acc[0]
          -0.167159081   imu[1].acc[1]
            9.67817783   imu[1].acc[2]
        -0.00111889921   imu[1].pqr[0]
        -0.00523020467   imu[1].pqr[1]
         0.00455262465   imu[1].pqr[2]
                     0   sampleTimeMs
                    50   state
           0x00B01000 * status
```

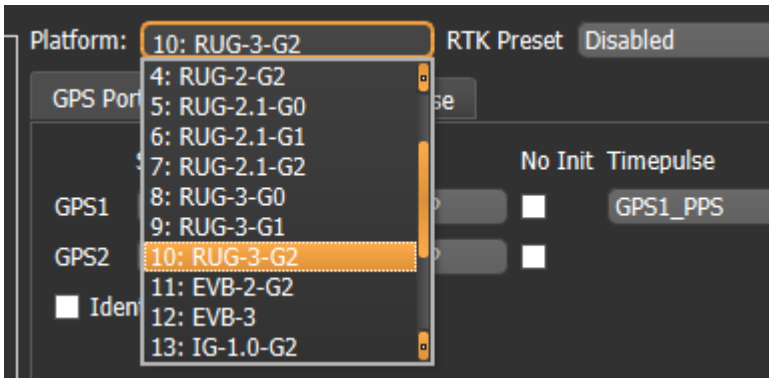

©2022

7.4.3 Platform Configuration

The `DID_FLASH_CONFIG.platformConfig` allows for specification of the IMX carrier board type and configuration settings. This is important and helpful for configuring I/O specific to the platform (carrier board). Values for the Platform Config are specified in the **enum ePlatformConfig** in the SDK data_sets.h.

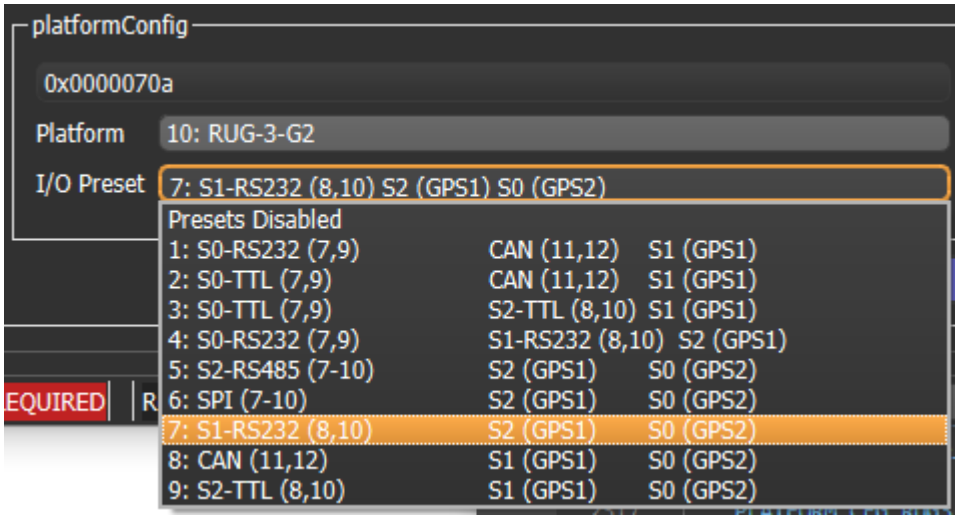
Platform Type

The platform config type can be set through the EvalTool General Settings and GPS Settings tabs. Setting the Platform Config type through the EvalTool acts as a convenience preset that automatically sets the GPS source, type, and timepulse pin selection for the selected platform.



I/O Presets

The pin assignments on the RUG-3 are software configurable using the `PLATFORM_CFG_PRESET_MASK` bits of the `DID_FLASH_CONFIG.platformConfig`. The `PLATFORM_CFG_TYPE` must be set to one of the RUG-3 types to enable the I/O Presets configuration on the RUG-3. The RUG-3 main connector pin numbers are listed in parenthesis in the I/O Preset.



©2022

7.4.4 System Configuration

See the Binary Protocol page for descriptions of each flash configuration value and enumeration bit values.

Serial Port Baud Rates

UART standard baud rates available on the IMX are: 921600, 460800, 230400, 115200, 57600, 38400, 19200, 9600. When operating within the standard baud rate range (<= 921600 bps), only these specific baud rates can be used. Non-standard high speed baud rates (>921600) listed in the following section allow for arbitrary custom baud rates.

HIGH SPEED BAUD RATES

Non-standard high speed UART baud rates (>921600 bps) can be set to arbitrary values up to 10 Mbps. Due to hardware limitations, the applied baud rate will be rounded to the closest available baud rate and reported back via the `DID_FLASH_CONFIG.serXBaudRate` parameter.

BAUD RATE CONFIGURATION

The IMX baud rate can be manually set by changing the following flash configuration parameters:

Configuration	Description
`DID_FLASH_CONFIG.ser0BaudRate`	baud rate for IMX serial port 0
`DID_FLASH_CONFIG.ser1BaudRate`	baud rate for IMX serial port 1
`DID_FLASH_CONFIG.ser2BaudRate`	baud rate for IMX serial port 2

These parameters can be changed using the EvalTool or the CLTool. The following examples show how the EvalTool and CLtool can be used to set the IMX serial port 1 baud rate to 460,800 bps.

`EvalTool -> Data Sets -> DATA_FLASH_CONFIG.ser1BaudRate = 460800`

```
cltool -c COM# -flashConfig=ser0BaudRate=460800
```


©2022

7.4.5 Time Synchronization

INS & GPS Timestamps

The IMX output messages are timestamped using GPS time-base because this time is known immediately following GPS signal reception. Conversion from GPS time to UTC time requires knowledge of the number of leap seconds (GPS-UTC) offset. This value is received periodically (every 12.5 minutes) and is available in the `DID_GNSS1_POS` and `DID_GNSS1_RTK_POS` (gnss_pos_t) messages. GPS leap seconds is 18 seconds as of December 31, 2016 and will change in the future.

The original designers of GPS chose to express time and date as an integer week number (starting with the first full week in January 1980) and a time of week (often abbreviated to TOW) expressed in seconds. Working with time/date in this form is easier for digital systems than the more "conventional" year/month/day, hour/minute/second representation. Most GNSS receivers use this representation internally and converting to a more "conventional form" externally.

GPS TO UTC TIME CONVERSION

UTC time is found by subtracting GPS leap seconds from the GPS time.

GPS Time Synchronization

Systems connected to the IMX can be time synchronized using the GPS PPS timepulse signal and any message containing GPS time. The actual time of the GPS PPS timepulse signal is the same as any message with GPS time rounded down to the second. The following pseudo code illustrates how this is done.

```
// GPS Time Synchronization - Find the difference between local time and GPS time:

// 1. Sample your local time on the rising edge of the GPS PPS timepulse signal.
double ppsLocalTime = localTime();

// 2. Read the GPS time from any message WITHIN ONE SECOND FOLLOWING the GPS PPS timepulse signal.
double gpsTime = readGpsMessageTime();             // within one second after GPS PPS

// 3. Find the difference between the GPS PPS local time and the GPS time rounded down to the
//    nearest second (443178.800 s down to 443178 s, or 443178800 ms down to 443178 s).
double localToGpsTimeTemp = ppsLocalTime - floor(gpsTime);

// 4. Error check to ensure you have a consistent solution
static double localToGpsTimeLast;
double localToGpsTime;

if (fabs(localToGpsTimeLast - localToGpsTimeTemp) < 0.002)        // within 2ms
{
        localToGpsTime = localToGpsTimeTemp;
}
localToGpsTimeLast = localToGpsTimeTemp; // Update history

// Local time can now be converted at anytime to GPS time using 'localToGpsTime' difference.
double currentGpsTime = localTime() + localToGpsTime;
```

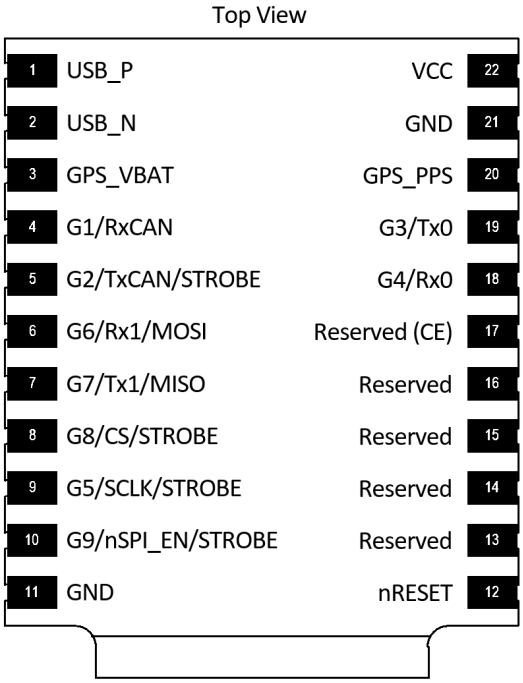
Using the Strobe Input Pins

The IMX has several strobe input pins which can be configured to cause the IMX to report both its internal time and full navigation solution at the moment when triggered.

STROBE I/O EVENTS

Strobe input and output (I/O) events are used for time and data synchronization.

©2022



STROBE pins on the IMX Module - Top View

STROBE INPUT (TIME SYNC INPUT)

Strobe inputs are used to timestamp digital events observed on any of the pins labeled STROBE, e.g. camera shutter signals. A STROBE input event occurs when the logic level of any STROBE pin is toggled. The transition direction can be set so that the STROBE event triggers on a rising edge, or a falling edge. An internal 100K pull-up or pull-down resistor is enabled, depending on the assertion direction. External pull-up or pull-down resistors are not necessary.

The STROBE input will trigger on the edge type specified. However, the minimum period between STROBE input pulses is 1 ms. The measurement and timestamp resolution are both 1 ms.

The following pins can be used for STROBE input.

Signal	Module Pin	EVB-1 Pin	Rugged Pin	EVB-2
G2	5	H2-4	12	H7-6
G5	9	H6-3		H7-9
G8	8	H6-6		H7-12
G9	10	Button "B"		H7-13

To use a pin as a Strobe Input pin, the I/O must be configured as a strobe input. Additionally, the triggering edge must be set using the following bits in `DID_FLASH_CONFIG.ioConfig`.

Bit Name	Bit Value	Description
(IO_CONFIG_STROBE_TRIGGER_HIGH unset)	0x00000000	Trigger strobe on falling edge
IO_CONFIG_STROBE_TRIGGER_HIGH	0x00000001	Trigger strobe on rising edge

Pushbutton “B” on the EVB asserts a logic low to G9 (pin 10) of the IMX and can be used to test the STROBE input functionality.

**Note: Holding pin 9 low at startup enables SPI which uses pins 5 and 8 making them unavailable to be used as Strobe Inputs. If pin 9 is not held low, the internal pullup resistor holds it high at startup. This sets pins 5 and 8 as inputs which can be used as Strobe Inputs.

©2022

A STROBE input event causes a timestamp message and INS2 message to be transmitted. The ASCII messages `$PSTRB` and `$PINS2` messages are sent by default but can be disabled and replaced by the binary messages `DID_STROBE_IN_TIME` and `DID_INS_2` if the RMC bit `RMC_BITS_STROBE_IN_TIME` is set for the given serial port.

Example:

```
rmc_t rmc;
rmc.bits = RMC_BITS_STROBE_IN_TIME;

int messageSize = is_comm_set_data_to_buf(buffer, bufferSize, comm, DID_RMC, sizeof(uint64_t), offsetof(rmc_t, bits), &rmc);
if (messageSize != serialPortWrite(serialPort, comm->buffer, messageSize))
{
    printf("Failed to write save persistent message\r\n");
}
```

Table 2 - DID_STROBE_IN_TIME message transmitted following a SYNC input event.

Field	Type	Description
week	uint32_t	Weeks since January 6th, 1980
timeOfWeekMs	uint32_t	Time of week (since Sunday morning) in milliseconds, GMT.
pin	uint32_t	STROBE input pin
count	uint32_t	STROBE serial index number

The STROBE input event also causes the HDW_STATUS_STROBE_IN_EVENT (0x00000020) bit of the hdwStatus field in INS output (DID_INS_1, DID_INS_2, DID_INS_3, and DID_INS_4) to be set, allowing users to identify strobe input events using the INS output.

Troubleshooting Input Strobe

If the STOBE input does not appear to be functioning properly, an oscilloscope or fast multi-meter can be used to probe the actual STROBE line to ensure the proper 0V to 3.3V voltage swing is present. The following two tests can be used to evaluate the proper function of the strobe source and the IMX strobe input.

TEST 1: Identify if the IMX strobe input is configured properly and has a low input impedance:

1. Disconnect your strobe source from the IMX. Use a 1K ohm resistor as a pull-up resistor between 3.3V and the IMX strobe input and measure the strong input line.
2. Repeat using the resistor as a pull-down resistor between ground and the strobe input and measure the strobe input line.

This will tell if the IMX strobe input is somehow being driven internally or not configured correctly. If it is functioning correctly, the line will toggle from 0V to +3.3V following the resistor pull-down and pull-up.

TEST 2: Identify if your strobe source is driving correctly:

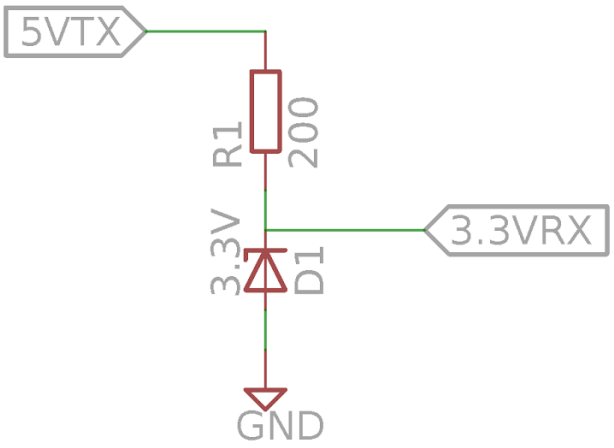
1. With the IMX strobe input disconnected from your strobe driving circuit, probe the output of the strobe driving circuit and observe what levels it toggles between.
2. Attach a 1M ohm pull-down resistor from ground to the strobe output and observe the strobe voltage swing.

If the circuit is working correctly, it should drive the strobe output from 0V to +3.3V despite the 1M ohm pull-down resistor.

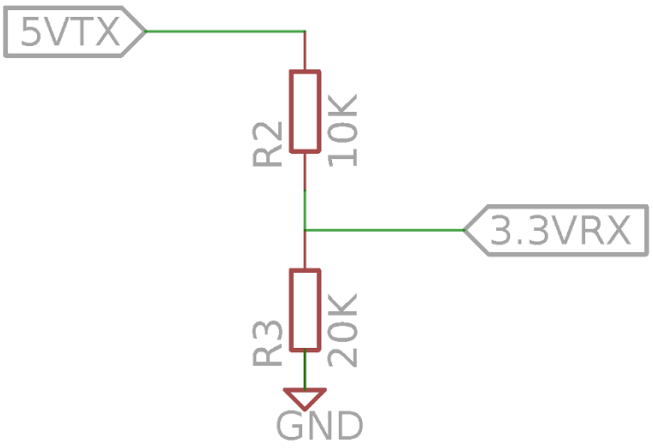
Input Voltage Level Shifter

The maximum input voltage for strobe lines (any pin on the IMX) is 3.6V. A level shifter may be used to convert any strobe signal that is larger than 3.3V. The following figure shows two passive level shifter circuits, a zener diode voltage clamp and a resistor voltage divider.

©2022



With a Zener diode



With a voltage divider

These circuits are beneficial because of their simplicity. An active, powered level shifter may also be used and necessary.

STROBE OUTPUT (PREINTEGRATED IMU PERIOD)

The STROBE output feature generates a 1 ms pulse on pin G9, with the leading edge marking the start of the preintegrated IMU (PIMU) integration period. To enable this output, set the `IO_CONFIG_G9_STROBE_OUTPUT_NAV` bit (0x00000020) in `DID_FLASH_CONFIG.ioConfig`. The polarity of the pulse is configured by setting the `IO_CONFIG_STROBE_TRIGGER_HIGH` bit (0x00000001) in the same field.

CONFIGURING MESSAGE OUTPUT

By default, triggering a strobe input event will cause the IMX to produce an NMEA PINS2 message as well as a PSTRB message which contains the time stamp of the strobe event.

To instead send a binary DID_INS_2 and DID_STROBE_IN_TIME message, set the `RMC_BITS_STROBE_IN_TIME` flag of `DID_RMC/bits` field.

©2022

7.4.6 UART Interface

The IMX has different UART TTL serial ports. These serial ports can be converted from TTL to RS232 or RS422 using a level converter, such as found on the Rugged-3, EVB-1, and EVB-2 carrier boards.

Actual UART Baud Rates

The serial ports use different peripherals so the actual baud rates of the ports differ.

Due to UART limitations, the ***actual baud rate*** that the hardware is capable of generating differs from the target or desired baud rate. This difference is more pronounced at higher baud rates (>921600 bps). The following table outlines these differences.

Target Baud Rate (bps)	**IMX-5** **Actual Baud Rate (bps)**	**IMX-3** **Actual Baud Rate (bps)**
19,200	19,198	19,191
38,400	38,406	38,422
57,600	57,595	57,515
115,200	115,273	115,030
230,400	230,547	231,481
460,800	459,770	457,317
921,600	919,540	937,500
	3,200,000	3,125,000
	4,000,000	3,750,000
	5,000,000	4,687,500
	8,000,000	6,250,000
	10,000,000	9,375,000

IMX-5 UART BAUD RATE EQUATION

The actual baud rate that the IMX-5 hardware is capable of generating is described in the following equation.

Baud rates <= 5 Mbps:

```
Divisor = floor( (80e6 + ((Target Baud Rate)/2)) / (Target Baud Rate) )
Actual Baud Rate = floor( 80e6 / Divisor )
```

Baud rates > 5 Mbps:

```
Divisor = floor( (160e6 + ((Target Baud Rate)/2)) / (Target Baud Rate) )
Actual Baud Rate = floor( 160e6 / Divisor )
```

IMX-3 UART BAUD RATE EQUATION

The actual baud rate that the IMX-5 hardware is capable of generating is described in the following equations.

```
Divisor = floor( (18750000 + ((Target Baud Rate)/2)) / (Target Baud Rate) )
Actual Baud Rate = floor( 18750000 / Divisor )
```


©2022

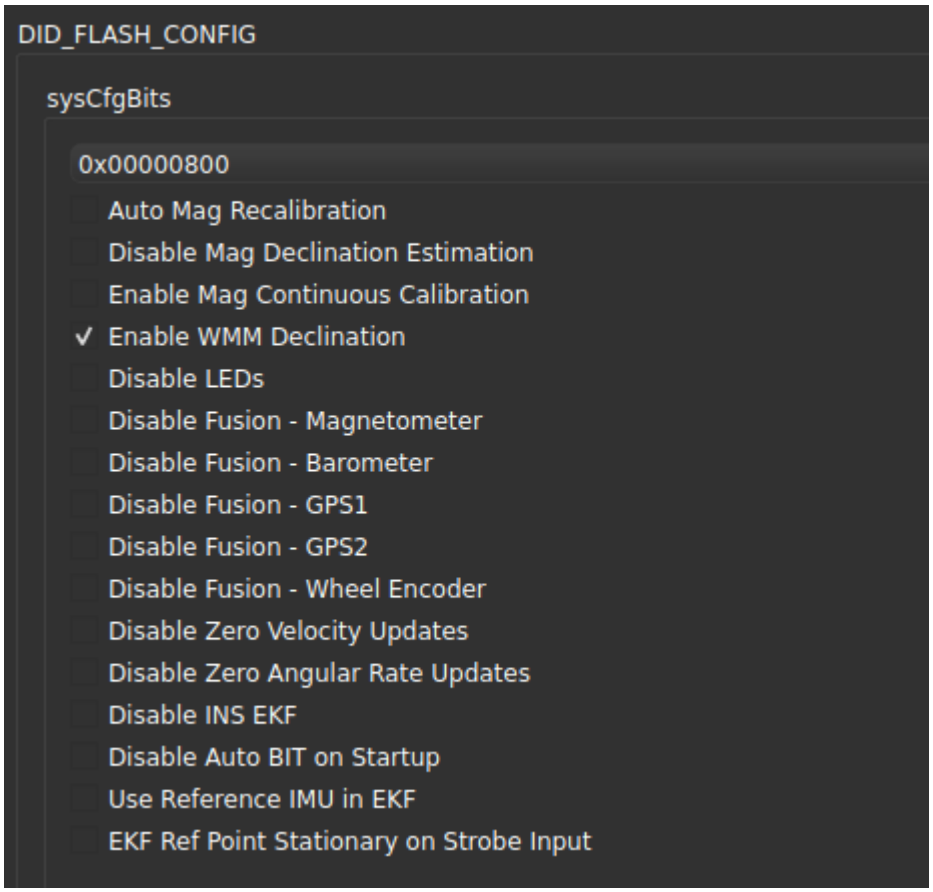
7.4.7 World Magnetic Model (WMM)

The World Magnetic Model (WMM) is a mathematical model that represents the Earth's magnetic field. It is developed by the National Centers for Environmental Information (NCEI) and the British Geological Survey (BGS) and is updated every five years to account for changes in the Earth's magnetic field.

The IMX-5 uses an on-board WMM to estimate the magnetic declination, which is the angle between magnetic north and true north.

Enabling WMM Declination

The WMM declination is enabled by setting bit `SYS_CFG_BITS_MAG_ENABLE_WMM_DECLINATION` in `DID_FLASH_CONFIG.sysCfgBits` on the IMX-5. This can be done in the EvalTool > Settings > General tab by checking the `Enable WMM Declination` option.



Using WMM Declination

When enabled, the WMM declination (magnetic declination) is computed based on the current position of the IMX-5. The current WMM declination can be viewed from the `DID_INL2_STATES.magDec` variable.

AHRS MODE

When running in AHRS mode and no GPS position has previously been available, the reference latitude, longitude position (`DID_FLASH_CONFIG.refLla`) is used to initialize the IMX position and WMM declination. The user must set this `refLla` location for the WMM to generate the correct declination.

NAV MODE

When GPS is available and the IMX is in GNSS aided navigation mode (Nav mode), the WMM declination will continuously update as the current position changes.

©2022

7.4.8 Zero Motion Command

The *Zero Motion Command* is user initiated and informs the EKF that the system is stationary on the ground. It is used to aid in IMU bias estimation which can reduce drift in the INS attitude. It works as a virtual velocity and angular rate sensor to provide velocity and angular rate observations when the INS is stationary (zero velocity and zero angular rate). This is done for a period of two seconds after the *Zero Motion Command* is received. The Zero Motion Command is beneficial for the following reasons:

- Overriding incorrect GPS motion caused by weak GPS signal.
- Speeding up gyro biases convergence time when there is no GPS signal.

In normal AHRS mode (stationary with or without GPS), only the IMU gyro biases are estimated by the EKF. Setting `DID_FLASH_CONFIG.dynamicModel = DYNAMIC_MODEL_STATIONARY (2)` is equivalent to continually issuing the zero motion command.

To use the *Zero Motion Command*:

1. Ensure the system is stationary on the ground.
2. Send the *Zero Motion Command* either once or continuously while the system is stationary. This can be done either by using the *Zero Motion* button in the EvalTool General Settings tab or by sending the DID_SYS_CMD binary message.
3. After sending the *Zero Motion Command*, wait for the INS_STATUS_STATIONARY_MODE status bit to clear in DID_INS_x.insStatus before moving the system. This flag takes about 2 seconds to clear following the last *Zero Motion Command*.

Applying this command more than one time can further improve the IMU bias estimation.

Warning

Issuing the *Zero Motion Command* while the system is moving can cause incorrect IMU bias estimates and lead to poor INS performance. It is important to make sure that the system is stationary when using the *Zero Motion Command*.

©2022

7.5 IMX System Status

7.5.1 Solution Status

Solution Alignment

Solution alignment occurs when aiding sensor and state estimation are in agreement and indicates that solution output can be trusted.

HEADING ALIGNMENT

Heading alignment varies based on available heading aiding sensors and conditions of motion.

`INS_STATUS_HDG_ALIGN_COARSE` and `INS_STATUS_HDG_ALIGN_FINE` flags indicate whether INS heading is aided by any heading sensor (including GPS or magnetometer). More accurate heading sensors (i.e. GPS) are prioritized over less accurate sensors (i.e. magnetometers) and will fall back to the less accurate sensors when the more accurate sensors are not available. A momentary blip in these alignment flags may occur during heading transition from higher to lower accuracy aiding sensors (i.e. GPS to magnetometer). `INS_STATUS_HDG_ALIGN_FINE` and `INS_STATUS_HDG_ALIGN_COARSE` flags will not be set when no heading aiding is available.

Heading aiding status is also identified when the `INS_STATUS_SOLUTION_MASK` of the `insStatus` equals:

- `INS_STATUS_SOLUTION_NAV`
- `INS_STATUS_SOLUTION_NAV_HIGH_VARIANCE`
- `INS_STATUS_SOLUTION_AHRS`
- `INS_STATUS_SOLUTION_AHRS_HIGH_VARIANCE`

The source of heading aiding can be identified by the following flags:

- `INS_STATUS_GNSS_AIDING_HEADING` - either dual or single GPS.
- `INS_STATUS_MAG_AIDING_HEADING` - when GPS heading is not available.

Status Flags

This section lists the commonly used status flags. A complete listing of status flags is available in data_sets.h.

©2022

INS STATUS FLAGS (INSSTATUS)

The INS status flags, **insStatus**, are found in the DID_INS_1, DID_INS_2, DID_INS_3, and DID_SYS_PARAMS messages. Bitmasks for the **insStatus** flags are defined in *eInsStatusFlags* in data_sets.h.

©2022

Flag	Description
INS_STATUS_HDG_ALIGN_COARSE	Heading estimate is usable but outside spec.
INS_STATUS_HDG_ALIGN_FINE	Heading estimate is within spec.
INS_STATUS_VEL_ALIGN_COARSE	Velocity estimate is usable but outside spec.
INS_STATUS_VEL_ALIGN_FINE	Velocity estimate is within spec.
INS_STATUS_POS_ALIGN_COARSE	Position estimate is usable but outside spec.
INS_STATUS_POS_ALIGN_FINE	Position estimate is within spec.
INS_STATUS_GNSS_AIDING_HEADING	INS heading is being corrected by GPS.
INS_STATUS_GNSS_AIDING_POS	INS position and velocity are being corrected by GPS.
INS_STATUS_GNSS_UPDATE_IN_SOLUTION	GPS update event occurred in INS, potentially causing discontinuity in position path.
INS_STATUS_MAG_AIDING_HEADING	INS heading is being corrected by magnetometer.
INS_STATUS_NAV_MODE	AHRS = 0 (no position or velocity), NAV = 1
INS_STATUS_MAG_RECALIBRATING	Magnetometer is recalibrating.
INS_STATUS_MAG_INTERFERENCE_OR_BAD_CAL_OR_NO_CAL	Magnetometer is experiencing interference or calibration is bad. Attention may be required to remove interference (move the device) or recalibrate the magnetometer.
INS_STATUS_SOLUTION_MASK	0=INS_STATUS_SOLUTION_OFF – The INS is not running **1=INS_STATUS_SOLUTION_ALIGNING** – The INS is aligning on startup **3=INS_STATUS_SOLUTION_NAV** – The INS is in NAV mode and the state estimate is good. **4=INS_STATUS_SOLUTION_NAV_HIGH_VARIANCE** – The INS is in NAV mode and the state estimate is experiencing high variance. This may be caused by excessive noise on one or more sensors, such as vibration, magnetic interference, poor GPS sky visibility and/or GPS multipath errors. See DID_INL2_VARIANCE. **5=INS_STATUS_SOLUTION_AHRS** – INS is in AHRS mode and the solution is good. There is no valid position correction data from GPS or other aiding sensor. Only the attitude states are estimated. **6=INS_STATUS_SOLUTION_AHRS_HIGH_VARIANCE** – INS is in AHRS mode and the state estimate has high variance. See DID_INL2_VARIANCE. **7=INS_STATUS_SOLUTION_VRS** - System is in VRS mode (no earth relative heading) and roll and pitch are good. **8=INS_STATUS_SOLUTION_VRS_HIGH_VARIANCE** System is in VRS mode (no earth relative heading) but roll and pitch uncertainty has exceeded the threshold.
INS_STATUS_RTK_COMPASSING_MASK	**0x00100000=INS_STATUS_RTK_COMPASSING_BASELINE_UNSE** - GPS compassing antenna offsets are not set in flashCfg. **0x00200000=INS_STATUS_RTK_COMPASSING_BASELINE_BAD** - GPS antenna baseline specified in flashCfg and measured by GPS do no match.
INS_STATUS_GNSS_NAV_FIX_MASK	GPS navigation fix type (see eGnssNavFixStatus)
INS_STATUS_RTK_COMPASSING_VALID	RTK compassing heading is accurate. (RTK fix and hold status)
INS_STATUS_RTK_ERROR_MASK	See eInsStatusFlags in data_sets.h.

©2022

HARDWARE STATUS FLAGS (HDWSTATUS)

The hardware status flags, **hdwStatus**, are found in the `DID_INS1`, `DID_INS2`, `DID_INS3`, and `DID_SYS_PARAMS` messages. Bitmasks for the **hdwStatus** flags are defined in `eHdwStatusFlags` in data_sets.h.

Field	Description
HDW_STATUS_MOTION_MASK	Unit is moving and NOT stationary
HDW_STATUS_GNSS_SATELLITE_RX_VALID	Antenna is connected to the GPS receiver and signal is valid. Unset indicates weak signal or no output from 1 or more enabled GPS receivers
HDW_STATUS_STROBE_IN_EVENT	Event occurred on strobe input pin
HDW_STATUS_GNSS_TIME_OF_WEEK_VALID	GPS time of week is valid and reported. Otherwise the timeOfWeek is local system time.
HDW_STATUS_SATURATION_MASK	Acc., Gyro, Mag or Baro is saturated
HDW_STATUS_SYSTEM_RESET_REQUIRED	System Reset is required for proper function
HDW_STATUS_ERR_GNSS_PPS_NOISE	GPS PPS timepulse signal has noise and occurred too frequently
HDW_STATUS_MAG_RECAL_COMPLETE	Magnetometer recalibration has finished (when INS_STATUS_MAG_RECALIBRATING is unset).
HDW_STATUS_ERR_COM_TX_LIMITED	Communications Tx buffer limited
HDW_STATUS_ERR_COM_RX_OVERRUN	Communications Rx buffer overrun
HDW_STATUS_ERR_NO_GNSS_PPS	GPS PPS timepulse signal has not been received or is in error
HDW_STATUS_GNSS_PPS_TIMESYNC	Time synchronized by GPS PPS
	(BIT) Built-in self-test mask
HDW_STATUS_ERR_TEMPERATURE	Outside of operational range
HDW_STATUS_RESET_CAUSE_BACKUP_MODE	Reset from backup mode (low-power state w/ CPU off)
HDW_STATUS_RESET_CAUSE_WATCHDOG_FAULT	Reset from watchdog fault
HDW_STATUS_RESET_CAUSE_SOFT	Reset from software
HDW_STATUS_RESET_CAUSE_HDW	Reset from hardware (NRST pin low)

7.5.2 Built-in Test (BIT)

Built-in test (BIT) is enabled by setting `DID_BIT.command` to any of the following values.

```
BIT_CMD_FULL_STATIONARY              = (int)2, // (FULL) Comprehensive test.  Requires system be completely stationary without vibrations.
BIT_CMD_BASIC_MOVING                 = (int)3, // (BASIC) Ignores sensor output.  Can be run while moving.  This mode is automatically run after bootup.
BIT_CMD_FULL_STATIONARY_HIGH_ACCURACY = (int)4, // (FULL-HA) Same as BIT_CMD_FULL_STATIONARY but with higher accuracy requirements.
```

BIT takes about 5 seconds to run, and is completed when `DID_BIT.state == BIT_STATE_DONE (1)`. All BIT tests except those related to GPS require the system to be stationary to be accurate.

©2022

BIT Hardware Flags (hdwBitStatus)

Hardware BIT flags are contained in **hdwBitStatus**, found in the `DID_BIT` message. Bitmasks for the **hdwBitStatus** flags are defined in `eHdwBitStatusFlags` in data_sets.h.

Field	Description
HDW_BIT_PASSED_ALL	All HBIT are passed
HDW_BIT_PASSED_NO_GNSS	All Self Tests passed without GPS signal
HDW_BIT_FAILED_MASK	One of the built-in tests failed
HDW_BIT_FAULT_GNSS_NO_COM	No GPS Signal
HDW_BIT_FAULT_GNSS_POOR_CNO	Poor GPS signal. Check Antenna
HDW_BIT_FAULT_GNSS_POOR_ACCURACY	Poor GPS Accuracy or Low number of satellites

calBitStatus – Calibration BIT Flags

Calibration BIT flags are contained in **calBitStatus**, found in the `DID_BIT` message. Bitmasks for the **calBitStatus** flags are defined in `eCalBitStatusFlags` in data_sets.h.

Field	Description
CAL_BIT_PASSED_ALL	Passed all calibration checks
CAL_BIT_FAILED_MASK	One of the calibration checks failed

©2022

7.5.3 Health Monitoring

This section illustrates tests used for system health monitoring in common application.

1. **Communication Check**

   To check that cabling between the unit and the application is working after initialization, expect that the initial expected packets are received within 3 seconds.

2. **Sensor Test (Must be Stationary)**

   These tests are ideal for manufacturing and periodic in-field testing. Initiate by setting `DID_BIT.command = 2`.

Test	Description
hdwBitStatus & HDW_BIT_PASSED_ALL	Hardware is good
calBitStatus & CAL_BIT_PASSED_ALL	Sensor calibration is good

1. **GPS Hardware Test**

   Initiate by setting `DID_BIT.command = 2`.

Test	Description
hdwBitStatus & HDW_BIT_FAULT_GNSS_NO_COM	No GPS serial communications.
hdwBitStatus & HDW_BIT_FAULT_GNSS_POOR_CNO	Poor GPS signal strength. Check antenna.

1. **GPS Lock Test**

Test	Description
insStatus & INS_STATUS_GNSS_AIDING_POS	GPS is being fused into INS solution

1. **INS Output Valid**

Test	Description
insStatus & INS_STATUS_HDG_ALIGN_FINE	Attitude estimates are valid
insStatus & INS_STATUS_VEL_ALIGN_FINE	Velocity estimates are valid
insStatus & INS_STATUS_POS_ALIGN_FINE	Position estimates are valid

1. **System Temperature**

   System temperature is available at DID_SYS_SENSORS.temp.

2. **Communications Errors**

   HDW_STATUS_COM_PARSE_ERROR_COUNT(DID_SYS_PARAMS.hdwStatus) is the number of parsed packet errors encountered.

©2022

7.6 Magnetometer

The magnetometer is used to estimate heading when the system is in any of the following conditions:

1. is in AHRS mode
2. has no GPS fix
3. has GPS fix and constant velocity (non-accelerating motion)

To have accurate heading under these conditions, the magnetometer must be calibrated. The system allows for two types of modes for recalibration, external initiated and automatically initiated re-calibration. Regardless of the recalibration mode, a slower online background calibration runs that continuously improves the magnetometer calibration to handle local magnetic environment changes. All magnetometer calibration is stored in flash memory and available on bootup.

7.6.1 Disable Magnetometer Updates

Magnetometer fusion into the INS and AHRS filter can be disabled by setting bit `SYS_CFG_BITS_DISABLE_MAGNETOMETER_FUSION` (0x00001000) of `DID_FLASH_CONFIG.sysCfgBits`.

7.6.2 Magnetometer Recalibration

Occasionally the magnetometer will require a complete recalibration, replacing the old calibration with an entirely new calibration. This is accomplished either through external or automatic initiated recalibration. Use of the different modes is generally governed by the particular use case for the end customer and is intended to allow for the most flexibility in an integrated product design.

External Recalibration

External magnetometer recalibration allows the most flexibility in determining when an end user will need to recalibrate the system. This control over the timing of the recalibration is critical for many use cases and allows product designers to implement their desired workflows for customers. Further there are use cases where automatic recalibration is not possible because the quality of the magnetometer calibration is not observable. Such use cases would include AHRS operation, extended periods without motion or no GNSS fix. External magnetometer recalibration, as the name suggests is triggered by an external command from the application managing the IMX hardware. The IMX provides a set of status messages indicating the quality of the magnetometer calibration and leaves the timing and implementation of a recalibration up to the product designer. Specifically, `INS_STATUS_MAG_INTERFERENCE_OR_BAD_CAL_OR_NO_CAL` is an indication of the quality of the magnetometer calibration (see system status flags for details).

During the calibration process, the system should be clear of steel, iron, magnets, or other ferrous materials (i.e. steel desks, tables, building structures). The IMX should be attached to the system in which it is operating and rotated together during the calibration process. The following is the

Force magnetometer recalibration procedure:

1. Set `DID_MAG_CAL.state` to either: * `MAG_CAL_STATE_MULTI_AXIS` (1) for Multi-Axis which is more accurate and requires 360º rotation about two different axes. * `MAG_CAL_STATE_SINGLE_AXIS` (2) for Single-Axis which is less accurate and requires 360º rotation about one axis.
2. Rotate the system accordingly.

Recalibration completion is indicated by either:

1. `INS_STATUS_MAG_RECALIBRATING` bit of the insStatus word set to zero. The insStatus word is found in standard INS output messages (`DID_INS_1`, `DID_INS_2`, `DID_INS_3`, and `DID_INS_4`).
2. `DID_MAG_CAL.progress` is 100.

©2022

Recalibration progress, indicated as a percentage (0-100%), can be observed in the variable `DID_MAG_CAL.progress`. The recalibration process can be canceled and the prior calibration restored anytime by setting `DID_MAG_CAL.state` = `MAG_CAL_STATE_ABORT` (101).

The “Mag used” indicator in the EvalTool INS tab will be green when magnetometer data is being fused into the solution, black when not being fused into the solution, and red during recalibrating.

Example code:

```
#include "com_manager.h"
// Set DID_MAG_CAL.state = 1 for multi-axis recalibration
int32_t value = MAG_CAL_STATE_MULTI_AXIS;
sendDataComManager(0, DID_MAG_CAL, &value, 4, offsetof(mag_cal_t, state));
// Enable broadcast of DID_MAG_CAL.progress every 100ms to observe the percent complete
sendDataComManager(0, DID_MAG_CAL, 0, sizeof(mag_cal_t), 100);
```

Automatic Recalibration

Automatic magnetometer recalibration is useful for systems where user intervention to start external calibration is not convenient or practical. In this mode, the solution will determine that the system needs to be recalibrated and will attempt to do so while in normal operation. In the period while the system is recalibrating, the uncalibrated magnetometer data is used to prevent the INS heading from drifting but it does not provide heading measurements to the state estimator. This feature is enabled by setting bit `SYS_CFG_BITS_AUTO_MAG_RECAL` (0x00000004) of `DID_FLASH_CONFIG.sysCfgBits` non-volatile word.

```
// Enable automatic magnetometer calibration.
DID_FLASH_CONFIG.sysCfgBits |= SYS_CFG_BITS_AUTO_MAG_RECAL;
// Disable automatic magnetometer calibration.
DID_FLASH_CONFIG.sysCfgBits &= ~SYS_CFG_BITS_AUTO_MAG_RECAL;
```

7.6.3 Magnetometer Continuous Calibration

To mitigate the need for recalibration (completely replace calibration data), continuous calibration improves the magnetometer calibration slowly over time. Continuous calibration always runs in the background.

7.6.4 Magnetometer Calibration Settings

The magnetometer calibration algorithm can produce higher quality calibrations when more data is collected across multiple axes of rotation. However, there are use cases where data collection beyond a single axis is impractical if not impossible. To address this issue there is a setting in the flash to configure the data requirement threshold for magnetometer calibration. The available settings include:

- Single Axis Calibration – This setting requires a full rotation in the yaw axis (relative to earth) to determine the calibration. Additional data that is collected via motion on other axes is used but not required. Once a full rotation is completed the calibration is calculated and the online continuous calibration is started.
- Multi Axis Calibration – This setting requires data to be collected across at least two axes, where one is the yaw axis. This calibration mode does not have any specific angular rotational requirements in any given axes, but it does require that data has been collected across a sufficient angular span. There is an indicator (**mag_cal_threshold_complete**) in the `DID_SYS_PARAMS` message that relates the total percent of the required threshold that has been collected. As more data is collected this value will increment to 100% at which point the calibration will be calculated and the online continuous calibration will continue to run

```
/*! Magnetometer recalibration. 0 = multi-axis, 1 = single-axis */
SYS_CFG_BITS_MAG_RECAL_MODE_MASK = (int)0x00000700,
SYS_CFG_BITS_MAG_RECAL_MODE_OFFSET = 8,
```


©2022

7.7 Dead Reckoning

7.7.1 Ground Vehicle Dead Reckoning

Overview

The IMX inertial navigation integrates IMU data to dead reckon (estimate position and velocity) when GPS position fix is not available. The amount of position error during dead reckoning can vary based on several factors including system runtime, motion experienced, and sensor bias stability.

Knowledge about the vehicle's kinematic constraints is applied to reduce drift and improve position estimation.

Installation

> **Important**
>
> It is critical to ensure the IMX remains fixed relative to the vehicle. Any shift or change in the IMX location relative to the vehicle will result in degraded or inaccurate dead reckoning solution.

> **Important**
>
> Heavy vibrations can degrade the IMX measurements and dead reckoning solution.

1. Mount the IMX and GNSS antenna at fixed locations on the vehicle.
2. Set the GPS antenna offsets relative to the IMX origin in meters. EvalTool > Data Sets > DID_FLASH_CONFIG > gnss1AntOffsetX/Y/Z.

ENABLING

Dead reckoning is enabled by setting the `DID_FLASH_CONFIG.dynamicModel` to 4 for ground vehicles. This is done automatically during Learning Mode and stored to flash memory.

LEARNING MODE

Learning mode is be used following installation or any change in the IMX position relative to the vehicle. Learning is used to estimate the vehicle kinematic calibration which is used during normal operation.

Learning Mode Instructions

1. `Start` learning mode.
2. Drive with sufficient motion for learning. This is identified with the EvalTool `GV: Cal` indicator in the INS tab turns GREEN (`DID_GROUND_VEHICLE.status & GV_STATUS_LEARNING_CONVERGED` is not zero). Either of the following patterns is typically adequate.

- At least 200 meters straight, 5 left turns (+90 degrees) and 5 right turns
- Three figure eight patterns.

3. `Stop` learning and save kinematic calibration to flash memory.

Using the EvalTool

©2022

From the EvalTool INS tab:

1. Press the `GV:` button to reveal the ground vehicle options.
2. Press the `Start` button to clear and start learning.
3. Press the `Stop` button to stop learning and save kinematic calibration to flash memory.

Using the DID_GROUND_VEHICLE Message

1. Enable learning mode by setting the `DID_GROUND_VEHICLE.mode` to any of the following commands. The `DID_GROUND_VEHICLE.mode` value will toggle to 1 indicating the system is in learning mode and 0 to indicate learning mode is off.

   2 "Start" - Start with user supplied values in the `DID_GROUND_VEHICLE.transform` and enable learning mode.

   3 "Resume" - Start with the existing calibration and enable learning mode.

   4 "Clear & Start" - Set transform to zero and start with aggressive learning mode. This is the same as the "Start" button in the EvalTool INS tab.

   5 "Stop & Save" - End learning mode and save kinematic calibration to flash memory.
2. Disable learning and save kinematic calibration to flash memory by setting `DID_GROUND_VEHICLE.mode` to 5.

WHEEL ENCODER INPUT (OPTIONAL)

Wheel encoder input is optional for dead reckoning operation. The implementation of wheel encoder input will improve dead reckoning performance. Encoders constrain drift along the axis of travel. A quadrature wheel encoder can be connected directly to the IMX-5's G5/G8 I/O pins (`IO_CONFIG_G5G8_QDEC_INPUT` in `DID_FLASH_CONFIG.ioConfig`). Alternatively, an external system can generate `DID_WHEEL_ENCODER` messages that are then streamed into an IMX-5 port.

©2022

DID_WHEEL_ENCODER struct is defined below. Only omega_l and omega_r are required to be populated. All other value can be left as zero. Once populated, the full struct is streamed to the IMX-5 using the Set Data method from the SDK.

```
/** (DID_WHEEL_ENCODER) Message to communicate wheel encoder measurements to GPS-INS */
typedef struct PACKED
{
    /** (Do not use, internal development only) Time of measurement in current GPS week */
    double timeOfWeek;

    /** Status */
    uint32_t status;

    /** (Do not use, internal development only) Left wheel angle (rad) */
    float theta_l;

    /** (Do not use, internal development only) Right wheel angle (rad) */
    float theta_r;

    /** Left wheel angular rate (rad/s). Positive when wheel is turning toward the forward direction of the vehicle. Use WHEEL_CFG_BITS_DIRECTION_REVERSE_LEFT
in DID_FLASH_CONFIG::wheelConfig to reverse this. */
    float omega_l;

    /** Right wheel angular rate (rad/s). Positive when wheel is turning toward the forward direction of the vehicle. Use
WHEEL_CFG_BITS_DIRECTION_REVERSE_RIGHT in DID_FLASH_CONFIG::wheelConfig to reverse this. */
    float omega_r;

    /** (Do not use, internal development only) Left wheel revolution count */
    uint32_t wrap_count_l;

    /** (Do not use, internal development only) Right wheel revolution count */
    uint32_t wrap_count_r;

    /** (rad^2/s^2) Wheel encoder velocity noise variance */
    float var_wheel_omega;

    /** (rad^2) Wheel encoder angle noise variance */
    float var_wheel_theta;

} wheel_encoder_t;
```

Examples

Dead reckoning examples can be found here.

©2022

7.7.2 IMX Dead Reckoning Examples

Dead Reckoning is the process of calculating the current position of a moving object by using a previously determined position, or fix, and then incorporating estimations of speed, heading direction, and course over elapsed time. Knowledge about the vehicle's kinematic constraints (i.e. wheels on the ground) is applied to reduce drift and improve position estimation.

Inertial Sense has added dead reckoning capability to IMX to estimate position for extended periods of time during GNSS outages. In this report RTK-GNSS is used.

The following are examples dead reckoning of a car test vehicle. No wheel sensors were used in these examples. The dead reckoning position is shown in the yellow "INS" line and GNSS position in the red "GNSS" line.

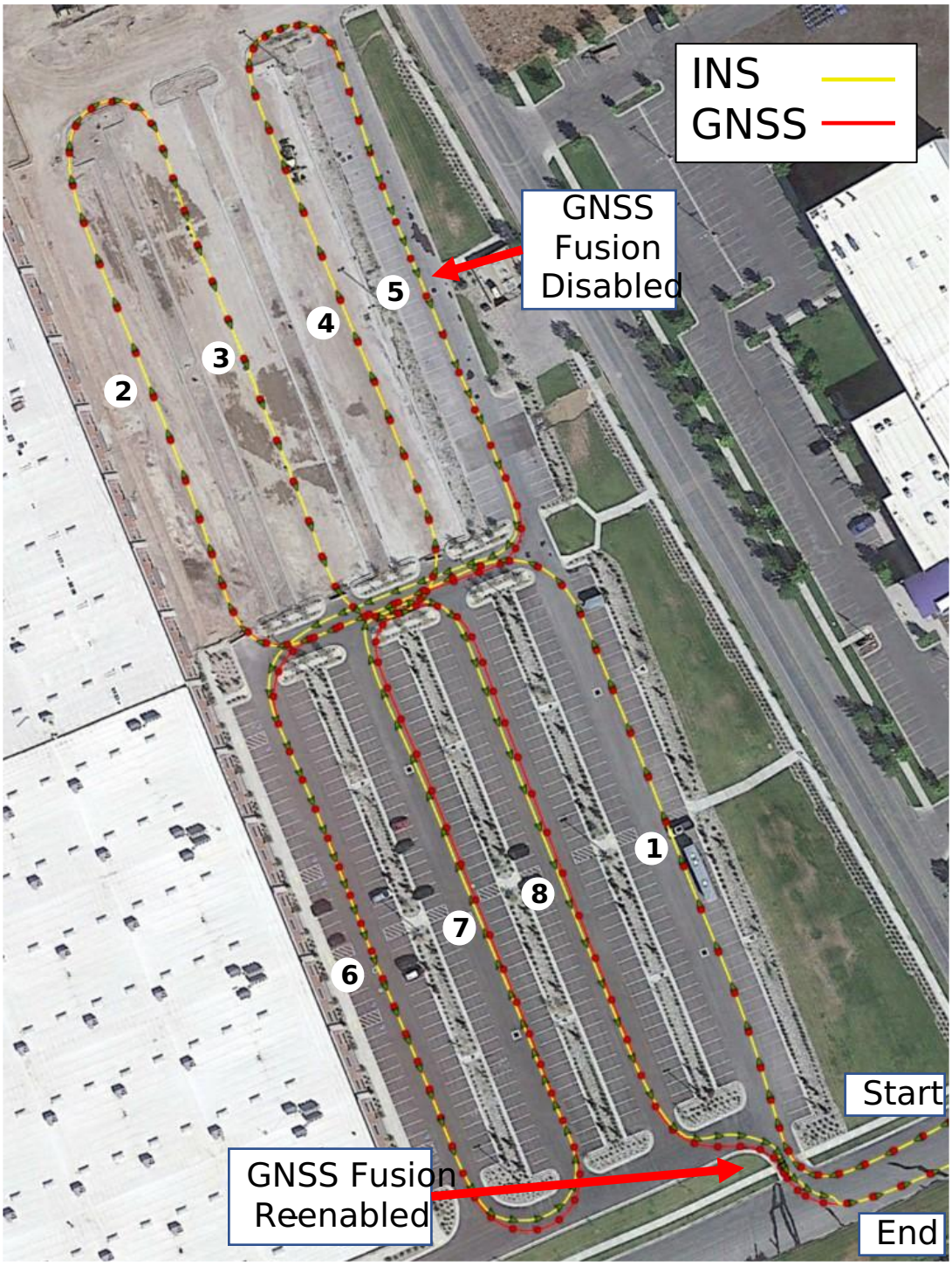
Parking Lot Simulated GNSS Outage

In this example GNSS outage was simulated by disabling GNSS fusion into the INS Kalman filter (EKF). This was done by setting the `Disable Fusion - GPS1` option found in the General settings of the EvalTool app. By disabling GPS fusion and keeping fix, we can use the GNSS position as truth and compare it to the dead reckoning solution.

Dead reckoning duration: **30 seconds**, **605 meters**

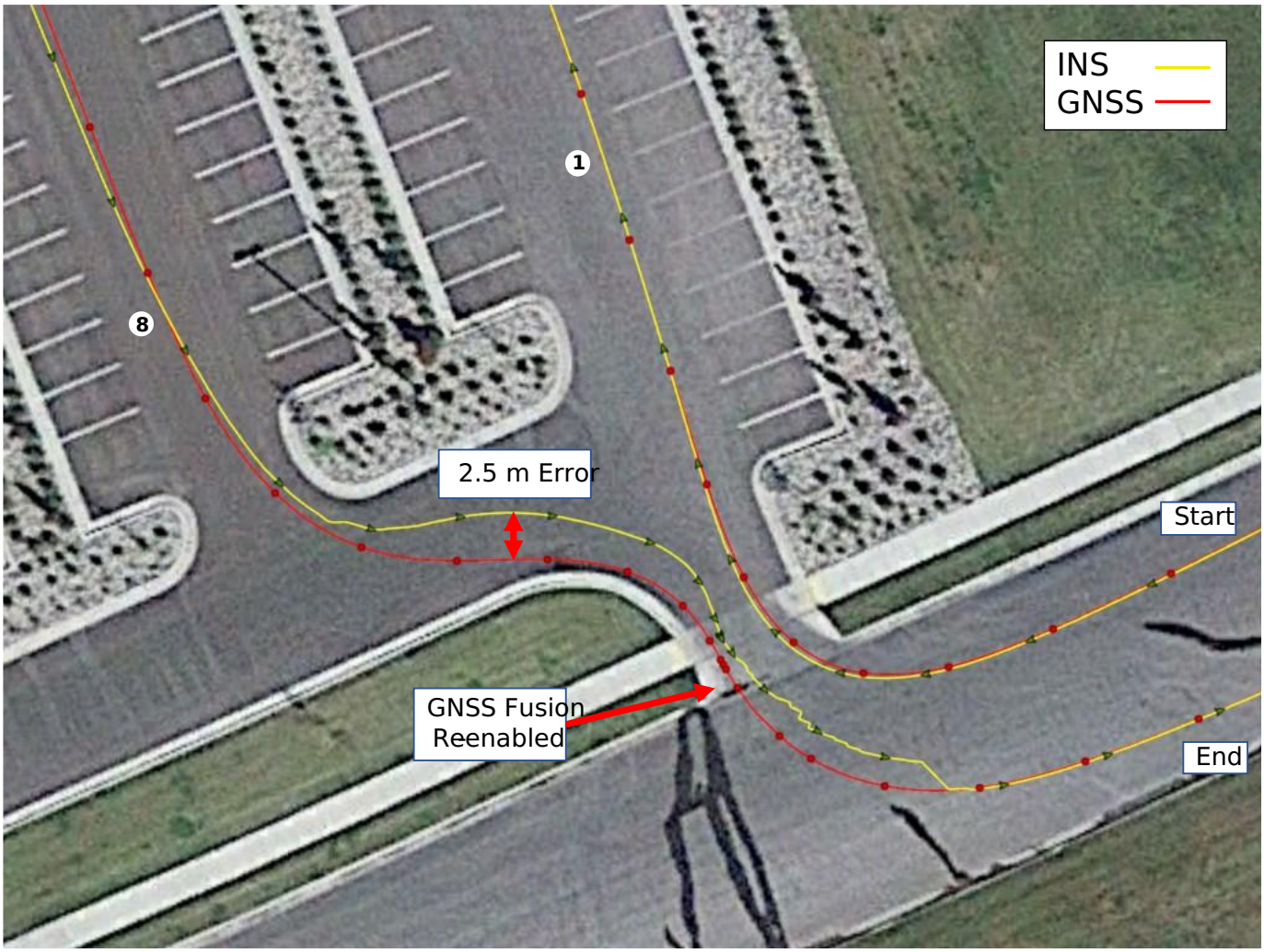
Max position error: **2.5 meters**, **0.4% drift**

©2022



In the drive the car starts and ends the drive at the bottom right corner of the image. The numbered path segments show the order of travel. GPS fusion was disabled in the middle of path segment 5.

©2022



When GNSS fusion is re-enabled, error in the INS solution is removed and the INS position estimate jumps back onto the GNSS position. There is 2.5m of error between the dead reckoning position and the GNSS position.

Multi-Level Parking Garage

In this example our test vehicle drove in and out of a parking garage. The drive consisted of starting outside with GNSS fix, entering the garage (losing GNSS fix), driving up one level, parking, and then following the path back down and out of the garage where GNSS fix was regained.

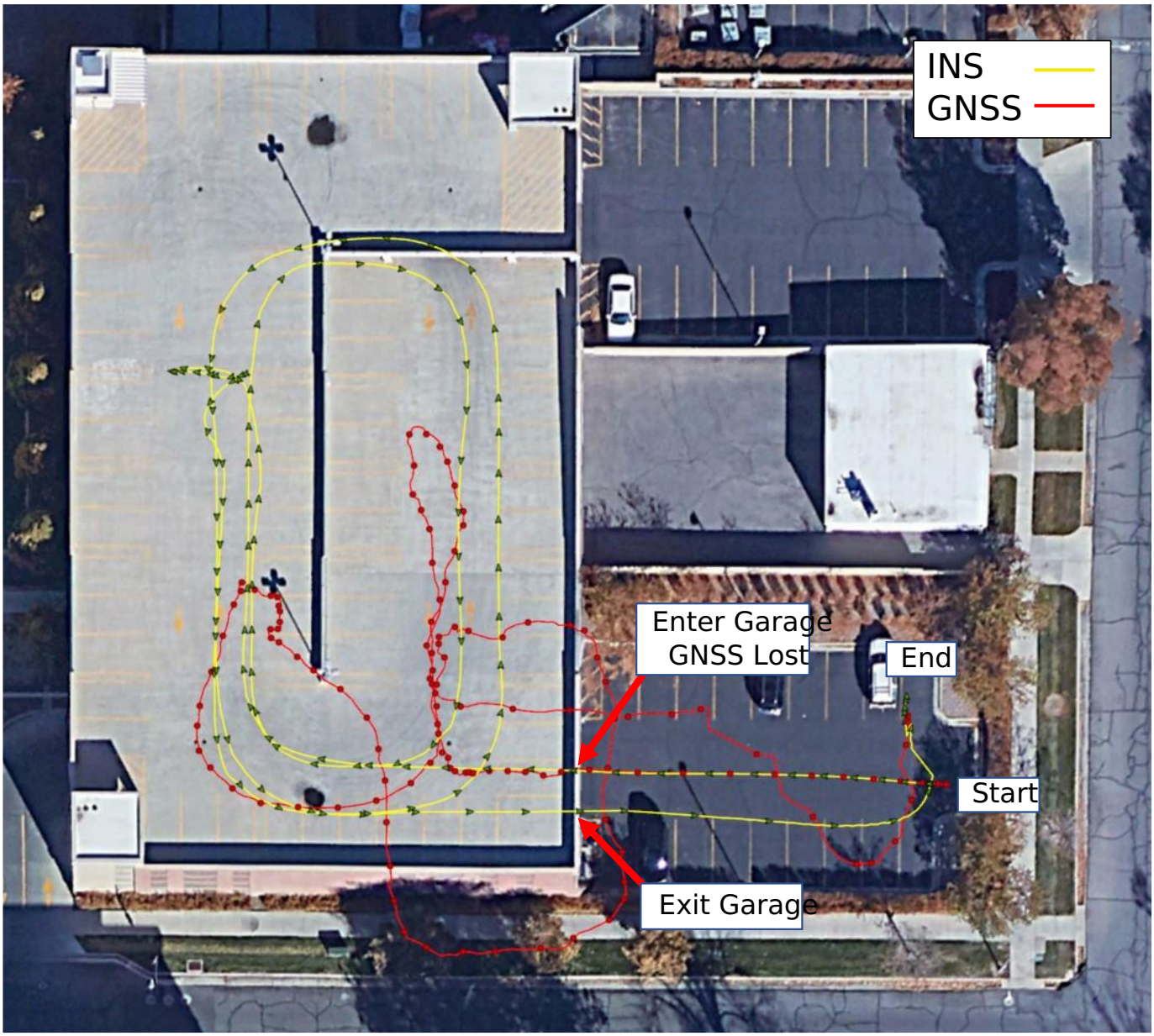
Dead reckoning duration: **105 seconds**, **349 meters**

Exit position error: **~2 meters**, **0.6% drift**

©2022

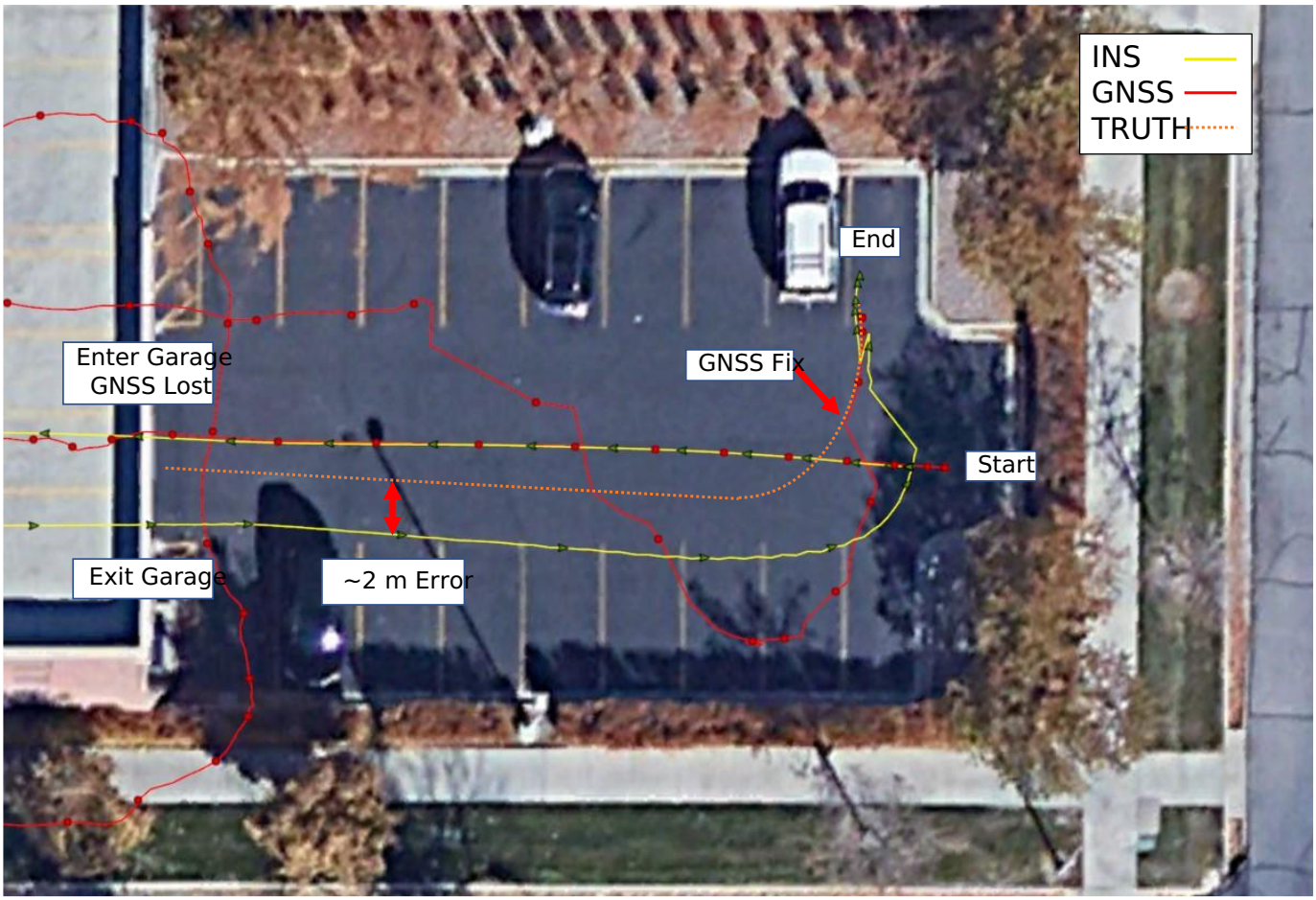
Here we see outside parking lot where the test vehicle started and ended. GNSS fix was lost upon entry of the garage and regained several seconds after exiting the garage.

©2022



Above is the top view of the parking garage. When inside the garage, the GNSS fix is lost shown by the red line erratic deviation. The dead reckoning (INS) position shown by the yellow line matches the actual driven path.

©2022



GNSS fix was not regained until about 20 meters after exiting the garage, just prior to parking at the top right corner of the outside parking lot. The actual position is shown by the orange truth dotted line. GNSS position is shown by the red line and dead reckoning by the yellow INS line. GNSS fix occurs when the red GNSS line jumps and joins the orange truth dotted line. When exiting the garage, the position error was approximately 2 meters following 105 seconds of dead reckoning from GNSS outage.

Conclusion

The IMX with dead reckoning and without wheel sensor can estimate position to within ~3m over 100 seconds of typical automotive parking lot driving.

INERTIALSENSE
autonomous navigation solutions

©2022

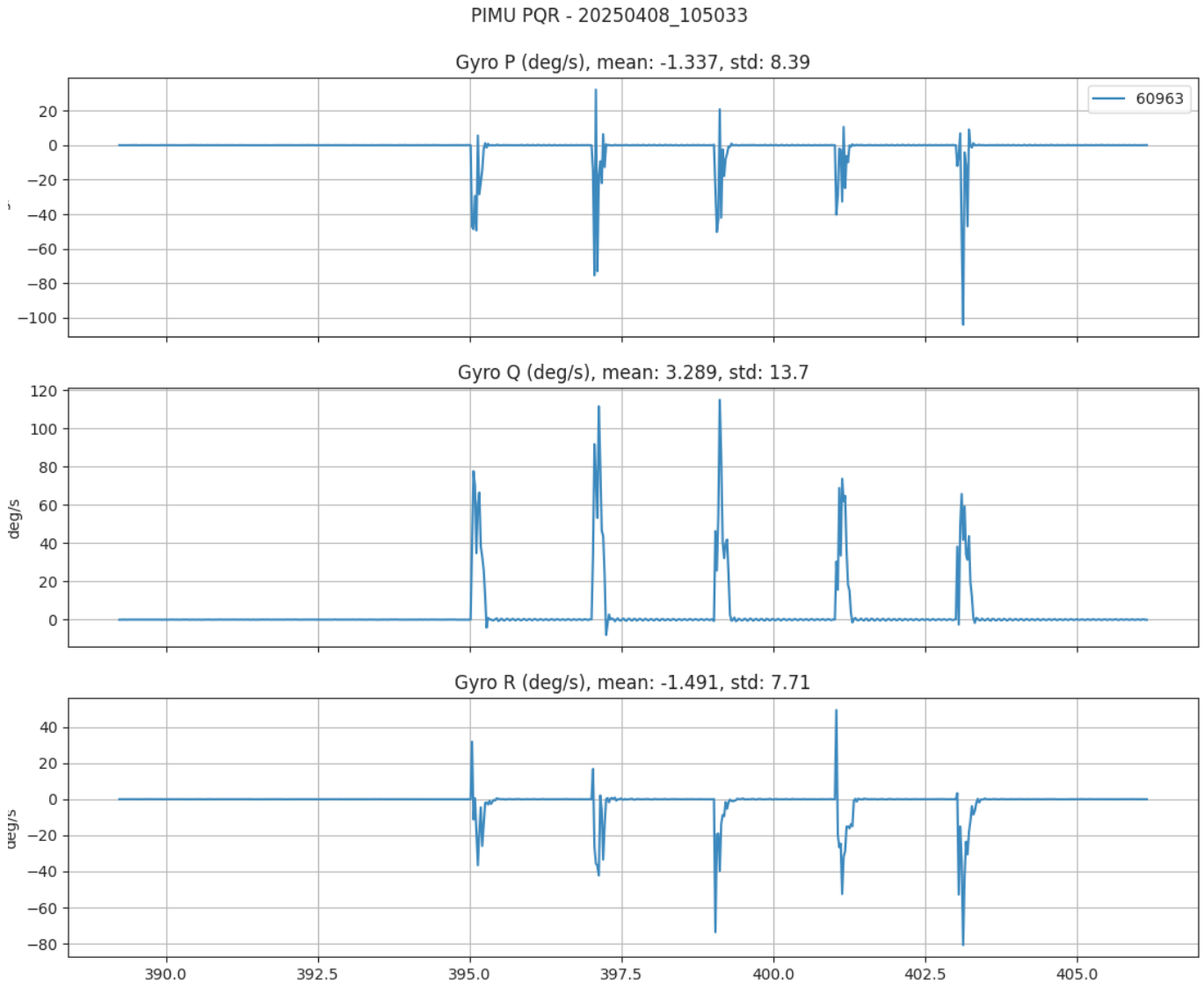
7.8 IMU Shock Rejection

7.8.1 Overview

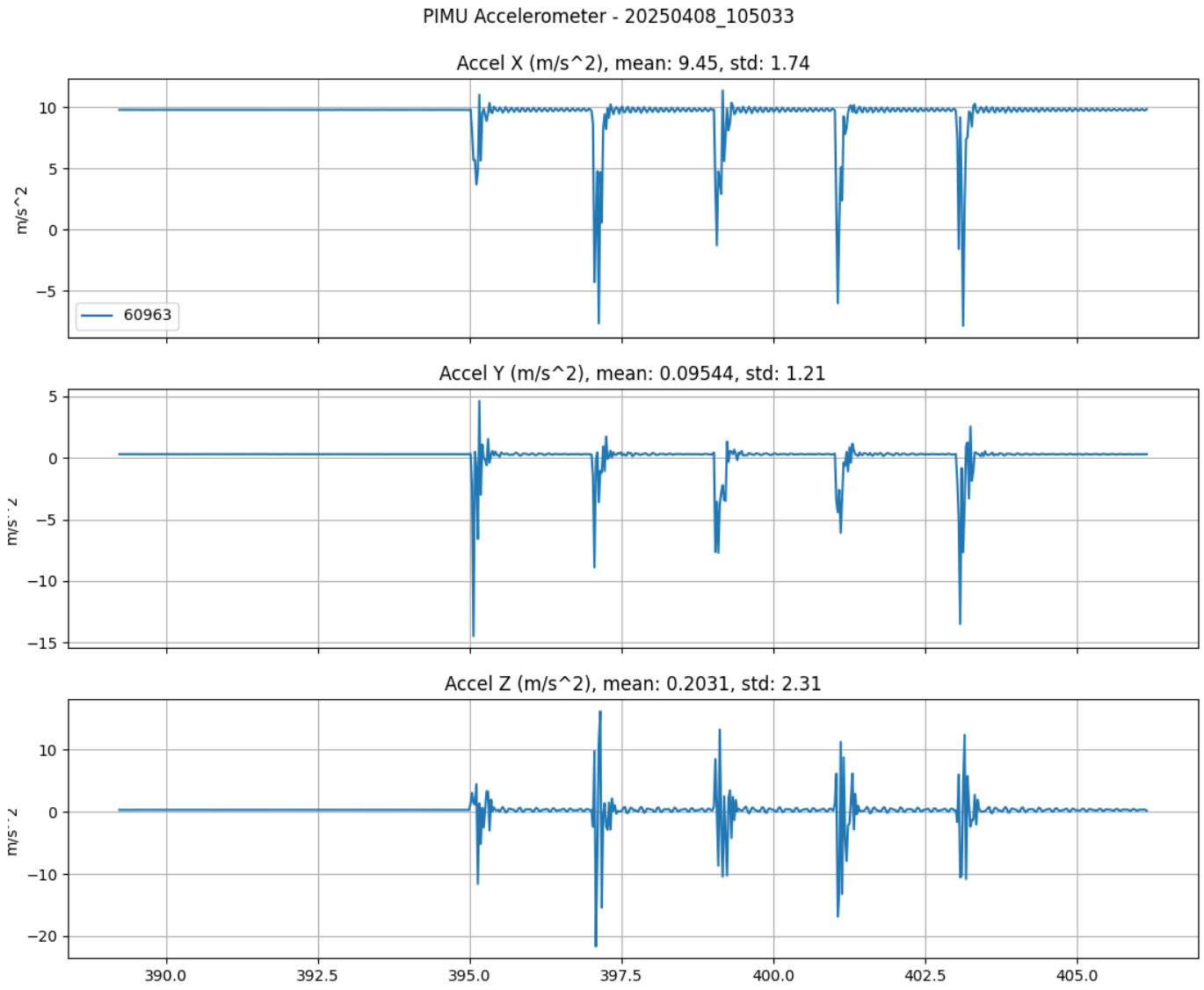
The IMU Shock Detection feature enables the system to identify and mitigate mechanical shock events that adversely affect Inertial Measurement Unit (IMU) measurements. Because impulse shocks can saturate sensors and exceed their effective bandwidth---producing invalid data that would otherwise corrupt Attitude and Heading Reference System (AHRS) and Inertial Navigation System (INS) estimates---the system employs specialized algorithms to detect, isolate, and reject shock-induced disturbances. When a shock is detected, the Extended Kalman Filter (EKF) is rewound to remove corrupted measurements and maintain navigation solution integrity.

7.8.2 Without Shock Rejection

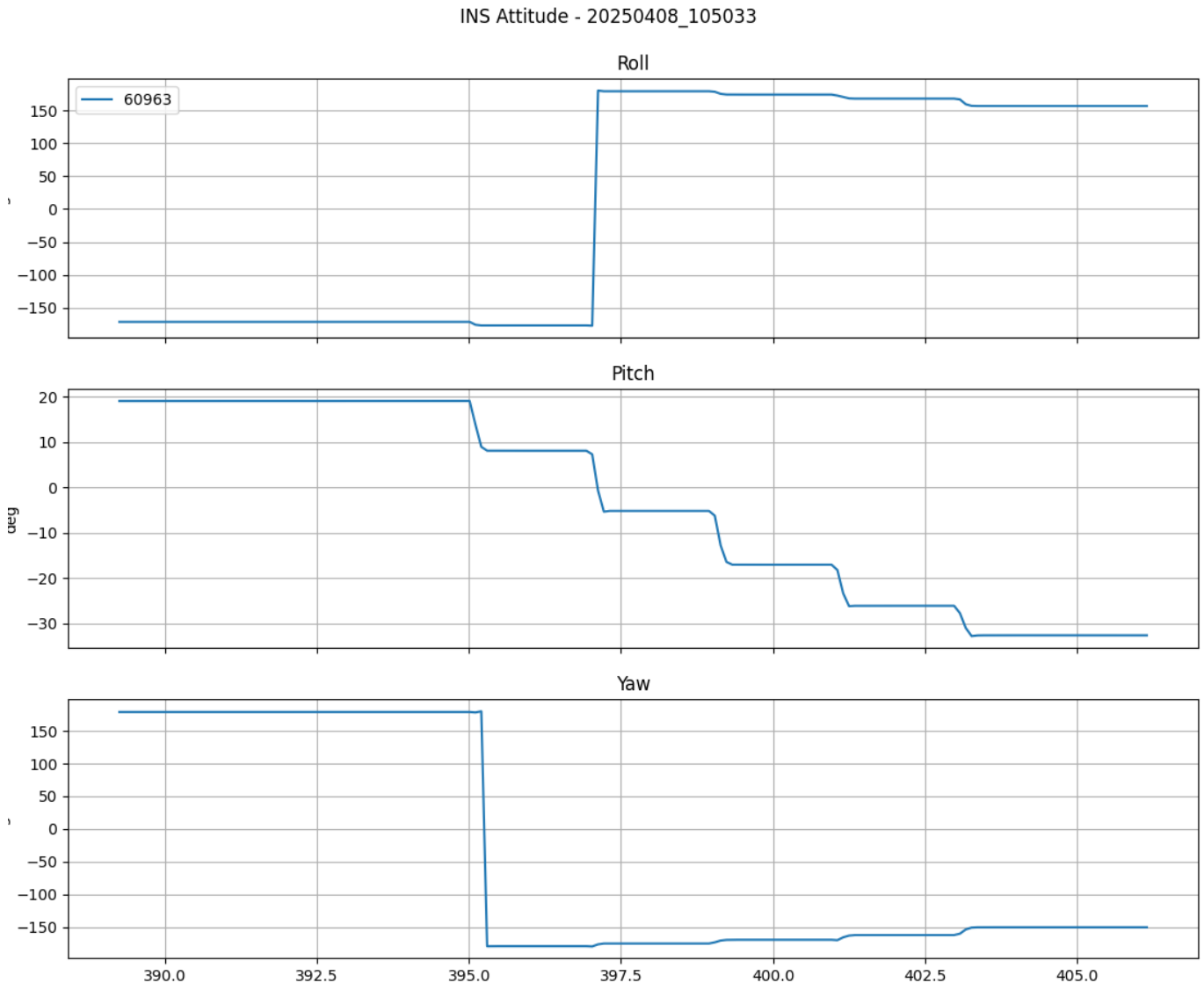
In the absence of a shock rejection mechanism on platforms such as the IMX-5, the attitude solution (comprising roll, pitch, and heading) can exhibit significant degradation, manifesting as attitude offset deviations of up to **+10 degrees** from the true value. This degradation underscores the critical need for a robust mitigation strategy.



©2022


PIMU Accelerometer - 20250408_105033
Accel X (m/s^2), mean: 9.45, std: 1.74
m/s^2
60963
Accel Y (m/s^2), mean: 0.09544, std: 1.21
Accel Z (m/s^2), mean: 0.2031, std: 2.31
390.0
392.5
395.0
397.5
400.0
402.5
405.0

©2022

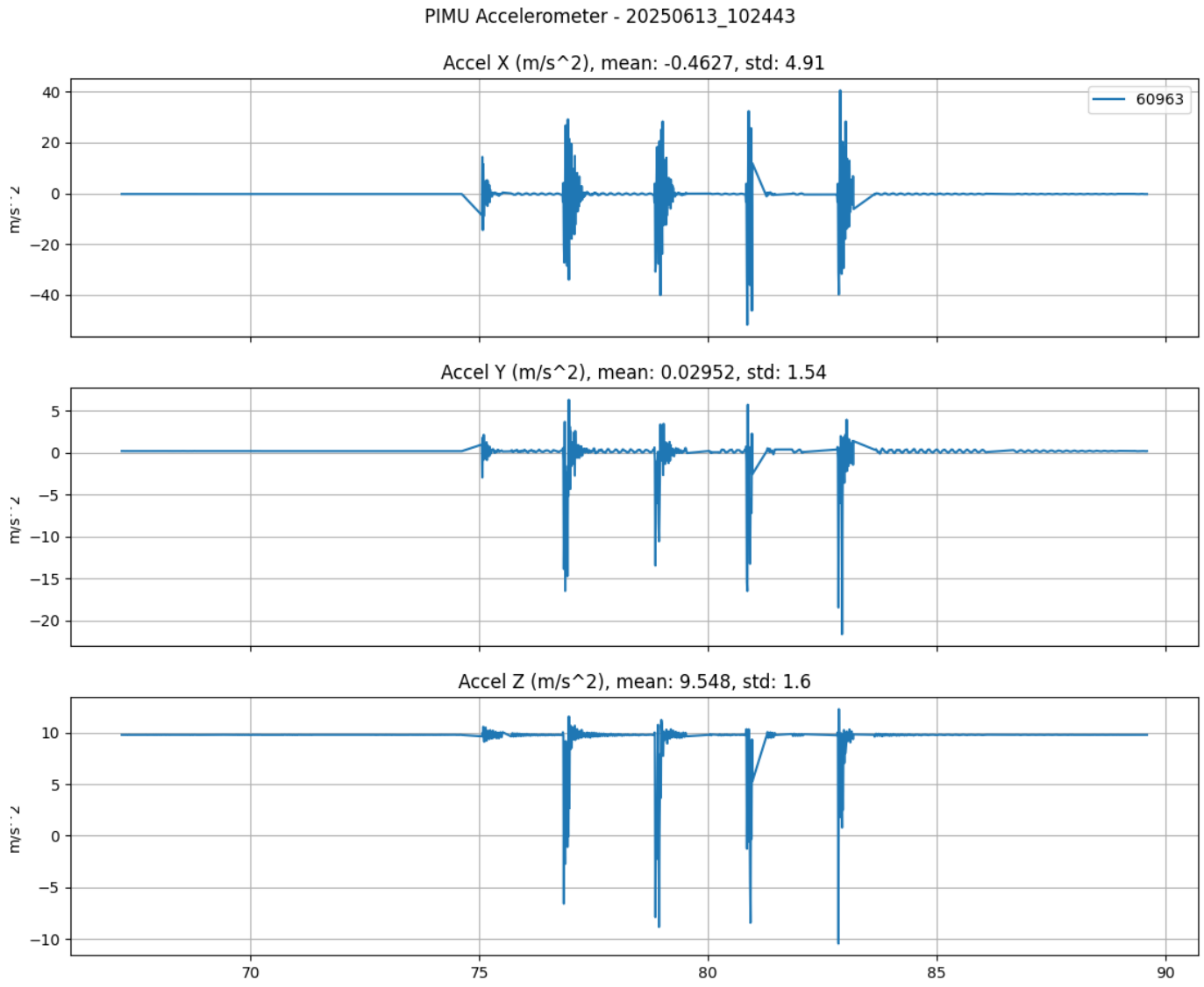


7.8.3 Performance With Shock Rejection

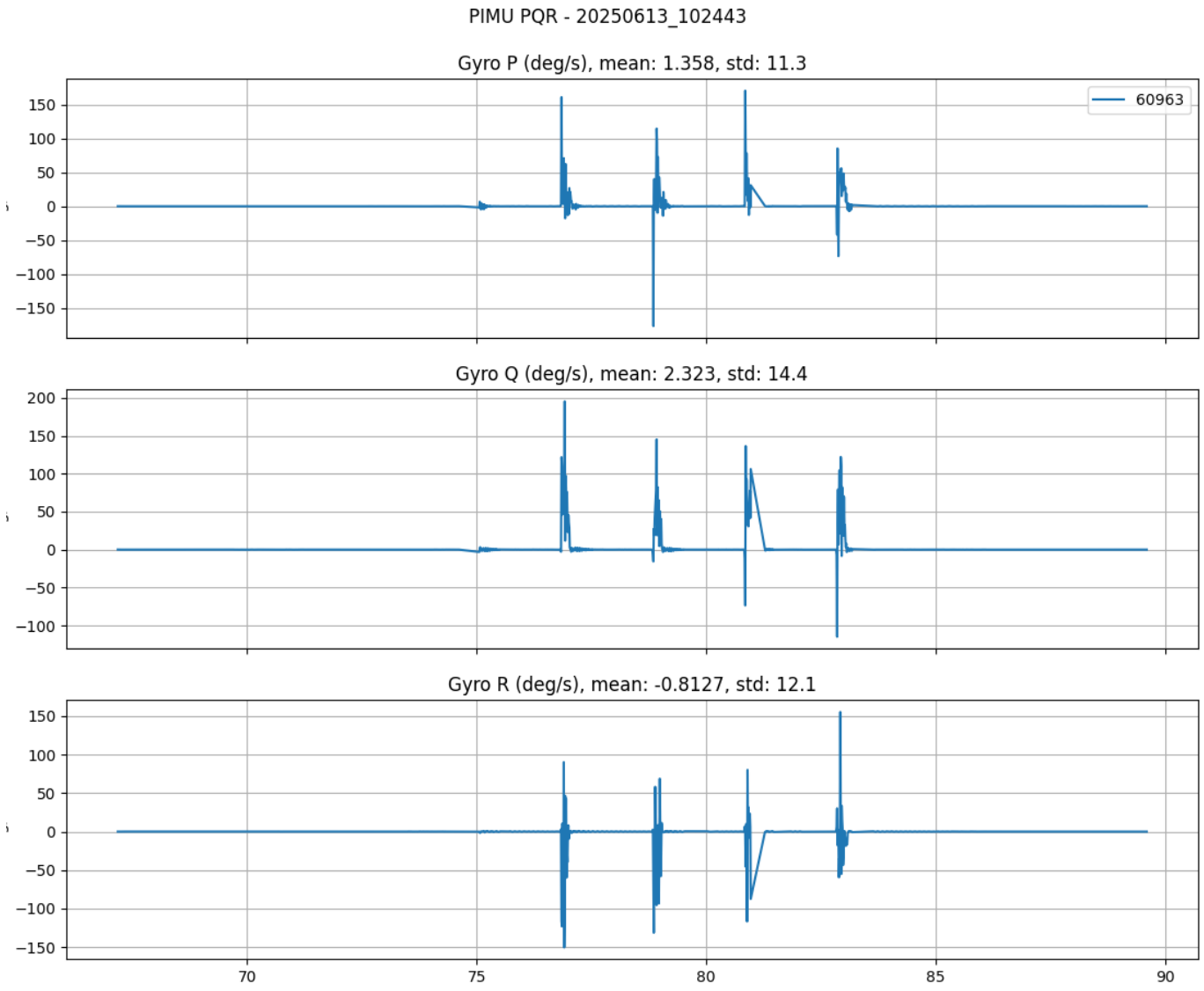
Shock Rejection Enabled

When the shock rejection feature is active, a substantial impulse can be observed on the IMU accelerometer's X-axis. The true peak magnitude of this shock is often unidentifiable due to the large impulse force exceeding the sensor's bandwidth limitations.

©2022

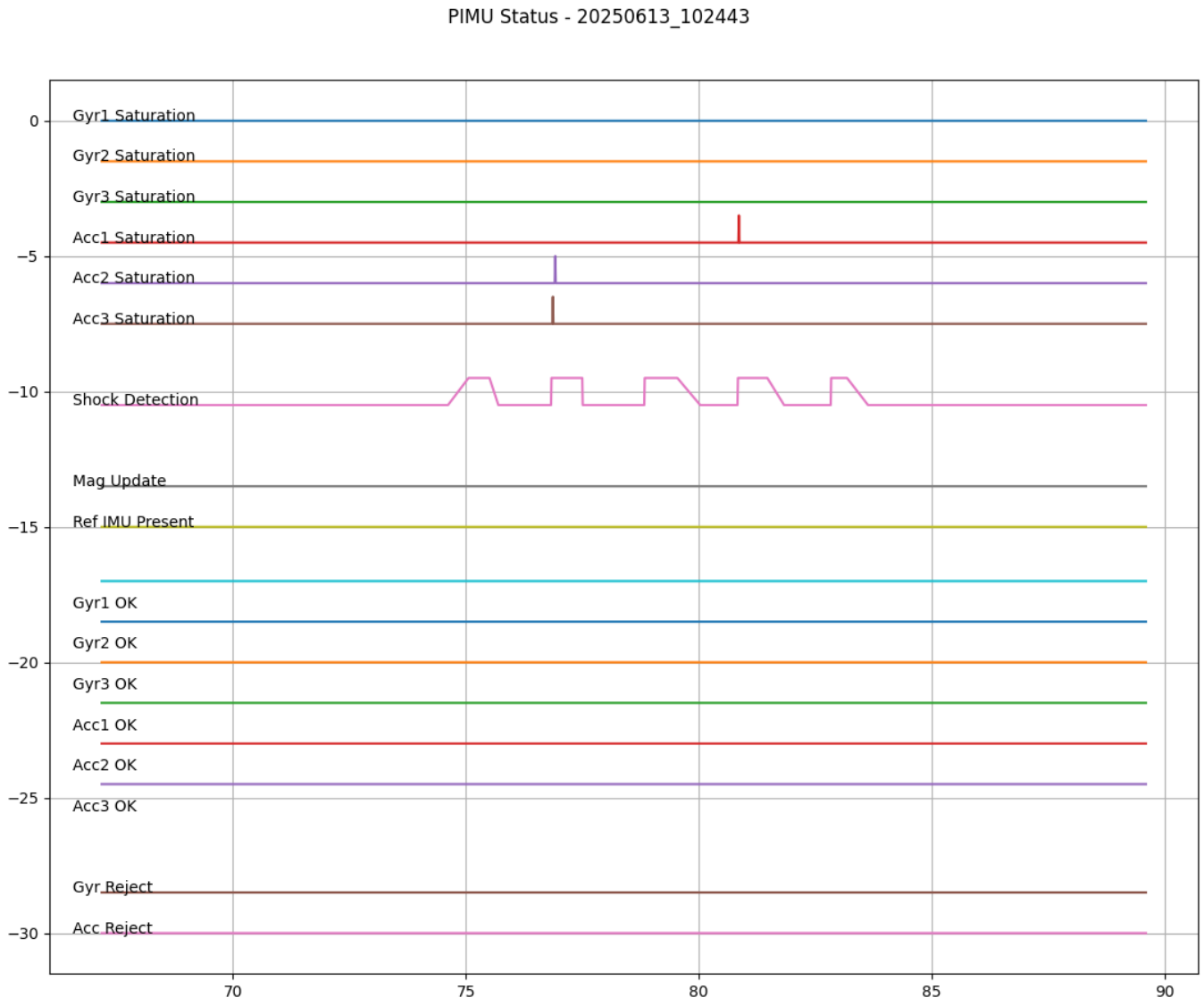

PIMU Accelerometer - 20250613_102443
Accel X (m/s^2), mean: -0.4627, std: 4.91
60963
m/s^2
Accel Y (m/s^2), mean: 0.02952, std: 1.54
m/s^2
Accel Z (m/s^2), mean: 9.548, std: 1.6
m/s^2
70
75
80
85
90

©2022



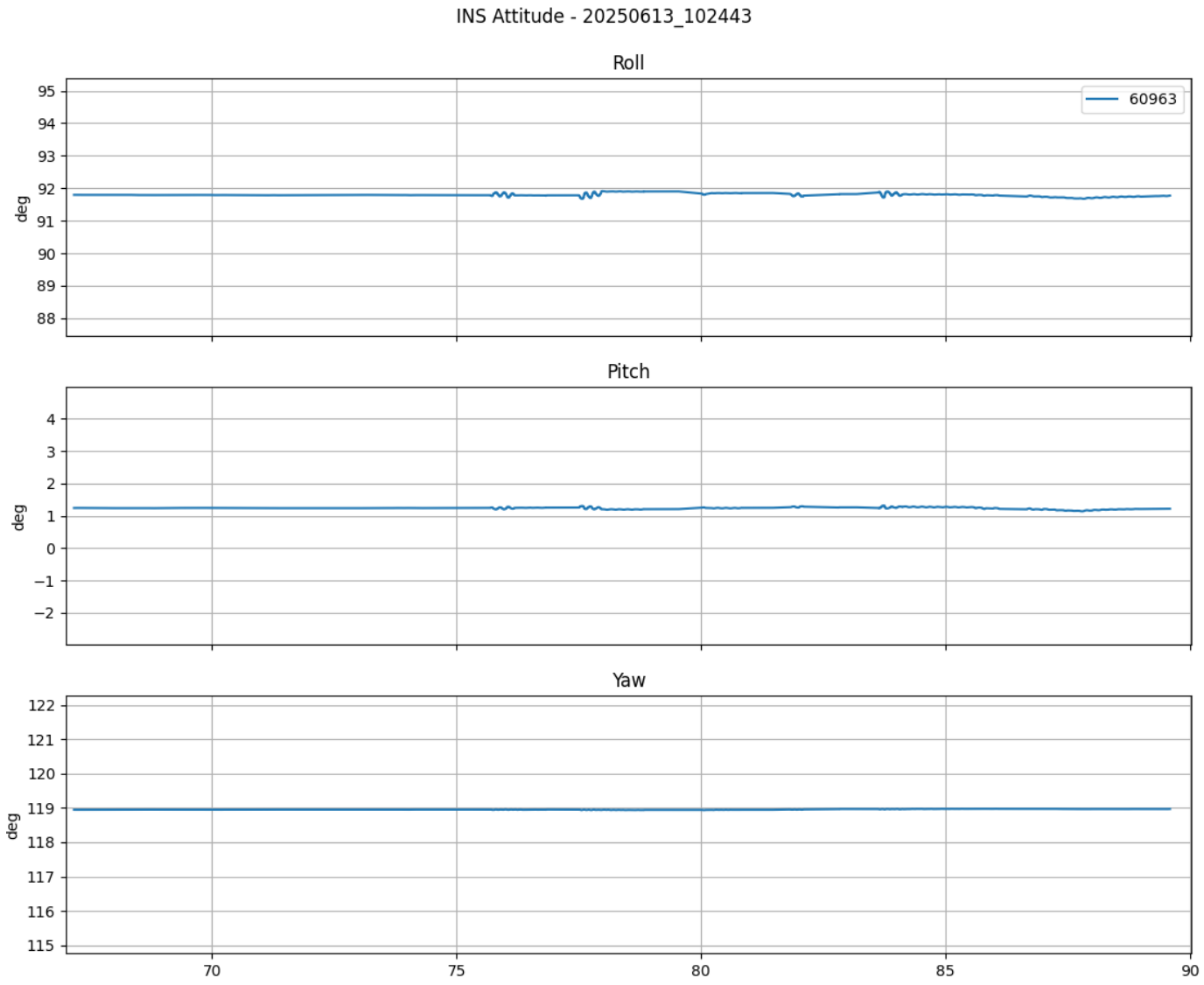
During the event, the IMU status flags provide crucial diagnostic information, recording both sensor saturation and successful shock detection. Critically, because the shock event is accurately identified and mitigated, the attitude estimate (roll, pitch, and yaw) remains largely impervious to the shock input, preserving the accuracy and reliability of the navigation solution.

©2022



Because the shock is properly identified, the attitude estimate (roll, pitch, yaw) is relatively unaffected by the shock input.

©2022



7.8.4 How to Enable Shock Rejection

The shock rejection feature is configured through specific bit settings within the Software Development Kit (SDK) and must be applied to the **DID_FLASH_CONFIG.imuShockOptions** register:

IMU_SHOCK_OPTIONS_ENABLE (0x01): Activates the IMU shock rejection functionality.

IMU_SHOCK_OPTIONS_FAST_RECOVERY (0x02): This option is particularly beneficial when impulse shocks occur concurrent with platform rotation. It minimizes the post-shock integration error and substantially reduces the time required for attitude convergence. However, it is important to note that this setting can marginally increase the attitude error induced directly by the shock event itself. Therefore, its use is generally discouraged for applications where the platform is stationary and not undergoing rotation during shock occurrences.

©2022

7.8.5 Configuration

The shock rejection can be tuned if needed using the following parameters.

Parameter	Type	Units	Purpose	Typical Range	Notes / Impact
imuShockDetectLatencyMs	uint8_t	ms	EKF rewind window after shock detection	10–100 ms	Larger values protect more history but add navigation latency
imuShockRejectLatchMs	uint8_t	ms	Time shock rejection remains active after shock ends	100–500 ms	Prevents rapid on/off oscillation; ensures stabilization
imuShockOptions	uint8_t	—	Bitfield controlling shock detection behavior	—	Enables detection, algorithm selection, EKF rewind behavior
imuShockDeltaAccHighThreshold	uint8_t	m/s²	Acceleration difference to trigger shock start	2–10 m/s²	Lower = more sensitive; higher = fewer false positives
imuShockDeltaAccLowThreshold	uint8_t	m/s²	Acceleration difference to confirm shock end	0.5–5 m/s²	Must be lower than high threshold for hysteresis
imuShockDeltaGyroHighThreshold	uint8_t	deg/s	Angular rate difference to trigger shock start	5–20 deg/s	Detects rotational shocks
imuShockDeltaGyroLowThreshold	uint8_t	deg/s	Angular rate difference to confirm shock end	1–10 deg/s	Lower than high threshold to avoid oscillation

7.8.6 Usage Example

Configuration for Vehicle Dynamics

```
config.imuShockDetectLatencyMs = 50;          // 50ms EKF rewind window
config.imuShockRejectLatchMs = 200;           // 200ms latch time after shock end
config.imuShockOptions = 0x01;                // Enable shock detection
config.imuShockDeltaAccHighThreshold = 5;     // 5 m/s² to trigger
config.imuShockDeltaAccLowThreshold = 2;      // 2 m/s² to confirm end
config.imuShockDeltaGyroHighThreshold = 10;   // 10 deg/s to trigger
config.imuShockDeltaGyroLowThreshold = 4;     // 4 deg/s to confirm end
```

7.8.7 Tuning Guidelines

For Sensitive Applications (e.g., precision navigation)

- Increase `imuShockDetectLatencyMs` to provide better protection

©2022

- Lower acceleration/gyroscope thresholds to catch subtle shocks
- Longer latch time to ensure full stabilization

For Rugged Environments (e.g., off-road vehicles)

- Consider reducing thresholds further to detect frequent impacts
- Balance latch time against desired responsiveness
- Monitor EKF rewind performance in field testing

For Minimal Interference

- Increase thresholds to only catch significant shocks
- Shorter latch times for faster recovery
- Reduce `imuShockDetectLatencyMs` if rewind latency is unacceptable

7.8.8 Shock Detection Algorithm

The system detects shocks by monitoring **inter-IMU differences**:

1. **Shock Start:** When differences exceed either the acceleration OR gyroscope high thresholds
2. **Active Shock:** Shock rejection remains active for the duration of the shock event
3. **Shock End:** When ALL differences drop below their respective low thresholds AND the latch timer expires
4. **EKF Rewind:** The navigation state is rewound by `imuShockDetectLatencyMs` to remove shock-corrupted estimates

---

7.8.9 Performance Considerations

- **Latency Impact:** EKF rewind introduces latency proportional to `imuShockDetectLatencyMs`
- **Computational Load:** Shock detection adds minimal overhead (difference calculations and comparisons)
- **IMU Health:** Requires all 3 IMUs to be operational for reliable shock detection

©2022

7.9 IMX-5 Bootloader

The IMX-5 bootloader is embedded firmware stored on the IMX and is used to update the IMX application firmware.

7.9.1 Application Firmware Update

The following are conditions for the IMX firmware update.

- **Bootloader Enable** - At start of the firmware update process, the IMX is sent a "bootloader enable" command to reboot the processor into bootloader mode. The IMX will not reboot into application mode until a valid firmware upload has completed.
- **Handshake Sequence** - A handshake sequence consisting of five consecutive sync characters (`U`) spaced 10 ms (minimum 5 ms) apart is sent to the IMX to initiate a given port and close all other ports to prevent interference.
- **Application Enable** - The IMX reboots into application mode only after a valid firmware upload has completed.

7.9.2 Bootloader Update

Updating the bootloader firmware is occasionally necessary when new functionality is required. The bootloader is checked and updated at the same time as loading new firmware. The following steps outline how to update the IMX bootloader and firmware.

1. **Ensure IMX Firmware is Running** - *(This step is not necessary if the IMX firmware is running and the EvalTool is communicating with the IMX)*. If the bootloader is running but the firmware is not, version information will not appear in the EvalTool. The LED will also be a fading cyan.
2. **Select Baud Rate** - Select a slower baud rate (i.e. 115,200 or 230,400) for systems with known baud rate limits.
3. **Update the Bootloader and Firmware** - Use the EvalTool "Update Firmware" button in the Settings tab to upload the latest bootloader and the latest firmware. **The bootloader can only be updated using serial0 or the native USB ports.**

7.9.3 Known Issues

The following are known issues in the IMX-5 bootloader.

- **UART 50s Disable** - (Version v5g and prior) All UARTs get disabled if no handshake sequence is received within 50 seconds of the bootloader start. The USB port does not automatically close due to no handshake reception.

©2022

7.10 Troubleshooting

7.10.1 IMX-5 Chip Erase

Please email support@inertialsense.com for assistance or to provide feedback on this user guide.

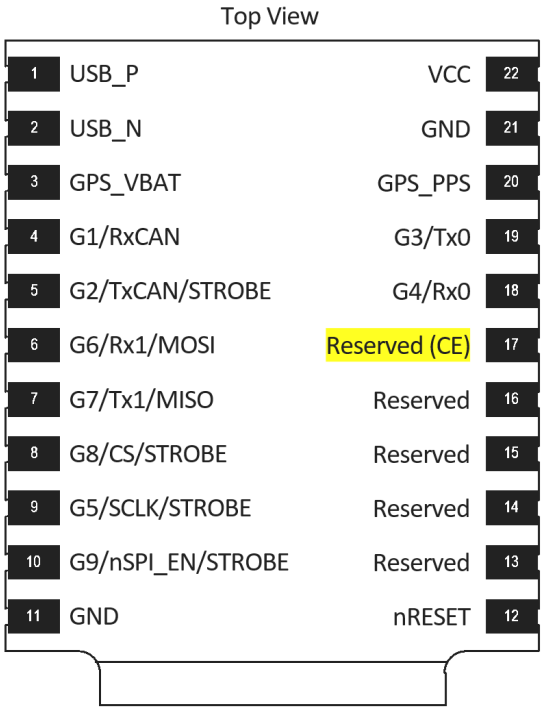
STEPS FOR CHIP-ERASE RECOVERY

> **Warning**
>
> The CHIP ERASE (Reserved (CE) pin 17) erases all flash memory including firmware, settings and calibration. CHIP ERASE should only be used as a last resort. This step should ONLY be used if the steps for Stuck in Bootloader Mode fail and there is NO other method to recover communications.

> **Important**
>
> Please notify support@inertialsense.com if this step is necessary so that we can keep track of cause of failures and provide you any necessary support.

IMX Chip Erase Pad

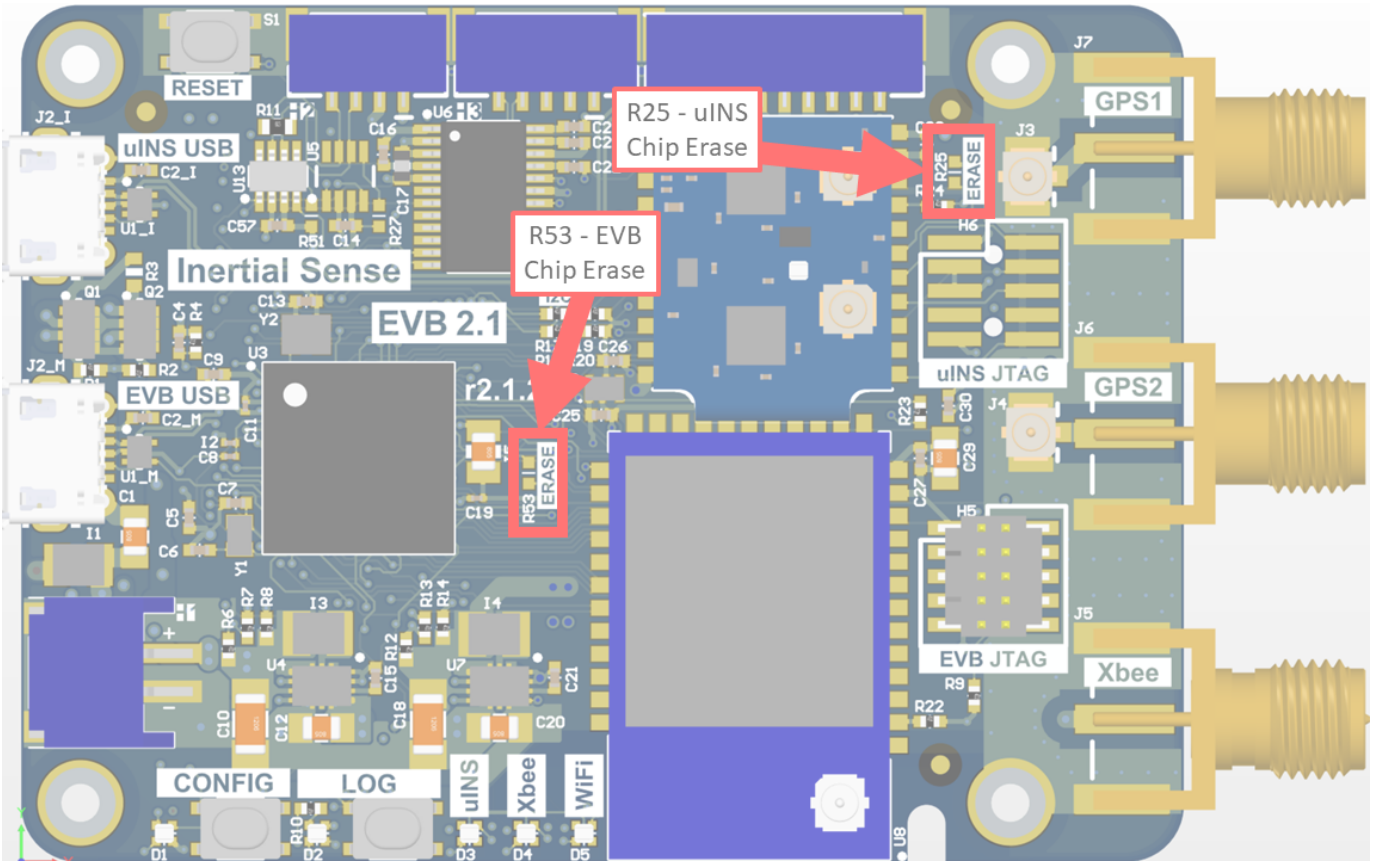
On the IMX-5, CHIP ERASE is enabled if +3.3V (available on pin 22) is applied to the chip erase (CE) pin 17 during boot up from power cycle or reset.



Connect +3.3V to pin 17 (CE) while power cycling the IMX to chip erase IMX.

©2022

EVB-2 Chip Erase Pads

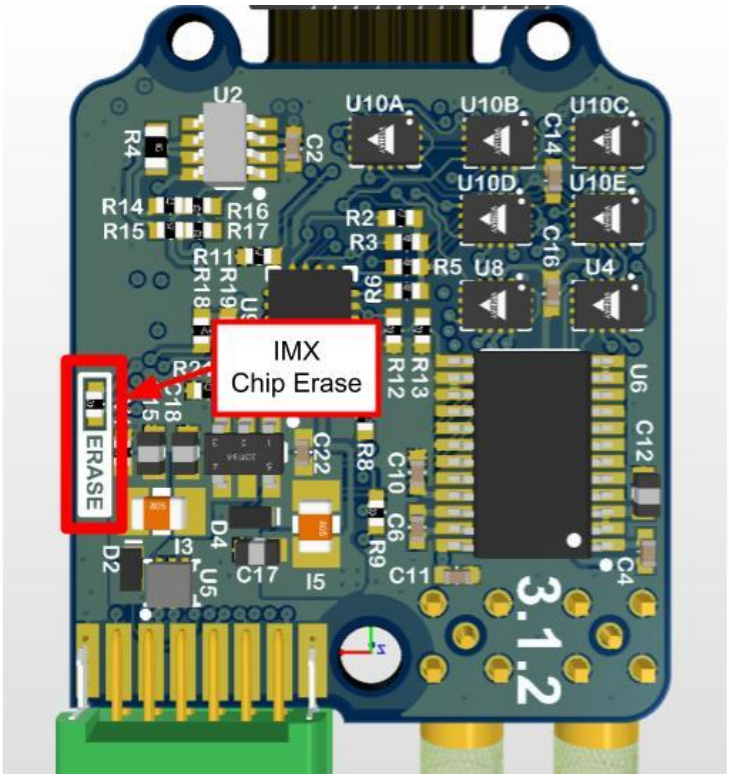


Short R25 pads together to chip erase IMX.
Short R53 pads together to chip erase EVB-2.

Rugged Chip Erase Pads

The chip erase pads on the Rugged-3 are a set of 0402 SMT pads with the label "ERASE". Shorting these pads together will apply +3.3V to the IMX chip erase pin 17. The power must be cycled while shorting these pads in order to apply chip erase to the IMX-5.

©2022



Short "ERASE" pads together and reset to chip erase.

©2022

Restore Firmware

1. **Power on system**
2. **Record your IMX Serial Number** - If you can read the serial number, record it for reference. On older firmware versions the serial number will be erased. New firmware versions store the serial number in a location that chip erase won't touch.
3. **Chip Erase IMX** - Assert Chip Erase (Reserved (CE) pin 17) on the IMX longer than 100ms by connecting to +3.3V. +3.3V is available on pin 2 of all EVB headers. **Warning!!!** - CHIP ERASE erases all flash memory (including firmware, settings, and calibration) and should only be used as a last resort. This step should ONLY be used if there is NO other method to recover communications.
4. **Reset the system**
5. **Enable EvalTool Internal Mode** - This exposes the "Manufacturing" tab used to upload calibration data.
6. **Restore the application and bootloader firmware** - Use the "Update Firmware" button in the EvalTool Settings tab to load the bootloader firmware and IMX firmware.

Enable EvalTool Internal Mode

EvalTool internal mode is used to access the EvalTool Manufacturing tab, used to restore serial numbers and calibration data.

1. Close the EvalTool so it isn't running.
2. Using a text editor, change the value of "DBGINT" to 99 (i.e. `"DBGINT": 99,`) in settings file: C:\Users\[USERNAME]\Documents\Inertial Sense\settings.json.
3. Restart EvalTool and verify "[INTERNAL MODE]" is in the title bar.

Restore Sensor Calibration

Contact InertialSense and provide your unit serial number to request the sensor calibration that corresponds with your unit. Use the EvalTool to upload the sensor calibration onto your unit.

1. Ensure the EvalTool is in Internal Mode which provides access the Manufacturing tab.
2. Ensure unit is communicating with EvalTool.
3. Upload calibration data: EvalTool -> Manufacturing Tab -> "Load" button next to "System Test" button.
4. Verify "TC Pts" which is the number of calibration points located just below the "Load" button changes from "0,0" to two numbers larger than 12 (i.e. "18,18").
5. Reset the unit.
6. **Run "Built-In Test"** - Verify the built-in test passes by pressing the "Built-In Test" button in the EvalTool INS tab.
7. **Verify IMU output** - Place the unit on a flat level surface. Using the EvalTool Sensor tab, verify that the gyro rates are near zero, the accelerometer X and Y axes are near zero, and accelerometer Z axis is near -9.8m/s^2 for gravity.

©2022

7.10.2 IMX-5 Bootloader

Windows USB Device Driver

In some cases updating the IMX-5 bootloader firmware (not the IMX-5 application firmware) will fail in Windows if the wrong device driver is used. The following instructions show how to install the correct device driver using the **Zadig generic USB driver installer** Windows application.

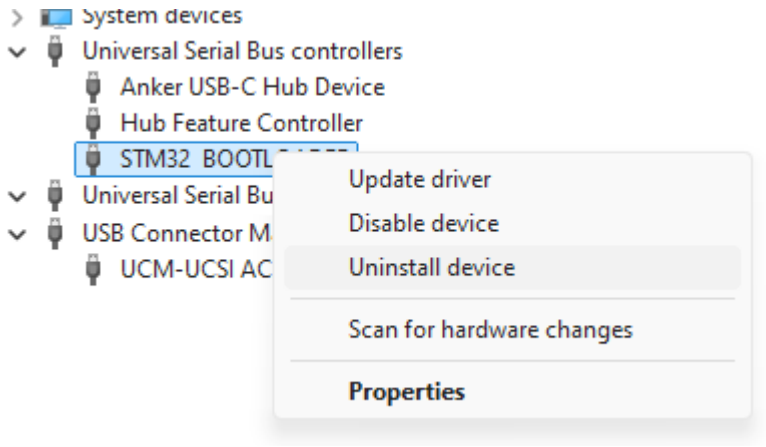
STEPS TO UPDATE IMX WINDOWS DEVICE DRIVER

When updating the bootloader, the IMX-5 will switch into ROM bootloader mode and communicate using DFU protocol over USB. In this mode the device is identified in the Window Device Manager as `STM32 BOOTLOADER`. This device will only appear in the Device Manager when the IMX-5 is in ROM bootloader mode and connected via USB. This device will not appear in the Device Manager during normal operation or when not connected over USB.

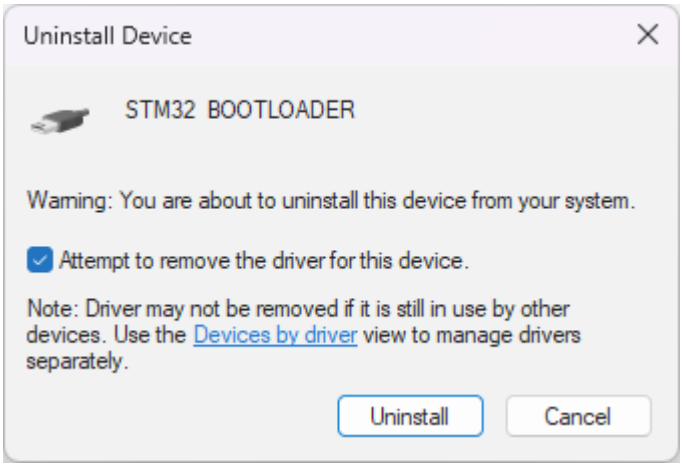
STEP 1 - Uninstall Old Device Driver - To uninstall the device driver, right click and select "Uninstall device" on the `STM32 BOOTLOADER` device.

NOTE: To see the device in the device manager, the IMX-5 should be connected over USB and the IMX-5 in ROM bootloader mode. Run the firmware update with bootloader **Force Update** option to enable ROM bootloader mode on the IMX-5.

NOTE: An incorrect version of the device driver can be identified when the `STM32 BOOTLOADER` device appears under `Universal Serial Bus Controllers`. The correct device driver will cause the `STM32 BOOTLOADER` device to appear under `Universal Serial Bus Devices`.

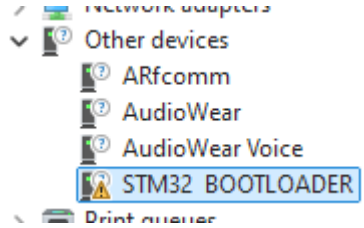


Delete the old device driver files by selecting **Attempt to remove the driver for this device** or **Delete driver files** checkbox and click **Uninstall**.

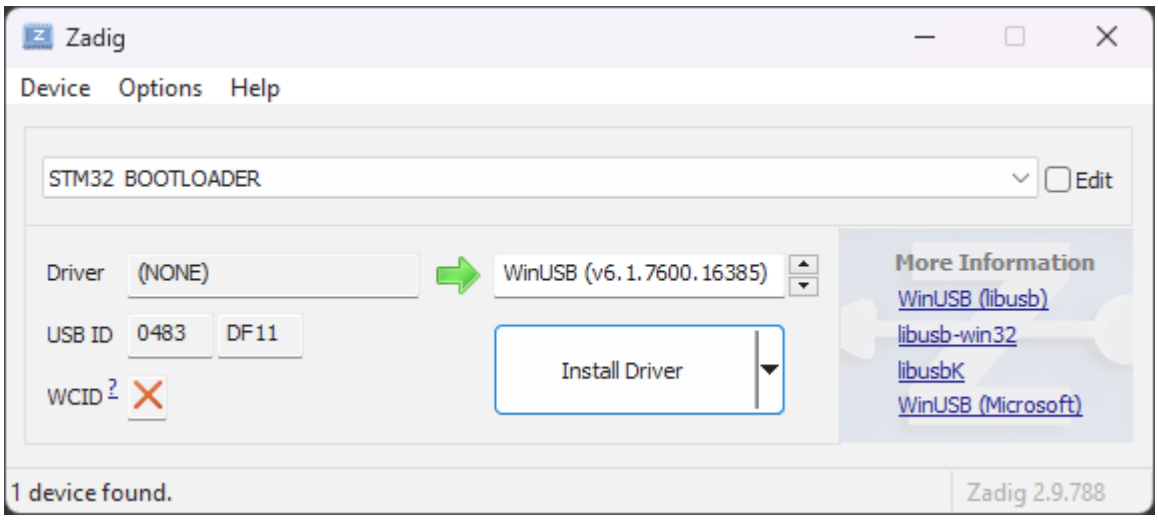


©2022

STEP 2 - Install New Device Driver - Download and run the Zadig USB driver installer. The Zadig USB driver installer detects the device that needs the driver installed. The `STM32 BOOTLOADER` device must appear in the Windows Device Manager for this to work. When there is no device driver installed, the `STM32 BOOTLOADER` device will appear under **Other devices** in the Windows Device Manager.



Run the Zadig generic USB driver application. Select update online and make sure `STM32 BOOTLOADER` is selected in the top dropdown menu. `WinUSB *=*v(6.1...` should appear in the driver option list. Click **Install Driver** and wait about one minute for the process to finish.



The `STM32 BOOTLOADER` device will appear in the Device Manager under **Universal Serial Bus devices** when the device driver is properly installed.

Run the firmware update to test the newly installed device driver.

NOTE: The negotiation at the beginning of the IMX-5 bootloader update can be interrupted if the IMX-5 receives data on other ports than that used for the update (i.e. from a connected GPS receiver). If this happens, the firmware update will fail but can be re-run. To prevent interruption so that the bootloader update occurs in one process, data stream into other ports of the IMX must be turned off.

©2022

8. GPX

8.1 GPX Configuration

8.1.1 Disable VCC_RF

The VCC_RF output (GPX pin 16) used to power active GNSS antennas can be disabled by setting bit `GPX_SYS_CFG_BITS_DISABLE_VCC_RF` (0x00000001) in `DID_GPX_FLASH_CFG.sysCfgBits`.

©2022

8.2 GPX System Status

Up-to-date information about GPX System Status can be found in the current version of the Inertial Sense SDK in `data_sets.h`. This information is output in the DID `DID_GPX_STATUS` and transmitted as a `gpx_status_t`.

8.2.1 Status

Status info can be found in the `gpx_status_t` in the field `status`. Please see `eGpxStatus` for more in-depth information.

8.2.2 Hardware Status

Status info can be found in the `gpx_status_t` in the field `hdwStatus`. Please see `eGPXHdwStatusFlags` for more in-depth information.

©2022

9. Communication Protocols

9.1 Protocol Overview

The Inertial Sense products support binary and NMEA protocol for communication.

9.1.1 Binary vs. NMEA

The following table compares the differences and advantages between the binary and NMEA protocols.

	NMEA Protocol	**Binary Protocol**
Data Efficient	No. Numbers must be converted to IEEE float and integers for application. Data occupies more memory.	Numbers are in floating point and integer binary format used in computers. Data occupies less memory.
Human Readable	Yes	No
Complexity	Packet are easier to parse.	Packet encoding, decoding, and parsing are MORE complicated. Using SDK is recommended.
SDK Support	Yes, less	Yes, more
Data Access	Limited to sensor and INS output.	Comprehensive access to all data and configuration settings.
Recommended Use	Rapid prototypes and simple projects. Devices supporting NMEA.	Moderate to advanced applications.
Apps and Examples	NMEA Communications Example	EvalTool, CLTool, Binary Communications Example, Firmware Update Example, Data Logger Example

©2022

9.2 Data Sets (DIDs)

9.2.1 Data Sets (DIDs)

Data Sets in the form of C structures are available through binary protocol and provide access to system configuration and output data. The data sets are defined in SDK/src/data_sets.h of the InertialSense SDK.

INS / AHRS Output

DID_INS_1

`ins_1_t`

Field	Type	Description
week	uint32_t	GPS number of weeks since January 6th, 1980
timeOfWeek	double	GPS time of week (since Sunday morning) in seconds
insStatus	uint32_t	INS status flags (eInsStatusFlags). Copy of DID_SYS_PARAMS.insStatus
hdwStatus	uint32_t	Hardware status flags (eHdwStatusFlags). Copy of DID_SYS_PARAMS.hdwStatus
theta	float[3]	Euler angles: roll, pitch, yaw in radians with respect to NED
uvw	float[3]	Velocity U, V, W in meters/second, in body frame. Convert to NED velocity using "vectorBodyToReference(uvw, theta, vel_ned)".
lla	double[3]	WGS84 latitude, longitude, height above ellipsoid (degrees, degrees, meters)
ned	float[3]	North, east, down (meters) offset from reference latitude, longitude, and altitude to current latitude, longitude, and altitude

DID_INS_2

`ins_2_t`

Field	Type	Description
week	uint32_t	GPS number of weeks since January 6th, 1980
timeOfWeek	double	GPS time of week (since Sunday morning) in seconds
insStatus	uint32_t	INS status flags (eInsStatusFlags). Copy of DID_SYS_PARAMS.insStatus
hdwStatus	uint32_t	Hardware status flags (eHdwStatusFlags). Copy of DID_SYS_PARAMS.hdwStatus
qn2b	float[4]	Quaternion body rotation with respect to NED: W, X, Y, Z
uvw	float[3]	Velocity U, V, W in meters/second, in body frame. Convert to NED velocity using "quatRot(vel_ned, qn2b, uvw)".
lla	double[3]	WGS84 latitude, longitude, height above ellipsoid in meters (not MSL)

©2022

DID_INS_3

`ins_3_t`

Field	Type	Description
week	uint32_t	GPS number of weeks since January 6th, 1980
timeOfWeek	double	GPS time of week (since Sunday morning) in seconds
insStatus	uint32_t	INS status flags (eInsStatusFlags). Copy of DID_SYS_PARAMS.insStatus
hdwStatus	uint32_t	Hardware status flags (eHdwStatusFlags). Copy of DID_SYS_PARAMS.hdwStatus
qn2b	float[4]	Quaternion body rotation with respect to NED: W, X, Y, Z
uvw	float[3]	Velocity U, V, W in meters/second, in body frame. Convert to NED velocity using "quatRot(vel_ned, qn2b, uvw)".
lla	double[3]	WGS84 latitude, longitude, height above ellipsoid in meters (not MSL)
msl	float	Height above mean sea level (MSL) in meters

DID_INS_4

`ins_4_t`

Field	Type	Description
week	uint32_t	GPS number of weeks since January 6th, 1980
timeOfWeek	double	GPS time of week (since Sunday morning) in seconds
insStatus	uint32_t	INS status flags (eInsStatusFlags). Copy of DID_SYS_PARAMS.insStatus
hdwStatus	uint32_t	Hardware status flags (eHdwStatusFlags). Copy of DID_SYS_PARAMS.hdwStatus
qe2b	float[4]	Quaternion body rotation with respect to ECEF: W, X, Y, Z
ve	float[3]	Velocity in ECEF (earth-centered earth-fixed) frame in meters per second
ecef	double[3]	Position in ECEF (earth-centered earth-fixed) frame in meters

Inertial Measurement Unit (IMU)

DID_IMU

`imu_t`

Field	Type	Description
time	double	Time since boot up in seconds. Convert to GPS time of week by adding gps.towOffset
status	uint32_t	IMU status flags (eImuStatus)
I	imui_t	Combined Inertial Measurement Unit (IMU) sample: angular rate and acceleration

©2022

DID_IMUS

`imus_t`

Field	Type	Description
time	double	Time since boot up in seconds. Convert to GPS time of week by adding gps.towOffset
status	uint32_t	IMUs status flags (eImusStatus)
I	imui_t[1]	Per-device Inertial Measurement Unit (IMU) samples: angular rate and acceleration

DID_IMUS_RAW

Multiple IMU data calibrated from DID_IMUS_UNCAL. We recommend use of DID_IMU or DID_PIMU as they are oversampled and contain less noise.

`imus_t`

Field	Type	Description
time	double	Time since boot up in seconds. Convert to GPS time of week by adding gps.towOffset
status	uint32_t	IMUs status flags (eImusStatus)
I	imui_t[1]	Per-device Inertial Measurement Unit (IMU) samples: angular rate and acceleration

DID_IMU_RAW

IMU data averaged from DID_IMUS_RAW. Use this IMU data for output data rates faster than DID_FLASH_CONFIG.startupNavDtMs. Otherwise we recommend use of DID_IMU or DID_PIMU as they are oversampled and contain less noise.

`imu_t`

Field	Type	Description
time	double	Time since boot up in seconds. Convert to GPS time of week by adding gps.towOffset
status	uint32_t	IMU status flags (eImuStatus)
I	imui_t	Combined Inertial Measurement Unit (IMU) sample: angular rate and acceleration

DID_PIMU

`pimu_t`

Field	Type	Description
time	double	Time since boot up in seconds. Convert to GPS time of week by adding gps.towOffset
dt	float	Integration period in seconds for delta theta and delta velocity. Configured using DID_FLASH_CONFIG.startupNavDtMs
status	uint32_t	IMU status flags (eImuStatus)
theta	float[3]	IMU delta theta: gyroscope {p,q,r} integral over dt, in radians, in sensor/body frame
vel	float[3]	IMU delta velocity: accelerometer {x,y,z} integral over dt, in meters/second, in sensor/body frame

©2022

Sensor Output

DID_BAROMETER

`barometer_t`

Field	Type	Description
time	double	Time since boot up in seconds. Convert to GPS time of week by adding gps.towOffset
bar	float	Barometric pressure in kilopascals (kPa)
mslBar	float	MSL altitude derived from barometric pressure sensor, in meters
barTemp	float	Temperature of barometric pressure sensor in Celsius
humidity	float	Relative humidity as a percent (%rH). Range is 0% - 100%

DID_MAGNETOMETER

`magnetometer_t`

Field	Type	Description
time	double	Time since boot up in seconds. Convert to GPS time of week by adding gps.towOffset
mag	float[3]	Magnetometer X, Y, Z in microtesla (uT), in body frame

DID_MAG_CAL

`mag_cal_t`

Field	Type	Description
state	uint32_t	Mag recalibration command/status. COMMANDS: 1=multi-axis, 2=single-axis, 101=abort, STATUS: 200=running, 201=done (see eMagCalState)
progress	float	(%) Mag recalibration progress indicator: 0-100
declination	float	(rad) Magnetic declination estimate

©2022

DID_SYS_SENSORS

`sys_sensors_t`

Field	Type	Description
time	double	Time since boot up, in seconds. Convert to GPS time of week by adding gps.towOffset
temp	float	IMU temperature, in Celsius
pqr	float[3]	Gyros {p,q,r}, in radians/second
acc	float[3]	Accelerometers {x,y,z}, in meters/second^2
mag	float[3]	Magnetometers {x,y,z} (uncalibrated units)
bar	float	Barometric pressure, in kilopascals
barTemp	float	Temperature of barometric pressure sensor, in Celsius
mslBar	float	MSL altitude derived from barometric pressure sensor, in meters
humidity	float	Relative humidity as a percent (%rH). Range is 0% - 100%
vin	float	EVB system input voltage, in volts. uINS pin 5 (G2/AN2). Use 10K/1K resistor divider between Vin and GND.
ana1	float	ADC analog input, in volts. uINS pin 4 (G1/AN1)
ana3	float	ADC analog input, in volts. uINS pin 19 (G3/AN3)
ana4	float	ADC analog input, in volts. uINS pin 20 (G4/AN4)

©2022

GPS / GNSS

DID_GNSS1_POS

`gnss_pos_t`

Field	Type	Description
week	uint32_t	GPS number of weeks since January 6th, 1980
timeOfWeekMs	uint32_t	GPS time of week (since Sunday morning) in milliseconds
status	uint32_t	GNSS status (see eGnssStatus): [0x000000xx] number of satellites used, [0x0000xx00] fix type, [0x00xx0000] status flags, NMEA input flag
ecef	double[3]	Position in ECEF {x,y,z}, in meters
lla	double[3]	Position in WGS84 latitude, longitude, height above ellipsoid (not MSL), in degrees, degrees, meters
hMSL	float	Height above mean sea level (MSL), in meters
hAcc	float	Horizontal accuracy, in meters
vAcc	float	Vertical accuracy, in meters
pDop	float	Position dilution of precision (unitless)
cnoMean	float	Average of all non-zero satellite carrier to noise ratios (signal strengths), in dBHz
towOffset	double	Time sync offset between local time since boot up and GPS time of week, in seconds. Add this to IMU and sensor time to get GPS time of week in seconds.
leapS	uint8_t	GPS leap second (GPS-UTC) offset, in seconds. Receiver's best knowledge of the leap seconds offset from UTC to GPS time. Subtract from GPS time of week to get UTC time of week. (18 seconds as of December 31, 2016)
satsUsed	uint8_t	Number of satellites used in the position solution
cnoMeanSigma	uint8_t	Standard deviation of cnoMean over the past 5 seconds, in dBHz x10
status2	uint8_t	Secondary GNSS status byte (see eGnssStatus2): [0x0X] spoofing/jamming status, [0xX0] unused

DID_GNSS1_RCVR_POS

GNSS 1 position data from GNSS receiver.

©2022

`gnss_pos_t`

Field	Type	Description
week	uint32_t	GPS number of weeks since January 6th, 1980
timeOfWeekMs	uint32_t	GPS time of week (since Sunday morning) in milliseconds
status	uint32_t	GNSS status (see eGnssStatus): [0x000000xx] number of satellites used, [0x0000xx00] fix type, [0x00xx0000] status flags, NMEA input flag
ecef	double[3]	Position in ECEF {x,y,z}, in meters
lla	double[3]	Position in WGS84 latitude, longitude, height above ellipsoid (not MSL), in degrees, degrees, meters
hMSL	float	Height above mean sea level (MSL), in meters
hAcc	float	Horizontal accuracy, in meters
vAcc	float	Vertical accuracy, in meters
pDop	float	Position dilution of precision (unitless)
cnoMean	float	Average of all non-zero satellite carrier to noise ratios (signal strengths), in dBHz
towOffset	double	Time sync offset between local time since boot up and GPS time of week, in seconds. Add this to IMU and sensor time to get GPS time of week in seconds.
leapS	uint8_t	GPS leap second (GPS-UTC) offset, in seconds. Receiver's best knowledge of the leap seconds offset from UTC to GPS time. Subtract from GPS time of week to get UTC time of week. (18 seconds as of December 31, 2016)
satsUsed	uint8_t	Number of satellites used in the position solution
cnoMeanSigma	uint8_t	Standard deviation of cnoMean over the past 5 seconds, in dBHz x10
status2	uint8_t	Secondary GNSS status byte (see eGnssStatus2): [0x0X] spoofing/jamming status, [0xX0] unused

DID_GNSS1_RTK_POS

GNSS RTK position data

©2022

`gnss_pos_t`

Field	Type	Description
week	uint32_t	GPS number of weeks since January 6th, 1980
timeOfWeekMs	uint32_t	GPS time of week (since Sunday morning) in milliseconds
status	uint32_t	GNSS status (see eGnssStatus): [0x000000xx] number of satellites used, [0x0000xx00] fix type, [0x00xx0000] status flags, NMEA input flag
ecef	double[3]	Position in ECEF {x,y,z}, in meters
lla	double[3]	Position in WGS84 latitude, longitude, height above ellipsoid (not MSL), in degrees, degrees, meters
hMSL	float	Height above mean sea level (MSL), in meters
hAcc	float	Horizontal accuracy, in meters
vAcc	float	Vertical accuracy, in meters
pDop	float	Position dilution of precision (unitless)
cnoMean	float	Average of all non-zero satellite carrier to noise ratios (signal strengths), in dBHz
towOffset	double	Time sync offset between local time since boot up and GPS time of week, in seconds. Add this to IMU and sensor time to get GPS time of week in seconds.
leapS	uint8_t	GPS leap second (GPS-UTC) offset, in seconds. Receiver's best knowledge of the leap seconds offset from UTC to GPS time. Subtract from GPS time of week to get UTC time of week. (18 seconds as of December 31, 2016)
satsUsed	uint8_t	Number of satellites used in the position solution
cnoMeanSigma	uint8_t	Standard deviation of cnoMean over the past 5 seconds, in dBHz x10
status2	uint8_t	Secondary GNSS status byte (see eGnssStatus2): [0x0X] spoofing/jamming status, [0xX0] unused

©2022

DID_GNSS1_RTK_POS_MISC

`gnss_rtk_misc_t`

©2022

Field	Type	Description
timeOfWeekMs	uint32_t	GPS time of week (since Sunday morning), in milliseconds
accuracyPos	float[3]	Accuracy: estimated standard deviations of the position solution assuming the a priori error model and error parameters used by the positioning options. []: {ECEF x,y,z} or {north,east,down}, in meters
accuracyCov	float[3]	Accuracy: estimated covariance of the position solution. []: absolute value of the square root of the estimated covariance {NE, EU, UN}, in meters
arThreshold	float	Ambiguity resolution threshold for validation (unitless)
gDop	float	Geometric dilution of precision (unitless)
hDop	float	Horizontal dilution of precision (unitless)
vDop	float	Vertical dilution of precision (unitless)
baseLla	double[3]	Base station position: latitude, longitude, height (deg, deg, m)
cycleSlipCount	uint32_t	Cycle slip counter
roverGpsObservationCount	uint32_t	Rover GPS observation element counter
baseGpsObservationCount	uint32_t	Base station GPS observation element counter
roverGlonassObservationCount	uint32_t	Rover GLONASS observation element counter
baseGlonassObservationCount	uint32_t	Base station GLONASS observation element counter
roverGalileoObservationCount	uint32_t	Rover Galileo observation element counter
baseGalileoObservationCount	uint32_t	Base station Galileo observation element counter
roverBeidouObservationCount	uint32_t	Rover BeiDou observation element counter
baseBeidouObservationCount	uint32_t	Base station BeiDou observation element counter
roverQzsObservationCount	uint32_t	Rover QZSS observation element counter
baseQzsObservationCount	uint32_t	Base station QZSS observation element counter
roverGpsEphemerisCount	uint32_t	Rover GPS ephemeris element counter
baseGpsEphemerisCount	uint32_t	Base station GPS ephemeris element counter
roverGlonassEphemerisCount	uint32_t	Rover GLONASS ephemeris element counter
baseGlonassEphemerisCount	uint32_t	Base station GLONASS ephemeris element counter
roverGalileoEphemerisCount	uint32_t	Rover Galileo ephemeris element counter
baseGalileoEphemerisCount	uint32_t	Base station Galileo ephemeris element counter
roverBeidouEphemerisCount	uint32_t	Rover BeiDou ephemeris element counter
baseBeidouEphemerisCount	uint32_t	Base station BeiDou ephemeris element counter
roverQzsEphemerisCount	uint32_t	Rover QZSS ephemeris element counter
baseQzsEphemerisCount	uint32_t	Base station QZSS ephemeris element counter
roverSbasCount	uint32_t	Rover SBAS element counter
baseSbasCount	uint32_t	Base station SBAS element counter
baseAntennaCount	uint32_t	Base station antenna position element counter
ionUtcAlmCount	uint32_t	Ionosphere model, UTC, and almanac element counter

©2022

Field	Type	Description
correctionChecksumFailures	uint32_t	Number of checksum failures from received corrections
timeToFirstFixMs	uint32_t	Time to first RTK fix, in milliseconds

DID_GNSS1_RTK_POS_REL

`gnss_rtk_rel_t`

Field	Type	Description
timeOfWeekMs	uint32_t	(ms) GPS time of week (since Sunday morning)
differentialAge	float	(s) Age of differential corrections
arRatio	float	Ambiguity resolution ratio factor for validation (unitless; higher indicates greater confidence the fixed integer ambiguity is correct)
baseToRoverVector	float[3]	(m) Vector from base to rover GNSS antennas {x,y,z} in ECEF. DID_GNSS1_RTK_POS_REL: RTK base station (base) to GNSS1 (rover). DID_GNSS2_RTK_CMP_REL (compassing): GNSS1 (base) to GNSS2 (rover)
baseToRoverDistance	float	(m) Distance from base to rover GNSS antennas (baseline length)
baseToRoverHeading	float	(rad) Heading of baseToRoverVector in the local tangent (NED) plane. Compassing mode: GNSS1 (base) to GNSS2 (rover)
baseToRoverHeadingAcc	float	(rad) Accuracy (standard deviation) of baseToRoverHeading
status	uint32_t	GNSS status (see eGnssStatus): [0x000000xx] number of satellites used, [0x0000xx00] fix type, [0x00xx0000] status flags, NMEA input flag
covEcefPacked	float[6]	RTK solution covariance in ECEF packed as [Pxx, Pyy, Pzz, Pxy, Pyz, Pzx], in meters^2

DID_GNSS1_SAT

`gnss_sat_t`

Field	Type	Description
timeOfWeekMs	uint32_t	GPS time of week (since Sunday morning) in milliseconds
numSats	uint32_t	Number of satellites in the sky (valid entries in the sat[] list below)
sat	gnss_sat_sv_t[80]	Per-satellite tracking information list

DID_GNSS1_VEL

`gnss_vel_t`

Field	Type	Description
timeOfWeekMs	uint32_t	GPS time of week (since Sunday morning) in milliseconds
vel	float[3]	GNSS velocity, in meters/second. In ECEF {vx,vy,vz} if status bit GNSS_STATUS_FLAGS_GNSS_NMEA_DATA (0x00008000) is NOT set; in local tangent plane with no vertical velocity {vNorth, vEast, 0} if that bit IS set.
sAcc	float	Speed accuracy, in meters/second
status	uint32_t	GNSS status (see eGnssStatus): [0x000000xx] number of satellites used, [0x0000xx00] fix type, [0x00xx0000] status flags, NMEA input flag

©2022

DID_GNSS1_VERSION

`gnss_version_t`

Field	Type	Description
swVersion	uint8_t[30]	GNSS receiver software version string
hwVersion	uint8_t[10]	GNSS receiver hardware version string
extension	gnss_extension_ver_t[6]	Additional 30-byte extension version-info strings reported by the receiver

DID_GNSS2_POS

GNSS 2 position data

`gnss_pos_t`

Field	Type	Description
week	uint32_t	GPS number of weeks since January 6^{th}, 1980
timeOfWeekMs	uint32_t	GPS time of week (since Sunday morning) in milliseconds
status	uint32_t	GNSS status (see eGnssStatus): [0x000000xx] number of satellites used, [0x0000xx00] fix type, [0x00xx0000] status flags, NMEA input flag
ecef	double[3]	Position in ECEF {x,y,z}, in meters
lla	double[3]	Position in WGS84 latitude, longitude, height above ellipsoid (not MSL), in degrees, degrees, meters
hMSL	float	Height above mean sea level (MSL), in meters
hAcc	float	Horizontal accuracy, in meters
vAcc	float	Vertical accuracy, in meters
pDop	float	Position dilution of precision (unitless)
cnoMean	float	Average of all non-zero satellite carrier to noise ratios (signal strengths), in dBHz
towOffset	double	Time sync offset between local time since boot up and GPS time of week, in seconds. Add this to IMU and sensor time to get GPS time of week in seconds.
leapS	uint8_t	GPS leap second (GPS-UTC) offset, in seconds. Receiver's best knowledge of the leap seconds offset from UTC to GPS time. Subtract from GPS time of week to get UTC time of week. (18 seconds as of December 31, 2016)
satsUsed	uint8_t	Number of satellites used in the position solution
cnoMeanSigma	uint8_t	Standard deviation of cnoMean over the past 5 seconds, in dBHz x10
status2	uint8_t	Secondary GNSS status byte (see eGnssStatus2): [0x0X] spoofing/jamming status, [0xX0] unused

DID_GNSS2_RTK_CMP_MISC

RTK Dual GNSS RTK compassing related data.

©2022

`gnss_rtk_misc_t`

©2022

Field	Type	Description
timeOfWeekMs	uint32_t	GPS time of week (since Sunday morning), in milliseconds
accuracyPos	float[3]	Accuracy: estimated standard deviations of the position solution assuming the a priori error model and error parameters used by the positioning options. []: {ECEF x,y,z} or {north,east,down}, in meters
accuracyCov	float[3]	Accuracy: estimated covariance of the position solution. []: absolute value of the square root of the estimated covariance {NE, EU, UN}, in meters
arThreshold	float	Ambiguity resolution threshold for validation (unitless)
gDop	float	Geometric dilution of precision (unitless)
hDop	float	Horizontal dilution of precision (unitless)
vDop	float	Vertical dilution of precision (unitless)
baseLla	double[3]	Base station position: latitude, longitude, height (deg, deg, m)
cycleSlipCount	uint32_t	Cycle slip counter
roverGpsObservationCount	uint32_t	Rover GPS observation element counter
baseGpsObservationCount	uint32_t	Base station GPS observation element counter
roverGlonassObservationCount	uint32_t	Rover GLONASS observation element counter
baseGlonassObservationCount	uint32_t	Base station GLONASS observation element counter
roverGalileoObservationCount	uint32_t	Rover Galileo observation element counter
baseGalileoObservationCount	uint32_t	Base station Galileo observation element counter
roverBeidouObservationCount	uint32_t	Rover BeiDou observation element counter
baseBeidouObservationCount	uint32_t	Base station BeiDou observation element counter
roverQzsObservationCount	uint32_t	Rover QZSS observation element counter
baseQzsObservationCount	uint32_t	Base station QZSS observation element counter
roverGpsEphemerisCount	uint32_t	Rover GPS ephemeris element counter
baseGpsEphemerisCount	uint32_t	Base station GPS ephemeris element counter
roverGlonassEphemerisCount	uint32_t	Rover GLONASS ephemeris element counter
baseGlonassEphemerisCount	uint32_t	Base station GLONASS ephemeris element counter
roverGalileoEphemerisCount	uint32_t	Rover Galileo ephemeris element counter
baseGalileoEphemerisCount	uint32_t	Base station Galileo ephemeris element counter
roverBeidouEphemerisCount	uint32_t	Rover BeiDou ephemeris element counter
baseBeidouEphemerisCount	uint32_t	Base station BeiDou ephemeris element counter
roverQzsEphemerisCount	uint32_t	Rover QZSS ephemeris element counter
baseQzsEphemerisCount	uint32_t	Base station QZSS ephemeris element counter
roverSbasCount	uint32_t	Rover SBAS element counter
baseSbasCount	uint32_t	Base station SBAS element counter
baseAntennaCount	uint32_t	Base station antenna position element counter
ionUtcAlmCount	uint32_t	Ionosphere model, UTC, and almanac element counter

©2022

Field	Type	Description
correctionChecksumFailures	uint32_t	Number of checksum failures from received corrections
timeToFirstFixMs	uint32_t	Time to first RTK fix, in milliseconds

DID_GNSS2_RTK_CMP_REL

Dual GNSS RTK compassing / moving base to rover (GNSS 1 to GNSS 2) relative info.

`gnss_rtk_rel_t`

Field	Type	Description
timeOfWeekMs	uint32_t	(ms) GPS time of week (since Sunday morning)
differentialAge	float	(s) Age of differential corrections
arRatio	float	Ambiguity resolution ratio factor for validation (unitless; higher indicates greater confidence the fixed integer ambiguity is correct)
baseToRoverVector	float[3]	(m) Vector from base to rover GNSS antennas {x,y,z} in ECEF. DID_GNSS1_RTK_POS_REL: RTK base station (base) to GNSS1 (rover). DID_GNSS2_RTK_CMP_REL (compassing): GNSS1 (base) to GNSS2 (rover)
baseToRoverDistance	float	(m) Distance from base to rover GNSS antennas (baseline length)
baseToRoverHeading	float	(rad) Heading of baseToRoverVector in the local tangent (NED) plane. Compassing mode: GNSS1 (base) to GNSS2 (rover)
baseToRoverHeadingAcc	float	(rad) Accuracy (standard deviation) of baseToRoverHeading
status	uint32_t	GNSS status (see eGnssStatus): [0x000000xx] number of satellites used, [0x0000xx00] fix type, [0x00xx0000] status flags, NMEA input flag
covEcefPacked	float[6]	RTK solution covariance in ECEF packed as [Pxx, Pyy, Pzz, Pxy, Pyz, Pzx], in meters^2

DID_GNSS2_SAT

GNSS 2 GNSS satellite information: sat identifiers, carrier to noise ratio, elevation and azimuth angles, pseudo range residual.

`gnss_sat_t`

Field	Type	Description
timeOfWeekMs	uint32_t	GPS time of week (since Sunday morning) in milliseconds
numSats	uint32_t	Number of satellites in the sky (valid entries in the sat[] list below)
sat	gnss_sat_sv_t[80]	Per-satellite tracking information list

DID_GNSS2_VEL

GNSS 2 velocity data

©2022

`gnss_vel_t`

Field	Type	Description
timeOfWeekMs	uint32_t	GPS time of week (since Sunday morning) in milliseconds
vel	float[3]	GNSS velocity, in meters/second. In ECEF {vx,vy,vz} if status bit GNSS_STATUS_FLAGS_GNSS_NMEA_DATA (0x00008000) is NOT set; in local tangent plane with no vertical velocity {vNorth, vEast, 0} if that bit IS set.
sAcc	float	Speed accuracy, in meters/second
status	uint32_t	GNSS status (see eGnssStatus): [0x000000xx] number of satellites used, [0x0000xx00] fix type, [0x00xx0000] status flags, NMEA input flag

DID_GNSS2_VERSION

GNSS 2 version info

`gnss_version_t`

Field	Type	Description
swVersion	uint8_t[30]	GNSS receiver software version string
hwVersion	uint8_t[10]	GNSS receiver hardware version string
extension	gnss_extension_ver_t[6]	Additional 30-byte extension version-info strings reported by the receiver

DID_GNSS_RTK_OPT

RTK options - requires little endian CPU.

©2022

`gnss_rtk_opt_t`

©2022

Field	Type	Description
mode	int32_t	Positioning mode (PMODE_???: e.g. single, DGPS, kinematic, static, moving-baseline)
soltype	int32_t	Solution type (0 = forward, 1 = backward, 2 = combined forward/backward)
nf	int32_t	Number of frequencies used (1 = L1, 2 = L1+L2, 3 = L1+L2+L5)
navsys	int32_t	Navigation systems bitmask (GPS/GLONASS/Galileo/BeiDou/QZSS/SBAS, SYS_??? bits)
elmin	float	Elevation mask angle, satellites below this are excluded (rad)
snrmin	int32_t	Minimum SNR/Cno for a satellite to be considered for RTK (0.25 dB-Hz units, see obsd_t.SNR)
snrrange	int32_t	SNR range from the highest-SNR satellite to consider (overrides snrmin if non-zero)
modear	int32_t	Integer ambiguity resolution (AR) mode (0 = off, 1 = continuous, 2 = instantaneous, 3 = fix-and-hold, 4 = PPP-AR)
glomodear	int32_t	GLONASS ambiguity resolution mode (0 = off, 1 = on, 2 = auto-calibrate inter-frequency bias, 3 = external calibration)
sbsmodear	int32_t	SBAS ambiguity resolution mode (0 = off, 1 = on)
bdsmodear	int32_t	BeiDou ambiguity resolution mode (0 = off, 1 = on)
arfilter	int32_t	Ambiguity-resolution filtering to reject bad satellites (0 = off, 1 = on)
maxout	int32_t	Consecutive observation outage count before resetting a satellite's carrier-phase bias
maxrej	int32_t	Consecutive rejection count before resetting a satellite's carrier-phase bias
minlock	int32_t	Minimum lock (continuous-tracking) count required before fixing an ambiguity
minfixsats	int32_t	Minimum number of satellites required to fix integer ambiguities
minholdsats	int32_t	Minimum number of satellites required to hold fixed integer ambiguities
mindropsats	int32_t	Minimum number of satellites below which satellites are dropped during ambiguity resolution
rcvstds	int32_t	Use receiver-reported stdev estimates to scale measurement variances (0 = off, 1 = on)
minfix	int32_t	Minimum consecutive fix count required before holding an ambiguity
armaxiter	int32_t	Maximum number of iterations used to resolve integer ambiguities
dynamics	int32_t	Dynamics model used by the filter (0 = none, 1 = velocity, 2 = acceleration)
intpref	int32_t	Interpolate reference (base) observations, used for post-mission processing (0 = off, 1 = on)
rovpos	int32_t	Rover position mode for fixed-position solutions
refpos	int32_t	Base station position mode for relative-positioning solutions
err	float[12]	Measurement error factor coefficients (indexed by error model term)
std	float[3]	Initial-state standard deviations: [0] carrier-phase bias, [1] ionosphere, [2] troposphere
prn	float[6]	Process-noise standard deviations: [0] bias, [1] iono, [2] trop, [3] horizontal accel, [4] vertical accel, [5] position

©2022

Field	Type	Description
sclkstab	double	Satellite clock stability (sec/sec, i.e. fractional frequency error)
thresar	float[8]	Ambiguity-resolution validation thresholds (e.g. ratio test and related AR acceptance criteria)
elmaskar	float	Elevation mask for ambiguity resolution of a newly rising satellite (rad)
elmaskhold	float	Elevation mask below which a held ambiguity is dropped (rad)
thresslip	float	Cycle-slip detection threshold on the geometry-free phase combination (m)
thresdop	float	Cycle-slip detection threshold based on Doppler-predicted phase (m)
varholdamb	float	Variance assigned to fix-and-hold pseudo-measurements of ambiguity (cycle^2)
gainholdamb	float	Gain applied to GLONASS and SBAS satellites when adjusting held ambiguities
maxtdiff	float	Maximum allowed time difference between rover and base observations (sec)
fix_reset_base_msgs	int	Number of base messages without a fix after which satellite biases are reset
maxinno	float[2]	Innovation (measurement residual) rejection thresholds: [0] carrier-phase, [1] code/pseudorange (m)
maxnis_lo	float[2]	Lower normalized innovation squared (NIS) rejection thresholds: [0] phase, [1] code
maxnis_hi	float[2]	Upper normalized innovation squared (NIS) rejection thresholds: [0] phase, [1] code
maxgdop	double	Rejection threshold on geometric dilution of precision (GDOP)
baseline	float[3]	Baseline length constraint: {constrained length, sigma before fix, sigma after fix} (m)
max_baseline_error	float	Maximum allowed baseline length error before the solution is considered invalid (m)
reset_baseline_error	float	Baseline length error above which the filter is reset (m)
max_ubx_error	float	Maximum error with respect to the receiver's (u-blox) reported position, triggers a reset if exceeded (m)
ru	double[3]	Rover position for fixed-position mode {x,y,z} (ECEF, m)
rb	double[3]	Base station position for relative-positioning mode {x,y,z} (ECEF, m)
maxaveep	int32_t	Maximum number of epochs used when averaging a position
outsingle	int32_t	Output a single-point solution on DGPS/float/fixed/PPP outage (0 = off, 1 = on)
velcon	float[2]	Velocity constraint variance in compassing/moving-baseline mode: {before fix, after fix} (m^2/s^2)
mp_bias_lpf_alpha	float	Low-pass-filter alpha for multipath bias estimation; smaller values apply heavier filtering
mp_var_lpf_alpha	float	Low-pass-filter alpha for multipath variance estimation; smaller values apply heavier filtering

GPX

DID_GPX_DEV_INFO

GPX device information

©2022

`dev_info_t`

Field	Type	Description
reserved	uint8_t	Reserved bits
buildFlags	uint8_t	Build flags: 0x1=debug mode, 0x2=dirty (see eBuildFlags)
hardwareType	uint8_t	Hardware Type: 1=uINS, 2=EVB, 3=IMX, 4=GPX (see eIsHardwareType)
hdwRunState	uint8_t	Device Run State: Bootloader, App, etc (see eHdwRunStates)
serialNumber	uint32_t	Serial number
hardwareVer	uint8_t[3]	Hardware version: [0]=major, [1]=minor, [2]=pcb/a revision
hardwareVariant	uint8_t	Hardware variant; an arbitrary number independent of version that identifies some variation of the hardware; most commonly, IMU population type on IMX-6 (see eImx6ImuPopulationType)
firmwareVer	uint8_t[4]	Firmware (software) version
buildNumber	uint32_t	Build number
protocolVer	uint8_t[4]	Communications protocol version
repoRevision	uint32_t	Repository revision number
manufacturer	char[24]	Manufacturer name
buildType	uint8_t	Build type (0=production, 'c'=release candidate, 'b'=beta, 'a'=alpha, 'd'=developer, 's'=snapshot, '^'=dirty)
buildYear	uint8_t	Build date year - 2000
buildMonth	uint8_t	Build date month
buildDay	uint8_t	Build date day
buildHour	uint8_t	Build time hour
buildMinute	uint8_t	Build time minute
buildSecond	uint8_t	Build time second
buildMillisecond	uint8_t	Build time millisecond
addInfo	char[24]	Additional info

©2022

DID_GPX_FLASH_CFG

`nvm_flash_cfg_t`

©2022

Field	Type	Description
size	uint32_t	Size of group or union, which is nvm_group_x_t + padding
checksum	uint32_t	Checksum, excluding size and checksum. 0xFFFFFFFF is invalid.
key	uint32_t	Manufacturer method for restoring flash defaults
startupImuDtMs	uint32_t	(ms) IMU sample (system input) period set on startup. Cannot be larger than startupNavDtMs. Zero disables sensor/IMU sampling.
startupNavDtMs	uint32_t	(ms) Navigation filter (system output) output period set on startup. Used to initialize sysParams.navOutputPeriodMs.
ser0BaudRate	uint32_t	(bps) Serial port 0 baud rate
ser1BaudRate	uint32_t	(bps) Serial port 1 baud rate
insRotation	float[3]	(rad) Rotation about the X,Y,Z axes from Sensor Frame to Intermediate Output Frame. Order applied: Z,Y,X.
insOffset	float[3]	(m) X,Y,Z offset from Intermediate Output Frame to INS Output Frame.
gnss1AntOffset	float[3]	(m) X,Y,Z offset in Sensor Frame to GNSS 1 antenna.
dynamicModel	uint8_t	INS dynamic platform model (see eDynamicModel). Options are: 0=PORTABLE, 2=STATIONARY, 3=PEDESTRIAN, 4=GROUND VEHICLE, 5=SEA, 6=AIRBORNE_1G, 7=AIRBORNE_2G, 8=AIRBORNE_4G, 9=WRIST. Used to balance noise and performance characteristics of the system. The dynamics selected here must be at least as fast as your system or you experience accuracy error. This is tied to the GNSS position estimation model and intend in the future to be incorporated into the INS position model.
debug	uint8_t	Debug
gnssSatSigConst	uint16_t	Satellite system constellation used in GNSS solution (see eGnssSatSigConst). 0x0003=GPS, 0x000C=QZSS, 0x0030=Galileo, 0x00C0=Beidou, 0x0300=GLONASS, 0x1000=SBAS
sysCfgBits	uint32_t	System configuration bits (see eSysConfigBits).
refLla	double[3]	(deg, deg, m) Reference latitude, longitude and height above ellipsoid for north east down (NED) calculations
lastLla	double[3]	(deg, deg, m) Last latitude, longitude, HAE (height above ellipsoid) used to aid GNSS startup. Updated when the distance between current LLA and lastLla exceeds lastLlaUpdateDistance.
lastLlaTimeOfWeekMs	uint32_t	(ms) Last LLA GPS time since week start (Sunday morning)
lastLlaWeek	uint32_t	Last LLA GPS number of weeks since January 6th, 1980
lastLlaUpdateDistance	float	(m) Distance between current and last LLA that triggers an update of lastLla
ioConfig	uint32_t	Hardware interface configuration bits (see eIoConfig).
platformConfig	uint32_t	Hardware platform specifying the IMX carrier board type (i.e. RUG, EVB, IG) and configuration bits (see ePlatformConfig). The platform type is used to simplify the GNSS and I/O configuration

©2022

Field	Type	Description
		process. Bit PLATFORM_CFG_UPDATE_IO_CONFIG is excluded from the flashConfig checksum and from determining whether to upload.
gnss2AntOffset	float[3]	(m) X,Y,Z offset in Sensor Frame origin to GNSS 2 antenna.
zeroVelRotation	float[3]	(rad) Euler (roll, pitch, yaw) rotation from INS Sensor Frame to Intermediate ZeroVelocity Frame. Order applied: heading, pitch, roll.
zeroVelOffset	float[3]	(m) X,Y,Z offset from Intermediate ZeroVelocity Frame to Zero Velocity Frame.
gnssTimeUserDelay	float	(sec) User defined delay for GPS time. This parameter can be used to account for GNSS antenna cable delay.
magDeclination	float	(rad) Earth magnetic field (magnetic north) declination (heading offset from true north)
gnssTimeSyncPeriodMs	uint32_t	(ms) Time between GPS time synchronization pulses. Requires reboot to take effect.
startupGnssDtMs	uint32_t	(ms) GNSS measurement (system input) update period set on startup. 200ms minimum (5Hz max).
RTKCfgBits	uint32_t	RTK configuration bits (see eRTKConfigBits).
sensorConfig	uint32_t	Sensor config to specify the full-scale sensing ranges and output rotation for the IMU and magnetometer (see eSensorConfig)
gnssMinimumElevation	float	(rad) Minimum elevation of a satellite above the horizon to be used in the solution. Low elevation satellites may provide degraded accuracy, due to the long signal path through the atmosphere.
ser2BaudRate	uint32_t	(bps) Serial port 2 baud rate
wheelConfig	wheel_config_t	Wheel encoder: euler angles describing the rotation from imu to left wheel, plus track width/radius and config bits (see eWheelCfgBits)
magInterferenceThreshold	float	Magnetometer interference sensitivity threshold. Typical range is 2-10 (3 default) and 1000 to disable mag interference detection.
magCalibrationQualityThreshold	float	Magnetometer calibration quality sensitivity threshold. Typical range is 10-20 (10 default) and 1000 to disable mag calibration quality check, forcing it to be always good.
gnssCn0Minimum	uint8_t	(dBHz) GNSS CN0 absolute minimum threshold for signals. Used to filter signals in RTK solution.
gnssCn0DynMinOffset	uint8_t	(dBHz) GNSS CN0 dynamic minimum threshold offset below max CN0 across all satellites. Used to filter signals used in RTK solution. To disable, set gnssCn0DynMinOffset to zero and increase gnssCn0Minimum.
imuRejectThreshGyroLow	uint8_t	IMU gyro fault rejection threshold low
imuRejectThreshGyroHigh	uint8_t	IMU gyro fault rejection threshold high
imuShockDetectLatencyMs	uint8_t	(ms) IMU shock detection latency. Time used for EKF rewind to prevent shock from influencing EKF estimates.
imuShockRejectLatchMs	uint8_t	

©2022

Field	Type	Description
		(ms) IMU shock rejection latch time. Time required following detected shock end to disable shock rejection.
imuShockOptions	uint8_t	IMU shock rejection options (see eImuShockOptions)
imuShockDeltaAccHighThreshold	uint8_t	(m/s^2) IMU shock detection. Min acceleration difference between the 3 IMUs to detect the start of a shock.
imuShockDeltaAccLowThreshold	uint8_t	(m/s^2) IMU shock detection. Max acceleration difference between the 3 IMUs within the latch time to detect the end of a shock.
imuShockDeltaGyroHighThreshold	uint8_t	(deg/s) IMU shock detection. Min angular rate difference between the 3 IMUs to detect the start of a shock.
imuShockDeltaGyroLowThreshold	uint8_t	(deg/s) IMU shock detection. Max angular rate difference between the 3 IMUs within the latch time to detect the end of a shock.
ioConfig2	uint8_t	Hardware interface configuration bits for GNSS2 PPS (see eIoConfig2).

DID_GPX_RMC

GPX rmc

`rmc_t`

Field	Type	Description
bits	uint64_t	Data stream enable bits for the specified ports (see RMC_BITS_...)
options	uint32_t	Options to select alternate ports to output data, persist across reboot, NMEA speed filtering, etc. (see RMC_OPTIONS_...)

©2022

DID_GPX_STATUS

`gpx_status_t`

Field	Type	Description
timeOfWeekMs	uint32_t	(ms) GPS time of week (since Sunday morning)
status	uint32_t	Status (see eGpxStatus)
grmcBitsSer0	uint64_t	GRMC message enable bits for serial port 0 (see GRMC_BITS_...)
grmcBitsSer1	uint64_t	GRMC message enable bits for serial port 1 (see GRMC_BITS_...)
grmcBitsSer2	uint64_t	GRMC message enable bits for serial port 2 (see GRMC_BITS_...)
grmcBitsUSB	uint64_t	GRMC message enable bits for USB (see GRMC_BITS_...)
grmcNMEABitsSer0	uint64_t	NMEA message enable bits for serial port 0 (see NMEA_MSG_ID...)
grmcNMEABitsSer1	uint64_t	NMEA message enable bits for serial port 1 (see NMEA_MSG_ID...)
grmcNMEABitsSer2	uint64_t	NMEA message enable bits for serial port 2 (see NMEA_MSG_ID...)
grmcNMEABitsUSB	uint64_t	NMEA message enable bits for USB (see NMEA_MSG_ID...)
hdwStatus	uint32_t	Hardware status flags (see eGPXHdwStatusFlags)
mcuTemp	float	(C) MCU temperature (GPX_INVALID_MCU_TEMP if not available)
navOutputPeriodMs	uint32_t	(ms) Navigation output period
flashCfgChecksum	uint32_t	Flash config checksum used with host SDK synchronization
rtkMode	uint32_t	RTK mode bits (see eRTKConfigBits)
gnssStatus	gpx_gnss_status_t[2]	Per-GNSS-receiver driver status (GNSS1, GNSS2)
gpxSourcePort	uint8_t	Port this status message was sourced from
upTime	double	(s) Time since system was started

Raw GPS Data

Raw GPS data is contained in the `DID_GNSS1_RAW`, `DID_GNSS2_RAW`, and `DID_GNSS_BASE_RAW` messages of type `gnss_raw_t`. The actual raw data is contained in the union member `gnss_raw_t.data` and should be interpreted based on the value of `gnss_raw_t.dataType` (i.e. as observation, ephemeris, SBAS, or base station position).

DID_GNSS1_RAW

GNSS raw data for rover (observation, ephemeris, etc.) - requires little endian CPU. The contents of data can vary for this message and are determined by dataType field. RTK positioning or RTK compassing must be enabled to stream this message.

`gnss_raw_t`

Field	Type	Description
receiverIndex	uint8_t	Source receiver: 1=RECEIVER_INDEX_GNSS1, 2=RECEIVER_INDEX_EXTERNAL_BASE, 3=RECEIVER_INDEX_GNSS2
dataType	uint8_t	Type of data in the data union (see eRawDataType): 1=observations, 2=ephemeris, 3=glonassEphemeris, 4=SBAS, 5=baseAntenna, 6=IonosphereModel
obsCount	uint8_t	Number of observations (obsd_t elements) present in data.obs when dataType==1 (raw_data_type_observation)
reserved	uint8_t	Reserved for alignment / future use
data	uGnssRawData	Raw data payload; interpret based on dataType (see eRawDataType)

©2022

DID_GNSS2_RAW

GNSS raw data for rover (observation, ephemeris, etc.) - requires little endian CPU. The contents of data can vary for this message and are determined by dataType field. RTK positioning or RTK compassing must be enabled to stream this message.

`gnss_raw_t`

Field	Type	Description
receiverIndex	uint8_t	Source receiver: 1=RECEIVER_INDEX_GNSS1, 2=RECEIVER_INDEX_EXTERNAL_BASE, 3=RECEIVER_INDEX_GNSS2
dataType	uint8_t	Type of data in the data union (see eRawDataType): 1=observations, 2=ephemeris, 3=glonassEphemeris, 4=SBAS, 5=baseAntenna, 6=IonosphereModel
obsCount	uint8_t	Number of observations (obsd_t elements) present in data.obs when dataType==1 (raw_data_type_observation)
reserved	uint8_t	Reserved for alignment / future use
data	uGnssRawData	Raw data payload; interpret based on dataType (see eRawDataType)

DID_GNSS_BASE_RAW

`gnss_raw_t`

Field	Type	Description
receiverIndex	uint8_t	Source receiver: 1=RECEIVER_INDEX_GNSS1, 2=RECEIVER_INDEX_EXTERNAL_BASE, 3=RECEIVER_INDEX_GNSS2
dataType	uint8_t	Type of data in the data union (see eRawDataType): 1=observations, 2=ephemeris, 3=glonassEphemeris, 4=SBAS, 5=baseAntenna, 6=IonosphereModel
obsCount	uint8_t	Number of observations (obsd_t elements) present in data.obs when dataType==1 (raw_data_type_observation)
reserved	uint8_t	Reserved for alignment / future use
data	uGnssRawData	Raw data payload; interpret based on dataType (see eRawDataType)

RAW GPS DATA BUFFER UNION

`uGnssRawData`

Field	Type	Description
obs	obsd_t[]	Satellite observation data, valid when dataType == raw_data_type_observation
eph	eph_t	Satellite non-GLONASS ephemeris data (GPS, Galileo, Beidou, QZSS), valid when dataType == raw_data_type_ephemeris
gloEph	geph_t	Satellite GLONASS ephemeris data, valid when dataType == raw_data_type_glonass_ephemeris
sbas	sbsmsg_t	Satellite-Based Augmentation Systems (SBAS) data, valid when dataType == raw_data_type_sbas
sta	sta_t	Base station information (base position, antenna position, antenna height, etc.), valid when dataType == raw_data_type_base_station_antenna_position
ion	ion_model_utc_alm_t	Ionosphere model and UTC parameters, valid when dataType == raw_data_type_ionosphere_model_utc_alm
buf	uint8_t[1000]	Byte buffer providing untyped access to the same union storage

©2022

GPS GALILEO QZSS EPHEMERIS

`eph_t`

©2022

Field	Type	Description
sat	int32_t	Satellite number in RTKLIB notation. GPS: 1-32, GLONASS: 33-59, Galileo: 60-89, SBAS: 90-95
iode	int32_t	IODE, Issue Of Data Ephemeris (ephemeris data-set version number)
iodc	int32_t	IODC, Issue Of Data Clock (clock data-set version number)
sva	int32_t	SV accuracy (URA index), see IS-GPS-200/IRN-IS-200H p.97
svh	int32_t	SV health for GPS/QZS (0 = ok)
week	int32_t	Ephemeris reference week number: GPS week for GPS/QZS, Galileo week (GST week) for GAL
code	int32_t	GPS/QZS: code on L2 (00 = invalid, 01 = P-code on, 11 = C/A-code on, 11 = invalid). GAL/CMP (BeiDou): data source indicator
flag	int32_t	GPS/QZS: L2 P-code data flag (nonzero indicates the NAV data stream is commanded OFF on the P-code of the L2 in-phase component). CMP (BeiDou): nav message type
toe	gtime_t	Time Of Ephemeris: ephemeris reference epoch (GPST)
toc	gtime_t	Clock data reference time (GPST), see IS-GPS-200 20.3.4.5
ttr	gtime_t	Transmission time of the message, T_trans (GPST)
A	double	Orbit semi-major axis (m)
e	double	Orbit eccentricity (dimensionless)
i0	double	Orbit inclination angle at reference time (rad)
OMG0	double	Longitude of ascending node of orbit plane at the weekly epoch (rad)
omg	double	Argument of perigee (rad)
M0	double	Mean anomaly at reference time (rad)
deln	double	Mean motion difference from computed value, delta-n (rad/s)
OMGd	double	Rate of right ascension / longitude of ascending node, OMEGA-dot (rad/s)
idot	double	Rate of inclination angle, i-dot (rad/s)
crc	double	Amplitude of the cosine harmonic correction term to the orbit radius (m)
crs	double	Amplitude of the sine harmonic correction term to the orbit radius (m)
cuc	double	Amplitude of the cosine harmonic correction term to the argument of latitude (rad)
cus	double	Amplitude of the sine harmonic correction term to the argument of latitude (rad)
cic	double	Amplitude of the cosine harmonic correction term to the angle of inclination (rad)
cis	double	Amplitude of the sine harmonic correction term to the angle of inclination (rad)
toes	double	Time Of Ephemeris in seconds within the week (s), the double-precision counterpart of toe above. toe is computed as eph->toe = gst2time(week, eph->toes); this value is the ephemeris expiration reference and is generally ~2 hours ahead of the current time
fit	double	Curve-fit interval (h) (0 = 4 hours, 1 = greater than 4 hours)
f0	double	SV clock bias, af0 (s)
f1	double	SV clock drift, af1 (s/s, dimensionless)
f2	double	SV clock drift rate, af2 (s/s^2)
tgd	double[4]	

©2022

Field	Type	Description
		Group delay parameters. GPS/QZS: tgd[0] = TGD (IRN-IS-200H p.103). Galileo: tgd[0] = BGD E5a/E1, tgd[1] = BGD E5b/E1. BeiDou: tgd[0] = BGD1, tgd[1] = BGD2
Adot	double	Semi-major axis rate, A-dot, for CNAV messages; not used
ndot	double	First derivative of mean motion n (equivalently second derivative of mean anomaly M), n-dot, for CNAV messages (rad/s^2); not used

GLONASS EPHEMERIS

`geph_t`

Field	Type	Description
sat	int32_t	Satellite number in RTKLIB notation. GPS: 1-32, GLONASS: 33-59, Galileo: 60-89, SBAS: 90-95
iode	int32_t	IODE, derived from bits 0-6 of the tb (time interval index) field
frq	int32_t	Satellite frequency channel number (GLONASS FDMA slot, -7..+13)
svh	int32_t	Satellite health flag
sva	int32_t	Satellite accuracy (URA-like indicator)
age	int32_t	Satellite age of operation (days)
toe	gtime_t	Ephemeris reference epoch within the week, GPS time system (GPST)
tof	gtime_t	Message frame transmission time, GPS time system (GPST)
pos	double[3]	Satellite position at toe (ECEF) (m)
vel	double[3]	Satellite velocity at toe (ECEF) (m/s)
acc	double[3]	Satellite (luni-solar) acceleration at toe (ECEF) (m/s^2)
taun	double	SV clock bias, -tau_n (s)
gamn	double	Relative frequency bias, gamma_n (dimensionless, s/s)
dtaun	double	Time delay between the L1 and L2 signal transmissions (s)

SBAS

`sbsmsg_t`

Field	Type	Description
week	int32_t	GPS week number of message reception
tow	int32_t	Time of week of message reception (s)
prn	int32_t	SBAS satellite PRN number
msg	uint8_t[29]	Raw SBAS message payload (226 bits), zero-padded to 29 bytes
reserved	uint8_t[3]	Reserved/unused (padding for alignment)

©2022

STATION PARAMETERS

`sta_t`

Field	Type	Description
deltype	int32_t	Antenna delta (offset) type: 0 = e/n/u (east/north/up), 1 = x/y/z (ECEF)
pos	double[3]	Station reference position (ECEF) (m)
del	double[3]	Antenna position delta from the station reference position, interpreted per deltype: e/n/u or x/y/z (m)
hgt	double	Antenna height above the marker/reference point (m)
stationId	int32_t	Station identifier (e.g. RTCM/RINEX station ID)

SATELLITE OBSERVATION

`obs_t`

Field	Type	Description
n	uint32_t	Number of observation slots currently used (valid entries in data[])
nmax	uint32_t	Number of observation slots allocated in data[]
data	obsd_t	Pointer to the observation data buffer, an array of obsd_t records

SATELLITE INFORMATION

`gnss_sat_sv_t`

Field	Type	Description
gnssId	uint8_t	GNSS constellation identifier (see eSatSvGnssId)
svId	uint8_t	Satellite identifier (PRN/slot number, meaning depends on gnssId)
elev	int8_t	Elevation, in degrees (range: ±90)
azim	int16_t	Azimuth, in degrees (range: ±180)
cno	uint8_t	Carrier to noise ratio (signal strength), in dBHz
status	uint16_t	Satellite status bitflags (see eSatSvStatus)

INERTIAL MEASUREMENT UNIT (IMU)

`imui_t`

Field	Type	Description
pqr	float[3]	Gyroscope P, Q, R (angular rate about body X, Y, Z) in radians/second
acc	float[3]	Acceleration X, Y, Z in meters/second^2, in body frame

©2022

Configuration

DID_FLASH_CONFIG

`nvm_flash_cfg_t`

©2022

Field	Type	Description
size	uint32_t	Size of group or union, which is nvm_group_x_t + padding
checksum	uint32_t	Checksum, excluding size and checksum. 0xFFFFFFFF is invalid.
key	uint32_t	Manufacturer method for restoring flash defaults
startupImuDtMs	uint32_t	(ms) IMU sample (system input) period set on startup. Cannot be larger than startupNavDtMs. Zero disables sensor/IMU sampling.
startupNavDtMs	uint32_t	(ms) Navigation filter (system output) output period set on startup. Used to initialize sysParams.navOutputPeriodMs.
ser0BaudRate	uint32_t	(bps) Serial port 0 baud rate
ser1BaudRate	uint32_t	(bps) Serial port 1 baud rate
insRotation	float[3]	(rad) Rotation about the X,Y,Z axes from Sensor Frame to Intermediate Output Frame. Order applied: Z,Y,X.
insOffset	float[3]	(m) X,Y,Z offset from Intermediate Output Frame to INS Output Frame.
gnss1AntOffset	float[3]	(m) X,Y,Z offset in Sensor Frame to GNSS 1 antenna.
dynamicModel	uint8_t	INS dynamic platform model (see eDynamicModel). Options are: 0=PORTABLE, 2=STATIONARY, 3=PEDESTRIAN, 4=GROUND VEHICLE, 5=SEA, 6=AIRBORNE_1G, 7=AIRBORNE_2G, 8=AIRBORNE_4G, 9=WRIST. Used to balance noise and performance characteristics of the system. The dynamics selected here must be at least as fast as your system or you experience accuracy error. This is tied to the GNSS position estimation model and intend in the future to be incorporated into the INS position model.
debug	uint8_t	Debug
gnssSatSigConst	uint16_t	Satellite system constellation used in GNSS solution (see eGnssSatSigConst). 0x0003=GPS, 0x000C=QZSS, 0x0030=Galileo, 0x00C0=Beidou, 0x0300=GLONASS, 0x1000=SBAS
sysCfgBits	uint32_t	System configuration bits (see eSysConfigBits).
refLla	double[3]	(deg, deg, m) Reference latitude, longitude and height above ellipsoid for north east down (NED) calculations
lastLla	double[3]	(deg, deg, m) Last latitude, longitude, HAE (height above ellipsoid) used to aid GNSS startup. Updated when the distance between current LLA and lastLla exceeds lastLlaUpdateDistance.
lastLlaTimeOfWeekMs	uint32_t	(ms) Last LLA GPS time since week start (Sunday morning)
lastLlaWeek	uint32_t	Last LLA GPS number of weeks since January 6th, 1980
lastLlaUpdateDistance	float	(m) Distance between current and last LLA that triggers an update of lastLla
ioConfig	uint32_t	Hardware interface configuration bits (see eIoConfig).
platformConfig	uint32_t	Hardware platform specifying the IMX carrier board type (i.e. RUG, EVB, IG) and configuration bits (see ePlatformConfig). The platform type is used to simplify the GNSS and I/O configuration

©2022

Field	Type	Description
		process. Bit PLATFORM_CFG_UPDATE_IO_CONFIG is excluded from the flashConfig checksum and from determining whether to upload.
gnss2AntOffset	float[3]	(m) X,Y,Z offset in Sensor Frame origin to GNSS 2 antenna.
zeroVelRotation	float[3]	(rad) Euler (roll, pitch, yaw) rotation from INS Sensor Frame to Intermediate ZeroVelocity Frame. Order applied: heading, pitch, roll.
zeroVelOffset	float[3]	(m) X,Y,Z offset from Intermediate ZeroVelocity Frame to Zero Velocity Frame.
gnssTimeUserDelay	float	(sec) User defined delay for GPS time. This parameter can be used to account for GNSS antenna cable delay.
magDeclination	float	(rad) Earth magnetic field (magnetic north) declination (heading offset from true north)
gnssTimeSyncPeriodMs	uint32_t	(ms) Time between GPS time synchronization pulses. Requires reboot to take effect.
startupGnssDtMs	uint32_t	(ms) GNSS measurement (system input) update period set on startup. 200ms minimum (5Hz max).
RTKCfgBits	uint32_t	RTK configuration bits (see eRTKConfigBits).
sensorConfig	uint32_t	Sensor config to specify the full-scale sensing ranges and output rotation for the IMU and magnetometer (see eSensorConfig)
gnssMinimumElevation	float	(rad) Minimum elevation of a satellite above the horizon to be used in the solution. Low elevation satellites may provide degraded accuracy, due to the long signal path through the atmosphere.
ser2BaudRate	uint32_t	(bps) Serial port 2 baud rate
wheelConfig	wheel_config_t	Wheel encoder: euler angles describing the rotation from imu to left wheel, plus track width/radius and config bits (see eWheelCfgBits)
magInterferenceThreshold	float	Magnetometer interference sensitivity threshold. Typical range is 2-10 (3 default) and 1000 to disable mag interference detection.
magCalibrationQualityThreshold	float	Magnetometer calibration quality sensitivity threshold. Typical range is 10-20 (10 default) and 1000 to disable mag calibration quality check, forcing it to be always good.
gnssCn0Minimum	uint8_t	(dBHz) GNSS CN0 absolute minimum threshold for signals. Used to filter signals in RTK solution.
gnssCn0DynMinOffset	uint8_t	(dBHz) GNSS CN0 dynamic minimum threshold offset below max CN0 across all satellites. Used to filter signals used in RTK solution. To disable, set gnssCn0DynMinOffset to zero and increase gnssCn0Minimum.
imuRejectThreshGyroLow	uint8_t	IMU gyro fault rejection threshold low
imuRejectThreshGyroHigh	uint8_t	IMU gyro fault rejection threshold high
imuShockDetectLatencyMs	uint8_t	(ms) IMU shock detection latency. Time used for EKF rewind to prevent shock from influencing EKF estimates.
imuShockRejectLatchMs	uint8_t	

©2022

Field	Type	Description
		(ms) IMU shock rejection latch time. Time required following detected shock end to disable shock rejection.
imuShockOptions	uint8_t	IMU shock rejection options (see eImuShockOptions)
imuShockDeltaAccHighThreshold	uint8_t	(m/s^2) IMU shock detection. Min acceleration difference between the 3 IMUs to detect the start of a shock.
imuShockDeltaAccLowThreshold	uint8_t	(m/s^2) IMU shock detection. Max acceleration difference between the 3 IMUs within the latch time to detect the end of a shock.
imuShockDeltaGyroHighThreshold	uint8_t	(deg/s) IMU shock detection. Min angular rate difference between the 3 IMUs to detect the start of a shock.
imuShockDeltaGyroLowThreshold	uint8_t	(deg/s) IMU shock detection. Max angular rate difference between the 3 IMUs within the latch time to detect the end of a shock.
ioConfig2	uint8_t	Hardware interface configuration bits for GNSS2 PPS (see eIoConfig2).

DID_NMEA_BCAST_PERIOD

`nmea_msgs_t`

Field	Type	Description
options	uint32_t	Options: Port selection[0x0=current, 0x1=ser0, 0x2=ser1, 0x4=ser2, 0x8=USB, 0x100=preserve, 0x200=Persistent] (see RMC_OPTIONS_...)
nmeaBroadcastMsgs	nmeaBroadcastMsgPair_t[20]	NMEA message to be set. Up to 20 message ID/period pairs. Message ID of zero indicates the remaining pairs are not used. (see eNmeaMsgId)

DID_RMC

`rmc_t`

Field	Type	Description
bits	uint64_t	Data stream enable bits for the specified ports (see RMC_BITS_...)
options	uint32_t	Options to select alternate ports to output data, persist across reboot, NMEA speed filtering, etc. (see RMC_OPTIONS_...)

Command

DID_SYS_CMD

is the bitwise inverse of command and must be sent in the same write for the command to be processed.

`system_command_t`

Field	Type	Description
command	uint32_t	System command (see eSystemCommand) 1=save current persistent messages, 5=zero motion, 97=save flash, 99=software reset. "invCommand" (following variable) must be set to bitwise inverse of this value for this command to be processed.
invCommand	uint32_t	Error checking field that must be set to bitwise inverse of command field for the command to take effect.

©2022

EVB-2

DID_EVB_FLASH_CFG

EVB configuration.

©2022

`evb_flash_cfg_t`

©2022

Field	Type	Description
size	uint32_t	Size of this struct
checksum	uint32_t	Checksum, excluding size and checksum
key	uint32_t	Manufacturer method for restoring flash defaults
cbPreset	uint8_t	Communications bridge preset (see eEvb2ComBridgePreset)
reserved1	uint8_t[3]	Reserved for 32-bit alignment
cbf	uint32_t[EVB2_PORT_COUNT]	Communications bridge forwarding, indexed by eEvb2CommPorts
cbOptions	uint32_t	Communications bridge options (see eEvb2ComBridgeOptions)
bits	uint32_t	Config bits (see eEvbFlashCfgBits)
radioPID	uint32_t	Radio preamble ID (PID), 0x0 to 0x9. Only radios with matching PIDs can communicate together. Different PIDs minimize interference between multiple sets of networks. Checked before the network ID.
radioNID	uint32_t	Radio network ID (NID), 0x0 to 0x7FFF. Only radios with matching NID can communicate together. Checked after the preamble ID.
radioPowerLevel	uint32_t	Radio transmitter output power level (XBee PRO SX 0=20dBm, 1=27dBm, 2=30dBm)
wifi	evb_wifi_t[3]	WiFi SSID and PSK presets
server	evb_server_t[3]	Server IP and port presets
encoderTickToWheelRad	float	(rad) Encoder tick to wheel rotation conversion factor. Encoder tick count per revolution on 1 channel x gear ratio x 2pi.
CANbaud_kbps	uint32_t	(kbps) CAN baudrate
can_receive_address	uint32_t	CAN receive address
uinsComPort	uint8_t	EVB port for uINS communications and SD card logging. 0=uINS-Ser0 (default), 1=uINS-Ser1, SP330=5, 6=GPIO_H8 (use eEvb2CommPorts)
uinsAuxPort	uint8_t	EVB port for uINS aux com and RTK corrections. 0=uINS-Ser0, 1=uINS-Ser1 (default), 5=SP330, 6=GPIO_H8 (use eEvb2CommPorts)
reserved2	uint8_t[2]	Reserved to ensure 32-bit alignment
portOptions	uint32_t	Enable radio RTK filtering, etc. (see eEvb2PortOptions)
h3sp330BaudRate	uint32_t	(bps) Baud rate for EVB serial port H3 (SP330 RS233 and RS485/422)
h4xRadioBaudRate	uint32_t	(bps) Baud rate for EVB serial port H4 (TTL to external radio)
h8gpioBaudRate	uint32_t	(bps) Baud rate for EVB serial port H8 (TTL)
wheelCfgBits	uint32_t	Wheel encoder configuration (see eWheelCfgBits)
velocityControlPeriodMs	uint32_t	

©2022

Field	Type	Description
		(ms) Wheel update period. Sets the wheel encoder and control update period.

DID_EVB_STATUS

`evb_status_t`

Field	Type	Description
week	uint32_t	GPS number of weeks since January 6th, 1980
timeOfWeekMs	uint32_t	(ms) GPS time of week (since Sunday morning)
firmwareVer	uint8_t[4]	Firmware (software) version
evbStatus	uint32_t	Status (see eEvbStatus)
loggerMode	uint32_t	Data logger control state (see eEvb2LoggerMode)
loggerElapsedTimeMs	uint32_t	(ms) Elapsed time of the current data log
wifiIpAddr	uint32_t	WiFi IP address
sysCommand	uint32_t	System command (see eSystemCommand). 99 = software reset
towOffset	double	(s) Time sync offset between local time since boot up to GPS time of week. Add this to IMU and sensor time to get GPS time of week in seconds.

©2022

General

DID_BIT

`bit_t`

Field	Type	Description
command	uint8_t	BIT input command (see eBitCommand). Ignored when zero.
lastCommand	uint8_t	BIT last input command (see eBitCommand)
state	uint8_t	BIT current state (see eBitState)
reserved	uint8_t	Unused
hdwBitStatus	uint32_t	Hardware BIT status (see eHdwBitStatusFlags)
calBitStatus	uint32_t	Calibration BIT status (see eCalBitStatusFlags)
tcPqrBias	float	(rad/s) Gyro bias residual from temperature calibration
tcAccBias	float	(m/s^2) Accelerometer bias residual from temperature calibration
tcPqrSlope	float	(rad/s per deg C) Gyro temperature-compensation slope error
tcAccSlope	float	(m/s^2 per deg C) Accelerometer temperature-compensation slope error
tcPqrLinearity	float	(rad/s) Gyro temperature-compensation curve-fit linearity error
tcAccLinearity	float	(m/s^2) Accelerometer temperature-compensation curve-fit linearity error
pqr	float	(rad/s) Gyro angular rate error measured during BIT
acc	float	(m/s^2) Accelerometer error measured during BIT
pqrSigma	float	(rad/s) Standard deviation of the gyro angular rate error, over the BIT sample window
accSigma	float	(m/s^2) Standard deviation of the accelerometer error, over the BIT sample window
testMode	uint8_t	Self-test/fault-simulation mode (see eBitTestMode)
testVar	uint8_t	Self-test mode bi-directional variable used with testMode
detectedHardwareId	uint16_t	Detected hardware type (see "Product Hardware ID"), used to ensure correct firmware is used

DID_CANFD_CONFIG

CAN FD configuration: FD message broadcast rates, transmit addresses, and baud rate. Shares the same data structure as DID_CAN_CONFIG.

©2022

`can_config_t`

Field	Type	Description
can_period_mult	uint16_t[]	Broadcast period multiple for each CAN message. 0 disables the message. Indices 0..NUM_CIDS-1 correspond to classic can_cid_t values. In CAN-FD mode the same array is reused: indices 0..NUM_FDCIDS-1 correspond to canfd_cid_t values (FDCID_INS_1=0, FDCID_INS_2=1, ...). NUM_FDCIDS < NUM_CIDS so there is no overlap.
can_transmit_address	uint32_t[]	Transmit address for each CAN message. Indices 0..NUM_CIDS-1 correspond to classic can_cid_t values. In CAN-FD mode indices 0..NUM_FDCIDS-1 correspond to canfd_cid_t values and are validated / defaulted by CAN_init() when FD is enabled.
can_setting	uint16_t	Baud rate (kbps) (See can_baudrate_t for valid baud rates). Bit 15 (CAN_BAUDRATE_KBPS_FD_ENABLE) enables CAN-FD on capable hardware.
can_receive_address	uint32_t	Receive address

DID_CAN_CONFIG

Addresses for CAN messages

`can_config_t`

Field	Type	Description
can_period_mult	uint16_t[]	Broadcast period multiple for each CAN message. 0 disables the message. Indices 0..NUM_CIDS-1 correspond to classic can_cid_t values. In CAN-FD mode the same array is reused: indices 0..NUM_FDCIDS-1 correspond to canfd_cid_t values (FDCID_INS_1=0, FDCID_INS_2=1, ...). NUM_FDCIDS < NUM_CIDS so there is no overlap.
can_transmit_address	uint32_t[]	Transmit address for each CAN message. Indices 0..NUM_CIDS-1 correspond to classic can_cid_t values. In CAN-FD mode indices 0..NUM_FDCIDS-1 correspond to canfd_cid_t values and are validated / defaulted by CAN_init() when FD is enabled.
can_setting	uint16_t	Baud rate (kbps) (See can_baudrate_t for valid baud rates). Bit 15 (CAN_BAUDRATE_KBPS_FD_ENABLE) enables CAN-FD on capable hardware.
can_receive_address	uint32_t	Receive address

©2022

DID_DEV_INFO

`dev_info_t`

Field	Type	Description
reserved	uint8_t	Reserved bits
buildFlags	uint8_t	Build flags: 0x1=debug mode, 0x2=dirty (see eBuildFlags)
hardwareType	uint8_t	Hardware Type: 1=uINS, 2=EVB, 3=IMX, 4=GPX (see eIsHardwareType)
hdwRunState	uint8_t	Device Run State: Bootloader, App, etc (see eHdwRunStates)
serialNumber	uint32_t	Serial number
hardwareVer	uint8_t[3]	Hardware version: [0]=major, [1]=minor, [2]=pcb/a revision
hardwareVariant	uint8_t	Hardware variant; an arbitrary number independent of version that identifies some variation of the hardware; most commonly, IMU population type on IMX-6 (see eImx6ImuPopulationType)
firmwareVer	uint8_t[4]	Firmware (software) version
buildNumber	uint32_t	Build number
protocolVer	uint8_t[4]	Communications protocol version
repoRevision	uint32_t	Repository revision number
manufacturer	char[24]	Manufacturer name
buildType	uint8_t	Build type (0=production, 'c'=release candidate, 'b'=beta, 'a'=alpha, 'd'=developer, 's'=snapshot, '^'=dirty)
buildYear	uint8_t	Build date year - 2000
buildMonth	uint8_t	Build date month
buildDay	uint8_t	Build date day
buildHour	uint8_t	Build time hour
buildMinute	uint8_t	Build time minute
buildSecond	uint8_t	Build time second
buildMillisecond	uint8_t	Build time millisecond
addInfo	char[24]	Additional info

DID_DIAGNOSTIC_MESSAGE

Diagnostic message

`diag_msg_t`

Field	Type	Description
timeOfWeekMs	uint32_t	GPS time of week (since Sunday morning), in milliseconds
messageLength	uint32_t	Message length, including null terminator, in bytes
message	char[256]	Message data (null-terminated string); max size of message is 256 bytes

©2022

DID_EVB_DEBUG_ARRAY

`debug_array_t`

Field	Type	Description
i	int32_t[9]	Debug integer values, meaning defined by current firmware debug build
f	float[9]	Debug float values, meaning defined by current firmware debug build
lf	double[3]	Debug double (long float) values, meaning defined by current firmware debug build

DID_EVB_DEV_INFO

EVB device information

`dev_info_t`

Field	Type	Description
reserved	uint8_t	Reserved bits
buildFlags	uint8_t	Build flags: 0x1=debug mode, 0x2=dirty (see eBuildFlags)
hardwareType	uint8_t	Hardware Type: 1=uINS, 2=EVB, 3=IMX, 4=GPX (see eIsHardwareType)
hdwRunState	uint8_t	Device Run State: Bootloader, App, etc (see eHdwRunStates)
serialNumber	uint32_t	Serial number
hardwareVer	uint8_t[3]	Hardware version: [0]=major, [1]=minor, [2]=pcb/a revision
hardwareVariant	uint8_t	Hardware variant; an arbitrary number independent of version that identifies some variation of the hardware; most commonly, IMU population type on IMX-6 (see eImx6ImuPopulationType)
firmwareVer	uint8_t[4]	Firmware (software) version
buildNumber	uint32_t	Build number
protocolVer	uint8_t[4]	Communications protocol version
repoRevision	uint32_t	Repository revision number
manufacturer	char[24]	Manufacturer name
buildType	uint8_t	Build type (0=production, 'c'=release candidate, 'b'=beta, 'a'=alpha, 'd'=developer, 's'=snapshot, '^'=dirty)
buildYear	uint8_t	Build date year - 2000
buildMonth	uint8_t	Build date month
buildDay	uint8_t	Build date day
buildHour	uint8_t	Build time hour
buildMinute	uint8_t	Build time minute
buildSecond	uint8_t	Build time second
buildMillisecond	uint8_t	Build time millisecond
addInfo	char[24]	Additional info

DID_EVB_RTOS_INFO

sizes the task[] array and is not itself a task ID.

©2022

`evb_rtos_info_t`

Field	Type	Description
freeHeapSize	uint32_t	Heap high water mark, in free bytes remaining (lowest historical value)
mallocSize	uint32_t	Total memory allocated using RTOS pvPortMalloc(), in bytes
freeSize	uint32_t	Total memory freed using RTOS vPortFree(), in bytes
task	rtos_task_t[]	Per-task status/profiling info, indexed by eEvbRtosTask

DID_EVENT

`did_event_t`

Field	Type	Description
time	double	Time (uptime in seconds)
senderSN	uint32_t	Serial number of the device that generated this event
senderHdwId	uint16_t	Hardware type of the sender: 0=Host, 1=uINS, 2=EVB, 3=IMX, 4=GPX (see "Product Hardware ID")
priority	int8_t	Event priority/severity (see eEventPriority)
res8	uint8_t	Reserved for byte alignment
msgTypeID	uint16_t	Type/format of the payload in data[] (see eEventMsgTypeID)
length	uint16_t	Number of valid payload bytes in data[]
data	uint8_t[1]	Variable-length payload, length bytes; interpretation depends on msgTypeID

DID_EVENT_HEADER_SIZE

`did_event_t`

Field	Type	Description
time	double	Time (uptime in seconds)
senderSN	uint32_t	Serial number of the device that generated this event
senderHdwId	uint16_t	Hardware type of the sender: 0=Host, 1=uINS, 2=EVB, 3=IMX, 4=GPX (see "Product Hardware ID")
priority	int8_t	Event priority/severity (see eEventPriority)
res8	uint8_t	Reserved for byte alignment
msgTypeID	uint16_t	Type/format of the payload in data[] (see eEventMsgTypeID)
length	uint16_t	Number of valid payload bytes in data[]
data	uint8_t[1]	Variable-length payload, length bytes; interpretation depends on msgTypeID

©2022

DID_EXT_AIDING_ATTITUDE

`ext_aiding_attitude_t`

Field	Type	Description
timeOfWeekMs	uint32_t	GPS time of week (since Sunday morning) in milliseconds
status	uint32_t	Attitude representation, 1=euler, 2=quaternion (see eExtAidingAttitudeType)
att	float[4]	attitude, interpreted per `status`: euler {roll,pitch,yaw,-} or quaternion {w,x,y,z}
var	float[9]	3x3 row-major attitude-error covariance (rad^2), in the roll/pitch/yaw tangent space. Must have a non-zero diagonal or the observation is discarded.

DID_EXT_AIDING_DIR_SPEED

`ext_aiding_dir_speed_t`

Field	Type	Description
timeOfWeekMs	uint32_t	GPS time of week (since Sunday morning) in milliseconds
status	uint32_t	reserved, set to 0
speed	float	speed along `direction` (m/s)
var	float	observation variance ($m^{2/s}2$). Must be non-zero or the observation is discarded.
offset	float[3]	point of measurement relative to IMU origin in IMU/body frame {x,y,z} (m)
direction	float[3]	unit vector, in IMU/body frame, along which `speed` is measured (e.g. [1,0,0] for forward-pointing airspeed)

DID_EXT_AIDING_HEADING

`ext_aiding_heading_t`

Field	Type	Description
timeOfWeekMs	uint32_t	GPS time of week (since Sunday morning) in milliseconds
status	uint32_t	Heading type, 1=true, 2=magnetic, 3=course over ground (see eExtAidingHeadingType)
heading	float	heading (rad), 0 = north, positive clockwise, range [-pi, pi]
var	float	observation variance (rad^2). Must be non-zero or the observation is discarded.

DID_EXT_AIDING_POS

`ext_aiding_pos_t`

Field	Type	Description
timeOfWeekMs	uint32_t	GPS time of week (since Sunday morning) in milliseconds
status	uint32_t	Frame of measurement, 1=ECEF, 2=NED (see eExtAidingFrame)
pos	double[3]	position {x,y,z} (m)
offset	float[3]	point of measurement relative to IMU origin in IMU/body frame {x,y,z} (m)
var	float[3]	observation variance, per axis, in NED (m^2). Must be non-zero or the observation is discarded.

©2022

DID_EXT_AIDING_SPEED

`ext_aiding_speed_t`

Field	Type	Description
timeOfWeekMs	uint32_t	GPS time of week (since Sunday morning) in milliseconds
status	uint32_t	Speed type, 1=3D magnitude, 2=horizontal magnitude (see eExtAidingSpeedType)
speed	float	speed (m/s)
var	float	observation variance ($m^2/s2$). Must be non-zero or the observation is discarded.
offset	float[3]	point of measurement relative to IMU origin in IMU/body frame {x,y,z} (m)

DID_EXT_AIDING_VEL

`ext_aiding_vel_t`

Field	Type	Description
timeOfWeekMs	uint32_t	GPS time of week (since Sunday morning) in milliseconds
status	uint32_t	Frame of measurement, 1=ECEF, 2=NED, 3=Body (see eExtAidingFrame)
vel	float[3]	velocity {vx,vy,vz} (m/s)
offset	float[3]	point of measurement relative to IMU origin in IMU/body frame {x,y,z} (m)
var	float[3]	observation variance, per axis, in NED ($m^2/s2$). Must be non-zero or the observation is discarded.

DID_EXT_IMU

`imu_t`

Field	Type	Description
time	double	Time since boot up in seconds. Convert to GPS time of week by adding gps.towOffset
status	uint32_t	IMU status flags (eImuStatus)
I	imui_t	Combined Inertial Measurement Unit (IMU) sample: angular rate and acceleration

DID_GNSS1_SIG

`gnss_sig_t`

Field	Type	Description
timeOfWeekMs	uint32_t	GPS time of week (since Sunday morning) in milliseconds
numSigs	uint32_t	Number of satellite signals in the following satellite signal list (valid entries in sig[] below)
sig	gnss_sig_sv_t[160]	Per-signal tracking information list

DID_GNSS1_TIMESYNC

GNSS1 PPS time synchronization.

©2022

`gnss_timepulse_t`

Field	Type	Description
towOffset	double	Week seconds offset from MCU to GPS time, in seconds
towGps	double	Week seconds for next timepulse (from start of GPS week), in seconds
timeMcu	double	Local MCU week seconds, in seconds
msgTimeMs	uint32_t	Local timestamp of TIM-TP message used to validate timepulse, in milliseconds
plsTimeMs	uint32_t	Local timestamp of time sync pulse external interrupt used to validate timepulse, in milliseconds
syncCount	uint8_t	Counter for successful timesync events
badPulseAgeCount	uint8_t	Counter for failed timesync events
ppsInterruptReinitCount	uint8_t	Counter for GNSS PPS interrupt re-initialization
plsCount	uint8_t	Counter of GNSS PPS via GPIO, not interrupt
lastSyncTimeMs	uint32_t	Local timestamp of last valid PPS sync, in milliseconds
sinceLastSyncTimeMs	uint32_t	Time since last valid PPS sync, in milliseconds

DID_GNSS2_SIG

GNSS 2 signal information.

`gnss_sig_t`

Field	Type	Description
timeOfWeekMs	uint32_t	GPS time of week (since Sunday morning) in milliseconds
numSigs	uint32_t	Number of satellite signals in the following satellite signal list (valid entries in sig[] below)
sig	gnss_sig_sv_t[160]	Per-signal tracking information list

DID_GNSS2_TIMESYNC

GNSS2 PPS time synchronization.

©2022

`gnss_timepulse_t`

Field	Type	Description
towOffset	double	Week seconds offset from MCU to GPS time, in seconds
towGps	double	Week seconds for next timepulse (from start of GPS week), in seconds
timeMcu	double	Local MCU week seconds, in seconds
msgTimeMs	uint32_t	Local timestamp of TIM-TP message used to validate timepulse, in milliseconds
plsTimeMs	uint32_t	Local timestamp of time sync pulse external interrupt used to validate timepulse, in milliseconds
syncCount	uint8_t	Counter for successful timesync events
badPulseAgeCount	uint8_t	Counter for failed timesync events
ppsInterruptReinitCount	uint8_t	Counter for GNSS PPS interrupt re-initialization
plsCount	uint8_t	Counter of GNSS PPS via GPIO, not interrupt
lastSyncTimeMs	uint32_t	Local timestamp of last valid PPS sync, in milliseconds
sinceLastSyncTimeMs	uint32_t	Time since last valid PPS sync, in milliseconds

DID_GPX_BIT

`gpx_bit_t`

Field	Type	Description
results	uint32_t	GPX built-in test status (see eGPXBit_results)
command	uint8_t	Command (see eGPXBit_CMD)
port	uint8_t	Port used with the test
testMode	uint8_t	Self-test mode (see eGPXBit_test_mode)
state	uint8_t	Built-in self-test state (see eGPXBit_state)
detectedHardwareId	uint16_t	The hardware ID detected (see "Product Hardware ID"). This is used to ensure correct firmware is used.
reserved	uint8_t[2]	Unused

DID_GPX_DEBUG_ARRAY

`debug_array_t`

Field	Type	Description
i	int32_t[9]	Debug integer values, meaning defined by current firmware debug build
f	float[9]	Debug float values, meaning defined by current firmware debug build
lf	double[3]	Debug double (long float) values, meaning defined by current firmware debug build

DID_GPX_PORT_MONITOR

Data rate and status monitoring for each communications port.

©2022

`port_monitor_t`

Field	Type	Description
port	port_stats_t[6]	Per-port data rate and status statistics, one entry per communications port (see port_stats_t)
activePorts	uint8_t	Number of ports in the port[] array. FIXME: This should be moved to BEFORE the port definition, so on the receiving end, we know how many ports to expect.

DID_GPX_RTOS_INFO

`gpx_rtos_info_t`

Field	Type	Description
freeHeapSize	uint32_t	Heap high water mark, in free bytes remaining (lowest historical value)
mallocSize	uint32_t	Total memory allocated using RTOS pvPortMalloc(), in bytes
freeSize	uint32_t	Total memory freed using RTOS vPortFree(), in bytes
task	rtos_task_t[]	Per-task status/profiling info, indexed by eGpxRtosTask

DID_GPX_SYS_FAULT

System fault information. This is broadcast automatically every 10s if a critical fault is detected.

`system_fault_t`

Field	Type	Description
upTime	uint32_t	Time of fault, uptime in milliseconds
status	uint32_t	System fault status (see eSysFaultStatus)
fileNum	uint32_t	File number (source file identifier) where the fault occurred
lineNum	uint32_t	Line number within the file where the fault occurred
haltReason	uint32_t	Zephyr halt reason code
lr	uint32_t	Link register value at time of fault
pc	uint32_t	Program Counter value at time of fault
psr	uint32_t	Program Status Register value at time of fault
taskALastFeed	uint32_t	Milliseconds since task A last ran
taskBLastFeed	uint32_t	Milliseconds since task B last ran
wdtLastFeed	uint32_t	Milliseconds since the watchdog timer was last fed
var0	uint32_t	Multi-purpose register 0, fault-specific diagnostic value
var1	uint32_t	Multi-purpose register 1, fault-specific diagnostic value
var2	uint32_t	Multi-purpose register 2, fault-specific diagnostic value
var3	uint32_t	Multi-purpose register 3, fault-specific diagnostic value

©2022

DID_GROUND_VEHICLE

`ground_vehicle_t`

Field	Type	Description
timeOfWeekMs	uint32_t	(ms) GPS time of week (since Sunday morning)
status	uint32_t	Ground vehicle status flags (see eGroundVehicleStatus)
mode	uint32_t	Current mode of the ground vehicle. Use this field to apply commands. (see eGroundVehicleMode)
wheelConfig	wheel_config_t	Wheel transform, track width, and wheel radius

DID_IMUS_UNCAL

`imus_t`

Field	Type	Description
time	double	Time since boot up in seconds. Convert to GPS time of week by adding gps.towOffset
status	uint32_t	IMUs status flags (eImusStatus)
I	imui_t[1]	Per-device Inertial Measurement Unit (IMU) samples: angular rate and acceleration

DID_IMU_MAG

`imu_t`

Field	Type	Description
time	double	Time since boot up in seconds. Convert to GPS time of week by adding gps.towOffset
status	uint32_t	IMU status flags (eImuStatus)
I	imui_t	Combined Inertial Measurement Unit (IMU) sample: angular rate and acceleration

DID_INFIELD_CAL

`infield_cal_t`

Field	Type	Description
state	uint32_t	Used to set and monitor the state of the infield calibration system. (see eInfieldCalState)
status	uint32_t	Infield calibration status. (see eInfieldCalStatus)
sampleTimeMs	uint32_t	(ms) Number of samples used in IMU average. sampleTimeMs = 0 means "imu" member contains the IMU bias from flash.
imu	imui_t[1]	Dual purpose variable. 1.) This is the averaged IMU sample when sampleTimeMs != 0. 2.) This is a mirror of the motion calibration IMU bias from flash when sampleTimeMs = 0.
calData	infield_cal_vaxis_t[3]	Collected data used to solve for the bias error and INS rotation. Vertical axis: 0 = X, 1 = Y, 2 = Z

DID_INL2_MAG_OBS_INFO

INL2 magnetometer calibration information.

©2022

`inl2_mag_obs_info_t`

Field	Type	Description
timeOfWeekMs	uint32_t	(ms) GPS time of week
Ncal_samples	uint32_t	Number of calibration samples collected
ready	uint32_t	Data ready to be processed
calibrated	uint32_t	Calibration data present. Set to -1 to force mag recalibration.
auto_recal	uint32_t	Allow mag to auto-recalibrate
outlier	uint32_t	Bad sample data detected/rejected
magHdg	float	(rad) Heading from magnetometer
insHdg	float	(rad) Heading from INS
magInsHdgDelta	float	(rad) Difference between mag heading and (INS heading plus mag declination)
nis	float	Normalized innovation squared (likelihood metric) of the current mag heading update
nis_threshold	float	Threshold for maximum NIS, above which the mag update is rejected as an outlier
Wcal	float[9]	Magnetometer calibration matrix (row-major 3x3). Must be initialized with a unit matrix, not zeros!
activeCalSet	uint32_t	Active calibration set (0 or 1)
magHdgOffset	float	(rad) Offset between magnetometer heading and estimated heading
Tcal	float	Scaled computed variance between calibrated magnetometer samples
bias_cal	float[3]	(uT) Magnetometer calibration bias. Calibrated magnetometer output can be produced using: Bcal = Wcal * (Braw - bias_cal)

DID_INL2_NED_SIGMA

`inl2_ned_sigma_t`

Field	Type	Description
timeOfWeekMs	unsigned	(ms) GPS time of week (since Sunday morning)
StdPosNed	float[3]	(m) NED position error sigma
StdVelNed	float[3]	(m/s) NED velocity error sigma
StdAttNed	float[3]	(rad) NED attitude error sigma
StdAccBias	float[3]	(m/s^2) Acceleration bias error sigma
StdGyrBias	float[3]	(rad/s) Angular rate bias error sigma
StdBarBias	float	(m) Barometric altitude bias error sigma
StdMagDeclination	float	(rad) Mag declination error sigma

©2022

DID_INL2_STATES

`inl2_states_t`

Field	Type	Description
timeOfWeek	double	GPS time of week (since Sunday morning), in seconds
qe2b	float[4]	Quaternion body rotation with respect to ECEF: W, X, Y, Z
ve	float[3]	Velocity in ECEF frame, in meters/second
ecef	double[3]	Position in ECEF frame, in meters
biasPqr	float[3]	Gyro bias estimate, in radians/second
biasAcc	float[3]	Accelerometer bias estimate, in meters/second^2
biasBaro	float	Barometer bias estimate (altitude), in meters
magDec	float	Magnetic declination estimate, in radians
magInc	float	Magnetic inclination estimate, in radians

DID_INL2_STATUS

`inl2_status_t`

Field	Type	Description
ahrs	int	Non-zero while the filter is running in AHRS-only mode (prior to full INS/GNSS navigation)
zero_accel	int	Non-zero when zero-acceleration condition is detected
zero_angrate	int	Non-zero when zero-angular-rate condition is detected
accel_motion	int	Non-zero when accelerometer-sensed motion is detected
rot_motion	int	Non-zero when rotational motion is detected
zero_vel	int	Non-zero when zero-velocity condition is detected
ahrs_gnss_cnt	int	Counter of sequential valid GNSS data (for switching from AHRS to navigation)
hdg_err	float	Estimated heading error, in radians
hdg_coarse	int	Flag whether a coarse (uncertain) initial heading has been established
hdg_aligned	int	Flag whether initial attitude error has converged (heading alignment complete)
hdg_aligning	int	Flag whether heading alignment is currently in progress
ekf_init_done	int	Hot EKF initialization completed
mag_cal_good	int	Flag whether the magnetometer calibration is good
mag_cal_done	int	Flag whether the magnetometer calibration process has completed
stat_magfield	int	Flag whether the magnetic field is stationary/consistent (suitable for mag calibration)

©2022

DID_MANUFACTURING_INFO

`manufacturing_info_t`

Field	Type	Description
serialNumber	uint32_t	Inertial Sense serial number
hardwareId	uint16_t	Hardware ID: packed identifier, encoding the Hardware Type, hardwareVer Major, and hardwareVer Minor (see ENCODE_HDW_ID/DECODE_HDW_TYPE macros)
lotNumber	uint16_t	Inertial Sense lot number
date	char[16]	Inertial Sense manufacturing date (YYYYMMDDHHMMSS)
key	uint32_t	Key - write: unlock manufacturing info, read: number of times OTP has been set, 15 max
platformType	int32_t	Platform / carrier board (ePlatformConfig::PLATFORM_CFG_TYPE_MASK). Only valid if greater than zero.
reserved	int32_t	Reserved
uid	uint32_t[4]	Microcontroller unique identifier, 128 bits for SAM / 96 for STM32

DID_PIMU_MAG

`pimu_t`

Field	Type	Description
time	double	Time since boot up in seconds. Convert to GPS time of week by adding gps.towOffset
dt	float	Integration period in seconds for delta theta and delta velocity. Configured using DID_FLASH_CONFIG.startupNavDtMs
status	uint32_t	IMU status flags (eImuStatus)
theta	float[3]	IMU delta theta: gyroscope {p,q,r} integral over dt, in radians, in sensor/body frame
vel	float[3]	IMU delta velocity: accelerometer {x,y,z} integral over dt, in meters/second, in sensor/body frame

DID_PORT_MONITOR

`port_monitor_t`

Field	Type	Description
port	port_stats_t[6]	Per-port data rate and status statistics, one entry per communications port (see port_stats_t)
activePorts	uint8_t	Number of ports in the port[] array. FIXME: This should be moved to BEFORE the port definition, so on the receiving end, we know how many ports to expect.

©2022

DID_POSITION_MEASUREMENT

`pos_measurement_t`

Field	Type	Description
timeOfWeek	double	GPS time of week (since Sunday morning), s
ecef	double[3]	Position in ECEF (earth-centered earth-fixed) frame, m
psi	float	Heading with respect to NED frame, rad
accuracyCovUD	float[6]	Upper Diagonal of the 3x3 position accuracy covariance matrix (indices: [0 1 2 / _ 3 4 / _ _ 5])

DID_REFERENCE_IMU

Raw reference or truth IMU used for manufacturing calibration and testing. Input from testbed.

`imu_t`

Field	Type	Description
time	double	Time since boot up in seconds. Convert to GPS time of week by adding gps.towOffset
status	uint32_t	IMU status flags (eImuStatus)
I	imui_t	Combined Inertial Measurement Unit (IMU) sample: angular rate and acceleration

DID_REFERENCE_MAGNETOMETER

Reference or truth magnetometer used for manufacturing calibration and testing

`magnetometer_t`

Field	Type	Description
time	double	Time since boot up in seconds. Convert to GPS time of week by adding gps.towOffset
mag	float[3]	Magnetometer X, Y, Z in microtesla (uT), in body frame

DID_REFERENCE_PIMU

Reference or truth IMU used for manufacturing calibration and testing

`pimu_t`

Field	Type	Description
time	double	Time since boot up in seconds. Convert to GPS time of week by adding gps.towOffset
dt	float	Integration period in seconds for delta theta and delta velocity. Configured using DID_FLASH_CONFIG.startupNavDtMs
status	uint32_t	IMU status flags (eImuStatus)
theta	float[3]	IMU delta theta: gyroscope {p,q,r} integral over dt, in radians, in sensor/body frame
vel	float[3]	IMU delta velocity: accelerometer {x,y,z} integral over dt, in meters/second, in sensor/body frame

DID_ROS_COVARIANCE_POSE_TWIST

INL2 EKF 6x6 covariance matrices packed in arrays containing their elements on main diagonal and below

©2022

`ros_covariance_pose_twist_t`

Field	Type	Description
timeOfWeek	double	GPS time of week (since Sunday morning), in seconds
covPoseLD	float[21]	Lower-diagonal pose covariance (attitude rad^2, position m^2); see index layout above
covTwistLD	float[21]	Lower-diagonal twist covariance (velocity (m/s)^2, angular rate (rad/s)^2); see index layout above

DID_RTOS_INFO

`rtos_info_t`

Field	Type	Description
freeHeapSize	uint32_t	Heap high water mark, in free bytes remaining (lowest historical value)
mallocSize	uint32_t	Total memory allocated using RTOS pvPortMalloc(), in bytes
freeSize	uint32_t	Total memory freed using RTOS vPortFree(), in bytes
task	rtos_task_t[]	Per-task status/profiling info, indexed by eImxRtosTask

DID_RUNTIME_PROFILER

System runtime profiler

`runtime_profiler_t`

Field	Type	Description
p	runtime_profile_t[4]	Timing statistics for each profiled code section, RUNTIME_PROFILE_COUNT entries

©2022

DID_SCOMP

`sensor_compensation_v1p4_t`

Field	Type	Description
timeMs	uint32_t	Time since boot up, in milliseconds
pqr	sensor_comp_unit_t[NUM_IMU_DEVICES_V1P4]	Per-IMU gyro temperature-compensation state
acc	sensor_comp_unit_t[NUM_IMU_DEVICES_V1P4]	Per-IMU accelerometer temperature-compensation state
mag	sensor_comp_unit_t[NUM_MAG_DEVICES_V1P4]	Per-magnetometer temperature-compensation state
referenceImu	imui_t	Reference/truth IMU sample used during calibration
referenceMag	float[3]	(uT) Reference/truth magnetometer sample used during calibration
sampleCount	uint32_t	Number of samples collected for the current calibration state
calState	uint32_t	Current calibration state/step
status	uint32_t	Calibration status flags
alignAccel	float[3]	(m/s^2) Accelerometer reading used for alignment/leveling during calibration

DID_SENSORS_ADC

`sys_sensors_adc_t`

Field	Type	Description
time	double	Time since boot up in seconds
imu	sensors_imu_w_temp_t[1]	Per-device raw IMU (gyro/accel) + temperature samples
mag	sensors_mag_t[1]	Magnetometers
bar	float	Barometric pressure
barTemp	float	(°C) Temperature of barometric pressure sensor
humidity	float	Relative humidity as a percent (%rH). Range is 0% - 100%
ana	float[4]	ADC analog input

©2022

DID_SENSORS_ADC_SIGMA

`sys_sensors_adc_t`

Field	Type	Description
time	double	Time since boot up in seconds
imu	sensors_imu_w_temp_t[1]	Per-device raw IMU (gyro/accel) + temperature samples
mag	sensors_mag_t[1]	Magnetometers
bar	float	Barometric pressure
barTemp	float	(°C) Temperature of barometric pressure sensor
humidity	float	Relative humidity as a percent (%rH). Range is 0% - 100%
ana	float[4]	ADC analog input

DID_SENSORS_MCAL

Temperature compensated and motion calibrated IMU output.

`sensors_w_temp_t`

Field	Type	Description
imus	imus_t	Per-device gyro/accelerometer samples (see imus_t)
temp	float[1]	(°C) Temperature of IMU. Units only apply for calibrated data.
mag	mag_xyz_t[1]	(uT) Magnetometers. Units only apply for calibrated data.

DID_SENSORS_TCAL

Temperature compensated IMU output.

`sensors_w_temp_t`

Field	Type	Description
imus	imus_t	Per-device gyro/accelerometer samples (see imus_t)
temp	float[1]	(°C) Temperature of IMU. Units only apply for calibrated data.
mag	mag_xyz_t[1]	(uT) Magnetometers. Units only apply for calibrated data.

DID_SENSORS_TC_BIAS

`sensors_t`

Field	Type	Description
time	double	Time since boot up in seconds. Convert to GPS time of week by adding gps.towOffset. Units only apply for calibrated data.
mpu	sensors_mpu_t[1]	Per-device combined IMU + magnetometer sample

DID_SENSORS_UCAL

Uncalibrated IMU output.

©2022

`sensors_w_temp_t`

Field	Type	Description
imus	imus_t	Per-device gyro/accelerometer samples (see imus_t)
temp	float[1]	(°C) Temperature of IMU. Units only apply for calibrated data.
mag	mag_xyz_t[1]	(uT) Magnetometers. Units only apply for calibrated data.

DID_STROBE_IN_TIME

Timestamp for input strobe.

`strobe_in_time_t`

Field	Type	Description
week	uint32_t	GPS number of weeks since January 6^{th}, 1980
timeOfWeekMs	uint32_t	(ms) GPS time of week (since Sunday morning)
pin	uint16_t	Strobe input pin (i.e. G1, G2, G5, G9, G11, G12, G13, G15)
count	uint16_t	Strobe serial index number, incremented once per detected strobe edge

DID_SURVEY_IN

`survey_in_t`

Field	Type	Description
state	uint32_t	State of current survey (see eSurveyInStatus)
maxDurationSec	uint32_t	Maximum duration the survey will run, in seconds, if minAccuracy is not first achieved (ignored if 0)
minAccuracy	float	Required horizontal accuracy for the survey to complete before maxDurationSec elapses, in meters (ignored if 0)
elapsedTimeSec	uint32_t	Elapsed time of the survey, in seconds
hAccuracy	float	Approximate horizontal accuracy of the survey's current position estimate, in meters
lla	double[3]	Current surveyed position: latitude, longitude, altitude (deg, deg, m)

DID_SYS_FAULT

instead encode a single small integer value (1-7) at SYS_FAULT_STATUS_CRITICAL_ERROR_pos identifying the specific critical fault that caused a reset, since only one critical fault can be active/reported at a time.

©2022

`system_fault_t`

Field	Type	Description
upTime	uint32_t	Time of fault, uptime in milliseconds
status	uint32_t	System fault status (see eSysFaultStatus)
fileNum	uint32_t	File number (source file identifier) where the fault occurred
lineNum	uint32_t	Line number within the file where the fault occurred
haltReason	uint32_t	Zephyr halt reason code
lr	uint32_t	Link register value at time of fault
pc	uint32_t	Program Counter value at time of fault
psr	uint32_t	Program Status Register value at time of fault
taskALastFeed	uint32_t	Milliseconds since task A last ran
taskBLastFeed	uint32_t	Milliseconds since task B last ran
wdtLastFeed	uint32_t	Milliseconds since the watchdog timer was last fed
var0	uint32_t	Multi-purpose register 0, fault-specific diagnostic value
var1	uint32_t	Multi-purpose register 1, fault-specific diagnostic value
var2	uint32_t	Multi-purpose register 2, fault-specific diagnostic value
var3	uint32_t	Multi-purpose register 3, fault-specific diagnostic value

DID_SYS_PARAMS

`sys_params_t`

Field	Type	Description
timeOfWeekMs	uint32_t	GPS time of week (since Sunday morning) in milliseconds
insStatus	uint32_t	INS status flags (see eInsStatusFlags)
hdwStatus	uint32_t	Hardware status flags (see eHdwStatusFlags)
imuTemp	float	IMU temperature, in Celsius
baroTemp	float	Barometer temperature, in Celsius
mcuTemp	float	MCU temperature, in Celsius (not available yet)
sysStatus	uint32_t	System status flags (see eSysStatusFlags)
imuSamplePeriodMs	uint32_t	IMU sample period, in milliseconds. Zero disables sampling.
navOutputPeriodMs	uint32_t	Preintegrated IMU (PIMU) integration period and navigation/AHRS filter output period, in milliseconds
sensorTruePeriod	double	Actual sample period relative to GNSS PPS, in seconds
flashCfgChecksum	uint32_t	Flash config checksum used with host SDK synchronization
navUpdatePeriodMs	uint32_t	Navigation/AHRS filter update period, in milliseconds
genFaultCode	uint32_t	General fault code descriptor (see eGenFaultCodes). Set to zero to reset fault code.
upTime	double	System up time, in seconds (double precision)

©2022

DID_TIME_PULSE

`is_time_t`

Field	Type	Description
week	uint32_t	GPS number of weeks since January 6th, 1980
timeOfWeekMs	uint32_t	(ms) GPS time of week (since Sunday morning) of the timepulse event
status	uint32_t	Timepulse status flags
syncCnt	uint32_t	Count of timepulse sync events since power-on
reserved	uint32_t[4]	Reserved for future use

DID_WHEEL_ENCODER

`wheel_encoder_t`

Field	Type	Description
timeOfWeek	double	(s) (Do not use, internal development only) Time of measurement, seconds into current GNSS week
status	uint32_t	Wheel encoder status bits
theta_l	float	(rad) (Do not use, internal development only) Left wheel angle
theta_r	float	(rad) (Do not use, internal development only) Right wheel angle
omega_l	float	(rad/s) Left wheel angular rate. Positive when wheel is turning toward the forward direction of the vehicle. Use WHEEL_CFG_BITS_DIRECTION_REVERSE_LEFT in DID_FLASH_CONFIG::wheelConfig to reverse this.
omega_r	float	(rad/s) Right wheel angular rate. Positive when wheel is turning toward the forward direction of the vehicle. Use WHEEL_CFG_BITS_DIRECTION_REVERSE_RIGHT in DID_FLASH_CONFIG::wheelConfig to reverse this.
wrap_count_l	uint32_t	(Do not use, internal development only) Left wheel revolution (wrap-around) count
wrap_count_r	uint32_t	(Do not use, internal development only) Right wheel revolution (wrap-around) count
var_wheel_omega	float	(rad$^{2/s}$2) Wheel encoder velocity noise variance
var_wheel_theta	float	(rad^2) Wheel encoder angle noise variance

DIDS

`gen_3axis_sensor_t`

Field	Type	Description
time	double	Time in seconds (meaning is source-dependent; typically time since boot up or GPS time of week)
val	float[3]	3-axis sensor value {x,y,z} (units are source-dependent)

9.2.2 Enumerations and Defines

System status and configuration is made available through various enumeration and #defines.

©2022

General

DID_FLASH_CONFIG.GNSSSATSIGCONST

(eGnssSatSigConst)

Field	Value
GNSS_SAT_SIG_CONST_GPS	0x0003
GNSS_SAT_SIG_CONST_QZS	0x000C
GNSS_SAT_SIG_CONST_GAL	0x0030
GNSS_SAT_SIG_CONST_BDS	0x00C0
GNSS_SAT_SIG_CONST_GLO	0x0300
GNSS_SAT_SIG_CONST_SBS	0x1000
GNSS_SAT_SIG_CONST_IRN	0x2000
GNSS_SAT_SIG_CONST_IME	0x4000

©2022

DID_FLASH_CONFIG.SENSORCONFIG

(eSensorConfig)

©2022

Field	Value
SENSOR_CFG_GYR_FS_250	0x00000000
SENSOR_CFG_GYR_FS_500	0x00000001
SENSOR_CFG_GYR_FS_1000	0x00000002
SENSOR_CFG_GYR_FS_2000	0x00000003
SENSOR_CFG_GYR_FS_4000	0x00000004
SENSOR_CFG_GYR_FS_MAX	0x00000007
SENSOR_CFG_GYR_FS_MASK	0x00000007
SENSOR_CFG_GYR_FS_OFFSET	(int)0
SENSOR_CFG_ACC_FS_2G	0x00000000
SENSOR_CFG_ACC_FS_4G	0x00000001
SENSOR_CFG_ACC_FS_8G	0x00000002
SENSOR_CFG_ACC_FS_16G	0x00000003
SENSOR_CFG_ACC_FS_32G	0x00000004
SENSOR_CFG_ACC_FS_MAX	0x00000007
SENSOR_CFG_ACC_FS_MASK	0x00000070
SENSOR_CFG_ACC_FS_OFFSET	(int)4
SENSOR_CFG_GYR_DLPF_250HZ	0x00000000
SENSOR_CFG_GYR_DLPF_184HZ	0x00000001
SENSOR_CFG_GYR_DLPF_92HZ	0x00000002
SENSOR_CFG_GYR_DLPF_41HZ	0x00000003
SENSOR_CFG_GYR_DLPF_20HZ	0x00000004
SENSOR_CFG_GYR_DLPF_10HZ	0x00000005
SENSOR_CFG_GYR_DLPF_5HZ	0x00000006
SENSOR_CFG_GYR_DLPF_MASK	0x00000F00
SENSOR_CFG_GYR_DLPF_OFFSET	(int)8
SENSOR_CFG_ACC_DLPF_218HZ	0x00000000
SENSOR_CFG_ACC_DLPF_218HZb	0x00000001
SENSOR_CFG_ACC_DLPF_99HZ	0x00000002
SENSOR_CFG_ACC_DLPF_45HZ	0x00000003
SENSOR_CFG_ACC_DLPF_21HZ	0x00000004
SENSOR_CFG_ACC_DLPF_10HZ	0x00000005
SENSOR_CFG_ACC_DLPF_5HZ	0x00000006
SENSOR_CFG_ACC_DLPF_MASK	0x0000F000
SENSOR_CFG_ACC_DLPF_OFFSET	(int)12
SENSOR_CFG_SENSOR_ROTATION_MASK	0x001F0000

©2022

Field	Value
SENSOR_CFG_SENSOR_ROTATION_OFFSET	(int)16
SENSOR_CFG_SENSOR_ROTATION_0_0_0	(int)0
SENSOR_CFG_SENSOR_ROTATION_0_0_90	(int)1
SENSOR_CFG_SENSOR_ROTATION_0_0_180	(int)2
SENSOR_CFG_SENSOR_ROTATION_0_0_N90	(int)3
SENSOR_CFG_SENSOR_ROTATION_90_0_0	(int)4
SENSOR_CFG_SENSOR_ROTATION_90_0_90	(int)5
SENSOR_CFG_SENSOR_ROTATION_90_0_180	(int)6
SENSOR_CFG_SENSOR_ROTATION_90_0_N90	(int)7
SENSOR_CFG_SENSOR_ROTATION_180_0_0	(int)8
SENSOR_CFG_SENSOR_ROTATION_180_0_90	(int)9
SENSOR_CFG_SENSOR_ROTATION_180_0_180	(int)10
SENSOR_CFG_SENSOR_ROTATION_180_0_N90	(int)11
SENSOR_CFG_SENSOR_ROTATION_N90_0_0	(int)12
SENSOR_CFG_SENSOR_ROTATION_N90_0_90	(int)13
SENSOR_CFG_SENSOR_ROTATION_N90_0_180	(int)14
SENSOR_CFG_SENSOR_ROTATION_N90_0_N90	(int)15
SENSOR_CFG_SENSOR_ROTATION_0_90_0	(int)16
SENSOR_CFG_SENSOR_ROTATION_0_90_90	(int)17
SENSOR_CFG_SENSOR_ROTATION_0_90_180	(int)18
SENSOR_CFG_SENSOR_ROTATION_0_90_N90	(int)19
SENSOR_CFG_SENSOR_ROTATION_0_N90_0	(int)20
SENSOR_CFG_SENSOR_ROTATION_0_N90_90	(int)21
SENSOR_CFG_SENSOR_ROTATION_0_N90_180	(int)22
SENSOR_CFG_SENSOR_ROTATION_0_N90_N90	(int)23
SENSOR_CFG_DISABLE_MAGNETOMETER	0x00400000
SENSOR_CFG_DISABLE_BAROMETER	0x00800000
SENSOR_CFG_IMU_FAULT_DETECT_MASK	0xFF000000
SENSOR_CFG_IMU_FAULT_DETECT_GYR	0x01000000
SENSOR_CFG_IMU_FAULT_DETECT_ACC	0x02000000
SENSOR_CFG_IMU_FAULT_DETECT_OFFLINE	0x04000000
SENSOR_CFG_IMU_FAULT_DETECT_LARGE_BIAS	0x08000000
SENSOR_CFG_IMU_FAULT_DETECT_SENSOR_NOISE	0x10000000

©2022

DID_FLASH_CONFIG.SYSCFGBITS

(eSysConfigBits)

Field	Value
UNUSED1	0x00000001
SYS_CFG_BITS_ENABLE_MAG_CONTINUOUS_CAL	0x00000002
SYS_CFG_BITS_AUTO_MAG_RECAL	0x00000004
SYS_CFG_BITS_DISABLE_MAG_DECL_ESTIMATION	0x00000008
SYS_CFG_BITS_DISABLE_LEDS	0x00000010
SYS_CFG_BITS_MAG_RECAL_MODE_MASK	0x00000700
SYS_CFG_BITS_MAG_RECAL_MODE_OFFSET	8
SYS_CFG_BITS_MAG_ENABLE_WMM_DECLINATION	0x00000800
SYS_CFG_BITS_DISABLE_MAGNETOMETER_FUSION	0x00001000
SYS_CFG_BITS_DISABLE_BAROMETER_FUSION	0x00002000
SYS_CFG_BITS_DISABLE_GNSS1_FUSION	0x00004000
SYS_CFG_BITS_DISABLE_GNSS2_FUSION	0x00008000
SYS_CFG_BITS_DISABLE_AUTO_ZERO_VELOCITY_UPDATES	0x00010000
SYS_CFG_BITS_DISABLE_AUTO_ZERO_ANGULAR_RATE_UPDATES	0x00020000
SYS_CFG_BITS_DISABLE_INS_EKF	0x00040000
SYS_CFG_BITS_DISABLE_AUTO_BIT_ON_STARTUP	0x00080000
SYS_CFG_BITS_DISABLE_WHEEL_ENCODER_FUSION	0x00100000
SYS_CFG_BITS_ENABLE_GNSS_ANTENNA_OFFSET_ESTIMATION	0x00200000
SYS_CFG_BITS_BOR_LEVEL_0	0x0
SYS_CFG_BITS_BOR_LEVEL_1	0x1
SYS_CFG_BITS_BOR_LEVEL_2	0x2
SYS_CFG_BITS_BOR_LEVEL_3	0x3
SYS_CFG_BITS_BOR_THRESHOLD_MASK	0x00C00000
SYS_CFG_BITS_BOR_THRESHOLD_OFFSET	22
SYS_CFG_USE_REFERENCE_IMU_IN_EKF	0x01000000
SYS_CFG_EKF_REF_POINT_STATIONARY_ON_STROBE_INPUT	0x02000000

©2022

DID_GPX_FLASH_CFG.SYSCFGBITS

(eGpxSysConfigBits)

Field	Value
GPX_SYS_CFG_BITS_DISABLE_VCC_RF	0x00000001
GPX_SYS_CFG_BITS_BOR_LEVEL_0	0x0
GPX_SYS_CFG_BITS_BOR_LEVEL_1	0x1
GPX_SYS_CFG_BITS_BOR_LEVEL_2	0x2
GPX_SYS_CFG_BITS_BOR_LEVEL_3	0x3
GPX_SYS_CFG_BITS_BOR_THRESHOLD_MASK	0x00C00000
GPX_SYS_CFG_BITS_BOR_THRESHOLD_OFFSET	22

©2022

DID_GPX_STATUS.HDWSTATUS

(eGPXHdwStatusFlags)

©2022

Field	Value
GPX_HDW_STATUS_GNSS1_SATELLITE_RX	0x00000001
GPX_HDW_STATUS_GNSS2_SATELLITE_RX	0x00000002
GPX_HDW_STATUS_GNSS1_TIME_OF_WEEK_VALID	0x00000004
GPX_HDW_STATUS_GNSS2_TIME_OF_WEEK_VALID	0x00000008
GPX_HDW_STATUS_GNSS1_RESET_COUNT_MASK	0x00000070
GPX_HDW_STATUS_GNSS1_RESET_COUNT_OFFSET	4
GPX_HDW_STATUS_FAULT_GNSS1_INIT	0x00000080
GPX_HDW_STATUS_GNSS1_FAULT_FLAG_OFFSET	7
GPX_HDW_STATUS_GNSS2_RESET_COUNT_MASK	0x00000700
GPX_HDW_STATUS_GNSS2_RESET_COUNT_OFFSET	8
GPX_HDW_STATUS_FAULT_GNSS2_INIT	0x00000800
GPX_HDW_STATUS_GNSS2_FAULT_FLAG_OFFSET	11
GPX_HDW_STATUS_GNSS_FW_UPDATE_REQUIRED	0x00001000
GPX_HDW_STATUS_LED_ENABLED	0x00002000
GPX_HDW_STATUS_SYSTEM_RESET_REQUIRED	0x00004000
GPX_HDW_STATUS_FLASH_WRITE_PENDING	0x00008000
GPX_HDW_STATUS_ERR_COM_TX_LIMITED	0x00010000
GPX_HDW_STATUS_ERR_COM_RX_OVERRUN	0x00020000
GPX_HDW_STATUS_ERR_COM_MASK	0x00030000
GPX_HDW_STATUS_ERR_NO_GNSS1_PPS	0x00040000
GPX_HDW_STATUS_ERR_NO_GNSS2_PPS	0x00080000
GPX_HDW_STATUS_ERR_PPS_MASK	0x000C0000
GPX_HDW_STATUS_ERR_LOW_CNO_GNSS1	0x00100000
GPX_HDW_STATUS_ERR_LOW_CNO_GNSS2	0x00200000
GPX_HDW_STATUS_ERR_CNO_GNSS1_IR	0x00400000
GPX_HDW_STATUS_ERR_CNO_GNSS2_IR	0x00800000
GPX_HDW_STATUS_ERR_CNO_MASK	0x00F00000
GPX_HDW_STATUS_BIT_RUNNING	0x01000000
GPX_HDW_STATUS_BIT_PASSED	0x02000000
GPX_HDW_STATUS_BIT_FAULT	0x03000000
GPX_HDW_STATUS_BIT_MASK	0x03000000
GPX_HDW_STATUS_BIT_OFFSET	24
GPX_HDW_STATUS_ERR_TEMPERATURE	0x04000000
GPX_HDW_STATUS_GNSS_PPS_TIMESYNC	0x08000000
GPX_HDW_STATUS_RESET_CAUSE_MASK	0x70000000

©2022

Field	Value
GPX_HDW_STATUS_RESET_CAUSE_BACKUP_MODE	0x10000000
GPX_HDW_STATUS_RESET_CAUSE_SOFT	0x20000000
GPX_HDW_STATUS_RESET_CAUSE_HDW	0x40000000
GPX_HDW_STATUS_FAULT_SYS_CRITICAL	0x80000000

©2022

DID_GPX_STATUS.RTKMODE

(eRTKConfigBits)

©2022

Field	Value
RTK_CFG_BITS_ROVER_MODE_RTK_POSITIONING_DEPRECATED	0x00000001
RTK_CFG_BITS_ROVER_MODE_RTK_POSITIONING	0x00000002
RTK_CFG_BITS_ROVER_MODE_RTK_COMPASSING	0x00000004
RTK_CFG_BITS_ROVER_MODE_RTK_COMPASSING_DEPRECATED	0x00000008
RTK_CFG_BITS_ROVER_MODE_RTK_POSITIONING_MASK	(RTK_CFG_BITS_ROVER_MODE_RTK_POSITIONING| RTK_CFG_BITS_ROVER_MODE_RTK_POSITIONING_DEPRECATED)
RTK_CFG_BITS_ROVER_MODE_RTK_COMPASSING_MASK	(RTK_CFG_BITS_ROVER_MODE_RTK_COMPASSING| RTK_CFG_BITS_ROVER_MODE_RTK_COMPASSING_DEPRECATED)
RTK_CFG_BITS_ROVER_MODE_MASK	0x0000000F
RTK_CFG_BITS_BASE_OUTPUT_GNSS1_UBLOX_SER0	0x00000010
RTK_CFG_BITS_BASE_OUTPUT_GNSS1_UBLOX_SER1	0x00000020
RTK_CFG_BITS_BASE_OUTPUT_GNSS1_UBLOX_SER2	0x00000040
RTK_CFG_BITS_BASE_OUTPUT_GNSS1_UBLOX_USB	0x00000080
RTK_CFG_BITS_BASE_OUTPUT_GNSS1_RTCM3_SER0	0x00000100
RTK_CFG_BITS_BASE_OUTPUT_GNSS1_RTCM3_SER1	0x00000200
RTK_CFG_BITS_BASE_OUTPUT_GNSS1_RTCM3_SER2	0x00000400
RTK_CFG_BITS_BASE_OUTPUT_GNSS1_RTCM3_USB	0x00000800
RTK_CFG_BITS_BASE_OUTPUT_GNSS2_UBLOX_SER0	0x00001000
RTK_CFG_BITS_BASE_OUTPUT_GNSS2_UBLOX_SER1	0x00002000
RTK_CFG_BITS_BASE_OUTPUT_GNSS2_UBLOX_SER2	0x00004000
RTK_CFG_BITS_BASE_OUTPUT_GNSS2_UBLOX_USB	0x00008000
RTK_CFG_BITS_BASE_OUTPUT_GNSS2_RTCM3_SER0	0x00010000
RTK_CFG_BITS_BASE_OUTPUT_GNSS2_RTCM3_SER1	0x00020000
RTK_CFG_BITS_BASE_OUTPUT_GNSS2_RTCM3_SER2	0x00040000
RTK_CFG_BITS_BASE_OUTPUT_GNSS2_RTCM3_USB	0x00080000
RTK_CFG_BITS_BASE_POS_MOVING	0x00100000
RTK_CFG_BITS_RESERVED1	0x00200000
RTK_CFG_BITS_RTK_BASE_IS_IDENTICAL_TO_ROVER	0x00400000
RTK_CFG_BITS_GNSS_PORT_PASS_THROUGH	0x00800000
RTK_CFG_BITS_BASE_OUTPUT_GNSS1_RTCM3_CUR_PORT	0x01000000
RTK_CFG_BITS_BASE_OUTPUT_GNSS2_RTCM3_CUR_PORT	0x02000000
RTK_CFG_BITS_BASE_OUTPUT_RTCM3_CLEAR_CUR_PORT	0x04000000
RTK_CFG_BITS_MULTIPATH_MITIGATION	0x08000000
RTK_CFG_BITS_BASE_OUTPUT_RTCM3_CUR_PORT_MASK	(RTK_CFG_BITS_BASE_OUTPUT_GNSS1_RTCM3_CUR_PORT| RTK_CFG_BITS_BASE_OUTPUT_GNSS2_RTCM3_CUR_PORT)
RTK_CFG_BITS_BASE_UBLOX_MASK	(RTK_CFG_BITS_BASE_GNSS1_UBLOX_MASK| RTK_CFG_BITS_BASE_GNSS2_UBLOX_MASK)

©2022

Field	Value
RTK_CFG_BITS_BASE_RTCM3_MASK	(RTK_CFG_BITS_BASE_GNSS1_RTCM3_MASK| RTK_CFG_BITS_BASE_GNSS2_RTCM3_MASK)
RTK_CFG_BITS_BASE_MODE	(RTK_CFG_BITS_BASE_UBLOX_MASK| RTK_CFG_BITS_BASE_RTCM3_MASK)
RTK_CFG_BITS_ROVER_MODE_ONBOARD_MASK	(RTK_CFG_BITS_ROVER_MODE_RTK_POSITIONING_DEPRECATED| RTK_CFG_BITS_ROVER_MODE_RTK_COMPASSING_DEPRECATED)
RTK_CFG_BITS_ALL_MODES_MASK	(RTK_CFG_BITS_ROVER_MODE_MASK| RTK_CFG_BITS_BASE_MODE)

©2022

DID_GPX_STATUS.STATUS

(eGpxStatus)

Field	Value
GPX_STATUS_COM_PARSE_ERR_COUNT_MASK	0x0000000F
GPX_STATUS_COM_PARSE_ERR_COUNT_OFFSET	0
GPX_STATUS_COM0_RX_TRAFFIC_DETECTED	0x00000010
GPX_STATUS_COM1_RX_TRAFFIC_DETECTED	0x00000020
GPX_STATUS_COM2_RX_TRAFFIC_DETECTED	0x00000040
GPX_STATUS_USB_RX_TRAFFIC_DETECTED	0x00000080
GPX_STATUS_UPDATE_CONFIRMED	0x00000100
GPX_STATUS_GENERAL_FAULT_MASK	0xFFDF0000
GPX_STATUS_FAULT_RTK_QUEUE_LIMITED	0x00010000
GPX_STATUS_FAULT_GNSS_RCVR_TIME	0x00100000
GPX_STATUS_FAULT_RTOS_TASK_PERIOD_OVERRUN	0x00200000
GPX_STATUS_FAULT_DMA	0x00800000
GPX_STATUS_FATAL_MASK	0x1F000000
GPX_STATUS_FATAL_OFFSET	24
GPX_STATUS_FATAL_RESET_LOW_POW	(int)1
GPX_STATUS_FATAL_RESET_BROWN	(int)2
GPX_STATUS_FATAL_RESET_WATCHDOG	(int)3
GPX_STATUS_FATAL_CPU_EXCEPTION	(int)4
GPX_STATUS_FATAL_UNHANDLED_INTERRUPT	(int)5
GPX_STATUS_FATAL_STACK_OVERFLOW	(int)6
GPX_STATUS_FATAL_KERNEL_OOPS	(int)7
GPX_STATUS_FATAL_KERNEL_PANIC	(int)8
GPX_STATUS_FATAL_UNALIGNED_ACCESS	(int)9
GPX_STATUS_FATAL_MEMORY_ERROR	(int)10
GPX_STATUS_FATAL_BUS_ERROR	(int)11
GPX_STATUS_FATAL_USAGE_ERROR	(int)12
GPX_STATUS_FATAL_DIV_ZERO	(int)13
GPX_STATUS_FATAL_SER0_REINIT	(int)14
GPX_STATUS_FATAL_UNKNOWN	0x1F
GPX_STATUS_FAULT_RP	0x20000000
GPX_STATUS_FAULT_UNUSED	0xC0000000

©2022

DID_SYS_CMD.COMMAND

(eSystemCommand)

©2022

Field	Value
SYS_CMD_NONE	0
SYS_CMD_SAVE_PERSISTENT_MESSAGES	1
SYS_CMD_ENABLE_BOOTLOADER_AND_RESET	2
SYS_CMD_ENABLE_SENSOR_STATS	3
SYS_CMD_ENABLE_RTOS_STATS	4
SYS_CMD_ZERO_MOTION	5
SYS_CMD_REF_POINT_STATIONARY	6
SYS_CMD_REF_POINT_MOVING	7
SYS_CMD_RESET_RTOS_STATS	8
SYS_CMD_ENABLE_GNSS_LOW_LEVEL_CONFIG	10
SYS_CMD_DISABLE_SERIAL_PORT_BRIDGE	11
SYS_CMD_ENABLE_SERIAL_PORT_BRIDGE_USB_TO_GNSS1	12
SYS_CMD_ENABLE_SERIAL_PORT_BRIDGE_USB_TO_GNSS2	13
SYS_CMD_ENABLE_SERIAL_PORT_BRIDGE_USB_TO_SER0	14
SYS_CMD_ENABLE_SERIAL_PORT_BRIDGE_USB_TO_SER1	15
SYS_CMD_ENABLE_SERIAL_PORT_BRIDGE_USB_TO_SER2	16
SYS_CMD_ENABLE_SERIAL_PORT_BRIDGE_SER0_TO_GNSS1	17
SYS_CMD_ENABLE_SERIAL_PORT_BRIDGE_CUR_PORT_TO_GNSS1	18
SYS_CMD_ENABLE_SERIAL_PORT_BRIDGE_CUR_PORT_TO_GNSS2	19
SYS_CMD_ENABLE_SERIAL_PORT_BRIDGE_CUR_PORT_TO_USB	20
SYS_CMD_ENABLE_SERIAL_PORT_BRIDGE_CUR_PORT_TO_SER0	21
SYS_CMD_ENABLE_SERIAL_PORT_BRIDGE_CUR_PORT_TO_SER1	22
SYS_CMD_ENABLE_SERIAL_PORT_BRIDGE_CUR_PORT_TO_SER2	23
SYS_CMD_ENABLE_SERIAL_PORT_BRIDGE_USB_LOOPBACK	24
SYS_CMD_ENABLE_SERIAL_PORT_BRIDGE_SER0_LOOPBACK	25
SYS_CMD_ENABLE_SERIAL_PORT_BRIDGE_SER1_LOOPBACK	26
SYS_CMD_ENABLE_SERIAL_PORT_BRIDGE_SER2_LOOPBACK	27
SYS_CMD_ENABLE_SERIAL_PORT_BRIDGE_CUR_PORT_LOOPBACK	28
SYS_CMD_ENABLE_SERIAL_PORT_BRIDGE_CUR_PORT_LOOPBACK_TESTMODE	29
SYS_CMD_GPX_ENABLE_BOOTLOADER_MODE	30
SYS_CMD_GPX_ENABLE_GNSS1_CHIPSET_BOOTLOADER	31
SYS_CMD_GPX_ENABLE_GNSS2_CHIPSET_BOOTLOADER	32
SYS_CMD_GPX_ENABLE_GNSS1_PASS_THROUGH	33
SYS_CMD_GPX_ENABLE_GNSS2_PASS_THROUGH	34
SYS_CMD_GPX_HARD_RESET_GNSS1	36

©2022

Field	Value
SYS_CMD_GPX_HARD_RESET_GNSS2	37
SYS_CMD_GPX_SOFT_RESET_GPX	38
SYS_CMD_GPX_ENABLE_SERIAL_BRIDGE_CUR_PORT_LOOPBACK	39
SYS_CMD_GPX_ENABLE_SERIAL_BRIDGE_CUR_PORT_LOOPBACK_TESTMODE	40
SYS_CMD_GPX_ENABLE_RTOS_STATS	41
SYS_CMD_GPX_SAVE_PERSISTENT_MESSAGES	42
SYS_CMD_GNSS_RCVR_QUIET_MODE	60
SYS_CMD_GNSS_RCVR_SOFT_RESET	61
SYS_CMD_GNSS_RCVR_HARD_RESET	62
SYS_CMD_GNSS_RCVR_PWR_CYCLE	63
SYS_CMD_RESET_EKF_STATES	70
SYS_CMD_CLEAR_ERROR_STATUS	71
SYS_CMD_SAVE_FLASH	97
SYS_CMD_SAVE_GNSS_ASSIST_TO_FLASH_RESET	98
SYS_CMD_SOFTWARE_RESET	99
SYS_CMD_MANF_UNLOCK	1122334455
SYS_CMD_MANF_ERASE_CALIBRATION_MOTION	1357924678
SYS_CMD_MANF_ERASE_CALIBRATION	1357924679
SYS_CMD_MANF_FACTORY_RESET	1357924680
SYS_CMD_MANF_CHIP_ERASE	1357924681
SYS_CMD_MANF_DOWNGRADE_CALIBRATION	1357924682
SYS_CMD_MANF_ENABLE_ROM_BOOTLOADER	1357924683
SYS_CMD_MANF_LED_ON	1357924684
SYS_CMD_MANF_LED_OFF	1357924685
SYS_CMD_FAULT_TEST_TRIG_MALLOC	57005
SYS_CMD_FAULT_TEST_TRIG_HARD_FAULT	57006
SYS_CMD_FAULT_TEST_TRIG_WATCHDOG	57007
SYS_CMD_FAULT_TEST_TRIG_FLASH_TORN_WRITE	57008

©2022

DID_SYS_PARAMS.GENFAULTCODE

(eGenFaultCodes)

Field	Value
GFC_INS_STATE_ORUN_UVW	0x00000001
GFC_INS_STATE_ORUN_LAT	0x00000002
GFC_INS_STATE_ORUN_ALT	0x00000004
GFC_UNHANDLED_INTERRUPT	0x00000010
GFC_GNSS_CRITICAL_FAULT	0x00000020
GFC_GNSS_TX_LIMITED	0x00000040
GFC_GNSS_RX_OVERRUN	0x00000080
GFC_INIT_SENSORS	0x00000100
GFC_INIT_SPI	0x00000200
GFC_CONFIG_SPI	0x00000400
GFC_GNSS1_INIT	0x00000800
GFC_GNSS2_INIT	0x00001000
GFC_FLASH_INVALID_VALUES	0x00002000
GFC_FLASH_CHECKSUM_FAILURE	0x00004000
GFC_FLASH_WRITE_FAILURE	0x00008000
GFC_SYS_FAULT_GENERAL	0x00010000
GFC_SYS_FAULT_CRITICAL	0x00020000
GFC_SENSOR_SATURATION	0x00040000
GFC_EKF_STATES_INVALID	0x00080000
GFC_INIT_IMU	0x00100000
GFC_INIT_BAROMETER	0x00200000
GFC_INIT_MAGNETOMETER	0x00400000
GFC_INIT_I2C	0x00800000
GFC_CHIP_ERASE_INVALID	0x01000000
GFC_EKF_GNSS_TIME_FAULT	0x02000000
GFC_GNSS_RECEIVER_TIME	0x04000000
GFC_GNSS_GENERAL_FAULT	0x08000000
GFC_EKF_INPUT_INVALID_IMU	0x10000000
GFC_GNSS_RTOS_ERROR	0x20000000
GFC_GPX_STATUS_COMMON_MASK	GFC_GNSS1_INIT|GFC_GNSS2_INIT|GFC_GNSS_TX_LIMITED| GFC_GNSS_RX_OVERRUN|GFC_GNSS_CRITICAL_FAULT| GFC_GNSS_RECEIVER_TIME|GFC_GNSS_GENERAL_FAULT

©2022

GPS NAVIGATION FIX TYPE

(eGnssNavFixStatus)

Field	Value
GNSS_NAV_FIX_NONE	0x00000000
GNSS_NAV_FIX_POSITIONING_3D	0x00000001
GNSS_NAV_FIX_POSITIONING_RTK_FLOAT	0x00000002
GNSS_NAV_FIX_POSITIONING_RTK_FIX	0x00000003

©2022

GPS STATUS

(eGnssStatus)

©2022

Field	Value
GNSS_STATUS_NUM_SATS_USED_MASK	0x000000FF
GNSS_STATUS_FIX_NONE	0x00000000
GNSS_STATUS_FIX_DEAD_RECKONING_ONLY	0x00000100
GNSS_STATUS_FIX_2D	0x00000200
GNSS_STATUS_FIX_3D	0x00000300
GNSS_STATUS_FIX_GNSS_PLUS_DEAD_RECK	0x00000400
GNSS_STATUS_FIX_TIME_ONLY	0x00000500
GNSS_STATUS_FIX_REF_LLA	0x00000600
GNSS_STATUS_FIX_UNUSED2	0x00000700
GNSS_STATUS_FIX_DGPS	0x00000800
GNSS_STATUS_FIX_SBAS	0x00000900
GNSS_STATUS_FIX_RTK_SINGLE	0x00000A00
GNSS_STATUS_FIX_RTK_FLOAT	0x00000B00
GNSS_STATUS_FIX_RTK_FIX	0x00000C00
GNSS_STATUS_FIX_MASK	0x00001F00
GNSS_STATUS_FIX_BIT_OFFSET	(int)8
GNSS_STATUS_FLAGS_FIX_OK	0x00010000
GNSS_STATUS_FLAGS_DGPS_USED	0x00020000
GNSS_STATUS_FLAGS_RTK_FIX_AND_HOLD	0x00040000
GNSS_STATUS_FLAGS_UNUSED_1	0x00080000
GNSS_STATUS_FLAGS_GNSS1_RTK_POSITION_ENABLED	0x00100000
GNSS_STATUS_FLAGS_STATIC_MODE	0x00200000
GNSS_STATUS_FLAGS_GNSS2_RTK_COMPASS_ENABLED	0x00400000
GNSS_STATUS_FLAGS_GNSS1_RTK_RAW_GNSS_DATA_ERROR	0x00800000
GNSS_STATUS_FLAGS_GNSS1_RTK_BASE_DATA_MISSING	0x01000000
GNSS_STATUS_FLAGS_GNSS1_RTK_BASE_POSITION_MOVING	0x02000000
GNSS_STATUS_FLAGS_GNSS1_RTK_BASE_POSITION_INVALID	0x03000000
GNSS_STATUS_FLAGS_GNSS1_RTK_BASE_POSITION_MASK	0x03000000
GNSS_STATUS_FLAGS_GNSS1_RTK_POSITION_VALID	0x04000000
GNSS_STATUS_FLAGS_GNSS2_RTK_COMPASS_VALID	0x08000000
GNSS_STATUS_FLAGS_GNSS2_RTK_COMPASS_BASELINE_BAD	0x00002000
GNSS_STATUS_FLAGS_GNSS2_RTK_COMPASS_BASELINE_UNSET	0x00004000
GNSS_STATUS_FLAGS_GNSS_NMEA_DATA	0x00008000
GNSS_STATUS_FLAGS_GNSS_PPS_TIMESYNC	0x10000000
GNSS_STATUS_FLAGS_MASK	0x1FFFE000

©2022

Field	Value
GNSS_STATUS_FLAGS_BIT_OFFSET	(int)16
GNSS_STATUS_FLAGS_RTK_COV_ECEF_PACKED_VALID	0x20000000
GNSS_STATUS_FLAGS_UNUSED_3	0x40000000
GNSS_STATUS_FLAGS_UNUSED_4	0x80000000

©2022

HARDWARE STATUS FLAGS

(eHdwStatusFlags)

©2022

Field	Value
HDW_STATUS_MOTION_GYR	0x00000001
HDW_STATUS_MOTION_ACC	0x00000002
HDW_STATUS_MOTION_MASK	0x00000003
HDW_STATUS_IMU_FAULT_REJECT_GYR	0x00000004
HDW_STATUS_IMU_FAULT_REJECT_ACC	0x00000008
HDW_STATUS_IMU_FAULT_REJECT_MASK	0x0000000C
HDW_STATUS_GNSS_SATELLITE_RX_VALID	0x00000010
HDW_STATUS_STROBE_IN_EVENT	0x00000020
HDW_STATUS_GNSS_TIME_OF_WEEK_VALID	0x00000040
HDW_STATUS_REFERENCE_IMU_RX	0x00000080
HDW_STATUS_SATURATION_GYR	0x00000100
HDW_STATUS_SATURATION_ACC	0x00000200
HDW_STATUS_SATURATION_MAG	0x00000400
HDW_STATUS_SATURATION_BARO	0x00000800
HDW_STATUS_IMU_SATURATION_MASK	(int)(HDW_STATUS_SATURATION_GYR| HDW_STATUS_SATURATION_ACC)
HDW_STATUS_SATURATION_MASK	0x00000F00
HDW_STATUS_SATURATION_OFFSET	8
HDW_STATUS_SYSTEM_RESET_REQUIRED	0x00001000
HDW_STATUS_ERR_GNSS_PPS_NOISE	0x00002000
HDW_STATUS_MAG_RECAL_COMPLETE	0x00004000
HDW_STATUS_FLASH_WRITE_PENDING	0x00008000
HDW_STATUS_ERR_COM_TX_LIMITED	0x00010000
HDW_STATUS_ERR_COM_RX_OVERRUN	0x00020000
HDW_STATUS_ERR_NO_GNSS_PPS	0x00040000
HDW_STATUS_GNSS_PPS_TIMESYNC	0x00080000
HDW_STATUS_COM_PARSE_ERR_COUNT_MASK	0x00F00000
HDW_STATUS_COM_PARSE_ERR_COUNT_OFFSET	20
HDW_STATUS_BIT_RUNNING	0x01000000
HDW_STATUS_BIT_PASSED	0x02000000
HDW_STATUS_BIT_FAILED	0x03000000
HDW_STATUS_BIT_MASK	0x03000000
HDW_STATUS_ERR_TEMPERATURE	0x04000000
HDW_STATUS_SPI_INTERFACE_ENABLED	0x08000000
HDW_STATUS_RESET_CAUSE_MASK	0x70000000
HDW_STATUS_RESET_CAUSE_BACKUP_MODE	0x10000000

©2022

Field	Value
HDW_STATUS_RESET_CAUSE_WATCHDOG_FAULT	0x20000000
HDW_STATUS_RESET_CAUSE_SOFT	0x30000000
HDW_STATUS_RESET_CAUSE_HDW	0x40000000
HDW_STATUS_FAULT_SYS_CRITICAL	0x80000000

IMU SHOCK OPTIONS

(eImuShockOptions)

Field	Value
IMU_SHOCK_OPTIONS_ENABLE	0x01
IMU_SHOCK_OPTIONS_FAST_RECOVERY	0x02

IMU STATUS

(eImuStatus)

Field	Value
IMU_STATUS_GYR_X_OK	0x00000001
IMU_STATUS_GYR_Y_OK	0x00000002
IMU_STATUS_GYR_Z_OK	0x00000004
IMU_STATUS_ACC_X_OK	0x00000008
IMU_STATUS_ACC_Y_OK	0x00000010
IMU_STATUS_ACC_Z_OK	0x00000020
IMU_STATUS_IMU_OK_BITSIZE	6
IMU_STATUS_IMU_OK_MASK	IMU_STATUS_GYR_X_OK|IMU_STATUS_GYR_Y_OK|IMU_STATUS_GYR_Z_OK| IMU_STATUS_ACC_X_OK|IMU_STATUS_ACC_Y_OK|IMU_STATUS_ACC_Z_OK
IMU_STATUS_GYR_OK_MASK	IMU_STATUS_GYR_X_OK|IMU_STATUS_GYR_Y_OK|IMU_STATUS_GYR_Z_OK
IMU_STATUS_ACC_OK_MASK	IMU_STATUS_ACC_X_OK|IMU_STATUS_ACC_Y_OK|IMU_STATUS_ACC_Z_OK
IMU_STATUS_SHOCK_PRESENT	0x00000040
IMU_STATUS_MAG_UPDATE	0x00000100
IMU_STATUS_REFERENCE_IMU_PRESENT	0x00000200
IMU_STATUS_GYR_FAULT_REJECT	0x01000000
IMU_STATUS_ACC_FAULT_REJECT	0x02000000
IMU_STATUS_SATURATION_GYR	0x40000000
IMU_STATUS_SATURATION_ACC	0x80000000
IMU_STATUS_SATURATION_MASK	0xC0000000

©2022

INS STATUS FLAGS

(eInsStatusFlags)

©2022

Field	Value
INS_STATUS_HDG_ALIGN_COARSE	0x00000001
INS_STATUS_VEL_ALIGN_COARSE	0x00000002
INS_STATUS_POS_ALIGN_COARSE	0x00000004
INS_STATUS_ALIGN_COARSE_MASK	0x00000007
INS_STATUS_WHEEL_AIDING_VEL	0x00000008
INS_STATUS_HDG_ALIGN_FINE	0x00000010
INS_STATUS_VEL_ALIGN_FINE	0x00000020
INS_STATUS_POS_ALIGN_FINE	0x00000040
INS_STATUS_ALIGN_FINE_MASK	0x00000070
INS_STATUS_GNSS_AIDING_HEADING	0x00000080
INS_STATUS_GNSS_AIDING_POS	0x00000100
INS_STATUS_GNSS_UPDATE_IN_SOLUTION	0x00000200
INS_STATUS_EKF_USING_REFERENCE_IMU	0x00000400
INS_STATUS_MAG_AIDING_HEADING	0x00000800
INS_STATUS_NAV_MODE	0x00001000
INS_STATUS_STATIONARY_MODE	0x00002000
INS_STATUS_GNSS_AIDING_VEL	0x00004000
INS_STATUS_KINEMATIC_CAL_GOOD	0x00008000
INS_STATUS_SOLUTION_MASK	0x000F0000
INS_STATUS_SOLUTION_OFFSET	16
INS_STATUS_SOLUTION_OFF	0
INS_STATUS_SOLUTION_ALIGNING	1
INS_STATUS_SOLUTION_NAV	3
INS_STATUS_SOLUTION_NAV_HIGH_VARIANCE	4
INS_STATUS_SOLUTION_AHRS	5
INS_STATUS_SOLUTION_AHRS_HIGH_VARIANCE	6
INS_STATUS_SOLUTION_VRS	7
INS_STATUS_SOLUTION_VRS_HIGH_VARIANCE	8
INS_STATUS_RTK_COMPASSING_BASELINE_UNSET	0x00100000
INS_STATUS_RTK_COMPASSING_BASELINE_BAD	0x00200000
INS_STATUS_RTK_COMPASSING_MASK	(INS_STATUS_RTK_COMPASSING_BASELINE_UNSET| INS_STATUS_RTK_COMPASSING_BASELINE_BAD)
INS_STATUS_MAG_RECALIBRATING	0x00400000
INS_STATUS_MAG_INTERFERENCE_OR_BAD_CAL_OR_NO_CAL	0x00800000
INS_STATUS_GNSS_NAV_FIX_MASK	0x03000000
INS_STATUS_GNSS_NAV_FIX_OFFSET	24

©2022

Field	Value
INS_STATUS_RTK_COMPASSING_VALID	0x04000000
INS_STATUS_RTK_RAW_GNSS_DATA_ERROR	0x08000000
INS_STATUS_RTK_ERR_BASE_DATA_MISSING	0x10000000
INS_STATUS_RTK_ERR_BASE_POSITION_MOVING	0x20000000
INS_STATUS_RTK_ERR_BASE_POSITION_INVALID	0x30000000
INS_STATUS_RTK_ERR_BASE_MASK	0x30000000
INS_STATUS_RTK_ERROR_MASK	(INS_STATUS_RTK_RAW_GNSS_DATA_ERROR| INS_STATUS_RTK_ERR_BASE_MASK)
INS_STATUS_RTOS_TASK_PERIOD_OVERRUN	0x40000000
INS_STATUS_GENERAL_FAULT	0x80000000

MAGNETOMETER RECALIBRATION MODE

(eMagCalState)

Field	Value
MAG_CAL_STATE_DO_NOTHING	(int)0
MAG_CAL_STATE_MULTI_AXIS	(int)1
MAG_CAL_STATE_SINGLE_AXIS	(int)2
MAG_CAL_STATE_ABORT	(int)101
MAG_CAL_STATE_RECAL_RUNNING	(int)200
MAG_CAL_STATE_RECAL_COMPLETE	(int)201
MAG_CAL_STATE_RECAL_MODE_NOT_SUPPORTED	(int)202

©2022

RTK CONFIGURATION

(eRTKConfigBits)

©2022

Field	Value
RTK_CFG_BITS_ROVER_MODE_RTK_POSITIONING_DEPRECATED	0x00000001
RTK_CFG_BITS_ROVER_MODE_RTK_POSITIONING	0x00000002
RTK_CFG_BITS_ROVER_MODE_RTK_COMPASSING	0x00000004
RTK_CFG_BITS_ROVER_MODE_RTK_COMPASSING_DEPRECATED	0x00000008
RTK_CFG_BITS_ROVER_MODE_RTK_POSITIONING_MASK	(RTK_CFG_BITS_ROVER_MODE_RTK_POSITIONING| RTK_CFG_BITS_ROVER_MODE_RTK_POSITIONING_DEPRECATED)
RTK_CFG_BITS_ROVER_MODE_RTK_COMPASSING_MASK	(RTK_CFG_BITS_ROVER_MODE_RTK_COMPASSING| RTK_CFG_BITS_ROVER_MODE_RTK_COMPASSING_DEPRECATED)
RTK_CFG_BITS_ROVER_MODE_MASK	0x0000000F
RTK_CFG_BITS_BASE_OUTPUT_GNSS1_UBLOX_SER0	0x00000010
RTK_CFG_BITS_BASE_OUTPUT_GNSS1_UBLOX_SER1	0x00000020
RTK_CFG_BITS_BASE_OUTPUT_GNSS1_UBLOX_SER2	0x00000040
RTK_CFG_BITS_BASE_OUTPUT_GNSS1_UBLOX_USB	0x00000080
RTK_CFG_BITS_BASE_OUTPUT_GNSS1_RTCM3_SER0	0x00000100
RTK_CFG_BITS_BASE_OUTPUT_GNSS1_RTCM3_SER1	0x00000200
RTK_CFG_BITS_BASE_OUTPUT_GNSS1_RTCM3_SER2	0x00000400
RTK_CFG_BITS_BASE_OUTPUT_GNSS1_RTCM3_USB	0x00000800
RTK_CFG_BITS_BASE_OUTPUT_GNSS2_UBLOX_SER0	0x00001000
RTK_CFG_BITS_BASE_OUTPUT_GNSS2_UBLOX_SER1	0x00002000
RTK_CFG_BITS_BASE_OUTPUT_GNSS2_UBLOX_SER2	0x00004000
RTK_CFG_BITS_BASE_OUTPUT_GNSS2_UBLOX_USB	0x00008000
RTK_CFG_BITS_BASE_OUTPUT_GNSS2_RTCM3_SER0	0x00010000
RTK_CFG_BITS_BASE_OUTPUT_GNSS2_RTCM3_SER1	0x00020000
RTK_CFG_BITS_BASE_OUTPUT_GNSS2_RTCM3_SER2	0x00040000
RTK_CFG_BITS_BASE_OUTPUT_GNSS2_RTCM3_USB	0x00080000
RTK_CFG_BITS_BASE_POS_MOVING	0x00100000
RTK_CFG_BITS_RESERVED1	0x00200000
RTK_CFG_BITS_RTK_BASE_IS_IDENTICAL_TO_ROVER	0x00400000
RTK_CFG_BITS_GNSS_PORT_PASS_THROUGH	0x00800000
RTK_CFG_BITS_BASE_OUTPUT_GNSS1_RTCM3_CUR_PORT	0x01000000
RTK_CFG_BITS_BASE_OUTPUT_GNSS2_RTCM3_CUR_PORT	0x02000000
RTK_CFG_BITS_BASE_OUTPUT_RTCM3_CLEAR_CUR_PORT	0x04000000
RTK_CFG_BITS_MULTIPATH_MITIGATION	0x08000000
RTK_CFG_BITS_BASE_OUTPUT_RTCM3_CUR_PORT_MASK	(RTK_CFG_BITS_BASE_OUTPUT_GNSS1_RTCM3_CUR_PORT| RTK_CFG_BITS_BASE_OUTPUT_GNSS2_RTCM3_CUR_PORT)
RTK_CFG_BITS_BASE_UBLOX_MASK	(RTK_CFG_BITS_BASE_GNSS1_UBLOX_MASK| RTK_CFG_BITS_BASE_GNSS2_UBLOX_MASK)

©2022

Field	Value
RTK_CFG_BITS_BASE_RTCM3_MASK	(RTK_CFG_BITS_BASE_GNSS1_RTCM3_MASK| RTK_CFG_BITS_BASE_GNSS2_RTCM3_MASK)
RTK_CFG_BITS_BASE_MODE	(RTK_CFG_BITS_BASE_UBLOX_MASK| RTK_CFG_BITS_BASE_RTCM3_MASK)
RTK_CFG_BITS_ROVER_MODE_ONBOARD_MASK	(RTK_CFG_BITS_ROVER_MODE_RTK_POSITIONING_DEPRECATED| RTK_CFG_BITS_ROVER_MODE_RTK_COMPASSING_DEPRECATED)
RTK_CFG_BITS_ALL_MODES_MASK	(RTK_CFG_BITS_ROVER_MODE_MASK| RTK_CFG_BITS_BASE_MODE)

RTK SOLUTION STATUS

(eRtkSolStatus)

Field	Value
rtk_solution_status_none	0
rtk_solution_status_fix	1
rtk_solution_status_float	2
rtk_solution_status_sbas	3
rtk_solution_status_dgps	4
rtk_solution_status_single	5

RAW GPS DATA TYPE

(eRawDataType)

Field	Value
raw_data_type_observation	1
raw_data_type_ephemeris	2
raw_data_type_glonass_ephemeris	3
raw_data_type_sbas	4
raw_data_type_base_station_antenna_position	5
raw_data_type_ionosphere_model_utc_alm	6

©2022

SYSTEM CONFIGURATION

(eSysConfigBits)

Field	Value
UNUSED1	0x00000001
SYS_CFG_BITS_ENABLE_MAG_CONTINUOUS_CAL	0x00000002
SYS_CFG_BITS_AUTO_MAG_RECAL	0x00000004
SYS_CFG_BITS_DISABLE_MAG_DECL_ESTIMATION	0x00000008
SYS_CFG_BITS_DISABLE_LEDS	0x00000010
SYS_CFG_BITS_MAG_RECAL_MODE_MASK	0x00000700
SYS_CFG_BITS_MAG_RECAL_MODE_OFFSET	8
SYS_CFG_BITS_MAG_ENABLE_WMM_DECLINATION	0x00000800
SYS_CFG_BITS_DISABLE_MAGNETOMETER_FUSION	0x00001000
SYS_CFG_BITS_DISABLE_BAROMETER_FUSION	0x00002000
SYS_CFG_BITS_DISABLE_GNSS1_FUSION	0x00004000
SYS_CFG_BITS_DISABLE_GNSS2_FUSION	0x00008000
SYS_CFG_BITS_DISABLE_AUTO_ZERO_VELOCITY_UPDATES	0x00010000
SYS_CFG_BITS_DISABLE_AUTO_ZERO_ANGULAR_RATE_UPDATES	0x00020000
SYS_CFG_BITS_DISABLE_INS_EKF	0x00040000
SYS_CFG_BITS_DISABLE_AUTO_BIT_ON_STARTUP	0x00080000
SYS_CFG_BITS_DISABLE_WHEEL_ENCODER_FUSION	0x00100000
SYS_CFG_BITS_ENABLE_GNSS_ANTENNA_OFFSET_ESTIMATION	0x00200000
SYS_CFG_BITS_BOR_LEVEL_0	0x0
SYS_CFG_BITS_BOR_LEVEL_1	0x1
SYS_CFG_BITS_BOR_LEVEL_2	0x2
SYS_CFG_BITS_BOR_LEVEL_3	0x3
SYS_CFG_BITS_BOR_THRESHOLD_MASK	0x00C00000
SYS_CFG_BITS_BOR_THRESHOLD_OFFSET	22
SYS_CFG_USE_REFERENCE_IMU_IN_EKF	0x01000000
SYS_CFG_EKF_REF_POINT_STATIONARY_ON_STROBE_INPUT	0x02000000

©2022

9.3 Inertial Sense Binary (ISB) Protocol

The Inertial Sense binary protocol provides the most efficient way to communicate with the IMX and GPX. It uses the native floating point and integer binary format used in computers. Binary protocol is not human readable like NMEA Protocol. Binary protocol uses Data Set (DID) C structures defined in SDK/src/data_sets.h of the InertialSense SDK.

9.3.1 Endianness

The IMX and GPX devices use little-endian byte order in the ISB protocol.

9.3.2 Communication

Writing to and reading from InertialSense products is done using "Set" and "Get" commands. The following helper function `portWrite()` which assists with writing data to the serial port is used throughout this document.

```
static int portWrite(int port, const unsigned char* buf, int len)
{
    return serialPortWrite(&serialPort, buf, len);
}
```

Setting Data

The `is_comm_set_data()` function will encode a message used to set data or configurations.

```
// Set INS output Euler rotation in radians to 90 degrees roll for mounting
void setInsOutputRotation()
{
    float rotation[3] = { 90.0f*C_DEG2RAD_F, 0.0f, 0.0f };
    uint8_t buf[200];
    int len = is_comm_set_data_to_buf(buf, sizeof(buf), comm, DID_FLASH_CONFIG, sizeof(float) * 3, offsetof(nvm_flash_cfg_t, insRotation), rotation);
    if (len > 0) portWrite(0, buf, len);
}
```

Getting Data

Data broadcasting or streaming is enabled by using the ***Realtime Message Controller*** (**RMC**) or the get data command.

GET DATA COMMAND

The `is_comm_get_data()` function will encode a PKT_TYPE_GET_DATA message that enables broadcast of a given message at a multiple of the *Data Source Update Rates*. Set the data rate (period multiple) to zero disable message broadcast and pull a single packet of data. Set the data size and offset to zero to request the entire data set.

```
// Ask for INS message w/ update 40ms period (4ms source period x 10).  Set data rate to zero to disable broadcast and pull a single packet.
uint8_t buf[200];
int len = is_comm_get_data_to_buf(buf, sizeof(buf), comm, DID_INS_1, 0, 0, 10);
if (len > 0) portWrite(0, buf, len);
```


©2022

DATA SOURCE UPDATE RATES

DID	Default Update Rate (Period)
DID_INS_[1-4]	(7ms default) Configured with DID_FLASH_CONFIG.startupNavDtMs
DID_IMU, DID_PIMU*	(14ms default) Configured with DID_FLASH_CONFIG.startupImuDtMs
DID_BAROMETER	~20ms
DID_MAGNETOMETER	~20ms
DID_GNSS[1-2]_[X] (Any DID beginning with DID_GNSS)	(200ms default) Configured with DID_FLASH_CONFIG. startupGnssDtMs
All other DIDs	1ms

*DID_PIMU integration period (dt) and output data rate are the same as DID_FLASH_CONFIG.startupNavDtMs and cannot be output at any other rate. If a different output data rate is desired, DID_IMU which is derived from DID_PIMU can be used instead.

REALTIME MESSAGE CONTROLLER (RMC)

The RMC is used to enable message broadcasting and provides updates from onboard data as soon as it becomes available with minimal latency. All RMC messages can be enabled using the *Get Data Command*, which is the preferred method, or by directly setting the RMC bits. The RMC bits are listed below. Message data rates are listed in the *Data Source Update Rates* table.

RMC Message
RMC_BITS_INS[1-4]
RMC_BITS_IMU, RMC_BITS_PIMU
RMC_BITS_BAROMETER
RMC_BITS_MAGNETOMETER
RMC_BITS_GNSS1_POS, RMC_BITS_GNSS2_POS
RMC_BITS_GNSS1_RTK_POS, RMC_BITS_GNSS1_RTK_POS_MISC
RMC_BITS_STROBE_IN_TIME

The following is an example of how to use the RMC. The `rmc.options` field controls whether RMC commands are applied to other serial ports. `rmc.options = 0` will apply the command to the current serial port.

```
rmc_t rmc;
// Enable broadcasts of DID_INS_1 and DID_GNSS1_POS
rmc.bits = RMC_BITS_INS1 | RMC_BITS_GNSS1_POS;
// Remember configuration following reboot for automatic data streaming.
rmc.options = RMC_OPTIONS_PERSISTENT;

uint8_t buf[200];
int len = is_comm_set_data_to_buf(buf, sizeof(buf), comm, DID_RMC, sizeof(rmc_t), 0, &rmc);
if (len > 0) portWrite(0, buf, len);
```

The update rate of the EKF is set by DID_FLASH_CONFIG.startupNavDtMs (reboot is required to apply the change). Independently, the DID_INS_x broadcast period multiple can be used to set the output data rate down to 1ms.

PERSISTENT MESSAGES

The *persistent messages* option saves the current data stream configuration to flash memory for use following reboot, eliminating the need to re-enable messages following a reset or power cycle.

- **To save persistent messages** - (to flash memory), bitwise OR `RMC_OPTIONS_PERSISTENT (0x200)` with the RMC option field or set `DID_SYS_CMD.command = 1` and `DID_SYS_CMD.invCommand = 0xFFFFFFFE`. See the save persistent messages example in the Binary Communications example project.
- **To disable persistent messages** - a stop all broadcasts packet followed by a *save persistent messages* command.

©2022

NMEA persistent messages are also available.

Enabling Persistent Messages - EvalTool

- Enable the desired messages in the EvalTool "Data Sets" tab.
- Press the "Save Persistent" button in the EvalTool "Data Logs" tab to store the current message configuration to flash memory for use following reboot.
- Reset the system and verify the messages are automatically streaming. You can use the EvalTool->Data Logs dialog to view the streaming messages.

To disable all persistent messages using the EvalTool, click the "Stop Streaming" button and then "Save Persistent" button.

Enabling Persistent Messages - CLTool

Persistent messages are enabled using the CLTool by including the `-persistent` option along with the options for the desired messages in the command line.

```
cltool -c /dev/ttyS3 -persistent -msgINS2 -msgGPS
```

EXAMPLE PROJECTS

Examples on how to use the Inertial Sense SDK for binary communications are found in the Binary Communications Example Project and cltool project. NMEA communications examples are found in the NMEA Example Project.

Parsing Data

The ISComm library in the InertialSenseSDK provides a communications parser that can parse InertialSense binary protocol as well as other protocols.

ONE BYTE (SIMPLE METHOD)

The following parser code is simpler to implement. This method uses the `is_comm_parse_byte()` function to parse one byte at a time of a data stream. The return value is a non-zero protocol_type_t when valid data is found.

```
uint8_t c;
protocol_type_t ptype;
// Read from serial buffer until empty
while (serialPortReadChar(&s_serialPort, &c) > 0)
{
    // timeMs = current_timeMs();
    switch (is_comm_parse_byte(&comm, inByte))
    {
    case _PTYPE_INERTIAL_SENSE_DATA:
        break;
    case _PTYPE_UBLOX:
        break;
    case _PTYPE_RTCM3:
        break;
    case _PTYPE_NMEA:
        break;
    }
}
```

SET OF BYTES (FAST METHOD)

The following parser code uses less processor time to parse data by copying multiple bytes at a time. This method uses `is_comm_free()` and `is_comm_parse()` along with a serial port read or buffer copy. The return value is a non-zero protocol_type_t when valid data is found.

```
// Read a set of bytes (fast method)
protocol_type_t ptype;

// Get available size of comm buffer.  is_comm_free() modifies comm->rxBuf pointers, call it before using comm->rxBuf.tail.
int n = is_comm_free(comm);

// Read data directly into comm buffer
if ((n = serialPortRead(comm->rxBuf.tail, n)))
{
    // Update comm buffer tail pointer
    comm->rxBuf.tail += n;

    // Search comm buffer for valid packets
    while ((ptype = is_comm_parse(comm)) != _PTYPE_NONE)
    {
```

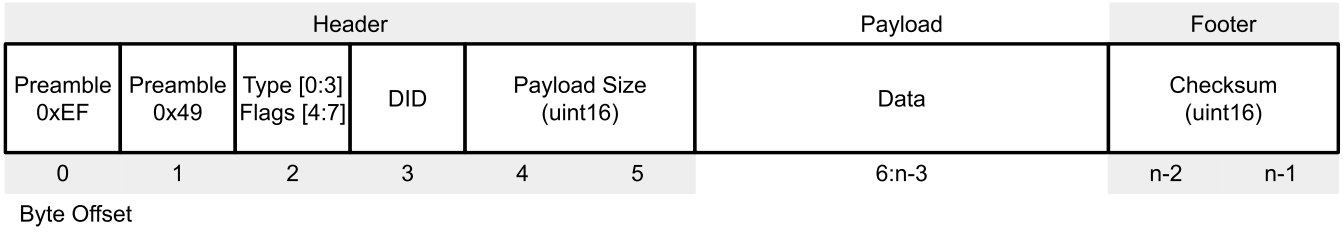

©2022

```
            switch (ptype)
            {
            case _PTYPE_INERTIAL_SENSE_DATA:
            case _PTYPE_INERTIAL_SENSE_CMD:
                break;
            case _PTYPE_UBLOX:
                break;
            case _PTYPE_RTCM3:
                break;
            case _PTYPE_NMEA:
                break;
            }
        }
    }
```

9.3.3 ISB Packet Overview

The IMX and GPX communicate using the Inertial Sense Binary (ISB) protocol. This section details the ISB protocol packet structure specific for protocol 2.x (software releases 2.x). Refer to the release 1.x ISB protocol document for a description of protocol 1.x (software releases 1.x). The Inertial-Sense-SDK ISComm provides functions to encode and decode ISB packets.

ISB Packet



The ISB packet structure is defined in the typedef `packet_t` found in ISComm.h.

HEADER TYPE AND FLAGS

The packet type and flags are found in the byte at offset 2 in the ISB packet. The **Type** is the lower nibble and the **Flags** are the upper nibble. The packet and is defined in ISComm.h.

```
typedef enum
{
    PKT_TYPE_INVALID                        = 0,    // Invalid packet id
    PKT_TYPE_ACK                            = 1,    // (ACK) received valid packet
    PKT_TYPE_NACK                           = 2,    // (NACK) received invalid packet
    PKT_TYPE_GET_DATA                       = 3,    // Request for data to be broadcast, response is PKT_TYPE_DATA. See data structures for list of possible
broadcast data.
    PKT_TYPE_DATA                           = 4,    // Data sent in response to PKT_TYPE_GET_DATA (no PKT_TYPE_ACK is sent)
    PKT_TYPE_SET_DATA                       = 5,    // Data sent, such as configuration options.  PKT_TYPE_ACK is sent in response.
    PKT_TYPE_STOP_BROADCASTS_ALL_PORTS      = 6,    // Stop all data broadcasts on all ports. Responds with an ACK
    PKT_TYPE_STOP_DID_BROADCAST             = 7,    // Stop a specific broadcast
    PKT_TYPE_STOP_BROADCASTS_CURRENT_PORT   = 8,    // Stop all data broadcasts on current port. Responds with an ACK
    PKT_TYPE_COUNT                          = 9,    // The number of packet identifiers, keep this at the end!
    PKT_TYPE_MAX_COUNT                      = 16,   // The maximum count of packet identifiers, 0x1F (PACKET_INFO_ID_MASK)
    PKT_TYPE_MASK                           = 0x0F, // ISB packet type bitmask

    ISB_FLAGS_MASK                          = 0xF0, // ISB packet flags bitmask (4 bits upper nibble)
    ISB_FLAGS_EXTENDED_PAYLOAD              = 0x10, // Payload is larger than 2048 bytes and extends into next packet.
    ISB_FLAGS_PAYLOAD_W_OFFSET              = 0x20, // The first two bytes of the payload are the byte offset of the payload data into the data set.
} eISBPacketFlags;
```

HEADER DID

The data ID (DID) values are defined at the top of data_sets.h and identify which data set is requested or contained in the ISB packet.

HEADER PAYLOAD SIZE

In the ISB protocol, the packet payload size is a 16-bit unsigned integer (uint16) that specifies the payload's byte length, excluding the packet header and footer/checksum. If the ISB_FLAGS_PAYLOAD_W_OFFSET flag (0x20) is set in the header's flags byte, the optional 16-bit offset value is included in the payload length.

©2022

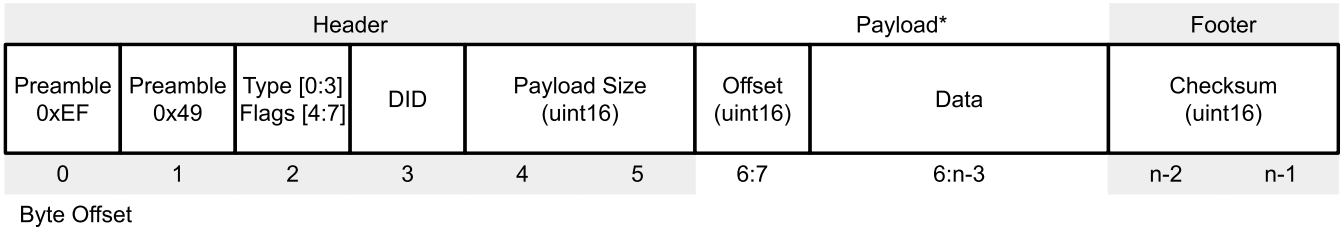
FOOTER CHECKSUM

The ISB packet footer contains a Fletcher-16 (16-bit integer). The following algorithm is used for this checksum and is found in ISComm.h.

```
uint16_t is_comm_fletcher16(uint16_t cksum_init, const void* data, uint32_t size)
{
    checksum16_u cksum;
    cksum.ck = cksum_init;
    for (uint32_t i=0; i<size; i++)
    {
        cksum.a += ((uint8_t*)data)[i];
        cksum.b += cksum.a;
    }
    return cksum.ck;
}
```

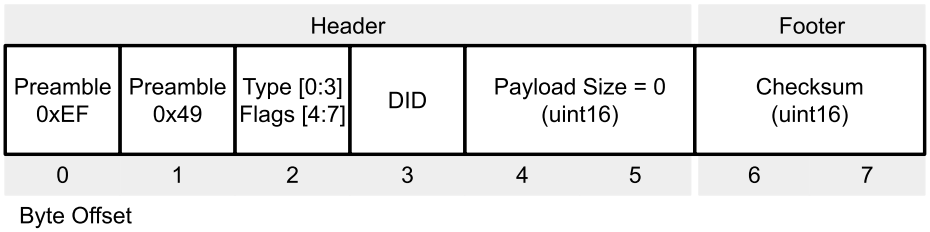
The ISB packet footer checksum is computed using the Fletcher-16 algorithm starting with an initial value of zero and sequencing over the entire packet (excluding the two footer checksum bytes).

ISB Packet with Data Offset



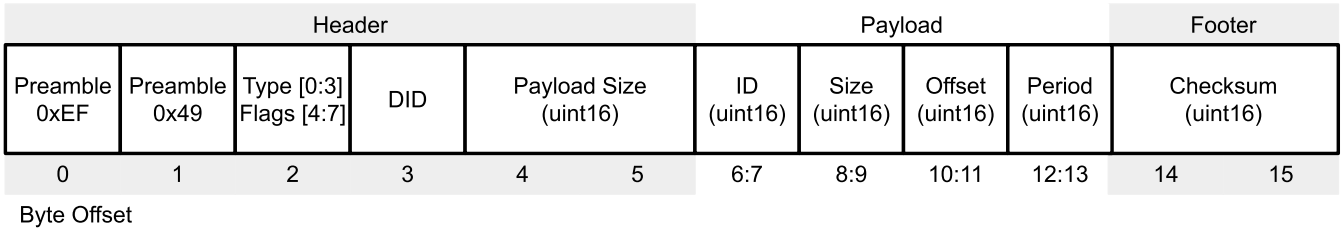
*The first two bytes of the payload may be a uint16 offset for the data offset in the target data set when the `ISB_FLAGS_PAYLOAD_W_OFFSET` flag is set in the header flags.

ISB Packet with No Payload



Packet types `PKT_TYPE_STOP_BROADCASTS_ALL_PORTS`, `PKT_TYPE_STOP_DID_BROADCAST`, `PKT_TYPE_STOP_BROADCASTS_CURRENT_PORT` have payload size zero and no payload.

ISB Get Data Packet



©2022

The ***Get Data*** packet of type `PKT_TYPE_GET_DATA` is used to query specific data according to data set ID, size, offset, and streaming period multiple. The payload size is 8. Setting the payload period to zero will result in a single response and a continuous stream of data for a non-zero period.

9.3.4 Stop Broadcasts Packets

> **Note**
>
> The NEMA $STPB stop broadcasts command is recommended as the protocol version-independent method for disabling data streaming.

Two *stop all broadcasts* packets are special packet types that will disable all binary and NMEA data streams. The following functions calls are provided in the SDK to generate the stop all broadcasts packets.

All Ports

```
uint8_t buf[200];
int len = is_comm_write_to_buf(buf, sizeof(buf), comm, PKT_TYPE_STOP_BROADCASTS_ALL_PORTS, 0, 0, 0, NULL);
if (len > 0) portWrite(0, buf, len);
```

The hexadecimal string to stop all broadcasts on all ports is:

```
0xef 0x49 0x06 0x00 0x00 0x00 0x3e 0x1f
```

Current Port Only

```
uint8_t buf[200];
int len = is_comm_write_to_buf(buf, sizeof(buf), comm, PKT_TYPE_STOP_BROADCASTS_CURRENT_PORT, 0, 0, 0, NULL);
if (len > 0) portWrite(0, buf, len);
```

The hexadecimal string to stop all broadcasts on the current port is:

```
0xef 0x49 0x08 0x00 0x00 0x00 0x40 0x27
```

9.3.5 RMC Presets

RMC Preset Stream PPD

The hexadecimal string for the RMC Preset enable PPD is:

```
0xef 0x49 0x05 0x09 0x0c 0x00 0xe2 0x3c 0x35 0x01 0x90 0x00 0x00 0xc0 0x00 0x01 0x00 0x00 0xf7 0xb0
```


©2022

9.4 NMEA 0183 (ASCII) Protocol

For simple use, the InertialSense device supports a human-readable NMEA protocol based on NMEA 0183. The NMEA protocol is human-readable in a command-line terminal, but it is less efficient than the binary protocol for the same data volume.

9.4.1 Communications Example

The NMEA Communications Example Project demonstrates how to implement the protocol.

9.4.2 Packet Structure

The Inertial Sense NMEA protocol follows the standard NMEA 0183 message structure:

- 1 byte – Start packet, `$` (`0x24`)
- n bytes – packet identifier
- 1 byte – comma (`0x2C`)
- n bytes – comma separated list of data, can include decimals and text
- 1 byte – checksum marker, `*` (`0x2A`)
- 2 bytes – checksum in hex format (i.e. `f5` or `0a`), zero-padded and lowercase
- 2 bytes – End packet, `\r\n` (`0x0D`, `0x0A`)

The packet checksum is an 8-bit integer and is calculated by XORing all bytes between (but not including) the $ and * bytes. The packet checksum byte is converted to a 2-byte NMEA hex code, and left-padded with 0 if necessary to ensure that it is always 2 bytes. The checksum is always lowercase hexadecimal characters. See NMEA 0183 message structure for more details. The NMEA string checksum is automatically computed and appended to the string when using the Inertial Sense SDK serialPortWriteAscii function or can be generated using an online checksum calculator. For example: MTK NMEA checksum calculator

9.4.3 Persistent Messages

The *persistent messages* option saves the current data stream configuration to flash memory for use following reboot, eliminating the need to re-enable messages following a reset or power cycle.

- **To save current NMEA persistent messages** - send the $PERS command.
- **To disable persistent messages** - send STPB](#stpb) followed by [PERS.

Binary persistent messages are also available.

Enabling Persistent Messages - EvalTool

To enable persistent NMEA messages using the EvalTool:

- Enable the desired NMEA messages in the EvalTool "Data Sets" tab. Select DID_NMEA_BCAST_PERIOD in the DID menu and set the desired NMEA messages period to a non-zero value.
- Press the "Save Persistent" button in the EvalTool "Data Logs" tab to store the current message configuration to flash memory.
- Reset the IMX and verify the messages are automatically streaming. You can use a generic serial port program like putty or the EvalTool->Data Logs->Data Log->Messages dialog to view the NMEA messages.

To disable all persistent messages using the EvalTool, click the "Stop Streaming" button and then "Save Persistent" button.

©2022

9.4.4 NMEA Input Messages

The following NMEA messages can be received by the IMX.

Message	Description
`$ASCE*14\r\n`	Query the broadcast rate of NMEA output messages.
ASCE	Set the broadcast period of selected NMEA output messages.
`$INFO*0E\r\n`	Query device information.
`$SRST*06\r\n`	Software reset.
`$PERS*14\r\n`	Save persistent messages to flash.
`$STPB*15\r\n`	Stop broadcast of all messages (NMEA and binary) on all ports.
`$STPC*14\r\n`	Stop broadcast of all messages (NMEA and binary) on current port.

ASCE

Enable NMEA message output streaming by specifying the NMEA message identifier or ID and broadcast period. The period is the multiple of the *data source period* (i.e., a GNSS message with period multiple of 2 and data source period of 200 ms (5 Hz) will broadcast every 400 ms). "xx" is the two-character checksum. The broadcast period for each message is configurable as a period multiple of the *Data Source Update Rates*. Up to 20 different NMEA messages can be enabled by repeating the message ID and period sequence within an ASCE message.

Note: A period value of 0 requests a single (one-shot) message and disables streaming for that message. This provides a convenient way to query an NMEA message without enabling continuous streaming.

```
$ASCE,OPTIONS,(ID,PERIOD)*xx\r\n
```

Index	Field	Description
1	`OPTIONS`	Port selection. Combine by adding options together: 0=current, 1=ser0, 2=ser1, 4=ser2, 8=USB, 512=persistent (remember after reset)
		Start of repeated group (1...20 times)
2+n*2	`ID`	Either **1.) message identifier string** (i.e., PPIMU, PINS1, GNGGA) excluding packet start character `$` or **2.) message ID** (eNmeaAsciiMsgId) of the NMEA message to be streamed. See the message ID in the NMEA output messages table.
3+n*2	`PERIOD`	Broadcast period multiple for specified message. Zero queries one message and disables streaming.
		End of repeated group (1...20 times)

EXAMPLE MESSAGES

The following example NMEA messages enable IMX data streaming output. The data period is 1, full data source rates, for those that do not specify the output rate.

The following two examples both enable the same NMEA message output:

```
$ASCE,0,PPIMU,1,PINS2,10,GNGGA,1*26\r\n
$ASCE,0,2,1,5,10,7,1*39\r\n
```

The following example will query one PINS1 message without streaming.

©2022

```
$ASCE,0,PINS1,0*0D\r\n
```

Message	Data (Output Rate)**
$ASCE,0,6,1,7,1,8,1,10,1,14,1*04\r\n	GGA, GLL, GSA, ZDA, GSV (all at 5Hz)
$ASCE,0,5,2,2,1,7,1*0A\r\n	PINS2 (35.71Hz), PPIMU (71.43Hz), GGA (5Hz)
$ASCE,0,0,1*09\r\n	PIMU
$ASCE,0,1,1*08\r\n	PPIMU
$ASCE,0,2,1*0B\r\n	PRIMU
$ASCE,0,3,1*0A\r\n	PINS1
$ASCE,0,4,1*0D\r\n	PINS2
$ASCE,0,5,1*0C\r\n	PGPSP
$ASCE,0,6,1*0F\r\n	GGA
$ASCE,0,7,1*0E\r\n	GLL
$ASCE,0,8,1*01\r\n	GSA
$ASCE,0,9,1*00\r\n	RMC
$ASCE,0,10,1*38\r\n	ZDA
$ASCE,0,11,1*39\r\n	PASHR
$ASCE,0,12,1*3A\r\n	PSTRB
$ASCE,0,13,1*3B\r\n	INFO
$ASCE,0,14,1*3C\r\n	GSV
$ASCE,0,15,1*3D\r\n	VTG

** These rates assume the default settings for data source rates.

PERS

Send this command to save current *persistent messages* to flash memory for use following reboot. This eliminates the need to re-enable messages following a reset or power cycle. To disable persistent messages, all messages must be disabled and then the 'save persistent messages' command should be sent.

```
$PERS*14\r\n
```

STPB

Stop all broadcasts (both binary and NMEA) on all ports by sending the following packet:

```
$STPB*15\r\n
```

The hexadecimal equivalent is:

```
24 53 54 50 42 2A 31 35 0D 0A
```

STPC

Stop all broadcasts (both binary and NMEA) on the current port by sending the following packet:

```
$STPC*14\r\n
```


©2022

The hexadecimal equivalent is:

```
24 53 54 50 43 2A 31 34 0D 0A
```

9.4.5 NMEA Output Messages

The following NMEA messages can be sent by the IMX. The message ID (`eNmeaAsciiMsgId`) is used with the `$ASCE` message to enable message streaming.

Identifier	ID	Description
ASCB		Broadcast period of NMEA output messages.
PIMU	1	IMU data (3-axis gyros and accelerometers) in the body frame.
PPIMU	2	Preintegrated IMU: delta theta (rad) and delta velocity (m/s).
PRIMU	3	Raw IMU data (3-axis gyros and accelerometers) in the body frame.
PINS1	4	INS output: euler rotation w/ respect to NED, NED position from reference LLA.
PINS2	5	INS output: quaternion rotation w/ respect to NED, ellipsoid altitude.
PGPSP	6	GPS position data.
GGA	7	Standard NMEA GGA GPS 3D location, fix, and accuracy.
GLL	8	Standard NMEA GLL GPS 2D location and time.
GSA	9	Standard NMEA GSA GPS DOP and active satellites.
RMC	10	Standard NMEA RMC Recommended minimum specific GPS/Transit data.
ZDA	11	Standard NMEA ZDA UTC Time/Date message.
PASHR	12	Standard NMEA PASHR (euler) message.
PSTRB	13	Strobe event input time.
INFO	14	Device information.
GSV	15	Standard NMEA GSV satellite info (all active constellations sent with corresponding talker IDs).
VTG	16	Standard NMEA VTG track made good and speed over ground.

The field codes used in the message descriptions are: lf = double, f = float, d = int.

ASCB

The `$ASCB` query reports the current NMEA broadcast configuration. Its payload contains the port options followed by `(ID,PERIOD)` pairs that mirror the `DID_NMEA_BCAST_PERIOD` dataset, letting you confirm which messages are enabled and at what rates.

NMEA Output GNSS Source

Following IMX power on/reset, the default source for NMEA output is GPS1 if available or GPS2 if GPS1 is disabled. This source is reported via bit SYS_STATUS_PRIMARY_GNSS_SOURCE_IS_GNSS2 (0x00000004) of DID_SYS_PARAMS.sysStatus where cleared means GNSS1 and set means GNSS2. Users may manually set or clear this bit to control the NMEA output GNSS source.

PIMU

IMU sensor data (3-axis gyros and accelerometers) in the body frame.

©2022

```
$PIMU,lf,f,f,f,f,f,f*xx\r\n
      1 2 3 4 5 6 7
```

Index	Field	Units	Description
1	time	sec	Time since system power up
2	IMU pqr[0]	rad/sec	IMU angular rate gyro – X
3	IMU pqr[1]	rad/sec	IMU angular rate gyro – Y
4	IMU pqr[2]	rad/sec	IMU angular rate gyro – Z
5	IMU acc[0]	m/s2	IMU linear acceleration – X
6	IMU acc[1]	m/s2	IMU linear acceleration – Y
7	IMU acc[2]	m/s2	IMU linear acceleration – Z

PPIMU

Preintegrated inertial measurement unit (IMU) sensor data, delta theta in radians and delta velocity in m/s in the body frame. Also known as coning and sculling integrals.

```
$PPIMU,lf,f,f,f,f,f,f,f*xx\r\n
       1 2 3 4 5 6 7 8
```

Index	Field	Units	Description
1	time	sec	Time since system power up
2	theta[0]	rad	IMU delta theta integral – X
3	theta[1]	rad	IMU delta theta integral – Y
4	theta[2]	rad	IMU delta theta integral – Z
5	vel[0]	m/s	IMU delta velocity integral – X
6	vel[1]	m/s	IMU delta velocity integral – Y
7	vel[2]	m/s	IMU delta velocity integral – Z
8	dt	s	Integration period for delta theta vel

PRIMU

Raw IMU sensor data (3-axis gyros and accelerometers) in the body frame (up to 8 kHz). Use this IMU data for output data rates faster than DID_FLASH_CONFIG.startupNavDtMs. Otherwise, we recommend use of PIMU or PPIMU as they are oversampled and contain less noise. 0 to disable.

©2022

```
$PRIMU,lf,f,f,f,f,f,f*xx\r\n
       1 2 3 4 5 6 7
```

Index	Field	Units	Description
1	time	sec	Time since system power up
2	Raw IMU pqr[0]	rad/sec	Raw IMU angular rate gyro – X
3	Raw IMU pqr[1]	rad/sec	Raw IMU angular rate gyro – Y
4	Raw IMU pqr[2]	rad/sec	Raw IMU angular rate gyro – Z
5	Raw IMU acc[0]	m/s2	Raw IMU linear acceleration – X
6	Raw IMU acc[1]	m/s2	Raw IMU linear acceleration – Y
7	Raw IMU acc[2]	m/s2	Raw IMU linear acceleration – Z

PINS1

INS output with Euler angles and NED offset from the reference LLA.

```
$PINS1,lf,d,d,d,f,f,f,f,f,f,lf,lf,lf,f,f,f*xx\r\n
       1 2 3 4 5 6 7 8 9 0  1  2  3 4 5 6
```

Index	Field	Units	Description
1	timeOfWeek	sec	Seconds since Sunday morning in GMT
2	GPS week	weeks	Number of weeks since January 1st of 1980 in GMT
3	insStatus		INS Status Flags
4	hdwStatus		Hardware Status Flags
5	theta[0]	rad	Euler angle – roll
6	theta[1]	rad	Euler angle – pitch
7	theta[2]	rad	Euler angle – yaw
8	UVW[0]	m/s	Velocity in body frame – X
9	UVW[1]	m/s	Velocity in body frame – Y
10	UVW[2]	m/s	Velocity in body frame – Z
11	Latitude	deg	WGS84 Latitude
12	Longitude	deg	WGS84 Longitude
13	HAE Altitude	m	Height above ellipsoid (vertical elevation)
14	NED[0]	m	Offset from reference LLA – North
15	NED[1]	m	Offset from reference LLA – East
16	NED[2]	m	Offset from reference LLA – Down

PINS2

INS output with quaternion attitude.

©2022

```
$PINS2,lf,d,d,d,f,f,f,f,f,f,f,lf,lf,lf*xx\r\n
       1 2 3 4 5 6 7 8 9 0 1  2  3  4
```

Index	Field	Units	Description
1	timeOfWeek	sec	Seconds since Sunday morning in GMT
2	GPS week	weeks	Number of weeks since January 1st of 1980 in GMT
3	insStatus		INS Status Flags
4	hdwStatus		Hardware Status Flags
5	qn2b[0]		Quaternion rotation (NED to body) – W
6	qn2b[1]		Quaternion rotation (NED to body) – X
7	qn2b[2]		Quaternion rotation (NED to body) – Y
8	qn2b[3]		Quaternion rotation (NED to body) – Z
9	UVW[0]	m/s	Velocity in body frame – X
10	UVW[1]	m/s	Velocity in body frame – Y
11	UVW[2]	m/s	Velocity in body frame – Z
12	Latitude	deg	WGS84 Latitude
13	Longitude	deg	WGS84 Longitude
14	HAE altitude	m	Height above ellipsoid (vertical elevation)

PGPSP

GPS navigation data.

```
$PGPSP,d,d,d,lf,lf,lf,f,f,f,f,f,f,f,f,f,f*xx\r\n
       1 2 3  4  5  6 7 8 9 0 1 2 3 4 5 6
```


©2022

```
$PGPSP,337272200,2031,1075643160,40.33057800,-111.72581630,1406.39,1425.18,0.95,0.37,0.55,-0.02,0.02,-0.03,0.17,39.5,337182.4521*4d\r\n
```

Index	Field	Units	Description
1	timeOfWeekMs	ms	GPS time of week in milliseconds since Sunday morning in GMT
2	GPS week	weeks	GPS number of weeks since January 1st of 1980 in GMT
3	status		(see eGnssStatus) GPS status: [0x000000xx] number of satellites used, [0x0000xx00] fix type, [0x00xx0000] status flags
4	Latitude	deg	WGS84 Latitude
5	Longitude	deg	WGS84 Longitude
6	HAE altitude	m	Height above WGS84 ellipsoid
7	MSL altitude	m	Elevation above mean sea level
8	pDOP	m	Position dilution of precision
9	hAcc	m	Horizontal accuracy
10	vAcc	m	Vertical accuracy
11	Velocity X	m/s	ECEF X velocity
12	Velocity Y	m/s	ECEF Y velocity
13	Velocity Z	m/s	ECEF Z velocity
14	sAcc	m/s	Speed accuracy
15	cnoMean	dBHz	Average of all satellite carrier-to-noise ratios (signal strengths) that are non-zero
16	towOffset	s	Time sync offset between local time since boot up to GPS time of week in seconds. Add this to IMU and sensor time to get GPS time of week in seconds.
17	leapS	s	GPS leap second (GPS-UTC) offset. Receiver's best knowledge of the leap seconds offset from UTC to GPS time. Subtract from GPS time of week to get UTC time of week.

GGA

NMEA GPS fix, 3D location and accuracy data.

©2022

```
$GPGGA,204153.200,4003.34331,N,11139.51872,W,1,25,0.93,1433.997,M,18.82,M,,*6d\r\n
              1         2 3          4 5 6  7    8     9    0     1 2 3 4
```

Index	Field	Units	Description	Example
1	HHMMSS.sss		UTC time (fix taken at 20:41:53.200 UTC)	204153.200
2,3	Latitude	deg,min	WGS84 latitude (DDmm.mmmmm,N)	4003.34331,N
4,5	Longitude	deg,min	WGS84 longitude (DDDmm.mmmmm,E)	11139.51872,W
6	Fix quality		0 = invalid, 1 = GPS fix (SPS), 2 = DGPS fix, 3 = PPS fix, 4 = RTK Fix, 5 = RTK Float, 6 = estimated (dead reckoning), 7 = Manual input mode, 8 = Simulation mode	1
7	# Satellites		Number of satellites in use	15
8	hDop	m	Horizontal dilution of precision	0.9
9,10	MSL_altitude	m	Elevation above mean sea level (MSL)	545.4,M
11,12	Undulation	m	Undulation of geoid. Height of the geoid above the WGS84 ellipsoid.	46.9,M
13	empty	s	Time since last DGPS update	
14	empty		DGPS station ID number	

GLL

NMEA geographic position, latitude / longitude and time.

```
$GPGLL,4916.45123,N,12311.12324,W,225444.800,A*33\r\n
              1 2          3 4         5 6
```

Index	Field	Units	Description	Example
1,2	Latitude	deg,min	WGS84 latitude (DDmm.mmmmm,N)	4916.45123,N
3,4	Longitude	deg,min	WGS84 longitude (DDDmm.mmmmm,E)	12311.12324,W
5	HHMMSS.sss		UTC time (fix taken at 22:54:44.8 UTC)	225444.800
6	Valid		Data valid (A=active, V=void)	A

GSA

NMEA GPS DOP and active satellites.

©2022

```
$GPGSA,A,3,04,05,,09,12,,,24,,,,,2.5,1.3,2.1*39\r\n
       1 2  3  4 ...             15  16  17
```

Index	Field	Units	Description	Example
1			Auto selection of 2D or 3D fix (M = manual)	A
2	Fix quality		Fix quality (1 = none, 2 = 2D, 3 = 3D)	3
3-14	Sat ID		Satellite ID (PRNs)	04,05,,09,12,,,24,,,,,
15	pDop	m	Dilution of precision	2.5
16	hDop	m	Horizontal dilution of precision	1.3
17	vDop	m	Vertical dilution of precision	2.1

RMC

NMEA GPS recommended minimum specific GPS/Transit data.

```
eg1. $GPRMC,081836,A,3751.65,S,14507.36,E,000.0,360.0,130998,011.3,E*62\r\n
eg2. $GPRMC,225446,A,4916.45,N,12311.12,W,000.5,054.7,191194,020.3,E*68\r\n

           225446       Time of fix 22:54:46 UTC
           A            Navigation receiver warning A = OK, V = warning
           4916.45,N    Latitude 49 deg. 16.45 min North
           12311.12,W   Longitude 123 deg. 11.12 min West
           000.5        Speed over ground, Knots
           054.7        Course Made Good, True
           191194       Date of fix  19 November 1994
           020.3,E      Magnetic variation 20.3 deg East
           *68          mandatory checksum

eg3. $GPRMC,220516,A,5133.82,N,00042.24,W,173.8,231.8,130694,004.2,W*70\r\n
             1    2    3    4    5     6    7    8      9     10  11 12

      1   220516     Time Stamp
      2   A          validity - A-ok, V-invalid
      3   5133.82    current Latitude
      4   N          North/South
      5   00042.24   current Longitude
      6   W          East/West
      7   173.8      Speed in knots
      8   231.8      True course
      9   130694     Date Stamp
      10  004.2      Variation
      11  W          East/West
      12  *70        checksum

eg4. $GPRMC,hhmmss.ss,A,llll.ll,a,yyyyy.yy,a,x.x,x.x,ddmmyy,x.x,a*hh\r\n
1    = UTC of position fix
2    = Data status (V=navigation receiver warning)
3    = Latitude of fix
4    = N or S
5    = Longitude of fix
6    = E or W
7    = Speed over ground in knots
8    = Track made good in degrees True
9    = UT date
10   = Magnetic variation degrees (Easterly var. subtracts from true course)
11   = E or W
12   = Checksum
```

VTG

NMEA GPS track made good and speed over ground.

```
eg1. $GPVTG,140.88,T,,M,8.04,N,14.89,K,D*05\r\n

0   Message ID $GPVTG
1   Track made good (degrees true)
2   T: track made good is relative to true north
3   Track made good (degrees magnetic)
4   M: track made good is relative to magnetic north
5   Speed, in knots
6   N: speed is measured in knots
7   Speed over ground in kilometers/hour (kph)
8   K: speed over ground is measured in kph
9   Mode indicator:
```


©2022

```
        A: Autonomous mode
        D: Differential mode
        E: Estimated (dead reckoning) mode
        M: Manual Input mode
        S: Simulator mode
        N: Data not valid
10  The checksum data, always begins with *
```

ZDA

NMEA GPS UTC Time and Date specification.

```
$GPZDA,213301.200,31,08,2023,00,00*41\r\n
              1  2  3    4  5  6
```

Index	Field	Units	Description	Example
1	HHMMSS.sss		UTC Time	213301.200
2	Day		Day	31
3	Month		Month	08
4	Year		Year	2023
5	localHrs		Local time zone hours	00
6	localMin		Local time zone minutes	00

GSV

NMEA GNSS satellites in view. `xx` corresponds to a constellation talker ID, shown in the table below.

Contains the number of sats in view, PRN numbers, elevation, azimuth and SNR value. Each message includes up to four satellites. When there are more than 4 sats for a constellation, multiple messages are sent. The total number of messages and the message number are included in each message.

Example:

```
$GPGSV,6,1,23,02,40,310,43,08,07,324,31,10,48,267,45,15,37,053,45*7C\r\n
$GPGSV,6,2,23,16,12,268,35,18,69,078,41,23,74,336,40,24,15,111,37*79\r\n
$GPGSV,6,3,23,26,02,239,31,27,35,307,38,29,12,162,37,32,14,199,39*7B\r\n
$GPGSV,6,4,23,44,43,188,43,46,40,206,43,522,48,267,45,527,37,053,26*73\r\n
$GPGSV,6,5,23,530,69,078,34,535,74,336,34,536,15,111,25,538,02,239,18*74\r\n
$GPGSV,6,6,23,539,35,307,27,541,12,162,21,544,14,199,25*73\r\n
$GAGSV,2,1,08,05,65,144,41,09,39,052,43,34,71,341,42,36,46,105,39*6A\r\n
$GAGSV,2,2,08,517,65,144,30,521,39,052,30,546,71,341,27,548,46,105,30*64\r\n
$GBGSV,3,1,10,11,09,141,34,14,52,047,44,27,32,313,43,28,80,263,44*64\r\n
$GBGSV,3,2,10,33,81,039,43,41,43,230,42,43,33,148,42,58,,,44*5B\r\n
$GBGSV,3,3,10,11,09,141,16,14,52,047,32*60\r\n
$GQGSV,1,1,01,02,45,101,30*49\r\n
$GLGSV,2,1,07,65,85,260,33,66,28,217,30,72,36,034,35,81,20,324,33*69\r\n
$GLGSV,2,2,07,87,47,127,35,88,73,350,34,87,47,127,20*53\r\n
```

Example NMEA version 4.11:

```
$GPGSV,4,1,14,02,40,310,43,08,07,324,31,10,48,267,45,15,37,053,45,1*67\r\n
$GPGSV,4,2,14,16,12,268,35,18,69,078,41,23,74,336,40,24,15,111,37,1*62\r\n
$GPGSV,4,3,14,26,02,239,31,27,35,307,38,29,12,162,37,32,14,199,39,1*60\r\n
$GPGSV,4,4,14,44,43,188,43,46,40,206,43,1*65\r\n
$GPGSV,3,1,09,10,48,267,45,15,37,053,26,18,69,078,34,23,74,336,34,6*68\r\n
$GPGSV,3,2,09,24,15,111,25,26,02,239,18,27,35,307,27,29,12,162,21,6*64\r\n
$GPGSV,3,3,09,32,14,199,25,6*58\r\n
$GAGSV,1,1,04,05,65,144,41,09,39,052,43,34,71,341,42,36,46,105,39,7*7E\r\n
$GAGSV,1,1,04,05,65,144,30,09,39,052,30,34,71,341,27,36,46,105,30,2*73\r\n
$GBGSV,2,1,08,11,09,141,34,14,52,047,44,27,32,313,43,28,80,263,44,1*71\r\n
$GBGSV,2,2,08,33,81,039,43,41,43,230,42,43,33,148,42,58,,,44,1*4E\r\n
$GBGSV,1,1,02,11,09,141,16,14,52,047,32,B*0D\r\n
$GQGSV,1,1,01,02,45,101,30,1*54\r\n
$GLGSV,2,1,06,65,85,260,33,66,28,217,30,72,36,034,35,81,20,324,33,1*75\r\n
$GLGSV,2,2,06,87,47,127,35,88,73,350,34,1*75\r\n
$GLGSV,1,1,01,87,47,127,20,3*41\r\n
```


©2022

Talker ID	Constellation
GP	GPS
GQ	QZSS
GA	Galileo
GL	Glonass
GB	BeiDou
GN	Combination of active constellations

Index	Field	Units	Description
1	numMsgs		Total number of messages for this constellation and epoch
2	msgNum		Message number
3	numSats		Total number of known satellites for the talker ID and signal ID
4+(n*5)	prn		Satellite PRN number
5+(n*5)	elev	deg	Elevation (0-90)
6+(n*5)	azim	deg	Azimuth (000 to 359)
7+(n*5)	snr	dB	SNR (00-99, empty when not tracking)
variable	system ID		GNSS system ID (distinguishes frequency band). This field is only output if the NMEA version is 4.11.

Where n is 0-3, for the four satellites supported by this message.

GSV OUTPUT FILTERING

Verbosity and size of the GSV NMEA message can be reduced to only select constellation and frequencies by using a Filtered GSV NMEA Message IDs instead of the standard GSV message ID `GPGSV` or `15` (`NMEA_MSG_ID_GNGSV`), either ASCII or integer. Note that the GSV output filter can only hide or mask information for satellites currently enabled in the `DID_FLASH_CONFIG.gnssSatSigConst` satellite system constellation. Usage:

```
$ASCE,[options],[Message ID]*[checksum]\r\n
```

For example, using message ID `GPGSV_1` or `3857` (`NMEA_MSG_ID_GPGSV_1`) will output only the GPS L1 frequency and prevent all other frequency and constellation satellite information from being displayed.

```
$ASCE,0,GPGSV_1,2*01\r\n
$ASCE,0,3857,2*33\r\n
```


©2022

Filtered GSV NMEA Message IDs

The following extended GSV NMEA message IDs are defined in data_sets.h.

All Constellations	Message ID ASCII	Message ID Int	Description
NMEA_MSG_ID_GNGSV_0	GNGSV_0	3840	Clear all constellations and frequencies
NMEA_MSG_ID_GNGSV_1	GNGSV_1	3841	Enable all constellations band1
NMEA_MSG_ID_GNGSV_2	GNGSV_2	3842	Enable all constellations band2
NMEA_MSG_ID_GNGSV_2_1	GNGSV_2_1	3843	Enable all constellations band1, band2
NMEA_MSG_ID_GNGSV_3	GNGSV_3	3844	Enable all constellations band3
NMEA_MSG_ID_GNGSV_3_1	GNGSV_3_1	3845	Enable all constellations band1, band3
NMEA_MSG_ID_GNGSV_3_2	GNGSV_3_2	3846	Enable all constellations band2, band3
NMEA_MSG_ID_GNGSV_3_2_1	GNGSV_3_2_1	3847	Enable all constellations band1, band2, band3
NMEA_MSG_ID_GNGSV_5	GNGSV_5	3848	Enable all constellations band5
NMEA_MSG_ID_GNGSV_5_1	GNGSV_5_1	3849	Enable all constellations band1, band5
NMEA_MSG_ID_GNGSV_5_2	GNGSV_5_2	3850	Enable all constellations band2, band5
NMEA_MSG_ID_GNGSV_5_2_1	GNGSV_5_2_1	3851	Enable all constellations band1, band2, band5
NMEA_MSG_ID_GNGSV_5_3	GNGSV_5_3	3852	Enable all constellations band3, band5
NMEA_MSG_ID_GNGSV_5_3_1	GNGSV_5_3_1	3853	Enable all constellations band1, band3, band5
NMEA_MSG_ID_GNGSV_5_3_2	GNGSV_5_3_2	3854	Enable all constellations band2, band3, band5
NMEA_MSG_ID_GNGSV_5_3_2_1	GNGSV_5_3_2_1	3855	Enable all constellations band1, band2, band3, band5

GPGSV - GPS	Message ID ASCII	Message ID Int	Description
NMEA_MSG_ID_GPGSV_0	GPGSV_0	3856	Disable all GPS frequencies
NMEA_MSG_ID_GPGSV_1	GPGSV_1	3857	Enable GPS L1
NMEA_MSG_ID_GPGSV_2	GPGSV_2	3858	Enable GPS L2
NMEA_MSG_ID_GPGSV_2_1	GPGSV_2_1	3859	Enable GPS L1, L2
NMEA_MSG_ID_GPGSV_5	GPGSV_5	3864	Enable GPS L5
NMEA_MSG_ID_GPGSV_5_1	GPGSV_5_1	3865	Enable GPS L1, L5
NMEA_MSG_ID_GPGSV_5_2	GPGSV_5_2	3866	Enable GPS L2, L5
NMEA_MSG_ID_GPGSV_5_2_1	GPGSV_5_2_1	3867	Enable GPS L1, L2, L5
NMEA_MSG_ID_GPGSV	GPGSV	3871	Enable all GPS frequencies

©2022

GAGSV - Galileo	Message ID ASCII	Message ID Int	Description
NMEA_MSG_ID_GAGSV_0	GAGSV_0	3888	Disable all Galileo frequencies
NMEA_MSG_ID_GAGSV_1	GAGSV_1	3889	Enable Galileo E1
NMEA_MSG_ID_GAGSV_5	GAGSV_5	3896	Enable Galileo E5
NMEA_MSG_ID_GAGSV_5_1	GAGSV_5_1	3897	Enable Galileo E1, E5
NMEA_MSG_ID_GAGSV	GAGSV	3903	Enable all Galileo frequencies

GBGSV - Beido	Message ID ASCII	Message ID Int	Description
NMEA_MSG_ID_GBGSV_0	GBGSV_0	3904	Disable all Beidou frequencies
NMEA_MSG_ID_GBGSV_1	GBGSV_1	3905	Enable Beidou B1
NMEA_MSG_ID_GBGSV_2	GBGSV_2	3906	Enable Beidou B2
NMEA_MSG_ID_GBGSV_2_1	GBGSV_2_1	3907	Enable Beidou B1, B2
NMEA_MSG_ID_GBGSV_3	GBGSV_3	3908	Enable Beidou B3
NMEA_MSG_ID_GBGSV_3_1	GBGSV_3_1	3909	Enable Beidou B1, B3
NMEA_MSG_ID_GBGSV_3_2	GBGSV_3_2	3910	Enable Beidou B2, B3
NMEA_MSG_ID_GBGSV_3_2_1	GBGSV_3_2_1	3911	Enable Beidou B1, B2, B3
NMEA_MSG_ID_GBGSV	GBGSV	3919	Enable all Beidou frequencies

GQGSV - QZSS	Message ID ASCII	Message ID Int	Description
NMEA_MSG_ID_GQGSV_0	GQGSV_0	3920	Disable all QZSS frequencies
NMEA_MSG_ID_GQGSV_1	GQGSV_1	3921	Enable QZSS L1
NMEA_MSG_ID_GQGSV_2	GQGSV_2	3922	Enable QZSS L2
NMEA_MSG_ID_GQGSV_2_1	GQGSV_2_1	3923	Enable QZSS L1, L2
NMEA_MSG_ID_GQGSV_5	GQGSV_5	3928	Enable QZSS L5
NMEA_MSG_ID_GQGSV_5_1	GQGSV_5_1	3929	Enable QZSS L1, L5
NMEA_MSG_ID_GQGSV_5_2	GQGSV_5_2	3930	Enable QZSS L2, L5
NMEA_MSG_ID_GQGSV_5_2_1	GQGSV_5_2_1	3931	Enable QZSS L1, L2, L5
NMEA_MSG_ID_GQGSV	GQGSV	3935	Enable all QZSS frequencies

©2022

GLGSV - Glonass	Message ID ASCII	Message ID Int	Description
NMEA_MSG_ID_GLGSV_0	GLGSV_0	3936	Disable all Glonass frequencies
NMEA_MSG_ID_GLGSV_1	GLGSV_1	3937	Enable Glonass L1
NMEA_MSG_ID_GLGSV_2	GLGSV_2	3938	Enable Glonass L2
NMEA_MSG_ID_GLGSV_2_1	GLGSV_2_1	3939	Enable Glonass L1, L2
NMEA_MSG_ID_GLGSV_3	GLGSV_3	3940	Enable Glonass L3
NMEA_MSG_ID_GLGSV_3_1	GLGSV_3_1	3941	Enable Glonass L1, L3
NMEA_MSG_ID_GLGSV_3_2	GLGSV_3_2	3942	Enable Glonass L2, L3
NMEA_MSG_ID_GLGSV_3_2_1	GLGSV_3_2_1	3943	Enable Glonass L1, L2, L3
NMEA_MSG_ID_GLGSV	GLGSV	3951	Enable all Glonass frequencies

VTG

NMEA GPS track made good and speed over ground.

```
$GPVTG,140.88,T,,M,8.04,N,14.89,K,D*05\r\n
          1 2 3 4   5 6     7 8 9
```

| Index | Field | Units | Description | Example | |-------|--------|-------|---|---------| | 1 | track true | deg | Ground track heading (true north) | 140.88 | | 2 | T | | Ground track heading is relative to true north | T | | 3 | track mag | deg | Ground track heading (magnetic north) | | | 4 | M | | Ground track heading is relative to magnetic north (track mag = track true + magVarCorrection) | M | | 5 | speed Kn | knots | Speed | 8.04 | | 6 | N | | Speed is measured in knots | N | | 7 | speed Km | kph | Speed over ground in kilometers/hour | 14.89 | | 8 | K | | Speed over ground is measured in kph | K | | 9 | mode ind | | Mode indicator: | D | | | | A: Autonomous mode | | | | | D: Differential mode | | | | | E: Estimated (dead reckoning) mode | | | | | M: Manual Input mode | | | | | S: Simulator mode | | | | | N: Data not valid | |

PASHR

NMEA heading, roll, pitch, and heave attitude message.

©2022

```
$PASHR,001924.600,95.81,T,+0.60,+1.05,+0.00,0.038,0.035,0.526,0,0*08\r\n
         1         2   3   4     5    6    7     8     9  10 11
```

Index	Field	Units	Description	Example
1	Time		UTC Time	001924.600
2	Heading		Heading value in decimal degrees	95.81
3	True Heading		T displayed if heading is relative to true north.	T
4	Roll	m	Roll in decimal degrees.	+0.60
5	Pitch	m	Pitch in decimal degrees.	+1.05
6	Heave	m	Instantaneous heave in meters.	+0.00
7	Roll Accuracy		Roll standard deviation in decimal degrees.	+0.038
8	Pitch Accuracy		Pitch standard deviation in decimal degrees.	0.035
9	Heading Accuracy		Heading standard deviation in decimal degrees.	0.526
10	GPS Status		GPS Status	0
11	INS Status		INS Status	0

PSTRB

Strobe input time. This message is sent when an assert event occurs on a strobe input pin.

```
$PSTRB,d,d,d,d*xx\r\n
       1 2 3 4
```

Index	Field	Units	Description
1	GPS week	weeks	Number of weeks since January 1st of 1980 in GMT
2	timeMsOfWeek	ms	Milliseconds since Sunday morning in GMT
3	pin		Strobe event input pin number
4	count		Strobe event serial index number

INFO

Device version information. Query this message by sending `$INFO*0E\r\n`.

©2022

```
$INFO,d,d.d.d.d,d.d.d.d,d,d.d.d.d,d,s,YYYY-MM-DD,hh:mm:ss.ms,s*xx\r\n
      1 2       3       4 5       6 7 8          9           10
```

Index	Field	Units	Description
1	Serial number		Manufacturer serial number
2	Hardware version		Hardware version
3	Firmware version		Firmware version
4	Build number		Firmware build number
5	Protocol version		Communications protocol version
6	Repo revision		Repository revision number
7	Manufacturer		Manufacturer name
8	Build date		Build date: [1] = year, [2] = month, [3] = day
9	Build time		Build date: [0] = hour, [1] = minute, [2] = second, [3] = millisecond
10	Add Info		Additional information
11	Hardware		Hardware: 1=uINS, 2=EVB, 3=IMX, 4=GPX
12	Reserved		Reserved for internal purpose.
13	Build type		Build type: 'a'=ALPHA, 'b'=BETA, 'c'=RELEASE CANDIDATE, 'r'=PRODUCTION RELEASE, 'd'=debug

9.4.6 NMEA Examples

Note

If the command strings below are altered, their checksum must be recalculated.

Note

All NMEA command strings must be followed with a carriage return and new line character (`\r\n` or `0x0D`, `0x0A`).

The NMEA string checksum is automatically computed and appended to string when using the Inertial Sense SDK serialPortWriteAscii function or can be generated using an online NMEA checksum calculator. For example: MTK NMEA checksum calculator

Stop streams on CURRENT port

```
$STPB*15
```

Stop all streams on ALL ports

```
$STPC*14
```

Query device version information

```
$INFO*0E
```


©2022

Response:

```
$INFO,30612,3.1.2.0,1.7.0.0,3522,1.2.74.7,6275,Inertial Sense Inc,0018-10-16,23:20:38.41,INL2*58
```

Stream INS1 @5Hz on port 0

```
$ASCE,1,3,1*0B
```

Stream INS1 @400ms on current port

```
$ASCE,0,3,2*09
```

Response:

```
$PINS1,244272.398,2021,427888998,805306448,0.0468,-0.3830,-0.0909,0.232,-0.083,-0.089,40.05574940,-111.65861580,1438.451,-1.678,-5.086,-9.697*11
$PINS1,244272.498,2021,427888998,805306448,0.0469,-0.3830,-0.0902,0.232,-0.081,-0.089,40.05575000,-111.65861550,1438.451,-1.611,-5.060,-9.697*18
$PINS1,244272.598,2021,427888998,805306448,0.0469,-0.3830,-0.0902,0.232,-0.081,-0.089,40.05575022,-111.65861562,1438.449,-1.587,-5.070,-9.695*1e
```

Stream INS1 @600ms on serial port 1

```
$ASCE,2,3,3*0A
```

Response:

```
$PINS1,256270.627,2021,427888998,1073741912,0.1153,-0.1473,-0.1628,0.001,0.001,0.003,40.05569486,-111.65864500,1416.218,-7.738,-7.570,12.536*3d
$PINS1,256270.647,2021,427888998,1073741912,0.1153,-0.1473,-0.1632,0.001,0.001,0.003,40.05569486,-111.65864500,1416.219,-7.738,-7.570,12.535*32
$PINS1,256270.667,2021,427888998,1073741912,0.1153,-0.1473,-0.1631,0.001,0.001,0.003,40.05569486,-111.65864500,1416.220,-7.738,-7.570,12.534*38
```

Stream PIMU @400ms and GGA @5Hz on current port

```
$ASCE,0,6,1,0,2*0D
```

Response:

```
$PIMU,3218.543,0.0017,-0.0059,-0.0077,-1.417,-1.106,-9.524,0.0047,0.0031,-0.0069,-1.433,-1.072,-9.585*1f
$GPGGA,231841,4003.3425,N,11139.5188,W,1,29,0.89,1434.16,M,18.82,M,,*59
$GPGGA,231841,4003.3425,N,11139.5188,W,1,29,0.89,1434.19,M,18.82,M,,*56
$PIMU,3218.763,0.0019,-0.0062,-0.0086,-1.426,-1.114,-9.509,0.0054,0.0029,-0.0070,-1.431,-1.085,-9.579*13
$GPGGA,231841,4003.3425,N,11139.5188,W,1,29,0.89,1434.16,M,18.82,M,,*59
$GPGGA,231841,4003.3425,N,11139.5188,W,1,29,0.89,1434.19,M,18.82,M,,*56
$PIMU,3218.543,0.0017,-0.0059,-0.0077,-1.417,-1.106,-9.524,0.0047,0.0031,-0.0069,-1.433,-1.072,-9.585*1f
```

9.4.7 Linux Command Line NMEA Query

This section illustrates how to request NMEA output using standard Linux commands without relying on the Inertial Sense SDK, cltool, or EvalTool. This provides a simple way to communicate directly with the device over a port, making it useful for quickly identifying device information, firmware version, or any available NMEA message.

Configure the serial port parameters (i.e. baudrate, etc).

```
stty -F /dev/ttyACM0 921600 raw -echo -crtscts
```

Stop message streaming on the current port. This prevents previously streamed messages from interfering with the response to a specific request.

```
cat < /dev/ttyACM0 & printf '%s\r\n' '$STPC*14' > /dev/ttyACM0
```

Request the device information.

```
cat < /dev/ttyACM0 & printf '%s\r\n' '$INFO*0E' > /dev/ttyACM0
```

Request the device information and exit after receiving the response.

```
printf '%s\r\n' '$INFO*0E' > /dev/ttyACM0 ; sleep 0.2 ; timeout 0.2 cat < /dev/ttyACM0
```


©2022

Request the $PINS1 solution output and exit after receiving the response.

```
printf '%s\r\n' '$ASCE,0,PINS1,0*0D' > /dev/ttyACM0 ; sleep 0.2 ; timeout 0.2 cat < /dev/ttyACM0
```


©2022

9.5 SPI Protocol

The SPI interface provides an alternate communication path for IMX and GPX modules, supporting the same protocols as the UART and USB interfaces. It operates in a master/slave configuration with the module as the slave; the master clocks data in and out. When the output buffer is empty, the module transmits zeros and the Data Ready (DR) pin goes low. Two read strategies are available: **fixed-size polling**, which reads a fixed block at a regular interval regardless of DR state, and **Data Ready gated**, which drives reads from the DR pin and packet framing. Fixed-size polling is simpler to implement; the DR-gated approach is more efficient and supports higher data rates.

9.5.1 Enable SPI

To enable SPI, hold pin G9 (nSPI_EN) low at startup.

Note: When an external GPS PPS timepulse signal is enabled on G9, the module will ignore the nSPI_EN signal and SPI mode will be disabled regardless of the G9 pin state.

9.5.2 Hardware

The Inertial Sense SPI interface uses 5 lines:

Line	Function
CS	Chip Select
SCLK	Clock Synchronization
MISO	Master In Slave Out
MOSI	Master Out Slave In
DR	Data Ready (optional)

Hardware Configuration

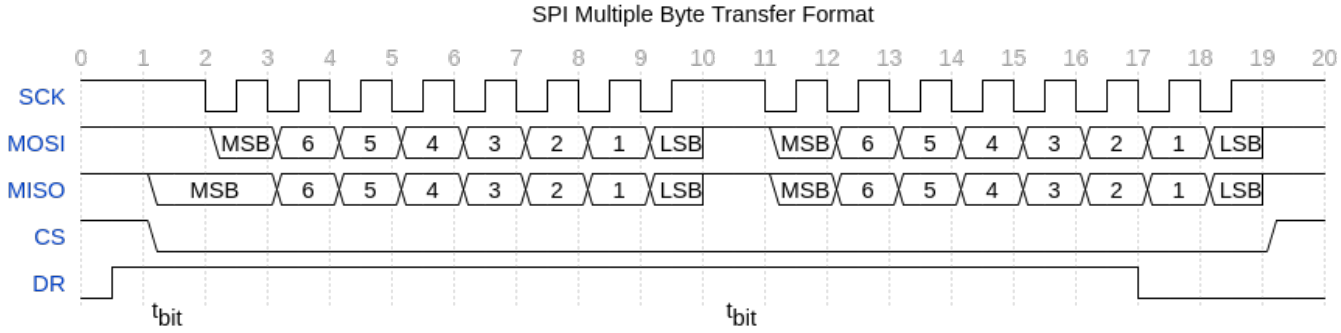
The IMX and GPX modules operate as an SPI slave device using **SPI Mode 3**:

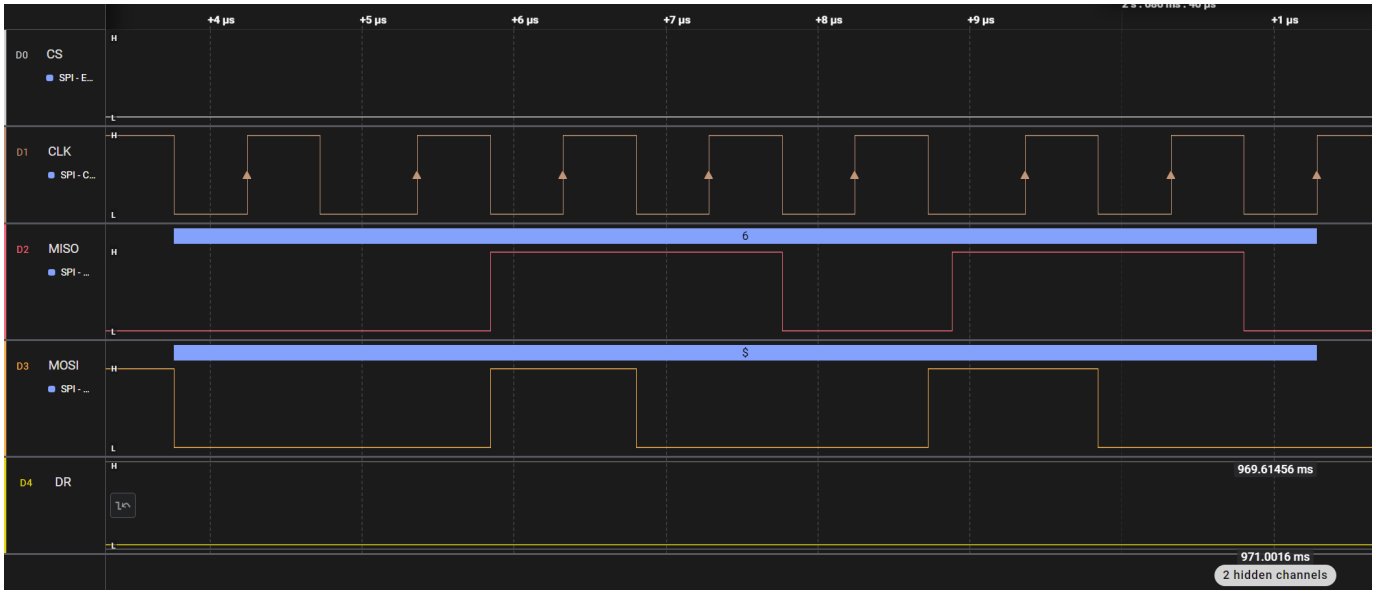
Setting	Value
SPI Mode	3
CPOL (Clock Polarity)	1 (idle high)
CPHA (Clock Phase)	1 (sample on rising edge, shift on falling edge)
Chip Select	Active low
Data Ready	Active high

Data Transfer

To ensure correct behavior in SPI slave mode, the master must provide a minimum delay of one t_{bit} between each byte transmission, where t_{bit} is the nominal time to transmit one bit. The module does not require a falling edge on CS to begin receiving — only a sustained low level. However, CS must be asserted at least one t_{bit} before the first clock cycle corresponding to the MSB. [1]

©2022





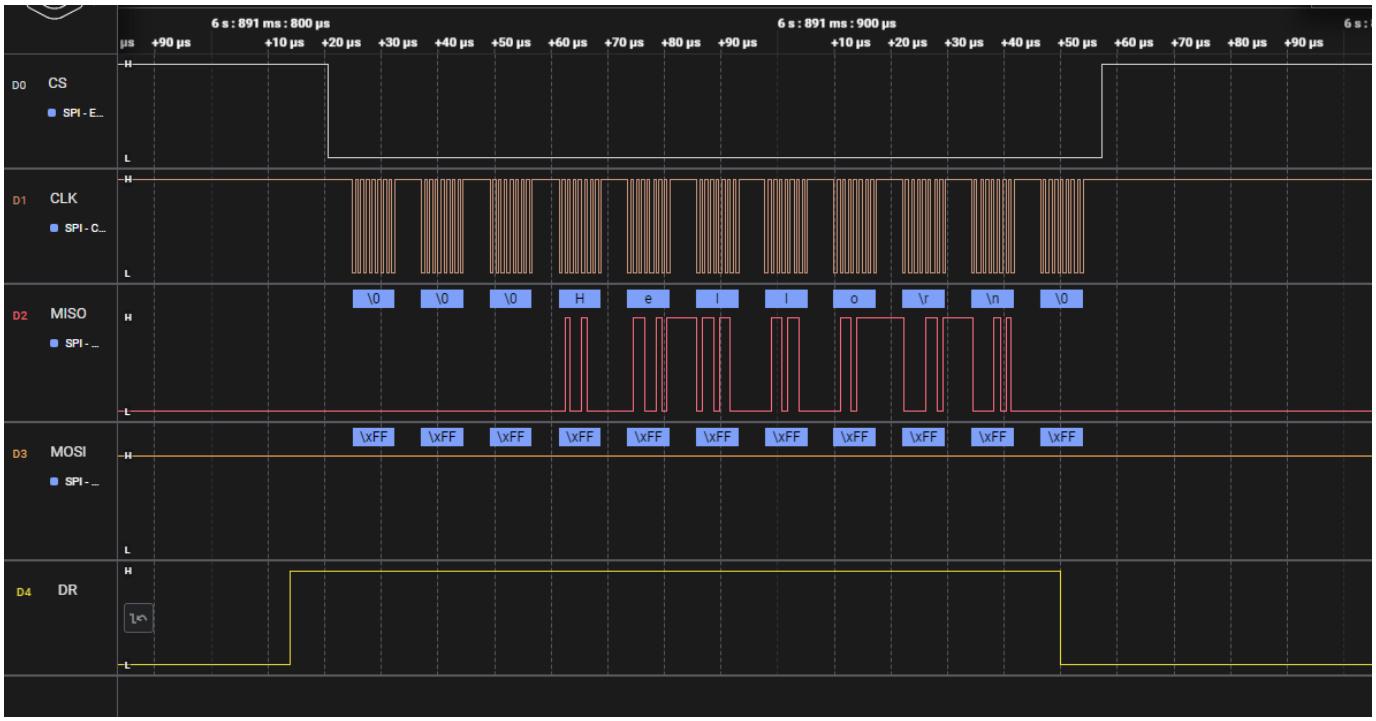
When no data is ready, the module transmits zeros.

Keeping CS continuously asserted is generally safe. However, if the master and slave clocks fall out of sync — due to noise, ground bounce, or a fast transient causing a spurious clock edge — there is no automatic recovery mechanism. Toggling CS (deasserting then reasserting) resets the shift register and resynchronizes the clocks.

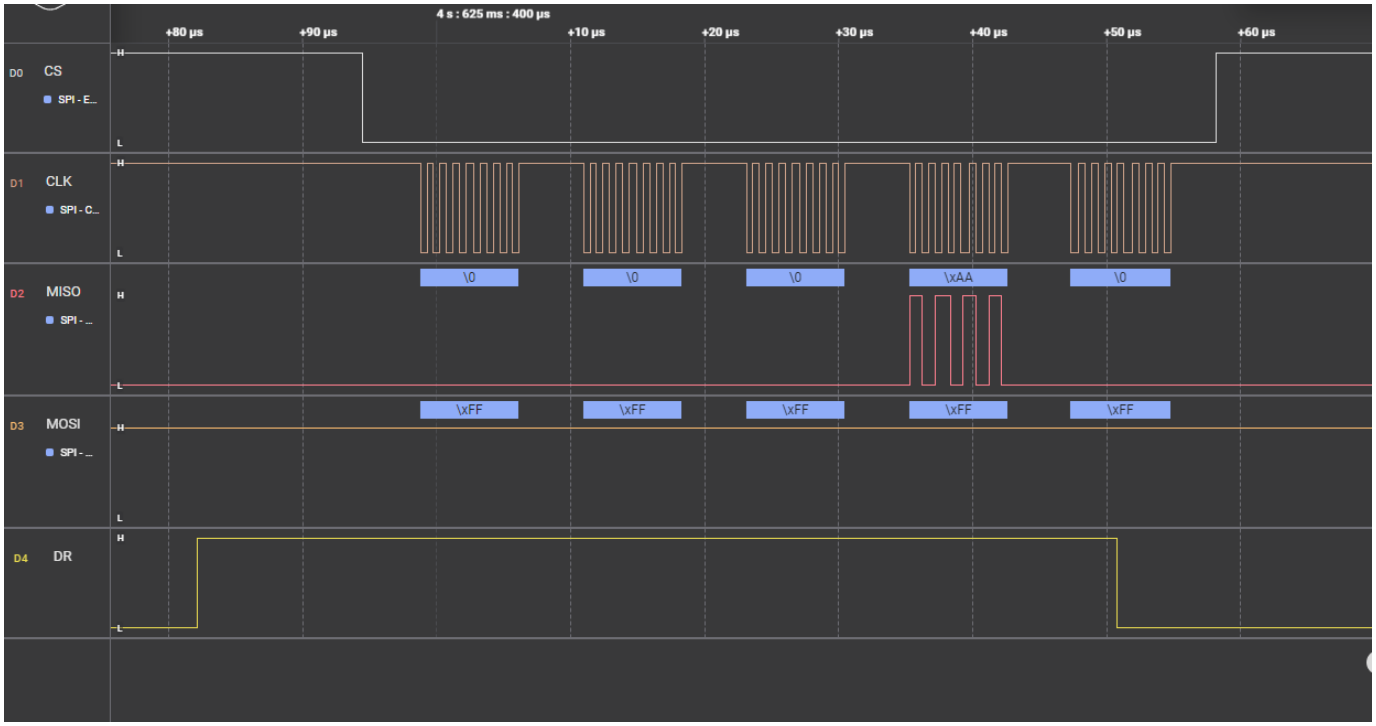
Data Ready Pin

The DR pin goes high when data is available and goes inactive one or two bytes before the end of the packet is clocked out. There is no dedicated end-of-packet character; framing relies on the DR pin and packet parsing.

©2022



Depending on timing, 1–4 bytes of zeros may precede the start of a packet.



If CS is deasserted during a packet transmission, the byte being clocked out may be lost. It is recommended to deassert CS only when outside of a data packet and the DR pin is inactive.

The internal SPI buffer is 4096 bytes. On overflow, the entire buffer is dropped — the module may be mid-packet and will transmit zeros until new data arrives. A buffer overflow is indicated by DR going high with no valid data present. Sending a request to the module or waiting for a periodic packet will restore normal operation.

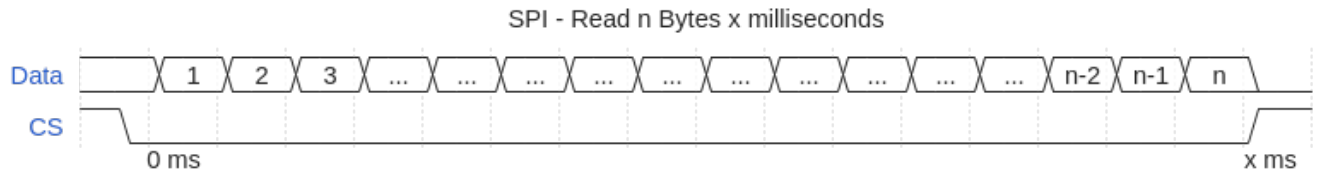
The SPI interface supports data rates up to **3 Mbps**. Using the Data Ready pin to gate reads (Strategy B below) allows operation up to **5 Mbps**.

©2022

Reading Data

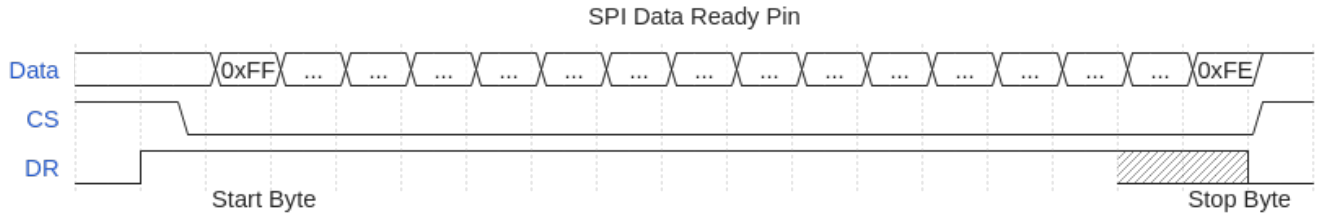
There are two strategies for reading data: **fixed-size polling**, where a fixed block is read at a regular interval regardless of DR state, and **Data Ready gated**, where reads are driven by the DR pin and packet framing. Fixed-size polling is simpler to implement; the DR-gated approach is more efficient.

A. Fixed-size polling — Read a fixed block at a regular interval, sized to reliably drain the buffer. For example, read 512 bytes every 4 ms.



The read buffer is zero-padded if the requested byte count exceeds the available data.

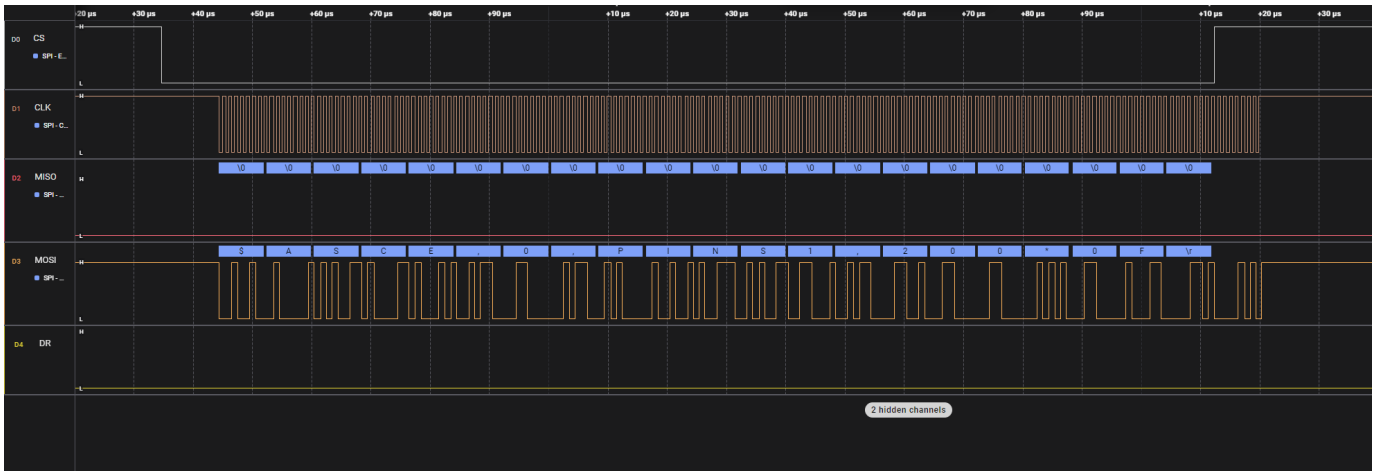
B. Data Ready gated — Read while the DR pin is active, or while inside a data packet. Because DR goes inactive a byte or two before the packet ends, the master must continue reading until the end-of-packet marker is found.



Pseudocode for Reading Data

1. Check the DR pin. If low, wait and check again.
2. Assert CS and read a block of data. Block size is arbitrary but should be large enough to contain most packets.
3. After the read, check DR again. If high, read more data without deasserting CS — raising CS while DR is high will cause data loss. If DR is low, deassert CS. On busy systems or at high data rates, DR may not go low between packets; in that case, keep CS asserted and continue reading.
4. Parse the incoming data for the start-of-packet byte (0xFF), discarding all preceding bytes.
5. Continue reading and parsing until the end-of-packet byte (0xFE) is found, then dispatch the complete packet. If another start-of-packet byte is encountered before the end is found, discard the partial packet and restart.

©2022



9.5.3 Troubleshooting

Every other byte is lost — The CS line may be toggling after each byte. CS should remain asserted for the duration of a transaction.

©2022

9.6 CAN Protocol

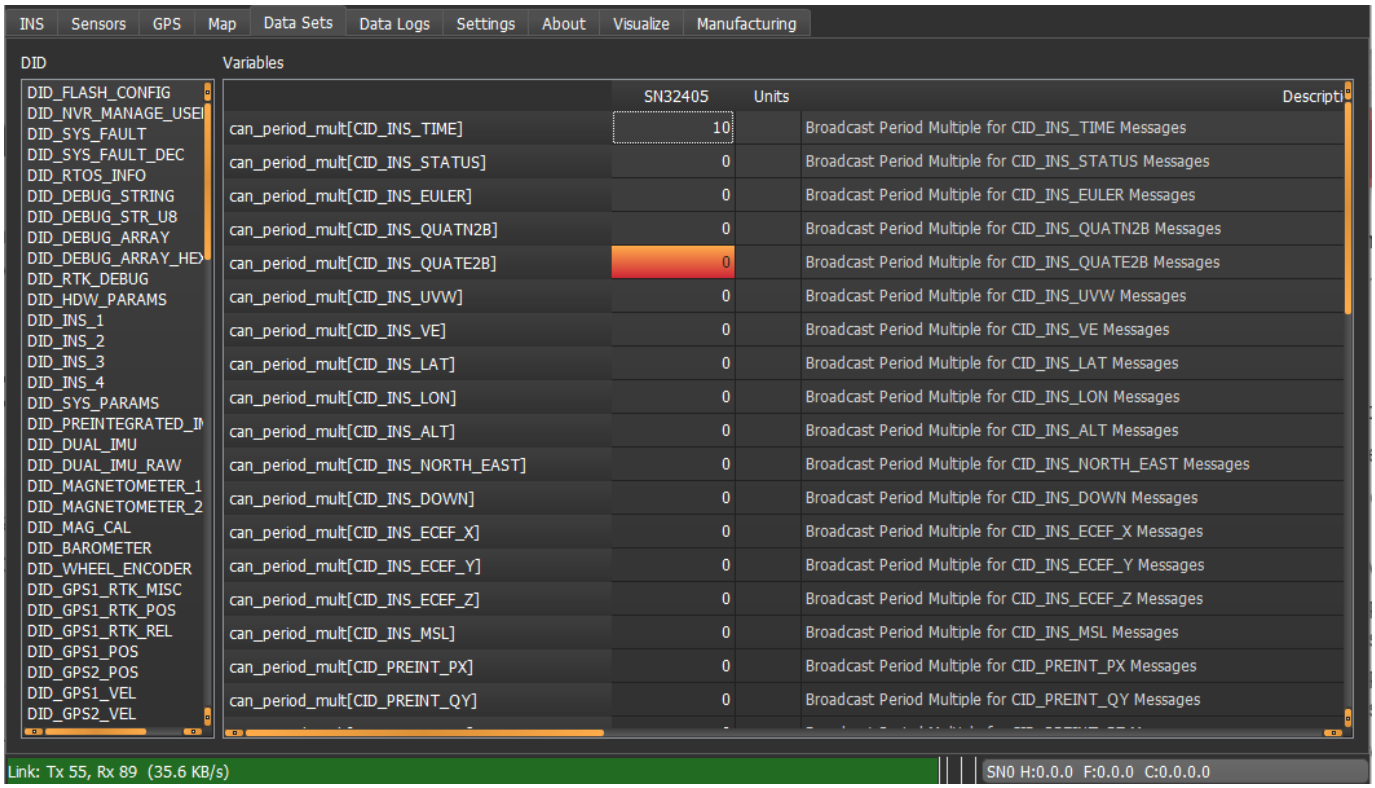
The CAN interface allows the output of the IMX to be published on a CAN bus. The Inertial Sense CAN implementation is based on CAN2.0b specification and follows a specific structure and format which is outlined below. All of the CAN configuration is done using the data set DID_CAN_CONFIG.

9.6.1 Enable CAN

To enable the CAN bus interface on the IMX, set bit `IO_CONFIG_G1G2_CAN_BUS` in `DID_FLASH_CONFIG.ioConfig`. This bit can be set using the EvalTool >> Settings >> General >> DID_FLASH_CONFIG >> ioConfig >> `Enable CAN Bus on G1,G2` option.

A CAN message is enabled by entering a non-zero value in the DID_CAN_CONFIG.*can_period_mult* field of the desired message. The *can_period_mult* field is an integer which is multiplied by the *Data Source Update Rate* to determine the message broadcast period. Set *can_period_mult* to zero disable message broadcasting.

In the image below the CID_INS_TIME message is set to broadcast data at 10 times the data source rate.



©2022

The baud rate is configurable by setting the field DID_CAN_CONFIG.can_setting. The following standard baud rates are supported:

- 20 kbps
- 33 kbps
- 50 kbps
- 83 kbps
- 100 kbps
- 125 kbps
- 200 kbps
- 250 kbps
- 500 kbps
- 1000 kbps

The message ID for each message can be entered into the can_transmit_address field corresponding to the desired message.

*Note: Any message ID greater than 0x7FF will be transmitted in the extended ID format.

The values set in any field of DID_CAN_CONFIG are saved to flash when a 'Save Persistent' command is received by the module. For example, this can be done in the EvalTool by clicking the Save Persistent button in the Data Logs tab. When the module is turned on, all the fields will be repopulated with the saved values.

All messages are disabled when a Stop Streaming message is received by the module. However, the values in each field will be repopulated to the values present when a 'Save Persistent' command was last received.

9.6.2 Hardware

Inertial Sense module exposes the RxCAN and TxCAN pins. The selection and implementation of a CAN transceiver is left to the user.

Line	Function
G1	RxCAN
G2	TxCAN

The Inertial Sense evaluation boards and Rugged unit have a built in transceiver.

9.6.3 CAN Data Sets (CIDs)

The CAN Data Sets, in the form of C structures, define the format of the output data. The data sets are defined in SDK\src\data_sets_canbus.h of the InertialSense SDK. The CID data is selected data from the standard Inertial Sense DIDs. The data types generally have been changed and scaled to fit the CAN2.0 8 byte payload restrictions.

CID_INS_TIME

INS time output

`is_can_time`

GMT information

Field	Type	Description
week	uint32_t	GPS number of weeks since January 6th, 1980
timeOfWeekMs	float	GPS time of week (since Sunday morning) in milliseconds

©2022

CID_INS_STATUS

`is_can_ins_status`

INS status flags

Field	Type	Description
insStatus	uint32_t	INS status flags (see eInsStatusFlags)
hdwStatus	uint32_t	Hardware status flags (see eHdwStatusFlags)

CID_INS_EULER

`is_can_ins_euler`

Euler angles: roll, pitch, yaw in radians with respect to NED (scaled by 10000)

Field	Type	Description
theta1	int16_t	Roll (4 decimal places precision)
theta2	int16_t	Pitch (4 decimal places precision)
theta3	int16_t	Yaw (4 decimal places precision)

CID_INS_QUATN2B

`is_can_ins_quatn2b`

Quaternion body rotation with respect to NED: W, X, Y, Z (scaled by 10000)

Field	Type	Description
qn2b1	int16_t	W (4 decimal places precision)
qn2b2	int16_t	X (4 decimal places precision)
qn2b3	int16_t	Y (4 decimal places precision)
qn2b4	int16_t	Z (4 decimal places precision)

CID_INS_QUATE2B

`is_can_ins_quate2b`

Quaternion body rotation with respect to ECEF: W, X, Y, Z (scaled by 10000)

Field	Type	Description
qe2b1	int16_t	W (4 decimal places precision)
qe2b2	int16_t	X (4 decimal places precision)
qe2b3	int16_t	Y (4 decimal places precision)
qe2b4	int16_t	Z (4 decimal places precision)

CID_INS_UVW

`is_can_uvw`

©2022

Velocity U, V, W in body frame in meters per second (scaled by 100).

Field	Type	Description
uvw1	int16_t	U (2 decimal places precision)
uvw2	int16_t	V (2 decimal places precision)
uvw3	int16_t	W (2 decimal places precision)

CID_INS_VE

`is_can_ve`

Velocity in ECEF (earth-centered earth-fixed) frame in meters per second (scaled by 100).

Field	Type	Description
ve1	int16_t	ve1 (2 decimal places precision)
ve2	int16_t	ve2 (2 decimal places precision)
ve3	int16_t	ve3 (2 decimal places precision)

CID_INS_LAT

`is_can_ins_lat`

WGS84 latitude.

Field	Type	Description
lat	double	Latitude (degrees) (more than 8 decimal places precision)

CID_INS_LON

`is_can_ins_lon`

WGS84 longitude.

Field	Type	Description
lon	double	Longitude (degrees) (more than 8 decimal places precision)

CID_INS_ALT

`is_can_ins_alt`

WGS84 height above ellipsoid and GPS status flags

Field	Type	Description
alt	float	Altitude (meters) (more than 8 decimal places precision)
status	uint32_t	(see eGnssStatus) GPS status: [0x000000xx] number of satellites used, [0x0000xx00] fix type, [0x00xx0000] status flags

CID_INS_NORTH_EAST

`is_can_north_east`

©2022

Offset from reference latitude, longitude, and altitude to current latitude, longitude, and altitude.

Type	Field	Description
float	ned1	North (meters)
float	ned2	East (meters)

CID_INS_DOWN

`is_can_down`

Down offset from reference LLA and INS status flags

Type	Field	Description
float	ned3	Down (meters)
float	insStatus	INS status flags

CID_INS_ECEF_X

`is_can_ecef_x`

X Position in ECEF (earth-centered earth-fixed) frame.

Type	Field	Description
ecef1	double	X (meters)

CID_INS_ECEF_Y

`is_can_ecef_y`

Y Position in ECEF (earth-centered earth-fixed) frame.

Type	Field	Description
ecef2	double	Y (meters)

CID_INS_ECEF_Z

`is_can_ecef_z`

Z Position in ECEF (earth-centered earth-fixed) frame.

Type	Field	Description
ecef2	double	Z (meters)

CID_INS_MSL

`is_can_msl`

Height above Mean Sea Level

Type	Field	Description
msl	float	MSL (meters)

CID_PREINT_PX

`is_can_preint_imu_px`

©2022

Preintegrated IMU values delta theta and delta velocity (X axis), and Integral period in body/IMU frame of accelerometer 0.

Type	Field	Description
theta0	int16_t	Delta theta (rad, scaled by 1000, 3 decimal places precision)
vel0	int16_t	Delta velocity (m/s, scaled by 100, 2 decimal places precision)
dt	uint16_t	Integral Period (seconds, scaled by 1000)

CID_PREINT_QY

`is_can_preint_imu_qy`

Preintegrated IMU values delta theta and delta velocity (Y axis), and Integral period in body/IMU frame of accelerometer 0.

Type	Field	Description
theta1	int16_t	Delta theta (rad, scaled by 1000, 3 decimal places precision)
vel1	int16_t	Delta velocity (m/s, scaled by 100, 2 decimal places precision)
dt	uint16_t	Integral Period (seconds, scaled by 1000)

CID_PREINT_RZ

`is_can_preint_imu_rz`

Preintegrated IMU values delta theta and delta velocity (Z axis), and Integral period in body/IMU frame of accelerometer 0.

Type	Field	Description
theta2	int16_t	Delta theta (rad, scaled by 1000, 3 decimal places precision)
vel2	int16_t	Delta velocity (m/s, scaled by 100, 2 decimal places precision)
dt	uint16_t	Integral Period (seconds, scaled by 1000)

CID_DUAL_PX

`is_can_dual_imu_px`

Dual IMU gyro and accelerometer values from accelerometer 0

Type	Field	Description
theta0	int16_t	Theta (rad/s, scaled by 1000, 3 decimal places precision)
vel0	int16_t	Acceleration (m/s^2, scaled by 100, 2 decimal places precision)
status	uint32_t	IMU status (see eImuStatus)

CID_DUAL_QY

`is_can_dual_imu_qy`

Dual IMU gyro and accelerometer values from accelerometer 0

Type	Field	Description
theta1	int16_t	Theta (rad/s, scaled by 1000, 3 decimal places precision)
vel1	int16_t	Acceleration (m/s^2, scaled by 100, 2 decimal places precision)
status	uint32_t	IMU status (see eImuStatus)

©2022

CID_DUAL_RZ

`is_can_dual_imu_rz`

Dual IMU gyro and accelerometer values from accelerometer 0

Type	Field	Description
theta2	int16_t	Theta (rad/s, scaled by 1000, 3 decimal places precision)
vel2	int16_t	Acceleration (m/s^2, scaled by 100, 2 decimal places precision)
status	uint32_t	IMU status (see eImuStatus)

CID_GNSS1_POS

`is_can_gnss_pos_status`

GPS CNO Mean and GPS status flags

Type	Field	Description
status	uint32_t	(see eGnssStatus) GPS status: [0x000000xx] number of satellites used, [0x0000xx00] fix type, [0x00xx0000] status flags
cnoMean	uint32_t	(dBHz) Average of all satellite carrier to noise ratios (signal strengths) that are non-zero

CID_GNSS1_RTK_POS_REL

`is_can_gnss_rtk_rel`

RTK-GPS positioning performance metrics

Type	Field	Description
arRatio	uint8_t	Ambiguity resolution ratio factor for validation
differentialAge	uint8_t	Age of differential (seconds)
distanceToBase	float	Distance to Base (m)
headingToBase	int16_t	Angle from north to vectorToBase in local tangent plane. (rad, scaled by 1000)

CID_GNSS2_RTK_CMP_REL

`is_can_gnss_rtk_rel`

RTK-GPS compassing performance metrics

Type	Field	Description
arRatio	uint8_t	Ambiguity resolution ratio factor for validation
differentialAge	uint8_t	Age of differential (seconds)
distanceToBase	float	Distance to Base (m)
headingToBase	int16_t	Angle from north to vectorToBase in local tangent plane. (rad, scaled by 1000)

CID_ROLL_ROLLRATE

`is_can_roll_rollRate`

©2022

Combination or INS roll estimation and preintegrated IMU of both accelerometers

Type	Field	Description
insRoll	int16_t	INS Roll (scaled by 10000, 4 decimal places precision)
pImu1	int16_t	Delta theta (rad, scaled by 1000, 3 decimal places precision)
pImu2	int16_t	Delta theta (rad, scaled by 1000, 3 decimal places precision)

©2022

10. GNSS - RTK

10.1 Multi-band GNSS

10.1.1 Advantages

The advent of multi-band GNSS (multiple frequency global navigation satellite systems) improves accuracy by reducing the impact of errors caused by multi-path and atmospheric distortion. When compared to traditional single-band GNSS, dual-band technology provides about a 2x reduction in average position error (circular error probable - CEP). Benefits of multi-band GNSS systems like the uBlox ZED-F9P or the Inertial Sense GPX-1 receiver include:

- Concurrent reception of GPS, GLONASS, Galileo and BeiDou for better coverage.
- Faster convergence time (GPS time to fix).
- More reliable / robust performance.
- ~2x reduction in average position error (CEP).
- Centimeter-level RTK position accuracy.
- Small and energy efficient module.
- Easy integration of RTK for fast time-to-market.

10.1.2 Overview

The IMX (GPS-INS) can interface with external multi-band (multi-frequency) GNSS receiver(s) to improve the precision of the EKF solution. The supported message protocols are uBlox binary, Septentrio binary and NMEA. The following are the GPS settings (accessible in the EvalTool GPS Settings tab and IMX `DID_FLASH_CONFIG.ioConfig` and `DID_FLASH_CONFIG.RTKCfgBits`):

Setting	Value
GPS Source	Serial port of the GNSS (serial 0 or 1)
GPS Type	GNSS model or protocol (ublox M8, ublox F9, Septentrio, Inertial Sense, or NMEA)
GPS RTK	*Position* for GPX-1 L1/L5 RTK precision positioning *Compass* for GPX-1 L1/L5 RTK Dual GNSS heading *F9 Position* for ZED-F9P multi-frequency RTK precision positioning *F9 Compass* for ZED-F9P multi-frequency Dual GNSS heading
GPS1 Timepulse	Source of the GNSS PPS time synchronization, uBlox GPS type only.

Refer to the Hardware section of this manual for serial port pinout information.

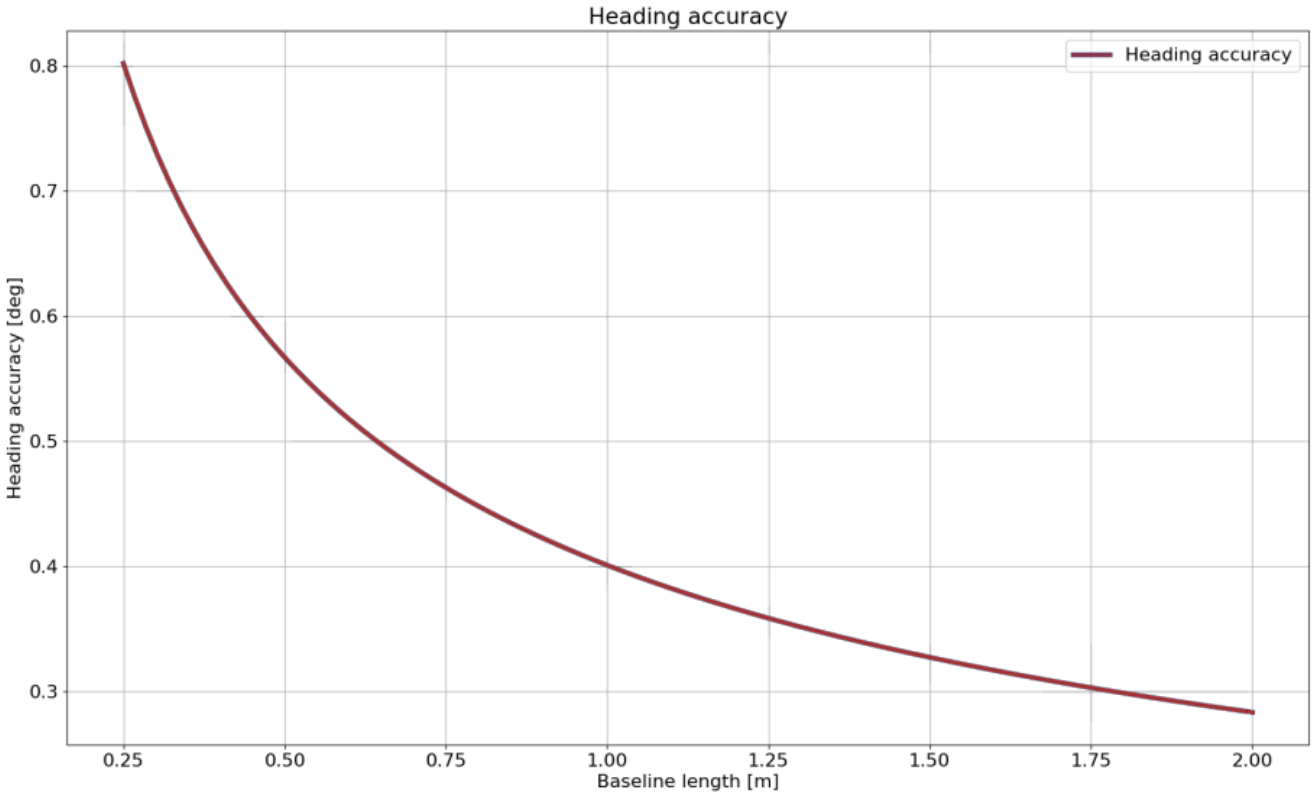
DUAL GNSS HEADING ACCURACY

When operating a system equipped with **dual multi-band GNSS receivers**—such as the **RUG-3-IMX-5-DUAL**—in moving-baseline RTK mode (RTK compassing), the resulting **baseline error** is determined by two primary components:

1. Receiver measurement noise, and
2. RTK solution error, which reflects satellite geometry, signal quality, and residual correction errors.

These error sources combine to define the achievable accuracy of the baseline vector and, consequently, the system's **heading accuracy**. Under ideal GNSS conditions, the expected heading performance is illustrated in the following plot.

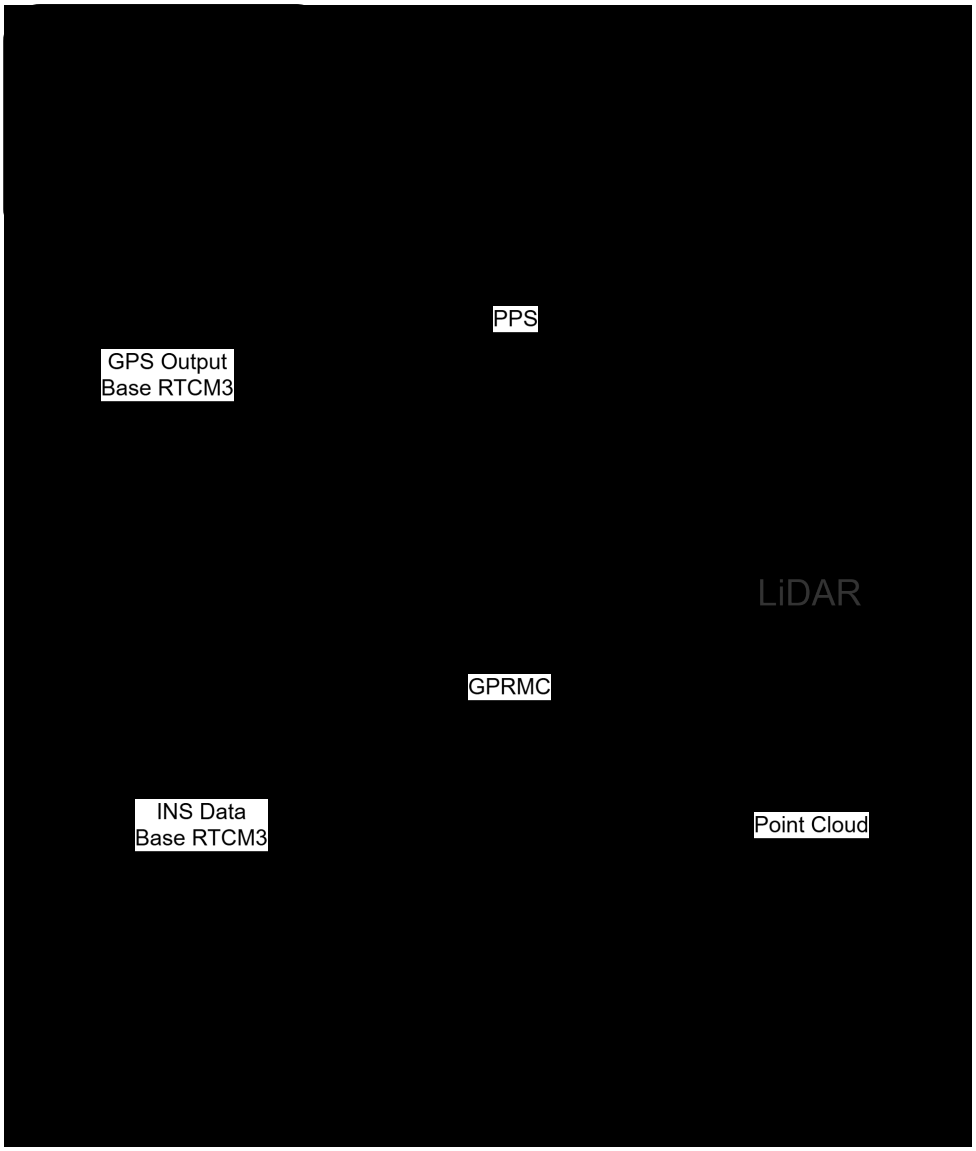
©2022



SINGLE GNSS RTK POSITIONING W/ LIDAR

RTK base messages (RTCM3) supplied to any of the IMX serial ports are forwarded to the GPX-1 for RTK positioning. The RTK precision position is used in the IMX EKF solution. The IMX can be configured to output NMEA messages such as GPGGA or GPRMC on any serial port.

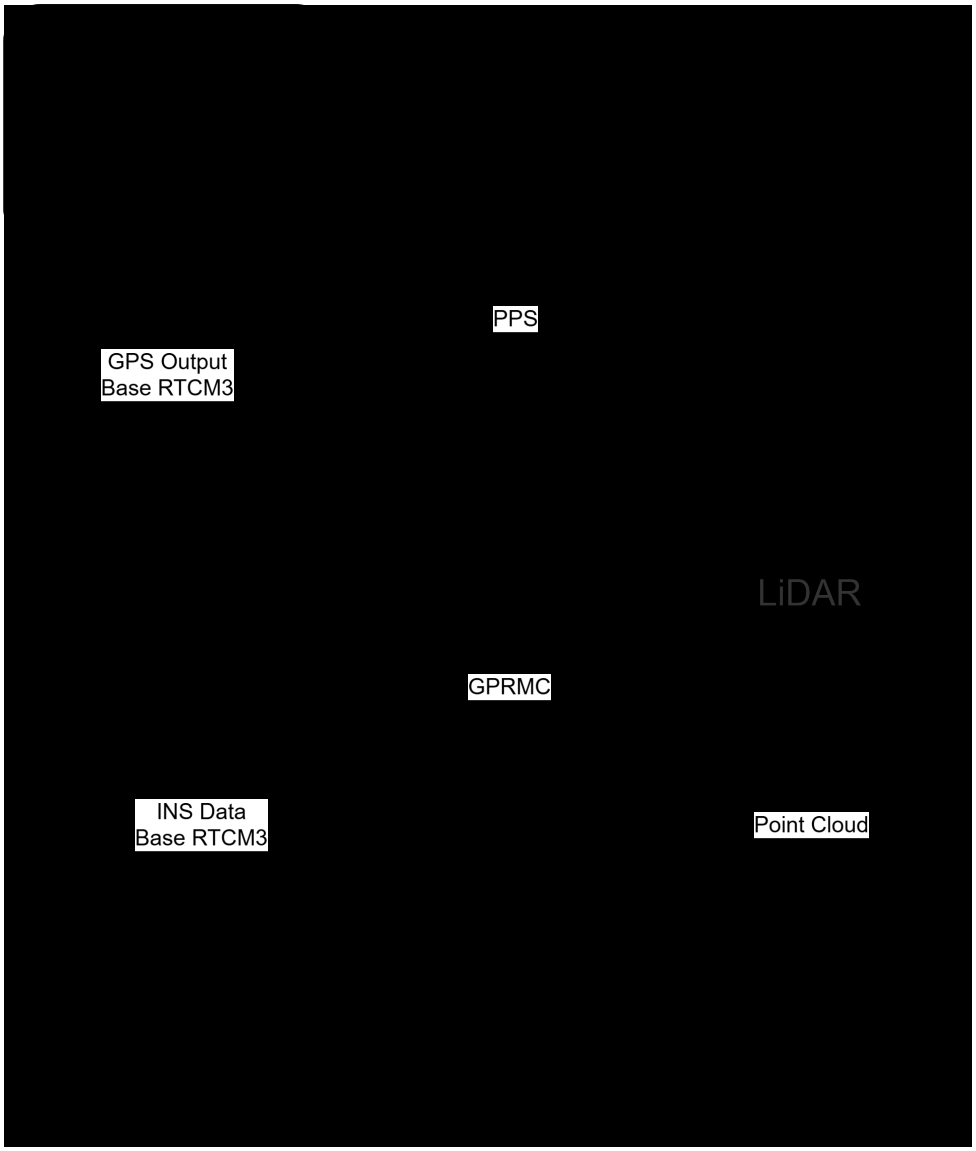
©2022



DUAL GNSS RTK POSITIONING AND RTK COMPASSING

RTK base messages (RTCM3) supplied to any of the IMX serial ports are forwarded to GPS1 for RTK positioning. RTK moving base messages from GPS1 are forwarded to GPS2 for RTK compassing. The RTK precision position from GPS 1 and the RTK compassing heading from GPS2 are used in the IMX EKF solution.

©2022


PPS
GPS Output
Base RTCM3
LiDAR
GPRMC
INS Data
Base RTCM3
Point Cloud

©2022

10.2 GNSS Antennas

10.2.1 Selecting a GNSS Antenna

- Using a passive GNSS antenna is possible but not recommended. This requires a high RHCP antenna gain, good view of the sky, and a short matched/tuned 50 Ω input impedance line. This option may be appropriate to minimize BOM costs.
- Best performance is achieved by using an active antenna with integrated LNA. The LNA gain must be >17dB for standard GPS-INS use.
- For RTK and dual antenna (GPS compassing) use, the following characteristics are recommended: gain >26dB, multipath signal rejection, better signal to noise ratio, and improved carrier phase linearity.
- Antennas with integrated SAW filter may be necessary to reject interference from near frequencies or harmonic signals, such as wireless and LTE.
- For RTK and dual antenna applications we recommend dual feed (dual element) GNSS antennas

10.2.2 GNSS Antenna Integration Considerations

GNSS Antenna Ground Plane

A GNSS antenna ground plane blocks multipath signals, creating a shadow area for the antenna to hide in. The ground plane is acting as an RF blocking device. It is made of any material that attenuates (or totally blocks or reflects) RF signals. It creates a shadow area for the antenna to hide in. That shadow is a cone above the ground plane. Any signals that come down from the satellites and are bouncing back upward from the earth can’t get to the antenna. Only signals coming directly from above can get to the antenna. The distance of the physical antenna above the ground plane changes the shape of the RF blocked shadow area.

The signal gain on some antennas can be improved by increasing the ground plane size up to a given size. Beyond that given size the antenna gain is not affected much.

A ground plane width of 8 to 12 cm is typically large enough for most applications.

HELPFUL LINKS:

u-Blox: RF design considerations for GNSS receivers Application Note

Taoglas: GPS Patch Integration Application Note

electronics.stackexchange.com: How big a ground plane does a GPS antenna need?

©2022

10.3 GNSS Satellite Constellations

The IMX supports onboard M8 and external (off-board) uBlox GNSS receivers. These receivers use multiple GNSS constellations in the global positioning solution.

The M8 receiver supports use of 3 concurrent constellations and the ZED-F9 receivers support 4 concurrent constellations (i.e. GPS, GLONASS, Galileo, and BeiDou).

The GPX-1 module supports 4 concurrent constellations.

10.3.1 Constellation Selection

The satellite constellations can be enabled or disabled by setting the corresponding enable bits in `DID_FLASH_CONFIG.gnssSatSigConst` as defined by eGnssSatSigConst in data_sets.h. The following are commonly used and recommended configuration groups.

```
// 3 constellations is supported by IMX onboard M8 receiver.
// (SBAS is not considered a constellation)
DID_FLASH_CONFIG.gnssSatSigConst = 0x133F   // GPS/QZSS, Galileo, GLONASS, SBAS
DID_FLASH_CONFIG.gnssSatSigConst = 0x10FF   // GPS/QZSS, Galileo, BeiDou, SBAS
DID_FLASH_CONFIG.gnssSatSigConst = 0x130F   // GPS/QZSS, GLONASS, SBAS

// 4 constellations is supported by ZED-F9 receiver and the GPX-1 (not IMX onboard M8 receiver).
DID_FLASH_CONFIG.gnssSatSigConst = 0x13FF   // GPS/QZSS, Galileo, GLONASS, BeiDou, SBAS
```


©2022

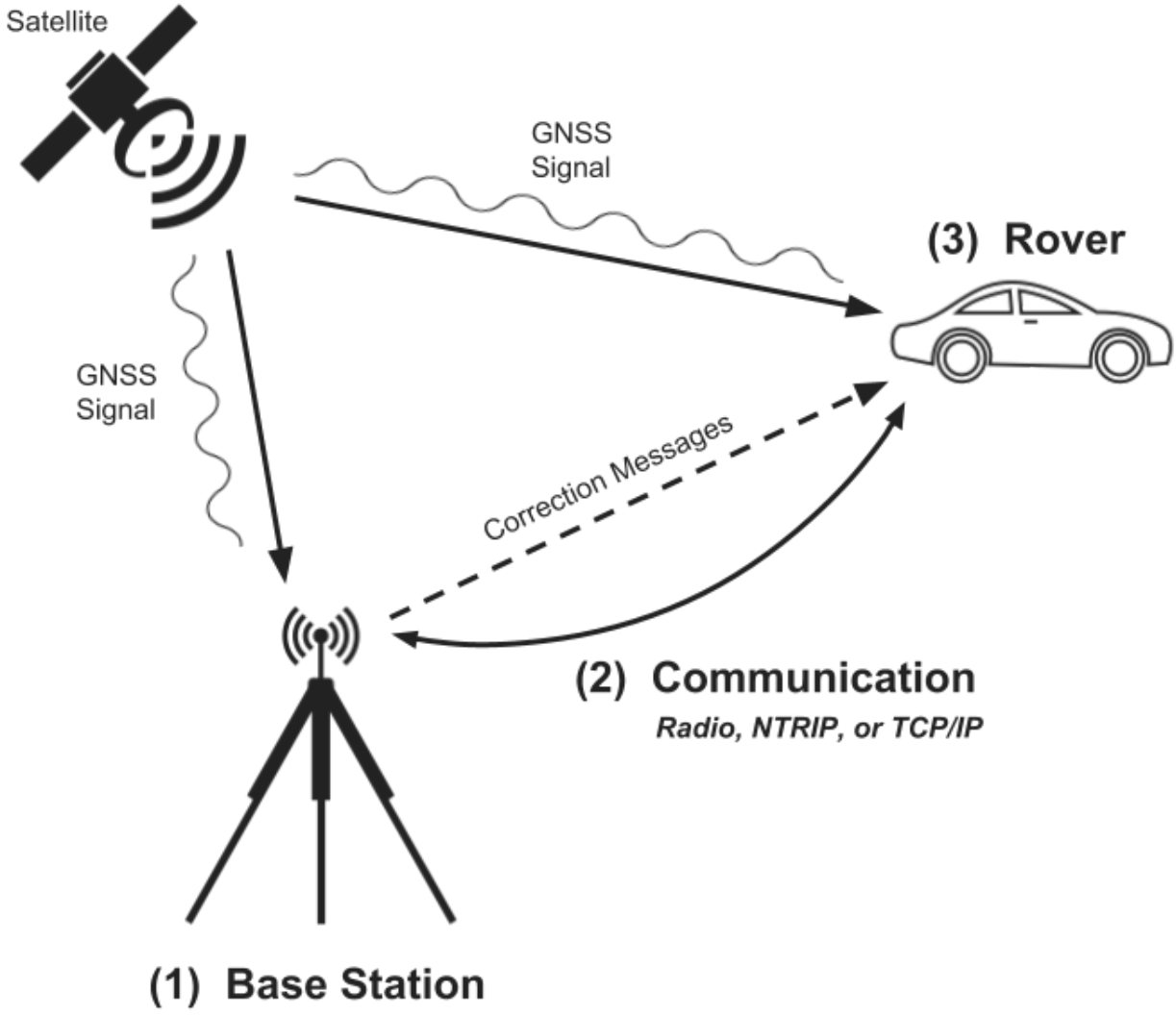
10.4 RTK Positioning

10.4.1 RTK Precision Positioning

Overview

Real Time Kinematic (RTK) is a precision satellite positioning technique which utilizes a base station to transmit position corrections to a receiver. The Inertial Sense RTK solution provides centimeter level position accuracy.

To use RTK, a base station, a rover (receiver), and a method to send corrections from the base to the rover are required.



See the multi-band GNSS section for details on using our multi-frequency ZED-F9 GNSS system.

©2022

RTK Hardware Setup

BASE STATION OPTIONS

Any of the following devices can be used as a RTK base station. All Inertial Sense base station options require a GPS antenna.

- **Inertial Sense IMX module, EVB-1, or Rugged** - Sends corrections on either serial port or USB that can then be forwarded to a rover using a communication method of choice.
- **3rd Party Base Station** - e.g. Emlid Reach Receiver.
- **Public NTRIP Caster** - e.g. CORS Network.

ROVER OPTIONS

The following configurations can be used for the RTK rover:

- **Inertial Sense Rugged** - Can receive corrections via either serial port or USB.

BASE TO ROVER COMMUNICATION

1. **Direct Serial** - Using USB, RS232, RS422/485, or TTL to pass corrections from Base to Rover.
2. **Radio Link** - For example, Digi Xbee to send RTK corrections. Other communication methods such as Bluetooth may also work for the chosen application.
3. **NTRIP** - Transmits RTK correction data over the Internet. To receive messages with NTRIP, the user must supply a URL, port number, and mount point . Often a username and password are also required.
4. **TCP/IP** - A protocol for communicating directly between computers. In order to receive messages using TCP/IP, an address (IP Address or DNS) must be supplied to the Base where the corrections will be transmitted.

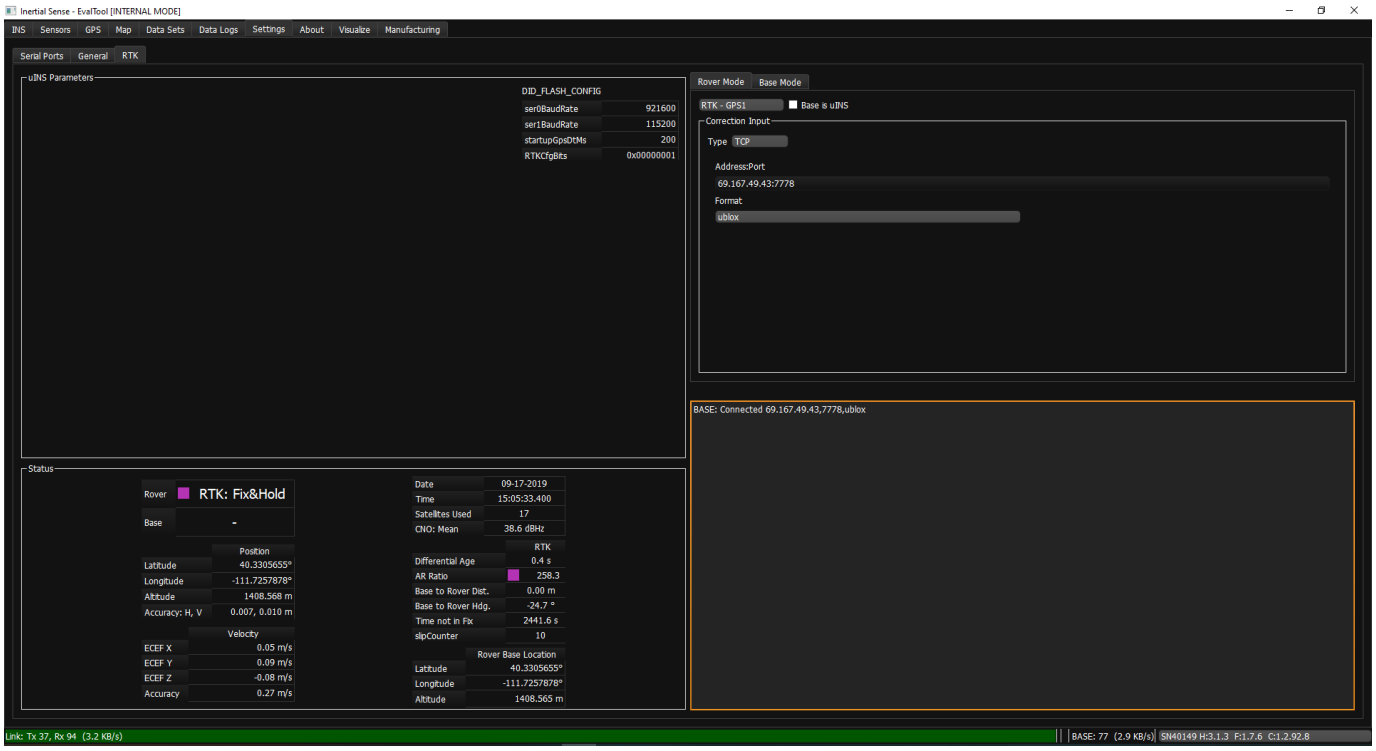
How to Know RTK is Working

USING THE EVALTOOL

1. Connect the IMX Rover to a computer with the EvalTool running. Open the comport for the unit in the Settings > Serial Ports.
2. Navigate to Settings > RTK.
3. Under the Status section, RTK functionality can be verified in 3 ways:

- Status field will show Single. Over the course of several minutes this status will change to Float then Fix.
- The Differential Age will show a timestamp that increments and resets back to zero about every second. This shows that the Rover is receiving Base messages.
- The Accuracy: H, V will show a large number at first. This number will decrease over time as the system acquires RTK Fix. Once in Fix, this number will average at +- 0.08, 0.14 m.

©2022



USING THE CLTOOL

1. Connect the IMX Rover to a computer with the CLTool running.
2. Include the argument -msgPresetPPD in the CLTool command.
3. Observe the DID_GPS_RTK_NAV message, Status: 0x******** (Single) over the course of several minutes this will change to (Float) then (Fix).

RTK Fix Status

LED INDICATORS

The LEDs on the IMX will indicate RTK fix status.

LED Behavior	Status	Description
	3D Fix, RTK Float	Allows improved accuracy up to ~1m
	RTK Fix	Allows increased accuracy up to ~3cm

RTK POSITIONING VALID FLAGS

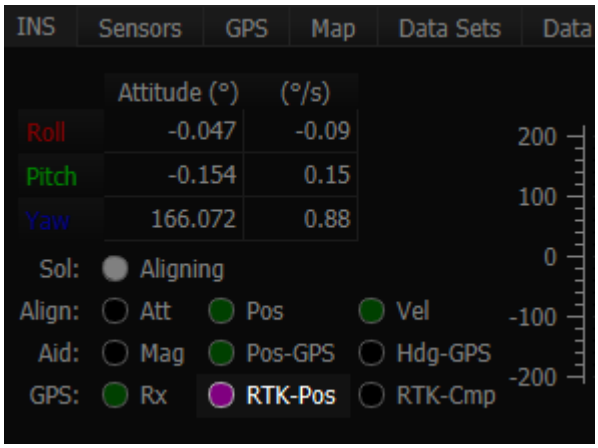
The RTK precision positioning fix status can be identified using the valid bit in the INS and GPS status flags.

```
// INS status
INS_STATUS_NAV_FIX_STATUS(DID_INS_1.insStatus) == GNSS_NAV_FIX_POSITIONING_RTK_FIX

// GPS status
DID_GNSS1_POS.status & GNSS_STATUS_FLAGS_GNSS1_RTK_POSITION_VALID
```

RTK precision positioning fix is indicated when the RTK-Pos radio button turns purple in the EvalTool INS tab.

©2022

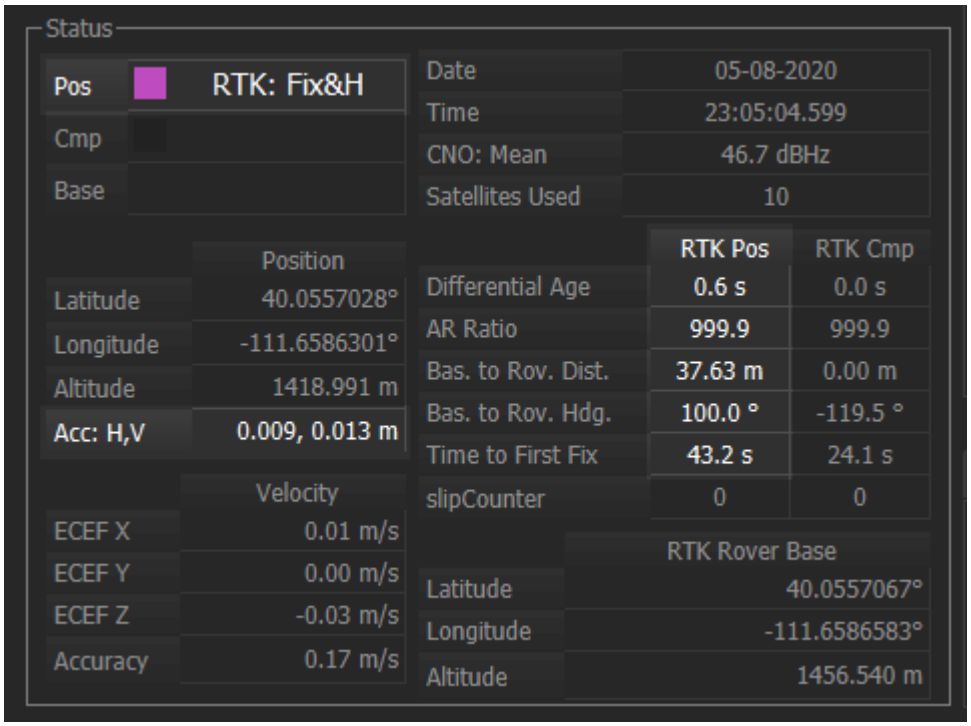


PROGRESS AND ACCURACY

The ambiguity resolution ratio, `arRatio`, is a metric that indicates progress of the solution that ranges from 0 to 999. Typically values above 3 indicate RTK fix progress.

```
DID_GNSS1_RTK_POS_REL.arRatio                    // Ambiguity resolution ratio
```

The DID_GNSS1_RTK_POS_REL status can be monitored in the EvalTool GPS tab.



RTK Base Messages

The IMX RTK solution accepts both RTCM3 and uBlox raw GNSS base correction messages. See the RTK Base or NTRIP pages for details on using base stations.

©2022

10.4.2 Rover Setup

System Configuration

The IMX must be configured as a Rover to receive RTK Base messages. This can be done through the EvalTool or the CLTool by enabling "Rover Mode".

EvalTool

1. Navigate to Settings > GPS > Rover > RTK.
2. Change the first drop-down menu to "Positioning (GPS1)", or one of the F9P options depending on the hardware setup.
3. Press Accept.
4. Verify the `RTKCfgBits` was automatically set correctly to any one of the rover modes listed in our binary communications protocol page.

CLTool

Use the `-flashConfig=rtkCfgBits=0x01` argument to configure the unit as rover where 0x01 can be any one of the rover modes listed in our binary communications protocol page.

Communications Setup

The IMX automatically parses data that arrives at any of the ports and recognizes base corrections data. Any communications method that sends the base corrections to one of the ports is suitable. Several common methods are described below.

EVB2 RADIO

EvalTool

The EVB-2 radio can be configured by pressing the "CONFIG" tactile switch until the light next to it is blue. This enables the radio and configures the radio settings. See the Configurations and EVB-2 Connections sections of the EVB-2 documentation.

1. Under "IMX Parameters" section verify the following:

- Check the Baud Rate for the serial port of the radio (`ser0BaudRate` or `ser1BaudRate`). This should match the Baud Rate of the radio. The Digi Xbee Pro SX module on the EVB2 runs at **115200** baud.

2. Navigate to Data Sets > `DID_EVB_FLASH_CFG`

- Change `cbPreset` - This should be set to `0x3` to enable the Digi Xbee Pro SX module.
- Change `radioPID` - Radio Preamble ID. Should be the same number used as the Base radio. (`0x0 to 0x9`)
- Change `radioNID` - Radio Network ID. Should be the same number used as the Base radio. (`0x0 to 0x7FFF`)
- Change `radioPowerLevel` - Used to adjust the radio output power level. (0=20dbm, 1=27dbm, and 2=30dbm)

3. Reset the EVB2 and Rover radio setup is complete.

For more information on `DID_EVB_FLASH_CFG` see DID-descriptions.

NTRIP CLIENT

For the Rover to receive messages from an NTRIP Caster, it must be connected to an interface with internet access (e.g. computer).

©2022

EvalTool

Follow these steps to set up the Rover to receive messages through NTRIP:

1. Navigate to Settings > RTK > Rover Mode.
2. Change the first drop-down menu to "RTK - GPS1"
3. Under Correction Input:

- Type = `NTRIP`
- Address:Port = : Ex: `rtgpsout.unavco.org:2101`
- Username/Password = Enter the Username and Password to the account used as the NTRIP Caster. Some Casters do not require this field.
- Format = RTCM3 or UBLOX
- Mount Point = Specify the mount point of the caster. Ex: `P016_RTCM3` 4. Press Apply.

CLTool

With the Rover IMX connected to the computer, use the -rover argument when running the CLTool executable:

- `-rover=TCP:` Set the type to "TCP".
- `PROTOCOL:` Set the protocol to "RTCM3" or "UBLOX". UBLOX requires more bandwidth and is not available from NTRIP casters.
- `URL:` The URL for the NTRIP Caster.
- `Port:` The port number will be provided by the NTRIP Caster.
- `MountPoint:` The mount point specifies which base station the corrections come from. This number will be provided by the NTRIP Caster.
- `Username:Password` The username and password for the account at the given URL (Not required by some public NTRIP casters).

Example:

```
cltool.exe -c COM10 -flashConfig=rtkCfgBits=0x01 -baud=57600 -rover=TCP:RTCM3:rtgpsout.unavco.org:2101:P016_RTCM3:username:password
```

TCP/IP

For the Rover to receive messages from a Base Station on a local network, it must be connected to an interface with network access (e.g. computer).

EvalTool

Follow these steps:

1. Navigate to Settings > GPS1
2. Under Correction Input:

- Type = `TCP`
- Address:Port = : e.g. `192.168.1.145:2001`
- Change Format to "ublox" or "RTCM3". Ublox requires more bandwidth but will result in better performance.

3. Press Accept.

> **Hint**
>
> For **serial ports**, view available comport numbers in the Settings tab of the EvalTool.

©2022

CLTool

With the IMX Rover connected to the computer, enter the -rover argument when running the CLTool executable:

- `-rover=TCP:` Set the type to "TCP".
- `RTCM3` Set the message type to "RTCM3" or "UBLOX". UBLOX requires more bandwidth and may be unavailable from some NTRIP Casters.
- `IP_Address` The IP Address of the Base Station to receive messages from.
- `Port` You may choose any number here. This should match the port number used for the Base Station.

Example: `cltool.exe -c COM10 -flashConfig=rtkCfgBits=0x01 -baud=57600 -rover=RTCM3:100.100.1.100:7777`

EVB2 WIFI

Using the EVB2 WiFi module to connect to the TCP/IP Base. EVB2 can save up to 3 Networks information. (Wifi[0], Wifi[1], Wifi[2]) Follow these steps using the EvalTool:

1. Under "IMX Parameters" section verify the following:
- Verify the `RTKCfgBits` was automatically set to 0x00000001
2. Navigate to Data Sets > `DID_EVB_FLASH_CFG`
- Change `cbPreset` - This should be set to `0x4` to enable the WiFi module.
- Change `wifi[0].ssid` - WiFi [0] Service Set Identifier or network name.
- Change `wifi[0].psk` - WiFi [0] Pre-Shared Key authentication or network password.
- Change `server[0].ipAddr` - server [0] IP address.
- Change `server[0].port` - server [0] port.
3. Reset the EVB2 and Rover WiFi setup is complete.

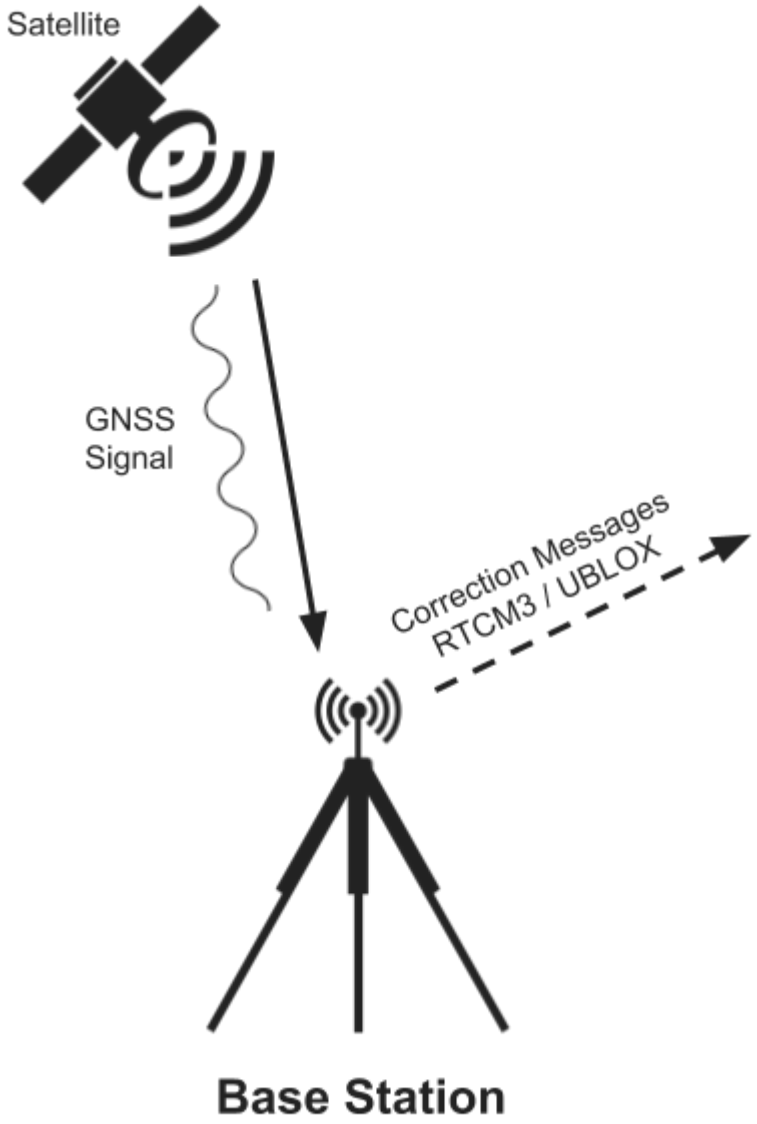
©2022

10.4.3 Base Setup

RTK Base Configuration

> **Note**
>
> If using an **NTRIP** service or **3rd Party Base Station** instead of your own base station, please skip this page and see the **NTRIP** page or reference the setup instructions for the 3rd Party Base Station. **NTRIP services** do not require additional setup.

An essential part of an RTK system is the Base Station which supplies correction messages from a known, surveyed location to the RTK Rover. The IMX Rover supports receiving RTCM3 and UBLOX correction messages.



Surveying In Base Position

> **Important**

©2022

Accuracy of the base position directly affects the rover absolute position accuracy. It is critical that the base position be surveyed in for rover absolute position accuracy.

The base survey cannot happen at the same time as base correction output messages are enabled. If a survey is started the base correction output will automatically be disabled.

The base position is stored in `DID_FLASH_CONFIG.refLla` and transmitted to the rover during RTK operation. The following steps outline how to survey in the base position.

1. **Mount base station in fixed location** - The location should not change during or following a survey.
2. **Set survey-in parameters** - This step can either be done using the EvalTool or programmatically using the data set (`DID_SURVEY_IN`).

EvalTool

1. Navigate to Settings > RTK > Base Mode.
2. In the "Survey In" section select one of the States:

- **Manual**: Direct entry of base position.
- **Average GPS 3D** - Requires standard GPS 3D lock (non-RTK mode) for survey.*
- **Average RTK Float** - Requires RTK float state for survey.*
- **Average RTK Fix** - Requires RTK fix for survey.

*The average methods will not run if the minimum requirements are not met. The system will wait until the requirements are met and then begin the survey.

3. Use the slider to select the Survey In runtime. Generally the longer the survey runs the more accurate the results will be.
4. Press the Start button.

> **Note**
>
> The current estimate of the survey is listed in the Position area above the Survey In section. If the survey completes successfully the results stored in flash memory (`DID_FLASH_CONFIG.refLla`) which will only change if the survey is re-run.

Using DID_SURVEY_IN

1. The location of the base can be manually entered using (`DID_FLASH_CONFIG.refLla`) if location is known.
2. Set `DID_SURVEY_IN.maxDurationSec` - Maximum time in seconds the survey will run. This is ignored if it is set to 0.
3. Set `DID_SURVEY_IN.minAccuracy` - Minimum horizontal accuracy in meters for survey to complete before maxDuration. This is ignored if it is set to 0.
4. Set (`DID_SURVEY_IN.state`) to begin the survey according to the desired survey State:

- **2 = Average GPS 3D** - Requires standard GPS 3D lock (non-RTK mode) or better for survey.*
- **3 = Average RTK Float** - Requires RTK float fix or better for survey.*
- **4 = Average RTK Fix** - Requires RTK fix for survey.

*The average methods will not run if the minimum requirements are not met. The system will wait until the requirements are met and then begin the survey.

Communications Setup

RADIO

The Base IMX must be configured to stream base corrections to the radio so it can be broadcast to the rover.

EvalTool

1. Open the COM port for the IMX under Settings > Serial Ports.

©2022

2. Navigate to Settings > RTK > Base Mode.
3. Under "Correction Output", find the fields for serial ports 0, 1, or USB. Select the serial port from which the corrections will be transmitted. This port must also be connected to the radio. Choose one of the options listed below. Leave the unused serial port off.

- "GPS1 - RTCM3": *Output standard RTCM3 messages.*
- "GPS1 - uBlox": *Output uBlox messages. This will provide more accuracy but requires significantly more bandwidth.*

4. Change the "Data Rate(ms)" field. This determines how many milliseconds pass between message outputs (e.g. Data Rate(ms) = 1,000 means one message/second). It is usually best to match the startupGnssDtMs value found in DID_FLASH_CONFIG.
5. In the "Position" section, the Base Station position is required so that it can transmit accurate corrections. Please refer to Surveying In Base Position if the base station location is unknown.
6. Click Apply, and reset the IMX. The unit will now start up in Base Station mode. Verify the base station is working by looking in the section labeled "Status". It will display the serial port of the radio and the message type. e.g. "SER1:UBX"
7. Navigate to Data Sets > `DID_EVB_FLASH_CFG`

- Change `cbPreset` - This should be set to `0x3` to enable the Digi Xbee Pro SX module.
- Change `radioPID` - Radio Preamble ID. Should be the same number used as the Rover radio. (`0x0 to 0x9`)
- Change `radioNID` - Radio Network ID. Should be the same number used as the Rover radio. (`0x0 to 0x7FFF`)
- Change `radioPowerLevel` - Used to adjust the radio output power level. (0 = 20dbm, 1 = 27dbm, and 2 = 30dbm)

8. Reset the EVB2 and Base radio setup is complete.

For more information on `DID_EVB_FLASH_CFG` see DID-descriptions.

CLTool

The RTK config bit must be set manually when using the CLTool. Use the following command line arguments when executing the CLTool from a prompt/terminal.

- `-c #` Open the COM port of the IMX. Windows users will use the name of the COM port, e.g. COM7. Linux users must enter the path to the correct COM port, e.g. /dev/ttyUSB0.
- `-baud=#` Set the baud rate for communications output (Replace # with baud rate number). This number will vary depending on setup. For lower quality radios it may be necessary to use a lower baud rate (ex: 57600).
- `-flashConfig=rtkCfgBits=0x00` Configure the unit to cast Base corrections. For more configuration options see eRTKConfigBits

Example:

```
cltool.exe -c COM29 -baud=57600 -flashConfig=rtkCfgBits=0x80
```

Warning

If the Base Station is not communicating properly, it may be necessary to verify that the baud rate is set to match that of the radios used. This rate varies depending on radio type.

TCP/IP SETUP

CLTool

It is required to manually set the RTK config bits in the CLTool. Pass these to the CLTool when run from the command prompt/ terminal.

- `-c #` Open the COM port of the IMX. Windows users will use the name of the COM port, e.g. COM7. Linux users must enter the path to the correct COM port, e.g. /dev/ttyUSB0.
- `-baud=#` Set the baud rate for communications output (Replace # with baud rate number).
- `` `-flashConfig=rtkCfgBits=0x00 `` Configure the unit to cast Base corrections. For more configuration options see eRTKConfigBits
- `-base=:#` Create the port over which corrections will be transmitted. Choose any unused port number.

©2022

Example:

```
cltool.exe -c COM29 -baud=921600 -flashConfig=rtkCfgBits=0x10 -base=:7777
```

Important

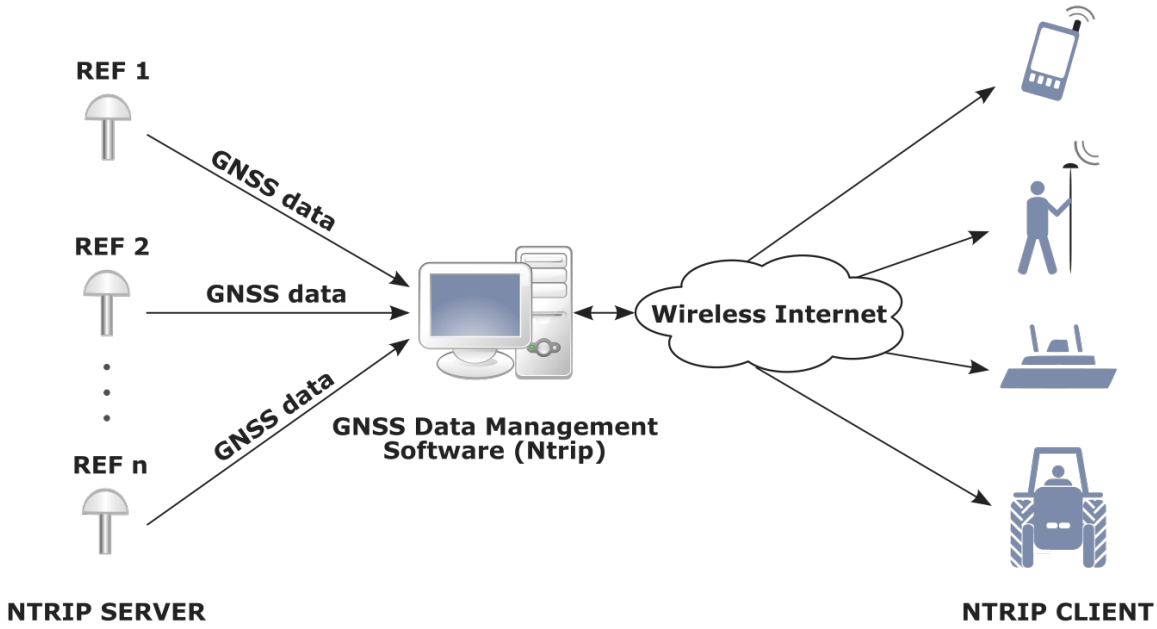
If the console displays the error **"Failed to open port at COMx"**, reset the device immediately after attempting to change the baud rate in the CLTool.

©2022

10.4.4 NTRIP

Networked Transport of RTCM via internet protocol, or NTRIP, is an open standard protocol for streaming differential data over the internet in accordance with specifications published by RTCM. There are three major parts to the NTRIP system: The NTRIP client, the NTRIP server, and the NTRIP caster:

1. The NTRIP server is a PC or on-board computer running NTRIP server software communicating directly with a GNSS reference station.
2. The NTRIP caster is an HTTP server which receives streaming RTCM data from one or more NTRIP servers and in turn streams the RTCM data to one or more NTRIP clients via the internet.
3. The NTRIP client receives streaming RTCM data from the NTRIP caster to apply as real-time corrections to a GNSS receiver.



The EvalTool/CLTool software applications provide NTRIP client functionality to be used with the IMX RTK rover. Typically an EvalTool NTRIP client connects over the internet to an NTRIP service provider. The EvalTool/CLTool NTRIP client then provides the RTCM 3.3 corrections to the IMX and ZED-F9P rover connected over USB or serial. Virtual reference service (VRS) is also supported by the EvalTool/CLTool NTRIP client.

Important

If using a **virtual reference service** (**VRS**), the rover must output the **NMEA GGA** message to return to the NTRIP caster. Without this, the NTRIP caster will not provide correction information.

NTRIP RTCM3 Messages

The NTRIP server must provide the necessary subset of RTCM3 messages supported by the IMX-RTK. The following is an example of compatible RTCM3 base output messages provided from a Trimble NTRIP RTK base station.

©2022

REQUIRED RTCM MESSAGES FOR RTK POSITIONING

Message Type	Description
RTCM 1005	Stationary RTK reference station ARP
RTCM 1074, 1075, or 1077	GPS MSM4, MSM5, or MSM7
RTCM 1084, 1085, or 1087	GLONASS MSM4, MSM5, or MSM7
RTCM 1094, 1095, or 1097	Galileo MSM4, MSM5, or MSM7
RTCM 1230	GLONASS code-phase biases

©2022

10.4.5 Using uBlox PointPerfect L-band Corrections

The IMX-5 can receive corrections from the uBlox D9S device, which provides L-band corrections through the uBlox PointPerfect solution. **PointPerfect IP-based corrections are currently not supported (L-band only)**

Firmware update

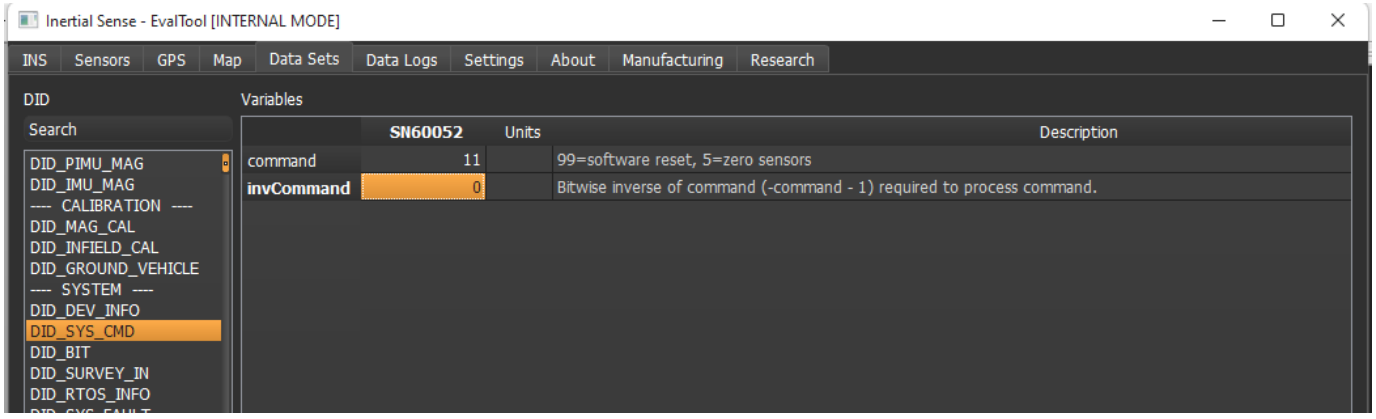
L-band corrections requires a later version of F9P firmware. Use FW version HPG 1.32 from the F9P downloads page on the uBlox site.

To check your firmware version, follow these steps:

1. Open the Inertial Sense EvalTool
2. Navigate to the Data Sets tab.
3. Select `DID_GPSx_VERSION`
4. The firmware version is displayed in extension[1]. For Example, FWVER=HPG 1.12

To update the firmware on the F9P, follow these steps:

1. Open the Inertial Sense EvalTool
2. Navigate to the Data Sets tab. Select `DID_SYS_CMD` from the sidebar (see the image below)
3. Set `command` to `12` and `invCommand` to `-13` to enable passthrough to GNSS1 (set `13` and `-14` for GNSS2)
4. Close the serial port (Settings tab)
5. Open the device in uBlox u-center (we currently use u-center 22.07)
6. Update the firmware in u-center per u-blox instructions. Baudrate should be set to 921600.



Configuration of the F9P

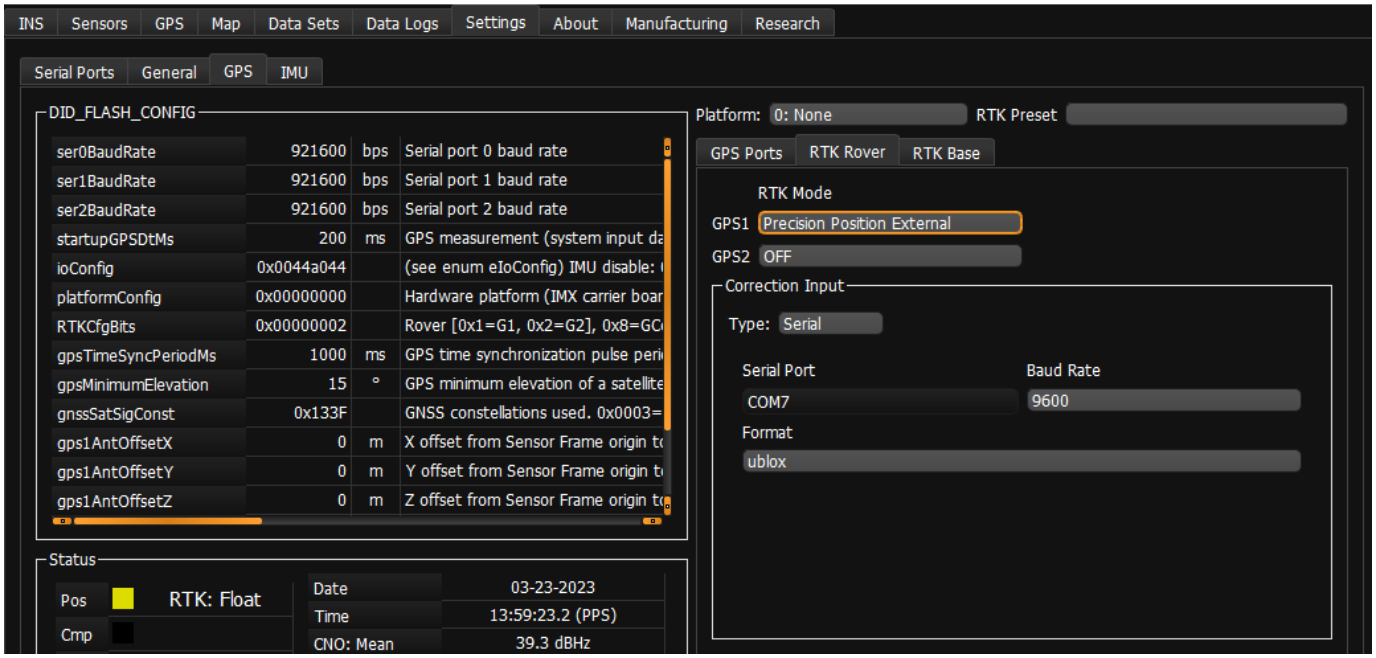
By default, the IMX-5 module updates settings on the F9P at bootup. Currently the IMX:

- Configures frequencies and constellations to use in the solution
- Changes the CFG-SPARTN setting to use L-band corrections. IP corrections are not currently supported through the IMX-5 due to a difference in the protocol.

The IMX acts as a transparent pipe for some UBX messages. In the mode shown below, the IMX passes the following messages:

- RXM-SPARTN (from F9P)
- RXM-SPARTNKEY (to/from F9P)
- RXM-PMP (to F9P)
- CFG-* (bidirectional)

©2022



For L-band, the messages are sent as UBX-RXM-PMP messages. No additional configuration on the IMX is needed to switch between these messages and RTCM3, either will work. We have not tested using them together, contact uBlox if you are considering this.

To set the SPARTNKEY message, use the u-center tool or send the proper message as specified in the HPG 1.32 manual to the IMX. SPARTNKEY messages are passed through the IMX.

If you need to access additional settings on the F9P, you can use the serial passthrough mode used for firmware update, or there are settings under the `RTK Base` tab that allow passthrough of all uBlox messages.

Corrections can be forwarded through the EvalTool using the `Correction Input` section of the tool. See the above screenshot for an example configuration.

©2022

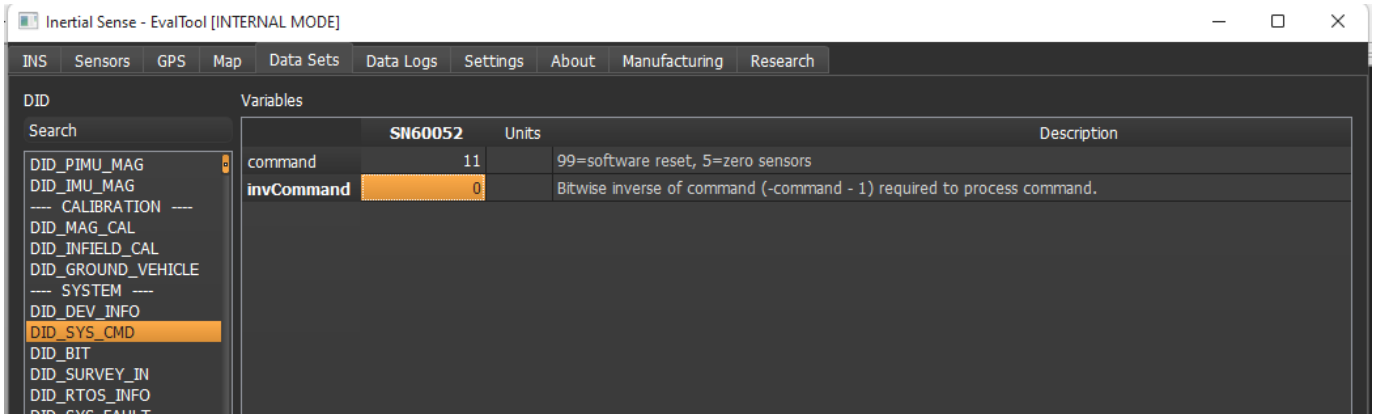
10.4.6 Using uBlox SBAS corrections

The uBlox F9P receivers can be configured to enable the SBAS corrections constelations

Firmware update

SBAS corrections require a later version of F9P firmware. Use FW version HPG 1.32 from the F9P downloads page on the uBlox site. To update the firmware on the F9P, follow these steps:

1. Open the Inertial Sense EvalTool
2. Navigate to the Data Sets tab. Open `DID_SYS_CMD` from the sidebar (see the image below)
3. Set `command` to `12` and `invCommand` to `-13` to enable passthrough to GNSS1 (set `13` and `-14` for GNSS2)
4. Close the serial port (Settings tab)
5. Open the device in uBlox u-center (u-center 22.07)
6. Update the firmware in u-center per u-blox instructions. Baudrate should be set to 921600.



Configuration of the F9P

With the uBlox 1.32 firmware installed on the F9P SBAS can be enabled using the standard constelation selection methods described in the GNSS Constelations page.

©2022

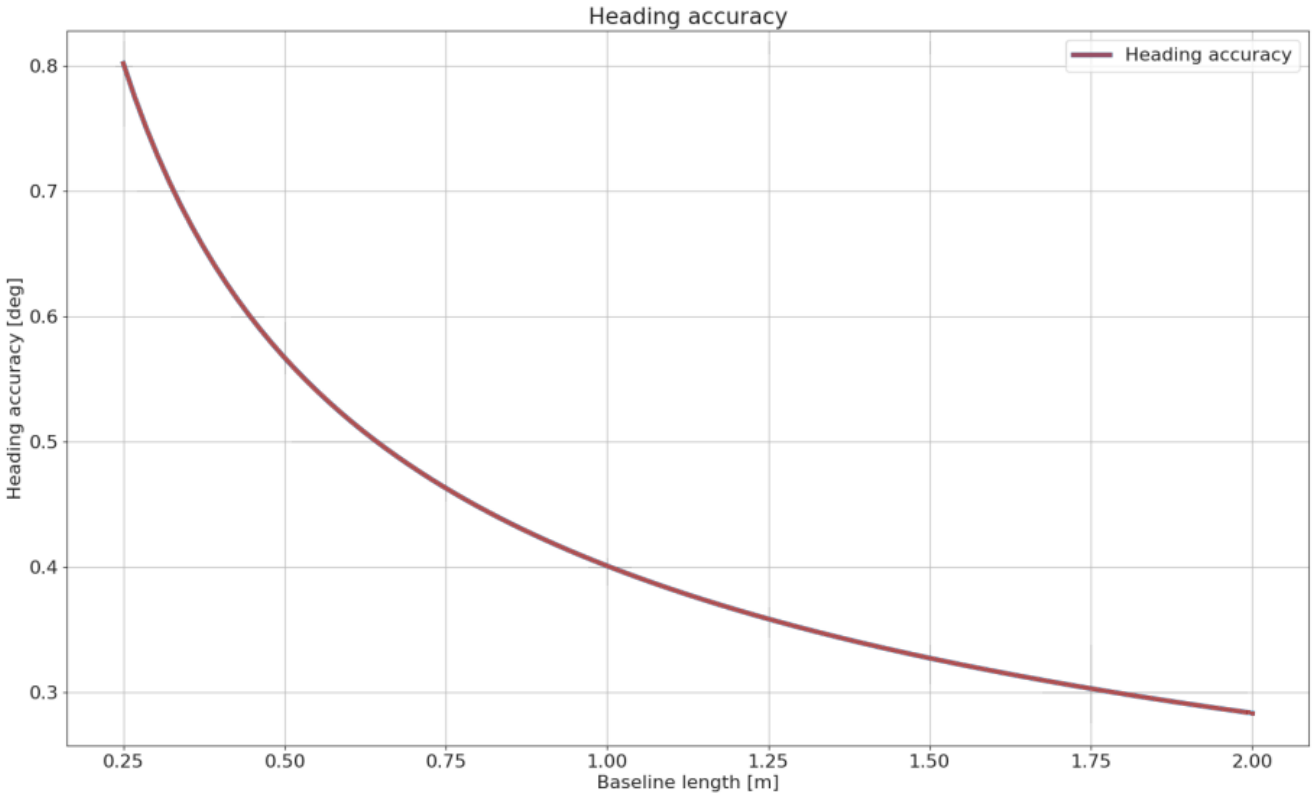
10.5 Dual GNSS RTK Compassing

10.5.1 Overview

RTK Compassing (Dual GNSS) is a system that determines heading by use of two GNSS receivers and antennas. It replaces the need for magnetometers which can be problematic in the presence of ferrous materials (e.g. steel) and EMI generating circuits (e.g. electric motors and drivers).

10.5.2 Heading Accuracy

The generalized heading accuracy under ideal conditions is shown in the following plot.



Recommended Minimum Baseline

The recommended minimum baseline (distance between dual GNSS antennas) is ***0.25 meters*** for multi-band GNSS compassing. The solution can operate at shorter baseline distances but is less robust and more susceptible to getting caught in a local minimum which may not converge to the correct heading.

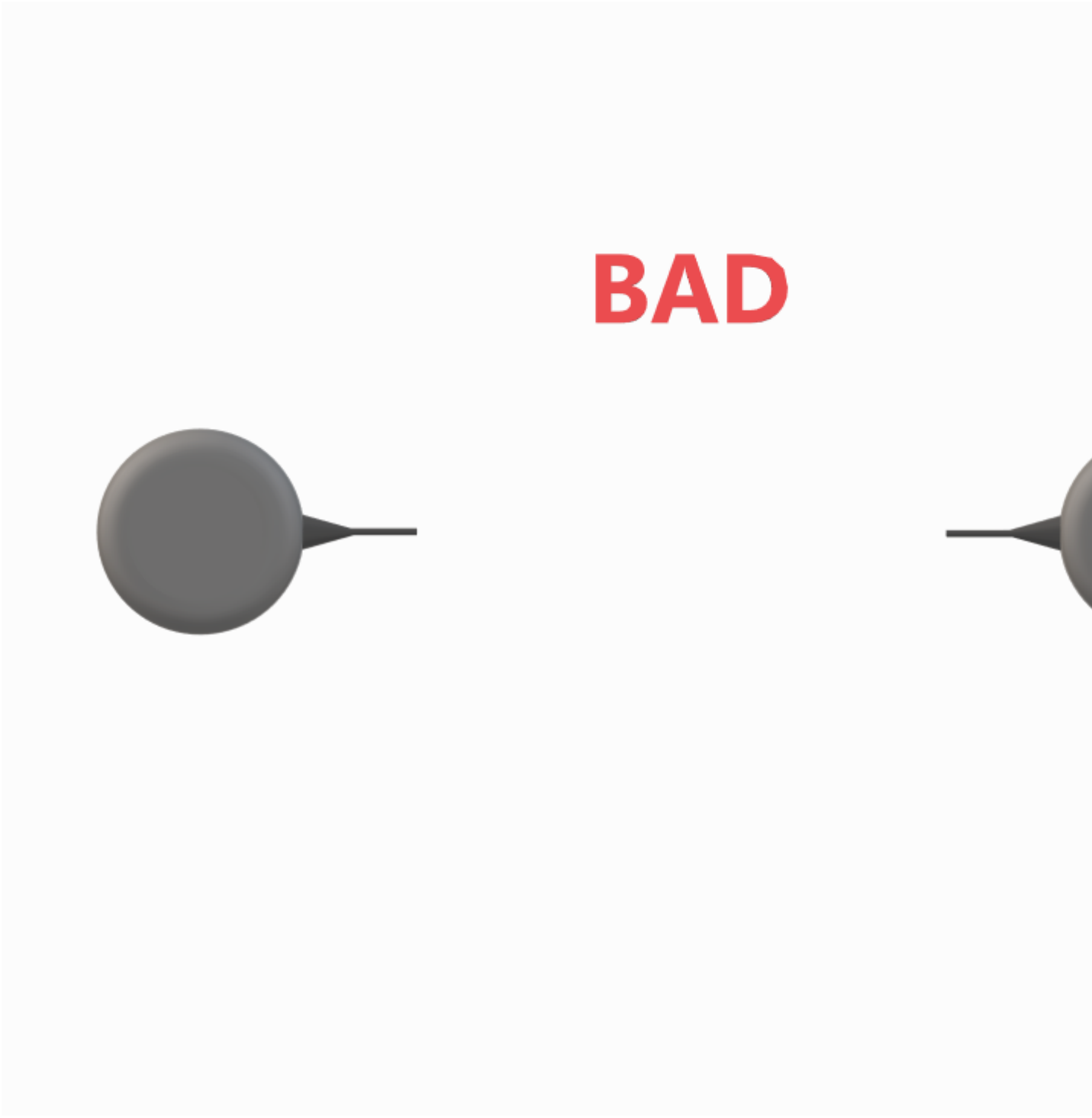
©2022

10.5.3 Antenna Orientation

Important

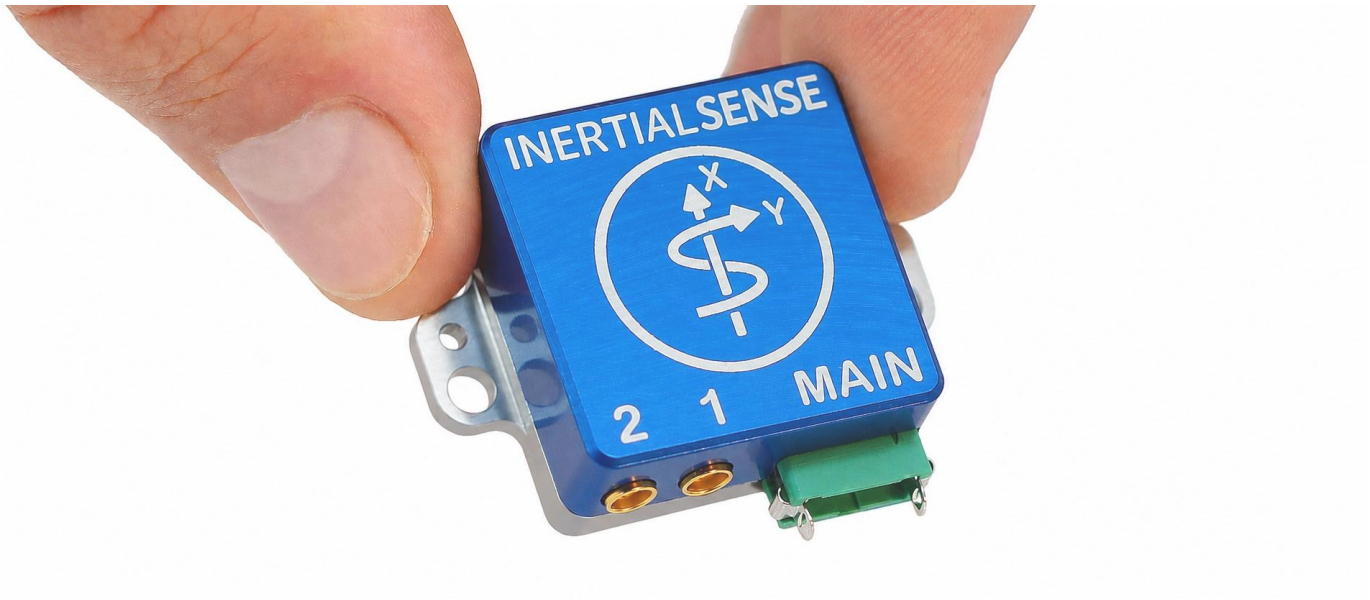
It is recommended that both GNSS antennas be identical and have the same physical orientation relative to each other (i.e. the antenna cable should exit in the same direction on both antennas). This will ensure best RF phase center alignment and heading accuracy. The actual RF phase center is often offset from the physical center of the antenna case.

Mismatch



©2022

10.5.4 Rugged GNSS Antenna Ports



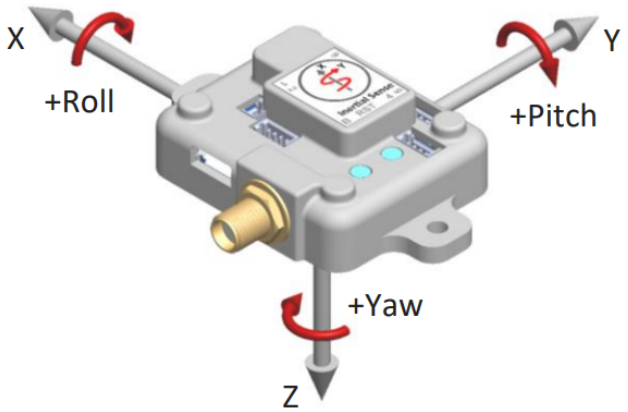
On the Rugged IMX, the MMCX ports labeled **1** and **2** are the ***GNSS1*** and ***GNSS2*** antenna inputs.

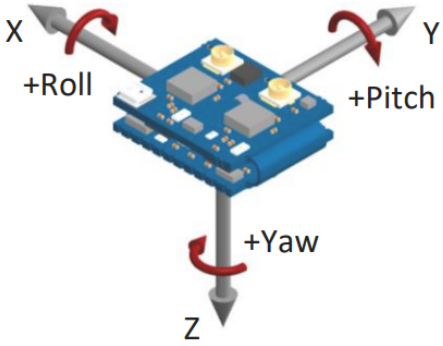
10.5.5 Dual Antenna Locations

The location for both GNSS antennas must be correctly specified by the user in the DID_FLASH_CONFIG variables within 1 cm accuracy:

```
DID_FLASH_CONFIG.gnss1AntOffset[X,Y,Z]
DID_FLASH_CONFIG.gnss2AntOffset[X,Y,Z]
```

These values describe the distance of each GNSS antenna from the IMX Sensor Frame origin in the direction of the Sensor Frame axes. The Sensor Frame is defined using DID_FLASH_CONFIG.sensorConfig.





Validate GNSS Antenna Offsets

To validate the GNSS antenna offset values, inspect the INS heading (`DID_INS_1.theta[2]`) and verify that it points in the forward direction of the sensor frame (and the platform). Note that the RTK compass heading reported in `DID_GNSS2_RTK_CMP_REL.baseToRoverHeading` always reflects the raw heading from the GNSS1 antenna to the GNSS2 antenna and does not change with the antenna offsets; the INS heading, however, incorporates the antenna offsets and orientation to rotate

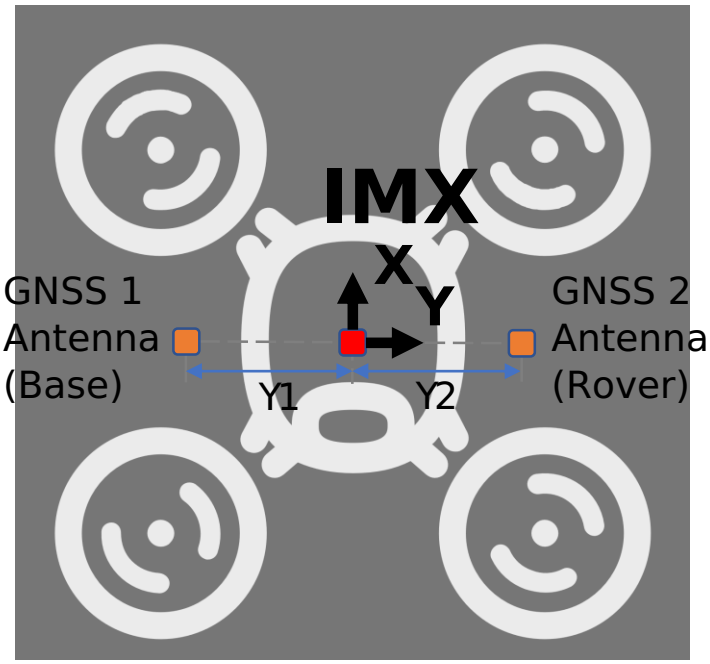
©2022

this RTK compass heading into the sensor frame. Because of this, the antenna offsets can be adjusted and re-validated immediately by simply watching the INS heading update.

Example Antenna Configurations

The following are examples that illustrate what the GNSS antenna offsets should be for two different antenna configurations.

DRONE

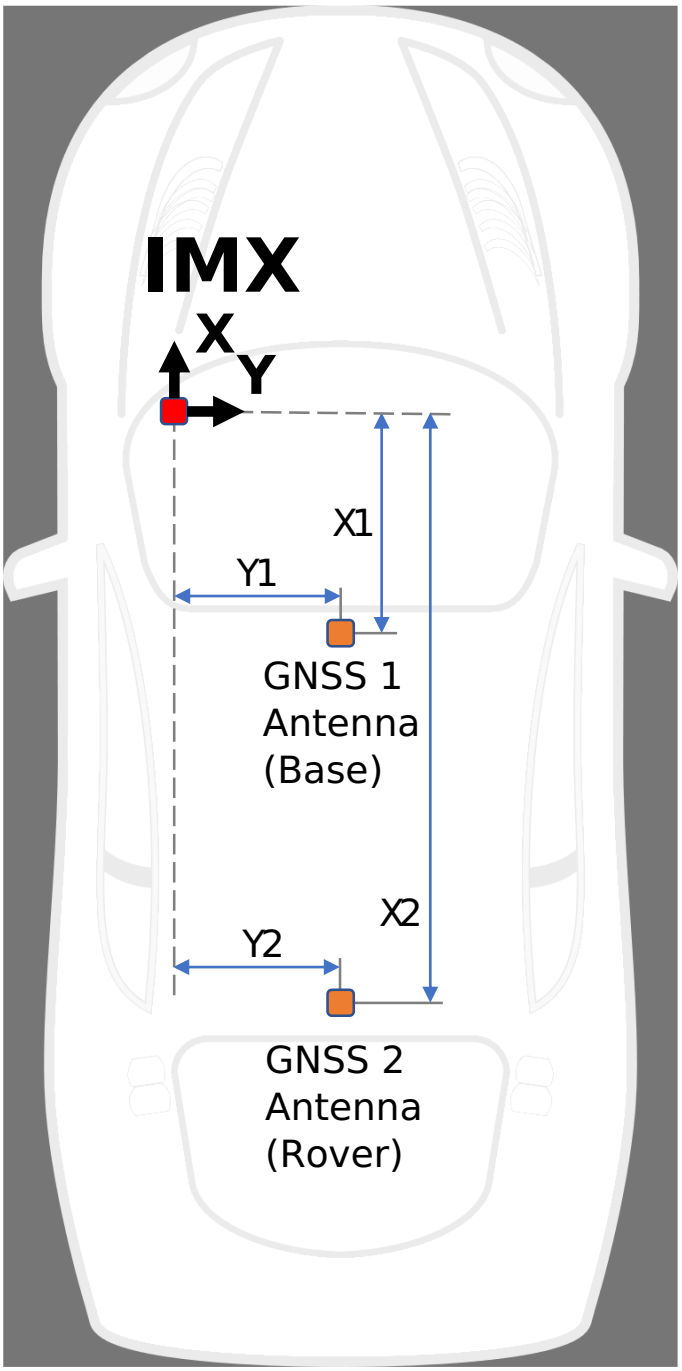



```
DID_FLASH_CONFIG.gnss1AntOffset[0] =  0.0
DID_FLASH_CONFIG.gnss2AntOffset[1] = -0.3   (negative direction of Y axis)
DID_FLASH_CONFIG.gnss2AntOffset[2] =  0.0

DID_FLASH_CONFIG.gnss2AntOffset[0] =  0.0
DID_FLASH_CONFIG.gnss2AntOffset[1] =  0.3
DID_FLASH_CONFIG.gnss2AntOffset[2] =  0.0
```


©2022

AUTOMOBILE




```
DID_FLASH_CONFIG.gnss1AntOffsetX = -0.5 (negative direction of X axis)
DID_FLASH_CONFIG.gnss1AntOffsetY =  0.5
DID_FLASH_CONFIG.gnss1AntOffsetZ = -0.5 (negative direction of Z axis, above IMX)

DID_FLASH_CONFIG.gnss2AntOffsetX = -1.5 (negative direction of X axis)
DID_FLASH_CONFIG.gnss2AntOffsetY =  0.5
DID_FLASH_CONFIG.gnss2AntOffsetZ = -0.5 (negative direction of Z axis, above IMX)
```


©2022

GNSS Antenna Ports

The following table explains how ports A and B on the Rugged IMX map to GNSS antennas 1 and 2.

Ports	Rugged IMX	IMX Module
GNSS 1 antenna port	A	1
GNSS 2 antenna port	B	2

10.5.6 Setup

Step 1 - Specify Offsets for Both Antennas

Refer to the Dual Antenna Locations section for a description of the GNSS antenna offset.

```
DID_FLASH_CONFIG.gnss1AntOffsetX = ?
DID_FLASH_CONFIG.gnss1AntOffsetY = ?
DID_FLASH_CONFIG.gnss1AntOffsetZ = ?

DID_FLASH_CONFIG.gnss2AntOffsetX = ?
DID_FLASH_CONFIG.gnss2AntOffsetY = ?
DID_FLASH_CONFIG.gnss2AntOffsetZ = ?
```

Using EvalTool - select `Data Sets -> DID_FLASH_CONFIG` and set `gnss1AntOffset[X,Y,Z]` and `gnss2AntOffset[X,Y,Z]` with the GNSS antenna offsets.

Using CLTool - run the CLTool using the following options replacing the `[OFFSET]` with the GNSS antenna offsets.

```
-flashconfig=gnss1AntOffsetX=[OFFSET]
-flashconfig=gnss1AntOffsetY=[OFFSET]
-flashconfig=gnss1AntOffsetZ=[OFFSET]
-flashconfig=gnss2AntOffsetX=[OFFSET]
-flashconfig=gnss2AntOffsetY=[OFFSET]
-flashconfig=gnss2AntOffsetZ=[OFFSET]
```

Step 2 - Enable GNSS Dual Antenna

Set the `RTK_CFG_BITS_ROVER_MODE_RTK_COMPASSING (0x00000004)` bit of RTKCfgBits.

```
DID_FLASH_CONFIG.RTKCfgBits |= RTK_CFG_BITS_ROVER_MODE_RTK_COMPASSING       // |= 0x00000004
```

Using EvalTool - go to `Settings -> RTK -> Rover Mode`, set the dropdown menu to `GNSS Compassing`, and press the `Apply` button.

Using CLTool - run the CLTool using the `-flashconfig=RTKCfgBits=0x4` option to enable GNSS Dual Antenna.

10.5.7 RTK Compassing Fix Status

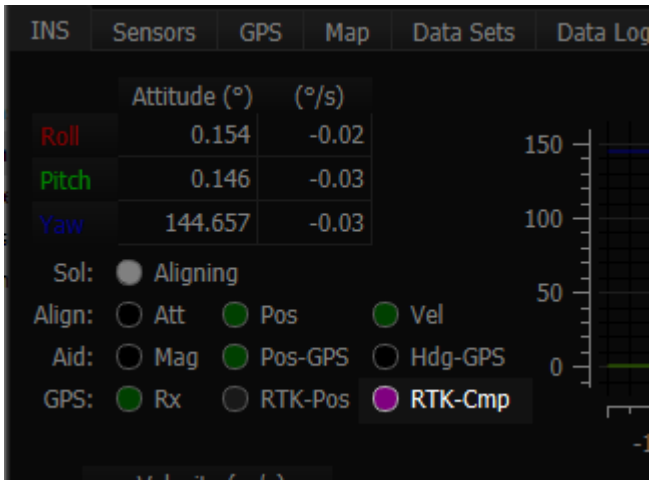
INS and GNSS Status Flags

The RTK compassing fix status can be identified using the valid bit in the INS and GNSS status flags.

```
DID_INS_1.insStatus & INS_STATUS_RTK_COMPASSING_VALID              // INS status - RTK heading is valid and aiding INS heading.
DID_GNSS1_POS.status & GNSS_STATUS_FLAGS_GNSS2_RTK_COMPASS_VALID       // GNSS status - RTK heading is valid and available in DID_GNSS2_RTK_CMP_REL.
```

RTK compassing fix is indicated when the RTK-Cmp radio button turns purple in the EvalTool INS tab.

©2022

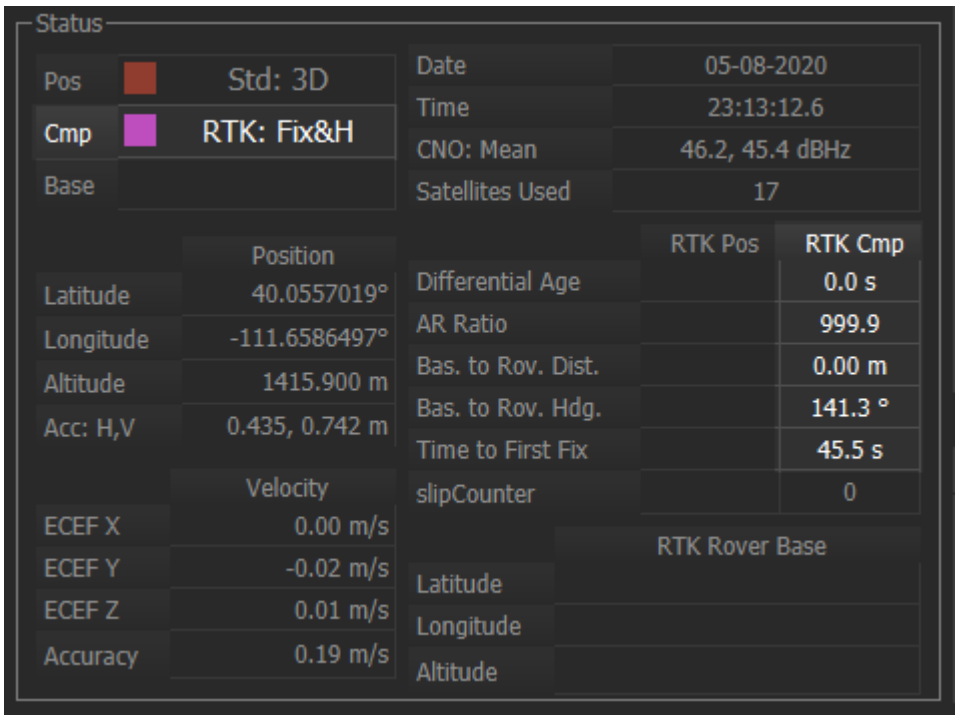


Progress and Accuracy

The base to rover heading accuracy indicates how much error is in the base to rover heading (RTK compassing heading). The ambiguity resolution ratio, `arRatio`, is a metric that indicates progress of the solution that ranges from 0 to 999. Typically values above 3 typically indicate RTK fix progress.

```
DID_GNSS2_RTK_CMP_REL.baseToRoverHeadingAcc        // (rad) RTK compassing accuracy
DID_GNSS2_RTK_CMP_REL.arRatio                      // Ambiguity resolution ratio
```

The DID_GNSS2_RTK_CMP_REL status can be monitored in the EvalTool GNSS tab.



10.5.8 Stationary Application

For RTK compassing stationary application, enabling the STATIONARY INS dynamic model (DID_FLASH_CONFIG.dynamicModel = 2) is recommended to reduce heading noise and drift. This will reduce heading error during RTK compassing fix or loss of fix. See INS-GNSS Dynamic Model and Zero Motion Command for details.

©2022

11. Data Logging/Plotting

11.1 Data Logging/Plotting

Inertial Sense provides a data logging capability in the EvalTool, CLTool, and SDK (C++) that can record data in binary, comma separated (.CSV), and KML file formats. This logging capability is useful for storing, replaying, and analyzing data.

11.1.1 Data Log Types

Comma Separated Values (`*.csv`)

The comma separated value (.csv) file format can be imported into many software packages, including Excel, Matlab, and Python.

KML (`*.kml`)

KML is a file format used to display geographic data in an Earth browser such as Google Earth.

Binary Data Log (`*.raw` and `*.dat`)

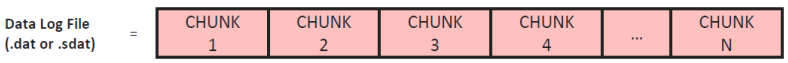
	Raw Logger (*.raw)	Serial Logger (*.dat)
Description	Data stored in the same byte for byte form as it appears over a serial port, without parsing and removing packet header/footer.	Stores data to file in the same serial order it was passed into the logger. This is the default logger used in the CLTool and EvalTool.
Advantages	Allows logging of all data/packet formats. Preserves all data in the original form as communicated over serial port. Can be logged by writing serial port data to file, no parsing needed.	Optimized for real-time data logging.
Source File	DeviceLogRaw.h / .cpp	DeviceLogSerial.h / .cpp
File extension	.raw	.dat

11.1.2 Binary Data Log Format

This section outlines the Inertial Sense binary data log types known as raw data and serial data (`.raw` and `.dat` file extensions). The .dat data log file type are composed of several data containers know as chunks. Each chunk contains a header, sub-header, and data.

File

The data log file name has the format *LOG_SNXXXXX_YYYYMMDD_HHMMSS_CNT.dat* which contains the device serial number, date, time, and log file count. The serial data log file format is `.dat`. This log consists of files containing a series of data Chunks.



©2022

Standard data types are stored in the log files and are defined as:

U32	unsigned int
U16	unsigned short
S8	char
U8	unsigned char

Chunk

The data log file is composed of Chunks. A Chunk is a data container that provides an efficient method for organizing, handling, and parsing data in a file. A Chunk starts with a header which has a unique identifiable marker and ends with the data to be stored.

CHUNK	=	CHUNK Header	CHUNK Data
Byte Size		40	

Chunk Header

The header, found at the start of each Chunk, is as follows:

CHUNK Header	=	Marker	Version	Classification	Name	Name Inverted	Data Size	Data Size Inverted	Group Number	Device Serial #	Port Handle	Reserved
Byte Size		4	2	2	4	4	4	4	4	4	4	4
Data Type		U32	U16	U16	S8 (x4)	S8 (x4)	U32	U32	U32	U32	U32	U32

The C structure implementation of the Chunk header is:

```
//!< Chunk Header
#pragma pack(push,1)
struct sChunkHeader
{
uint32_t marker; //!< Chunk marker (0xFC05EA32)
uint8_t version; //!< Chunk Version
uint8_t dataOffset; //!< offset from this position until the start of the chunk data
char protocolVersion[2]; //!< major/minor version of the underlying line/protocol
char name[4]; //!< Chunk name
char invName[4]; //!< Bitwise inverse of chunk name
uint32_t dataSize; //!< Chunk data length in bytes
uint32_t invDataSize; //!< Bitwise inverse of chunk data length
uint32_t grpNum; //!< Chunk Group Number: 0 = serial data, 1 = sorted data...
uint32_t devSerialNum; //!< Device serial number
uint16_t portId; //!< Device port id
uint16_t portType; //!< Device port type
char fwVersion[4]; //!< Device firmware version
};
#pragma pack(pop)
```

Chunk Data

The Chunk data is defined for the `.dat` file types.

CHUNK Data (.dat file)	=	DATA SET Header	DATA SET Data	DATA SET Header	DATA SET Data	...	DATA SET Header	DATA SET Data
Byte Size		12		12			12	

CHUNK Data (.sdat file)	=	CHUNK Sub-Header	DATA SET Serial #	DATA SET Data	DATA SET Serial #	DATA SET Data	...	DATA SET Serial #	DATA SET Data
Byte Size		16	4		4			4	
Data Type			U32		U32			U32	

©2022

Data Set Header

The Data set header is used for the `.dat` file types.

DATA SET Header	=	DATA SET ID	DATA SET Size	DATA SET Offset
Byte Size		4	4	4
		U32	U32	U32

©2022

11.2 Logging

The SDK logging interface is defined in SDK/src/ISLogger.h. Data logs can be converted between file formats using the Inertial Sense data logger. The logging interface is used in the Inertial Sense software described below.

11.2.1 Logging using Inertial Sense software

EvalTool

1. Go to **“Data Logs”** tab in EvalTool.
2. Select the **“Format”** file type from the drop-down menu.
3. Select the data to record within the Data Streams section of the **"Data Logs**" tab:

a. **Manual Selection** – Allows the user to select the specific datasets to stream and their update rates by setting the checkbox and period multiple in Manual Selection table.

b. **INS** – Log INS output (attitude, velocity, position) at 100 Hz by selecting "**INS**" from the RMC Presets dropdown.

c. **Post Process** – Used for beta testing and internal testing. Includes IMU, GPS, INS and other messages. Log by selecting "**PPD**" from the RMC Presets dropdown.

4. Press **“Enable”** to begin logging data.
5. Press **“Disable”** to stop logging data.
6. The **“Open Folder”** button opens the File Explorer location to the data logs, i.e. `C:\Users\[username]\Documents\Inertial_Sense\logs`.
7. To change the root log folder in the Eval Tool, edit `Documents/Inertial Sense/settings.json`, and add or change the logger key: "Directory": "FOLDER_FOR_LOGS".

CLTool

The CLTool, provided in the SDK, is a command line application that can record post process data. The CLTool help menu is displayed using the option `-h`. See the CLTool section for more information on using the CLTool.

11.2.2 Post Process Data (PPD) Logging Instructions

Post process data (PPD) logs include both the input to and output from the navigation filter. The data is used for analyzing, troubleshooting, and improving system performance. PPD logs can be recorded using the EvalTool, CLTool, or SDK.

PPD RMC bits Preset

PPD logs are created by enabling PPD data streaming by setting the RMC bits to `RMC_PRESET_IMX_PPD` and logging this stream to a .dat binary file. `RMC_PRESET_IMX_PPD` is defined in data_sets.h.

Logging PPD in EvalTool

The following steps outline how to record post process data in the EvalTool

1. Go to the "**Data Logs**" tab in the EvalTool.
2. Press the "**Data Log: PPD Log**" button to start logging.
3. Toggle the "**Data Log: Disable**" button to stop logging.
4. The "**Open Folder**" button will open the directory where the data logs are stored.

Logging PPD in CLTool

Streaming and logging a PPD log using the CLTool is done using the `-presetPPD -lon` options:

©2022

```
cltool -c /dev/ttyS2 -presetPPD -lon
```

See the CLTool section for more information on using the CLTool.

©2022

11.3 Plotting

11.3.1 Log Inspector

Log Inspector is a convenient way to quickly plot Inertial Sense PPD logs that are of the .dat format. The source code is in the SDK and can be modified and expanded.

11.3.2 CLTool

The CLTool can be used to load and replay `.dat` log files. The source code for the CLTool is located in the SDK and can be expanded by a user to analyze log data.

- `-rp PATH` replay data log from specified path
- `-rs=SPEED` replay data log at x SPEED

The following example replays data at 1x speed from the specified directory.

```
cltool -rp IS_logs/20180801_222310
```

The following example will replay data as fast as possible in quiet mode (without printing to the screen). This is useful to quickly reprocess the data.

```
cltool -rp IS_logs/20180801_222310 -rs=0 -q
```

11.3.3 3rd Party Software

The various file types described in the overview section can be analyzed using various software packages. Matlab, Python, and Excel are popular choices and are well suited for Inertial Sense data logs.

©2022

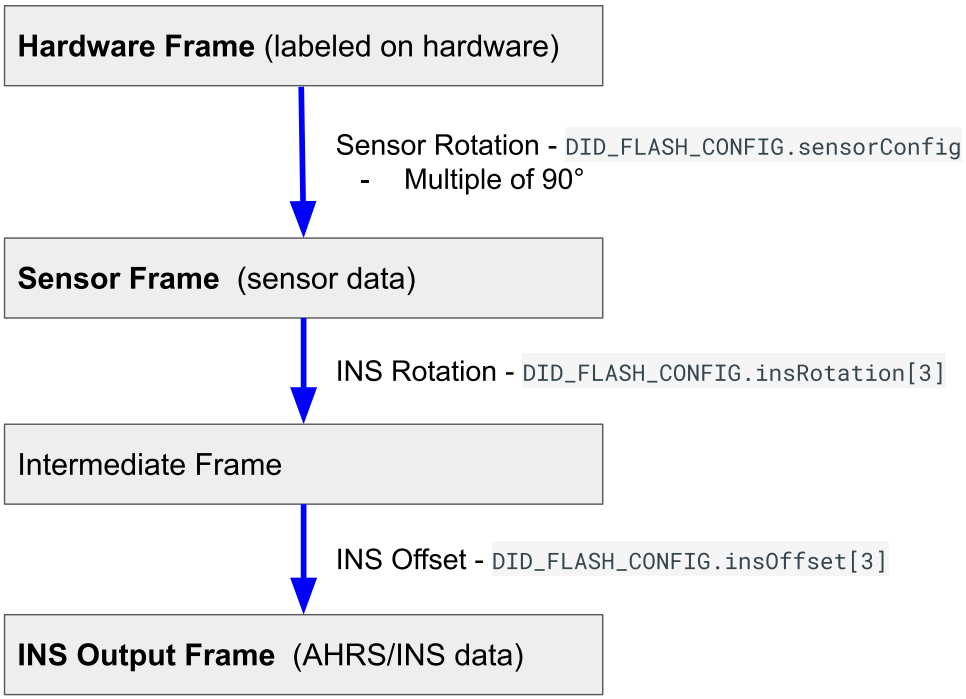
12. Reference

12.1 Coordinate Frames

In this manual, coordinate frame systems are simply referred to as frames. This page is to assist the developer in choosing and implementing the appropriate coordinate frames for their respective application. It should be noted that the following frames are in relation to the IMX itself.

12.1.1 Coordinate Frame Relationship

The relationship between the Hardware Frame, Sensor Frame, and INS Output Frame are as follows.

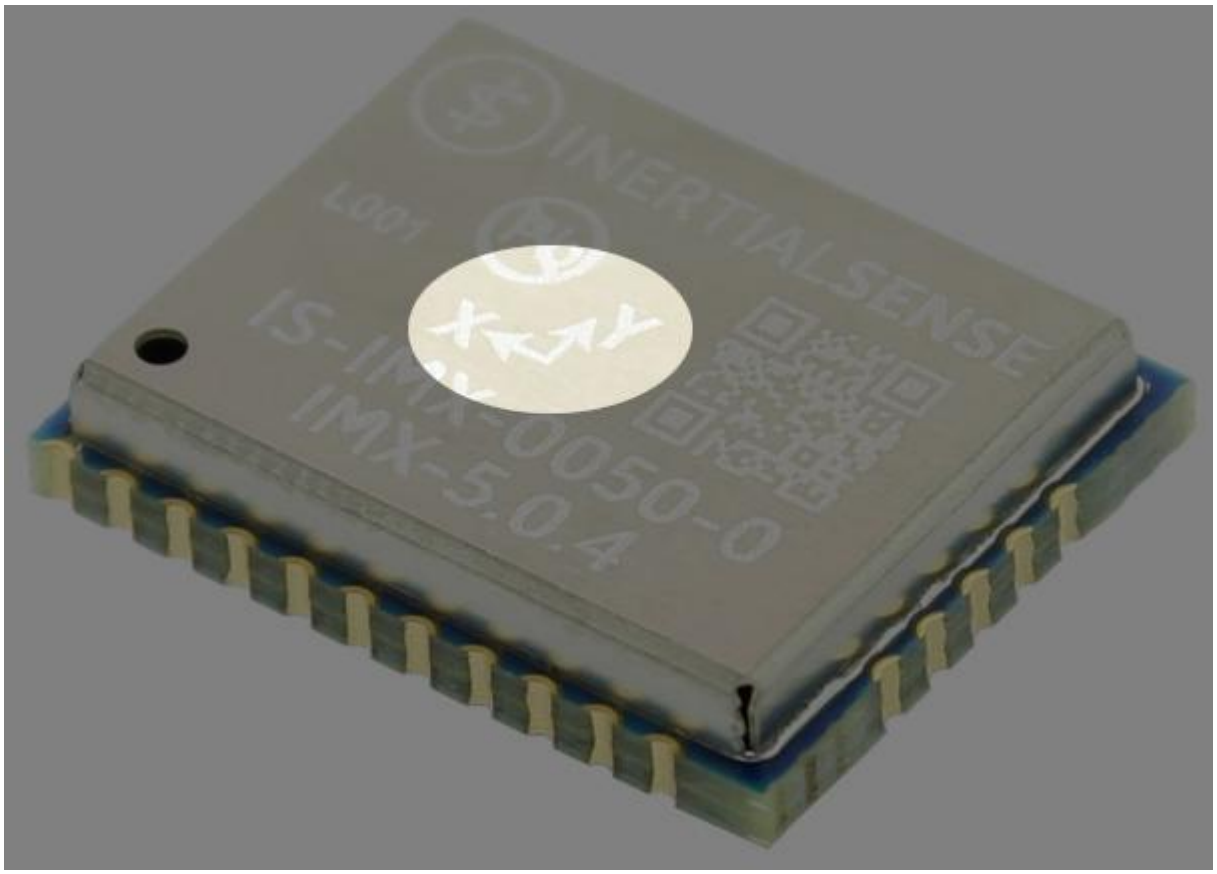


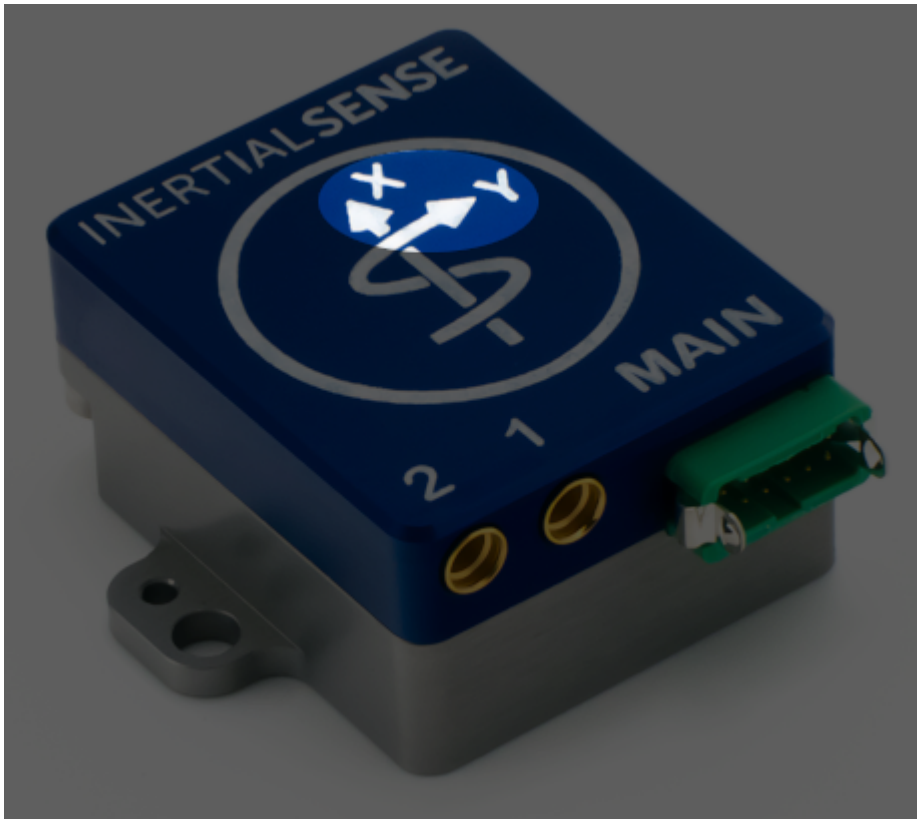
NOTE: The ***Hardware Frame*** and ***Sensor Frame*** are equivalent when the sensor rotation in `DID_FLASH_CONFIG.sensorConfig` is zero. The ***Sensor Frame*** and ***INS output Frame*** are equivalent when the `DID_FLASH_CONFIG.insRotation` and `DID_FLASH_CONFIG.insOffset` are zero.

12.1.2 Hardware Frame

The Hardware Frame is labeled "X" and "Y" on the hardware indicating the direction of the sensing elements in the IMX.

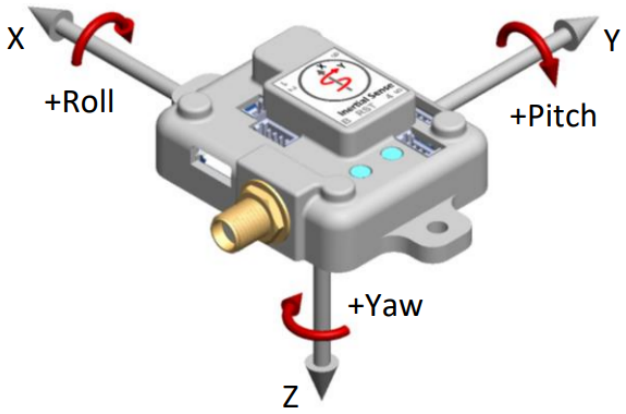
©2022

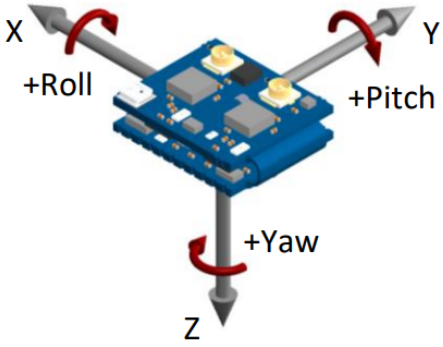




The IMX follows the right-hand rule for XYZ axis relative direction and angular rotation.

©2022





12.1.3 Sensor Frame

The IMU and magnetometer data (i.e. messages DID_IMU and DID_MAGNETOMETER) are in the Sensor Frame. The ***Hardware Frame*** is rotated into the ***Sensor Frame*** in multiples of 90° using the `SENSOR_CFG_SENSOR_ROTATION_MASK` bits of the `DID_FLASH_CONFIG.sensorConfig` as defined in `enum eSensorConfig`.

12.1.4 INS Output Frame

The INS output data (DID_INS_1, DID_INS_2, DID_INS_3) is in the INS Output Frame. Translation from Sensor Frame to INS Output Frame is defined as:

1. Sensor Frame → Intermediate Output Frame by rotation of `DID_FLASH_CONFIG.insRotation` euler angles (in order of heading, pitch, roll angle) In radians.
2. Intermediate Output Frame → INS Output Frame: Offset by `DID_FLASH_CONFIG.insOffset` in meters.

If `DID_FLASH_CONFIG.insRotation` and `DID_FLASH_CONFIG.insOffset` are zero, the Sensor Frame and the INS Output Frame are the same.

12.1.5 North-East-Down (NED) Frame

Position estimates can be output in the North-East-Down (NED) coordinate frame defined as follows:

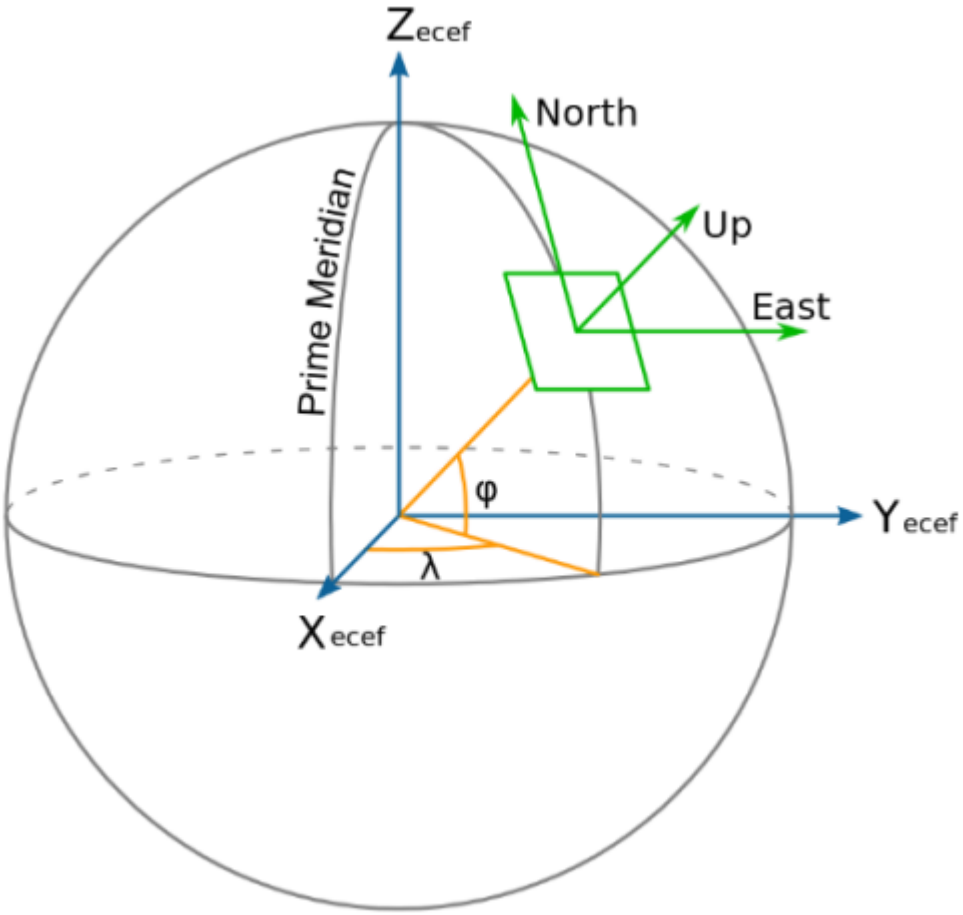
* Right-handed, Cartesian, non-inertial, geodetic frame with origin located at the surface of Earth (WGS84 ellipsoid). * Positive X-axis points towards North, tangent to WGS84 ellipsoid. * Positive Y-axis points towards East, tangent to WGS84 ellipsoid. * Positive Z-axis points down into the ground completing the right-handed system.

12.1.6 Earth-Centered Earth-Fixed (ECEF) Frame

The Earth-Centered Earth-Fixed (ECEF) frame is defined as follows:

* Right-handed, Cartesian, non-inertial frame with origin located at the center of Earth. * Fixed to and rotates with Earth. * Positive X-axis aligns with the WGS84 X-axis, which aligns with the International Earth Rotation and Reference Systems Service (IERS) Prime Meridian. * Positive Z-axis aligns with the WGS84 Z-axis, which aligns with the IERS Reference Pole (IRP) that points towards the North Pole. * Positive Y-axis aligns with the WGS84 Y-axis, completing the right-handed system.

©2022



12.1.7 Coordinate Frames Transformation Functions

This section is intended to be an example of how to rotate between frames using utility functions defined in the InertialSenseSDK.

Body frame to NED frame

The following example converts body velocity `DID_INS_2.uvw` to NED velocity `vel_ned`.

```
#include "SDK/src/ISPose.h"
quatRot( vel_ned, DID_INS_2.qn2b, DID_INS_2.uvw );
```

The following computes ground track heading based on `DID_INS_2.uvw` body velocity in the local tangent plane.

```
#include "SDK/src/ISPose.h"
quatRot( vel_ned, DID_INS_2.qn2b, DID_INS_2.uvw );
ground_track_heading = atan2f( vel_ned[1], vel_ned[0] );
```

This following example removes gravity from the IMU measured acceleration.

```
#include "SDK/src/ISPose.h"
Vector gravityNED = { 0, 0, -9.80665 }; // m/s^2
Vector gravityBody;
Vector accMinusGravity;
// Rotate gravity into body frame
quatConjRot( gravityBody, DID_INS_2.qn2b, gravityNED );
// Subtract gravity from IMU acceleration output
sub_Vec3_Vec3( accMinusGravity, DID_IMU.I[0].acc, gravityBody );
```


©2022

ECEF frame to NED frame

This example converts ECEF velocity `vel_ecef` to NED velocity `vel_ned`.

```
#include "SDK/src/ISPose.h"
quat_ecef2ned( lla[0], lla[1], qe2n );
quatConj( qn2e, qe2n );
quatRot( vel_ned, qn2e, vel_ecef );
```


©2022

12.2 Definitions

12.2.1 GPS Time To Fix

The time it takes for the GPS receiver to get “fix” or produce a navigation solution from the visible satellites is affected by the following GPS startup conditions:

- Cold start - In cold start mode, the receiver has no information from the last position (e.g. time, velocity, frequency etc.) at startup. Therefore, the receiver must search the full time and frequency space, and all possible satellite numbers. If a satellite signal is found, it is tracked to decode the ephemeris (18-36 seconds under strong signal conditions), whereas the other channels continue to search satellites. Once there are enough satellites with valid ephemeris, the receiver can calculate position and velocity data. Other GNSS receiver manufacturers call this startup mode Factory Startup.
- Warm start - In warm start mode, the receiver has approximate information for time, position, and coarse satellite position data (Almanac). In this mode, after power-up, the receiver normally needs to download ephemeris before it can calculate position and velocity data. As the ephemeris data usually is outdated after 4 hours, the receiver will typically start with a Warm start if it has been powered down for more than 4 hours.
- Hot start - In hot start mode, the receiver was powered down only for a short time (4 hours or less), so that its ephemeris is still valid. Since the receiver doesn't need to download ephemeris again, this is the fastest startup method.

Battery backed-up power supplied to the IMX preserves the GPS time, position, and coarse satellite position (almanac) while off. GPS almanac data is typically valid for several weeks while the GPS is off.

12.2.2 Preintegrated IMU

Also known as Coning and Sculling Integrals, Δ Theta Δ Velocity, or Integrated IMU. For clarification, we will use the name "Preintegrated IMU" or "PIMU" throughout the User Manual. They are integrated by the IMU at IMU update rates (2, 4, or 8 kHz). These integrals are reset each time they are output. Preintegrated IMU data acts as a form of compression, adding the benefit of higher integration rates for slower output data rates, preserving the IMU data without adding filter delay. It is most effective for systems that have higher dynamics and lower communications data rates. The IMX-5 uses Bortz for angular rates (which yield rotation from the previous pose) and full dual gyro and accelerometer integration for change in body velocities from the previous pose. The PIMU is NOT integrated from each axis separately.

12.2.3 IMU Bias Repeatability (Turn-on to Turn-on Bias)

The initial bias will be different for each power up of the IMU due to signal processing initial conditions and physical properties. A more repeatable bias allows for better tuning of INS parameters and faster estimate of the bias, whereas a more variable initial turn-on bias causes more difficult and longer INS convergence startup time.

12.2.4 IMU Bias Stability (In-Run Bias)

Describes the amount of bias change during any one run-time following power-on. This change is caused by temperature, time, and mechanical stress. The INS navigation filter estimates the IMU biases in order to improve the state estimate. The IMU bias stability directly impacts the accuracy of the INS output.

12.2.5 Random Walk

The IMU sensors measure a signal as well as noise or error, described as a stochastic process. During IMU integration in the INS, sensor noise is accumulated and produces a random walk or drift on the final solution. Random walk has a direct effect on the accuracy of the INS output.

12.2.6 Sensor Orthogonality (Cross-Axis Alignment Error)

The three axis gyro and accelerometer sensors found in an IMU have measurement axes at 90 degrees from each other, maximizing the observability of the system. In practice, these sensing axes in a three axis sensor are not perfectly at 90 degrees

©2022

of each other, or misaligned slightly due to manufacturing imperfection. This misalignment results in integration error in the INS and impacts accuracy. To correct for cross-axis alignment error, the IMU is calibrated during manufacturing in a controlled motion environment.

©2022

12.3 IMU Specifications

12.3.1 IMU Noise Specification Conversion to Standard Deviation

The following calculations convert the noise specifications from the IMX-5 inertial measurement unit (IMU) datasheet into usable standard deviation values for simulating sensor noise at a sampling rate of 100 Hz. IMUs typically provide specifications for gyroscope and accelerometer noise in terms of "Angular Random Walk" (ARW) and "Velocity Random Walk" (VRW), expressed per \sqrt{\text{hours}}. These values represent the rate at which random walk (drift) accumulates over time. To model this noise accurately in simulations, we need to translate the datasheet specifications into standard deviations that correspond to the chosen sampling rate (100 Hz, or 0.01 seconds per sample). This involves converting the ARW and VRW values from per \sqrt{\text{hour}} to per \sqrt{\text{second}} and then adjusting them based on the sampling interval, yielding noise characteristics that realistically represent the IMU's behavior in a simulated environment.

Given IMU specifications for the **IMX-5**:

- **Gyro Angular Random Walk (ARW)**: 0.16 \, ^{\circ} / \sqrt{\text{hr}}
- **Accelerometer Velocity Random Walk (VRW)**: 0.02 \, \text{m/s} / \sqrt{\text{hr}}
- **Sampling Rate**: 100 Hz (which corresponds to a time interval, \Delta t, of 0.01 seconds)

Time Conversion Factor

Since the random walk values are given per \sqrt{\text{hr}}, we need to convert from hours to seconds. 1 hour is 3600 seconds, so: To convert the noise specifications from per \sqrt{\text{hr}} to per \sqrt{\text{s}}, divide by 60.

1. Gyroscope Noise (° and °/s)

Convert the **Gyro Angular Random Walk (ARW)** to per \sqrt{\text{s}}:

Now, to get the **angle drift** at a 100 Hz sampling rate, multiply by the square root of the time interval: So, the **angle drift standard deviation at 100 Hz** is approximately **0.000267 °**.

To get the **angular rate noise** at a 100 Hz sampling rate, divide ARW by the square root of the time interval:

2. Accelerometer Noise (m/s and m/s²)

VELOCITY DRIFT STANDARD DEVIATION (M/S)

Convert **VRW** from per \sqrt{\text{hr}} to per \sqrt{\text{s}}:

Now, multiply by the square root of the time interval to get the standard deviation at 100 Hz in terms of velocity: So, the **velocity drift standard deviation at 100 Hz** is approximately **0.0000333 m/s**.

ACCELEROMETER STANDARD DEVIATION IN TERMS OF ACCELERATION (M/S²)

To express the accelerometer noise as standard deviation in terms of acceleration, divide the VRW (converted per \sqrt{\text{s}}) by the square root of the time interval: Thus, the **accelerometer noise standard deviation in terms of acceleration at 100 Hz** is approximately **0.00333 m/s²**.

Summary of Results:

- **Angle Drift Standard Deviation at 100 Hz**: **0.000267 °** (denoted as \sigma_{\text{angle}})
- **Gyro Angular Rate Noise Standard Deviation at 100 Hz**: **0.0267 °/s** (denoted as \sigma_{\text{gyro}})
- **Velocity Drift Standard Deviation at 100 Hz**: **0.0000333 m/s**
- **Acceleration Noise Standard Deviation at 100 Hz**: **0.00333 m/s²**

These values represent the Gaussian noise standard deviations for each sensor at 100 Hz sampling rate.

©2022

12.4 Interference Considerations

Electrical interference or noise can be coupled into the Inertial Sense module in the form of electromagnetic interference (EMI) through the air or electrically conducted through wiring. Sources for interference include:

- EMI at the GPS antenna.
- EMI at the IMX module.
- EMI conducted through the power supply or I/O lines.

Common sources for noise and interference are digital lines, USB 3.x, noisy power supplies, etc.

12.4.1 Detecting Interference

To detect if interference is being coupled into the Inertial Sense sensor module, it can be compared with a stock EVB demo unit to compare noise figures. This is done by using the following steps. If both steps pass, there is no noise being coupled into the module. Optionally, multiple sensor modules can be connected to the EvalTool in parallel to compare noise.

1. **Evaluate the IMU sensor** - Make sure the unit is stationary (on a table or non-moving surface) and not seeing any vibrations. Watch the standard deviation columns labeled "σ" in the Sensors tab of the EvalTool. This shows the noise level over the past 5 seconds, which means the device needs to be completely stable for 5 seconds to be accurate. Compare this figure between the integrated sensor module and EVB demo unit.
2. **Evaluate GPS sensitivity** – In clear view of the sky, monitor the satellite signal strength through the `DID_GNSS1_POS.cnoMean` field in the EvalTool "Data Sets" tab or in the EvalTool "GPS" tab. See that the CNO values are roughly the same between the integrated sensor module and the EVB demo unit.

12.4.2 Interference Mitigation

The best solution is to stop the EMI (emitted) or conducted noise at its source. If it is not possible to completely eliminate the source, the following methods should be considered depending on the cause of interference:

- **GPS antenna location** - Position the GPS antenna at the top of the vehicle, clear of obstructions and away from noise sources such as motors, processors, USB cables, digital devices, etc. USB 3.x typically generates quite a bit of EMI and interferes with the GPS. Added shielding and/or signal inline USB filters can be added USB 3.x to reduce EMI and mitigate GPS interference. Symptoms of GPS interference are poor GPS CNO signal strength, long times to lock, slip out of lock, etc.
- **GPS antenna ground plane** - Adding a 2"-3" diameter metallic ground plane below the GPS antenna will improve CNO noise ratio and improve sensitivity. This can help in noisy environments. You can use any scrap metal or PCB to test the concept by simply placing it below the GPS antenna (no electrical grounding required). The ground plane can have holes to reduce the weight and aerodynamic effects.

©2022

- **Shielding around the Inertial Sense module** - This can further prevent EMI from being absorbed by potentially GPS sensitive circuitry on the module.
- **Digital signals** - Making sure best practices for electrical current return paths for both common mode and differential mode signals can be key for this. Customers have seen GPS interference caused by USB 3.x and have resolved the issues using shielding and inline USB filters.
- **Power supply filtering** - This may be necessary on systems with significant digital noise. LC filters or similar filters can be added inline between the power supply and the IMX supply input (Vcc). Common switching mode/buck voltage regulators should be fine for use with the IMX module and not require additional filtering.

Please contact us at support@inertialsense.com if further support is needed.

12.4.3 Magnetic Interference

Magnetic interference may impact IMX magnetometer performance if surrounded by steel or ferrous material or near motors, motor drivers, or other electronics that cause EMI. This interference can be observed in the magnetometer output, magnetometer status, and INS heading. Make sure all components are fixed in location during this test. While powering and actuating the various interference sources, observe the following:

- **Magnetometer Output** should remain constant and not deviate. (EvalTool Sensors tab)
- **Magnetometer Status** should remain good and not indicate interference. (EvalTool INS tab, "Mag used" green = good, yellow = interference)
- **INS Heading** (yaw) estimate should not drift or change direction. (EvalTool INS tab, "Yaw")

If any of these items are affected during the test, the system may result in incorrect magnetometer and heading values.

12.4.4 Mechanical Vibration

The system accuracy may degrade in the presence of mechanical vibrations that exceed 3 g of acceleration. Empirical data shows degradation at approximately 100 - 150 Hz. Adding **vibration isolation** to the mount may be necessary to reduce the vibrations seen by the product and to improve accuracy.

12.4.5 Temperature Sensitivity

The system is designed to compensate for the effects of temperature drift, which can be found in typical operation. However, rapid hardware temperature changes can result in degraded accuracy of the IMU calibration, GPS position, and INS estimate. Rapid temperature change can be caused by direct exposure to wind, sun, and other elements.

©2022

13. User Manual PDF

This document is provided to allow users to capture a snapshot of our current documentation. **The PDF is an automatically generated printout of the current online documents so there are known link and image sizing issues.** We encourage use the online documentation which is maintained for customer accessibility.

Download PDF

©2022

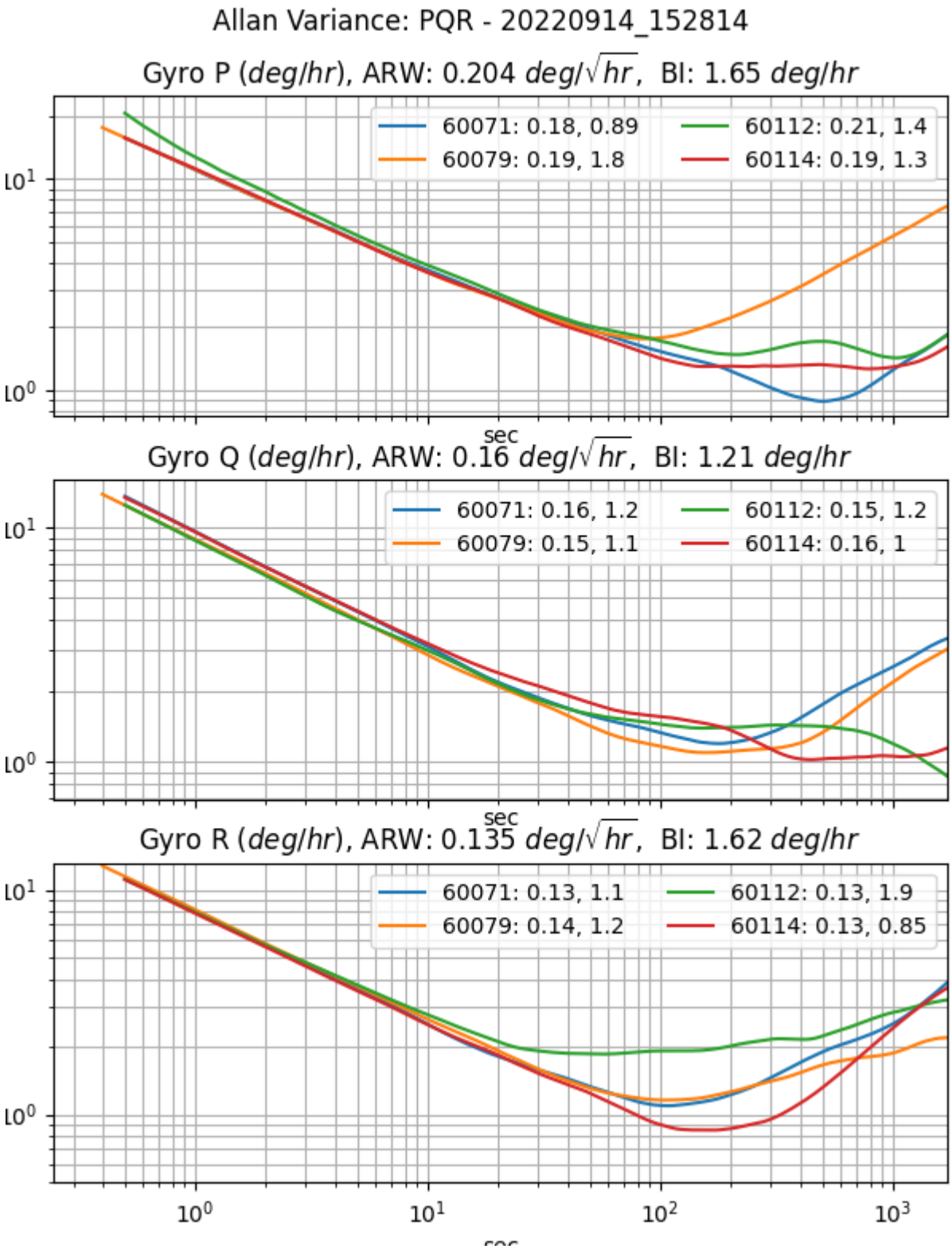
14. Frequently Asked Questions

14.1 What is a Tactical Grade IMU?

An Inertial Measurement Unit (IMU) is industry qualified as "Tactical Grade" when the In Run Bias Stability (IRBS) of the gyroscopes is between 0.5 deg/hour and 5 deg/hour. The IRBS represents the IMU stability during benign conditions (i.e. ideal integration time, stable temperature, and no inertial motion).

IEEE-STD-952-1997 defines IRBS as the minima on the Allan Variance curve. The following plots identify the Allan Variance representation of four IMX-5 tactical grade IMUs (serial numbers 60071, 60079, 60112, and 60114).

©2022



14.2 What is Inertial Navigation?

Inertial navigation is a technique of estimating position, velocity, and orientation (roll, pitch, heading) by integrating IMU inertial motion data from gyros and accelerometers to continuously calculate the dead reckoning position. The inertial sensors are supplemented with other sensors such as GPS, altimeter, and magnetometer. Inertial navigation is commonly used on moving vehicles such as mobile robots, ships, aircraft, submarines, guided missiles, and spacecraft.

©2022

14.3 What does an Inertial Navigation System (INS) offer over GPS alone?

Dead Reckoning - An inertial navigation system (INS) integrates the IMU data to dead reckon (estimate position and velocity) between GPS updates and during GPS outage.

Higher Data Rates - Typical GPS receivers data rates vary from 1Hz to 20Hz whereas INS systems like the IMX have data rates up to 8 kHz.

Signal Conditioning - An INS filters out noise in the GPS data and provides a smoother, more continuous data stream.

Orientation Data - An INS is capable of observing the orientation (roll, pitch, and heading) of the system regardless of the motion or direction of travel. This is because of how an INS fuses inertial data with GPS data. A GPS with one antenna can measure direction of travel (ground track heading) but cannot estimate vehicle roll, pitch, or heading.

14.4 Our Sensors - IMU vs AHRS vs INS

Inertial Measurement Unit (IMU) - Uses gyros and accelerometers to measure angular rate and linear acceleration.

Attitude Heading Reference System (AHRS) - Adds sensor fusion to IMU and magnetometer output to estimate orientation or roll, pitch, and heading.

Inertial Navigation System (INS) - Adds sensor fusion to IMU, GPS, magnetometer, and barometer data to estimate orientation, velocity, and position.

14.4.1 I have my own GPS system and just need raw motion data, which sensor is for me?

If you have your own filters in place and need raw data for measuring motion, then the **IMU** is the best option. The IMU provides raw, calibrated data for temperature, 3D acceleration (accelerometer), 3D magnetic field (magnetometer), and 3D rate of turn.

14.4.2 Which sensor will also provide attitude (roll/pitch/yaw) and heading data?

The **AHRS** sensor has all the capabilities of the IMU plus data on roll, pitch, and yaw. This sensor uses algorithms to fuse raw data from the IMU with the Earth’s gravity to provide orientation and is perfect for a robotic arm or indoor floor cleaner.

I need geographic positional data. Which sensor contains a GPS?

If the AHRS doesn't get you what you need, packaging this with one or more GPS sensors will give you the geographic positional data you need. Like the AHRS, your sensor should provide some sort of "sensor fusion" by combining all the data from each of these sensors to give a more accurate and holistic view of your rover's state.

What do I need to get started?

We recommend beginning with some sort of Development Kit. Ours comes with all of the necessary components to simplify testing and integration: the sensor you selected, the needed cable and antennas for connectivity, the firmware & software, and 3-5 hrs of complimentary engineering support from a team of Inertial Sense engineers.

Additionally, Inertial Sense provides customers with a custom datalogger, called the EvalTool. An easy to use data logging software to test and troubleshoot your new sensor is **essential** to your sensor integration experience.

14.5 How long can the IMX dead reckoning estimate position without GPS?

The IMX inertial navigation integrates the IMU data to dead reckoning position and velocity estimation between GPS updates and for a short period of time during GPS outages. This dead reckoning is designed to filter out GPS noise and provide cleaner faster updates than are available via GPS alone. The IMX dead-reckons, or estimates position and velocity, between GPS updates and through brief GPS outages. However, it is not designed for extended position navigation without GPS aiding. Dead reckoning is disabled after 7 seconds of GPS outage in order to constrain position and velocity drift. The amount of position drift during dead reckoning can vary based on several factors, including system runtime, motion experienced, and bias stability.

©2022

14.6 Can the IMX estimate position without GPS?

IMX can estimate the position for extended periods of time without GPS for ground vehicle dead reckoning applications only (see Ground Vehicle Dead Reckoning). When in standard mode (not ground vehicle dead reckoning) GPS is required to provide initial position estimation and to aid in IMU bias estimation. The IMX can dead reckon (estimate position without GPS) for brief periods of time. However, the quality of dead reckoning is a function of IMU bias estimation, which improves while the GPS is aiding the INS.

14.7 How does the IMX estimate roll/pitch during airborne coordinate turns (acceleration only in the Z axis and not in the X and Y axes)?

Using acceleration alone will incorrectly indicate the platform is level. GPS informs the IMX extended Kalman filter (EKF) about any motion the system experiences and as a result the EKF can distinguish between acceleration due to motion and gravity. Without a GPS (in AHRS mode) the IMX EKF can only assume the direction of gravity equals the average direction of acceleration measured slowly over time. The IMX can estimate roll/pitch under accelerated conditions during a coordinated turn because gyro integration prevents attitude drift over short term and level coordinate turns are cyclical allowing the average gravity for an entire heading rotation to be observed. However, the IMX AHRS solution under accelerated conditions (moving) will have degraded attitude accuracy. The IMX solution is more accurate when aided by GPS.

14.8 How does vibration affect navigation accuracy?

The IMX accuracy may degrade in the presence of mechanical vibrations that exceed 3g of acceleration. Empirical data shows degradation at approximately 100 - 150 Hz. Adding vibration isolation to the mount may be necessary to reduce the vibrations seen by the product and to improve accuracy.

14.9 Can the IMX operate underwater?

The IMX can only dead reckon for short periods of time and in general requires GPS to provide position and velocity data. The GPS antenna must be above the water surface in order for the GPS to function properly. It is ideal that the GPS antenna be fixed relative to the IMX (IMU) module in order to maintain precision when moving faster than 2 m/s or 0.8 m/s^2. However, the GPS antenna may be tethered above the IMX, where the GPS antenna is floating on the water surface and the IMX is below the water surface. System position will reflect the GPS antenna position and attitude (roll, pitch, heading) will reflect the IMX module orientation.

14.10 Can the IMX operate at >4g acceleration?

Typical L1 GPS receivers lose fix above 4g acceleration because the doppler variation starts to get too large and the receiver may become unstable or not be able to get/keep a fix. Additionally, the acceleration begins to affect the stability of the GPS XTAL oscillator.

On the IMX, the GPS will regain fix within seconds after acceleration drops below 4g. The IMX will track the velocity and position using inertial navigation for up to 5 seconds of GPS outage. As long as GPS outage is below 5 seconds, the IMX should be able to track position through a launch.

14.11 Customer Support

Have other questions or needs? Please email us at support@inertialsense.com.

©2022

15. Inertial Sense Navigation Glossary

This glossary defines terminology used in inertial navigation, GNSS positioning, and Inertial Sense navigation systems.

16. Navigation Systems

16.0.1 IMU (Inertial Measurement Unit)

A sensor device containing:

- gyroscopes
- accelerometers

It measures **angular velocity** and **linear acceleration**, which are used to estimate motion and orientation.

16.0.2 AHRS (Attitude and Heading Reference System)

A system that estimates **orientation** using IMU and magnetometer data.

Outputs typically include:

- roll
- pitch
- heading

16.0.3 INS (Inertial Navigation System)

A navigation system that computes:

- position
- velocity
- orientation

by integrating IMU measurements over time.

16.0.4 Strapdown INS

A modern INS architecture where the IMU is rigidly attached to the vehicle and navigation equations are solved in software.

This contrasts with older **gimbaled INS systems**.

16.0.5 GNSS (Global Navigation Satellite System)

Satellite navigation systems providing global positioning and timing.

©2022

Examples include:

- GPS
- GLONASS
- Galileo
- BeiDou

16.0.6 GNSS-INS Integration

Combining GNSS and inertial measurements to provide:

- robust positioning
- high-rate navigation
- improved reliability

16.0.7 Dead Reckoning

Position estimation obtained by integrating velocity and heading over time without external measurements.

16.0.8 Sensor Fusion

The process of combining measurements from multiple sensors to estimate system state more accurately.

17. Navigation State Variables

17.0.1 Position

Vehicle location expressed as:

- Latitude
- Longitude
- Altitude

This geodetic representation is often called **LLA**.

Position can also be expressed in Earth-fixed Cartesian (ECEF) coordinates:

```
p_ecef = [X, Y, Z]
```

or in local coordinates:

```
p = [North, East, Down]
```

17.0.2 Velocity

Rate of change of position.

In the navigation frame:

```
v = [v_N, v_E, v_D]
```


©2022

Velocity can also be expressed in Earth-fixed Cartesian (ECEF) coordinates:

```
v_ecef = [v_X, v_Y, v_Z]
```

17.0.3 Attitude

The orientation of a vehicle relative to a reference frame.

Representations include:

- Euler angles
- quaternions
- direction cosine matrices

18. Coordinate Frames

18.0.1 Body Frame

Coordinate frame fixed to the IMU.

Typical convention:

```
X → Forward
Y → Right
Z → Down
```

18.0.2 Navigation Frame

A local Earth-referenced coordinate frame used for navigation.

Common choice:

NED (North-East-Down)

18.0.3 NED Frame

Local tangent plane coordinate system:

```
X → North
Y → East
Z → Down
```

18.0.4 ECEF Frame

Earth-Centered Earth-Fixed coordinate system.

Origin:

Earth's center of mass.

Coordinates are represented as:

```
[X, Y, Z]
```


©2022

in meters, fixed to Earth.

18.0.5 LLA (Latitude, Longitude, Altitude)

Geodetic position representation on the Earth reference ellipsoid.

Common units:

- latitude/longitude in degrees
- altitude in meters

LLA is often used as an alternative to ECEF coordinates depending on the application.

18.0.6 ECI Frame

Earth-Centered Inertial coordinate frame that does not rotate with Earth.

Used primarily in orbital mechanics.

18.0.7 Sensor Frame

Coordinate frame defined by the axes of the IMU sensors.

18.0.8 Vehicle Frame

Coordinate frame aligned with the vehicle body.

19. Attitude Representations

19.0.1 Euler Angles

Orientation defined by three sequential rotations:

```
Roll  (φ)
Pitch (θ)
Yaw   (ψ)
```

Limitations:

- gimbal lock near ±90° pitch

19.0.2 Quaternion

A four-element representation of orientation:

```
q = [q_w, q_x, q_y, q_z]
```

Advantages:

- no singularities
- efficient for filtering

©2022

19.0.3 Direction Cosine Matrix (DCM)

3×3 rotation matrix that converts vectors between coordinate frames.

Example:

```
C_bn
```

Transforms body frame vectors into navigation frame.

20. Strapdown Navigation Equations

20.0.1 Attitude Propagation

Orientation updated using gyro measurements:

```
q̇ = ½ Ω(ω) q
```

where

```
ω = angular rate vector
Ω = quaternion rate matrix
```

20.0.2 Velocity Equation

Velocity evolves as:

```
v̇ = C_bn f_b + g_n - (2Ω_ie + Ω_en)v
```

where:

- `f_b` = specific force
- `g_n` = gravity vector
- `Ω_ie` = Earth rotation rate
- `Ω_en` = transport rate

20.0.3 Position Equation

Position updated by integrating velocity:

```
ṗ = v
```

21. IMU Measurement Terms

21.0.1 Angular Rate

Rotation rate measured by gyroscopes.

Units:

```
deg/s
rad/s
```


©2022

21.0.2 Specific Force

Acceleration measured by accelerometers excluding gravity.

21.0.3 Linear Acceleration

True acceleration of the body.

21.0.4 Magnetic Field

Earth magnetic field measured by magnetometers for heading estimation.

21.0.5 Barometric Pressure

Atmospheric pressure measurement used for altitude estimation.

22. Sensor Error Models

22.0.1 Bias

Constant offset in sensor output.

```
measurement = truth + bias
```

22.0.2 Bias Instability

Slow variation of sensor bias over time.

Often characterized using Allan variance.

22.0.3 Angular Random Walk (ARW)

Gyroscope white noise causing attitude error growth.

Units:

```
° / √hr
```

Error growth:

```
σ_angle = ARW √t
```

22.0.4 Velocity Random Walk (VRW)

Accelerometer white noise causing velocity error growth.

Units:

```
m/s / √hr
```


©2022

22.0.5 Scale Factor Error

Error in sensor gain.

```
measurement = (1 + scale_factor_error) * truth
```

22.0.6 Misalignment Error

Small angular errors between sensor axes and the reference coordinate frame.

22.0.7 Cross-Axis Sensitivity

Sensor response to motion on another axis.

23. Allan Variance Terms

23.0.1 Allan Variance

A time-domain method used to analyze sensor noise processes.

23.0.2 Allan Deviation

Square root of Allan variance.

Used to identify noise characteristics.

23.0.3 Bias Instability (Allan)

Estimated from the minimum of the Allan deviation curve.

Conversion:

```
bias_instability = Allan_min / 0.664
```

23.0.4 Rate Random Walk

Low-frequency gyro noise process.

23.0.5 Quantization Noise

Noise introduced by digital resolution limits.

©2022

24. GNSS Terminology

24.0.1 Pseudorange

Measured distance between receiver and satellite based on signal travel time.

24.0.2 Carrier Phase

High precision measurement of GNSS signal phase.

Used for centimeter-level positioning.

24.0.3 RTK (Real-Time Kinematic)

Carrier-phase GNSS positioning using base station corrections.

Accuracy:

```
1–2 cm
```

24.0.4 Base Station

Stationary GNSS receiver that provides correction data.

24.0.5 Rover

GNSS receiver whose position is being estimated.

24.0.6 Differential GNSS (DGNSS)

Positioning method using corrections from a reference station.

24.0.7 Satellite Ephemeris

Precise orbital data describing satellite position.

24.0.8 Dilution of Precision (DOP)

Measure of satellite geometry quality.

Types include:

- GDOP
- PDOP
- HDOP
- VDOP

©2022

25. GNSS Measurement Models

25.0.1 Pseudorange Equation

```
ρ = r + c(dt - dT) + I + T + ε
```

where:

- `r` = geometric range
- `dt` = receiver clock bias
- `dT` = satellite clock bias
- `I` = ionospheric delay
- `T` = tropospheric delay
- `ε` = measurement noise

25.0.2 Carrier Phase Measurement

```
Φ = r + c(dt - dT) + λN + ε
```

where:

- `λ` = carrier wavelength
- `N` = integer ambiguity

26. Kalman Filtering

26.0.1 Kalman Filter

An optimal recursive estimator used to estimate system state.

26.0.2 Extended Kalman Filter (EKF)

A nonlinear version of the Kalman filter commonly used in navigation.

26.0.3 State Vector

Vector of estimated system variables.

Example INS state:

```
x = [position, velocity, attitude, biases]
```

26.0.4 Process Model

Mathematical model predicting how system state evolves.

©2022

26.0.5 Measurement Model

Relates sensor measurements to system state.

26.0.6 Covariance Matrix

Represents uncertainty in the state estimate.

26.0.7 Innovation

Difference between predicted and measured observation.

```
innovation = measurement − prediction
```

26.0.8 Observability

Ability to estimate a state variable from available measurements.

27. INS/GNSS Integration

27.0.1 Loosely Coupled Integration

INS and GNSS are integrated at the **position/velocity level**.

27.0.2 Tightly Coupled Integration

GNSS measurements integrated at the **pseudorange level**.

Allows operation with fewer satellites.

27.0.3 Deep Coupling

GNSS tracking loops assisted directly by INS measurements.

27.0.4 Lever Arm

Offset between IMU and GNSS antenna.

```
r_lever = position_GNSS − position_IMU
```

27.0.5 Time Synchronization

Alignment of timestamps between sensors.

©2022

28. Navigation Performance Metrics

28.0.1 CEP (Circular Error Probable)

Radius containing 50% of position errors.

28.0.2 RMS Error

Root-mean-square error of measurements.

28.0.3 Drift Rate

Rate at which inertial navigation error grows without external aiding.

29. Inertial Sense Protocol

29.0.1 DID (Data Identifier)

Numeric identifier defining a data structure in the Inertial Sense binary protocol.

Examples include:

```
DID_INS_1
DID_IMU
DID_GNSS1_POS
```

29.0.2 Data Set

Binary structure containing navigation or configuration data.

29.0.3 ISB (Inertial Sense Binary)

Binary communication protocol used by Inertial Sense devices.

29.0.4 NMEA

ASCII protocol used for GNSS data output.

Example:

```
$GPGGA
```

30. Software Tools

30.0.1 SDK

Software development kit used to integrate Inertial Sense devices.

©2022

30.0.2 EvalTool

Graphical interface used to visualize and configure devices.

30.0.3 CLTool

Command-line tool used to communicate with devices.

31. Hardware Products

31.0.1 IMX

Inertial Sense navigation module containing:

- IMU
- magnetometer
- barometer
- navigation processor

31.0.2 GPX

Multi-band GNSS receiver used with IMX modules.

31.0.3 Rugged Systems

Industrialized versions of Inertial Sense modules.

©2022

32. Common Abbreviations

Acronym	Meaning
IMU	Inertial Measurement Unit
INS	Inertial Navigation System
GNSS	Global Navigation Satellite System
AHRS	Attitude and Heading Reference System
ARW	Angular Random Walk
VRW	Velocity Random Walk
RTK	Real-Time Kinematic
EKF	Extended Kalman Filter
NED	North-East-Down
ECEF	Earth-Centered Earth-Fixed
LLA	Latitude, Longitude, Altitude

©2022

33. Troubleshooting

33.1 Firmware Troubleshooting

Please email support@inertialsense.com for assistance or to provide feedback on this user guide.

33.1.1 Data doesn't look right

If the EvalTool or SDK are from a different release from the firmware on the unit, there may be communication protocol related issues. It's best to keep both the software and firmware in sync with each other. The EvalTool should flag a protocol mismatch in the settings tab.

33.1.2 Bootloader Not Responding

Check the following:

- The input supply is at 3.3V and clean without noise.
- The serial connection is grounded (no floating grounds).
- The serial wires between the IMX module and the next active device (buffer, converter, or processor) are not longer than 1 meter when bootloading firmware.
- Reset or power cycle the IMX and promptly run the firmware update within 30 seconds of reset. A known issue in the IMX-5 bootloader version v6g and prior versions that disables all UARTS if no handshake is received within 50 second following startup. Resetting the IMX will re-enable UARTs for 50 seconds.

33.1.3 Bootloader Update fails first time

If updating the bootloader firmware and using the USB direct connection on the IMX module (pins 1 and 2) or the EVB-2 (EVB USB connector), the serial port number will change when the device switches from application mode to Bootloader Update mode. This is expected and requires reselecting the new serial port and running the Bootloader Update process a second time.

33.1.4 System in AHRS mode despite GPS messages being received

If attempting to enter NAV mode but the system reports AHRS despite GPS data being received, then assure your units are not set to Rover RTK mode. This will override your ability to lock in GPS Nav mode.

33.1.5 "IMX-5 Bricked" System Recovery

Assert chip erase pin high (3.3V) while booting (power cycle or reset) to erase all flash memory and place IMX into ROM bootloader (DFU) mode.

Note: the IMX bootloader will timeout and disable all UARTS (not USB) after 30 seconds if the sync handshake is not received. This will render the IMX unresponsive over UART. To prevent this, do not interrupt the standard firmware update process. To recover the IMX, reset the IMX and then re-apply the firmware update within 30 seconds of reset. https://docs.inertialsense.com/user-manual/reference/bootloader/#known-issues

Note: Following chip erase: Update firmware using standard procedure including app and bootloader firmware images. Upload IMU calibration.

33.1.6 "GPX and IMX Bricked" System Recovery

Assert boot mode pin high (3.3V) while booting (power cycle or reset) to put the device into bootloader mode. Inertial Sense customer support is required to facilitate bootloader communications with this device.

©2022

33.1.7 "IMX (uINS) Bricked" System Recovery

There are different reasons a system may appear unresponsive and not communicate. The following sections describe how to recover a system from these states.

> **Attention**
>
> The ONLY indicator that the bootloader is running is the fading cyan module LED. NO communications will appear in the EvalTool or CLTool. **Attempt to update the firmware before performing a chip erase.**

> **Attention**
>
> Hardware v3.1.3 and firmware IS_uINS-3_v1.2.1.0_b287_2017-09-17_103826.hex and older will not communicate and require following these instructions to be recovered. Do NOT use the chip erase procedure for this scenario.

Stuck in Bootloader Mode

In some cases, the bootloader may fail to completely update firmware. This is indicated by the fading cyan status LED on the IMX module. This can happen if older bootloader firmware is on the IMX and firmware version 1.7.1 is uploaded. If this happens, the system will appear to be unresponsive in the EvalTool. The following process can be used to recover the system to a working state:

If the **bootloader is running**, identified with the fading cyan color LED on the IMX module, following these steps:

1. **Ensure IMX Firmware is Running** - *(This step is not necessary if the IMX firmware is running and the EvalTool is communicating with the IMX).* Select the device serial port and update the firmware using 1.6.4 or earlier. If the bootloader is running, version information will not appear in the EvalTool. The following bootloader update step will not work unless the EvalTool is communicating with the IMX firmware.
2. **Update the Bootloader** - Use the EvalTool "Update Bootloader" button in the Settings tab to upload the latest bootloader firmware. If it has a fading cyan color on the IMX module, the bootloader is running and ready for new firmware to be loaded. **The bootloader can only be updated using serial0 or the native USB connection.**
3. **Update the Firmware** - Use the EvalTool "Update Firmware" button to upload the latest IMX firmware.

If neither the bootloader or the IMX firmware are running, identified with the solid or no LED status on the IMX module, please contact us.

Recovery for Firmware v1.2.1.0

Hardware v3.1.3 and newer and firmware IS_uINS-3_v1.2.1.0_b287_2017-09-17_103826.hex and older result in a system that runs but will not communicate properly. This older firmware was not designed for the newer hardware and consequently runs the processor at a slower speed, which alters all of the predefined baud rates to non-standard irregular baud rates. A symptom of this problem is the LED flashing to indicate the processor activity and the module never communicates properly.

The following steps are provided to recover communications with the system.

1. Install and run the hotfix release 1.1.3 EvalTool.
2. Select the special baud rate **560,000** in the EvalTool and open the serial port.
3. Update the firmware using any version **newer than** IS_uINS-3_v1.2.1.0_b287_2017-09-17_103826.hex.

The latest EvalTool, CLTool, SDK, and firmware can be used once the firmware has been updated on the module.

©2022

33.1.8 Troubleshooting with EvalTool

Units Not Connecting

In the case that your units do not connect properly to the EvalTool, verify:

1. The baud rate is the same that you previously had when the Com Ports last opened correctly.
2. The LED on the unit is not showing solid white, flashing white, or solid red. These mean a failure occurred in loading the bootloader (see User Guide for full LED descriptions).
3. If you are using a USB3.0 connection, the com port might take longer to show up than with USB2.0
4. Check your computer's Device Manager to see if your unit shows up there. If it doesn't show up, you may have an FTDI driver issue.

a. If you suspect you don't have the FTDI driver installed on your Windows computer, use the following links to download the driver: - Executable for the FTDI USB driver: - https://ftdichip.com/wp-content/uploads/2023/09/CDM-v2.12.36.4-WHQL-Certified.zip - Drives without executable. - http://www.ftdichip.com/Drivers/D2XX.htm

33.1.9 Downgrading IMX to 1.8.x Firmware

The following steps can be used to downgrade the IMX firmware to version 1.8.x (or older):

1. Ensure the IMX is running 1.9.x (or newer) firmware.
2. Send the system commands `SYS_CMD_MANF_UNLOCK` and `SYS_CMD_MANF_DOWNGRADE_CALIBRATION` to the IMX to downgrade the IMU calibration and put the system into bootloader update mode. This can be done using the EvalTool, cltool, or SDK .

a. **EvalTool** (version 1.9.1 or later): Use the firmware "**Downgrade**" button (EvalTool -> Settings -> General -> Factory -> Downgrade).

b. **cltool** (version 1.10 or later): Use option `-sysCmd=1357924682` to send the downgrade command:

```
./cltool -c /dev/ttyACM0 -sysCmd=1357924682
```

cltool alternate method: use option `-edit 7` to edit the DID_SYS_CMD and send the downgrade command:

```
./cltool -c /dev/ttyACM0 -edit 7
```

Use `w` and `s` to move the cursor up or down (arrow keys do not work) and enter to submit the new value. The `invCommand` value is the bitwise inverse of the command and is required to validate the command. The `command` value will change to zero when the IMX accepts the `command` and `invCommand` values.

Send manufacture unlock command:

```
command = 1122334455
invCommand = 3172632840
```

Send downgrade command:

```
command = 1357924682
invCommand = 2937042613
```

3. Verify the IMX has rebooted into bootloader update mode. The host serial port will disappear and reappear. The IMX will NOT support normal DID binary or NMEA communications in this mode, but will be ready to update the bootloader.
4. Update the bootloader and firmware using the 1.8.x EvalTool, cltool, or SDK. Be sure to use the bootloader v5d (or older) with the 1.8.x firmware.

©2022

Chip Erase Downgrade

The above process using the `SYS_CMD_MANF_DOWNGRADE_CALIBRATION` command is recommended as it prevents the need to reload the IMU calibration onto the IMX. However, an alternative method to downgrade IMX to 1.8.x firmware is as follows:

1. Chip erase the IMX.
2. Load v5b (or older) bootloader and 1.8.x (or older) firmware using the 1.8.x EvalTool, cltool, or SDK.
3. Restore the IMU calibration.

33.1.10 1.7.6 Bug RTK Base GPS Raw work around

If you are having base raw errors on your Rover, in the bottom right of the Evaltool, or a climbing Diffrential Age, in Data Sets DID_GNSS1_RTK_REL, you maybe having this bug. Try this workaround.

1. Go-to settings tab, open the Base serial COM port.
2. Go-to Data Logs tab, under RCM Presets dropdown select PPD.
3. **NOTE:** You must leave the comport open on the Base.
4. Check your Rover to see if its still getting raw errors messages.

©2022

33.2 IMX Firmware Troubleshooting

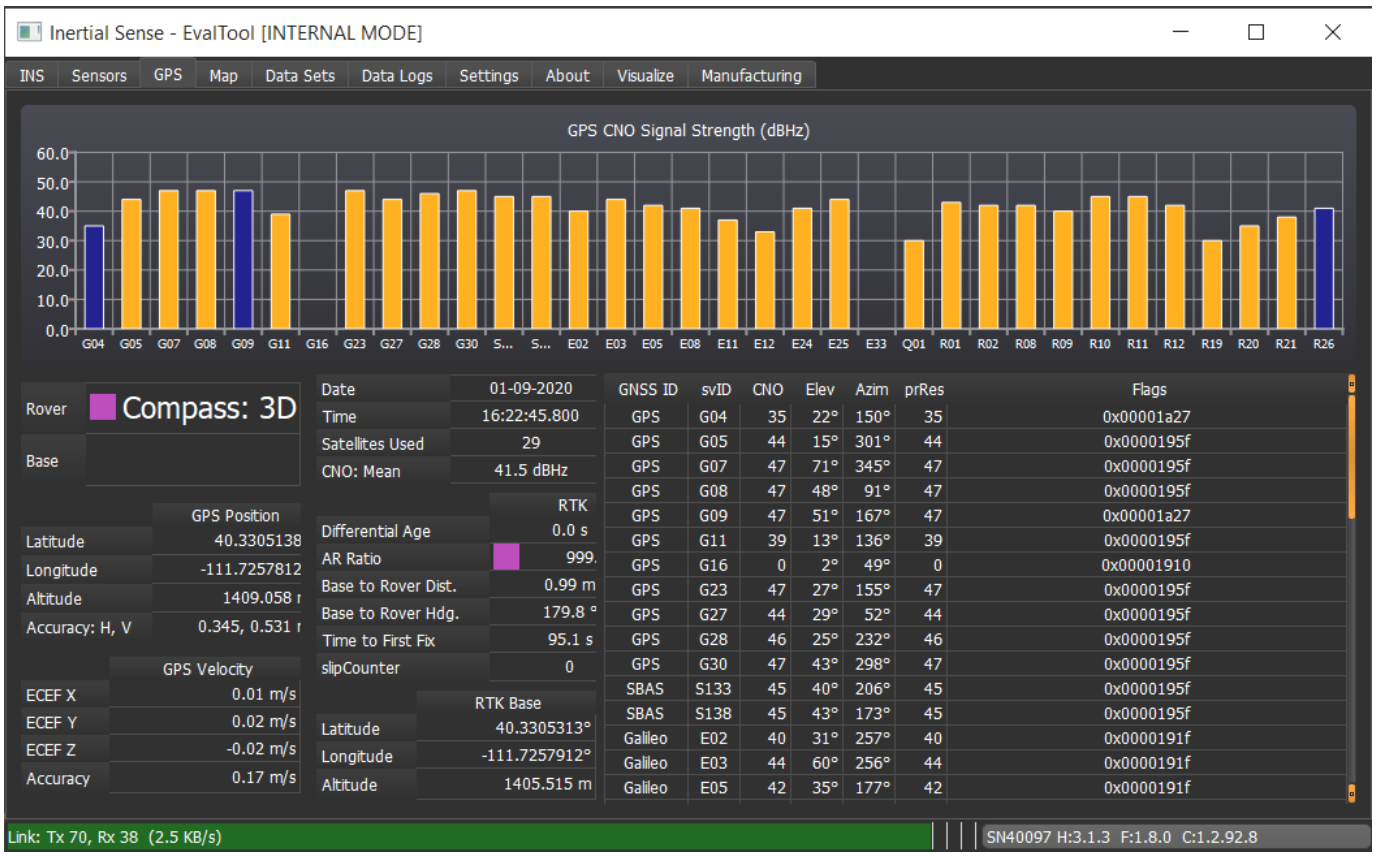
33.2.1 Antenna Baseline

Separation between GNSS antennas (or baseline distance) impacts the accuracy and fix time of the solution. Typical Dual GNSS heading fix time is 60-90 seconds using a 1 meter baseline. Baseline distances shorter than 1 meter will impact both heading accuracy and time to fix. However, having a short baseline of 0.35m should not cause an extremely long fix time.

ITEM TO TEST: Try increasing the antenna baseline to 0.5m or greater during initial testing.

33.2.2 Satellite CNO Strength, RFI and EMI

What is the satellite CNO (signal strength) level? The mean CNO would ideally be above 38-40. Anything lower could indicate the presence of RF interference (RFI) or electromagnetic interference (EMI). You can see this in the EvalTool GNSS tab or the DID_GNSS1_POS message.



ITEM TO TEST: Try powering off portions of the system while running the IMX. You may even try running the IMX independently to a separate computer to monitor the system so you can completely power off your system with the IMX still running. Pay close attention to the GPS CNO during each change.

33.2.3 USB Interface

USB-3 has been known to interfere with wireless and GNSS systems. It is the most common source of interference that has been experienced. Properly shielded cables or signal filters can address this.

ITEM TO TEST: Disable USB-3, digital busses, or various switching supplies and observe the satellite CNO level. CNO mean should be near or above 40 dB/Hz.

©2022

33.2.4 Antenna Ground Planes

The ground planes should be adequately sized. Larger ground planes help but are generally not the root cause of poor performance. Separate and common ground planes are both acceptable.

ITEM TO TEST: Try doubling the ground plane below the antenna. A simple sheet of metal placed below the antenna is fine. This is also not likely the root cause but worth testing.

33.2.5 Antenna Cable Ground Loops

In some cases the GNSS antenna cable can form an electrical loop and cause interference.

ITEM TO TEST: Try to ensure cables never loop back or are bundled. If possible shorten cables to smallest required length. Monitor GPS CNO before and after.

33.2.6 Local Interference

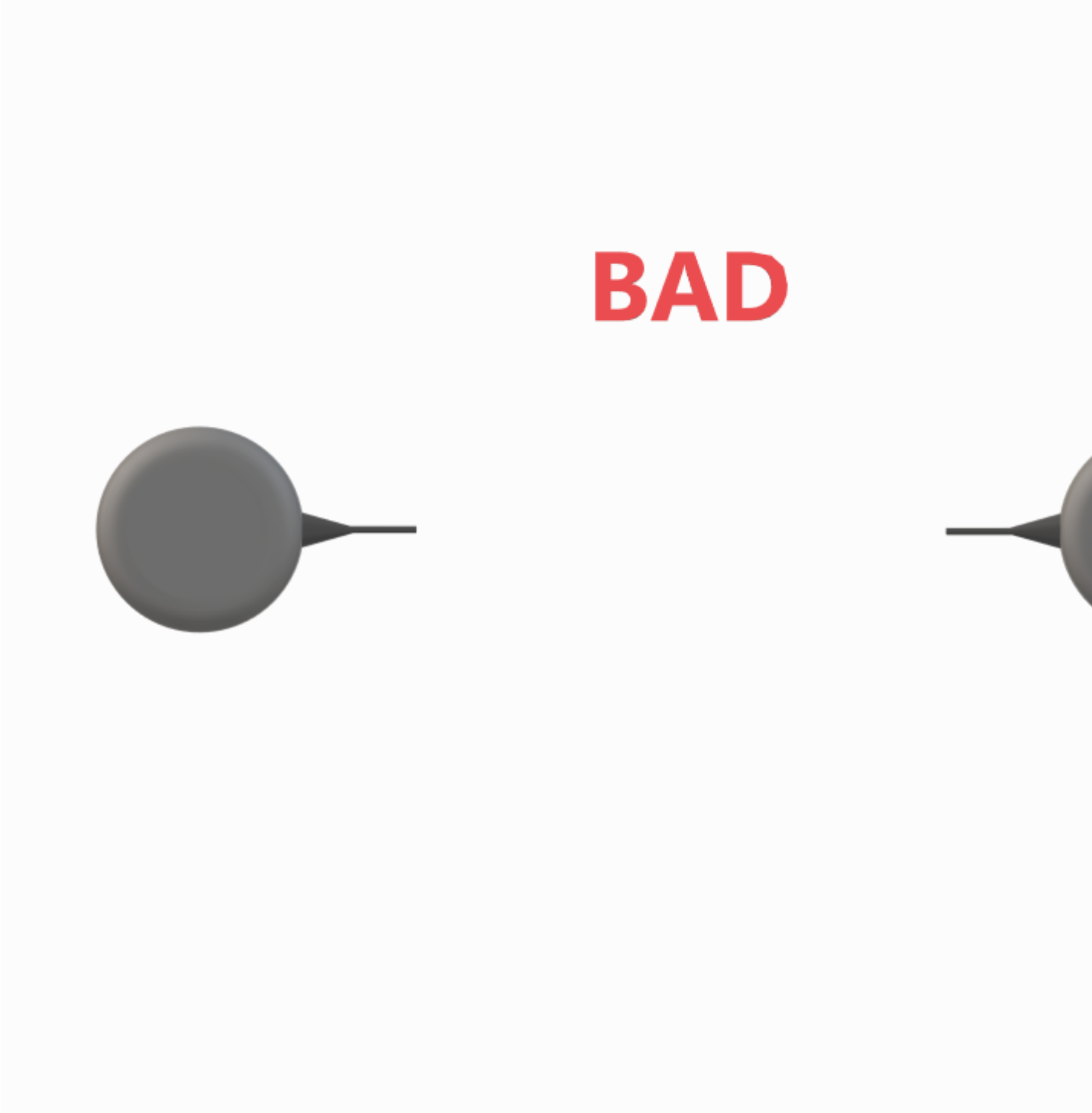
In some cases we have seen objects in close proximity to the GNSS antennas act as a multi-path surface, reflecting GNSS signals onto the GNSS antennas. Ensure no objects are near the antennas above the plane of the antennas.

©2022

33.2.7 Antenna Orientation

A more accurate heading can be achieved if the GNSS antennas have the same antenna orientation (point the same way). You should consider rotating one or both GNSS antennas so the coax cables exit in the same direction on each antenna.

Mismatch



©2022